UNIVERSITÀ DI PARMA

IoT Lab

→ LEC. 1

Internet of Things

Prof. Gianluigi Ferrari

Dipartimento di Ingegneria e Architettura – Pal. 2
Internet of Things (IoT) Lab
Università di Parma
gianluigi.ferrari@unipr.it
http://www.tlc.unipr.it/ferrari
http://iotlab. unipr.it/

Tel: 0521 90**6513**

Our Calendar (Spring 2026)

February						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
23	24	25	26	27	28	

March						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31					

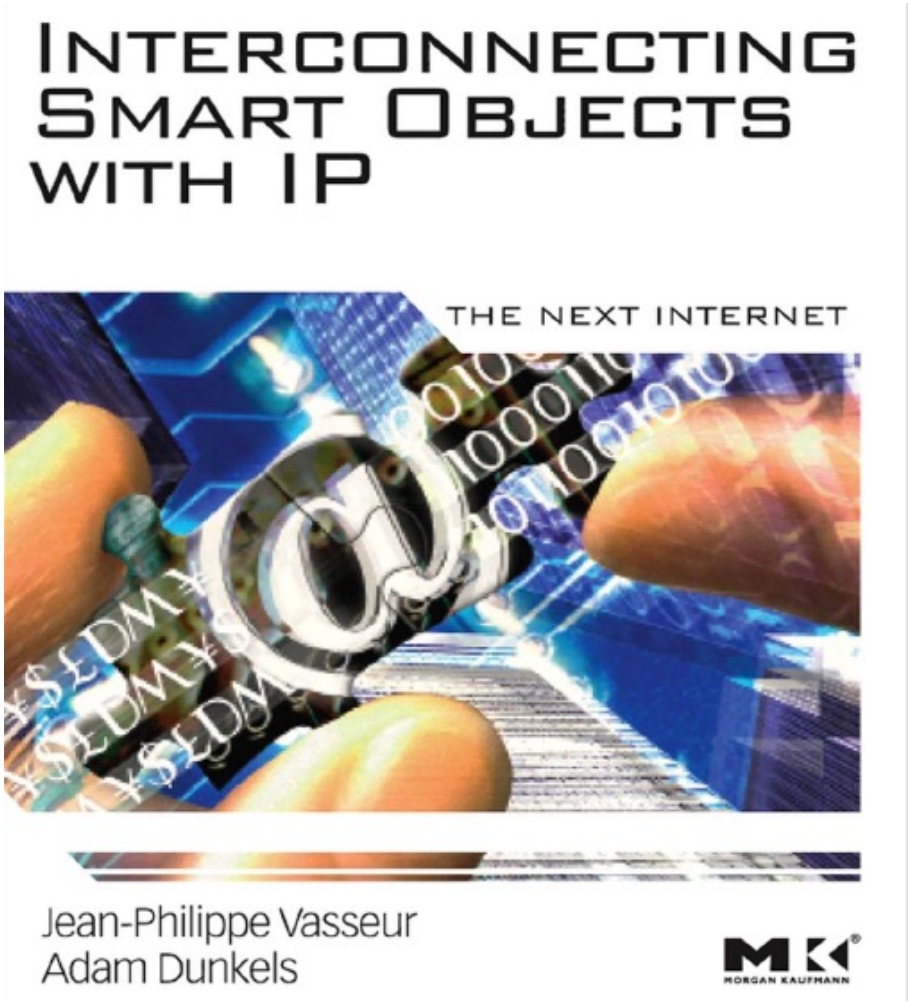
April						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

May						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
				1	2	3
4	5	6 -Midterm	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

June						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
1	2	3	4	5		

	theory
	practice
	no lecture
	Midterm (theory)

Our Main Reference



Outline

Theory

- Introduction to IoT
- Transport Layer Protocols: UDP and TCP
- IPv6
- Communication Mechanisms
- Energy-efficient MAC Protocols
- 6LowPAN
- Routing in LLNs & RPL
- Communication paradigms for the IoT
- Operating Systems and Hardware for the IoT
- Constrained Application Protocol (CoAP)
- IoT Security
- LoRa and LoRaWAN

Practice (Seminars, Lab & project)

- CoAP: Introduction to Java, Californium (Cf) Framework **or** Applied project (real devices)
- Lab & project assignment

UNIVERSITÀ DI PARMA

IoT Lab

Introduction to IoT

Main sources

- C. Bormann, “Protocols for the Real “Internet of Things””, keynote speech at the Internet of Things and Smart Cities Ph.D. School 2014 Lerici (SP), Italy - September 8th - 13th, 2014

- J. P. Vasseur and A. Dunkels, `Interconnecting Smart Objects with IP,` Morgan Kaufmann, 2010. Chapter 1.

- M. R. Palattella; N. Accettura; X. Vilajosana; T. Watteyne; L. A. Grieco; G. Boggia; M. Dohler, "Standardized Protocol Stack for the Internet of (Important) Things," *IEEE Communications Surveys & Tutorials*, IEEE , vol.15, no.3, pp.1389,1406, Third Quarter 2013. doi: 10.1109/SURV.2012.111412.00158

Definition and Preliminaries

The Internet of Things (IoT)

- Extending the current Internet and providing connection, communication, and inter-networking between devices and physical objects, or "Things," is a growing trend that is often referred to as the *Internet of Things*.

- "The technologies and solutions that enable integration of real world data and services into the current information networking technologies are often described under the umbrella term of the Internet of Things (IoT)"

Other Definitions (1/2)

(1) The Internet of Things, also called The Internet of Objects, refers to a wireless network between objects, usually the network will be wireless and self-configuring, such as household appliances.

----- Wikipedia

(2) By embedding short-range mobile transceivers into a wide array of additional gadgets and everyday items, enabling new forms of communication between people and things, and between things themselves.

----- WSIS 2005

Other Definitions (2/2)

(3) The term "Internet of Things" has come to describe a number of technologies and research disciplines that enable the Internet to reach out into the real world of physical objects.

----- IoT 2008

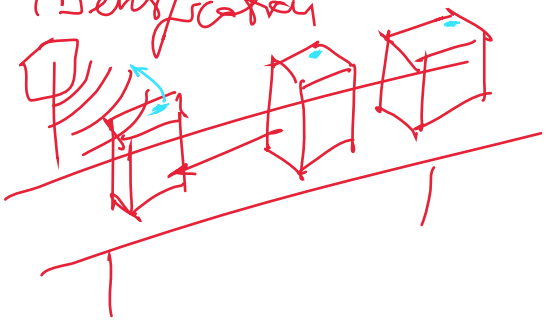
(4) “Things having identities and virtual personalities operating in smart spaces using intelligent interfaces to connect and communicate within social, environmental, and user contexts”.

------ IoT in 2020

SIoT = Social IoT

A brief History of IoT (RFID-oriented)

1997 "The Internet of Things" is the seventh in the series of ITU Internet Reports originally launched in 1997 under the title "Challenges to the Network" → RFID: bringing an object's identity to the Internet



1999 Auto-ID Center founded in MIT

2003 EPC Global founded in MIT

2005 Four important technologies of the internet of things was proposed in WSIS (World Summit on Information Society) conference.

2008 First international conference of internet of things: The IOT 2008 was held at Zurich.

Another History of IoT (Internet-oriented)

Year	Event
1967	REMBASS Remotely Monitored Battlefield Sensor System
1978	Distributed Sensor Networks for Aircraft Detection Lincoln Labs - Lacoss
1992	RAND Workshop - Future Technology Driven Revolutions in Military Conflict. Concepts behind Smart Dust emerge.
1993-1996	DARPA ISAT studies - many WSN ideas and applications discussed. Deborah Estrin leads one of the studies.
1994	LWIM - Low Power Wireless Integrated Microsensors - Bill Kaiser (UCLA)
1997	Smart Dust proposal written, Kris Pister (Berkeley)
1998	Seth Hollar makes wireless mouse collars
1999	Endeavour project proposed by Randy Katz, David Culler (Berkeley) PicoRadio project started by Jan Rabaey (Berkeley)
2000	Crossbow begins selling 'Berkeley motes'
2001	Multiple demos proving viability
2002	Dust, Ember, Millennial, Sensicast founded
2003	IEEE802.15.4-2003 standard Moteiv (now Sentilla) founded
2004	ZigBee 1.0 standard ratified TSMP 1.1 shipping
2005	Arch Rock founded
2006	ZigBee 2006 standard ratified IEEE802.15.4-2006 standard
2007	WirelessHART standard ratified IETF 6LoWPAN's RFC4944 published WirelessHART shown to achieve 99.999% reliability [20]
2008-2009	IETF workgroup Routing Over Low-power Lossy links (ROLL) created. IEEE802.15.4e work group created
2010-2011	IEEE802.15.4e's MAC protocol ratified IEFT 6LoWPAN's RFC4944 updated IETF ROLL's RPL routing protocol ratified

Internet of Things and Smart Cities Ph.D. School (2013,2014)

International speakers from academia and industry gave lectures tailoring their research field for an interdisciplinary audience. A dedicated discussion panel focused on the interaction and the collaboration between academia and industry in order to depict the future vision of Smart Cities and IoT

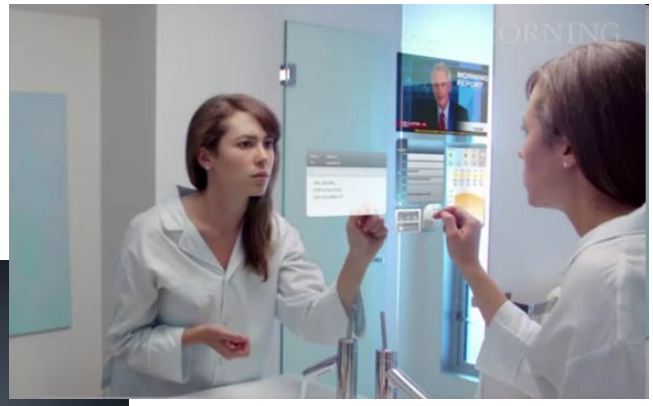
Some numbers:
36 students from 11 countries in Europe
17 speakers from Universities and Industry

2018 This Is What Happens In An Internet Minute
facebook
973,000 Logins
18 Million Text Messages
YouTube
4.3 Million Videos Viewed
375,000 Apps Downloaded
174,000 Scrolling Instagram
481,000 Tweets Sent
1.1 Million Swipes
tinder
187 Million Emails Sent
936,073 Views
twitch
67 Voice-First Devices Shipped
amazon echo
38 Million Messages
25,000 GIFs Sent via Messenger
2.4 Million Snaps Created
$862,823 Spent Online
NETFLIX
266,000 Hours Watched
3.7 Million Search Queries
Google
60 SECONDS
Created By:
@LoriLewis
@OfficiallyChadd

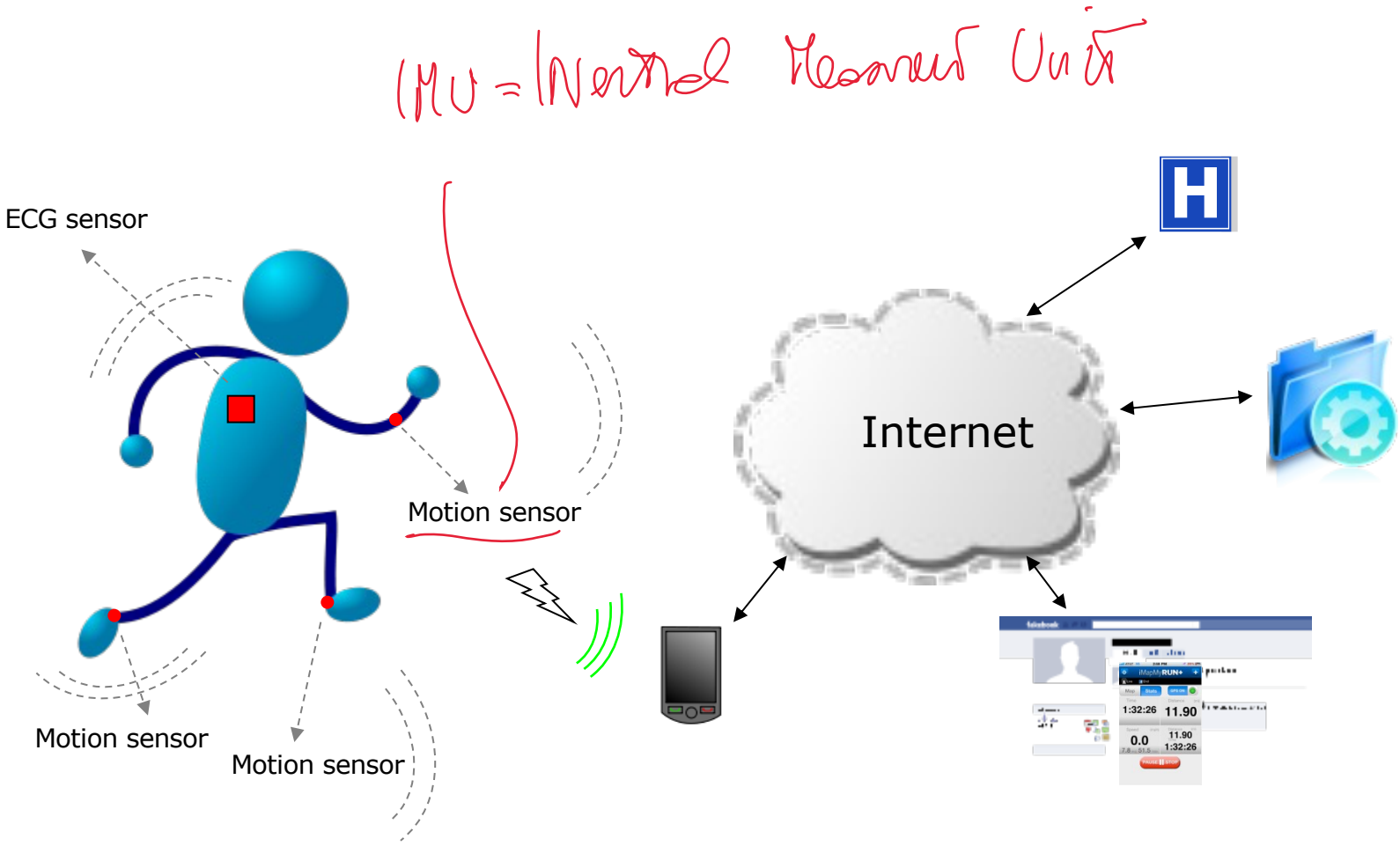
2019 This Is What Happens In An Internet Minute
facebook
1 Million Logging In
18.1 Million Texts Sent
YouTube
4.5 Million Videos Viewed
390,030 Apps Downloaded
347,222 Scrolling Instagram
87,500 People Tweeting
1.4 Million Swipes
tinder
188 Million Emails Sent
1 Million Views
twitch
41 Music Streaming Subscriptions
180 Smart Speakers Shipped
amazon echo
4.8 Million Gifs Served
GIPHY
41.6 Million Messages Sent
2.1 Million Snaps Created
$996,956 Spent Online
694,444 Hours Watched
NETFLIX
3.8 Million Search Queries
Google
60 SECONDS
Created By:
@LoriLewis
@OfficiallyChadd

More “Things” are being connected

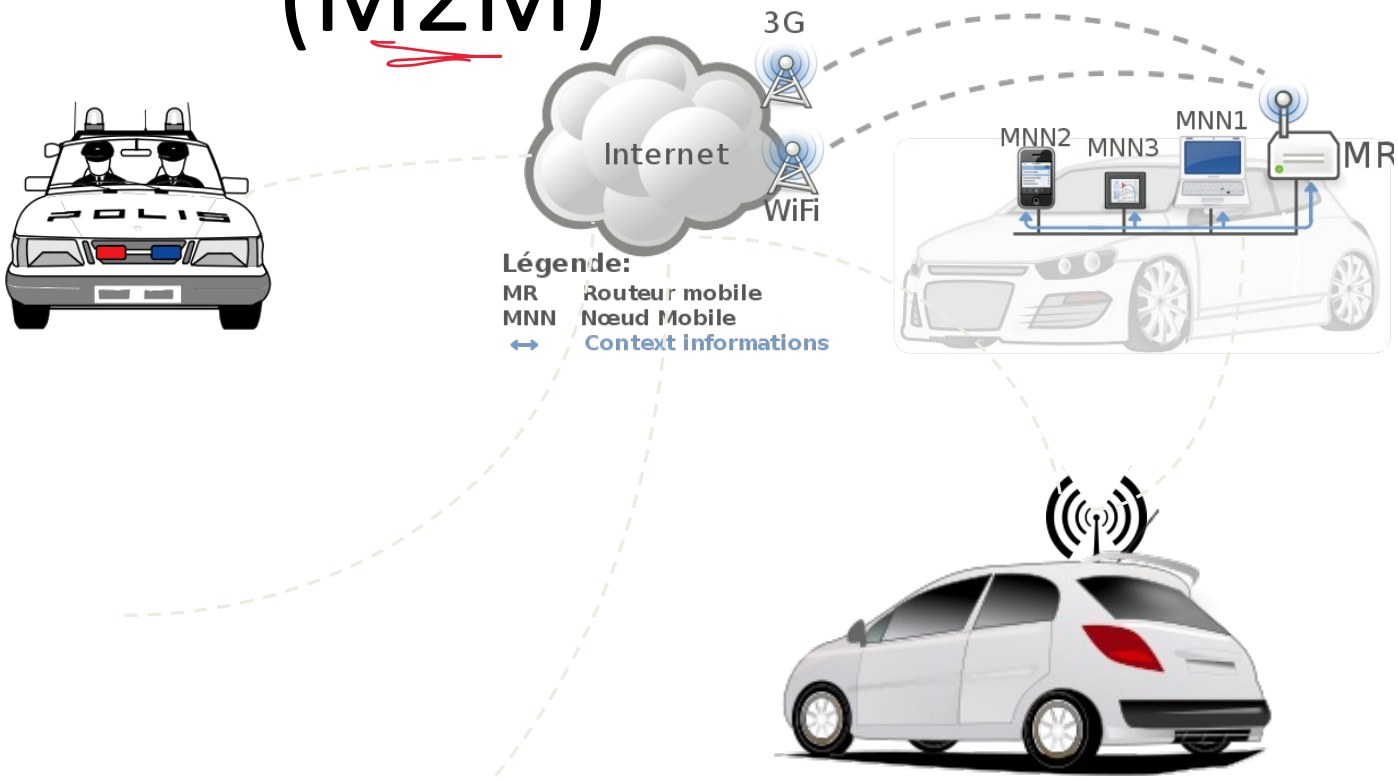
Home/daily-life devices
Business and
Public infrastructure
Health-care
…



People Connecting to Things



Things Connecting to Things (M2M)

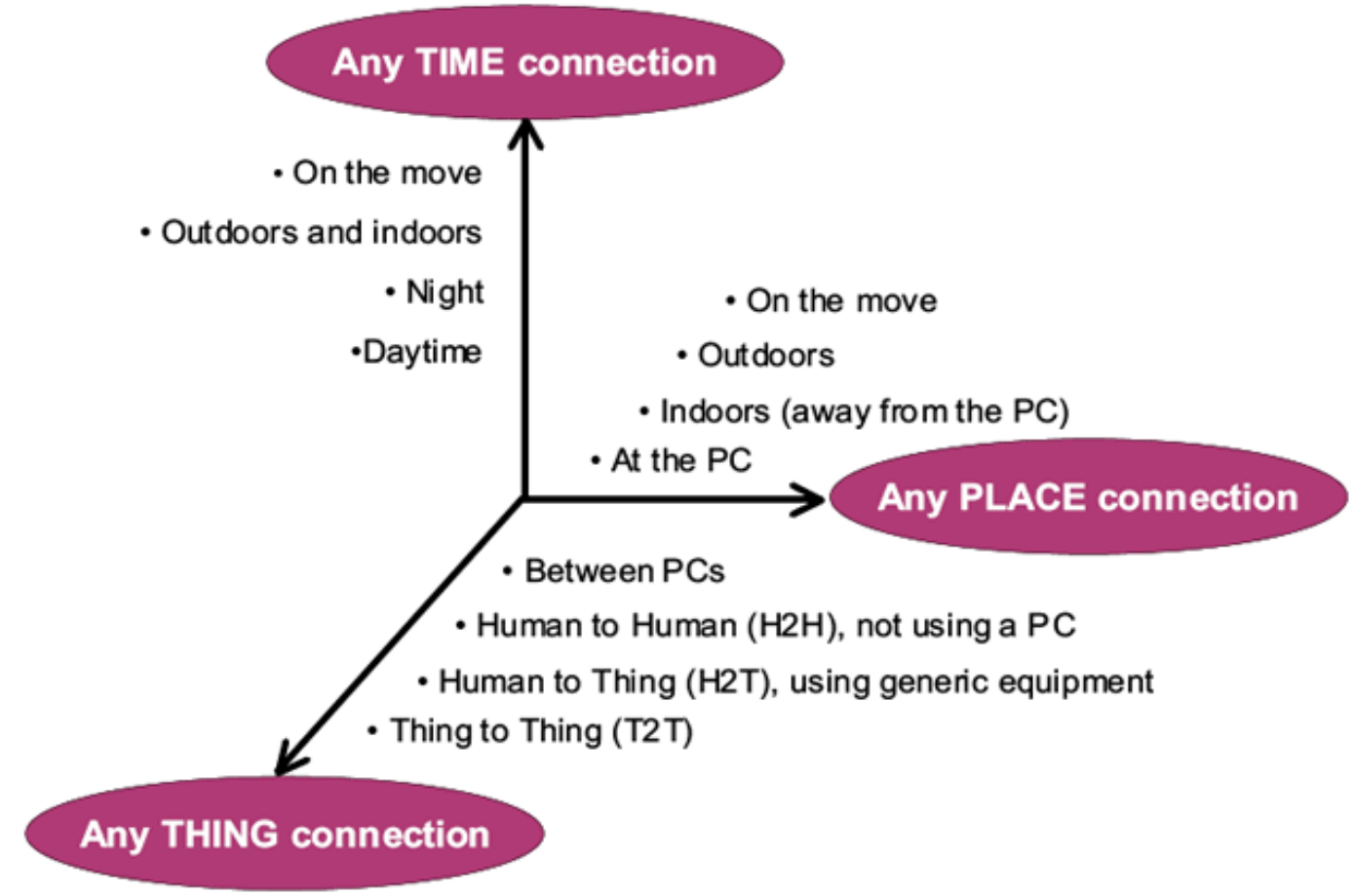


- Complex and heterogeneous resources and networks

How are the networks changing?

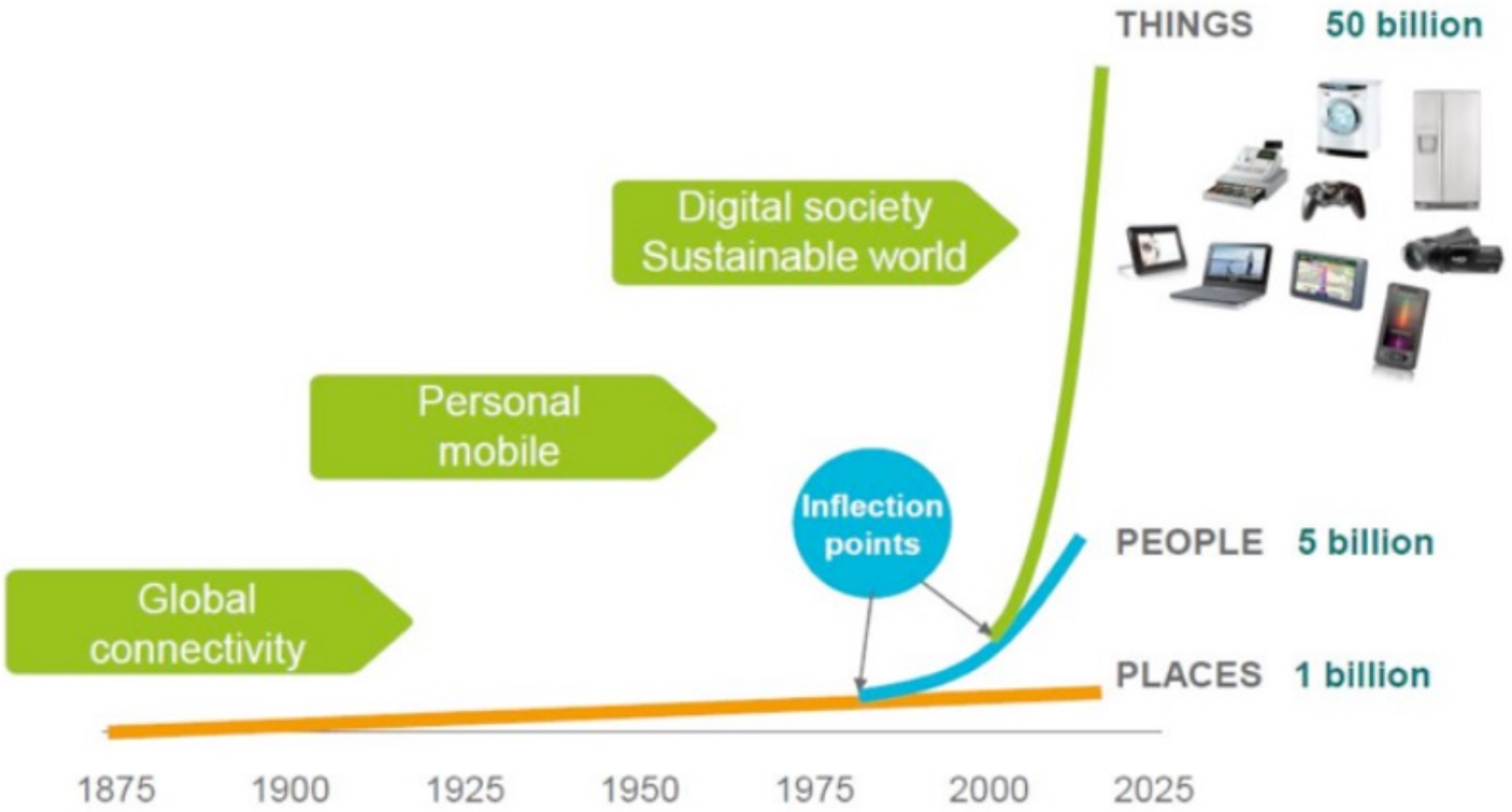
- Extensions
  - More nodes, more connections, IPv6, 6LowPan,...
  - Any TIME, Any PLACE + Any THING
  - M2M, IoT
    - Billions of interconnected devices,
    - Everybody connected.
- Expansions
  - Broadband
- Enhancements
  - Smart networks
  - Data-centric and content-oriented networking
  - Context-aware (autonomous) systems

Future Networks



Source: ITU adapted from Nomura Research Institute

Connecting:
Places → People → Things



A New Perspective

- Scale up:
  - Number of nodes (50 billion by 2020)
- Scale down:
  - Node
  - Cost
  - Complexity
  - Reuse: cent, kylobyte, megahertz

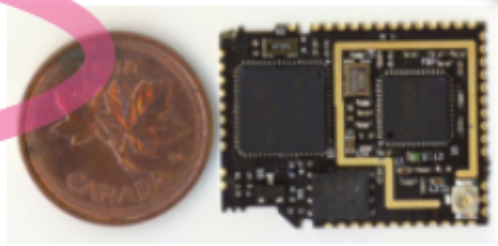
Constrained nodes: orders of magnitude

10/100 vs. 50/250

- There is not just a single class of "constrained node"

RFC 7228

- Class 0: too small to securely run on the Internet
  - "too constrained"
- Class 1: ~10 KiB data, ~100 KiB code
  - "quite constrained", "10/100"
- Class 2: ~50 KiB data, ~250 KiB code
  - "not so constrained", "50/250"

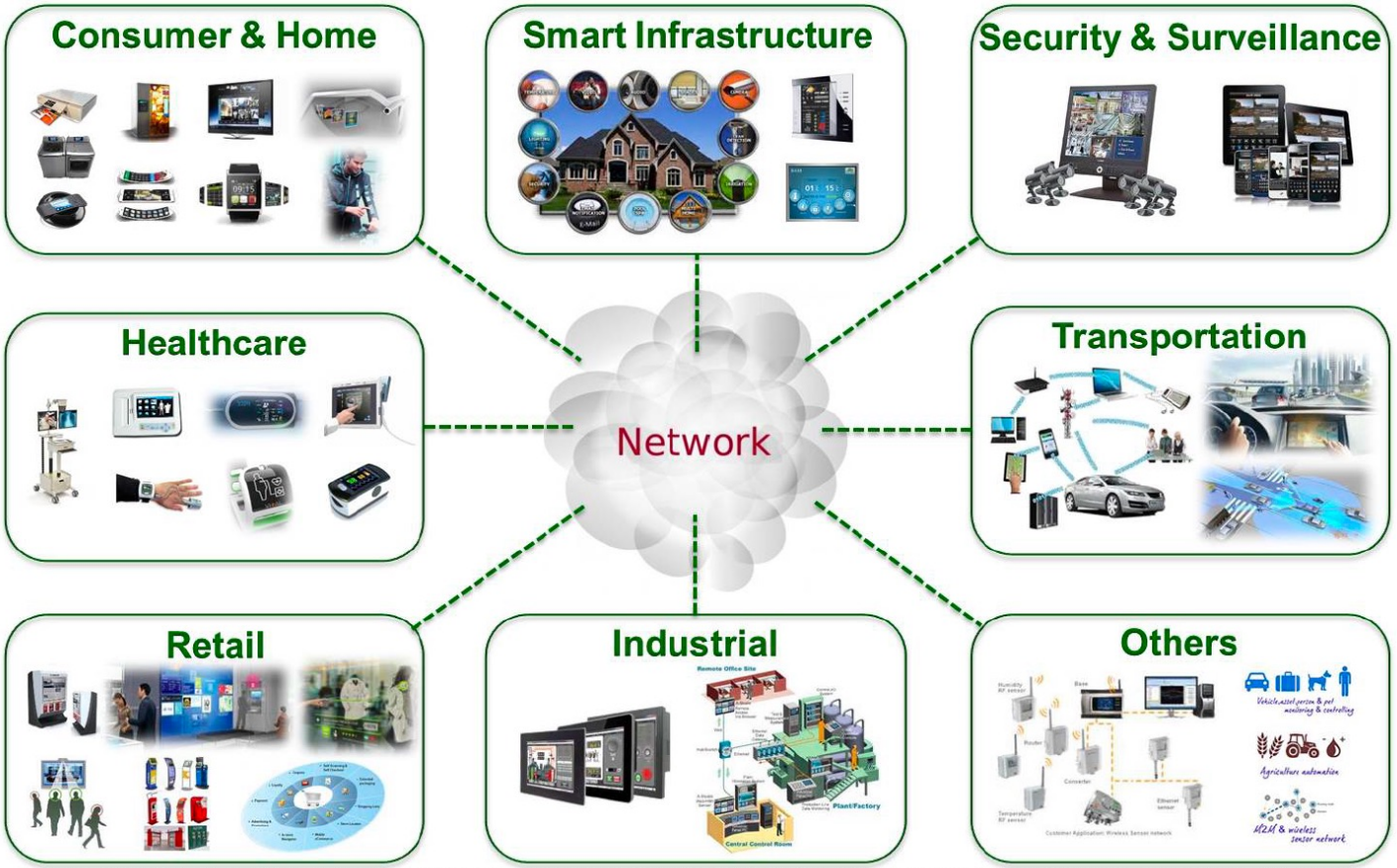


- These classes are not clear-cut, but may structure the discussion and help avoid talking at cross-purposes

Smart agriculture

EXPO 2015: UK Pavillion
Accelerometers to understand the status of a beehive

IoT Applications



Vivante and the Vivante logo are trademarks of Vivante Corporation. All other product, image or service names in this presentation are the property of their respective owners. © 2013 Vivante Corporation

Today's Smart Objects (1)

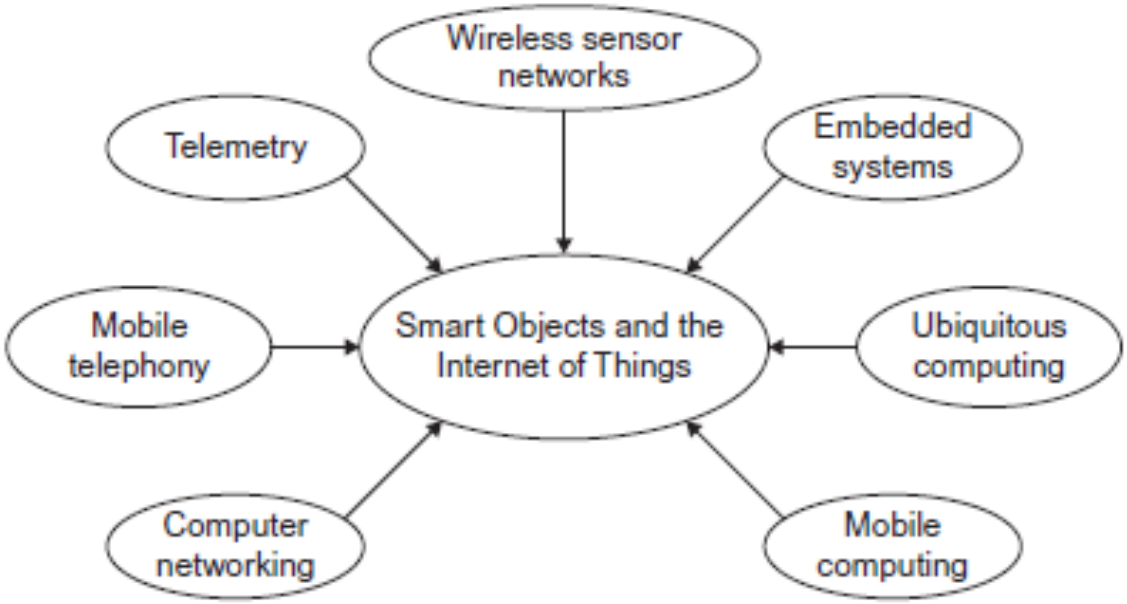


FIGURE 1.1

Smart objects are the intersection of embedded systems, ubiquitous computing, mobile telephony, telemetry, wireless sensor networks, mobile computing, and computer networking.

Embedded Systems (1)

- A computer (microprocessor) embedded in something other than a computer
- Traditionally: real-time systems (strict time response requirements, industry) → Real-Time Operating System (RTOS)
- Commonalities (emb. sys. - smart obj.):
  - Hardware
  - Constraints in terms of computational power and memory
  - Same microcontrollers (ARM, RISC-V)
- Differences:
  - Communication is not core for embedded systems (e.g., a car engine which cannot communicate can still work as car engine)
  - Communication is crucial for a smart object

Embedded Systems (2)

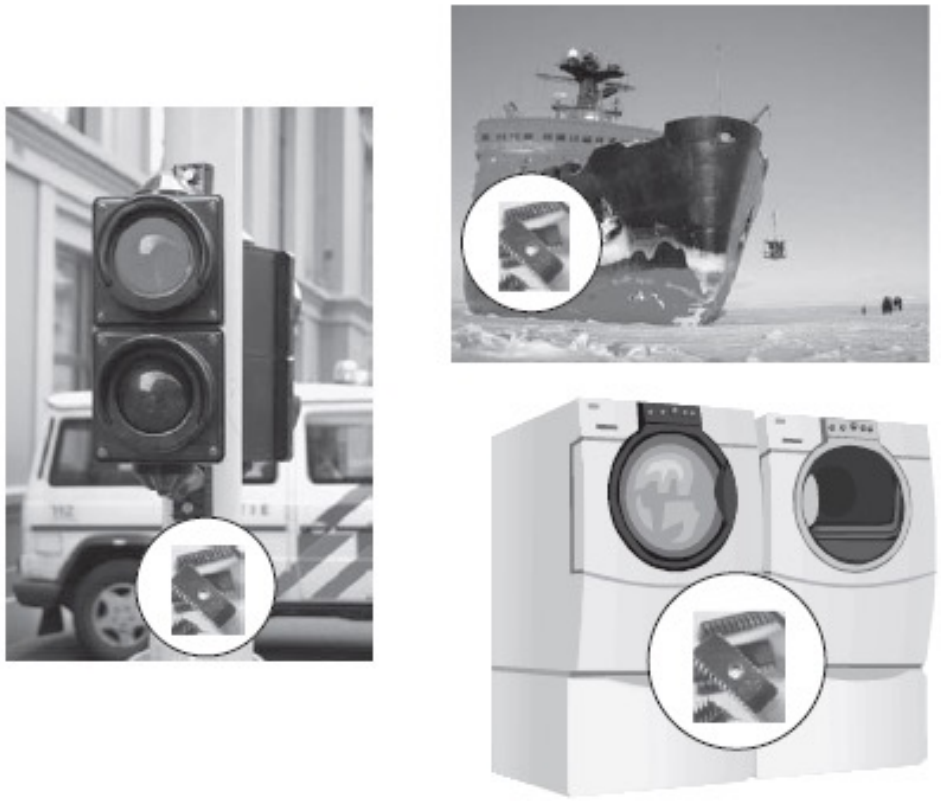


FIGURE 1.2

Embedded systems are microprocessor-equipped systems and devices that interact with the physical world. Examples include traffic lights, a ship's rudder controllers, and washing machine controllers.

Ubiquitous and Pervasive Computing (1/2)

- Field of study based on concept of what happens when computers move away from the desktop and immerse in the environment (MIT, 1988): "computing will no longer be done by desktop pcs, but will move into our daily environment"
- Ubiquitous computing: successful measure is its invisibility
- Examples:
  - Active badges (AT&T@ Cambrige, UK, late 80's-early 90's) with readers
  - AmbientROOM project (MIT, 1996)
  - Wearable computing: computing infrastructure moves on the body (**our research**)

Ubiquitous and Pervasive Computing (2/2)

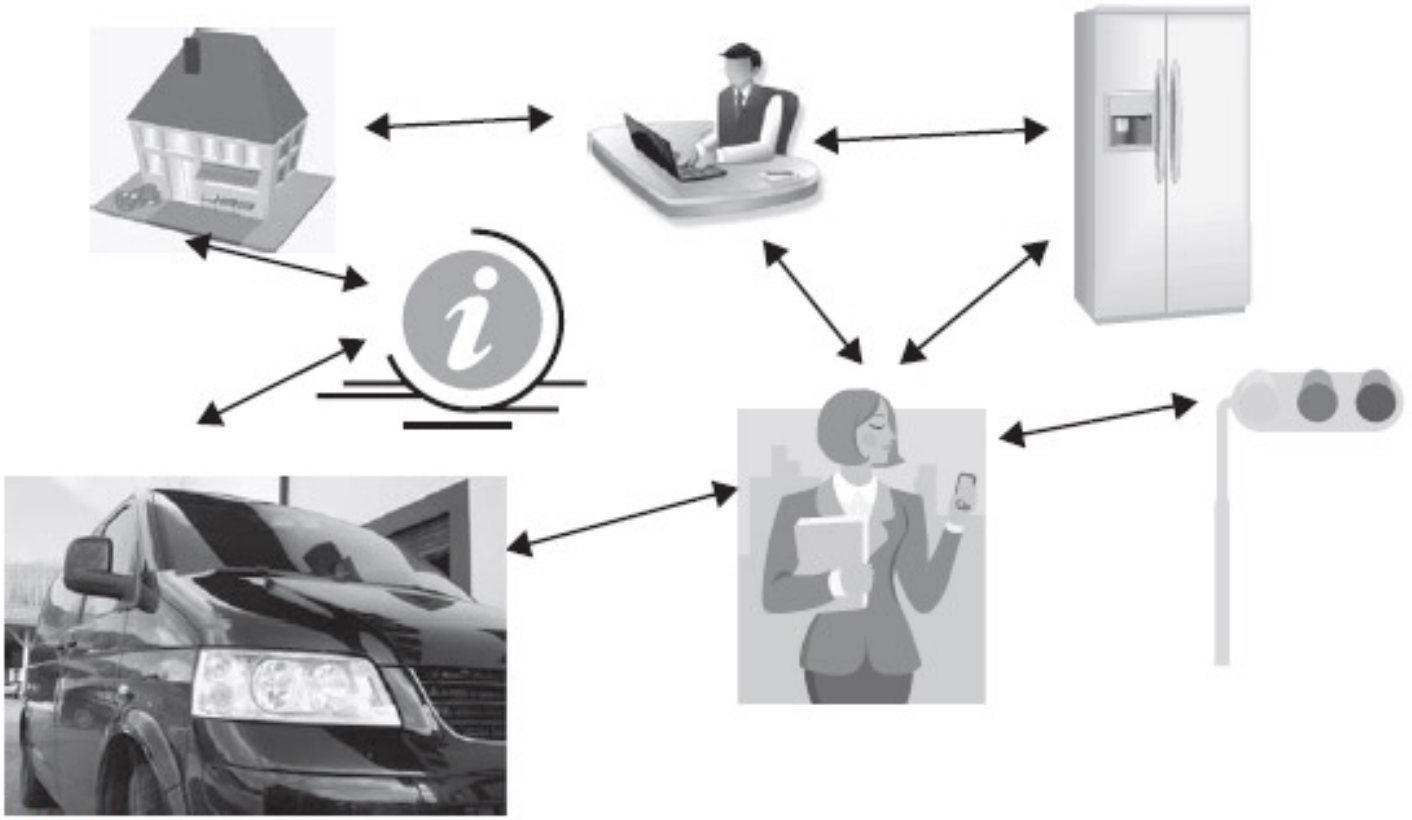


FIGURE 1.3

Ubiquitous computing is a vision for the future of computers where computing moves into everyday objects.

Mobile Telephony

- Grew from telephony industry with the promise of ubiquitous access to telephony
- Typically thought of as cellular telephony
- Smartphones: far more than cellular phones
  - Several other wireless interfaces: WiFi, Bluetooth, NFC
  - A large number of on-board sensors
- Mobile telephony has revolutionized access to the Internet → really seamless
- So, smartphones are likely to play a key role in IoT

Telemetry and M2M

- Telemetry: remote measurements
- M2M: generalization of telemetry, which implies autonomic communication between non-human operated machines and central to the concept of telemetry
- Telemetry:
  - Long-range: temperature, humidity, wind from remote weather stations (cellular network or Inmarsat satellite network); Formula 1
  - Short-range: pacemaker (telemetry coil, to measure the activity without surgery: low-power communication)

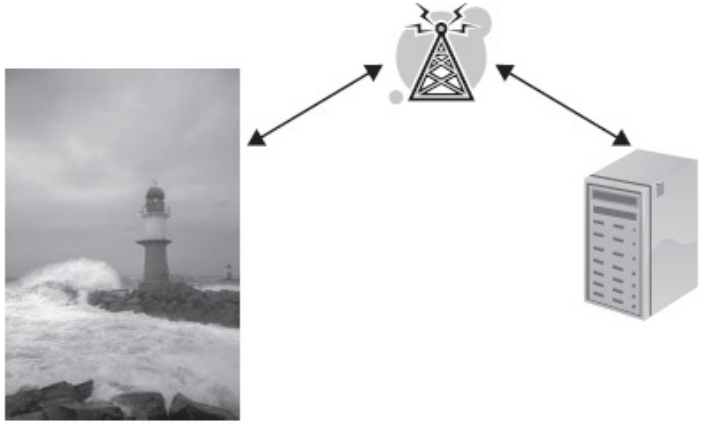


FIGURE 1.4

Telemetry allows reading measurements from remote systems such as weather stations. Data are typically transported using mobile telephony systems.

Wireless Sensor Networks

- Derive from the idea that wireless sensors can collect information from the physical world in a large number of situations (wild fire tracking, agriculture management, etc.)
- Significant research activity in the 2000's:
  - Smart Dust: Berkeley, 1999
  - First prototypes with larger sizes
- Idea: try to be different from the Internet → cross-layer optimization → no scalability
- Several results (energy savings, algorithms, protocols) have been moved to smart objects (not only data gathering, but actuation/control)

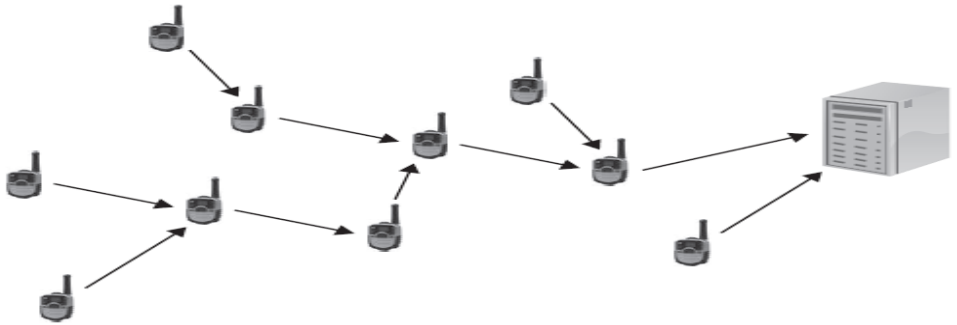
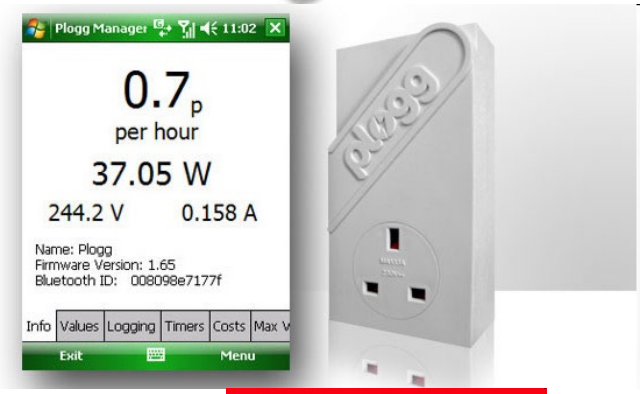
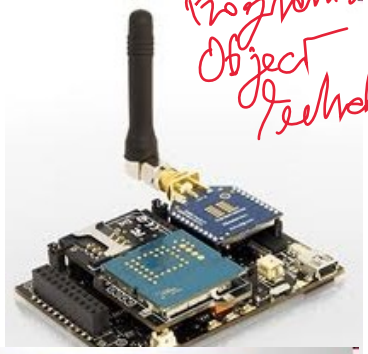


FIGURE 1.5
Wireless sensor networks provide large-scale measurements of physical properties using large amounts of sensors that transport their data wirelessly to a base station.

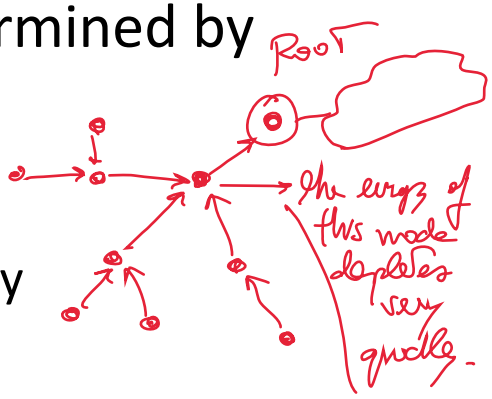
Sensor devices are becoming widely available

- Programmable devices
- Off-the-shelf gadgets/tools





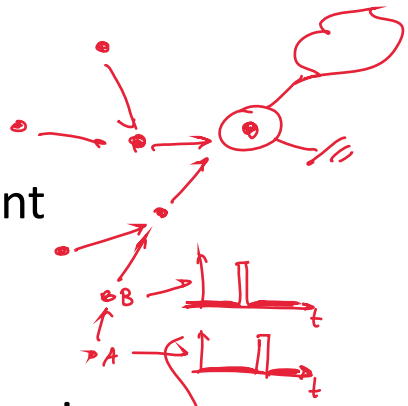
Challenges for Smart Objects



- **Node-level** internals: power consumption and physical size
- **Network-level** mechanisms and structures: determined by the entire ensemble of smart objects
- Aspects are highly interrelated:
  - power consumption (node-level) is highly influenced by communication patterns (network-level);
  - a routing protocol (network-level) should take into account the energy statuses of the nodes (node-level)
- Internet Protocol for Smart Objects (IPSO) Alliance: set up to spread awareness of the technology around smart objects → smart objects need evolvable technology → technology around IP is the right one

Node-level Challenge: Power Consumption

- Power consumption: battery-power or external low-power energy source (physical vibrations)
- Design implications for hardware, software, network protocols and architecture
  - Hardware designer: low-power components, minimal current leakage, power-efficient sleep mode
  - Software designer: make the most out of hardware
  - Operating systems for smart objects: natively provide mechanisms for low-power operation
- Power efficiency significantly affects network architectures and protocols design
  - Communication patterns→ use efficiently resources (sleeping)
  - Hardware and software keep track of energy spent and communicate to network layer



Node-level Challenges: Physical Size and Cost

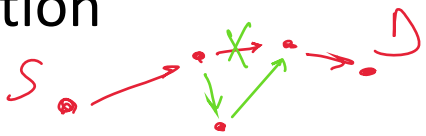
- Physical size: size and form factor determine the potential applications for a given smart object system
- Hardware designer
  - Hardware small
  - Number of components small
  - Each component small and inexpensive
- Software designer
  - Microprocessor smaller → Reduced computational speed and memory size
  - Each "line of code" (protocol "feature") counts: a few kBytes (e.g., efficient multicast in 6LoWPANs)
- Network protocols: designed minimizing the amount of information a node keeps about the network and other nodes
- Memory constraints: network level ←→ node level

Network-level Challenge: Network Size

- Large networks of small objects (e.g., thousands of nodes) → large amount of generated data
  - Example: Smart object network that samples the temperature inside a building. Temperature is generally a slow-moving phenomenon, so the nodes do not need to sample very often: sample the temperature twice every minute → 737,280 readings per year. If the system is designed to work for ten years, there will be over seven million readings, from each node, during the lifetime of the system
- Network size → impact on routing protocol design
- Routing protocol is related to:
  - amount of data the network can sustain
  - speed at which these data can be successfully transported through the network
  - achievable lifetime of the network as a whole
- Routing protocol must make well-informed choices: information on the network as a whole and on the neighbors

Network-level Challenge: Lossy Communications

- Unreliable nature of smart object networks
- Lossyness: inherent property of smart object networks → design for lossy networks, so that the communication protocol is robust for both lossy and non-lossy



- Lossy links → impact on routing protocol design
- Lossyness: illusive property, particularly in wireless networks.
  - Affected by environmental factors (e.g., temperature and humidity)
  - Electromagnetic interference
- Routing protocols for smart object networks must be prepared for this illusiveness.

↳ Need to have automatic mechanisms.

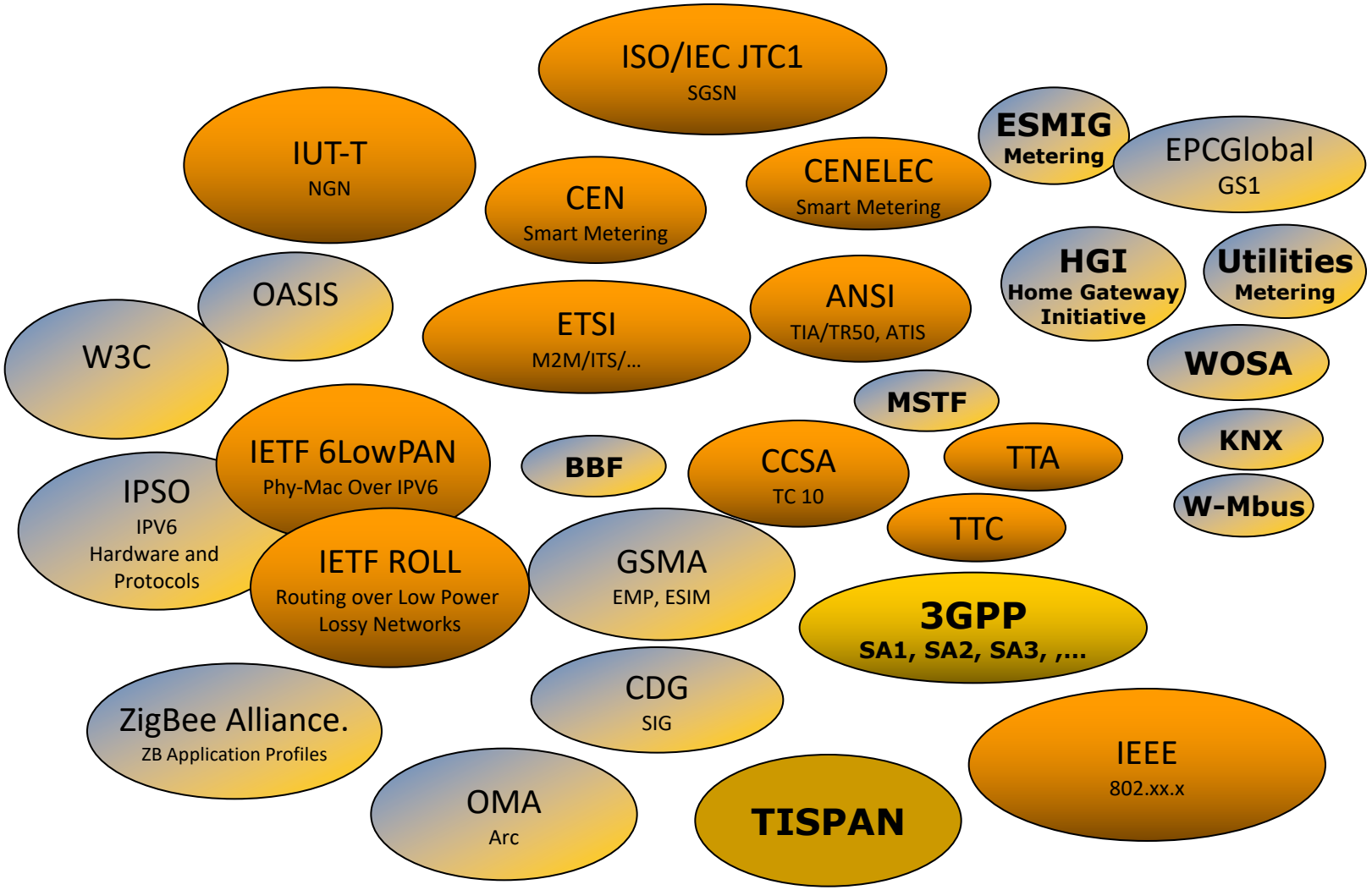
Other Network-level Challenges

- Node addressing: each smart object must be addressable uniquely
  - Within a single (even if large) network
  - With virtually any other SO of any other network → exponential growth → IPv6
- Network management: very challenging (no manual intervention; self-organization)
- Scale of smart object networks: amount of different environments and types of systems in which smart objects are used → various communication technologies
- Smart object network: provide mechanisms for external access to itself

Other relevant aspects: Standardization

- Different technology manufacturers have different specializations
  - For example, environment control system is controlled by an advanced IT system that receives its input from the humidity sensors
- With standardized technology, the technology is independent of its vendors, producers, and users → high acceptance
- Open standards produced through established practices of international standardization organizations
  - Patents for technology standardized must be freely accessible (protection against "submarine" patents)
- Standardization of smart object technology → challenge with several organization (different levels of protocol stack)

Standards Development Organizations (SDOs) and Industrial Groups



M2M: Vertical Fragmentation

- **Fragmented industry verticals:**
  - most industries are solving their M2M needs on their own;
  - often converging upon similar architectures and elements;
  - the M2M "wheel" is reinvented from industry to industry.
- **Customized solutions abound:**
  - since solutions are on an industry-by-industry basis, each solution requires its own design, production, and implementation cycle;
  - unique solutions often result in sole-sourcing of customized hardware;
  - software design is limited to a small group of developers who understand the architecture and API's for a particular industry or platform, thus leading to high development costs and high costs for support.
- **Lack of scale for components:**
  - economies of scale are difficult to achieve in any given industry since solutions are so varied and unique.

Examples of Verticals (1/2)

Security	Surveillance applications, alarms, object / people tracking
Transportation	Fleet management, emission control, toll payment, road safety (e.g. eCall stow pipe)
Health care	e-Health, and personal security
Utilities	Smart Metering and Grid such as oil, water, electricity, heat, and others
Manufacturing	Production chain monitoring and automation

Examples of Verticals (2/2)

Supply and provisioning	Freight supply and distribution monitoring, Vending Machines
Facility Management	Home / building / campus automation

. . .

Future	Multitude of new M2M applications based on standardized Service Capabilities and API [Ericsson CEO vision of 50 Billion Connected Objects by 2020]

Industrial Standardization for M2M

- M2M is here, and has massive potential for growth to be unleashed by standards.
- M2M first step towards future Internet of things (IoT) and will support Smart Metering/Grids.
- Market value increases with the “square” of interoperable/ accessible services.
- Operators and vendors see ETSI as a good place to do the work, however, progress requires wider consensus. (IETF, OMA and BBF contributes parts)

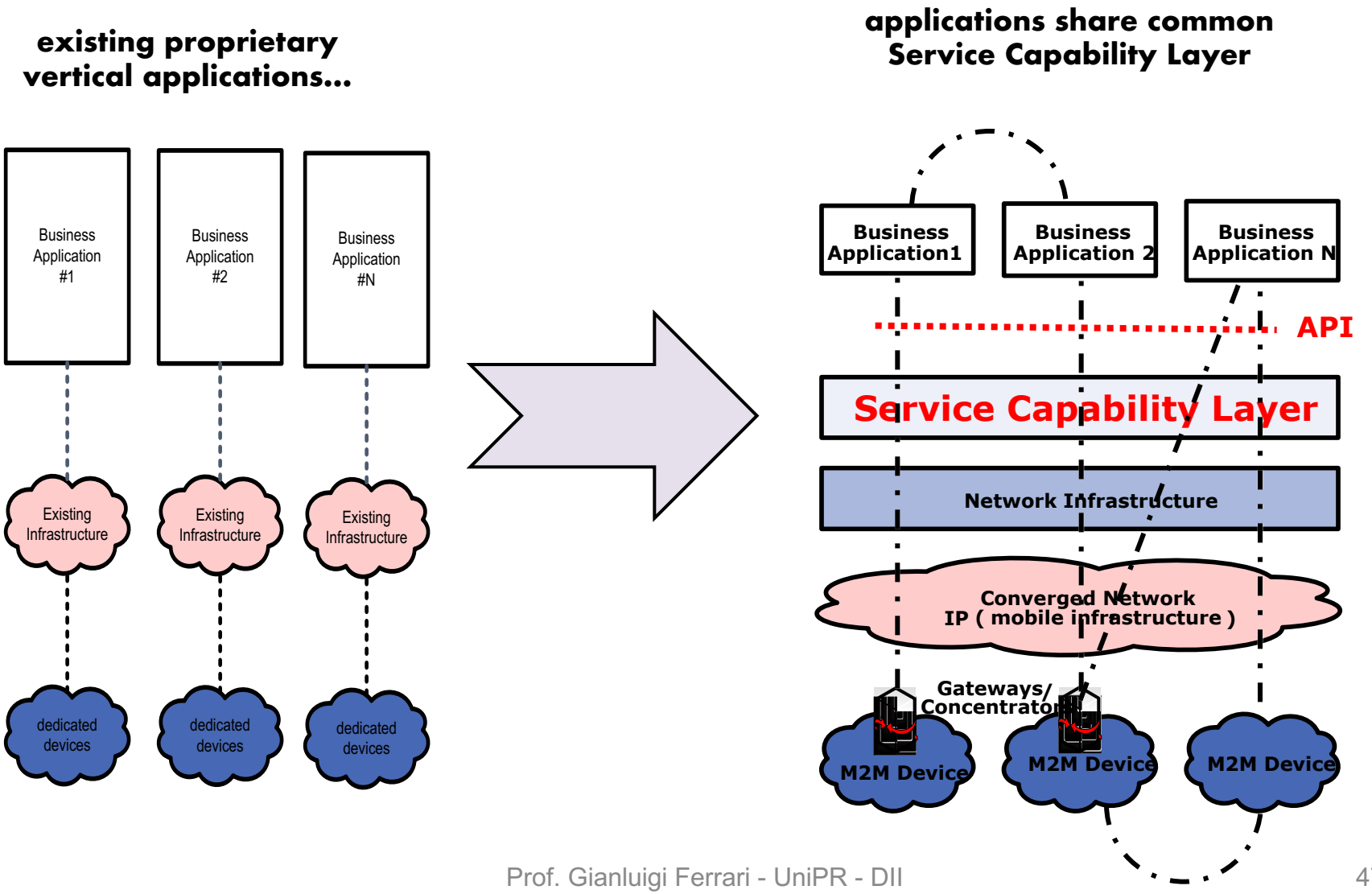
ITU-T M2M Service Layer Focus Group established in Jan. 2012

- M2M is included under the larger umbrella of the "Internet of Things" (IoT).
- The group will study and evaluate the M2M landscape and M2M work currently being undertaken by regional and national standards development organizations (SDOs), with a view to identifying a common set of requirements.
- The Focus Group will initially focus on the APIs and protocols to support e-health applications and services.
- The Focus Group will work closely with all ITU-T Study Groups and with other relevant SDOs, forums/consortia, regulators, policy makers, industry and academia. Within the ITU, the group will work particularly closely with the Internet of Things Global Standards Initiative (IoT-GSI). http://www.itu.int/en/ITU-T/gsi/iot/Pages/default.aspx.

Merits of Service Capabilities and API

- Service Capabilities provide functions that are to be shared by different applications in an interoperable manner.
- Service Capabilities expose functionalities through open interfaces (e.g. an API).
  - This will support interoperable ubiquitous services, however, area-specific data-models may be required (e.g. for Smart Metering /AMR).
- Service Capabilities simplify and optimize applications development and deployment and hide technology specifics.
- Service Capabilities may be M2M specific or generic, i.e. providing support to other than M2M applications.

Needs to invert the stove pipes



IETF (Academy-friendly)

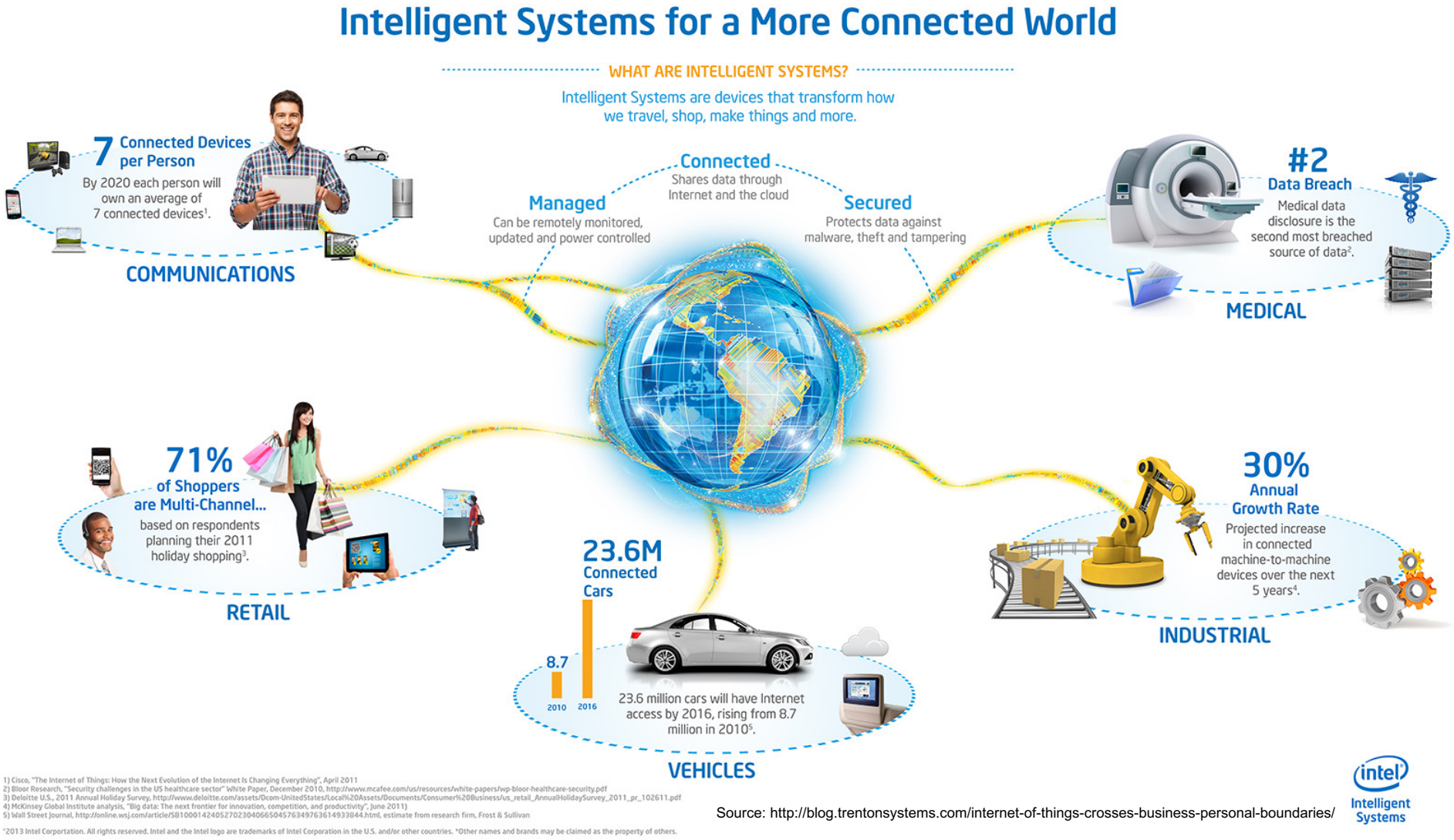
- The IETF work of the 6LoWPAN, ROLL and CoRE Working Groups (WGs) is of particular relevance for IoT/M2M.
- The CoRE WG is providing a framework for resource-oriented (i.e. RESTful) applications intended to run on constrained IP networks.
- The CoRE WG has defined a Constrained Application Protocol (CoAP) for the manipulation of Resources on a Device. The protocol supports the communications aspects of the M2M service layer.
- CoAP and related protocols will continue to be developed by the IETF.
- **CoAP is adopted by ETSI M2M**.
- The CoRE infrastructure includes a Link format (discovery mechanism), an event subscription mechanism and a scalable device proxy. Devices are viewed as RESTFul resources and can directly be interacted with using HTTP or CoAP protocol operations (PUT, GET, POST, DELETE, etc.).

Another relevant aspect: Interoperability

- Interoperability: the ability of equipment and systems from different vendors to operate together
- Smart objects must interoperate from the physical layer up to the application or integration layer
  - At physical level: agreement on physical frequencies, signal modulation, information rate
  - At network level: agreement on the format of the information that is sent and received over the physical channel; how nodes are addressed; how messages should be transported through a network of smart objects
  - At application or integration level: share a common view on how data should be entered or extracted from a smart object network, as well as how the smart objects should be reached from outside systems
- Imperative: technical architecture of smart objects defined to ease interoperability → IP-based architecture

Smart Things/Objects – Smart Environments

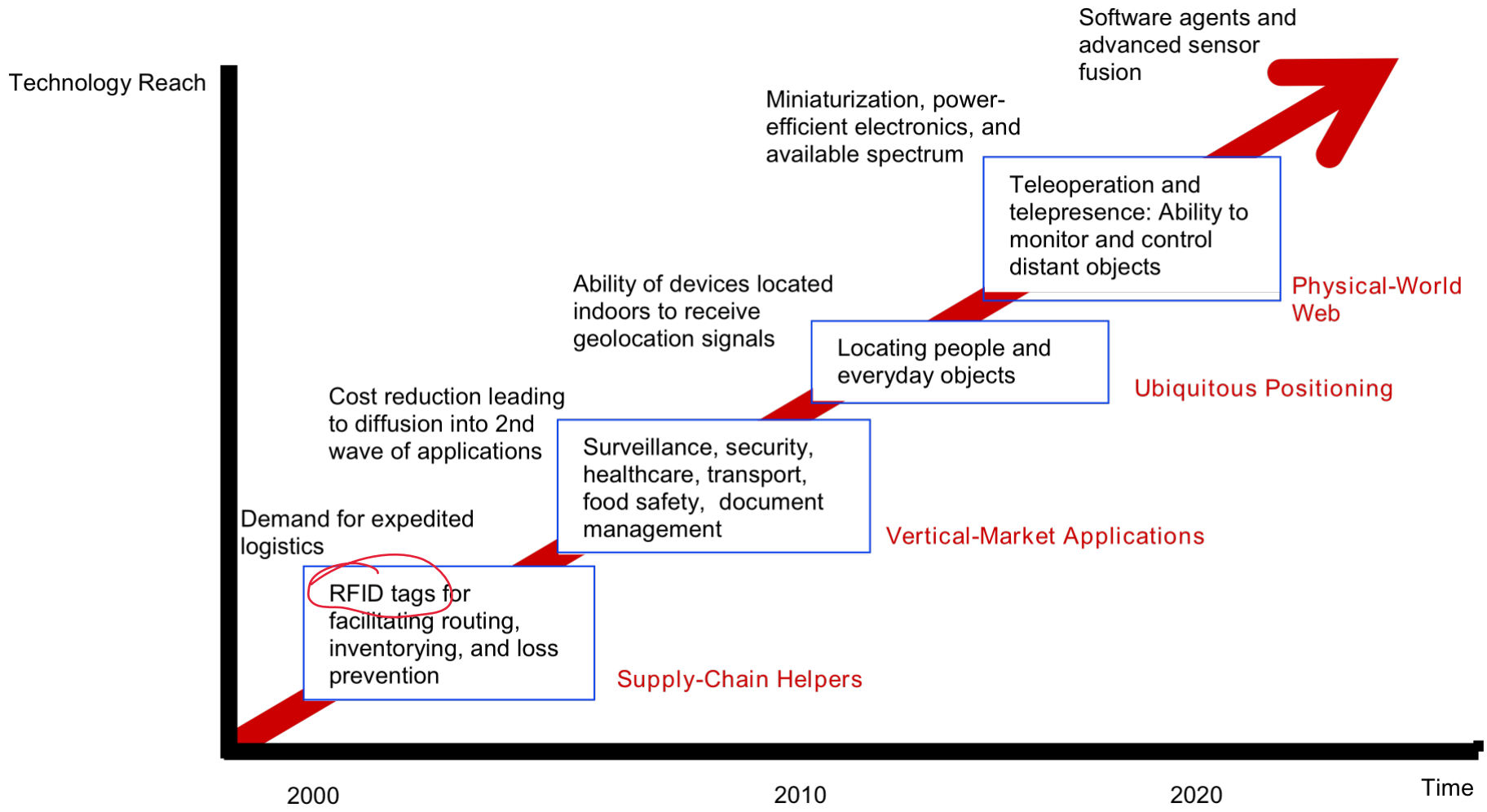
Opportunities



Source: http://blog.trentonsystems.com/internet-of-things-crosses-business-personal-boundaries/

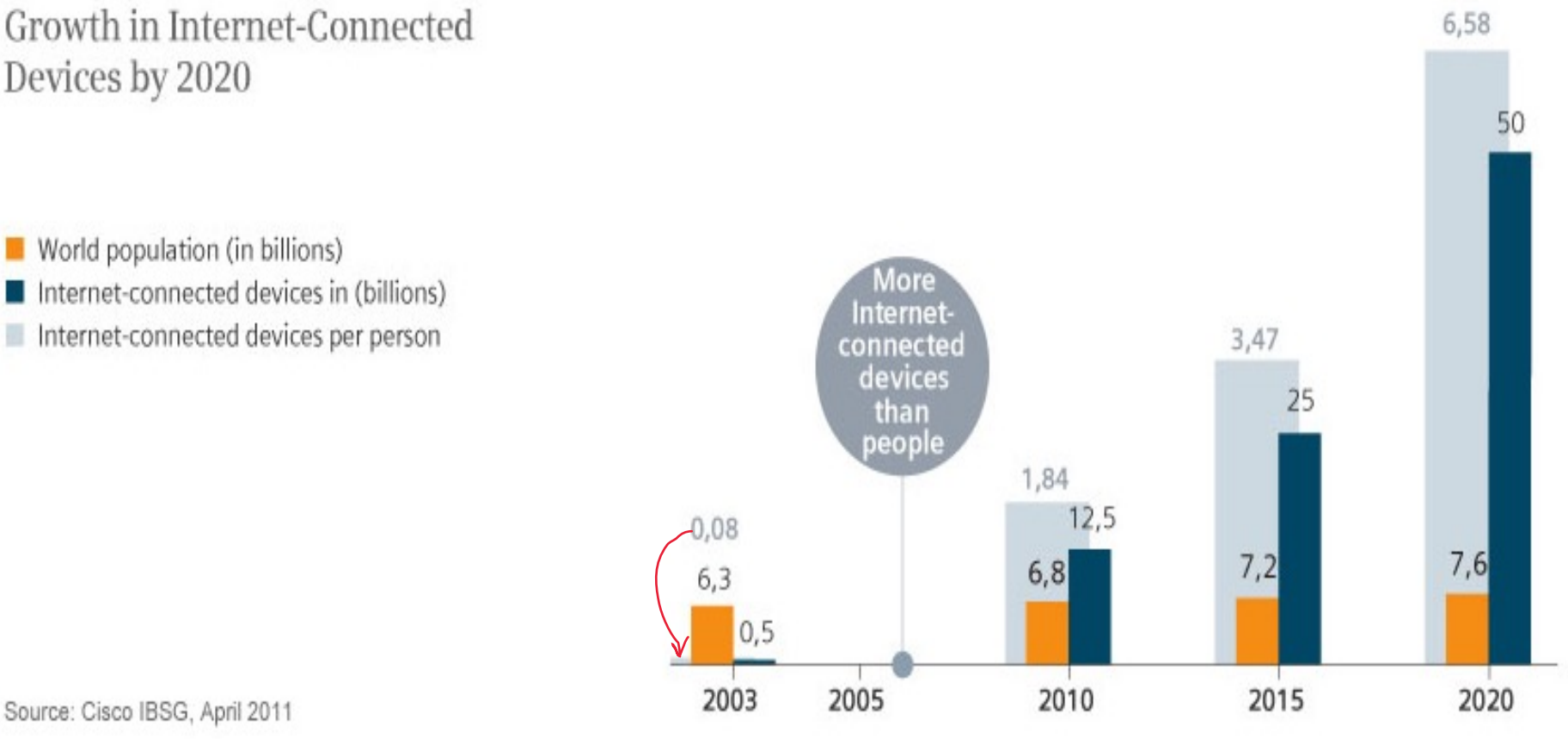
Technology trend

TECHNOLOGY ROADMAP: THE INTERNET OF THINGS



Source: SRI Consulting Business Intelligence

Internet-connected devices



Source: Siemens, http://www.siemens.com/innovation/apps/pof_microsite/_pof-fall-2012/_html_en/facts-and-forecasts-growth-market-of-the-future.html

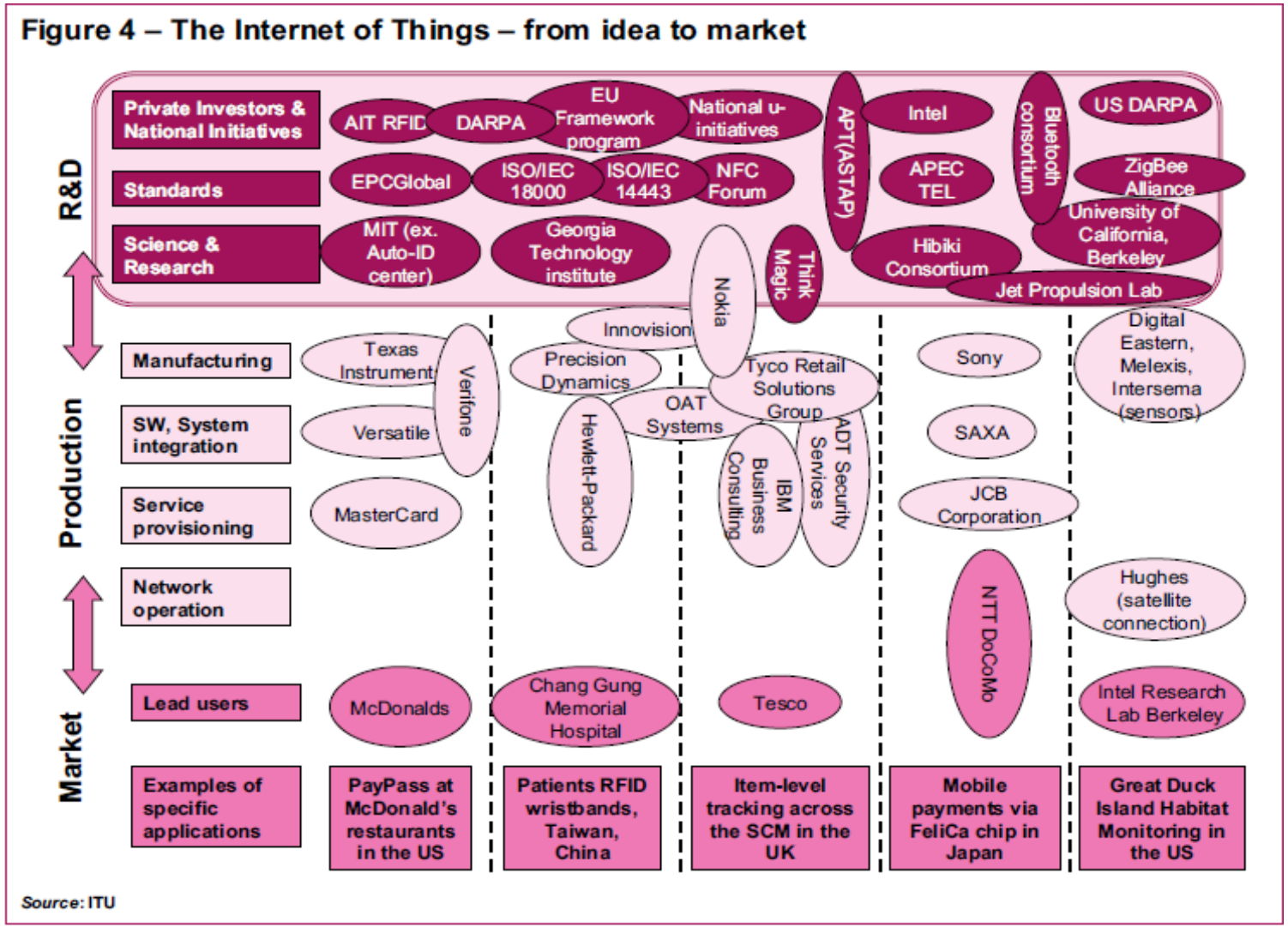
Global Data Generation

- Everyday around 20 quintillion (10^18) bytes of data are produced (Source: http://www-01.ibm.com/software/data/bigdata/).

- This data includes textual content (unstructured, semi-structured, structured) to multimedia content (images, video and audio), on a variety of platforms (enterprise, social media, and sensors).

- Big data!

State of the Art of IoT



An Outlook on Hardware

Main Hw Components

- The hardware of smart objects consists of four main components
  - [mouth/ears] Communication device (either a wireless transceiver with antenna or wired connection)
  - [brain] Microcontroller: defines the smart object's behavior
  - [hands] Set of sensors or actuators: interact with the physical world
  - [food] Power source: needed because the smart object contains electrical circuits

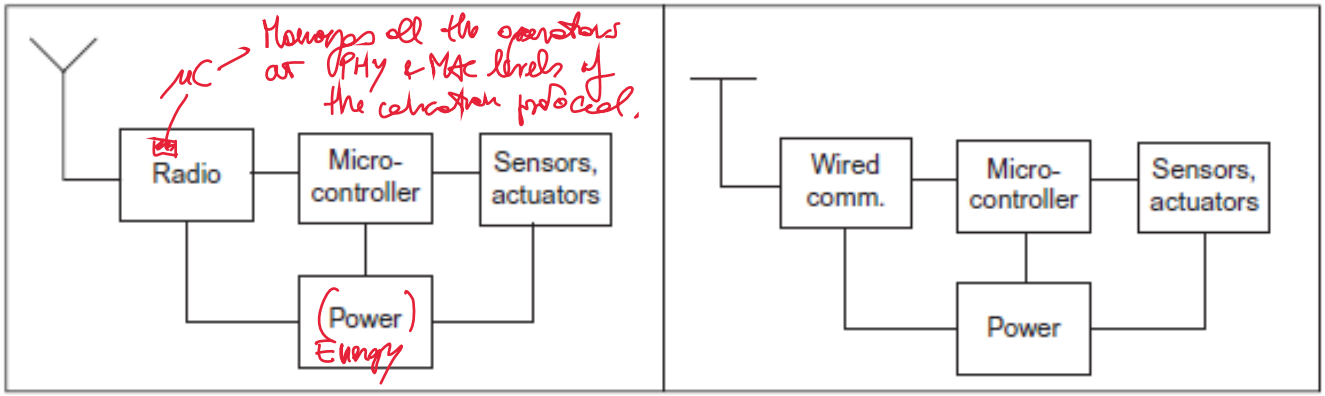


FIGURE 11.1

The hardware architecture of two smart objects: a radio-equipped, wireless smart object (left) and a smart object with wired communication (right).

An example: MicaZ

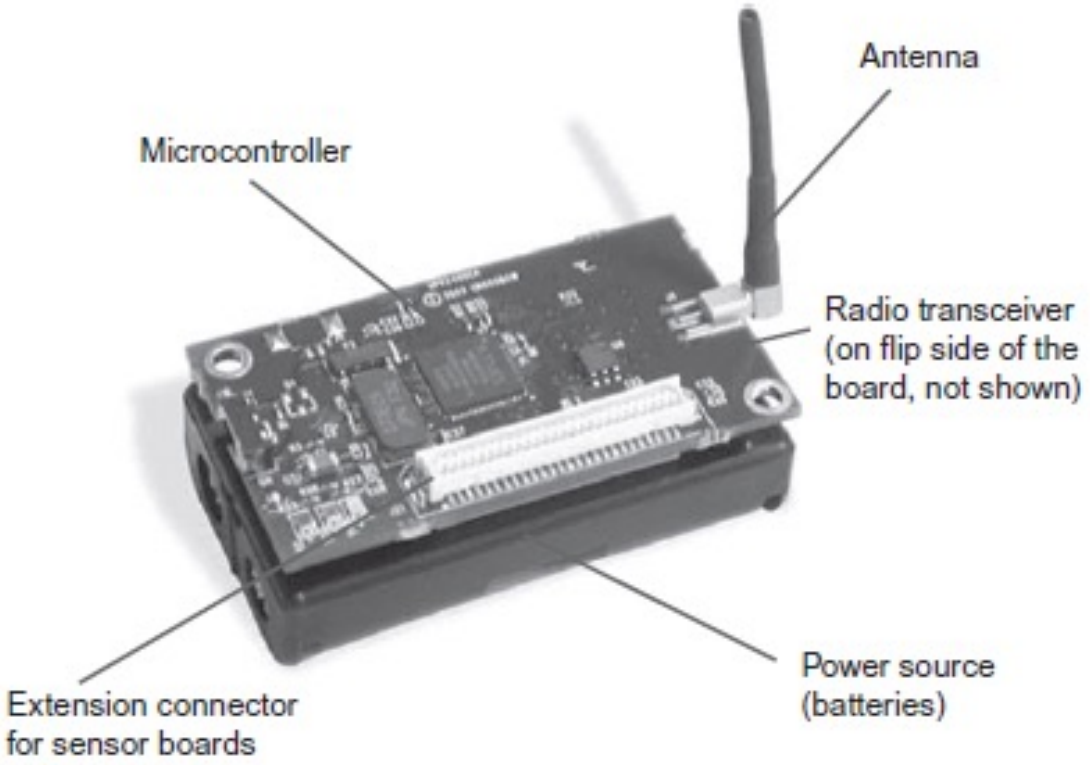


FIGURE 11.2

A MicaZ prototype board with a microcontroller, power source, communication device, and sensor connectors.

Photo courtesy of Crossbow Technology.

Communication Device

- Different types of radio transceivers have different amounts of built-in processing capabilities: (simple) send and receive individual bits of information into the air; (more powerful) package the information into packets, form headers, and even encrypt and decrypt the data using secure encryption methods
- Radio is usually the most power-consuming component (10 times the power consumption of the microcontroller or the sensors): modulation (tx) and demodulation (rx) –> idle listening as expensive as transmission
- Saving energy: duty cycling → need synchronization (see later)

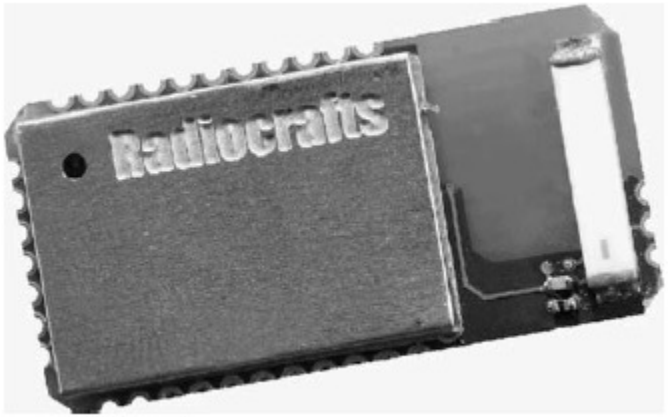


FIGURE 11.3
Texas Instruments CC2430 single-chip radio transceiver with integrated 8051 microcontroller and on-board antenna manufactured by Radiocrafts. The size of the board is $1.2 \times 1.0\,cm^2$.
Photos courtesy of Radiocrafts.

Example of system-on-chip (SoC)

Microcontroller

- Runs the software of the smart object & connects radio with sensors/actuators
- Microcontroller: microprocessor with built-in memory, timers, and hw for external connections
- Microcontrollers used in smart objects (cost lower than 10$) much smaller than those used in PCs
  - Smart object: a few Kbytes of on-chip memory ; clock rate of a few MHz
  - PC: Gbytes of memory; clock rate of several GHz
- Memory: ROM (behaviour) and RAM (temporary sw for tasks)

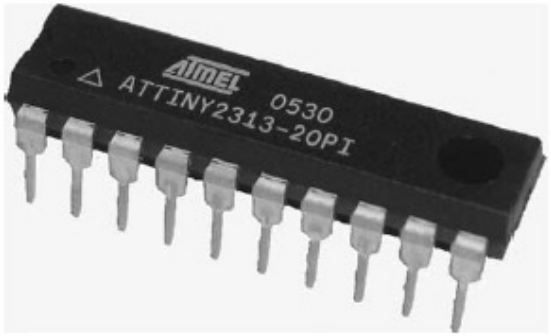


FIGURE 11.4

An Atmel ATTINY 2313 smart object microcontroller with 20 pins. The ATTINY 2313 has 2 kB of ROM and 128 bytes of RAM. This represents the low end of smart object microcontrollers.

Table 11.1 Microcontrollers Used in Smart Objects

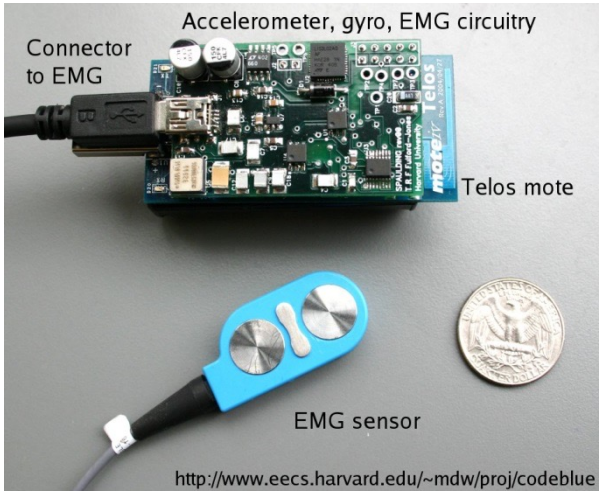
Name	Manufacturer	RAM (kB)	ROM (kB)	Current consumption (active/sleep), mA
MSP430xF168	Texas Instruments	10	48	2/0.001
AVR ATmega128	Atmel	8	128	8/0.02
8051	Intel	0.5	32	30/0.005
PIC18	Microchip	4	128	2.2/0.001

Note: Each manufacturer has several models of each device. This table lists only one example from each manufacturer.

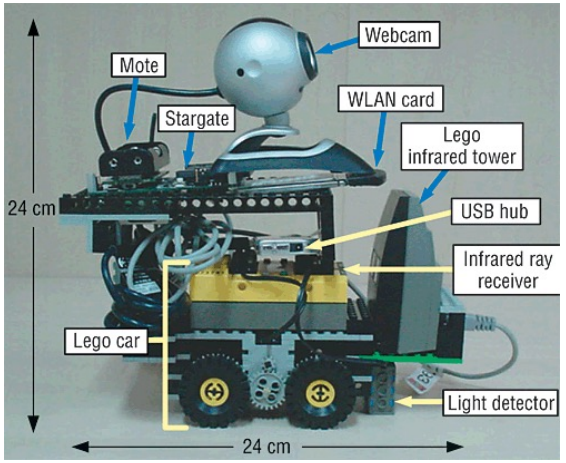
Sensors and Actuators

- Sensors: sense the environment
- Actuators: affect or change the environment
- From very simple (temperature sensor) to complex (ultrasonic range device or camera for fence monitoring)

Video recording



ElectroMioGraphy



Energy Power Sources

- Most common power source: battery (Lithium, non rechargeable)
  - (+) lifetime of many years on a single standard AA-size battery (low-power electronics and power-saving software)
  - (-) difficult to recycle
- Power scavenging: harvest from the environment (solar cells, piezoelectricity, electromagnetic energy-RFID)

FIGURE 11.5
PowerCast P2100 module converts radio energy from the air into electricity that powers an electrical circuit. The module requires an external antenna to pick up the radio signals (not shown).

Table 11.2 Different Power Sources for Smart Objects, Their Maximum Current Draw, and the Amount of Charge They Store

Power source	Typical maximum current (mA)	Typical charge (mAh)
CR2032 button cell	20	200
AA alkaline battery	20	3000
Solar cell	40	Limitless
RF power	25	Limitless

Energy Consumption

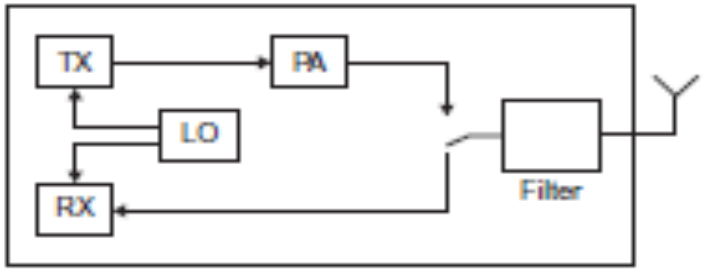


FIGURE 11.17

Schematic drawing of a radio transceiver.

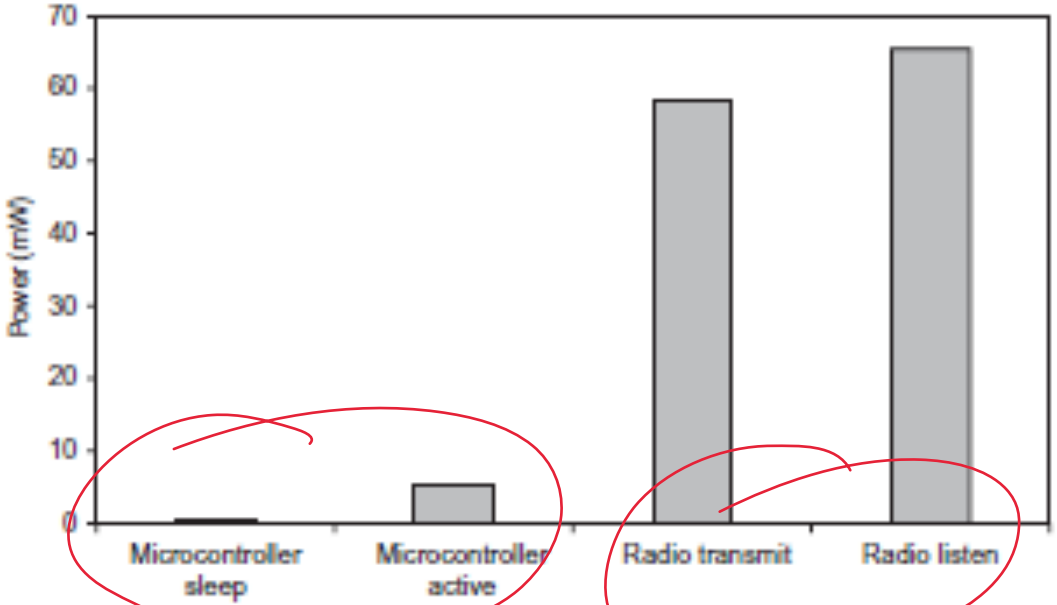


FIGURE 11.16

Power consumption of the microcontroller and the radio on the Tmote Sky smart object prototyping board. The power consumption of the microcontroller sleep mode is so low that it is not visible in the graph.

An Outlook on Software

Programming a Smart Object

- Programs written in a high-level programming language (e.g., C, maybe Java) and compiled with a compiler to machine code for the microcontroller
- Machine code written into ROM, run when the smart object is powered on
- Operating systems are used by smart objects (see next slide)
- Memory footprints makes programming (e.g., communication protocol stack) challenging (see later)

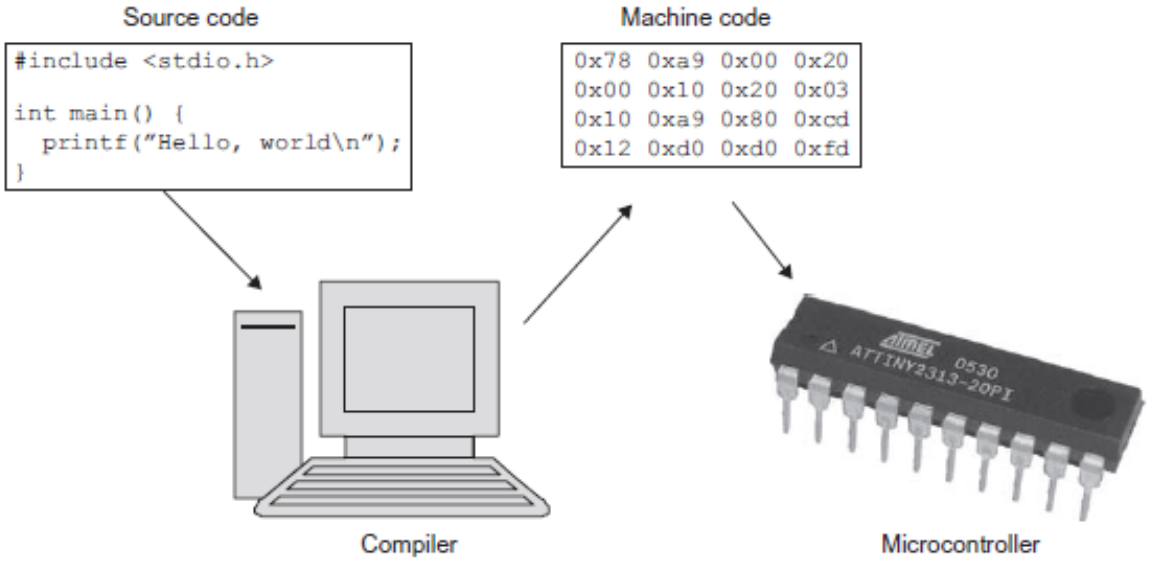


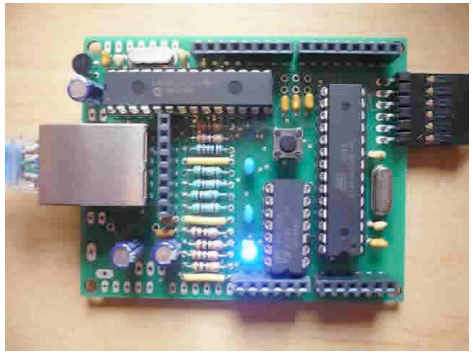
FIGURE 11.8

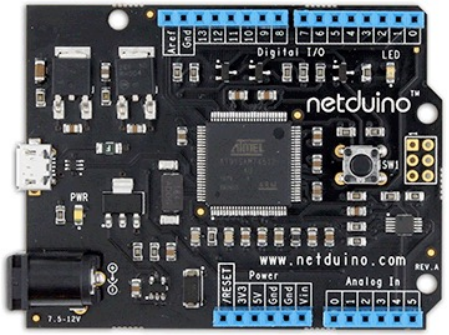
The process of software development for a smart object. Source code is compiled to machine code that is written to the ROM of the smart object microcontroller.

Operating Systems

- Very different from general-purpose operating systems
  - Tailored to specific requirements of smart objects and hardware constraints (memory constraints and processing speed)
  - No graphical interface (hidden deeply within the microcontroller)
- Main operating systems (all open-source)
  - **Contiki** (2003-, SICS-Thingsquare): first OS which guarantees IP communication with uIP TCP/IP stack (5 kB), incorporating also uIPv6 (11 kB) in 2008; useful simulator (Cooja); many components used in industry (uIP TCP/IP, protothread programming abstraction); written in C (ported to more than 12 microprocessors/controllers); direct communication with other IP-based application and web services (e.g., Twitter, mail)
  - **TinyOS** (2000-, Berkeley-Stanford): focus on wireless sensor networks and academia; used uIP for IP communication and has recently incorporated independent IPv6 implementation; written in nesC; event-driven programs
  - **FreeRTOS**: designed for embedded systems, providing real-time guarantees (control applications); provides TCP/IP support (uIP and lwIP); uses a preemptive, multi-threaded (multiple programs simultaneously running-large memory) programming model; ported to over 50 microprocessors/controllers

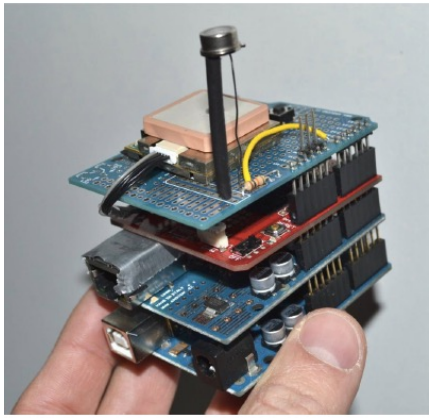
IoT Devices ("homegrown")





Netduino

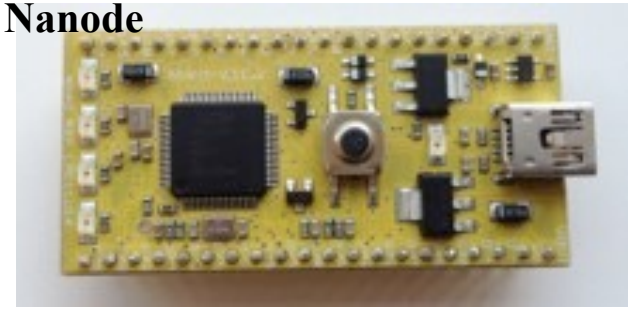
COTS
Comercial
off
the
Shelf



52North SenseBox light

Arduino + Ethernet + GPS ? ~ 100 €

Nanode



mbed.org ARM® Cortex™-M0

- Open source electronic platforms
- Must add sensors, comms, positioning
- Prices dropping rapidly

Twine

WiFi, temperature and vibration sensors, $100+ plus cloud service

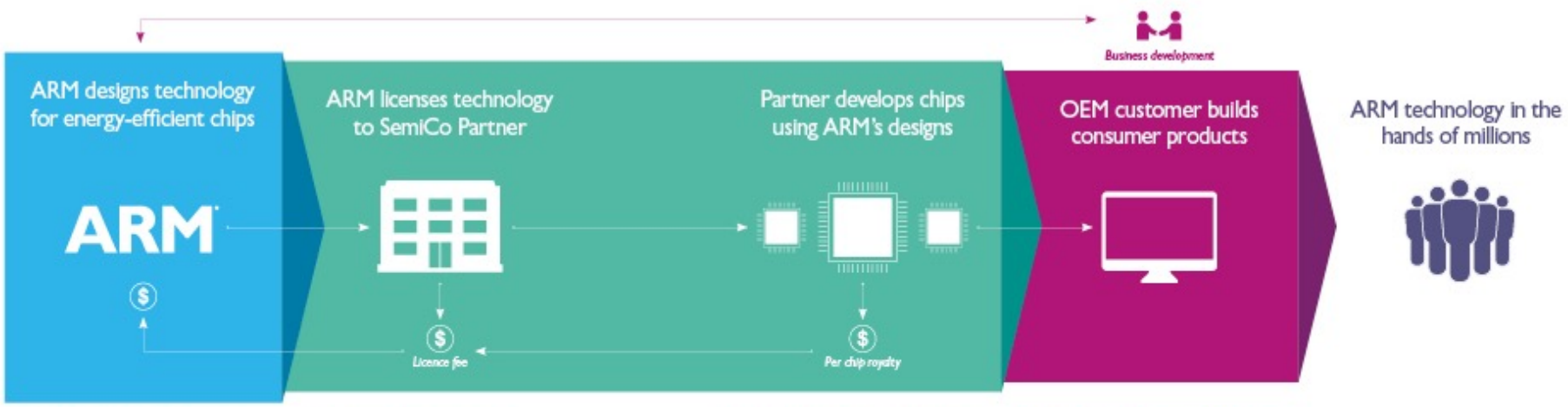
© 2012 Open Geospatial Consortium

Illustrative Spectrum of Embedded Resources: Cortex

MCU capability	Class	System Cost $ (IC + required crystals, memory, etc)	RAM (B)	Flash (B)	OS	Connectivity	Application	Capability
High	Low A	12	>16M	>64M	Linux, Android	Full IP stack	Gateway, local hub, smart pen	Full internet stack, rich OS (Linux, Android), typical gateway device
Med High	M4	8	1M	1M				
Med	M3, M4	5	48k	256k	RTOS, Embedded OS	6LowPAN	Smart watch, washing machine controller	COAP + DTLS, application developed using rich ecosystem of frameworks and libraries, capable of some audio and GUI
Low	M0, low M3	2	10k	64k	Embedded OS, none	6LowPAN	Light fitting, motion sensor	COAP (+DTLS?), 6LowPAN (just), quite constrained application
Ultra low	M0+	<1	<4k		none	Proprietary low-rate / short range	Single use temperature sensor, mm^3	Highly constrained, carefully optimised code, uplink via an application gateway device

The Architecture for the Digital World® ARM®

ARM Business Model



1. ARM designs technology to go into energy-efficient chips for a broad range of end markets. ARM licenses each design to multiple semiconductor companies. Every company pays an upfront licence fee to gain access to the design.

2. The semiconductor company will incorporate the ARM technology design into their chip. It can take 2–3 years to build a chip and a further year for an OEM to build their product, such as a digital TV or mobile phone, containing the chip. When the chip starts to ship, ARM receives a royalty on every chip that uses the design. Typically our royalty is based on the price of the chip.

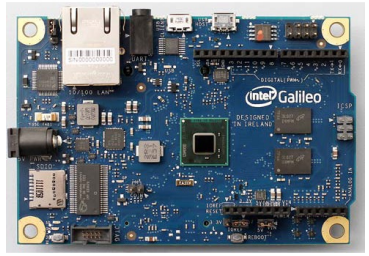
3. Each ARM design is suitable for a wide range of end applications and so can be re-used in different chip families addressing multiple markets. Each new chip family generates a new stream of royalties. An ARM design may be used in many different chips and may ship for over 20 years.

4. ARM's flexible business model has proven to be suitable for different technologies, end products, licensing strategies and a broad range of ecosystem Partners.

Deployment of an IoT testbed

150+ nodes

(3/4G, WiFi, 802.15.4, SubGHz, UWB, BLE,NFC,...)



intel Galileo

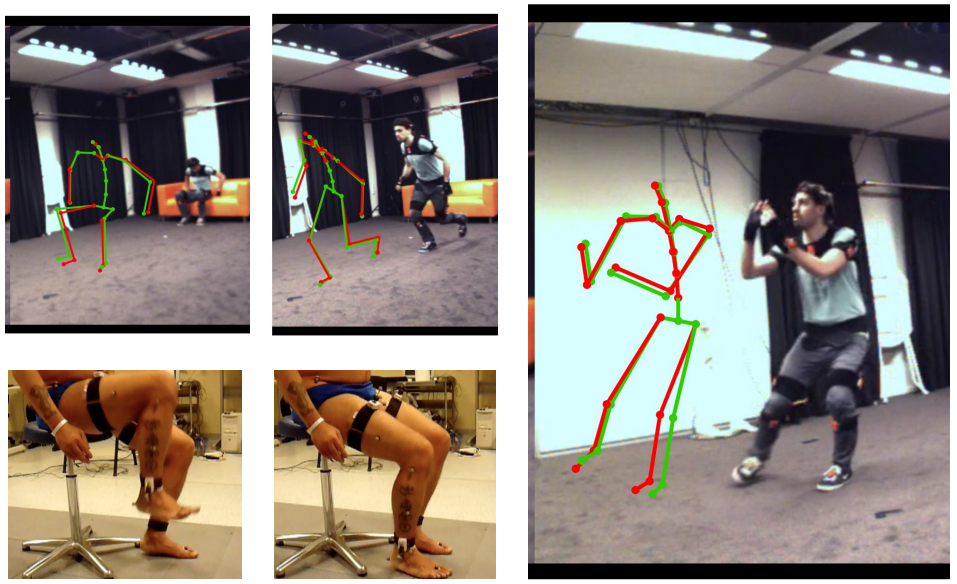
Affiliated with the Intel Galileo University Program

Body Sensor Networks

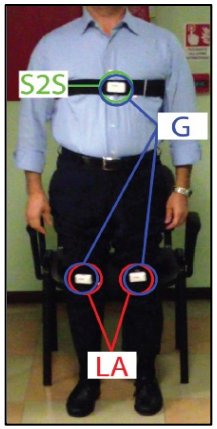
A Body Sensor Network (BSN) is a sensor network composed by nodes placed on a user body.The network nodes are, typically, devices supplied with inertial (i.e., accelerometers and gyroscopes) and magnetic (i.e., magnetometers) sensors.

Applications:

- motion capture
- posture recognition
- activity classification
- tele-rehabilitation (Parkinson's Disease)
- rehabilitation monitoring (post-stroke)



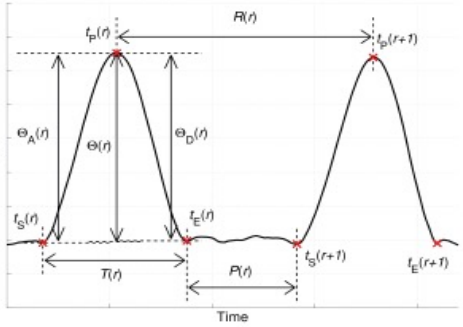
BSN-based Evaluation of PD Motor Tasks

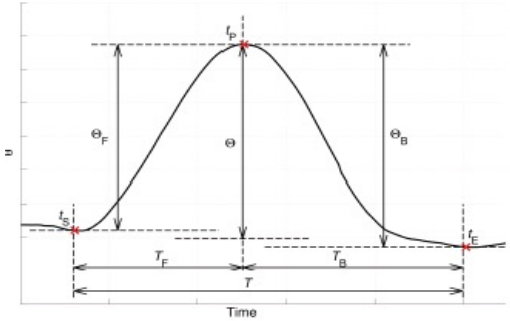


- **Main goal**: provide an automatic assessment of PD motor tasks
  - 3 UPDRS tasks: Leg Agility (LA), Sit-to-Stand (S2S), and Gait (G)
  - More objective and continuous evaluation of the symptoms
- **Hardware:** low-complexity BSN (3 IMUs - Shimmer)
- **Kinematic Characterization:** extraction of task-relevant features in both time and frequency domains

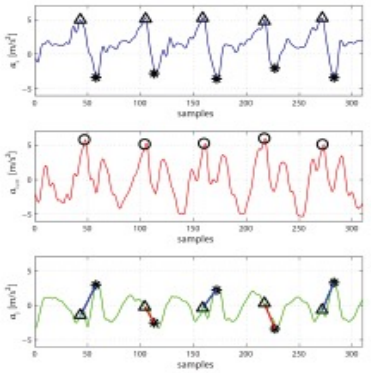
ISTITUTO AUXOLOGICO ITALIANO
Istituto di ricovero e cura a carattere scientifico

LA

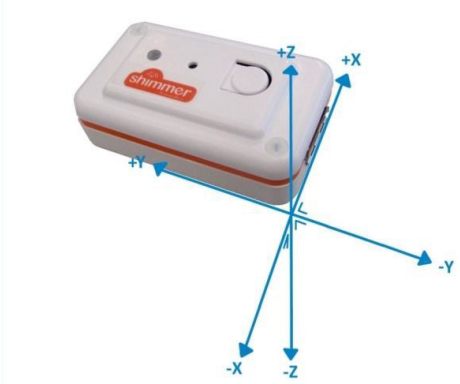




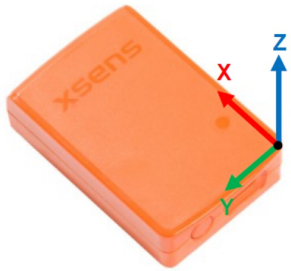
S2S



IMUs @ IoT Lab



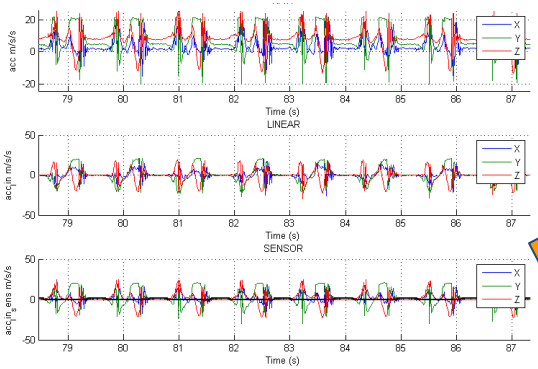
- Shimmer 2r node
- Small-sized, low-power, Bluetooth interface
- 9 DOF: 3-axial accelerometer, 3-axial gyroscope, 3-axial magnetometer
- Sampling Rate: 102.4 Hz

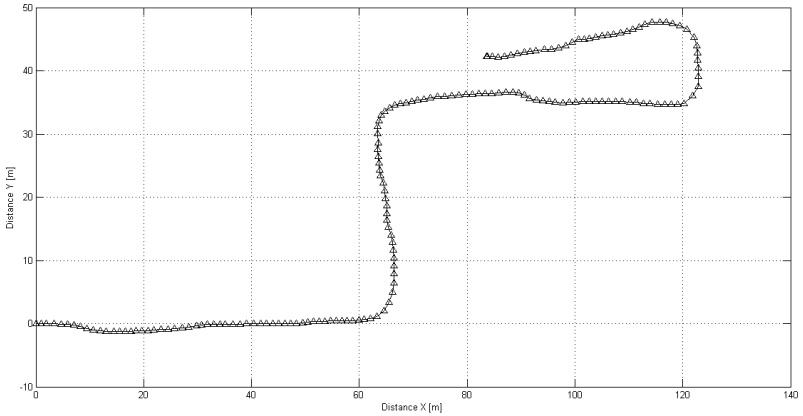


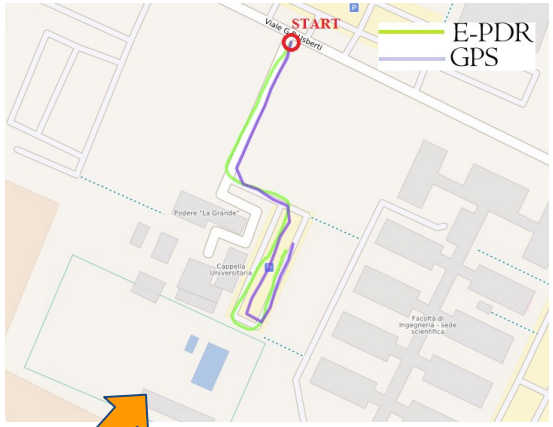
- 6 MTw Awinda IMU (accelerometer, gyroscope, magnetometer, barometer)
- Awinda Full Body straps set
- Awinda USB receiving dongle
- 6 slot Xsens docking station

- LPMS-B2
- Size: 39 x 39 x 8 mm
- Bluetooth Low Enery (LE) 4.1
- Communication distance < 20 m
- Orientation range Roll: ±180°; Pitch: ±90°; Yaw: ±180°
- Resolution < 0.01°
- Accuracy < 0. 5°(static), < 2° RMS (dynamic)
- Accelerometer 3 axes, ±2 / ±4 / ±8 / ±16 g, 16 bits
- Gyroscope 3 axes, ±125 / ±245 / ±500 / ±1000 / ±2000 °/s, 16 bits
- Magnetometer 3 axes, ± 4 / ± 8 / ± 12 / ± 16 gauss, 16 bits

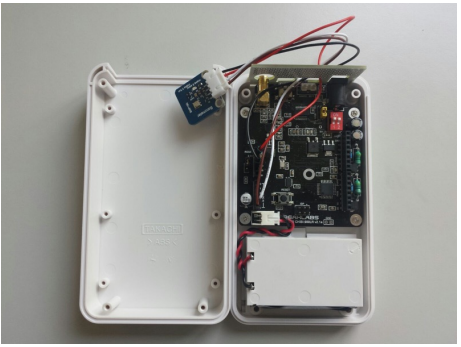
Inertial navigation and tracking: outdoor

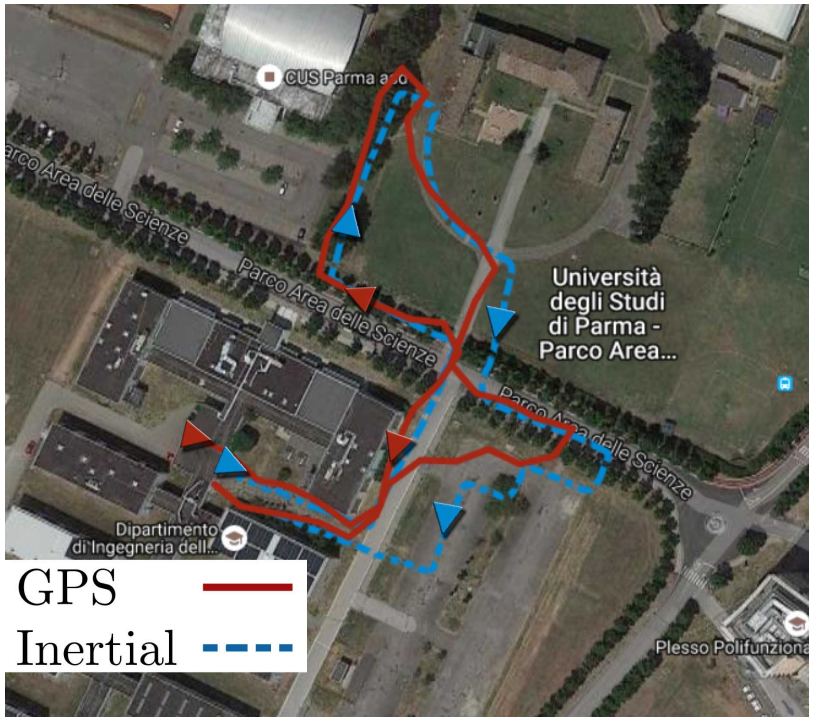






Evolutions of Inertial Navigation





- Hybrid inertial/barometric sensor → multi-floor navigation
- Navigation and tracking on non-flat surfaces (e.g., cross training)

A Demo (IPIN 2016, October 2016)

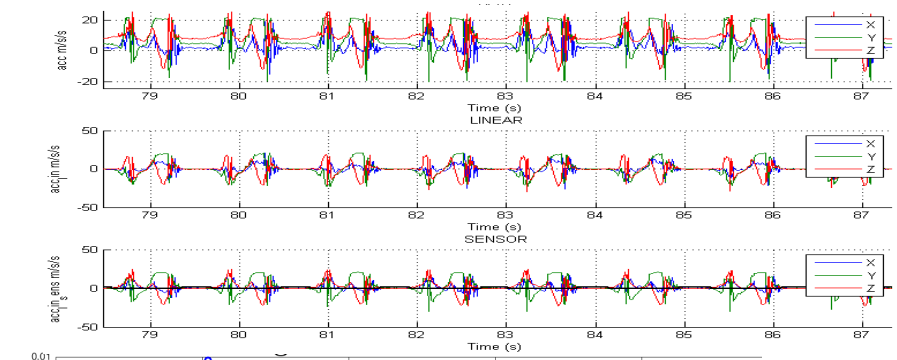
IoT Lab @ IPIN 2016
SEVENTH INTERNATIONAL CONFERENCE ON
INDOOR POSITIONING AND INDOOR NAVIGATIO
October 4-7, 2016 / Alcalá de Henares, Madrid, Spain

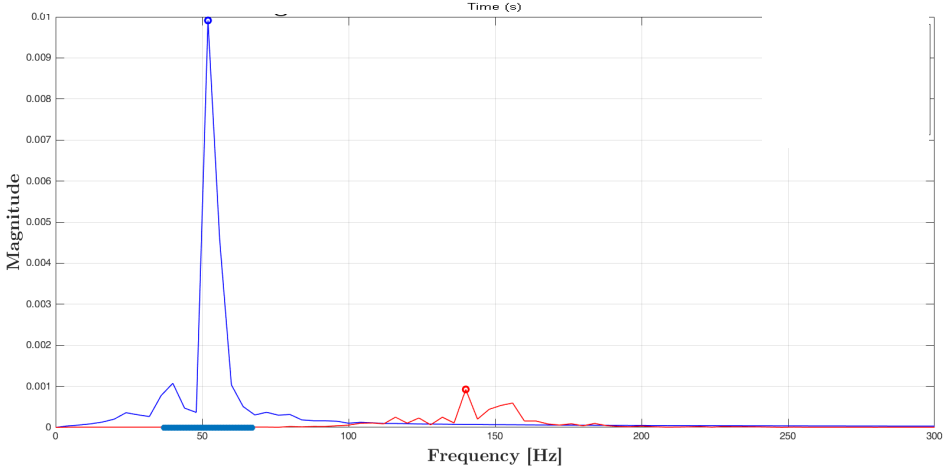
Track 2 Competition
3-rd Place Classified

- IPIN 2016 indoor localization competition (Alcalá de Henares, Madrid)

- IPIN tracks
  - Track 1: Smartphone-based
  - Track 2: Pedestrian dead reckoning positioning
  - Track 3: Smartphone-based (off-site)
  - Track 4: Indoor mobile robot positioning

Vibration Analysis



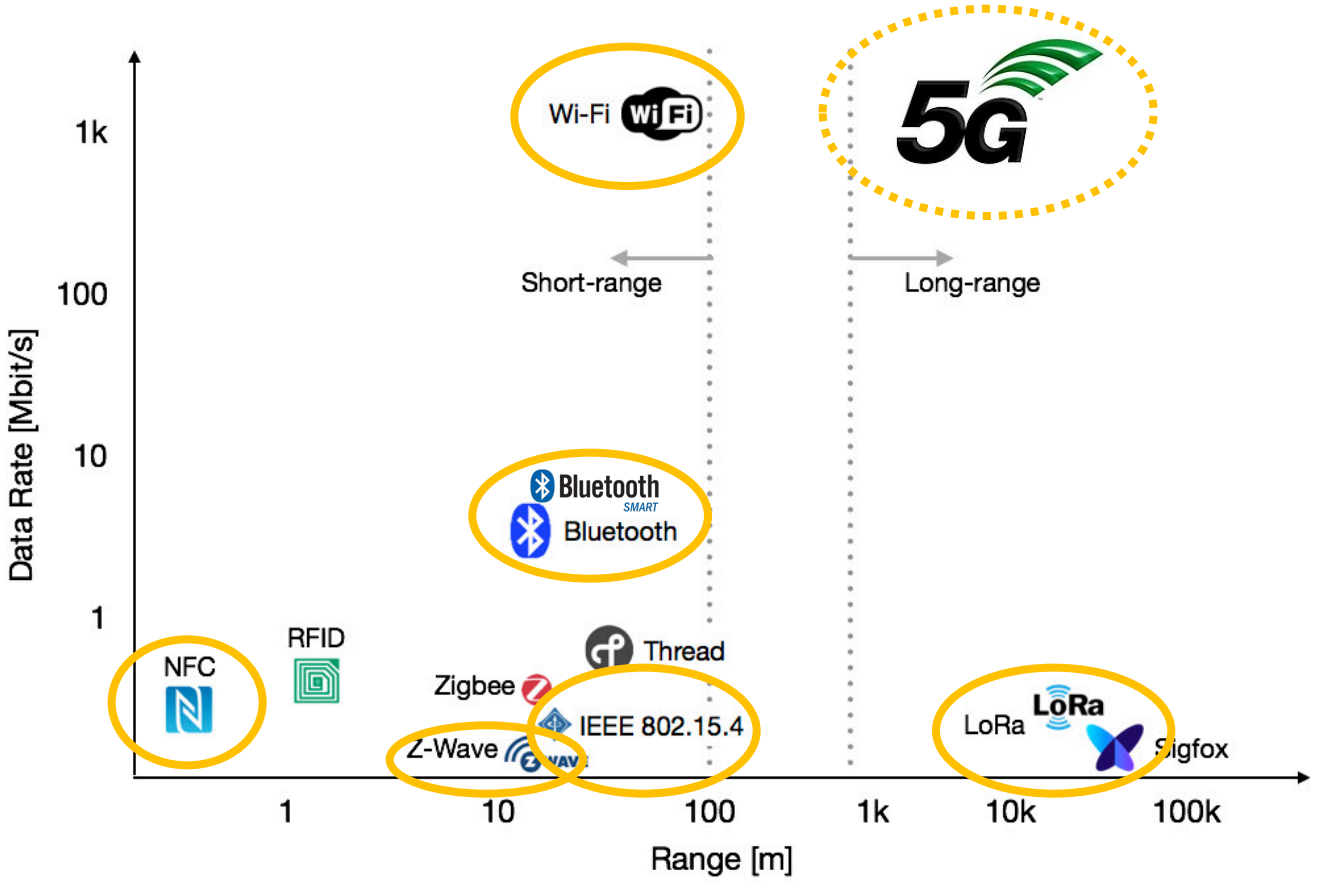


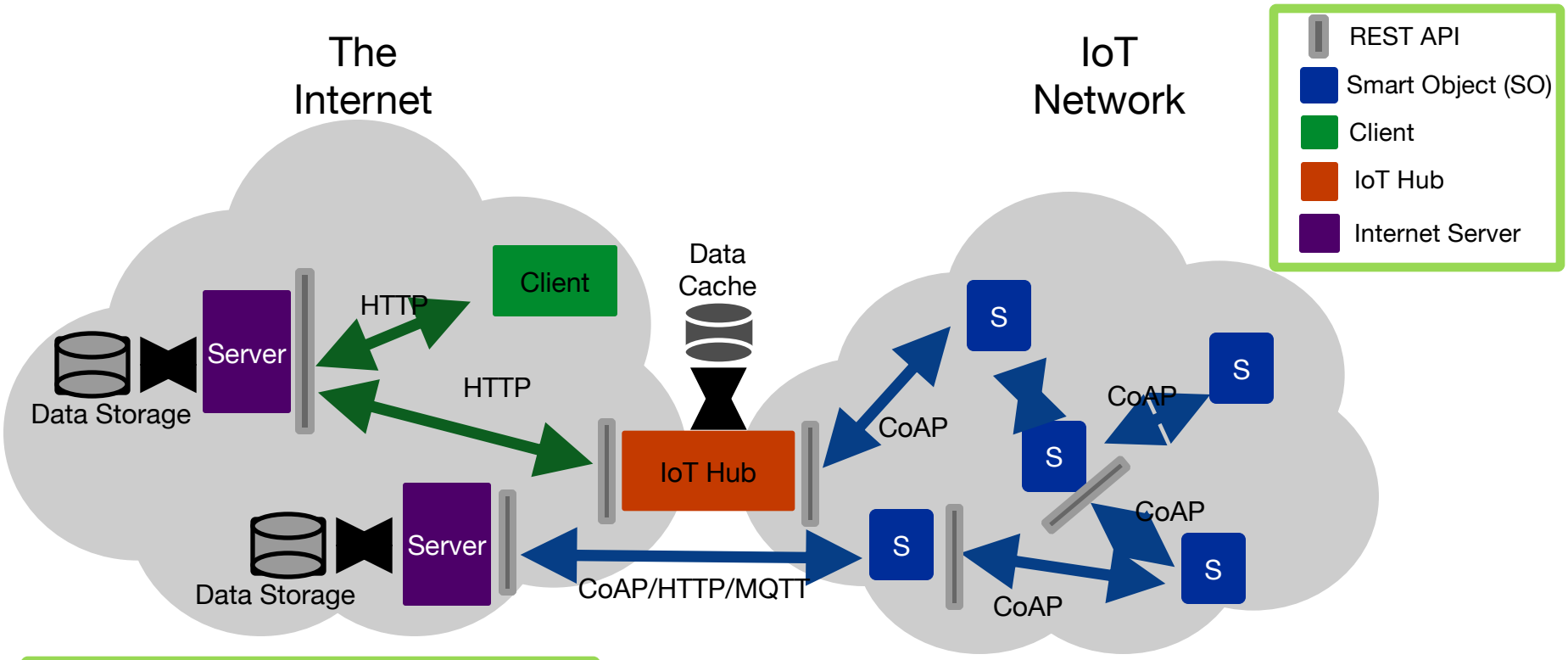
- Simple and non-invasive
- Combine an inertial sensor with an embedded system
- Extension: combine with audio sensor

Application: real-time RPM estimation

- Hybrid time/frequency processing

“Our” Wireless Communication Technologies

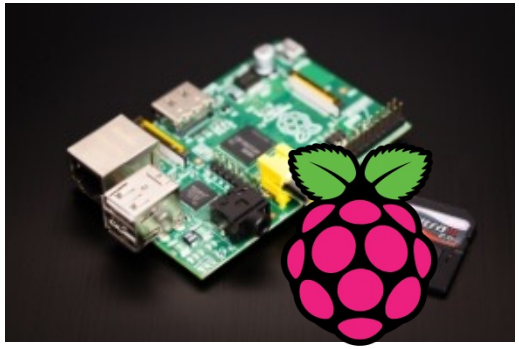




Data Storage, Processing, Aggregation and Distribution

Data Source/Stream, manage Incoming and Outgoing requests/responses SO

Infrastructure nodes



RaspberryPi

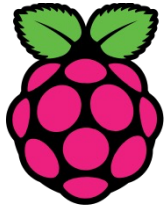
- 802.11
- Ethernet
- Camera

Arduino Yùn

- 802.11
- Ethernet

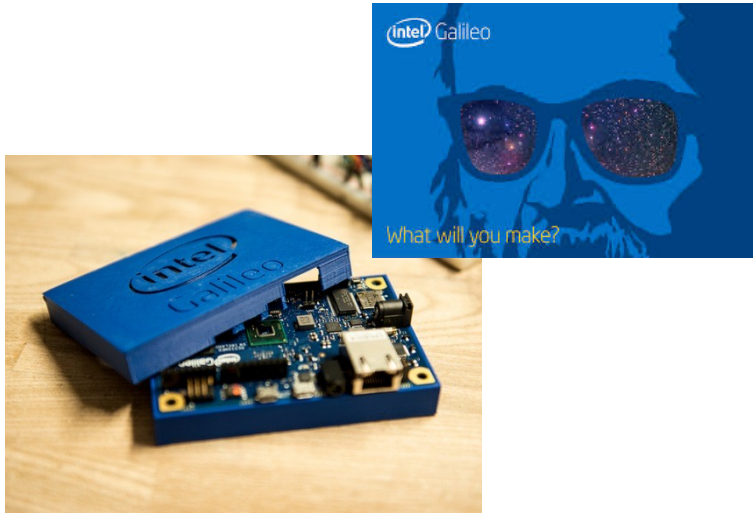
12 AP Cisco

- 802.11
- Ethernet



Gateway = RaspeberryPi & Telos B

- 802.15.4
- 802.11
- Ethernet



INTEL Galileo

- Ethernet
- WiFi (dongle)

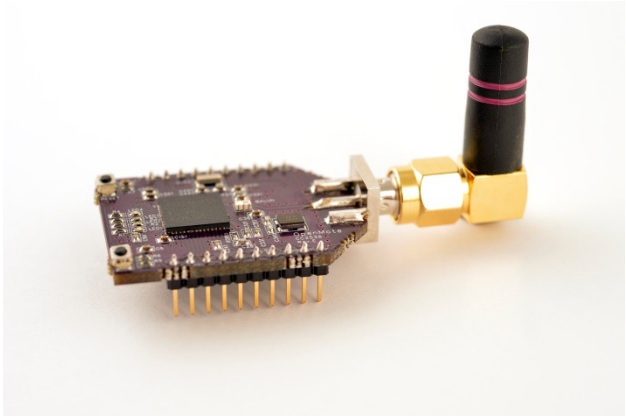
Mobile nodes

Telos B

- 802.15.4
- Contiki

OpenMote

- 802.15.4
- Contiki



UDC. 2

Zolertia Z1

- 802.15.4
- Contiki

UNIVERSITÀ DI PARMA

IoT Lab

→ Lec. 3

Transport Layer Protocols: UDP and TCP

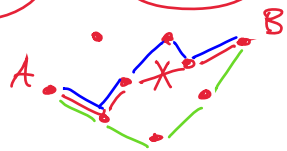
Main sources

- J. P. Vasseur and A. Dunkels, `Interconnecting Smart Objects with IP,` Morgan Kaufmann, 2010. Chapters 2 and 6.

- Princeton (COS 461: Computer Networks, Spring 2006) and Virginia Tech (Internet Teaching Lab, http://www.cs.virginia.edu/~itlab/) lectures

Original TCP/IP Goals (70's)

- Internet communication must continue despite loss of networks or gateways (" in the presence of link or node failures " in today's terms).



- The Internet must support multiple types of communication services.
- Internet architecture must:
  - accommodate a variety of networks (" networks " means link and physical layers)
  - permit distributed management of its resources
  - be cost-effective
  - permit host attachment with little effort
- Resources used in the Internet architecture must be accountable .

Scalability of TCP/IP

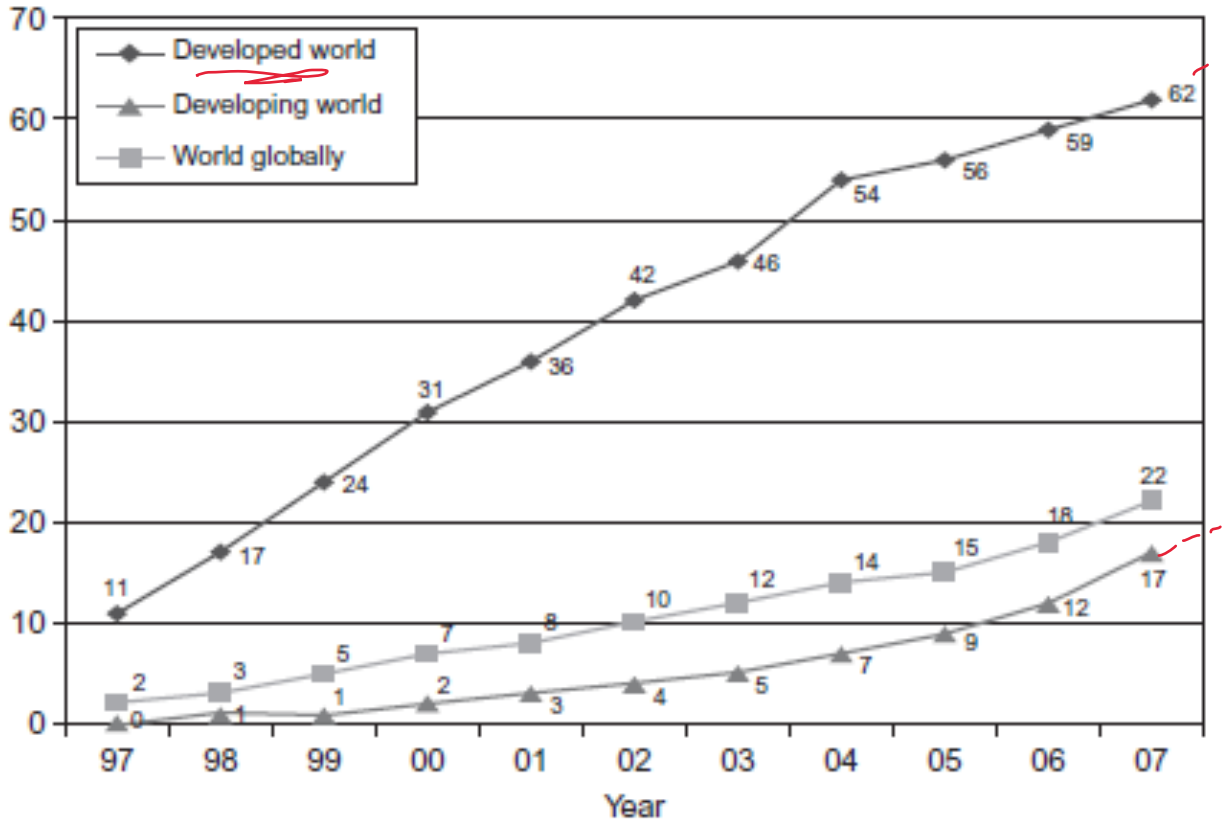


FIGURE 2.1

Level of penetration of the Internet regarding user number.

Protocol Stack (layers)

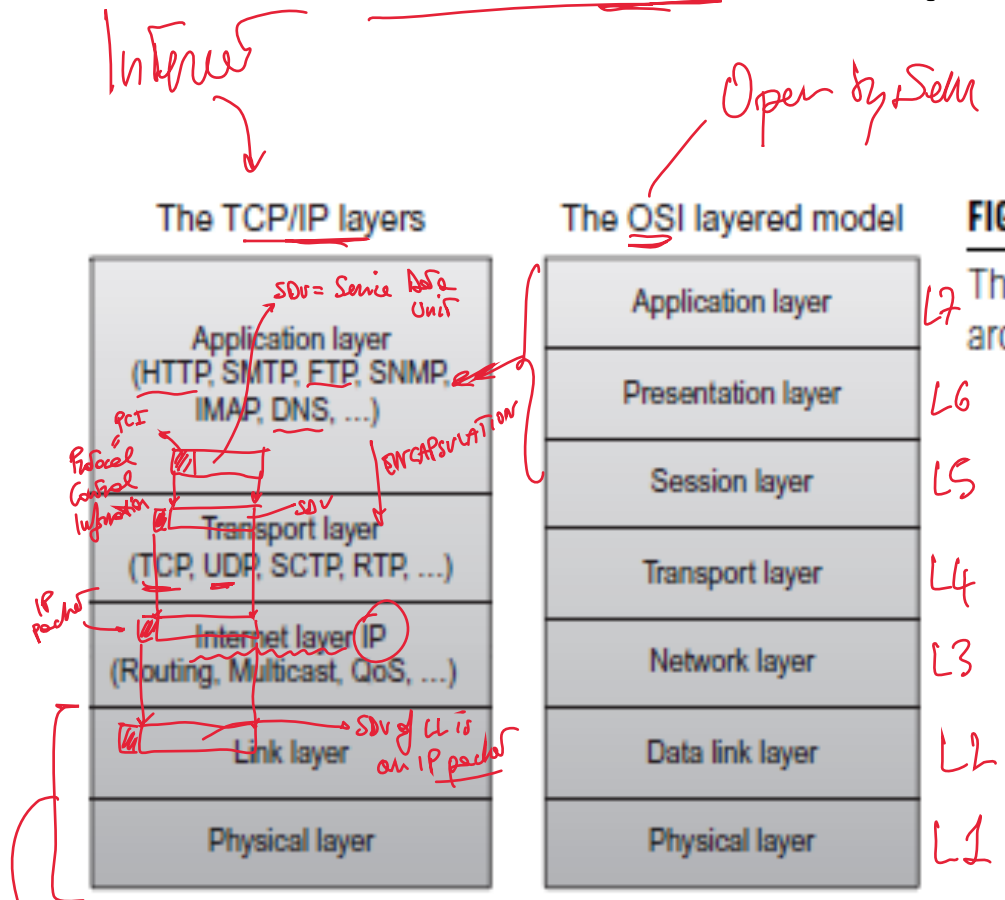
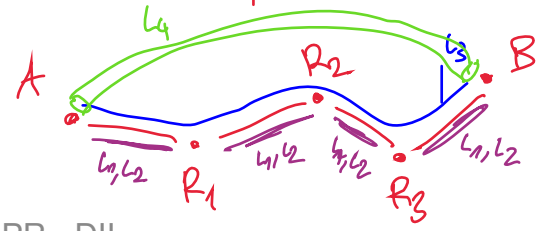


FIGURE 2.2
The concept of layering in the TCP/IP and OSI architectures.



Link Layer

- Usually refers to the physical and data link layers (the use of the PHY/MAC acronym is fairly common).
- Responsible for forwarding the IP packet on a link between two devices. This involves several functions such as: medium access control (MAC), error detection and (sometimes) retransmission, and flow control.
- Link layer protocol information is added in the form of a frame that carries the IP packet.
- Some links provide a very limited set of functions whereas others implement fairly sophisticated services that often include a link layer " routing " function.

Internet/Network Layer (IP)

- Responsible for providing an unreliable service for sending a packet between a source and a destination across the network, where host and routers are uniquely identified by their IP (IPv4 or IPv6) addresses, using a hierarchical addressing scheme.

- Routing is one of the main tasks accomplished by the IP layers

- Protocols such as ICMP (Internet Control Message Protocol) and IGMP (Internet Group Management Protocol) for multicast traffic are both considered part of the IP layer.

Transport Layer

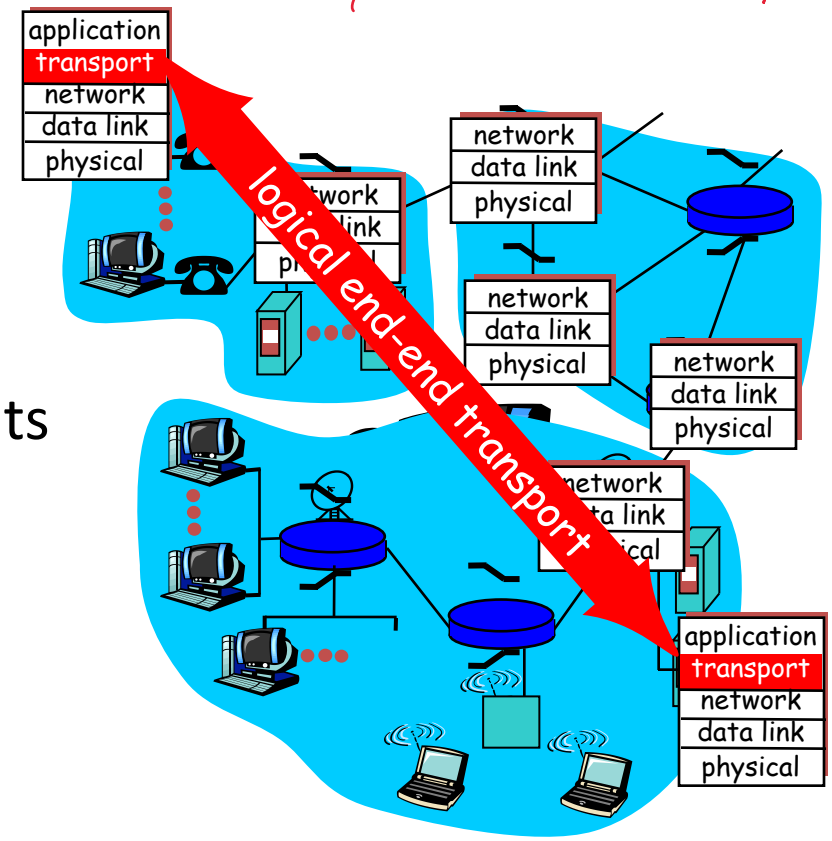
- Responsible for end-to-end communication between two devices.
- TCP (states are maintained at the end devices) provides a reliable transport mechanism with error detection and retransmission, flow control using dynamic windowing techniques, security mechanisms, and so forth.
- UDP (stateless) is mostly used for application addressing and optional error detection (done by an optional checksum in IPv4; note that the UDP checksum is mandatory in IPv6).
- Other transport protocols:
  - the message stream transport protocol SCTP (Stream Control Transmission Protocol) offers additional capabilities such as the bundling of multiple user messages in a single SCTP packet, the support of multi-homing, and so forth.
  - Real-time Transport Protocol (RTP) is another transport protocol designed for realtime applications such as streaming audio traffi c and video .

Application Layer

- Refers to higher level protocol(s) that supports the applications.
- The list of application layer protocols is fairly long, but few well known include:
  - File Transfer Protocol (FTP)
  - Trivial File Transfer Protocol (TFTP)
  - Simple Network Management Protocol (SNMP)
  - HypertextTransport Protocol (HTTP)
  - Network News Transfer Protocol (NNTP)
  - Telnet

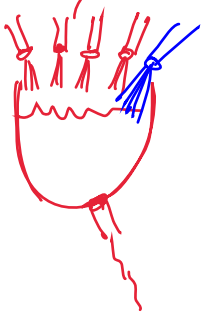
Transport Protocols

- Provide *logical communication* between application processes running on different hosts
- Run on end hosts
  - Sender: breaks application messages into segments, and passes to network layer
  - Receiver: reassembles segments into messages, passes to application layer
- Multiple transport protocol available to applications
  - Internet: TCP and UDP



Internet Transport Protocols

- Datagram messaging service (UDP)
  - No-frills extension of "best-effort" IP
- Reliable, in-order delivery (TCP)
  - Connection set-up
  - Discarding of corrupted packets
  - Retransmission of lost packets
  - Flow control
  - Congestion control
- Other services not available
  - Delay guarantees
  - Bandwidth guarantees



Transport Protocols in the Internet

- The Internet supports 2 transport protocols

UDP - User Datagram Protocol

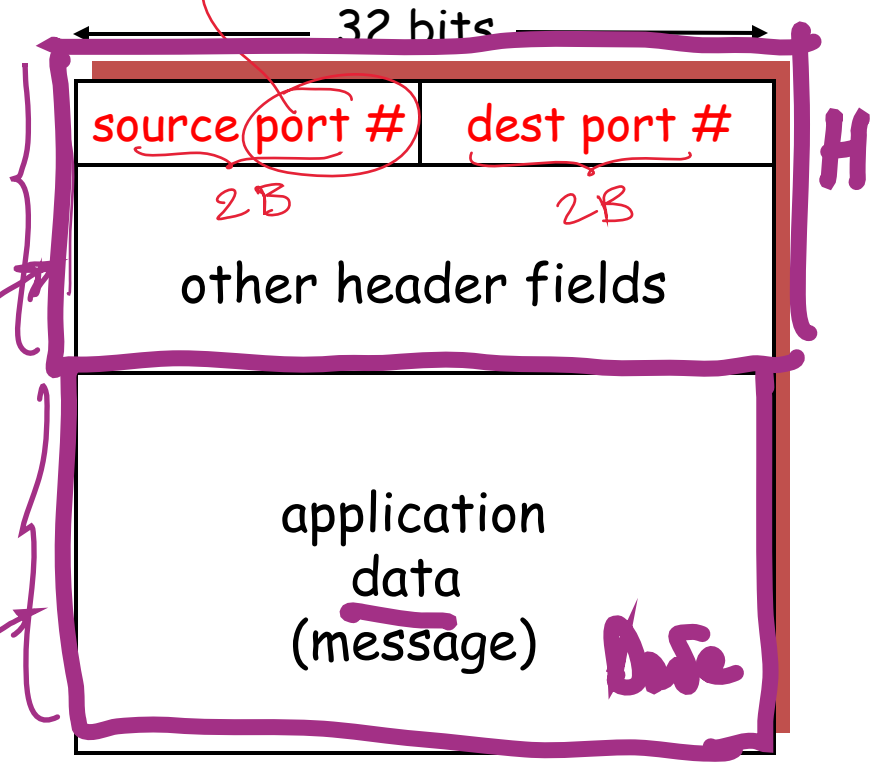
- datagram oriented
- unreliable, connectionless
- simple
- unicast and multicast
- useful only for few applications, e.g., multimedia applications
- used a lot for services
  - network management (SNMP, Simple Network Management Protocol), routing (Routing Information Protocol, RIP), naming (DNS), etc.

TCP - Transmission Control Protocol

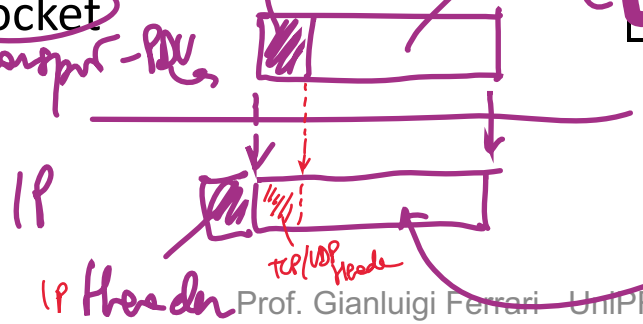
- stream oriented
- reliable, connection-oriented
- complex
- only unicast
- used for most Internet applications:
  - web (http), email (smtp), file transfer (ftp), terminal (telnet), etc.

Multiplexing and Demultiplexing

- Host receives IP datagrams
  - Each datagram has source and destination IP addresses
  - Each datagram carries one transport-layer segment
  - Each segment has source and destination port number
- Host uses IP addresses and port numbers to direct the segment to appropriate socket



TCP/UDP segment format



User Datagram Protocol (UDP)

- UDP is the simplest protocol in the TCP/IP suite
- Lightweight communication between processes
  - Avoid overhead and delays of ordered, reliable delivery
  - Send messages to and receive them from a socket
- UDP in a nutshell:
  - IP plus port numbers to support (de)multiplexing
  - Optional error checking on the packet contents

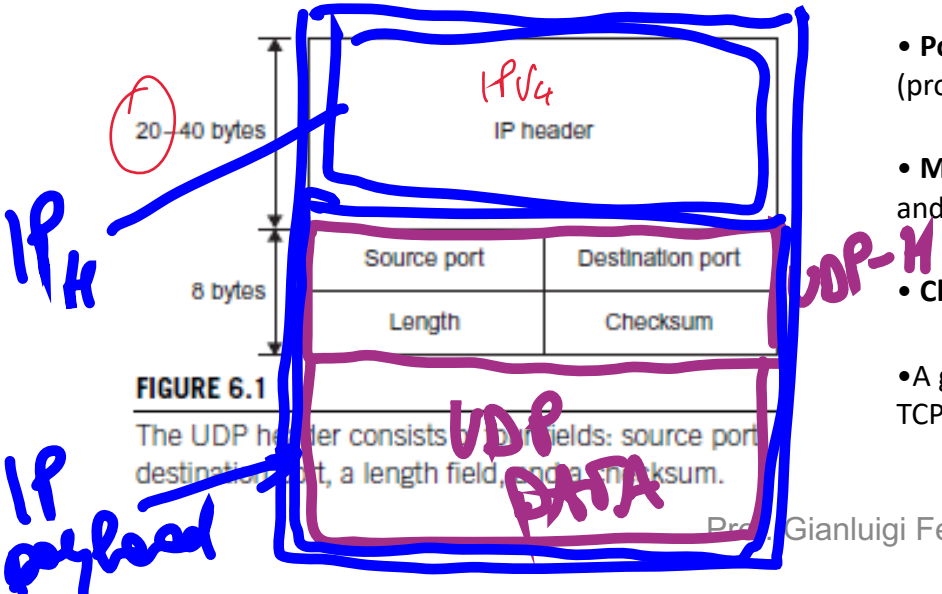


FIGURE 6.1

The UDP header consists of four fields: source port, destination port, a length field, and a checksum.

- **Port numbers** identify sending and receiving applications (processes). Maximum port number is $2^{16}-1= 65,535$
- **Message Length** is at least 8 bytes (I.e., Data field can be empty) and at most 65,535
- **Checksum** is for header (of UDP and some of the IP header fields)
- A globally unique address at the transport layer (for both UDP and TCP) is a tuple **<IP address, port number>**

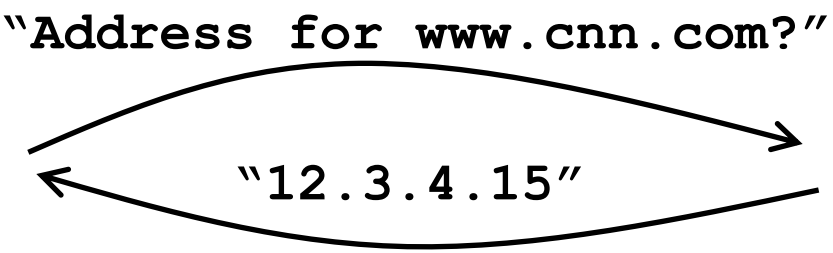
Why Would Anyone Use UDP?

- Finer control over what data is sent and when
  - As soon as an application process writes into the socket
  - … UDP will package the data and send the packet
- No delay for connection establishment
  - UDP just blasts away without any formal preliminaries
  - … which avoids introducing any unnecessary delays
- No connection state
  - No allocation of buffers, parameters, sequence #s, etc.
  - … making it easier to handle many active clients at once
- Small packet header overhead
  - UDP header is only eight-byte long
- **All excellent reasons for IoT scenarios**

Popular Applications That Use UDP

- Multimedia streaming
  - Retransmitting lost/corrupted packets is not worthwhile
  - By the time the packet is retransmitted, it's too late
  - E.g., telephone calls, video conferencing, gaming

- Simple query protocols like Domain Name System
  - Overhead of connection establishment is overkill
  - Easier to have application retransmit if needed



LEC. 3

Transmission Control Protocol (TCP)

- TCP provides a reliable byte stream on top of the best-effort packet service provided by the IP layer
- TCP hides the packet-oriented IP network beneath a virtual circuit abstraction, in which each virtual circuit is called a connection
- TCP provides a reliable stream transport service on top of the best-effort IP layer with three mechanisms:
  - Acknowledgments and retransmissions: All data sent with TCP are acknowledged by the receiver.
  - Sequence numbers: Every byte in the TCP byte stream is given a sequence number. The sequence numbers are used when matching acknowledgments with the corresponding data.
  - Sliding window (flow control): The receiver advertises how many bytes it is currently able to receive and the sender sends only as much data as the receiver can receive. As the receiver receives the data, it is able to receive more data.

An Analogy: Talking on a Cell Phone

- Alice and Bob on their cell phones
  - Both Alice and Bob are talking
- What if Alice couldn't understand Bob?
  - Bob asks Alice to repeat what she said
- What if Bob hasn't heard Alice for a while?
  - Is Alice just being quiet?
  - Or, have Bob and Alice lost reception?
  - How long should Bob just keep on talking?
  - Maybe Alice should periodically say "uh huh"
  - … or Bob should ask "Can you hear me now?" ☺

Some Take-Aways from the Example

- Acknowledgments from receiver
  - Positive: "okay" or "ACK"
  - Negative: "please repeat that" or "NACK"
- Timeout by the sender ("stop and wait")
  - Don't wait indefinitely without receiving some response
  - … whether a positive or a negative acknowledgment
- Retransmission by the sender
  - After receiving a "NACK" from the receiver
  - After receiving no feedback from the receiver

Segmented TCP Stream

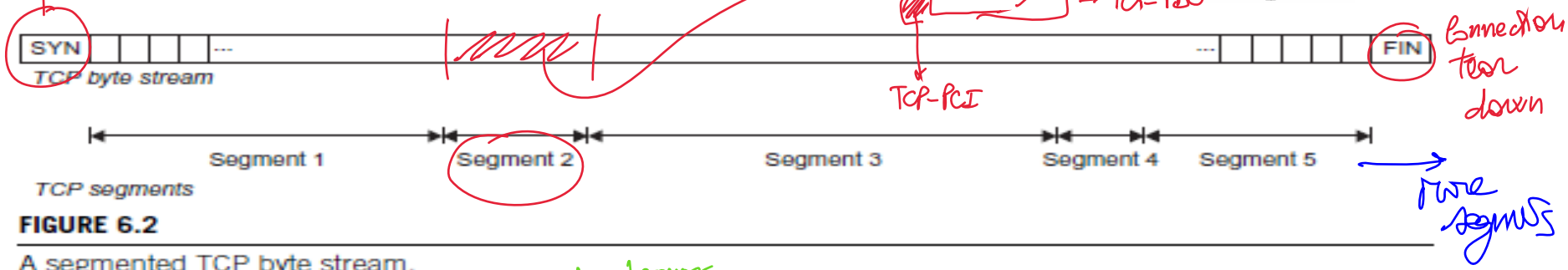


FIGURE 6.2

A segmented TCP byte stream.

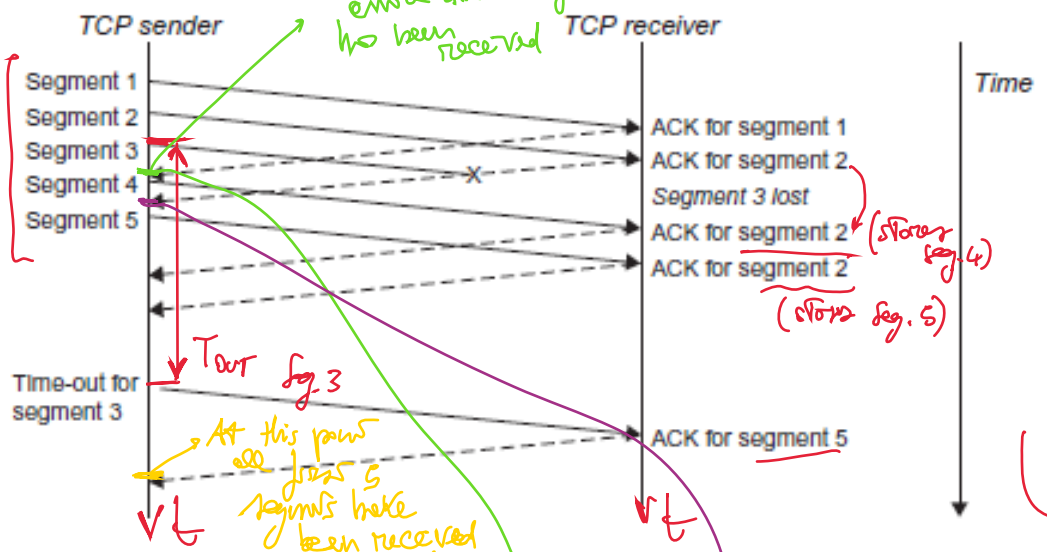


FIGURE 6.4

Loss of a TCP segment and the corresponding time-out.

(Fast retransmission: see p. 124)

Challenges of Reliable Data Transfer

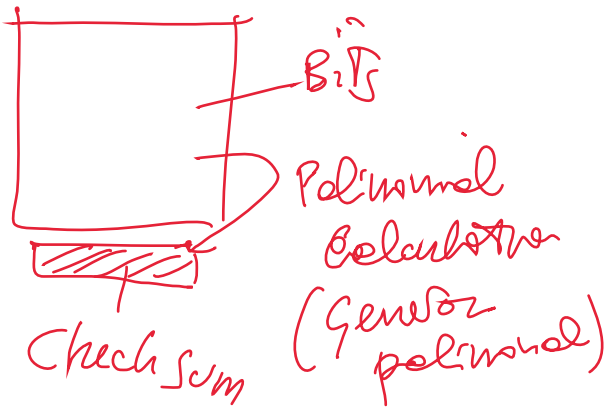
- Over a perfectly reliable channel (DREAM)
  - All of the data arrives in order, just as it was sent
  - Simple: sender sends data, and receiver receives data
- Over a channel with bit errors (ex. fiber optic channel)
  - All of the data arrives in order, but some bits corrupted
  - Receiver detects errors and says "please repeat that"
  - Sender retransmits the data that were corrupted
- Over a lossy channel with bit errors → (ex. wireless channel)
  - Some data are missing and some bits are corrupted
  - Receiver detects errors but cannot always detect loss
  - Sender must wait for acknowledgment ("ACK" or "OK")
  - … and retransmit data after some time if no ACK arrives

and no ACK will be transmitted

If a link is affected by fading, the packet will never reach the destination

TCP Support for Reliable Delivery

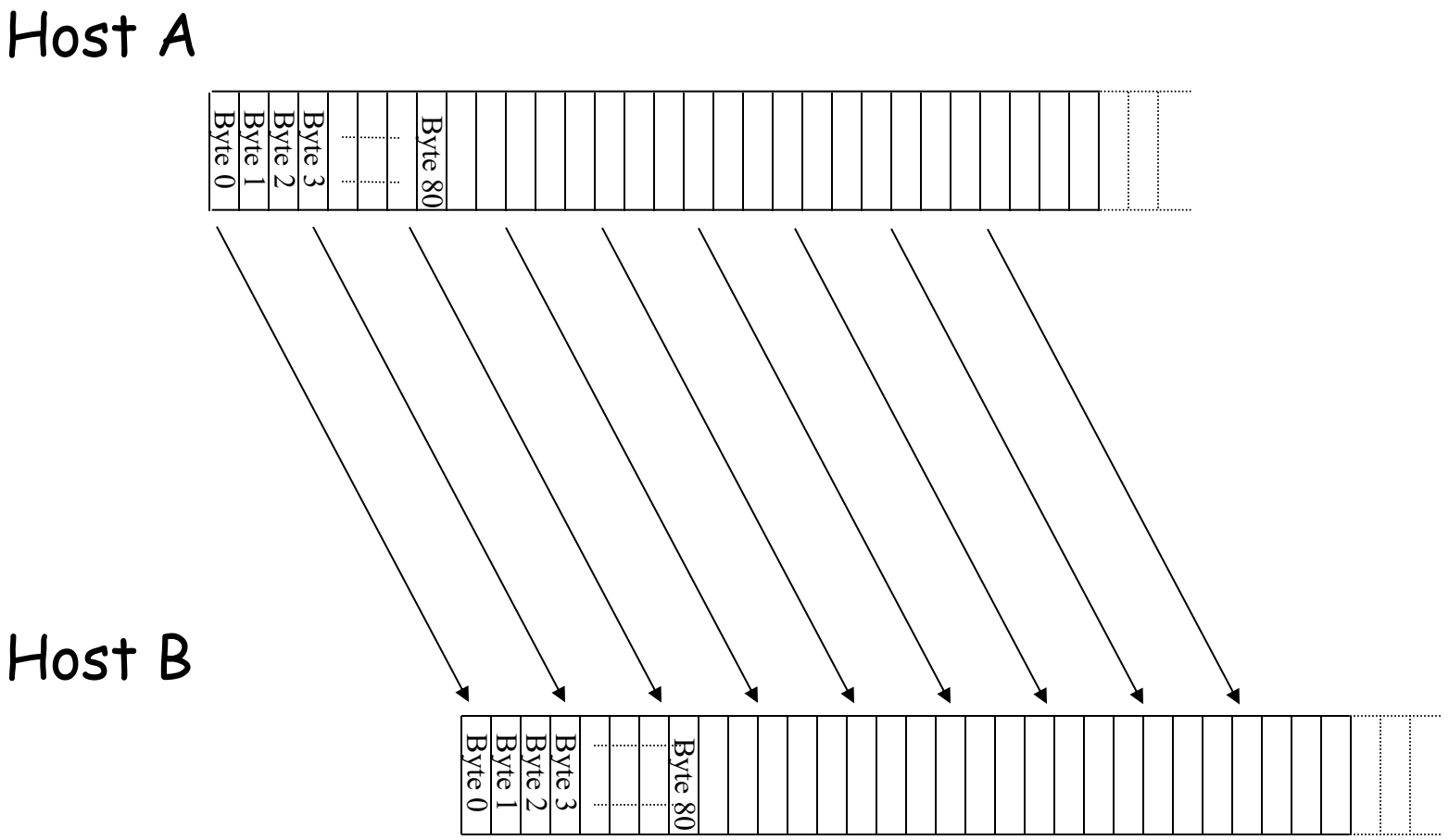
- Checksum
  - Used to detect corrupted data at the receiver
  - …leading the receiver to drop the packet
- Sequence numbers
  - Used to detect missing data
  - … and for putting the data back in order
- Retransmission
  - Sender retransmits lost or corrupted data
  - Timeout based on estimates of round-trip time
  - Fast retransmit algorithm for rapid retransmission



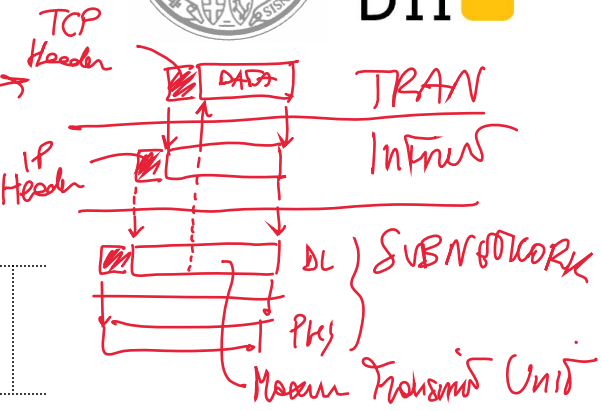
(No waiting for the time-out and immediate retransmission)

TCP Segments

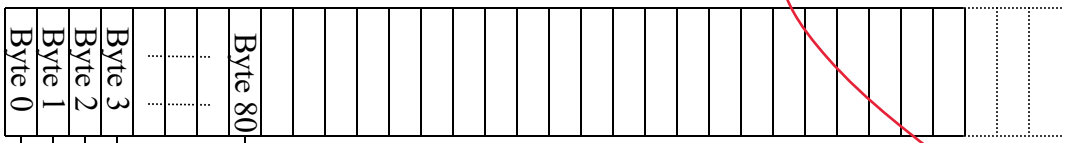
TCP "Stream of Bytes" Service



…Emulated Using TCP "Segments"



Host A



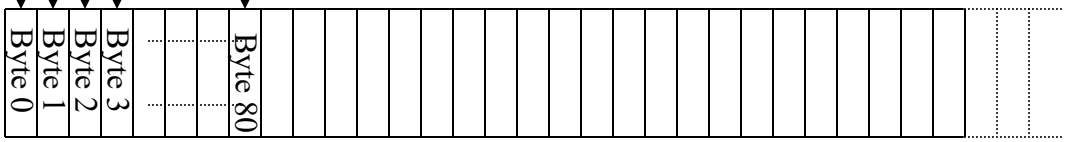
TCP Data

Segment sent when:

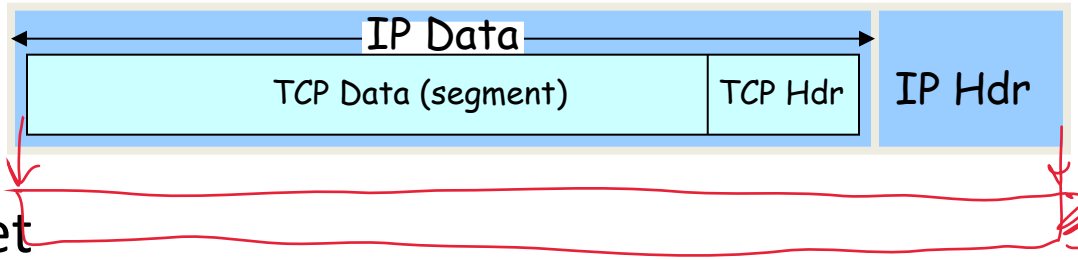
1. Segment full (Max Segment Size),
2. Not full, but times out, or
3. "Pushed" by application.

TCP Data

Host B

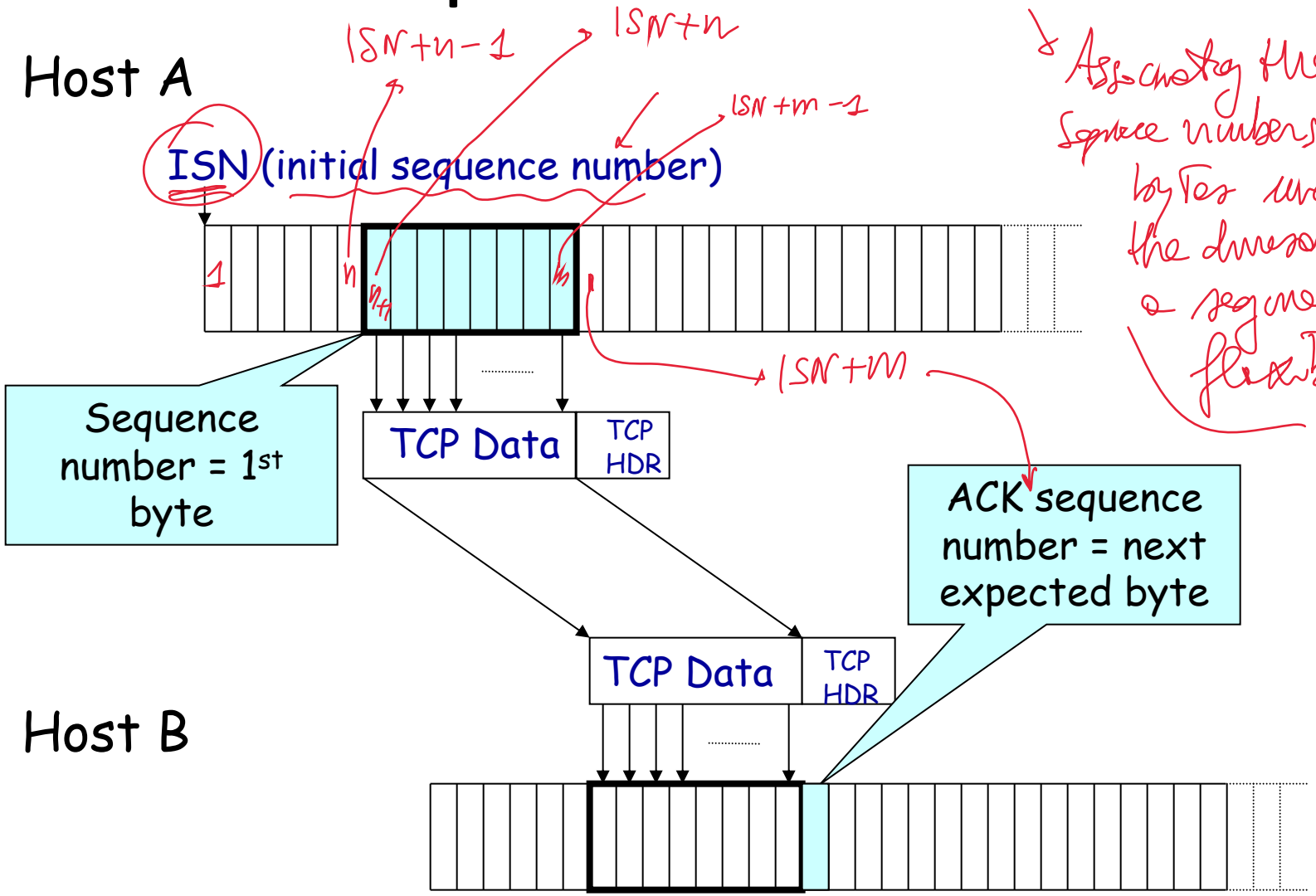


TCP Segment



- IP packet
  - No bigger than Maximum Transmission Unit (MTU)
  - E.g., up to 1500 bytes on an Ethernet
- TCP packet
  - IP packet with a TCP header and data inside
  - TCP header is typically 20 bytes long
- TCP segment
  - No more than Maximum Segment Size (MSS) bytes
  - E.g., up to 1460 consecutive bytes from the stream

Sequence Numbers



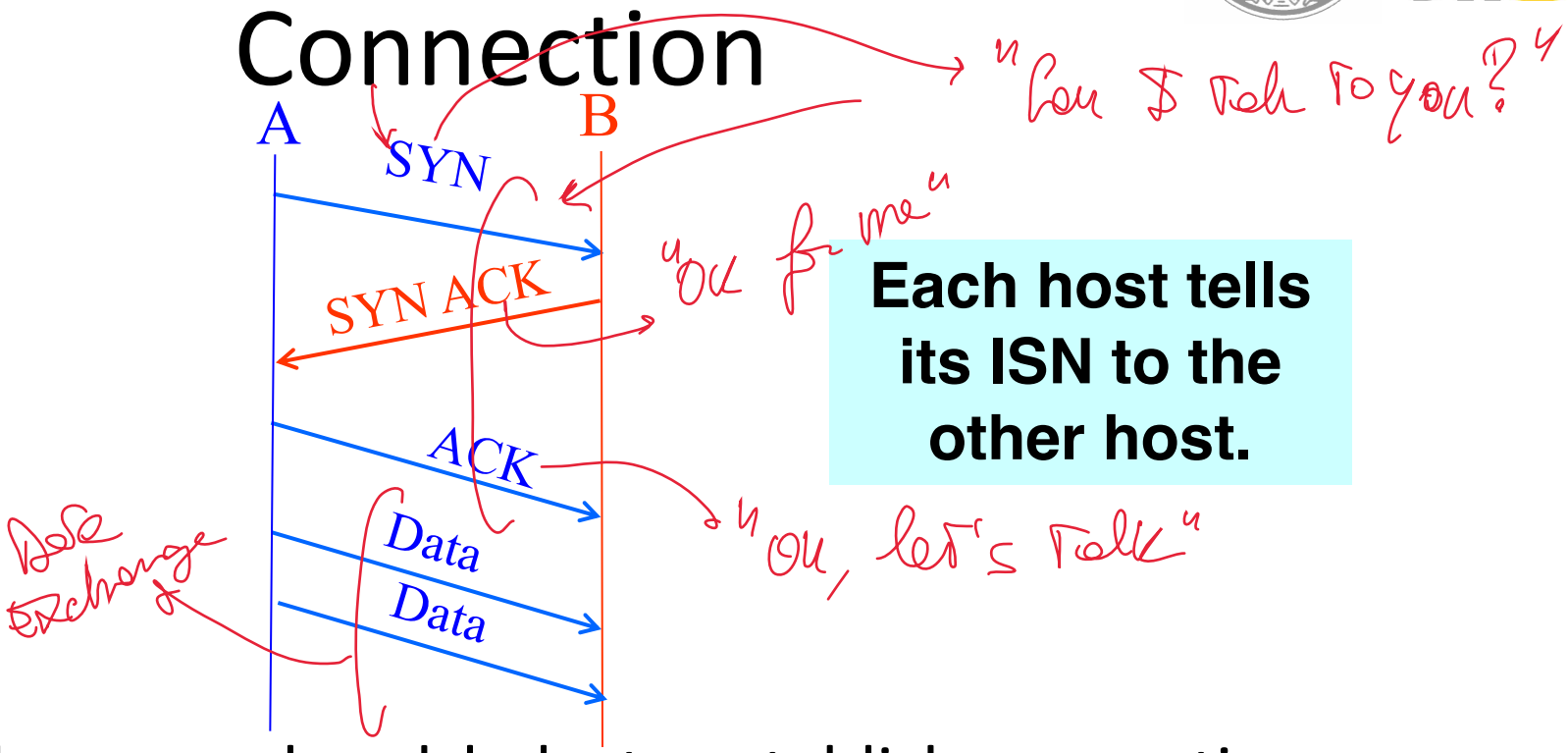
Initial Sequence Number (ISN)

- Sequence number for the very first byte
  - E.g., Why not a de facto ISN of 0?
- Practical issue
  - IP addresses and port #s uniquely identify a connection
  - Eventually, though, these port #s do get used again
  - … and there is a chance an old packet is still in flight
  - … and might be associated with the new connection
- So, TCP requires changing the ISN over time
  - Set from a 32-bit clock that ticks every 4 microseconds
  - … which only wraps around once every 4.55 hours!
- But, this means the hosts need to exchange ISNs

TCP Three-Way Handshake

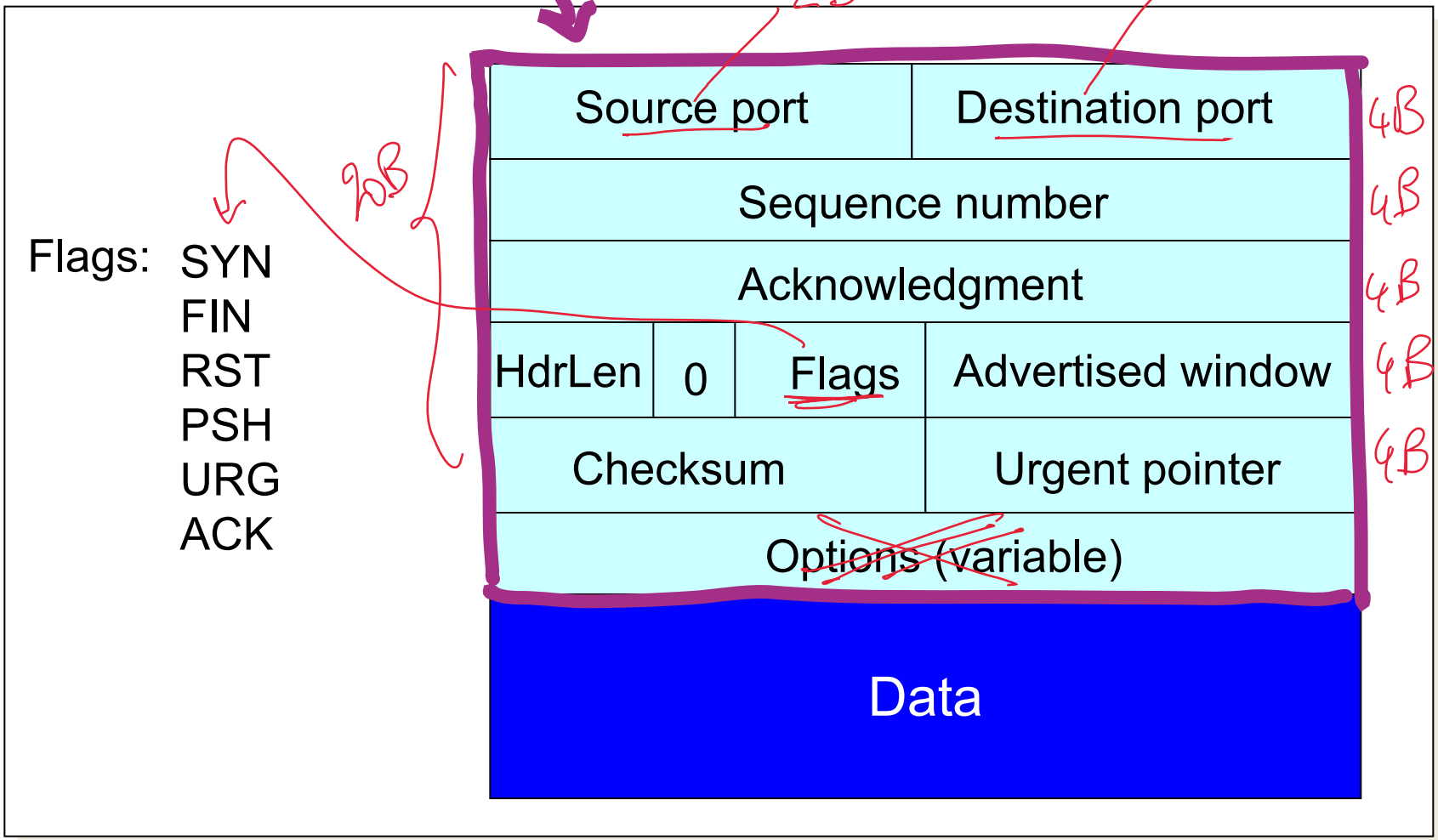
(Connection set-up)

Establishing a TCP Connection



- Three-way handshake to establish connection
  - Host A sends a **SYN** (open) to the host B
  - Host B returns a SYN acknowledgment (**SYN ACK**)
  - Host A sends an **ACK** to acknowledge the SYN ACK

TCP Header (20 bytes)



Step 1: A's Initial SYN Packet (A → B)

Flags: SYN
FIN
RST
PSH
URG
ACK

A's port			B's port
A's Initial Sequence Number			
Acknowledgment			
20 B	0	Flags	Advertised window
Checksum			Urgent pointer
Options (variable)			

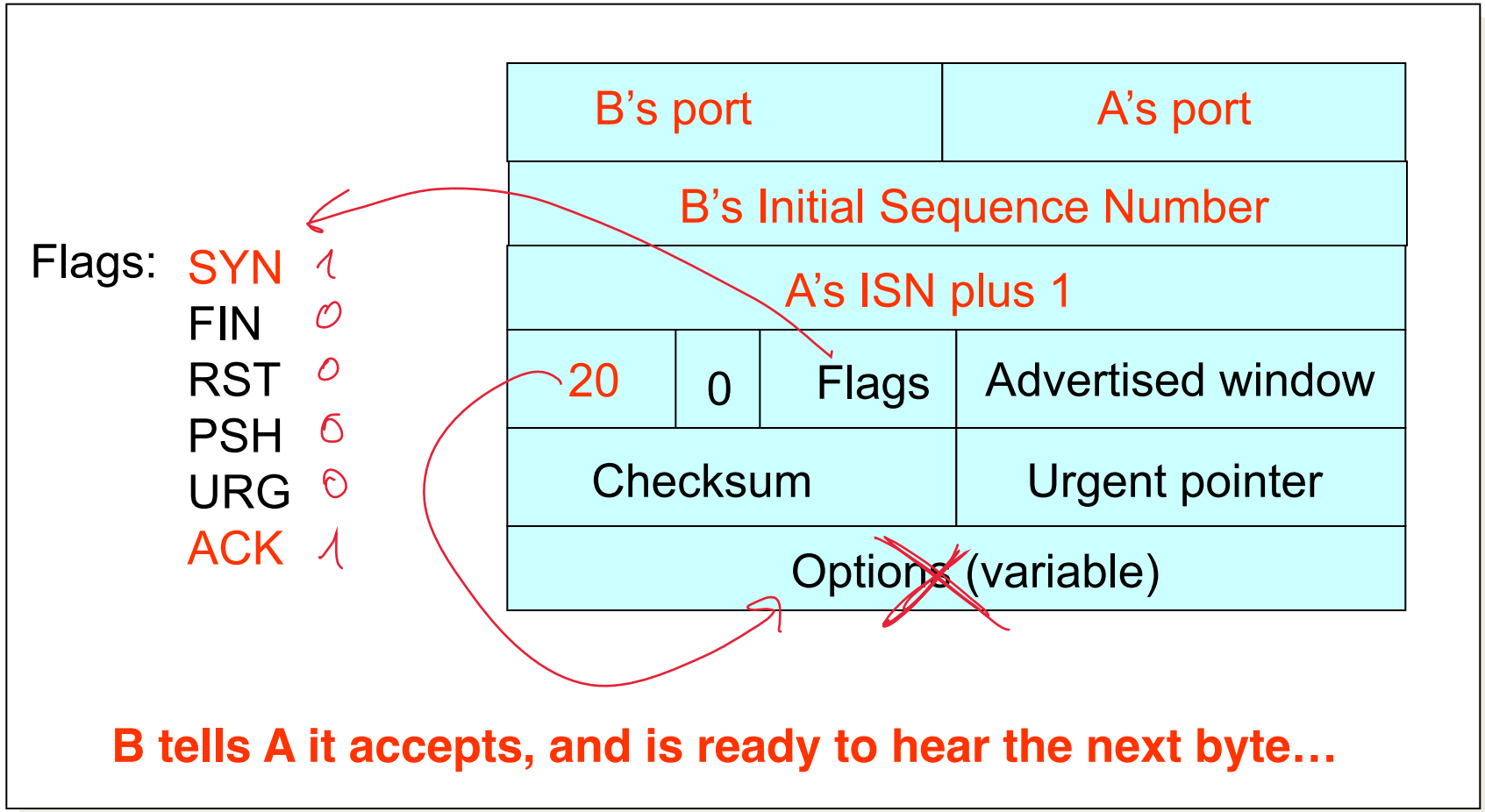
Header length

Flow window

A tells B it wants to open a connection…

(B → A)

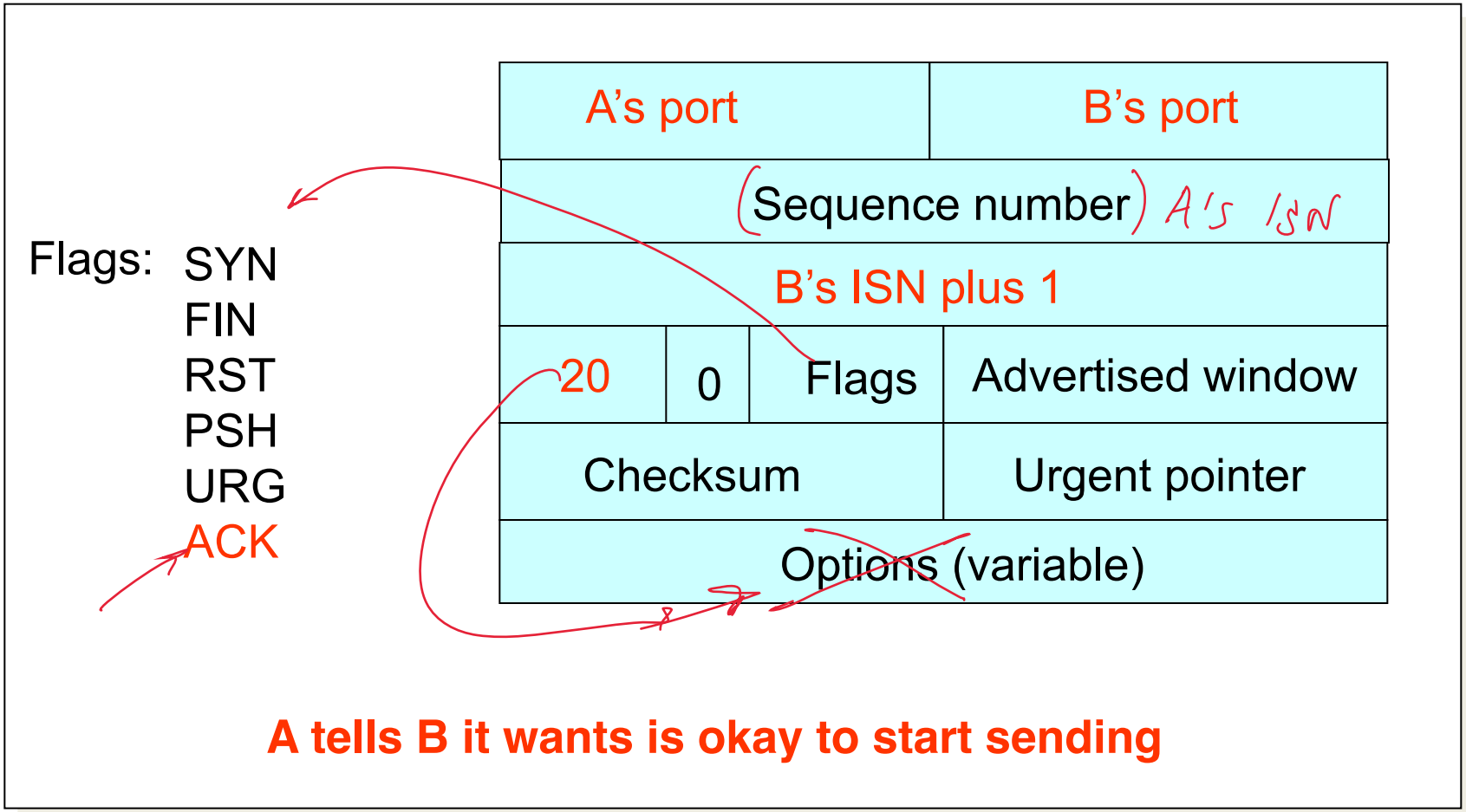
Step 2: B's SYN-ACK Packet



B tells A it accepts, and is ready to hear the next byte…

… upon receiving this packet, A can start sending data

Step 3: A's ACK of the SYN-ACK (A → B)



A tells B it wants is okay to start sending

… upon receiving this packet, B can start sending data

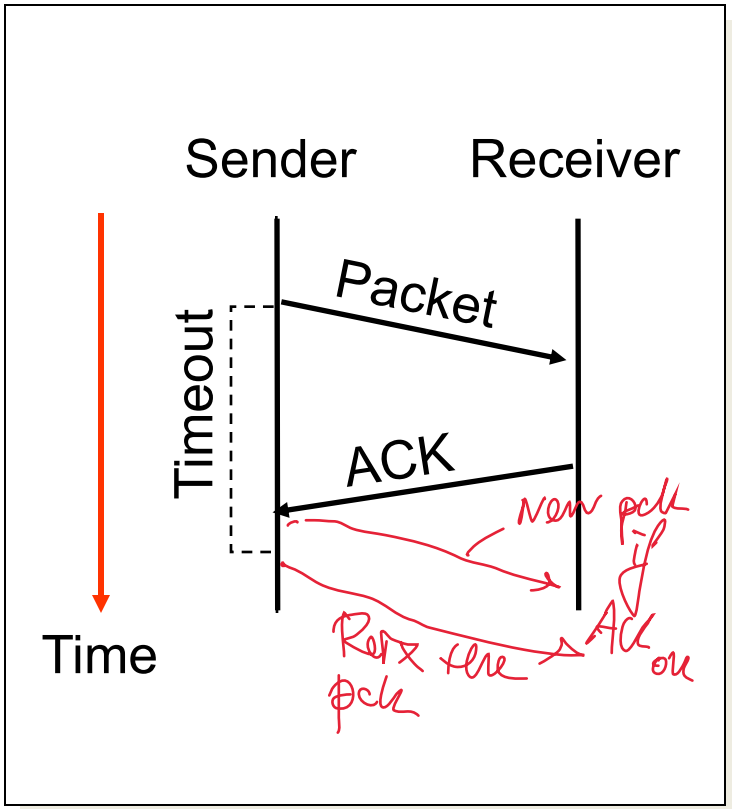
What if the SYN Packet Gets Lost?

- Suppose the SYN packet gets lost
  - Packet is lost inside the network, or
  - Server rejects the packet (e.g., listen queue is full)
- Eventually, no SYN-ACK arrives
  - Sender sets a timer and wait for the SYN-ACK
  - … and retransmits the SYN-ACK if needed
- How should the TCP sender set the timer?
  - Sender has no idea how far away the receiver is
  - Hard to guess a reasonable length of time to wait
  - Some TCPs use a default of 3 or 6 seconds

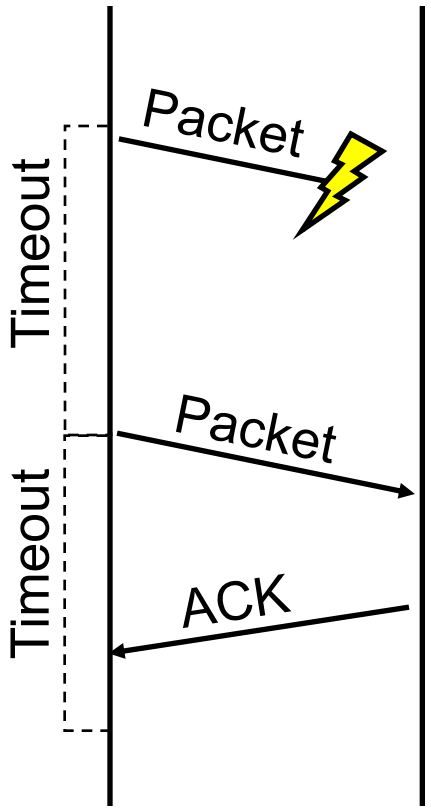
TCP Retransmissions

Automatic Repeat reQuest (ARQ)

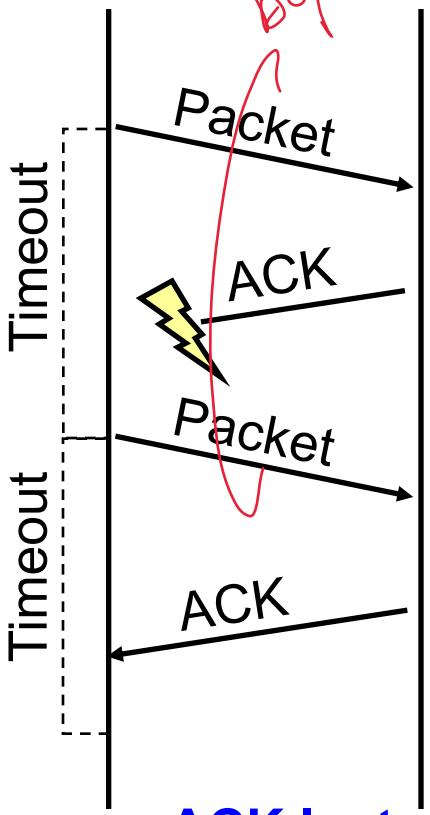
- Automatic Repeat Request
  - Receiver sends acknowledgment (ACK) when it receives packet
  - Sender waits for ACK and timeouts if it does not arrive within some time period
- Simplest ARQ protocol
  - Stop and wait
  - Send a packet, stop and wait until ACK arrives



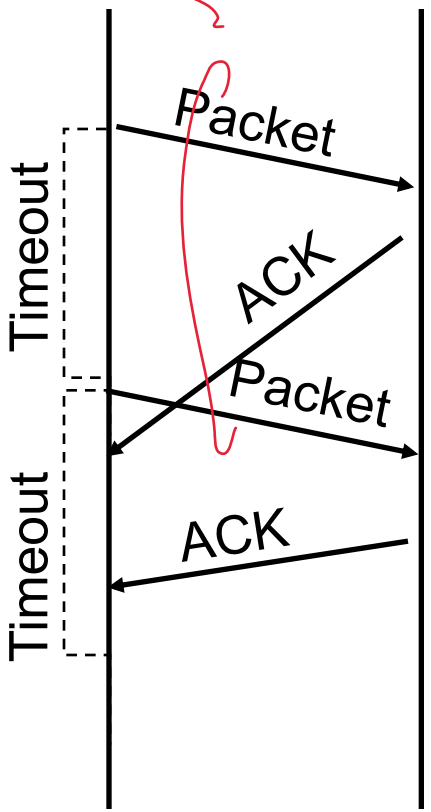
Reasons for Retransmission



Packet lost

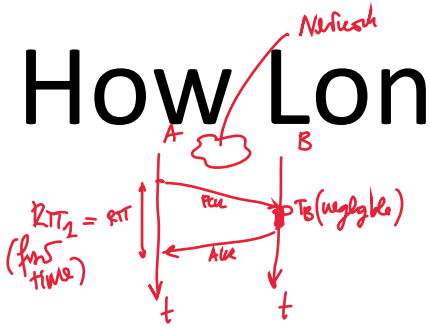


ACK lost
DUPLICATE PACKET



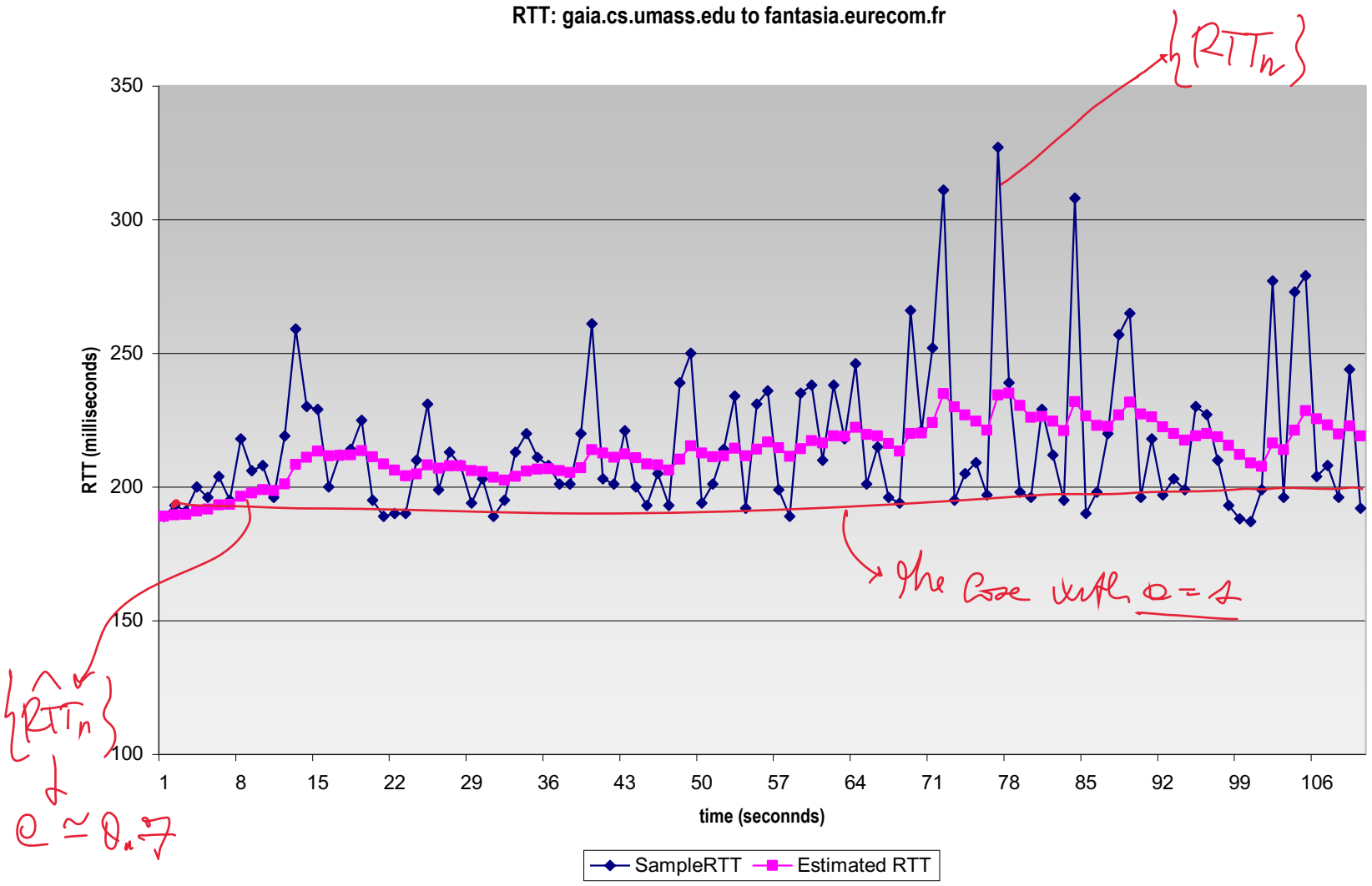
Early timeout
DUPLICATE PACKETS

How Long Should Sender Wait?



- Sender sets a timeout to wait for an ACK
  - Too short: wasted retransmissions
  - Too long: excessive delays when packet lost
- TCP sets timeout as a function of the RTT (Round Trip Time)
  - Expect ACK to arrive after an RTT
  - … plus a fudge factor to account for queuing
- But, how does the sender know the RTT?
  - Can estimate the RTT by watching the ACKs
  - Smooth estimate: keep a running average of the RTT
    - EstimatedRTT = a * EstimatedRTT + (1 –a) * SampleRTT
  - Compute timeout: TimeOut = 2 * EstimatedRTT

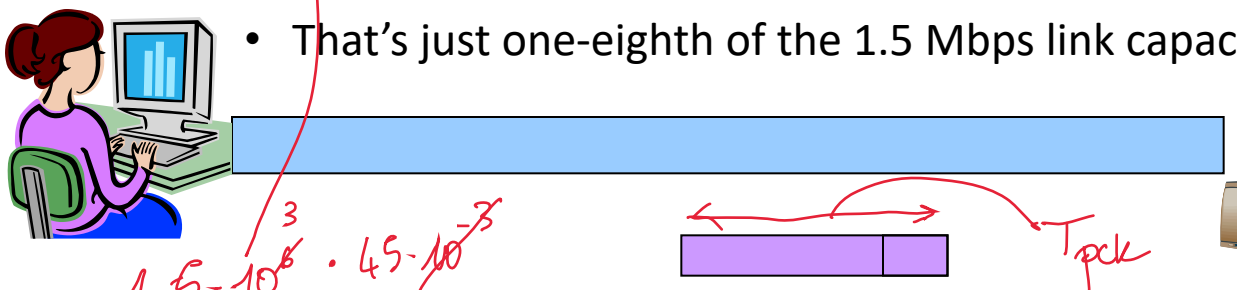
Example RTT Estimation



TCP Sliding Window

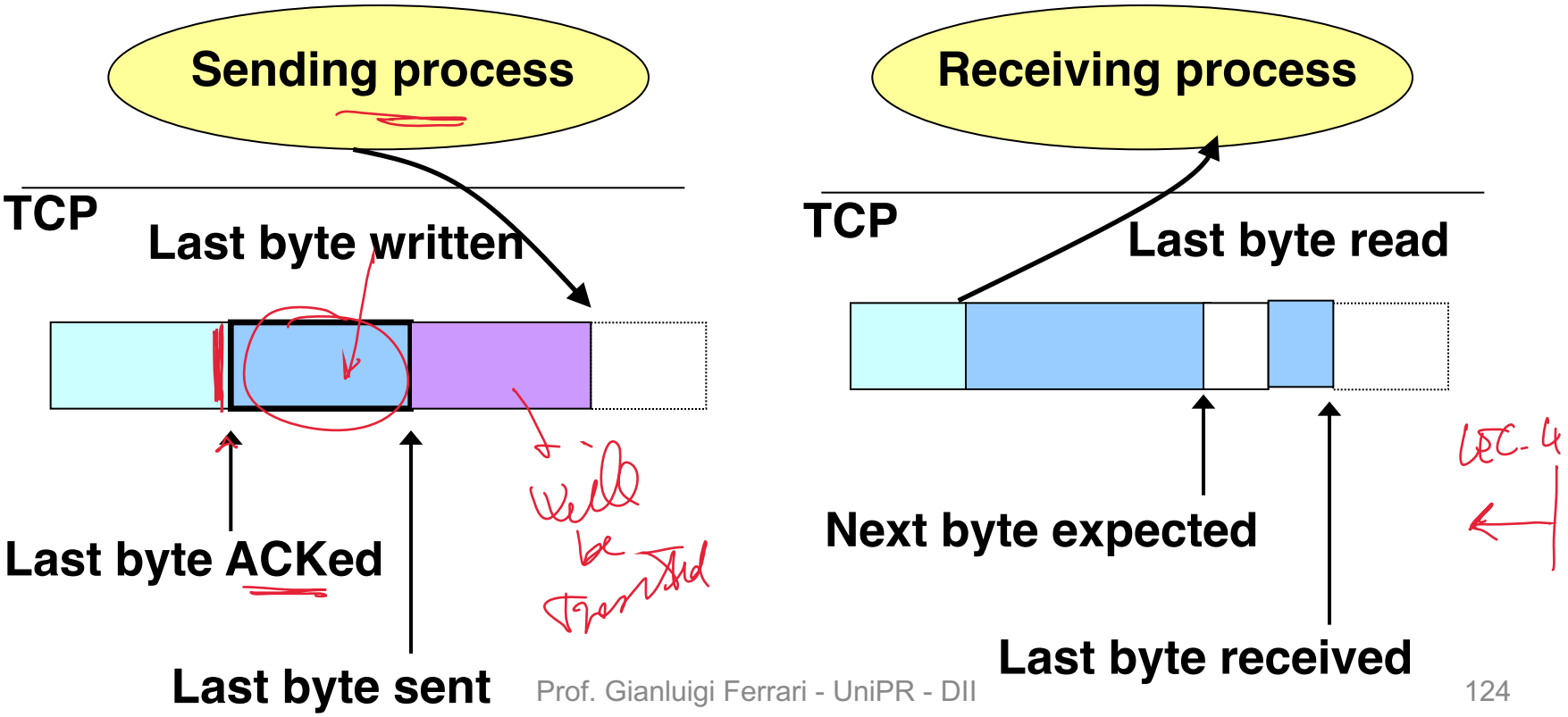
Motivation for Sliding Window

- Stop-and-wait is inefficient
  - Only one TCP segment is "in flight" at a time
  - Especially bad when delay-bandwidth product is high
- Numerical example
  - 1.5 Mbps link with a 45 msec round-trip time (RTT)
    - Delay-bandwidth product is 67.5 Kbits (or 8 KBytes)
  - But, sender can send at most one packet per RTT
    - Assuming a segment size of 1 KB (8 Kbits)
    - … leads to 8 Kbits/segment / 45 msec/segment ➔ 182 Kbps
    - That's just one-eighth of the 1.5 Mbps link capacity



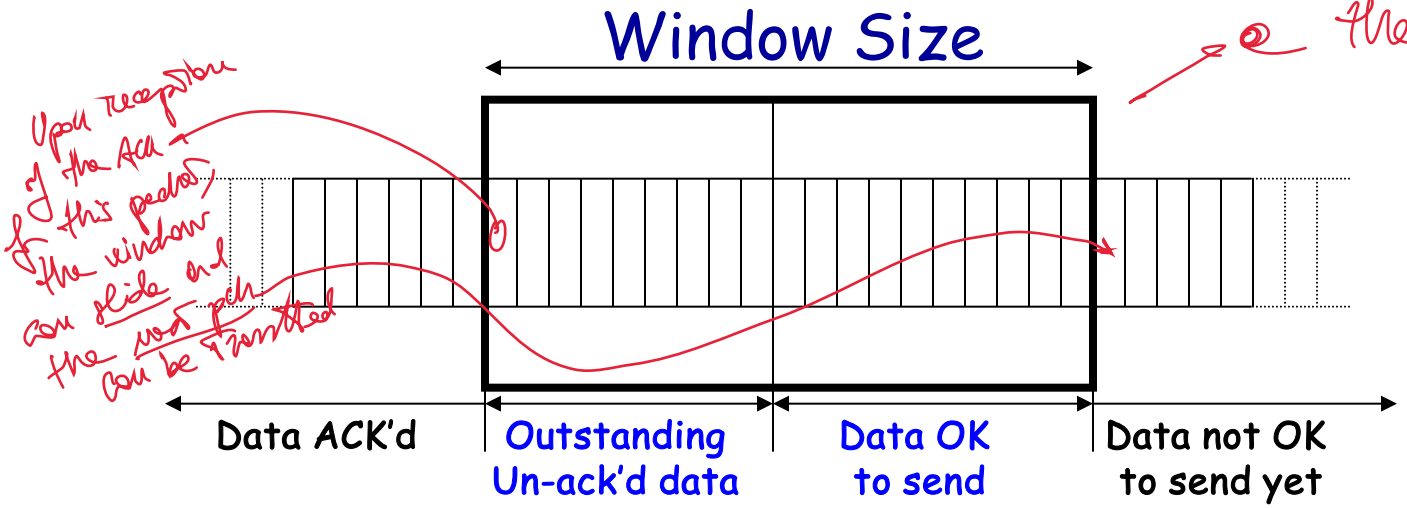
Sliding Window

- Allow a larger amount of data "in flight"
  - Allow sender to get ahead of the receiver
  - … though not *too far* ahead

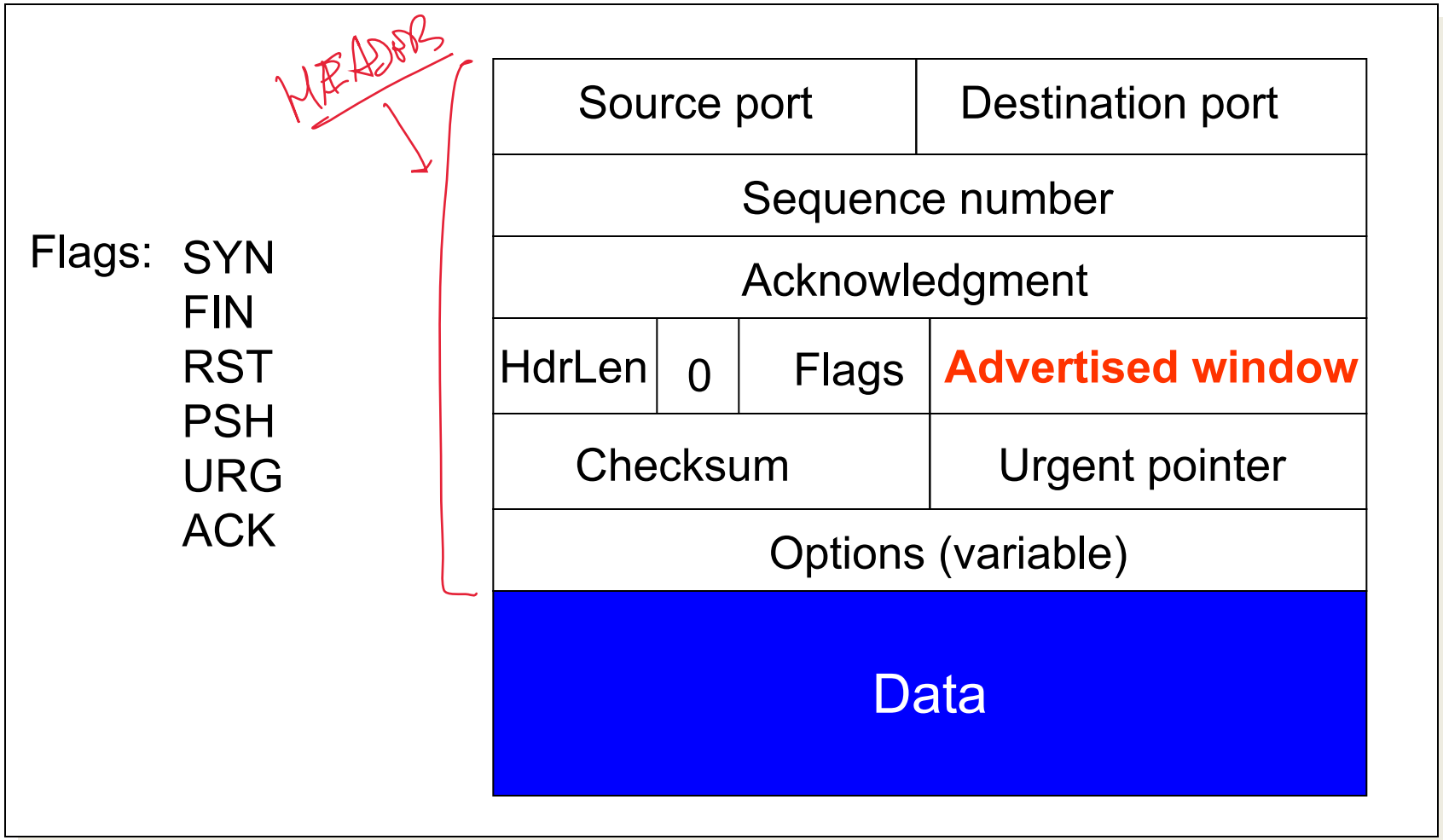


Receiver Buffering

- Window size
  - Amount that can be sent without acknowledgment
  - Receiver needs to be able to store this amount of data
- Receiver advertises the window to the receiver
  - Tells the receiver the amount of free space left
  - … and the sender agrees not to exceed this amount



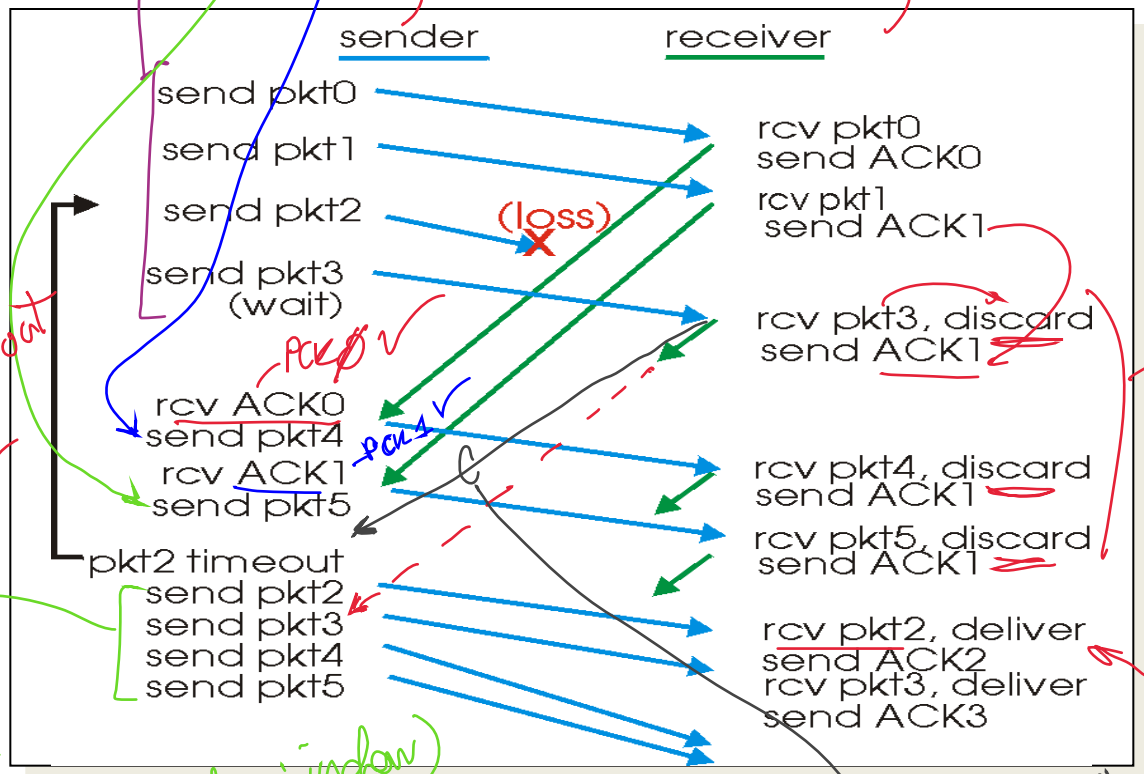
TCP Header for Receiver Buffering



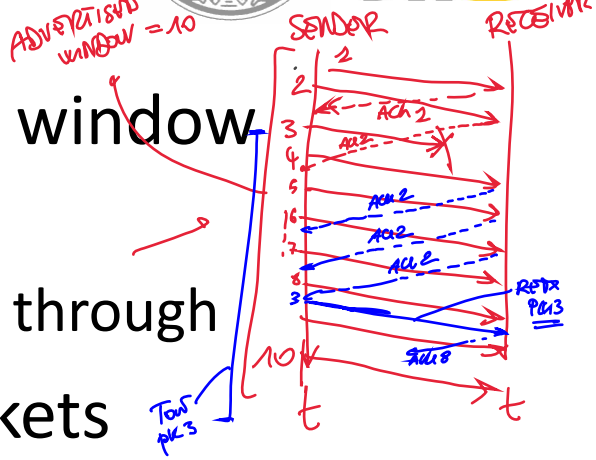
Fast Retransmission

Timeout is Inefficient

- Timeout-based retransmission
  - Sender transmits a packet and waits until timer expires
  - … and then retransmits from the lost packet onward

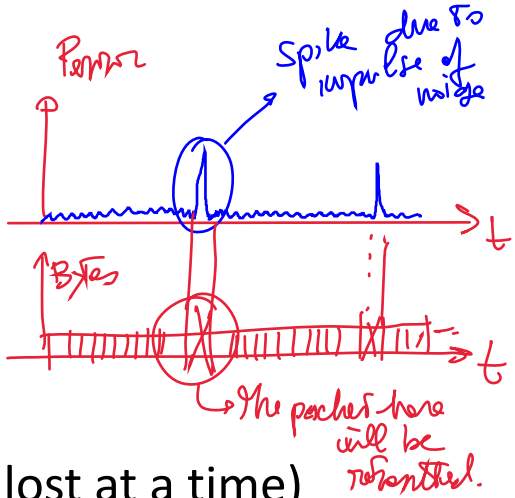


Fast Retransmission



- Better solution possible under sliding window
  - Although packet n might have been lost
  - … packets n+1, n+2, and so on might get through
- Idea: have the receiver send ACK packets
  - ACK says that receiver is still awaiting n^{th} packet
    - And *repeated* ACKs suggest later packets have arrived
  - Sender can view the "duplicate ACKs" as an early hint
    - … that the n^{th} packet must have been lost
    - … and perform the retransmission early
- Fast retransmission
  - Sender retransmits data after the triple duplicate ACK

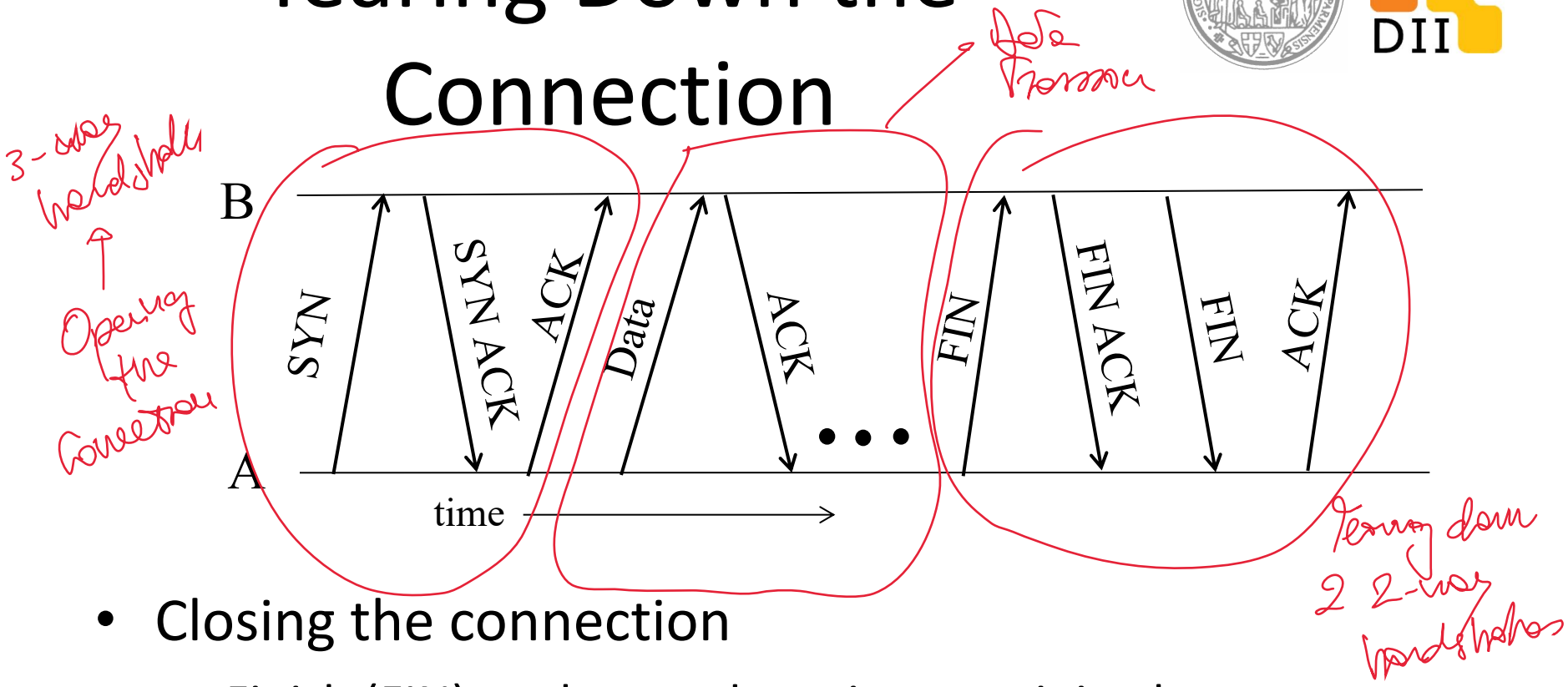
Effectiveness of Fast Retransmit



- When does Fast Retransmit work best?
  - Long data transfers
    - High likelihood of many packets in flight
  - Large window size
    - Support for many packets in flight
  - Low burstiness in packet losses (only one pck lost at a time)
    - Higher likelihood that later packets arrive successfully
- Implications for Web traffic
  - Most Web transfers are short (e.g., 10 packets)
    - Short HTML files or small images
  - So, often there aren't many packets in flight
  - … making fast retransmit less likely to "kick in"
  - Forcing users to like "reload" more often… ☺

Tearing Down the Connection

Tearing Down the Connection



- Closing the connection
  - Finish (FIN) to close and receive remaining bytes
  - And other host sends a FIN ACK to acknowledge
  - Reset (RST) to close and not receive remaining bytes

Sending/Receiving the FIN Packet

- Sending a FIN: close()
  - Process is done sending data via the socket
  - Process invokes “close()” to close the socket
  - Once TCP has sent all of the outstanding bytes…
  - … then TCP sends a FIN
- Receiving a FIN: EOF
  - Process is reading data from the socket
  - Eventually, the attempt to read returns an EOF

TCP State Diagram

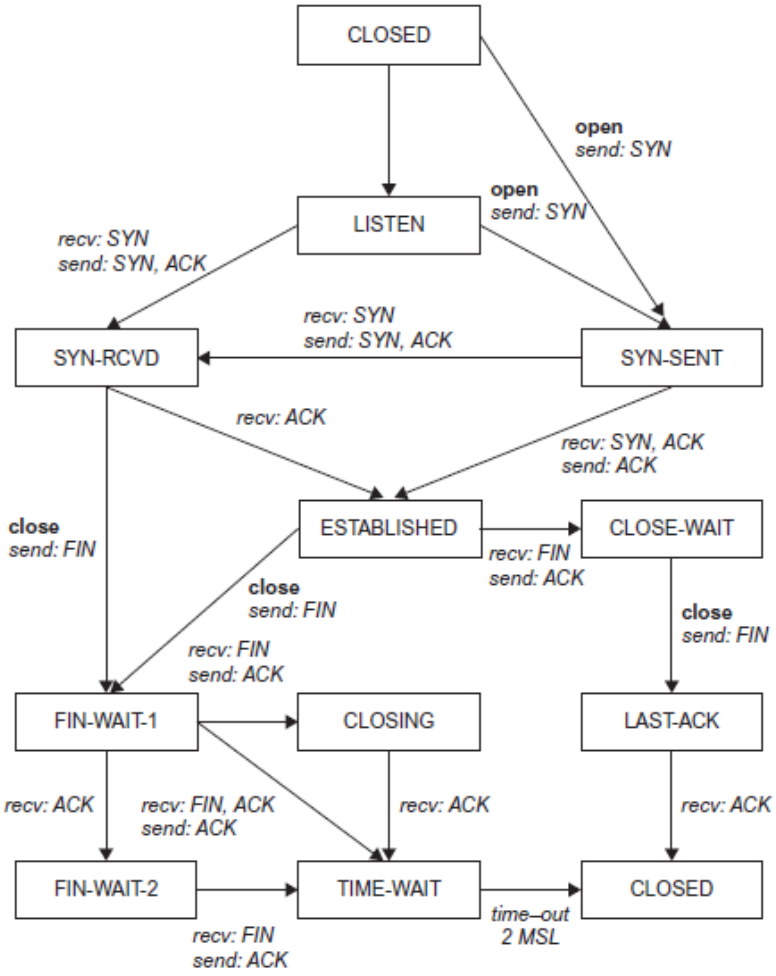


FIGURE 6.6

TCP state diagram.

UDP for Smart Objects

- Advantages
  - Very low overhead (header size and protocol logic) → low energy consumption; more room in each packet for application layer data; memory footprint
  - Simplicity of UDP fits well with several applications (e.g., temperature monitoring: losing a reading is not crucial) → in general, low reliability demands
  - Well suited to applications which require their own routing (application overlay mechanism)
  - UDP maps well to underlying multicast mechanisms
- Drawbacks
  - Loss of packets and no recovery mechanism → application has to recover (increase complexity)
  - Smart object networks have small packet sizes and UDP does not provide mechanisms to split their data → application layer decides proper packet size

TCP for Smart Objects

- Advantages
  - Smart object networks are typically lossy → reliable transport
  - TCP has problems in high-throughput applications over wireless links (not designed for wireless links), but high throughput is typically not relevant
  - Smart object networks interact with existing systems using TCP
  - TCP MSS (Maximum Segment Size) option is very useful (e.g., for 802.15.4 packets)
  - Core of TCP is simple (many of the complex mechanisms, e.g., sliding window and congestion control, aim at achieving high throughput)
  - Even though TCP header is larger than UDP header, it can be compressed
- Drawbacks
  - Reduced TCP implementations for smart objects do not achieve high throughput (as full-blown TCP stack): (i) no sliding window mechanism (only one TCP packet active at a time); (ii) delayed ACKs reduce throughput

UNIVERSITÀ DI PARMA

IoT Lab

IPv6

Main sources

- Slides by Prof. Luca Veltri (see following slides)
- J. P. Vasseur and A. Dunkels, `Interconnecting Smart Objects with IP,` Morgan Kaufmann, 2010. Chapters 6 and 15.
- Ansar Pasha and Patrick Grossetete, “IPv6 @ Cisco”

UNIVERSITA' DEGLI STUDI DI PARMA
Dipartimento di Ingegneria dell'Informazione

IPv6

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Internet of Things, 2015/2016

IPv4

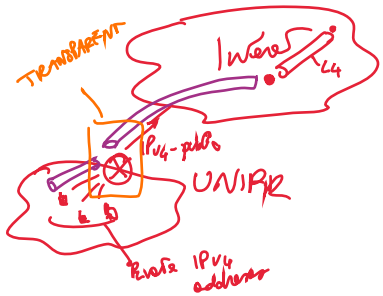
- RFC 791 del 1981
- More or less the same in 30 years
- Main issues:
  - **Limited addressing space**
  - **Growing of Routing Tables**
  - **Host configuration**
  - **Security**
  - **Quality of service**
  - **Mobility**

IPv4 issues: Addressing

- IPv4 addresses: 32-bit
  - **4,294,967,296 IPv4 "theoretical" addresses, but the exact number is far below**
- Depletion of IPv4 address
  - **early predictions (made at the time IPv6 was in its early phase of design, end of 90's): IPv4 address depletion as early as 2002**
- Mainly due to:
  - **growing countries (e.g. China)**
  - **Mobile phones**
  - **Always-on devices**
  - **IoT**

IPv4 issues: Addressing

- Several mitigation strategies were developed:
  - **better address allocation and subnetting**
  - **Classless inter-domain routing (CIDR)**
    - it also reduces the routing table sizes
  - **Network Address Translation (NAT)**
    - same public address shared by several hosts with private addresses
    - Some problems
      - introduce states in the network between end points
      - Some application protocols require specific application level gatways (ALG)
        - » e.g. FTP, SIP, DNS, ICMP
        - » need to maintain various protocol states
    - Incompatible with some security mechanism
      - e.g. IPSec



Internet Protocol v6

- IPv6 (RFC 2460) = the next generation Internet Protocol
  - **Complete redesign of IP addressing**
  - **Hierarchical 128-bit address with decoupled host identifier**
  - **Stateless auto-configuration**
  - **Simple routing and address management**
- Majority of traffic not yet IPv6 but..
  - **Most PC operating systems already have IPv6**
  - **Most routers already have IPv6 support**
  - **IoT is coming**

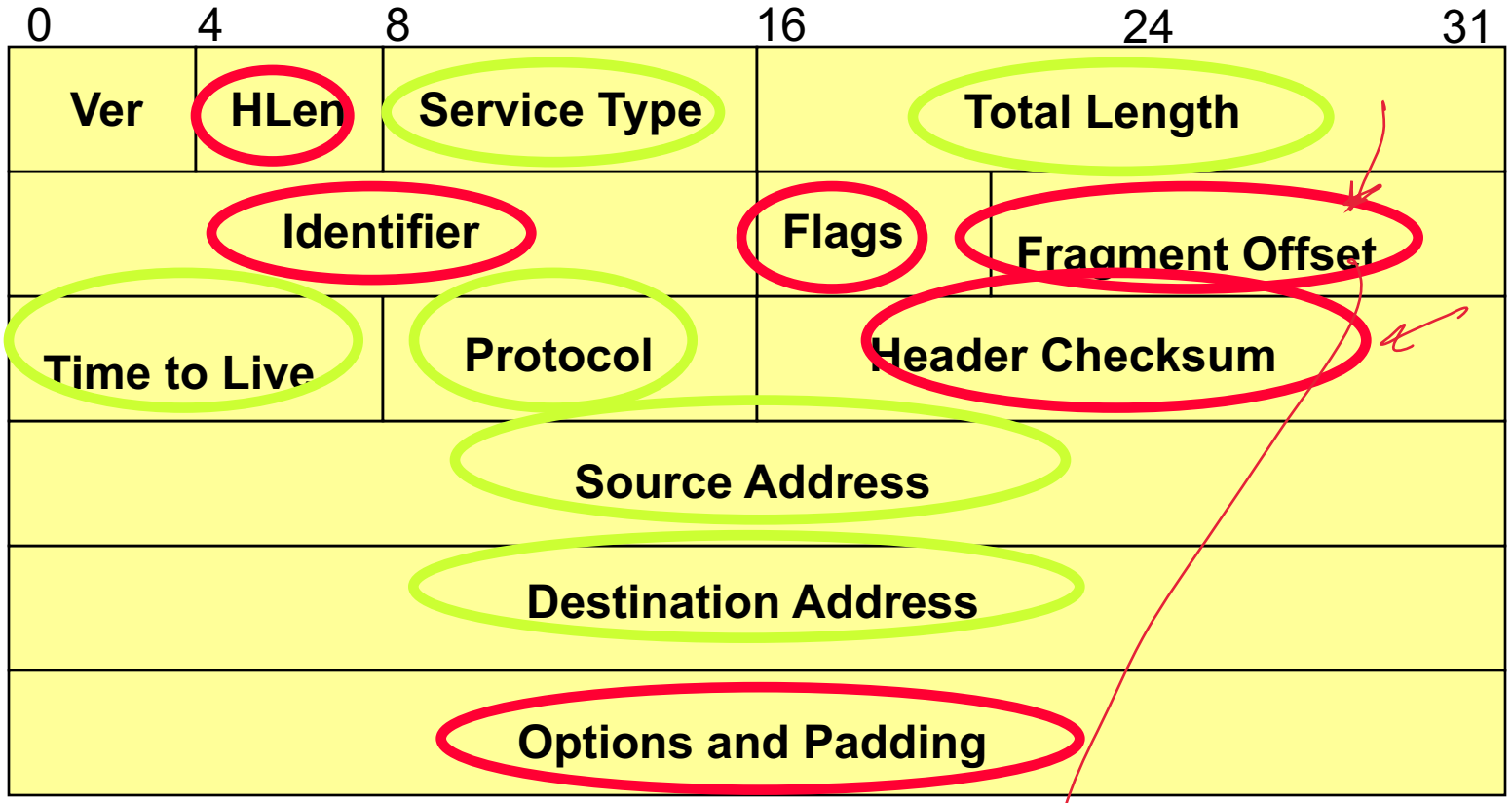
→ LEC. 6

IPv6 main characteristics

- Larger address space
  - **quadrupled to 16 bytes=128 bits**
  - **required for large-scale networks (urban networks, smart grids, industrial automation networks,..)**
- Larger number of levels of addressing hierarchy
  - **routing table efficiency**
- Autoconfiguration
  - **each node should be able to generate its own unique IPv6 address**
- Mobility support
  - **Mobile IPv6**
- Security
  - **IPSec/AH/ESP/IKE**

starting from its own unique MAC address (depending on the used interface)

IPv4 vs IP6 Header format



- 20 octects + options
- 13 mandatory fields
  - **one of them is also formed by 3 bits of flags**

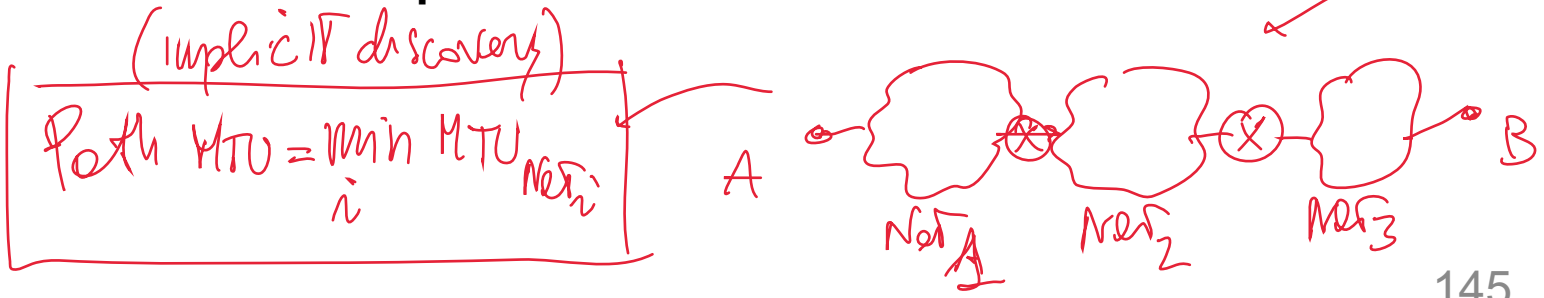
In order to limit the computational load of intermediate nodes, fragmentation is allowed only at the source node.

Modified

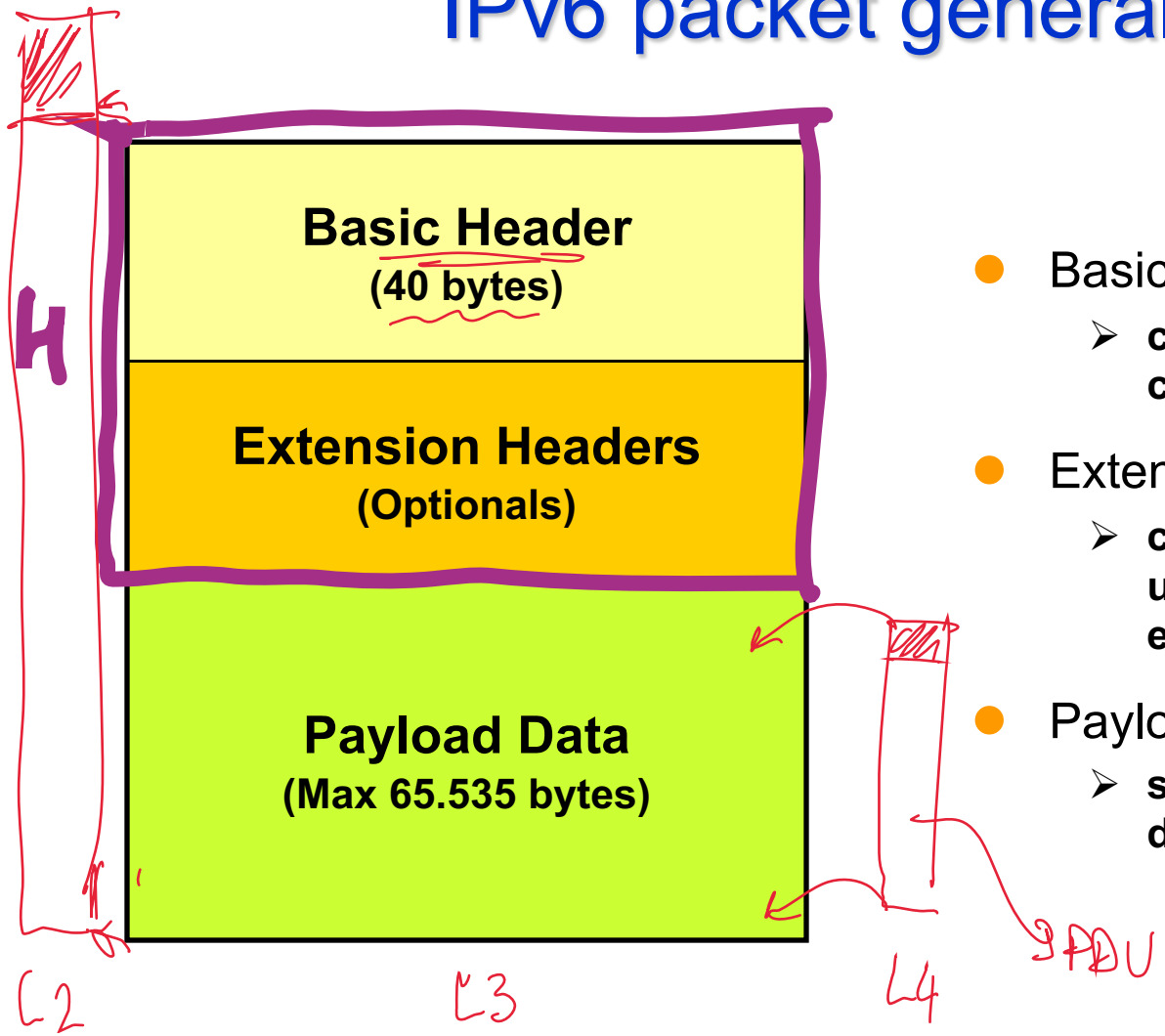
Removed

IPv4 vs IP IP6 Header format

- No checksumming at the IP network layer → transport layer needs to take care
- No hop-by-hop segmentation: path MTU (Maximum Transmission Unit) discovery
- Header change
  - **Unused IPv4 header fields have been removed**
  - **Simpler structure with potentially daisy-chained headers**
  - **IPv6 header is twice as long (40 bytes) as IPv4 header without options (20 bytes)**
  - **64 bits aligned (various fields can be grouped in groups of 64 bits) → 64-bit processors!**

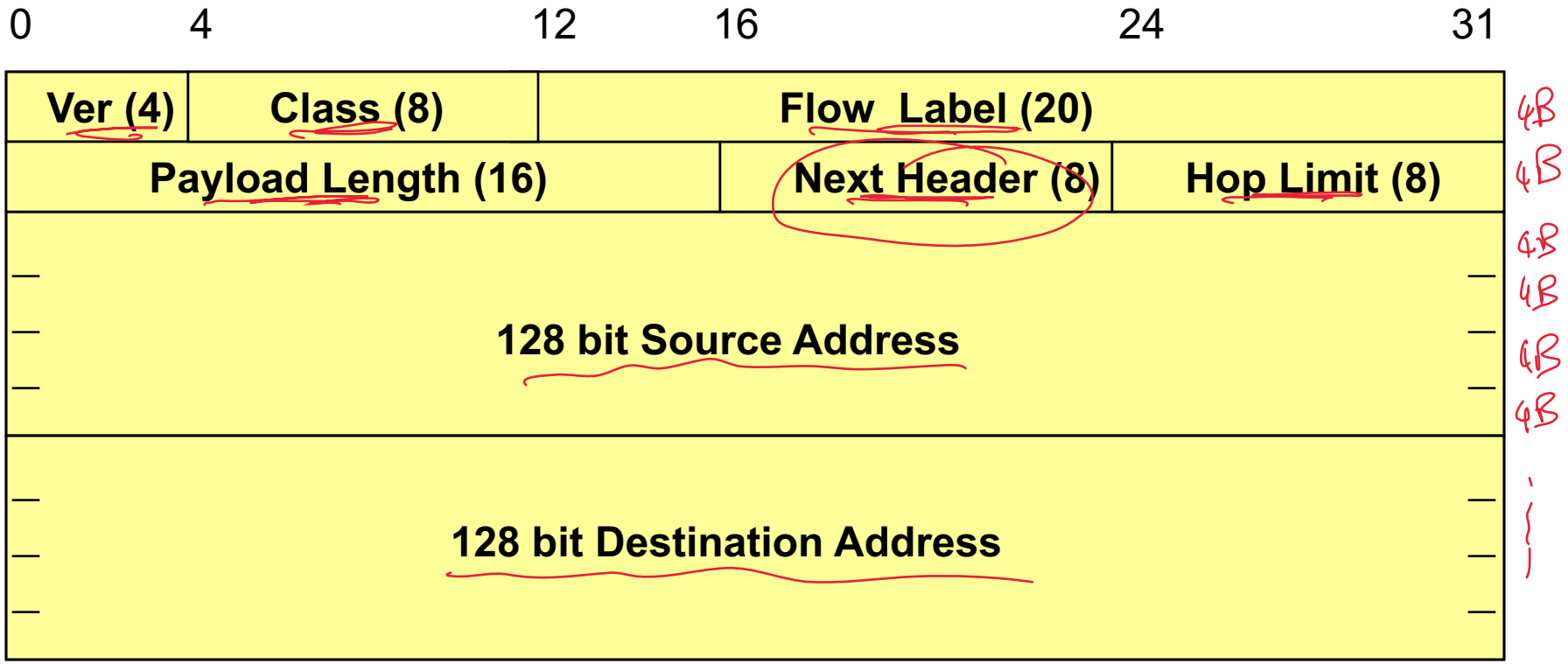


IPv6 packet general format



- Basic header
  - **contiene le informazioni comuni a tutti i datagrammi**
- Extension Headers
  - **contengono le opzioni utilizzate dai router intermedi e/o dall'host di destinazione**
- Payload Data
  - **sono i bit informativi elaborati dall'host di destinazione**

IPv6 Basic Header



40 octects
8 header fiels

IPv6 Basic Header

- Version (4 bits): IP versions number 6
- Traffic class (8 bits): used to indicate the Class of Service (CoS) of the packet
- Flow label (20 bits): a label may be used by a source node to refer to a sequence of packets identifying a flow that requires specific handling of the packet by routers along the path to its destination
- Payload length (16 bits): indicates the length of the payload (excluding the basic header)
- Next header (8 bits): identifies the header that follows the IPv6 packet header. It provides a flexible way to add optional extension headers
- Hop limit (8 bits): decremented each time the packet is forwarded by a node. When the hop limit field is equal to 0 (2^8 hops), the packet is discarded
- Source address: 128-bit IPv6 source address of the packet
- Destination address: 128-bit IPv6 destination address of the packet

IPv6 Basic Header

- Payload Length (16 bit)
  - **indicates the length of the payload (excluding the basic header)**
  - **normal maximum length is 65.535 byte**
  - **It is possible to extend it using "jumbo payload" (hop-by-hop options header)**
- Next Header (8 bit)
  - **identifies the header that follows the IPv6 packet header**
  - **Some values:**

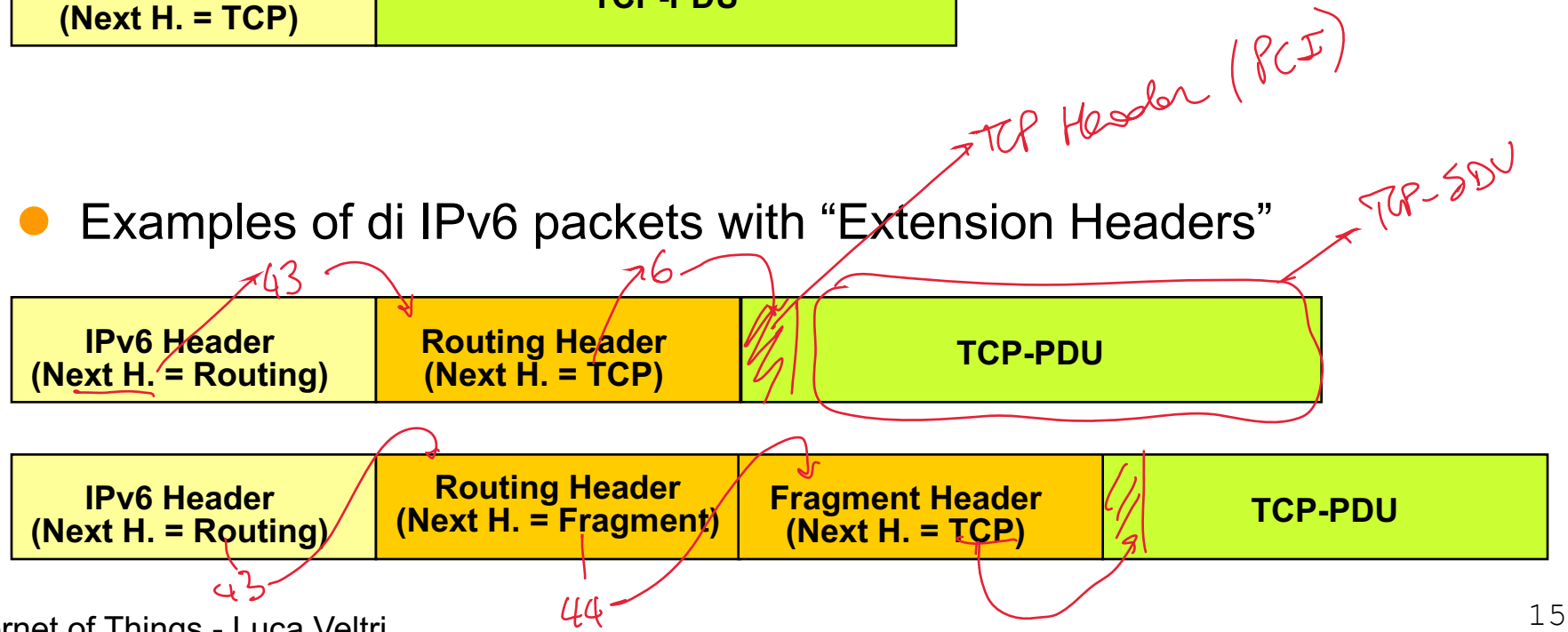
0	Hop-by-hop options header	46	Resource Reservation Protocol
4	Internet Protocol	50	Encapsulating Security Payload
6	Transmission Control Protocol	51	Authentication Header
17	User Datagram Protocol	58	Internet Control Message Protocol
43	Routing Header	59	No Next Header
44	Fragment Header	60	Destination Options Header

IPv6 Extension Headers

- Extension headers are inserted between the IPv6 basic header and the payload (upper layer data, e.g. TCP PDU)

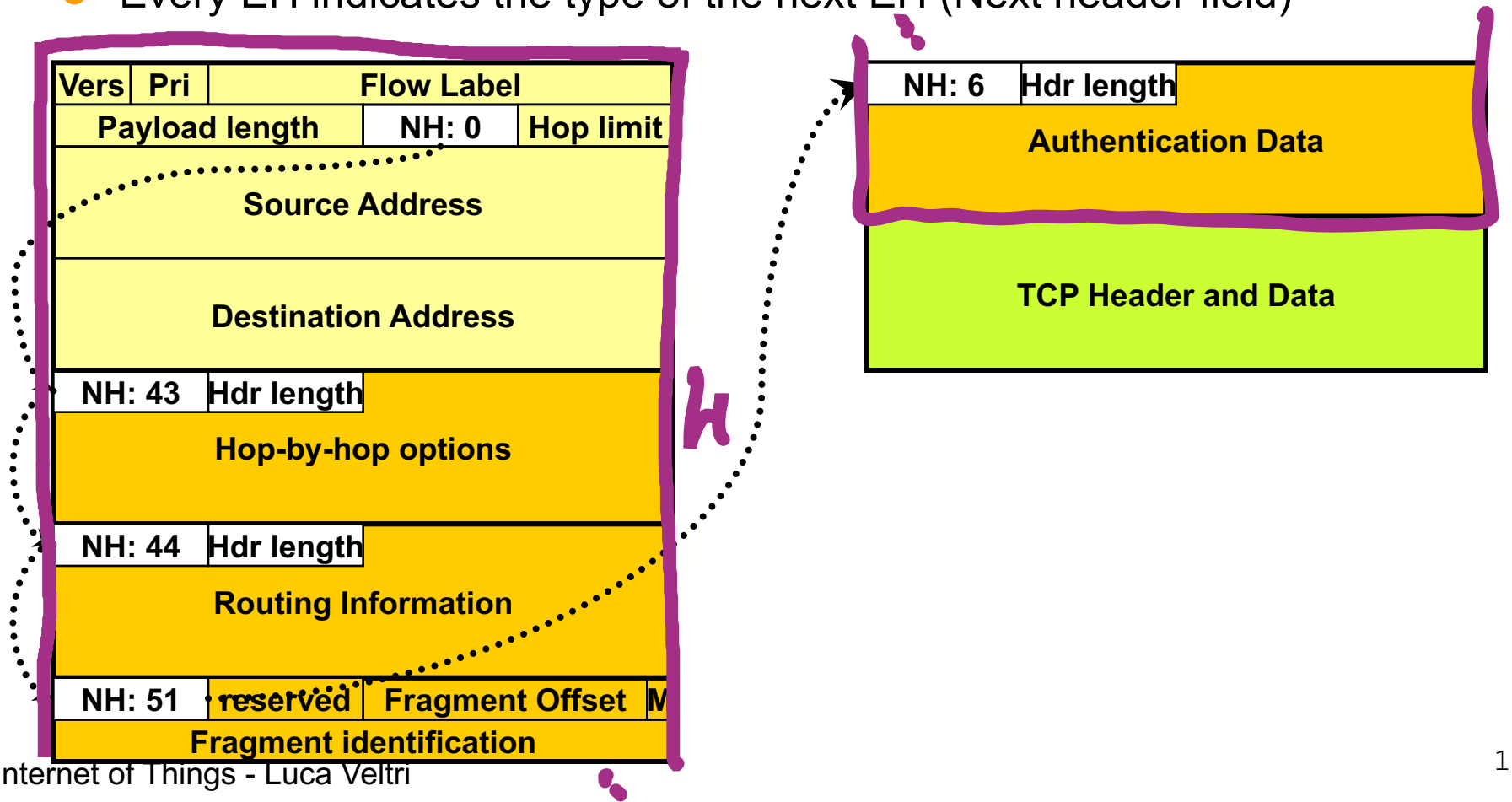
- IPv6 "normal" packet

- Examples of di IPv6 packets with "Extension Headers"



IPv6 Extension Headers (cont.)

- Every EH has a length of 8*(1+n) byte, where *n* is the Header Extension Length value
- Every EH indicates the type of the next EH (Next header field)



IPv6 Extension Headers (cont.)

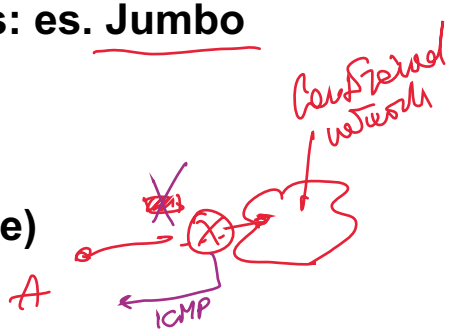
- Basic standard EH (in the right order):
  - **Hop-by-hop Options header**
  - **Destination Options header**
    - **only in prsence of Routing header**
  - **Routing header**
  - **Fragment header (only destination)**
  - **Authentication header (only destination)**
  - **Encapsulation security payload header (only destination)**
  - **Destination Options header (only destination)**
- Every EH may occur only 1 time
  - **The only exception is the Destination Options (at most 2 times)**

EHs relevant for intermediate nodes

EHs relevant only for the destination

IPv6 Extension Headers (cont.)

- Hop-by-hop Options (Type=0)
  - **It includes the options which involve all traversed routers: es. Jumbo Option (for packets with >65.575 bytes)**
- Routing (Type=43)
  - **It specifies a list of routers to be traversed (Strict o Loose)**
- Fragment (Type=44)
  - **It is useful to send fragmented packets (only from the source)**
  - **It a packets exceeds the link MTU (MTU min = 576 in IPv6!) it has to be discared and an ICMP msg is sent to the source, which can**
    - i) reduce the length of the subsequent packets
    - ii) send packets with the EH Fragment
- Destination Options (Type=60)
  - **It includes the options which need to be processed only at the destination node/s**
- Authetication (Type=51)
- Encapsulating Security Payload (ESP) (Type=50)



Hop-by-Hop Options

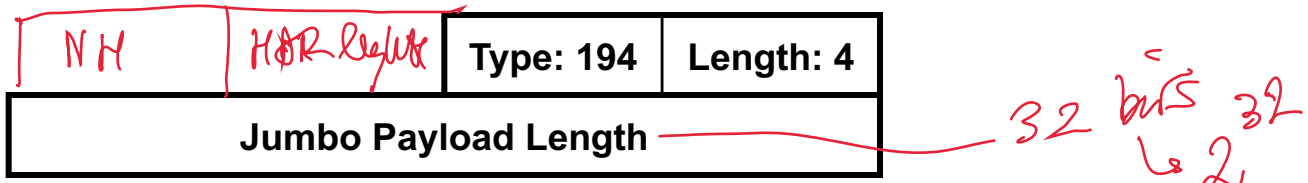
- It contains the options for each intermediate system over the path of the datagram
  - **It is processed also by intermediate nodes**
- It is constitute by a vsariable number of options codified as triplets TLV (Type-Length-Value)
  - **Type (8 bit): indicates the type of option**
  - **Length (8 bit): indicates the length of the field Value**
  - **Value: it transports the option and some indications for the router useful to process the option**
- Defined options:

Type	Option	Size	Allineamento
0	Pad 1	1 byte	nessuno
1	Pad N	2+n bytes	nessuno
194	Jumbo Payload Length	2+4 bytes	4·n + 2

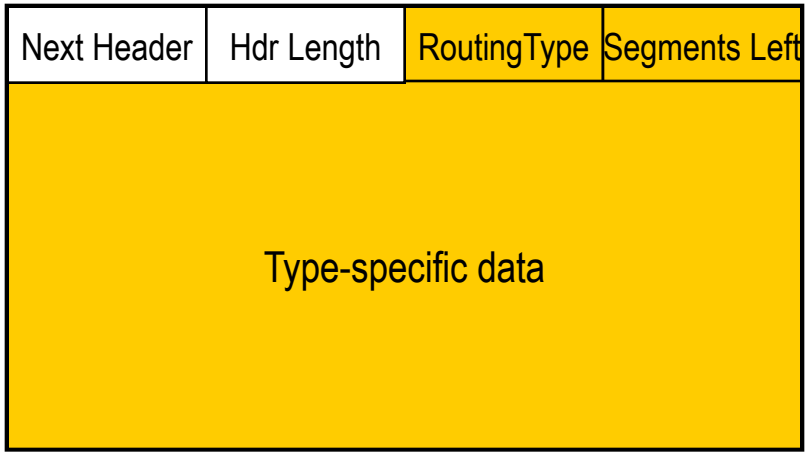
Hop-by-Hop Options Header

- Pad1 e PadN options
  - **Needed to insert filling octets in the header for alignment puroposes**
  - **Allow to increase the router processing speed**
    - Pad1: 1 byte: [type=0]
    - Pad2: 2+n bytes: [type=1 (1 byte), len (1 byte), N bytes]

- Jumbo Payload Length Option
  - **Allows to increase the maximum length of the datagram with respect to the value allowed by the basic header**
  - **If this option is used, the field payload length of the basic header has to be set to the value 0**

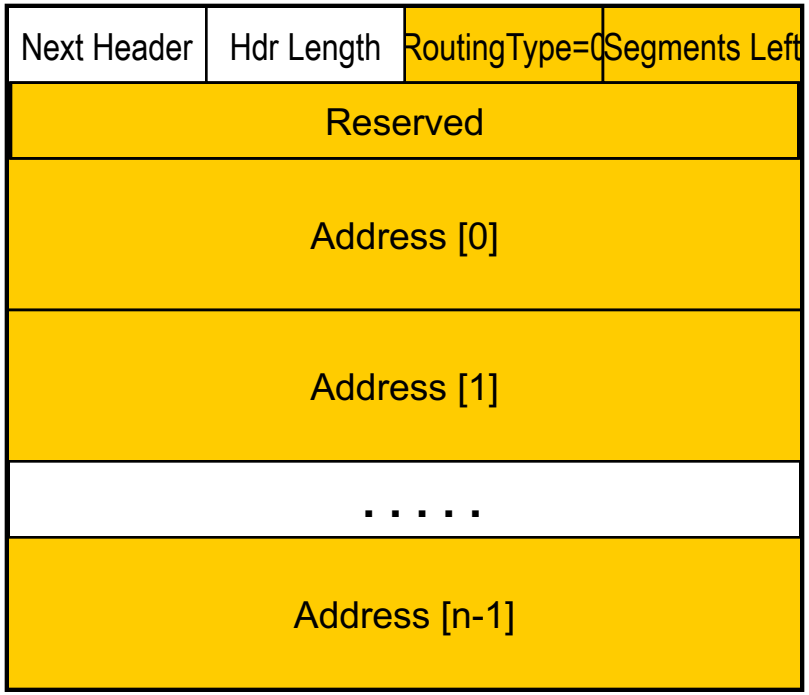


Routing Header



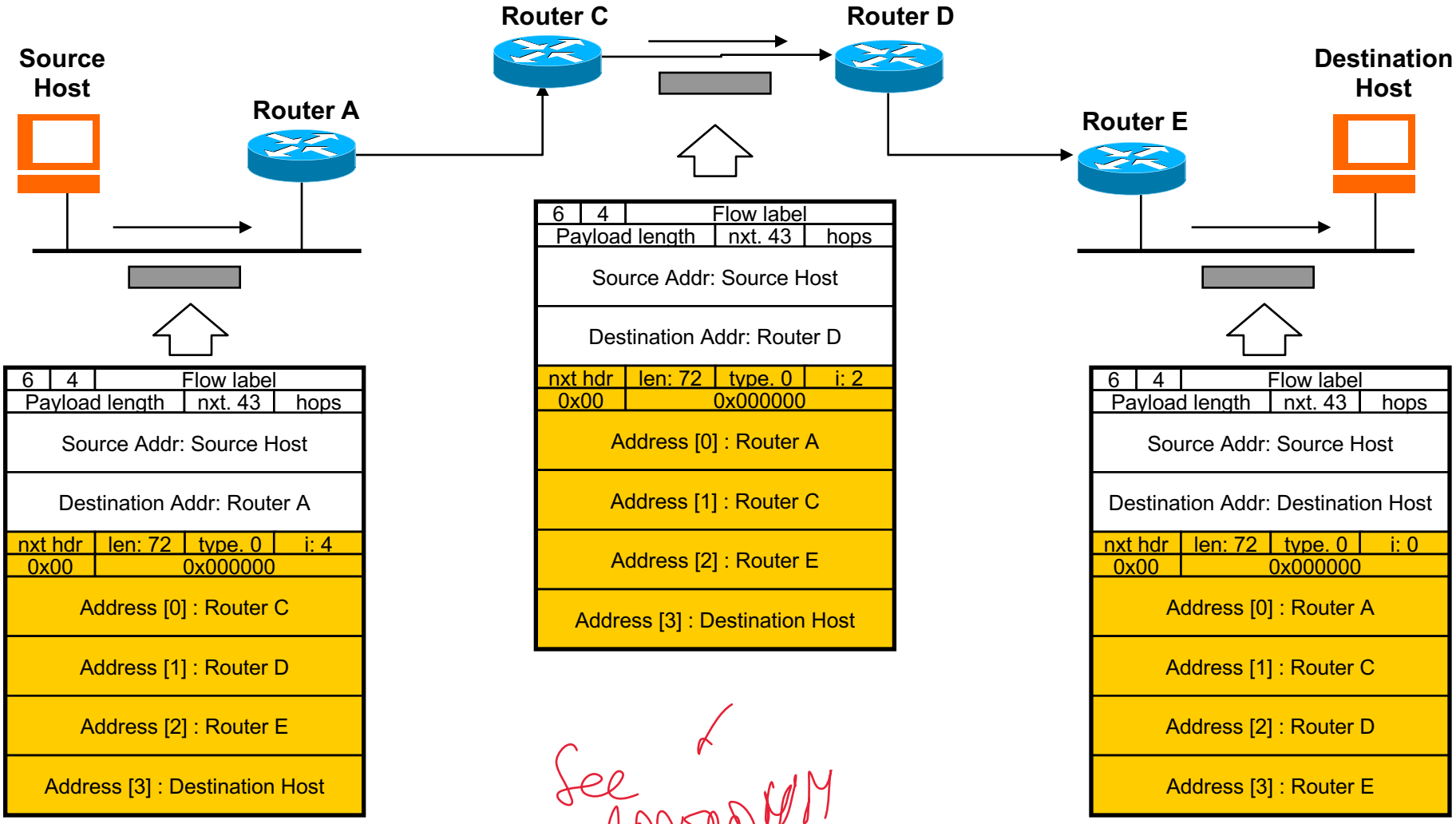
- Used by the source node to control the path followed by a packet
  - **similar to the Option *Loose Source* e *Record Route* of IPv4**
  - **processed only at the nodes with address in the *Dest Address***
- Format:
  - **Routing Type**
    - Identifies the type of routing
  - **Segments left**
    - It indicates the number of addresses which still need to be processed
    - Decremented by every node which processes it
  - **Type-specific data**
    - Field of variable length, multiple of 8 bytes
    - Depends on Routing Type

Routing Type 0 - Loose Source Route



- Address [0..n-1]
  - **Vector of *n* IPv6 addresses (*n* x 128 bit)**
  - **Adress[i] = (*i+1*)-th router along the path**
  - **At each hop, the index *i*=(*n-SegmentLeft*) is computed; the address *Address[i]* is exchanged with the *DA* (del basic Header), and the *Segment Left* field is decremented**
    - At the beginning:
      - DA= first node
      - Address[0]= second node
      - Address[n-1]= last nodo
    - At the end one has:
      - DA= last node
      - Address[0]= first node
      - Address[n-1]= before than last node

Loose Source Route - Example



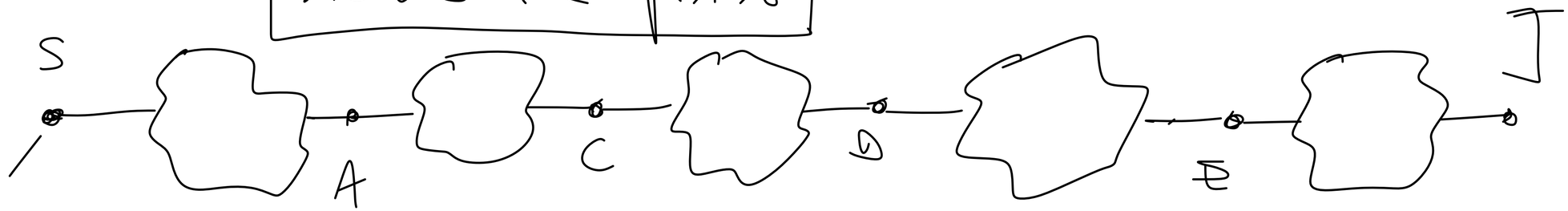
See Address

Lec. 6

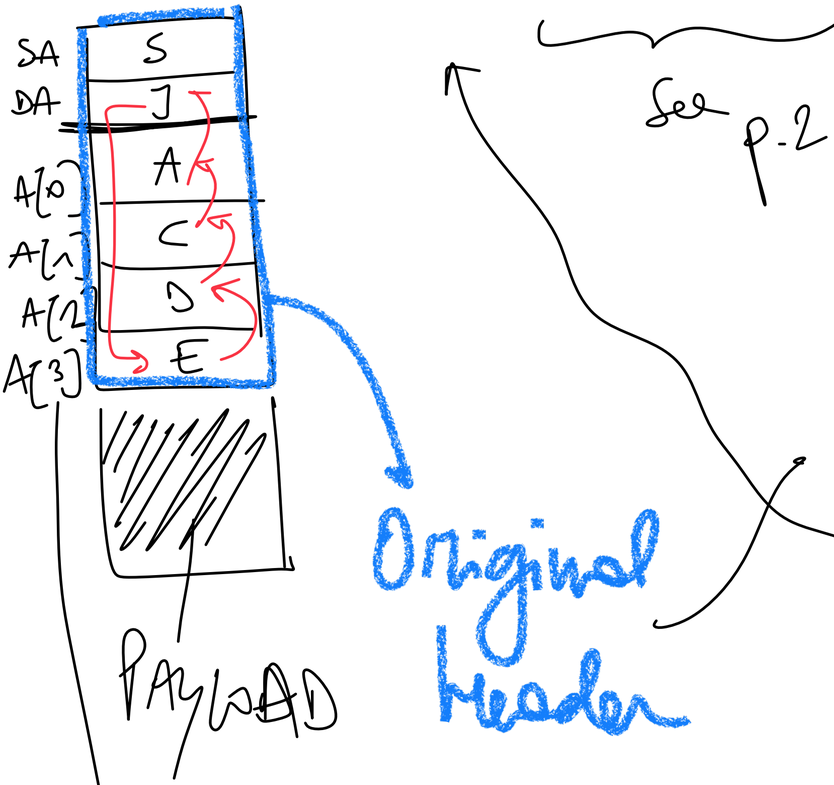
IoT - Lecture 6 - March 12, 2026

S → SA (Source Address)
] → DA (Destination Address)

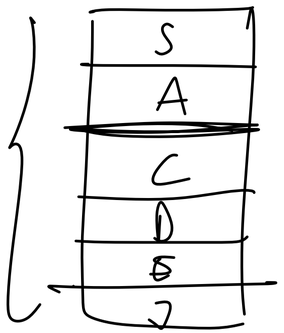
ADDENDUM @ p. 158



See p. 2

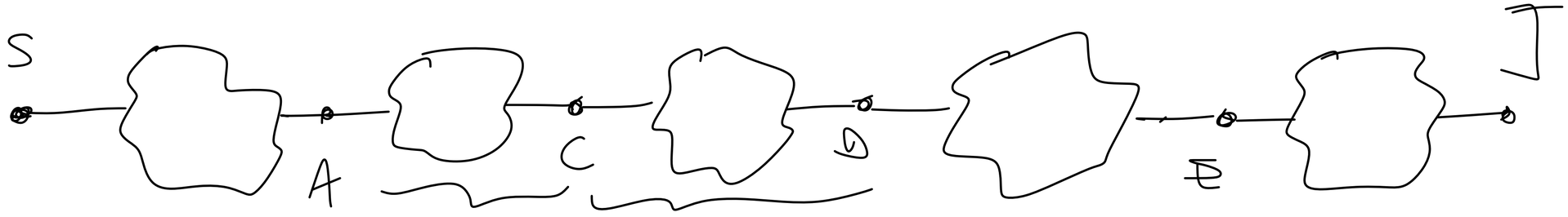


At Router @ S, the fields are changed as indicated →, obtaining



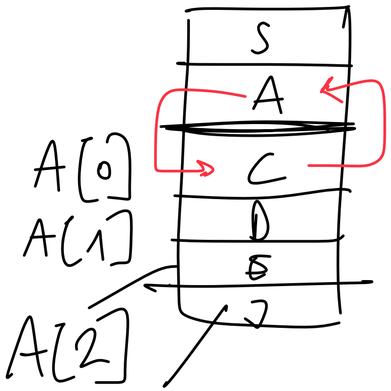
⇒ Pck flowing between S and segments legs = 4 (fold)

1



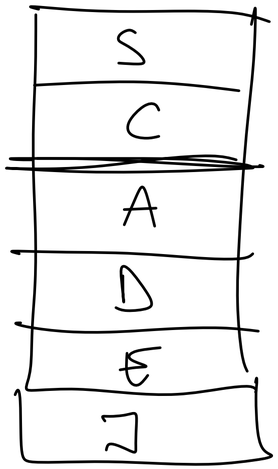
see p.3

Checks segmss legs = 4
then layer is DA
the address i = n − n = 0



they, if decrements
segmentslegs → 3

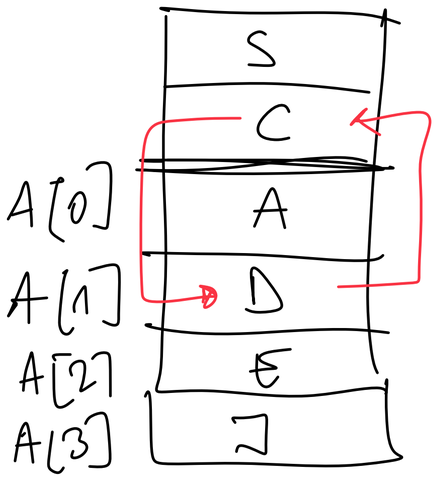
the packet formatted from A to C
is



and segments legs = 3

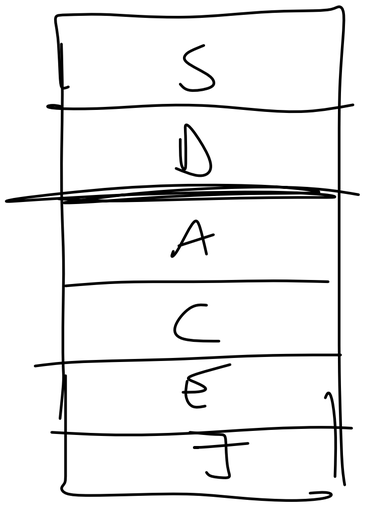
2

Node C checks segments_left = 3 → it has to retransmit
→ it inserts as DA the address of V = 4 - 3 = 1 → D



and segments_left is set to 2

⇓ the packet transmitted to D is

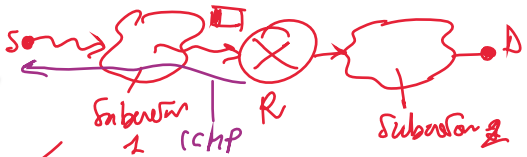


w segments_left = 2

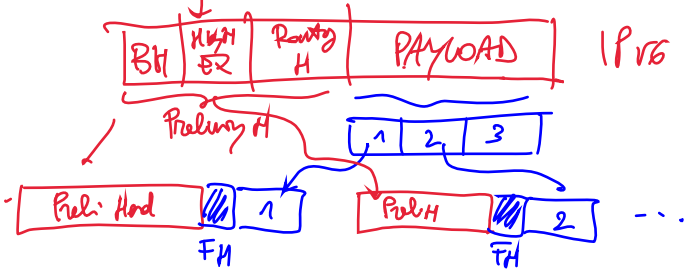
etc....

3

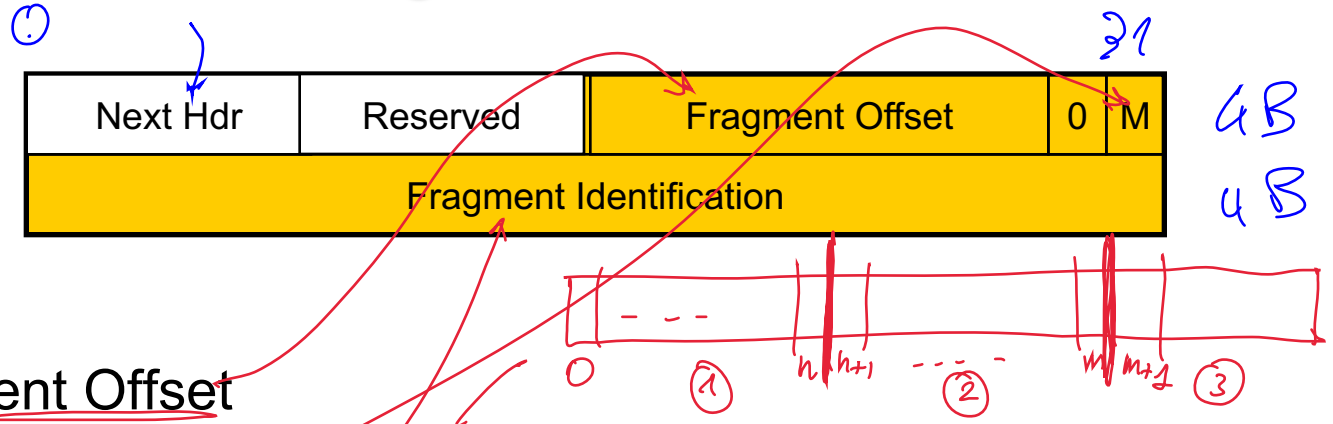
Fragment Header



- Header of fixed length equal to 64bit
- Used for segmentation and reconstruction of datagrams
- In IPv6 only the source can fragment a datagram
  - **When an intermediate node receives a datagram with length longer than the MTU, it discards it and notifieds the error to the sender through ICMP**
- Every datagram is composed by a non-fragmentable and a fragmentable part
  - **The non-fragmentable part is formed by the header which need to be processed also in intermediate nodes (Basic Header, Hop by Hop Header e Routing Header)**
- All fragments but the last one have the same length multiple of 8 bytes

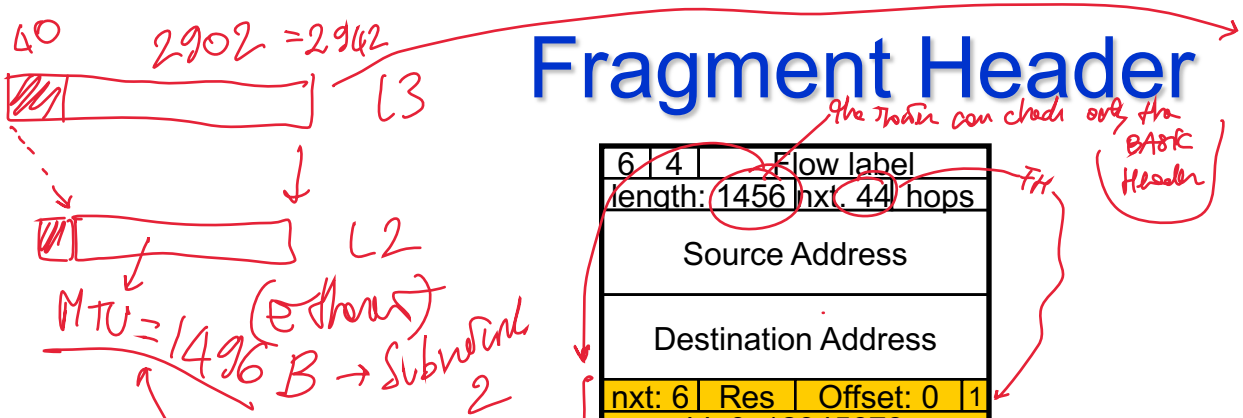


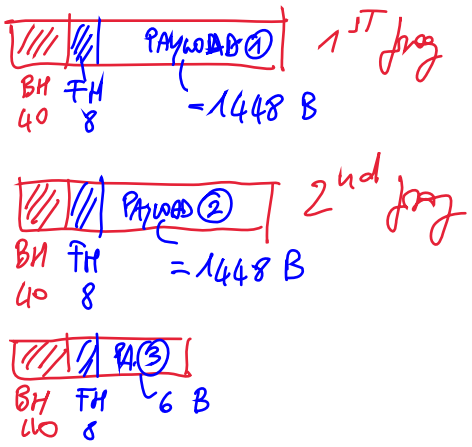
Fragment Header



- Fragment Offset
  - **Specifies the position (in units multiple of 8 bytes) of the first byte of the fragment in the original datagram**
- Bit M (More Fragments)
  - **Its value is set to 1 in all fragments of the datagram but in the last one, where it is placed to 0**
- Fragment identification
  - **Identifies the datagram which the fragment belongs to**

Fragment Header





Fragment 1

6	4	Flow label	
length: 1456		nxt: 44	hops
Source Address			
Destination Address			
nxt: 6	Res	Offset: 0	1
id: 0x12345678			
Payload data (1448 bytes)			

Fragment 2

6	4	Flow label	
length: 1456		nxt: 44	hops
Source Address			
Destination Address			
nxt: 6	Res	Offset: 181	1
id: 0x12345678			
Payload data (1448 bytes)			

Fragment 3

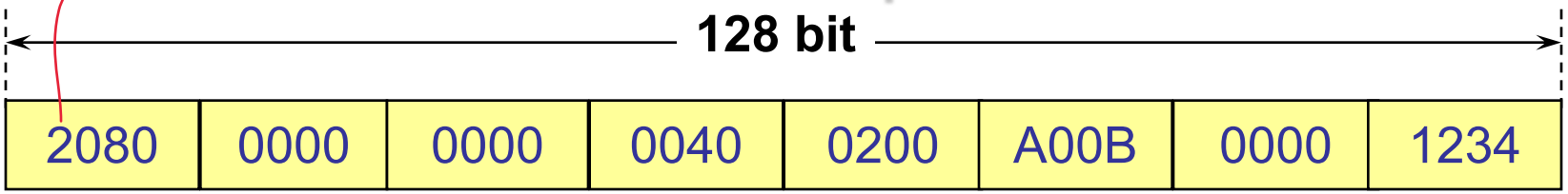
6	4	Flow label	
length: 1456		nxt: 44	hops
Source Address			
Destination Address			
nxt: 6	Res	Offset: 384	0
id: 0x12345678			
Payload data (6 bytes)			

6	4	Flow label	
length: 2902		nxt: 6	hops
Source Address			
Destination Address			
TCP Header			
Payload data (2902 bytes)			

IPv6 Addressing

- The designers of IPv6 had the benefit of IPv4 large experience and realized there would be tremendous advantages to designing the unicast address structure to reflect the overall topology of the Internet. These include:
  - **Easier allocation of address blocks at various levels of the Internet topological hierarchy**
  - **IP network addresses that automatically reflect the hierarchy by which routers move information across the Internet, allowing routes to be easily aggregated for more efficient routing**
  - **Flexibility for organizations like ISPs to subdivide their address blocks for customers**
  - **Flexibility for end-user organizations to subdivide their address blocks to match internal networks, much as subnetting did in IPv4**
  - **Greater "meaning" to IP addresses. Instead of just being a string of 128 bits with no structure, it would become possible to look at an address and know certain things about it**

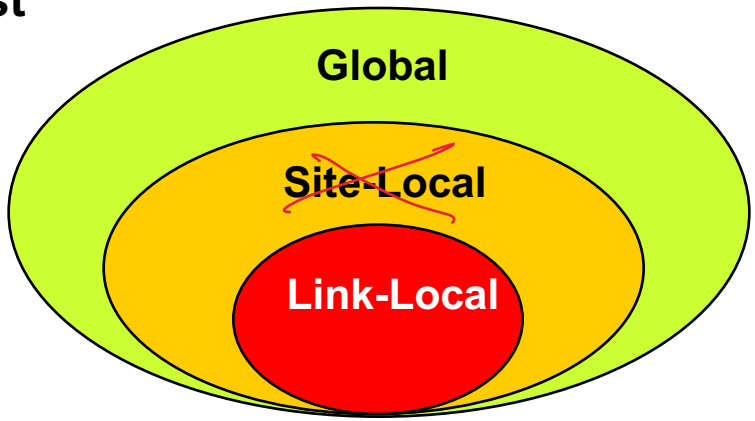
IPv6 Address Representation



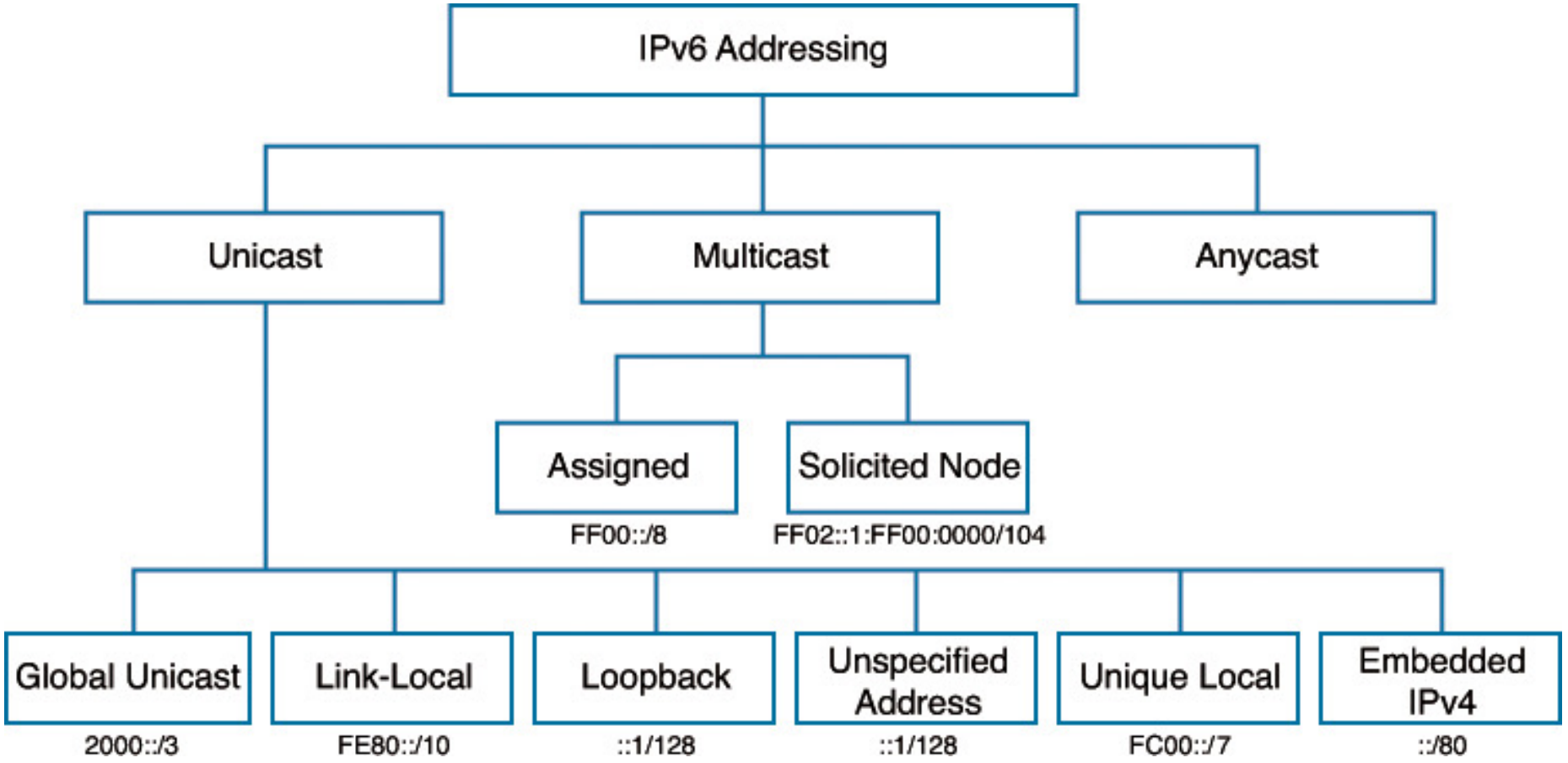
- 16-bit fields in case-insensitive colon hexadecimal (0,…,9,A,…,F) representation
  - 2080:0000:0000:0040:0200:A00B:0000:1234
- Some simplifications:
  - **leading zeroes in a field are optional and can be omitted**
    - 2080:0:0:40:200:A00B:0:1234
  - **successive fields of 0 represented as ::, but only once**
    - 2080::40:200:A00B:0:1234
    - 0:0:0:0:0:0:0:1 → ::1
    - 0:0:0:0:0:0:0:0 → ::
- IPv4-compatible address representation:
  - **0:0:0:0:0:0:10.0.0.1 = 0:0:0:0:0:0:A00:1 = ::A00:1 = ::10.0.0.1**

IPv6 Addressing

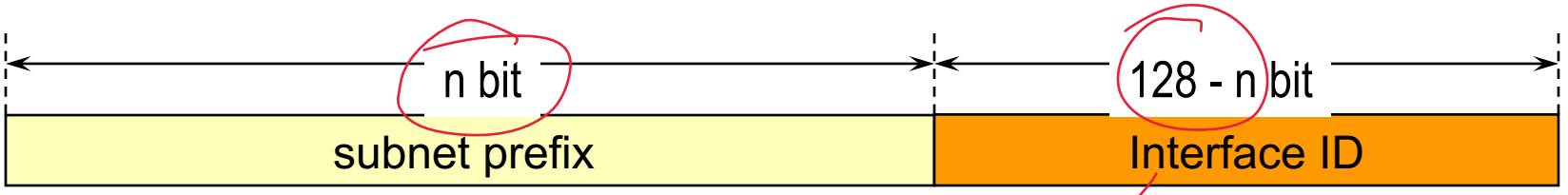
- IPv6 Addressing rules are covered by multiples RFC's
- Address Types are:
  - **Unicast : One-to-One (Global, Link local, Site local, Compatible)**
  - **Anycast : One to Nearest (Allocated from Unicast)**
  - **Multicast : One to Many**
  - **Reserved**
- A single interface may be assigned multiple IPv6 addresses of any type (unicast, anycast, multicast)
  - **No Broadcast Address → Use Multicast**
- Different address visibility
  - **Global**
  - **Site-Local (deprecated)**
  - **Link-Local**



IPv6 Addressing



IPv6 Addresses: Subnet Prefix



- Subnet prefix identifies:
  - **Type of address**
  - **Network/subnetwork that the address belongs to**
- The prefix length is specified by adding "/N" where N is the lenght (in bits)
  - **example:**
    - FEDC:0123:8700::1:2:3:4 /36
- Interface ID identifies:
  - **a node interface (within the given network)**

IPv6 Addresses

- IPv6 addresses start with a variable length "Prefix Format" (PF) indicating the address type

Address type	Binary prefix	IPv6 notation
Unspecified	00...0 (128 bits)	::/128
Loopback	00...1 (128 bits)	::1/128
Multicast	1111 1111	FF00::/8
Link-local unicast	1111 1110 10	FE80::/10
Site-local unicast	11111110 11	FEC0::/10
Global unicast	001	2000::/3

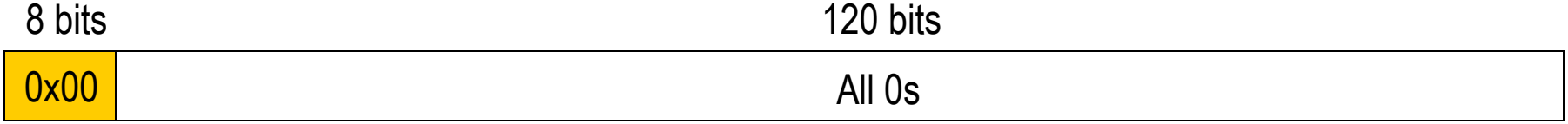
- Anycast addresses are taken by unicast addresses spaces

IPv6 Addresses (cont.)

Binary prefix	*Address type*
0000 0000	Reserved
0000 0001	Unassigned
0000 001	Reserved for NSAP (non-IP addresses used by ISO)
0000 010	Reserved for IPX (non-IP addresses used by IPX)
0000 011	Unassigned
0000 1	Unassigned
0001	Unassigned
001	Unicast Address Space
010	Unassigned
011	Unassigned
100	Unassigned
101	Unassigned
110	Unassigned
1110	Unassigned
1111 0	Unassigned
1111 10	Unassigned
1111 110	Unassigned
1111 1110 0	Unassigned
1111 1110 10	Link Local Use addresses
1111 1110 11	Site Local Use addresses
1111 1111	Multicast addresses

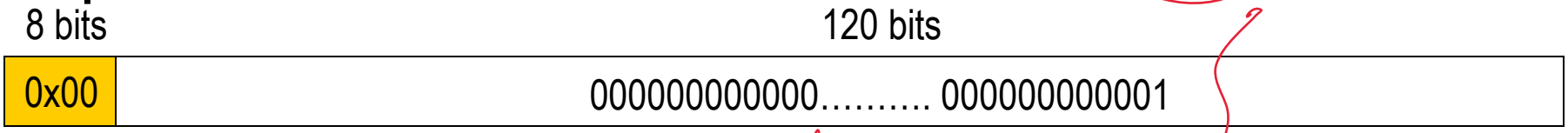
Reserved Addresses with prefix 0x00

Unspecidfied address (only as source address)



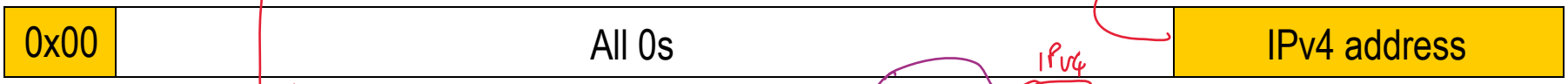
::0

Loopback address



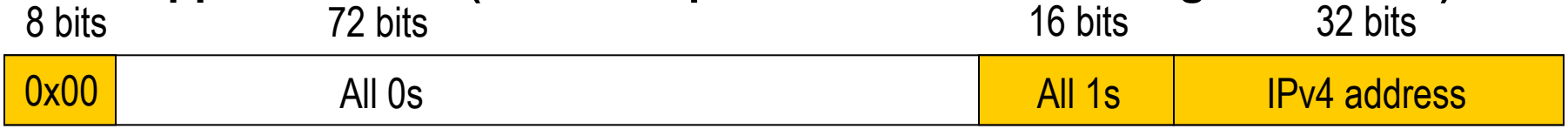
::1

IPv4 compatible address (to send a pkt to a IPv6 host through IPv4 nets)



::x.y.z.w

IPv4 mapped address (to send a pkt to a IPv4 host through IPv6 nets)



::FFFF:x.y.z.w

Unspecified and Loopback Addresses

- Unspecified address 0:0:0:0:0:0:0:0
  - **::**
  - **It must never be assigned to any node**
  - **It indicates the absence of an address**
  - **e.g. as src addr of any pkts sent by an initializing host**
  - **must not be used as the dest addr of pkts**
  - **a packet with a src addr :: must never be forwarded by an IPv6 router**
- Loopbackaddress 0:0:0:0:0:0:0:1
  - **::1**
  - **It may be used by a node to send an IPv6 packet to itself**
  - **It must not be assigned to any physical interface**
  - **It is treated as having link-local scope (of a virtual interface called "loopback interface")**
  - **must not be used as src addr in pkts sent outside of a single node**
  - **a packet with dest addr of loopback must never be sent outside**

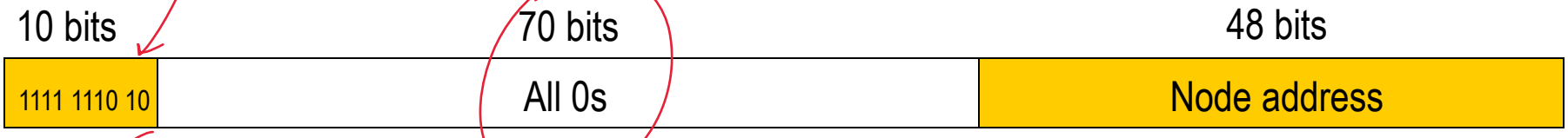
IPv6 Addresses with Embedded IPv4 Addresses

- IPv4-Compatible IPv6 Address
  - **was defined to assist in the IPv6 transition (to send a pkt to a IPv6 host through IPv4 nets)**
  - **is now deprecated because the current IPv6 transition mechanisms no longer use these addresses**

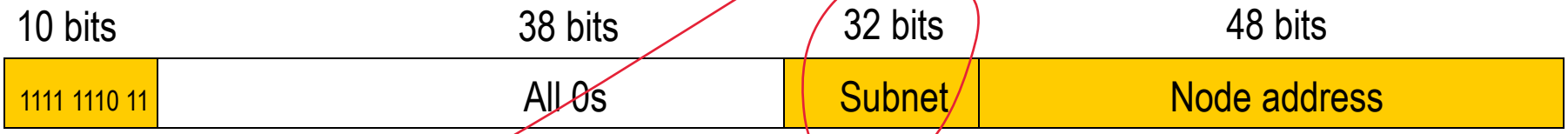
- IPv4-Mapped IPv6 Address
  - **IPv6 address that holds an embedded IPv4 address is defined**
  - **it is used to send a pkt to a IPv4 host through IPv6 nets**

Local Addresses

- Link local address (only within a subnet, e.g. LAN)



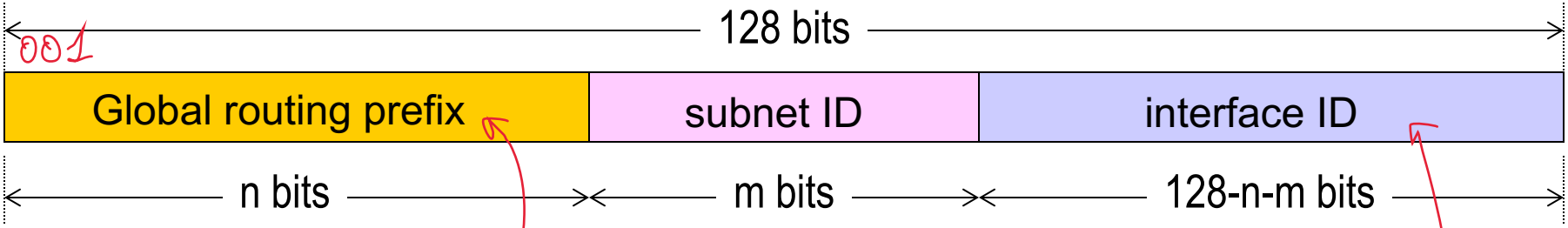
- Site local address (only though private subnets)



- Note:
  - **Site-local addresses were originally designed to be used for addressing inside of a site without the need for a global prefix**
  - **Site-Local addresses are now deprecated**

Global Unicast Addresses

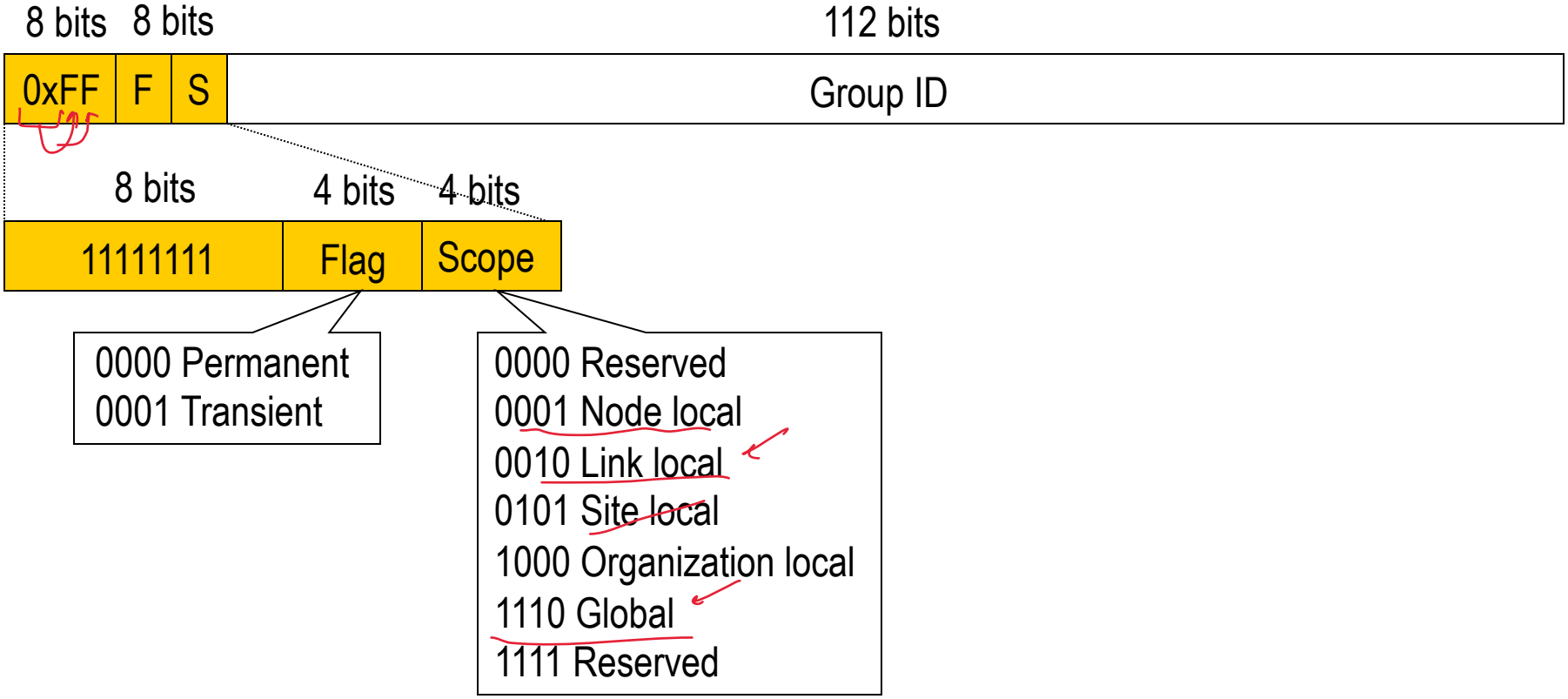
- The general format for IPv6 global unicast addresses is as follows



- where
  - **global routing prefix is a (typically hierarchically-structured) value assigned to a site (a cluster of subnets/links)**
  - **subnet ID is an identifier of a link within the site**
  - **interface ID uniquely identify interfaces on a link; it can be derived directly from that interface's link-layer address**
- All global unicast addresses other than those that start with binary 000 have a 64-bit interface ID field (e.g. n=48, m=16)
  - **the 64 bits interface ID is constructed in Modified EUI-64 format**
- Global unicast addresses that start with binary 000 have no such constraint on the size or structure of the interface ID field

Multicast Addresses

Multicast addresses



ICMPv6

- Internet Control Message Protocol per IPv6, used for:
  - **error reporting**
  - **diagnostic**
  - **neighbor discovery**
  - **management of multicast groups**
- It replaces some functions previously done (for IPv4) by:
  - **ICMP**
  - **ARP (Address Resolution Protocol)**
  - **IGMP (Internet Group Membership Protocol)**
- Two classes of messages:
  - **Error messages**
  - **Informational messages**
    - diagnostic
    - neighbor discovery
    - management of multicast groups

Sent by a router to indicate to a source node that it needs to fragment the packet

ICMPv6 messages

- Error messages (types from 0 to 127)
  - **Destination Unreachable**
  - **Packet Too Big**
  - **Time Exceeded**
  - **Parameter Problem**
- Informational messages (types from 128 to 255)
  - **Echo Request**
  - **Echo Reply**
  - **Router Solicitation**
  - **Router Advertisement**
  - **Neighbor Solicitation**
  - **Neighbor Advertisement**
  - **Redirect**
  - **Group management (Multicast Listener Discovery Protocol)**

Neighbor Discovery protocol

skip

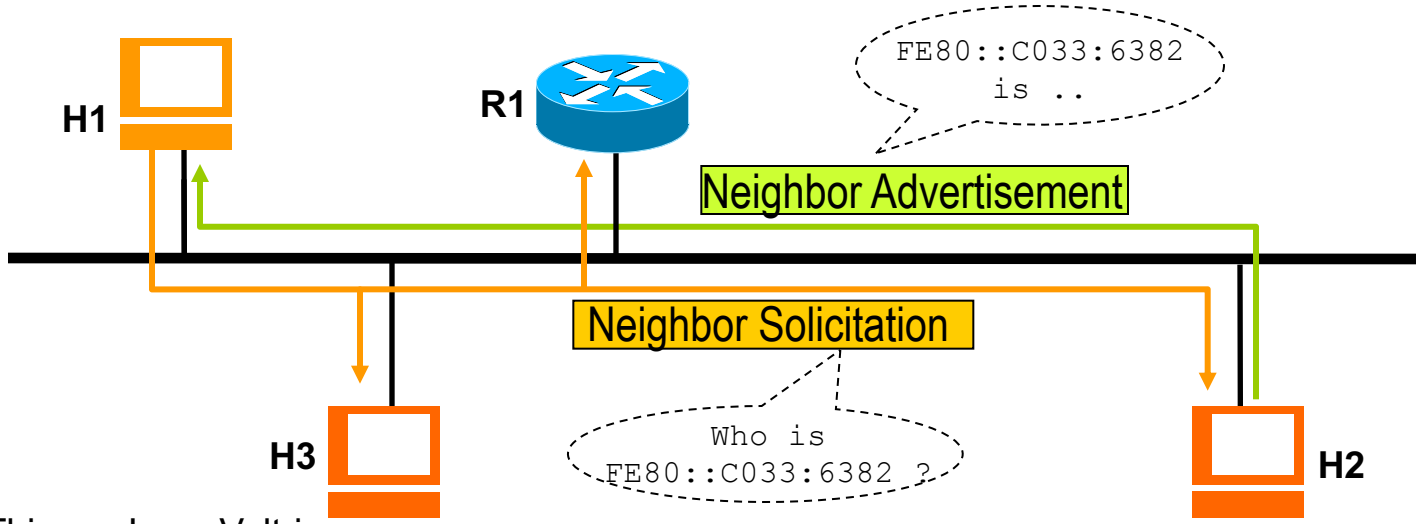
- It solves a set of problems related to the interaction between nodes attached to the same link
  - **Router Discovery**
    - how hosts locate routers that reside on an attached link
  - **Prefix Discovery**
    - how hosts discover the set of address prefixes that define which destinations are on-link for an attached link
  - **Parameter Discovery**
    - How a node learns such link parameters as the link MTU or such Internet parameters as the hop limit value to place in outgoing packets
  - **Address Autoconfiguration**
    - Introduces the mechanisms needed in order to allow nodes to automatically configure an address for an interface

Neighbor Discovery protocol (cont.) skip

- **Address resolution**
  - How nodes determine the link-layer address of an on-link destination (e.g., a neighbor) given only the destination's IP address
- **Next-hop determination**
  - The algorithm for mapping an IP destination address into the IP address of the neighbor to which traffic for the destination should be sent
  - The next-hop can be a router or the destination itself
- **Neighbor Unreachability Detection**
  - How nodes determine that a neighbor is no longer reachable
  - For neighbors used as routers, alternate default routers can be tried
- **Duplicate Address Detection**
  - How a node determines that an address it wishes to use is not already in use by another node
- **Redirect**
  - How a router informs a host of a better first-hop node to reach a particular destination

Address Resolution

- The process through which a node determines the link-layer address of a neighbor given only its IP address
- Neighbor Solicitation
  - **Sent by a node when it has a unicast packet to send to a neighbor, but does not know the neighbor's link-layer address**
  - **Used also for Duplicate Address Detection**
- Neighbor Advertisement
  - **In reply to Neighbor Solicitation**
  - **Sent also for notifying a change of address**



Host Autoconfiguration

- IPv6 supports different type of host auto-configurations
- Two “plug-and-play” ways for host configuration:
  - **Stateful**
    - DHCPv6 (Dynamic Host Configuration Protocol)
    - Similar to DHCP for IPv4
  - **Stateless**
    - Message exchange between Host and Router
    - Network prefix from ICMP Router Solicitation/Advertisement
    - The host forms its own address by appending its PH/DL interface address (e.g. MAC address)
- The two modes may coexist

Multicast Listener Discovery Protocol (MLDv2)

- Multicast Listener Discovery Protocol (MLD)
- Used by IPv6 routers to
  - **discover the presence of multicast listeners (i.e., nodes that wish to receive multicast packets) on their directly attached links, and**
  - **discover specifically which multicast addresses are of interest to those neighboring nodes**
- MLDv2 is a translation of the IGMPv3 protocol for IPv6 semantics

Transition mechanisms

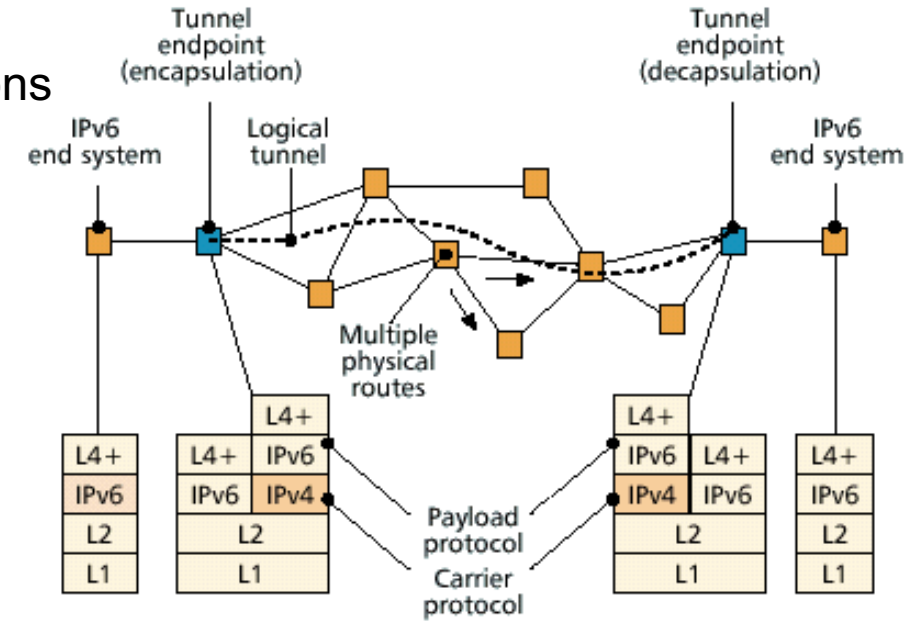
Transition mechanisms

- Transition mechanisms generally come in one one three forms
  - **Dual stack**
  - **Translation**
  - **Tunneling**

- Dual stack
  - **two protocol stacks are separately maintained and operate in parallel**
  - **the device can operate via either protocols**
  - **can be implemented in both end systems and network nodes**
    - in end systems: it enables both IPv4 and IPv6 applications to operate on the same node
    - in network nodes: it allows handling of both IPv4 and IPv6 packets

Transition mechanisms

- Tunneling
  - **used to bridge compatible nodes (e.g. IPv6) across incompatible networks (e.g. IPv4)**
  - **IPv6-over-IPv4 or IPv4-over-IPv6**
  - **main problem is the configuration of tunnels (on tunnel endpoints)**
  - **tunnel endpoint addresses are generally attained:**
    - manually or by tools (e.g. tunnel broker)
    - through existing services such as DNS or DHCP options
    - by embedding information in the link layer addresses or IP addresses



UNIVERSITÀ DI PARMA

IoT Lab

→ lez. 8

Communication Mechanisms

Main sources

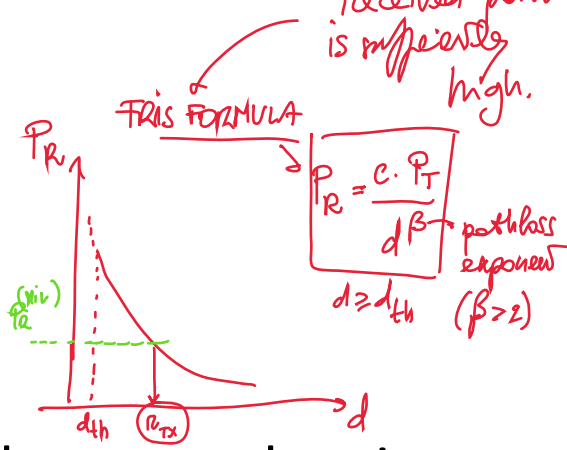
- J. P. Vasseur and A. Dunkels, "Interconnecting Smart Objects with IP,` Morgan Kaufmann, 2010. Chapter 12.

Communication Patterns

- Smart object communication: one-to-one, one-to-many, and many-to-one.
- Each communication pattern is used in different situations. Many applications use a combination of the patterns.
- Smart objects often communicate over unreliable communication channels: (i) interference (shared medium and free spectrum band); (ii) obstacles on the path
- Need for reliable communication protocols (best-effort may be sufficient in some cases).
- Different requirements: static (e.g., industrial WSN) vs dynamic (BSN).
- Power consumption: switch off as much as possible.

One-2-one Communication

FIGURE 12.1
One-to-one communication in a smart object network.



- One source and one destination, with possibly more relays in the middle (ad-hoc network)
- Could rely on the extensive body of knowledge on ad-hoc wireless networks

One-2-many Communication

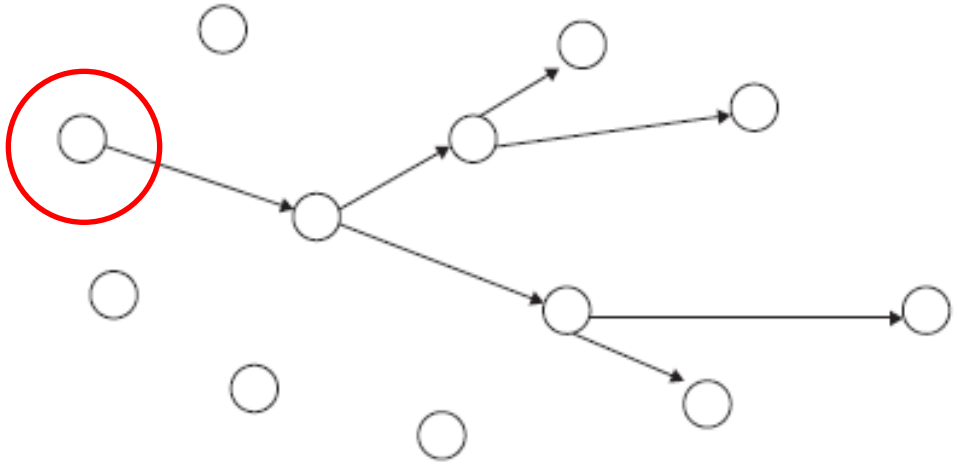


FIGURE 12.2
One-to-many communication in a smart object network.

- Need to reach all nodes (e.g., sending a command)
- Various mechanisms: flooding; irresponsible forwarding (IF); trickle (explicitly designed for low-power networks→RPL)
- Used in routing protocols for one-2-one communications (e.g., Ad-hoc On-demand Distance Vector, AODV) → iAODV

Many-2-one Communication

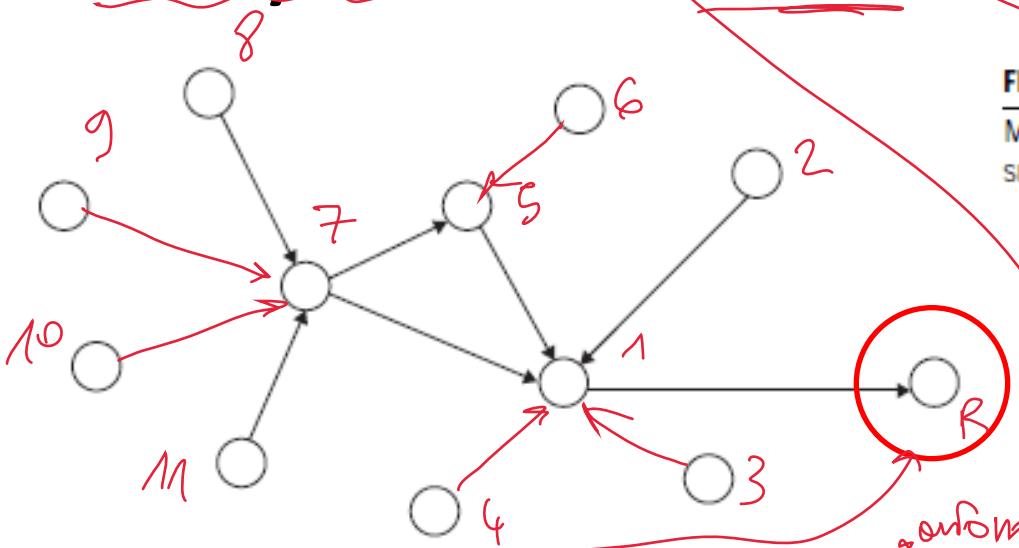
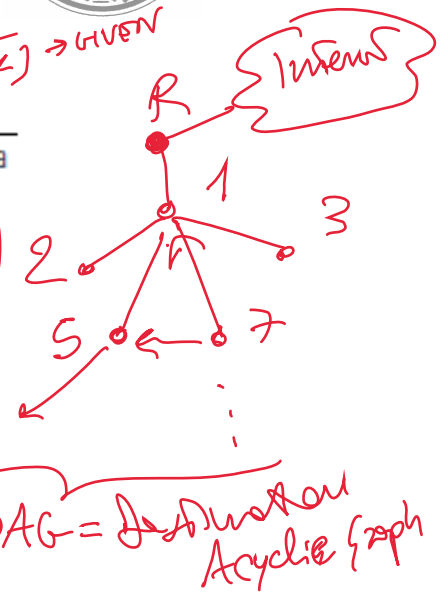


FIGURE 12.3
Many-to-one communication in a smart object network.



- Collect at a *sink*: sensor data or network health status (e.g., periodic status reports)
- There may be more than one sink inside the network.
- Set-up: build a tree structure with its root at the sink (iterative process from sink to all nodes) → hop count-based routing
- To account for radio quality in addition to hop count, several cost metrics can be considered → expected transmissions (ETX) is a good metric
- Collection Tree Protocol (CTP) based on ETX

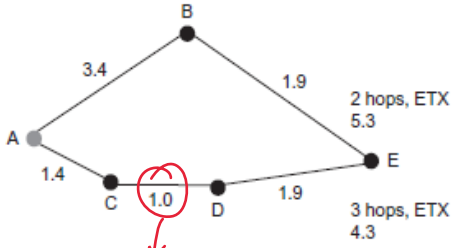


FIGURE 12.4
ETX in a five-node network.

Physical Communication Standards

- Of interest: IEEE 802.15.4 (wireless), IEEE 802.11 (wireless), Power Line Communications (PLC, wired)
- Transmission ranges:
  - IEEE 802.15.4: short range (a few m)
  - IEEE 802.11: medium range (up to several hundred m)
  - PLC: potentially long range (depends on eletric loads connected to the cable)
- Network formation
  - PLC: all nodes connected to the same cable (same connectivity level)
  - IEEE 802.15.4: topology change (mobile nodes or environment change)
  - IEEE 802.11: similar to IEEE 802.15.4 (problems less pronounced)
- Routing
  - IEEE 802.15.4: routing is crucial (nodes act often as relays)
  - PLC: no real need for routing (broacast in the cable)
  - IEEE 802.11: for extended network coverage, APs are connected

IEEE 802.15.4: Basics

- Developed by the 802.15 personal area network (PAN) Working Group within the Institute of Electrical and Electronics Engineers (IEEE).
  - Maximum data rate: 250 kb/s
  - Maximum transmit power: 1 mW
  - Nominal transmit range: a few tens of meters
- Focus: to allow low-cost and low-complexity transceivers
- Low-power radio stacks are built on IEEE 802.15.4: WirelessHART, ISA100a, IPv6, and ZigBee.
- The IEEE 802.15.4 standard specifies two layers: PHY and MAC (several mechanisms are proposed, but not all are used)
- Maximum PHY packet (payload) size: 127 bytes → data for upper layers is between 86 and 116 bytes (besides the MAC header) → upper layers need to fragment large data portions into 802.15.4 frames
- IEEE 802.15.4 is typically implemented as a combination of hw (PHY and part of MAC) and sw (MAC logic)

IEEE 802.15.4: Topologies

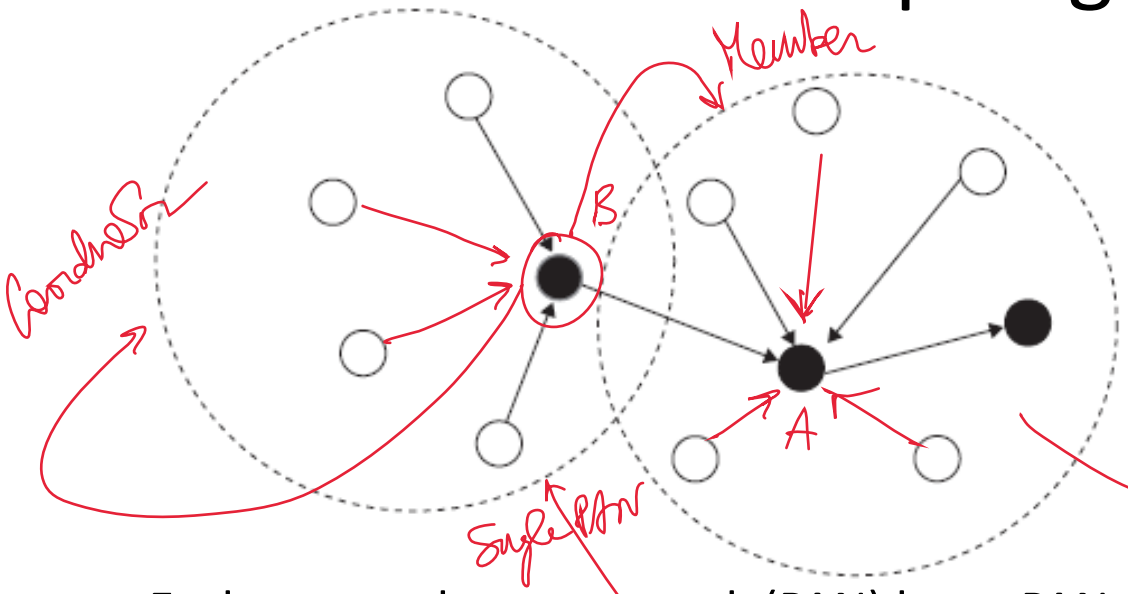
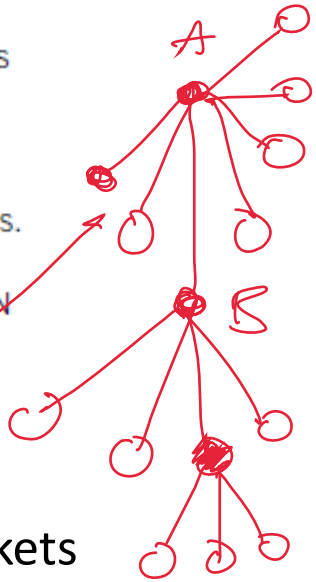


FIGURE 12.5

An IEEE 802.15.4 network with FFDs shown as dark dots and RFDs shown as hollow dots. Two of the FFDs are PAN coordinators in the two PANs, shown as dotted circles. The right PAN contains two FFDs, but only one is the PAN coordinator.



- Each personal area network (PAN) has a PAN coordinator and packets carry a 16-bit identifier.
- A device can participate in one PAN as the PAN coordinator and simultaneously participate as a PAN member in another PAN.
- Two types of nodes: fully functional devices (FFDs, can be coordinators) and reduced function devices (RFDs, cannot be coordinators)
- Three types of network topology: star, mesh, and cluster tree
- Routing is minor → protocols on top of 802.15.4 build their own topology and routing protocols

IEEE 802.15.4: Addresses

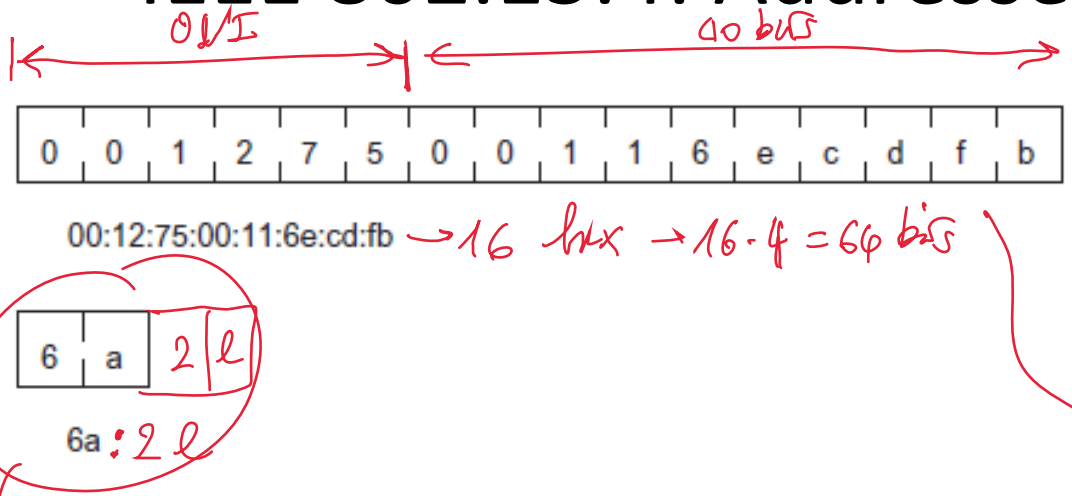


FIGURE 12.6
IEEE 802.15.4 supports two addressing formats: long (64-bit) addresses and short (16-bit) addresses.

MAC address !
fixed

- Each node in an 802.15.4 network has a 64-bit address that uniquely identifies the device: very long!
- Long addresses are globally unique and each 802.15.4 device is assigned an address when manufactured (24-bit for organizational unique identifier (OUI) from the IEEE; 40 bit assigned by the manufacturer), To the specific producer
- Short addresses (16-bit) are assigned (run-time) by the PAN coordinator and are valid only within the context of a PAN

IEEE 802.15.4: PHY layer (1/2)

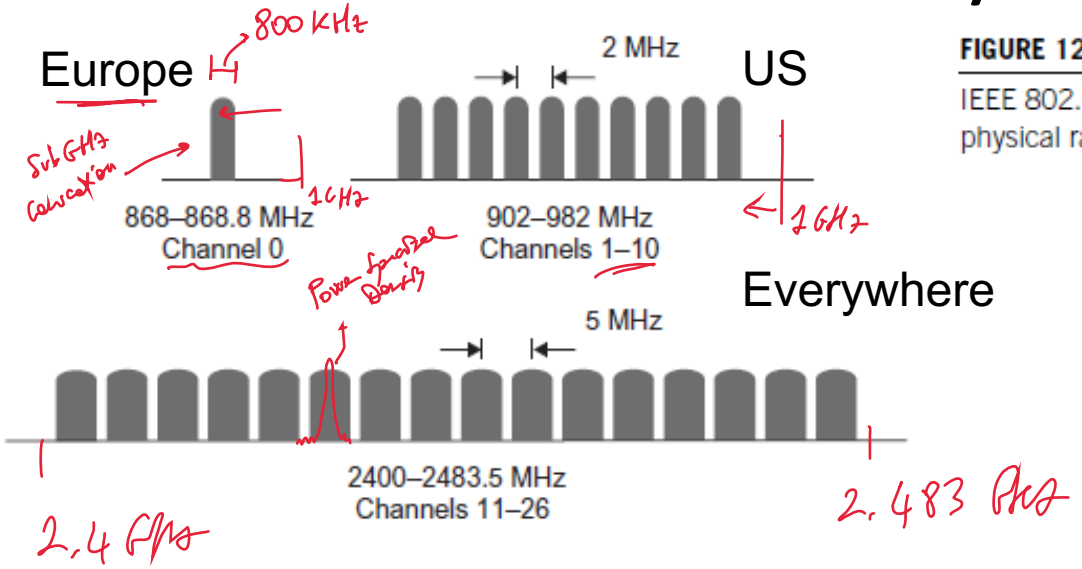


FIGURE 12.7

IEEE 802.15.4 defines 26 physical radio channels.

- **Modulations:**
  - Channels 0-10: BPSK
  - Channels 11-26: QPSK
- **Bit rates:**
  - ch. 0: 20 kb/s
  - ch. 1-10: 40 kb/s
  - ch. 11-26: 250 kb/s

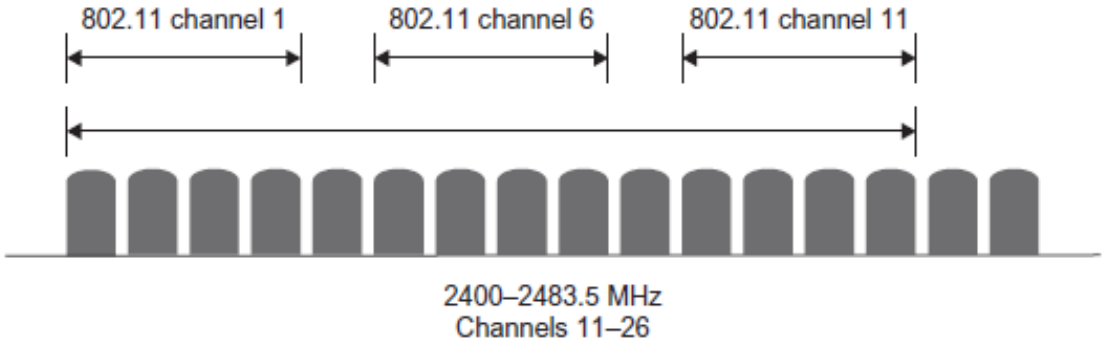
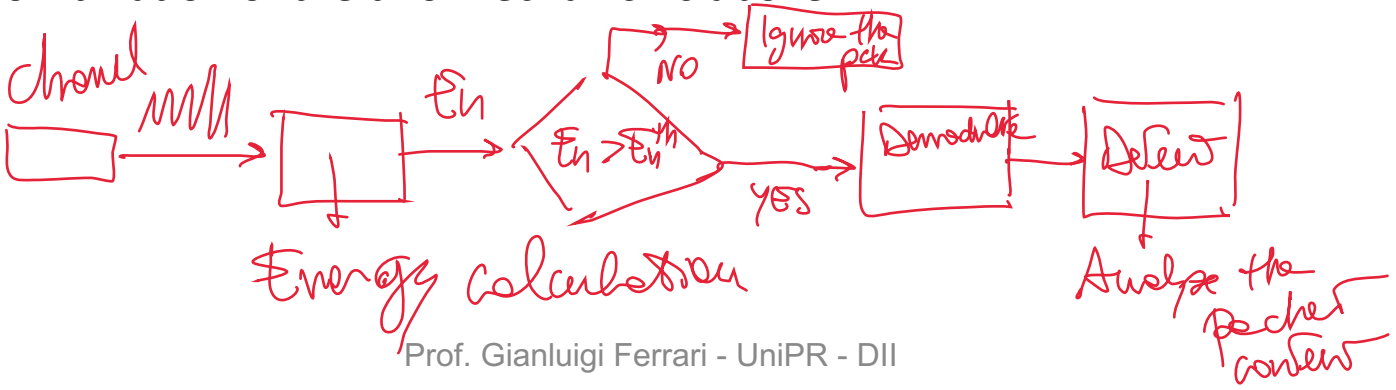


FIGURE 12.8

IEEE 802.15.4 channels 11–24 overlap the 802.11 channels. Channels 25 and 26 are not covered by 802.11 channels. When the non-overlapping 802.11 channels 1, 6, and 11 are used, two additional 802.15.4 channels are undisturbed by 802.11.

IEEE 802.15.4: PHY layer (2/2)

- Interference from IEEE 802.11 devices (WiFi) is a serious issue → lossy links
- PHY layer: provides mechanisms to measure the radio energy for a given radio channel: clear channel assessment (CCA) → used by MAC layer
- CCA uses three mechanisms:
  - measure the radio energy and compare it with a predefined energy threshold
  - demodulate the incoming radio signal to see if it is a valid 802.15.4 signal
  - a combination of the two mechanisms above



IEEE 802.15.4: MAC layer

- MAC layer: control access to the radio medium → control when the medium is idle (CCA) and how to coordinate access by multiple nodes
- IEEE 802.15.4 MAC layer provides: channel access management, validation of incoming frames, and acknowledgment of frame reception.
- Optional mechanisms for a time-division multiple access (TDMA) mechanism for medium access: the PAN coordinator assigns time slots to PAN devices (beacon mode: typically not of interest)
- Validation of incoming frames by computing: 16-bit cyclic redundancy check (CRC) of the entire frame
- If an incoming frame has the acknowledgment bit set → MAC layer broadcasts an acknowledgment frame → interference!

IEEE 802.15.4: Frame formats

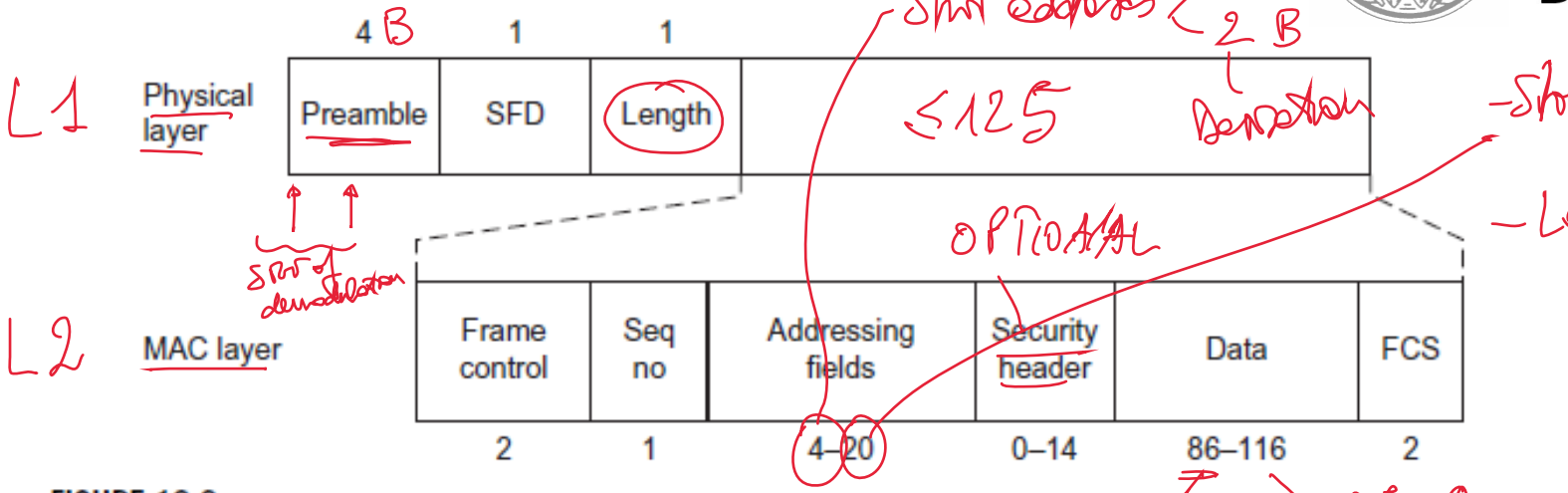


FIGURE 12.9

The IEEE 802.15.4 physical layer and MAC layer header formats.

- **PHY frame**:
  - Preamble: used to synchronize
  - Start of Frame Delimiter (SFD): indicates end of preamble
  - Length: between 0 and 2^7-1=127
- **MAC frame**:
  - Frame control: flags to interpret the rest of the header and request of ack
  - Sequence number: used by ack
  - Addressing fields: optional (sender/receiver, PAN)
  - Frame check sequence (FCS): contains the CRC to be checked

IEEE 802.15.4: Power consumption

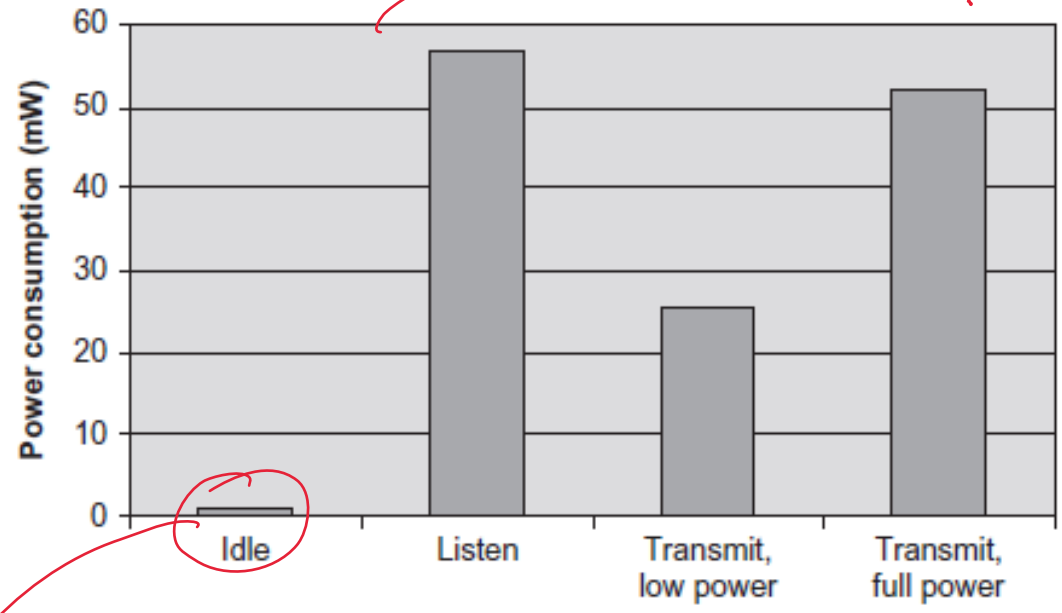
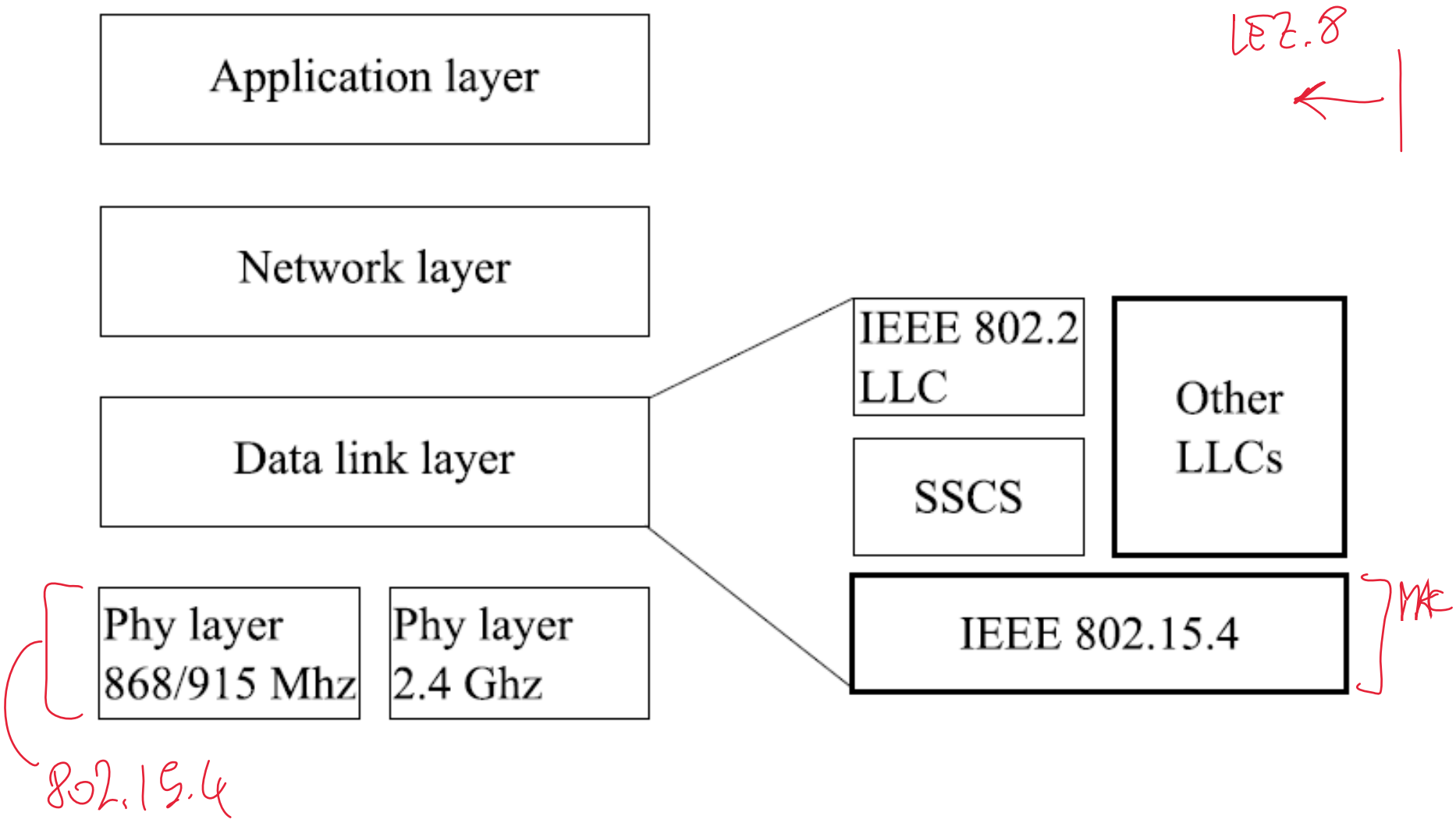


FIGURE 12.10

The power consumption of the CC2420 IEEE 802.15.4 radio transceiver.

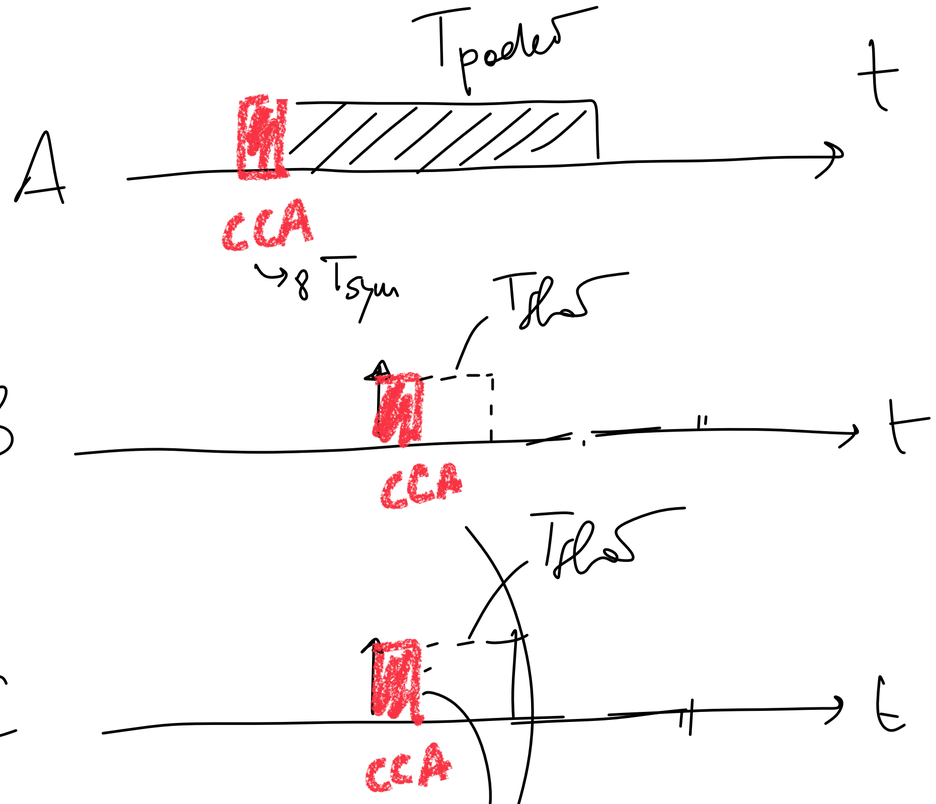
- Listen power consumption higher than transmit power consumption (needs to demodulate even if no useful signal is present)
- Transmit power can be set on a per-packet basis (between low and full)

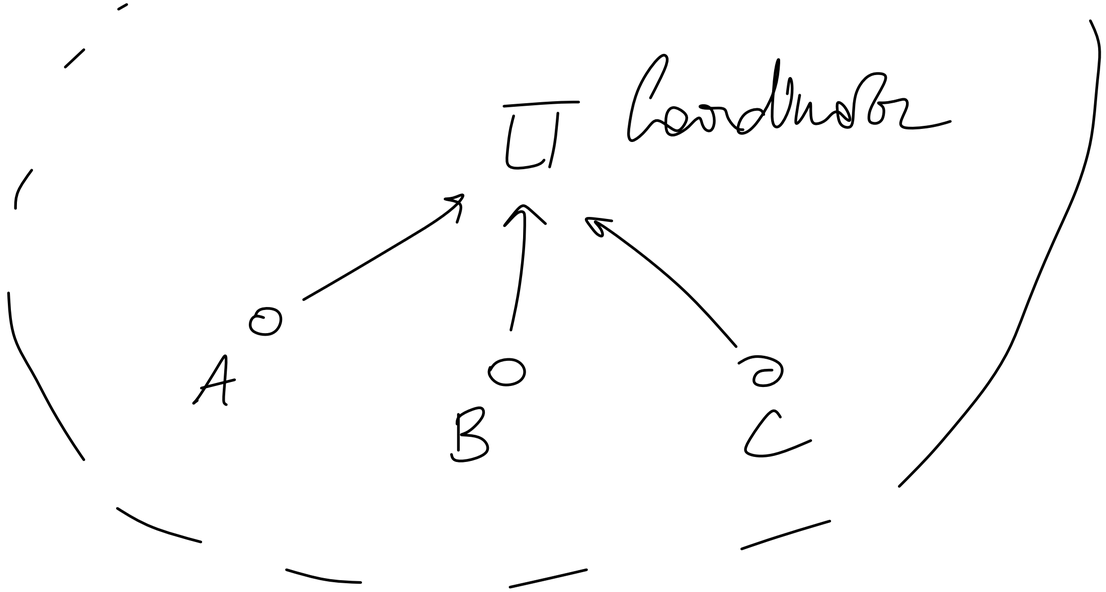
The ISO/OSI Stack of IEEE 802.15.4



IoT - Lecture 9 - Mar. 25, 2026

802.15.4





CSMA/CA

Before sending a packet, every node senses the channel

$BE_1 = 1$

1) B, C → Wait for a number of T slots randomly selected in the set $\{0, 2^{BE_1} - 1\} = \{0, 1\}$ — CA Mechanism

Denote as $n_{slot}^{(B)}$ and $n_{slot}^{(C)}$ the number of slots

$n_{slot}^{(B)} \in \{0, 1\}$ $n_{slot}^{(C)} \in \{0, 1\}$

After this number of T_{slot} intervals have elapsed → they do CCA again

2) If the channel is sensed busy again

$$BE_2 = BE_1 + 1 = 1 + 1 = 2$$

$$\{0, \dots, 2^{BE_2} - 1\} = \{0, 1, 2, 3\}$$

A node selects randomly a number in this set and waits for $n^{(B)}_{slot} \cdot T_{slot}$ (or $n^{(C)}_{slot} \cdot T_{slot}$).

3) If the channel is sensed busy $\rightarrow$ $BE_3 = BE_2 + 1 = 3$

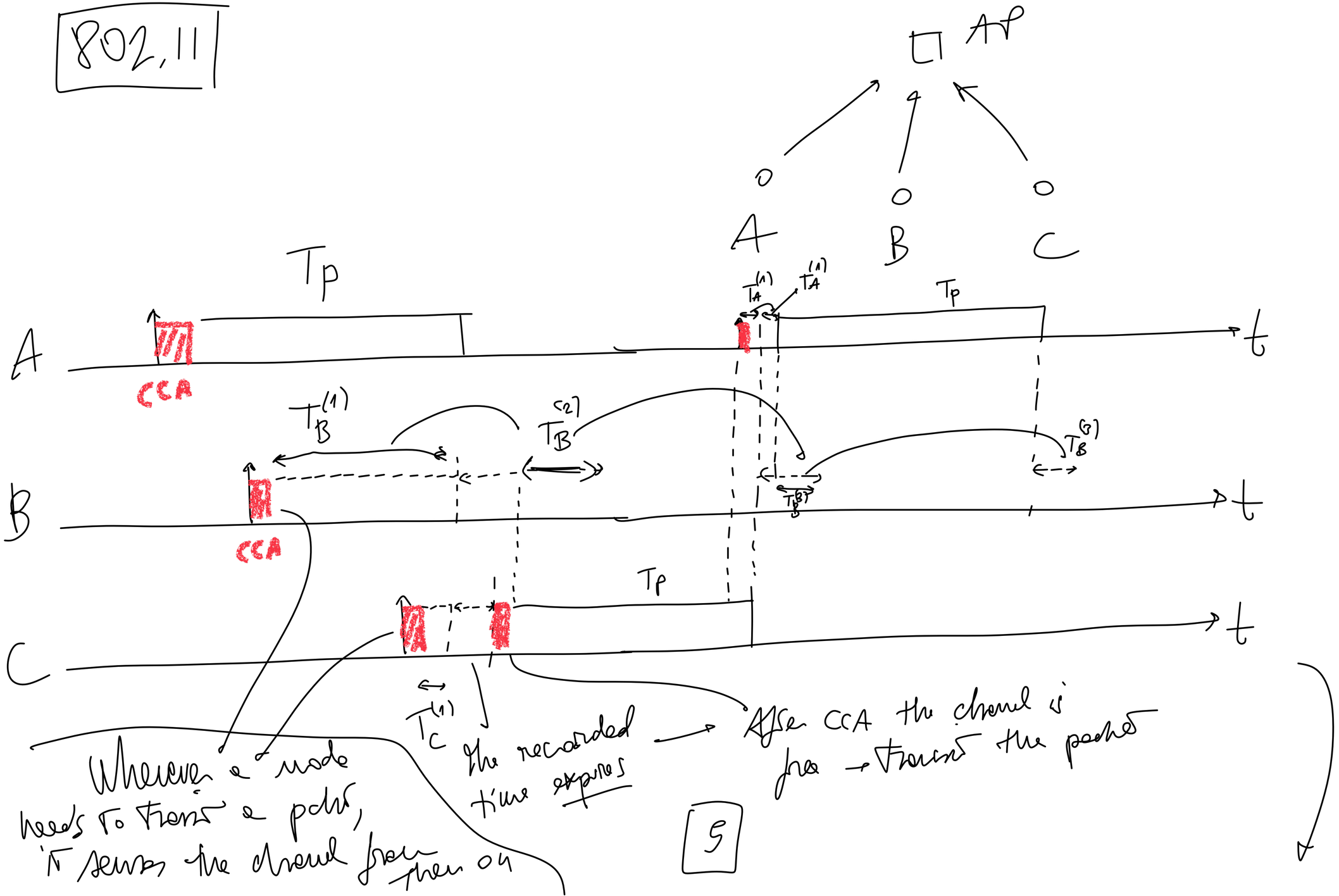
$\{0, \dots, 2^{BE_3} - 1\} = \{0, 1, \dots, 7\}$

3

↳ Why is this a Collision Avoidance (CA) mechanism?

Intuitively, if the BE increases, then the set of possible delays increases ⇒ more unlikely that two nodes select the same delay and start transmitting colliding.

802.11


AP
A
B
C
Tp
CCA
CCA
t
t
t
Tp
Tp
Whenever a node needs to transmit a packet, it senses the channel for then on
The recorded time expires
After CCA the channel is free → transmit the packet

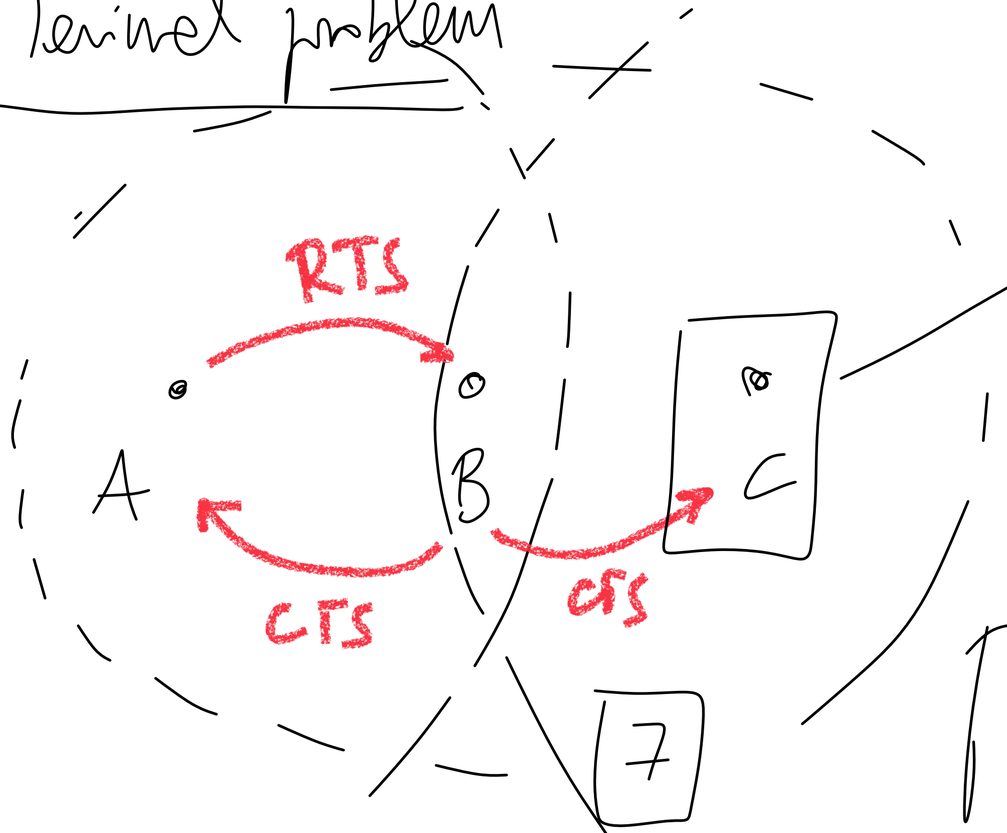
Expert CA mechanism because it is not from (B knows the first one to generate a packet after A started transmitting and is the last one to finish it)

RTS & CTS → Control packets

Ready To send

Clear To send

Hidden Terminal problem

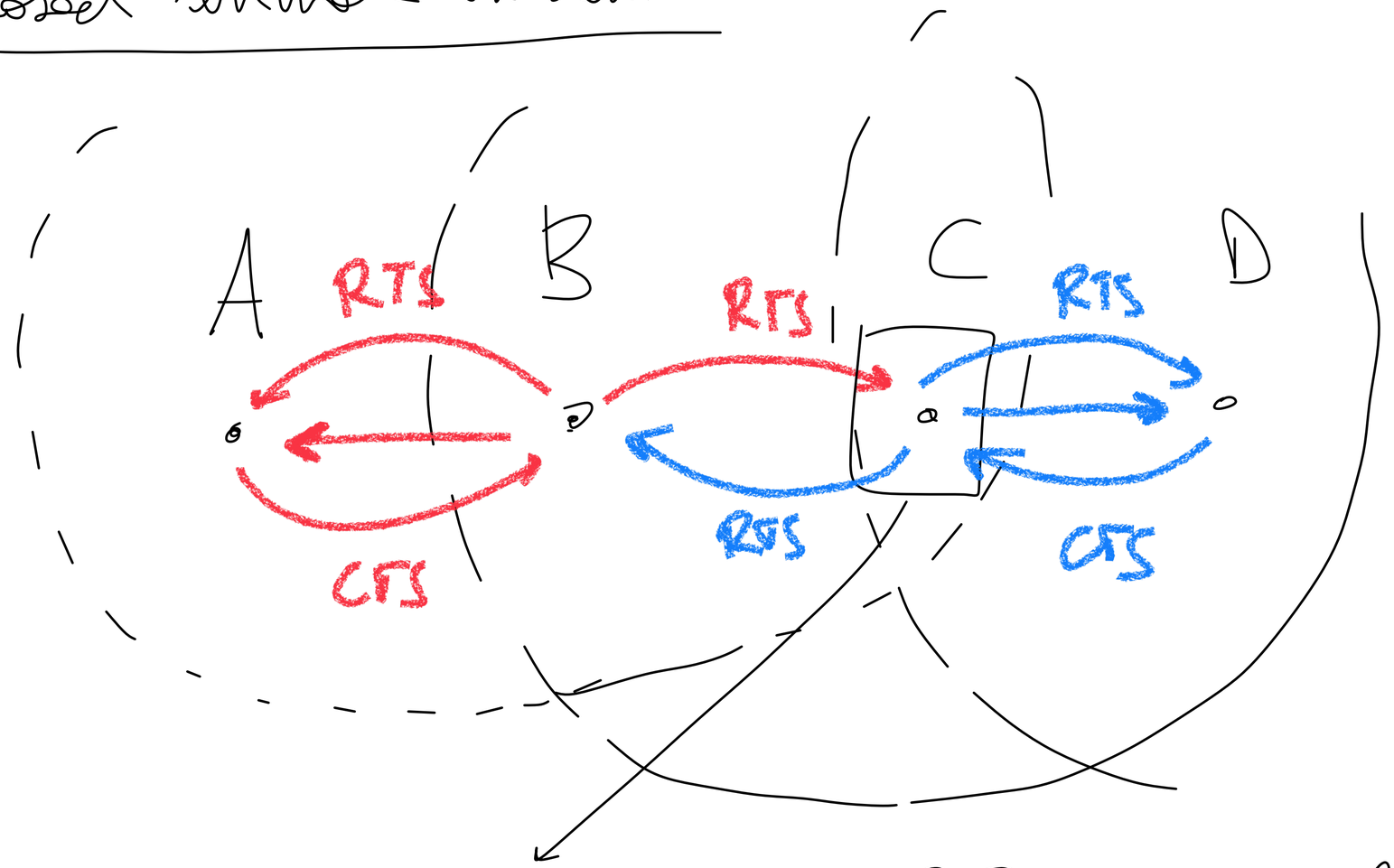


C receives a CTS (addressed To A)

did not receive the RTS

Concludes that it is within the transmission range of receiver

Exposed Terminal Problem

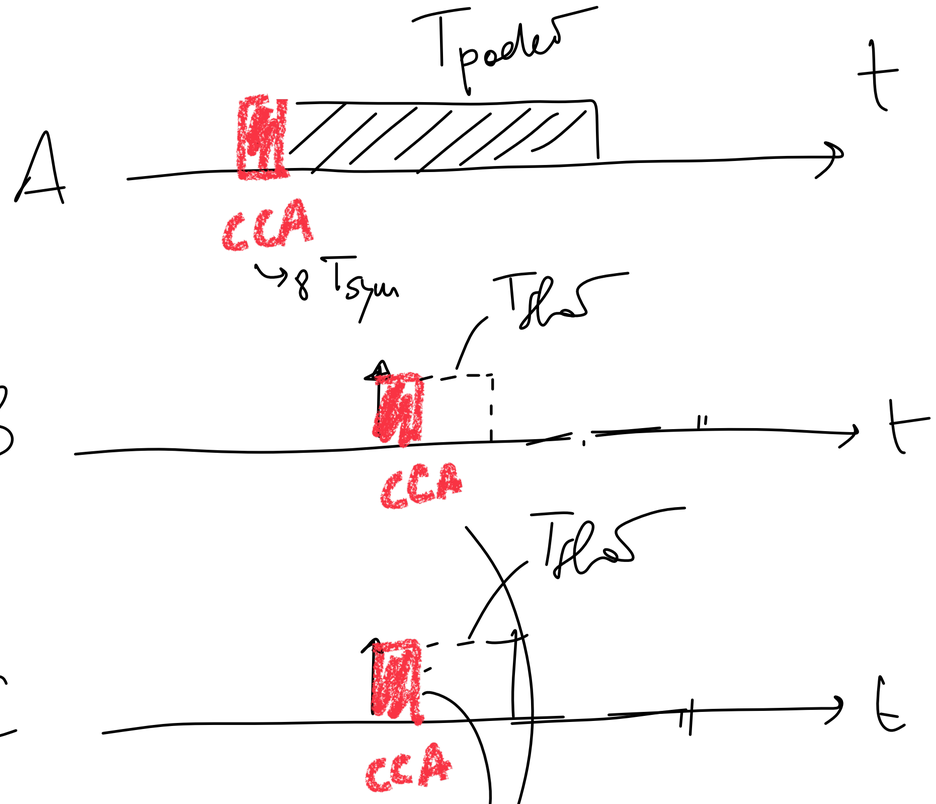


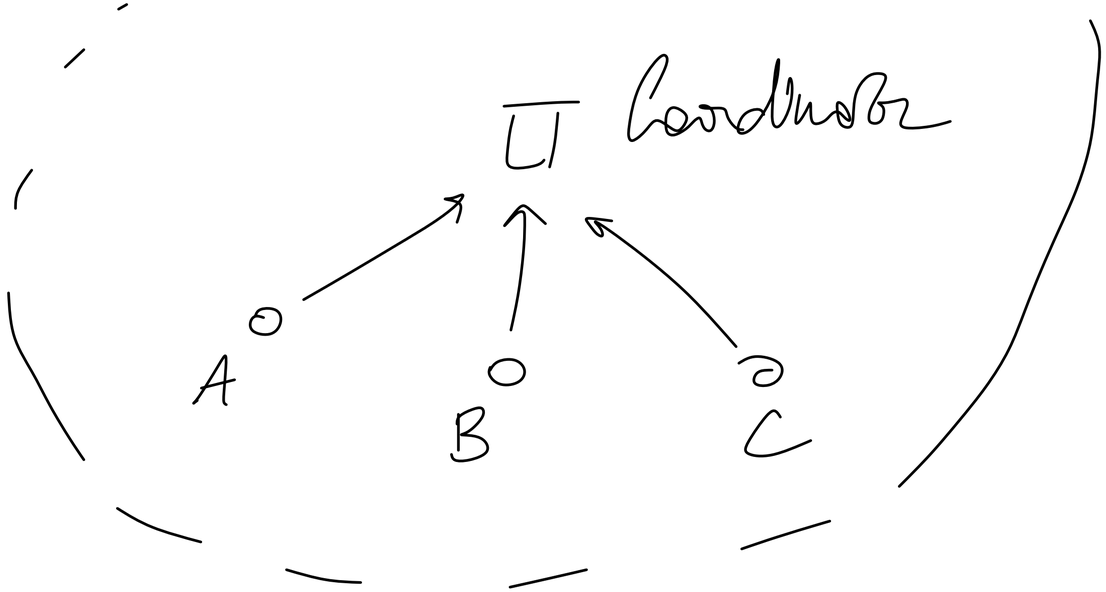
C → receives the RTS
→ did not receive the CTS
⇒ C concludes that it is within the transmission range of the transmitter, but out of the receiver

Can start transmitting to D!

IoT - Lecture 9 - Mar. 25, 2026

802.15.4





CSMA/CA

Before sending a packet, every node senses the channel

$BE_1 = 1$

1) B, C → Wait for a number of T slots randomly selected in the set $\{0, 2^{BE_1} - 1\} = \{0, 1\}$ — CA Mechanism

Denote as $n_{slot}^{(B)}$ and $n_{slot}^{(C)}$ the number of slots

$n_{slot}^{(B)} \in \{0, 1\}$ $n_{slot}^{(C)} \in \{0, 1\}$

After this number of T_{slot} intervals have elapsed → they do CCA again

2) If the channel is sensed busy again

$$BE_2 = BE_1 + 1 = 1 + 1 = 2$$

$$\{0, \dots, 2^{BE_2} - 1\} = \{0, 1, 2, 3\}$$

A node selects randomly a number in this set and waits for $n^{(B)}_{slot} \cdot T_{slot}$ (or $n^{(C)}_{slot} \cdot T_{slot}$).

3) If the channel is sensed busy $\rightarrow$ $BE_3 = BE_2 + 1 = 3$

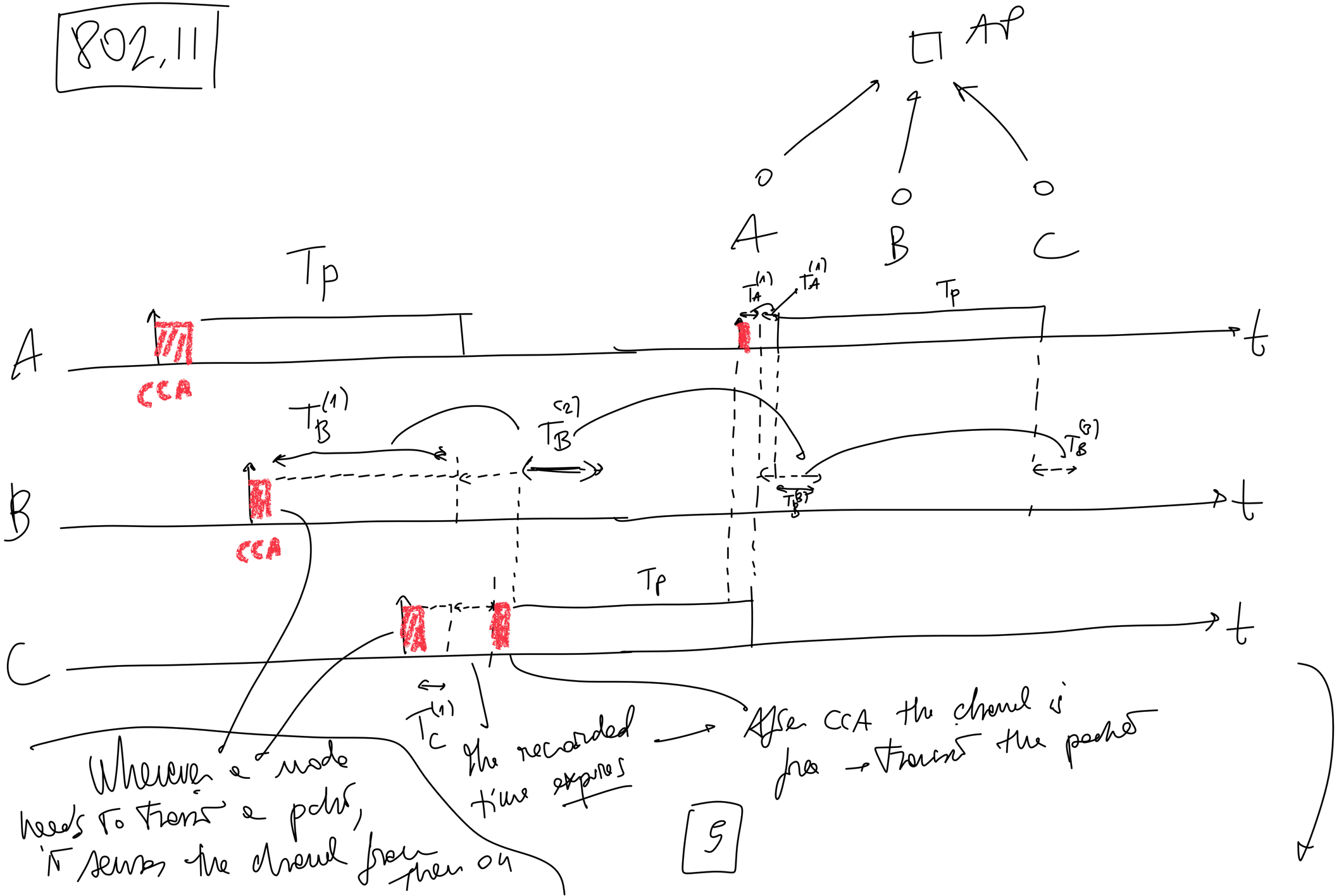
$\{0, \dots, 2^{BE_3} - 1\} = \{0, 1, \dots, 7\}$

3

↳ Why is this a Collision Avoidance (CA) mechanism?

Intuitively, if the BE increases, then the set of possible delays increases ⇒ more unlikely that two nodes select the same delay and start transmitting colliding.

802.11


AP
A
B
C
Tp
CCA
CCA
t
t
t
Tp
Tp
Whenever a node needs to transmit a packet, it senses the channel for then on
The recorded time expires
After CCA the channel is free → transmit the packet

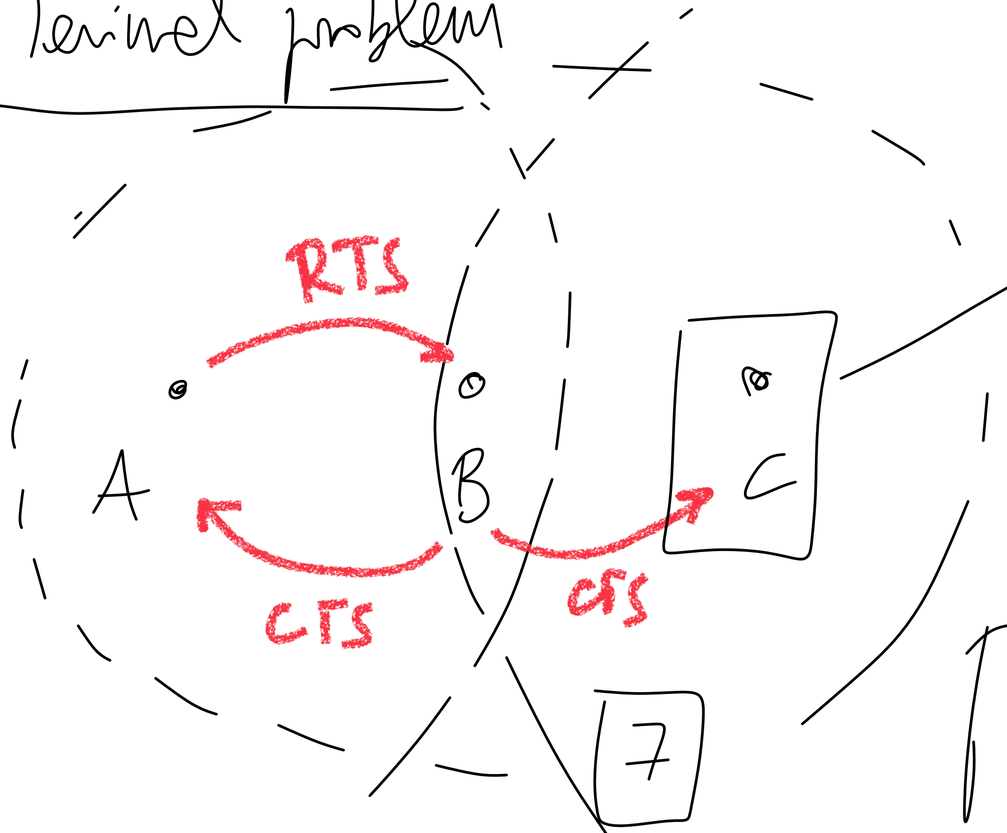
Expert CA mechanism because it is not from (B knows the first one to generate a packet after A started transmitting and is the last one to finish it)

RTS & CTS → Control packets

Ready To send

Clear To send

Hidden Terminal problem

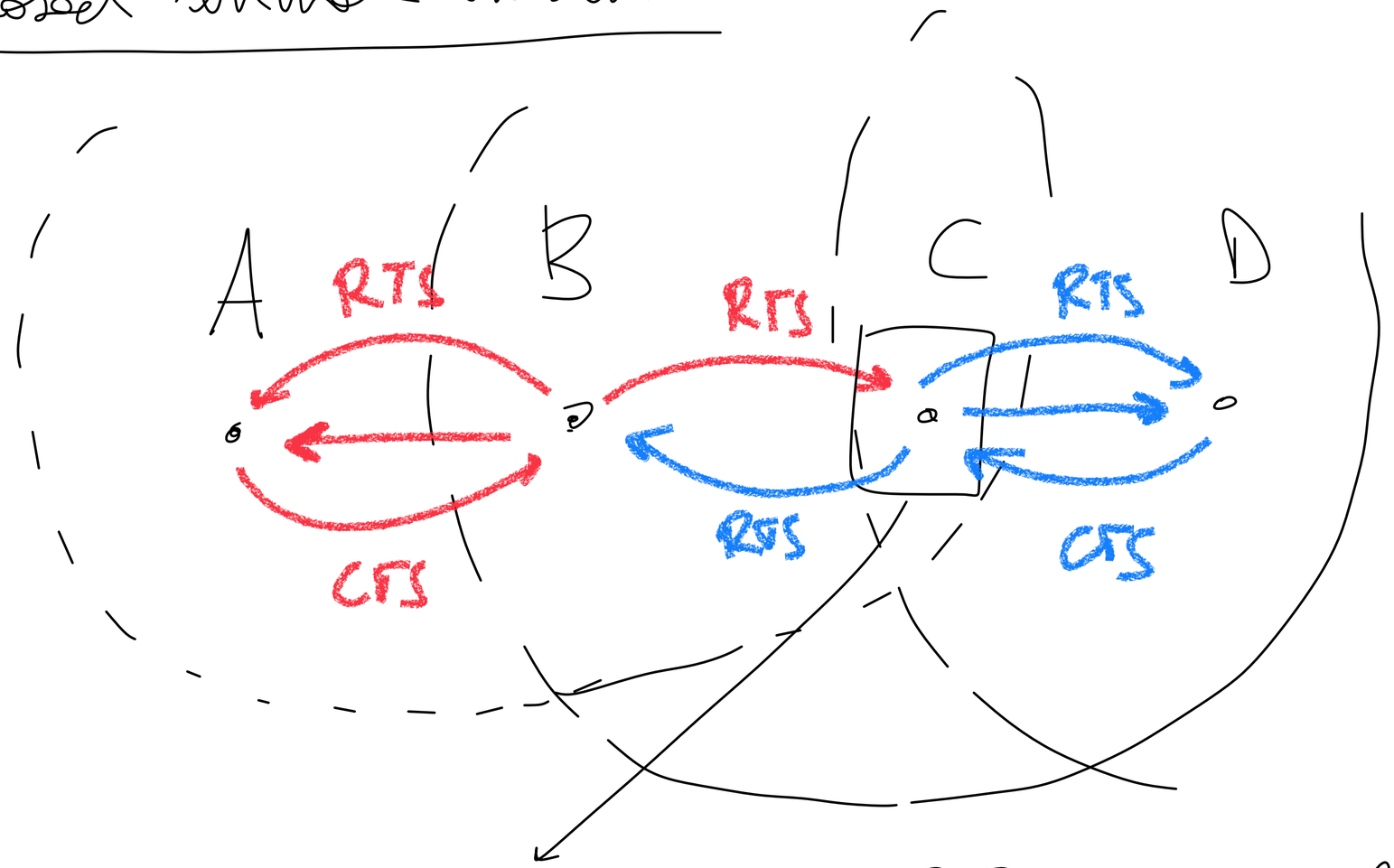


C receives a CTS (addressed To A)

did not receive the RTS

Concludes that it is within the transmission range of receiver

Exposed Terminal Problem

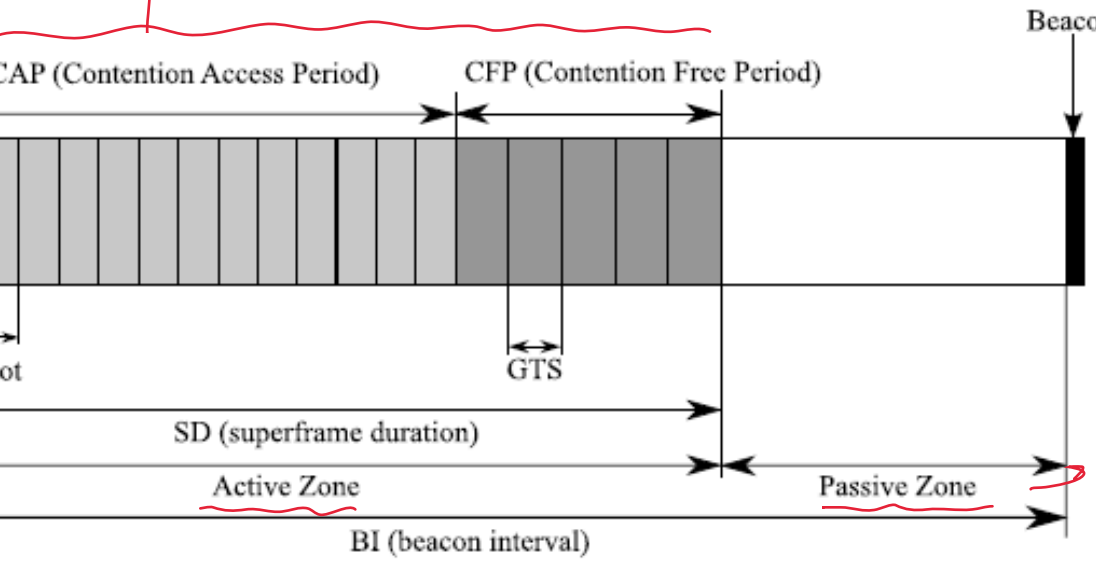


C → receives the RTS
→ did not receive the CTS
⇒ C concludes that it is within the transmission range of the transmitter, but out of the receiver

Can start transmitting to D!

Becon-Enabled Mode

- The use of beacon packets allows to synchronize tne nodes.
- The time is divided into superframes.
- The nodes have, therefore, higher complexity, but duty cycle and channel access are better managed (GTSs).



Beaconless Mode

- No synchronization between nodes.
- Nodes' structures are simpler, but, obviously the performance of beacon-enabled networks is better.
- The MAC protocol differs in terms of time management, synchronization, and packet retransmissions.

CSMA/CA MAC Protocol: Unslotted Version

- The source node waits for a number of slots randomly generated in the interval $[0, 2^{BE_i} - 1]$.
- Once elapsed this time, the node verifies the channel status through a Clear Channel Assessment (CCA), i.e., the energy on the RF interface is measured over $8T_{sym}$.
- If the energy is above a threshold, the channel is busy; otherwise, it is assumed idle.
- If channel is idle, the node can transmit a packet in its queue. Otherwise, NB (number of backoff cycles for this packet) and m (maximum number of backoff cycles, denoted as backoff stage) are compared.
- If $NB = m$, the CSMA/CA is declared as failed. Otherwise, BE is incremented by 1. If $BE_i < BE_{max}$, the procedure is repeated.

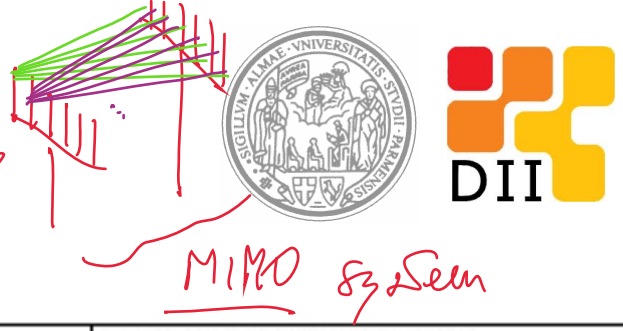
CSMA/CA MAC Protocol: Slotted Version

- The main difference is that all operations must be performed in the correspondence of a backoff boundary.
- CCAs of different nodes can collide. Therefore, two CCAs should be performed to prevent from collisions. However, this leads to waste in channel utilization.
- Obviously, in all cases collisions are still possible!!!

IEEE 802.11: Overview

- IEEE 802.11: wireless communication standard originally designed (late 90's) as a high-speed, short-range communication mechanism for laptops and general purpose PCs.
- WiFi is a brand name of the WiFi Alliance: identify equipment and software that is compatible with other WiFi and 802.11 systems
- Number of 802.11 devices worldwide: billions
- The widespread adoption of 802.11 makes deployment of smart objects easy
- Availability of 802.11 chipsets, routers, and network access cards reduces the cost of hardware for 802.11-enabled smart objects.
- Power consumption: typically one order of magnitude higher than that of 802.15.4 devices → new generation of low-power 802.11 transceivers (significant improvement of power consumption during sleeping periods and low power mode: turn off most of the functions, but resume quickly)

IEEE 802.11: standards



IEEE 802.11	Release Date	Frequency (GHz)	Max Data Rate (Mbps)	Range (m)		Status and Comments
				Indoor	Outdoor	
-1997	1997	2.4	2	20	100	Obsolete
a	1999	5/3.7	54	35/-	120/5k	Legacy systems
b	1999	2.4	11	35	140	Legacy systems
g	2003	2.4	54	38	140	Legacy systems
n	2009	2.4/5	600	70	250	Current systems
ac	2013	2.4/5	450/7,000	35		Next generation – just starting to be deployed.
ad	2012	60	7,000	10		Known as WiGig. Short-reach high data rate data transfers.
af	Est. 2016	0.470-0.710	568		6,000	Being called White-Fi because it uses unused TV spectrum.
ah		0.9	40			In development.
aj		45/60	7,000	10		Modification of 802.11ad for 45GHz band for use in China
ax	Est. 2019	2.4/5	450/10,000	35		Revision to 802.11ac to increase efficiency.

IEEE 802.11: foci of the standards

Task Group	Purpose	Status
PHY	Develop Three Physical Layer Options at 1–2 Mbps	Standard
MAC	Develop One Common MAC Layer	Standard
802.11a	High-Speed (54 Mbps) Physical Layer Extension in the 5.8 GHz Band	Standard
802.11b	High-Speed (11 Mbps) Physical Layer in the 2.4 GHz Band	Standard
802.11e	MAC Enhancements for QoS	Standard
802.11g	Even Higher Speed (54 Mbps) Physical Layer Extension in the 2.4 GHz Band	Standard
802.11i	MAC Enhancements for Enhanced Security	Standard
802.11n	Very High-Speed (100 Mbps) Physical Layer	Draft
802.11r	Mobility Handoffs	Draft
802.11s	Mesh Networks	Draft

IEEE 802.11: Low-Power

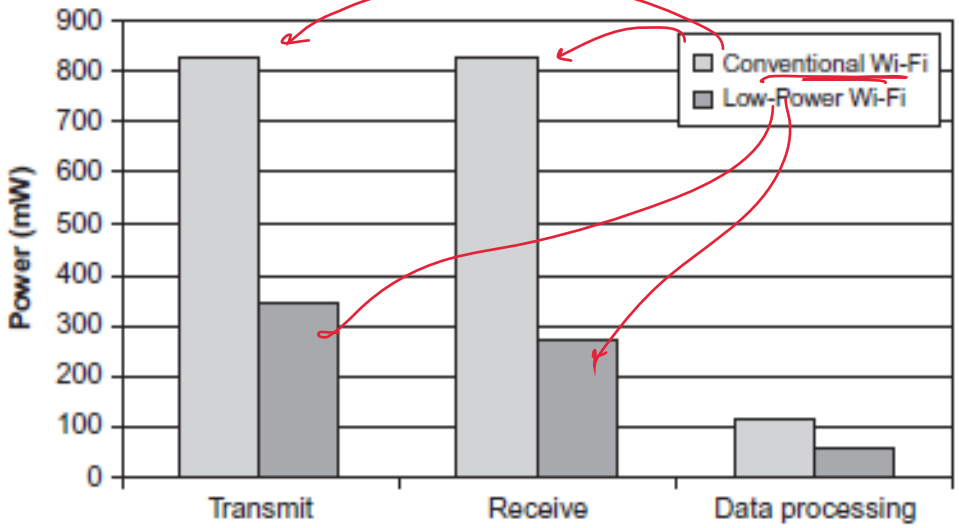


FIGURE 12.15

Comparison of power consumption in transmit, receive, and processing mode for conventional 802.11 and low-power 802.11.

Table 12.1 Power Consumption of Standby and Sleep Modes of a Conventional 802.11 Transceiver and of a Low-power 802.11 Transceiver

	Conventional 802.11 (mW)	Low-power 802.11 (mW)
Standby mode	N/A	0.018
Sleep mode	13	0.2

Note: Conventional 802.11 transceivers do not have a standby mode.

IEEE 802.11: network topology

- Two operational modes: infrastructure (access points) and ad-hoc (direct communications)

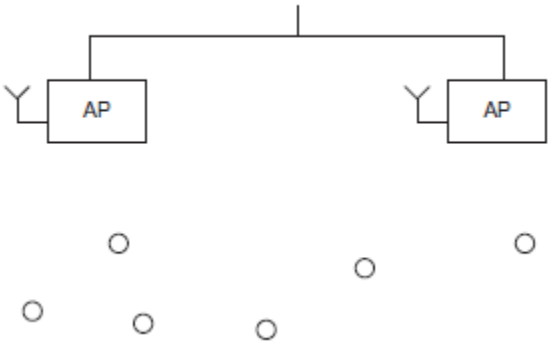


FIGURE 12.11
A network topology of 802.11 in access point mode. Each 802.11 transceiver is associated with an access point. A network may be served by more than one access point. The access points typically have wired network access.

- A node must associate itself with the AP → a group of nodes with an AP is called Basic Service Set (BSS) → with more than one AP, Extended Service Set (ESS) → Service Set IDentifier (SSID)
  - Node probes for a known or unknown SSID
  - AP replies with a probe response packet, with configuration parameters
  - If the network is ok, the node sends an authentication request frame
  - AP replies with association reply frame

IEEE 802.11: PHY layer

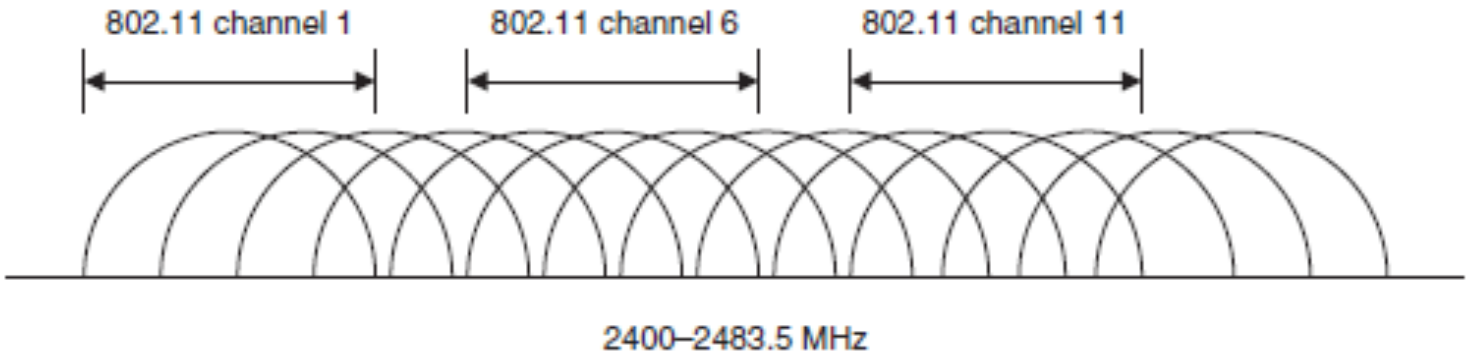


FIGURE 12.12

IEEE 802.11 operates on 14 different channels in the 2.4 GHz band. Channels 1, 6, and 11 do not overlap. Channels 12, 13, and 14 are not available in every country due to radio frequency licensing restrictions.

- Each 802.11 packet is sent with a fixed bit rate, but the bit rate may differ between packets (higher rate with a good communication channel)
- Broadcast packets are sent with a lower bit rate than packets addressed to a specific host to be compatible with older versions

IEEE 802.11: MAC layer

- MAC layer of 802.11: based on Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)

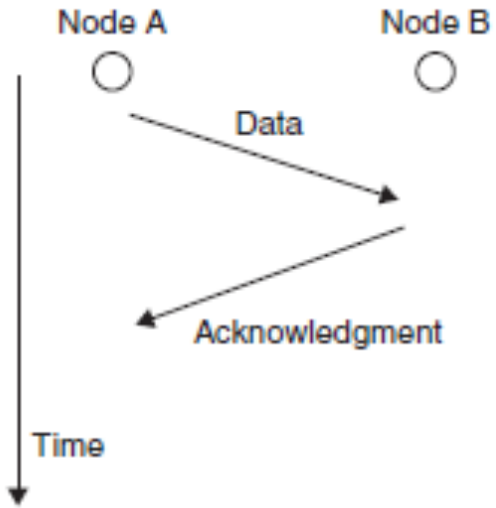


FIGURE 12.13

The IEEE 802.11 MAC layer uses acknowledgment packets that inform the sender that a packet was successfully received by the receiver.

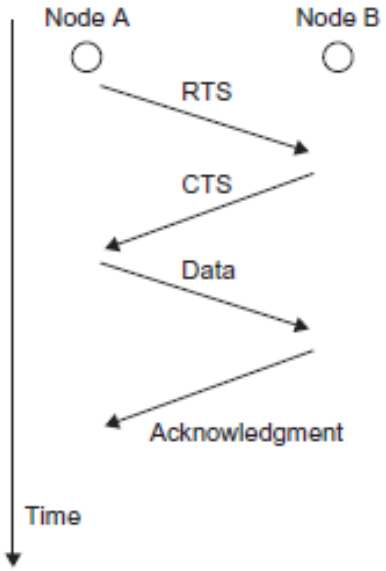


FIGURE 12.14

The RTS/CTS mechanism of the 802.11 MAC layer. To send a packet, a node first sends an RTS packet. If the access point grants the request, it sends a CTS packet and the node can send its packet.

- Mechanism of ready-to-send (RTS) and clear-to-send (CTS) solves the hidden and exposed terminal problems (see later)

PLC: a Quick Overview (1/2)

- Power line communications: eletrical lines are everywhere
- PLC: provides smart objects with data and energy (no need for batteries)

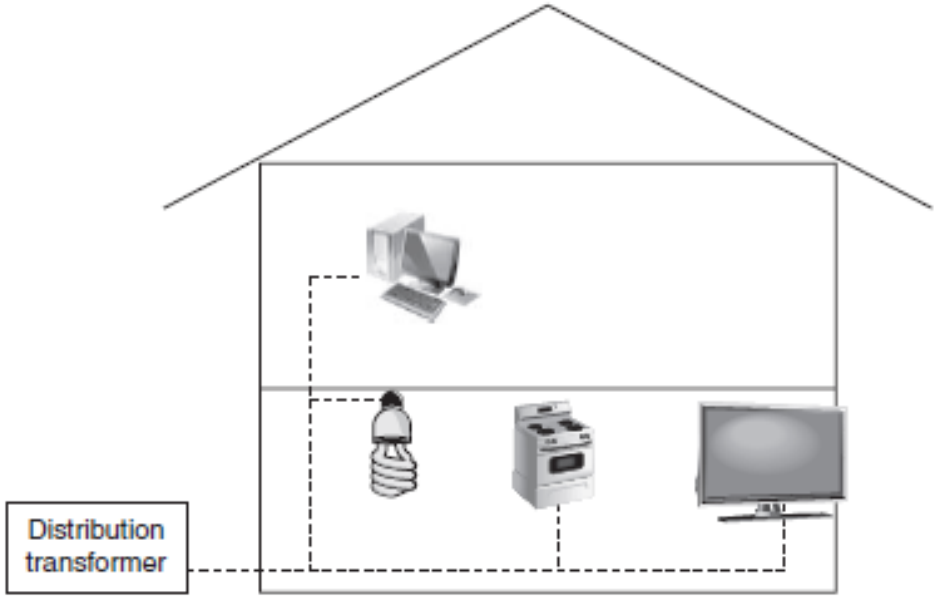


FIGURE 12.16

PLC network within a home.

PLC: a Quick Overview (2/2)

- Data transported across the 50 Hz or 60 Hz electrical distribution network: electrical equipment may add a significant noise
- PLC for short-range communication: only within one electrical domain such as a home or an office building (modulated data signal cannot traverse the electrical voltage transformers outside the electrical domain)
- PHY: copper wires (carry high-power AC signal → noisy; add interferences from plugged-in devices) → very robust communication mechanisms → data rates from kb/s to Mb/s
- MAC: CSMA/CA (unlike transceivers for Ethernet cables, a node transmitting over an electrical line cannot receive)
- Power savings still important: (i) reduce overheating and (ii) avoid compromising overall power savings

UNIVERSITÀ DI PARMA

IoT Lab

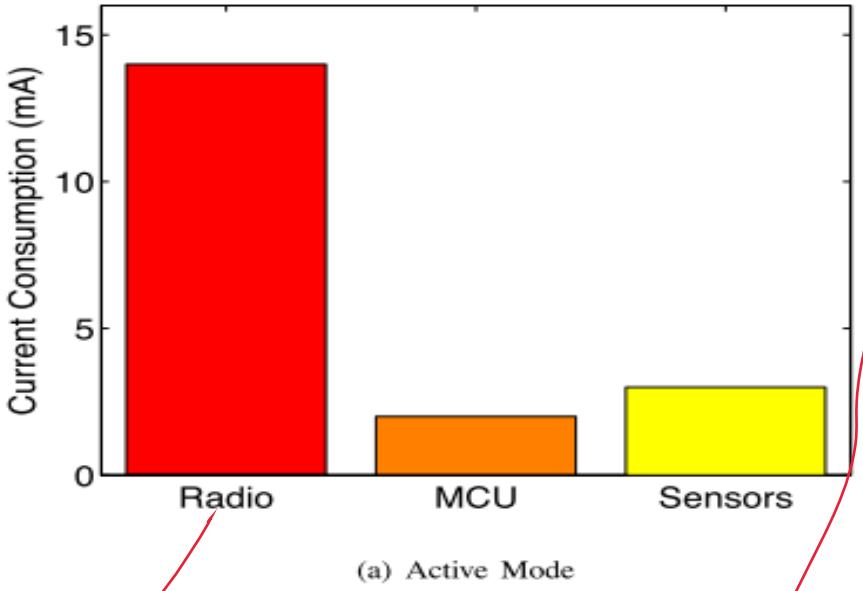
Energy-efficient MAC Protocols

Main sources

- A. Bachir, M. Dohler, T. Watteyne, K. K. Leung, "MAC Essentials for Wireless Sensor Networks," *IEEE Communications Surveys & Tutorials,* vol.12, no.2, pp.222-248, Second Quarter 2010. doi: 10.1109/SURV.2010.020510.00058
- [MACA] P. Karn,"MACA - a New Channel Access Method for Packet Radio", in ARRL/CRRL Amateur Radio 9tt,~ Computer Networking Conference, pp. 134-140, 1990.
- [PAMAS] Suresh Singh and C. S. Raghavendra. 1998. PAMAS—power aware multi-access protocol with signalling for ad hoc networks. *SIGCOMM Comput. Commun. Rev.* 28, 3 (July 1998), 5-26. DOI=10.1145/293927.293928 http://doi.acm.org/10.1145/293927.293928
- [STEM] C. Schurgers, V. Tsiatsis, S. Ganeriwal, M. Srivastava, "Optimizing sensor networks in the energy-latency-density design space," *IEEE Transactions on Mobile Computing*, vol.1, no.1, pp.70,80, Jan-Mar 2002. doi: 10.1109/TMC.2002.1011060
- [B-MAC] J. Polastre, J. Hill, and D. Culler, "Versatile low power media access for wireless sensor networks." Proc. 2nd international conference on Embedded networked sensor systems (SenSys '04), pp. 95-107, Baltimore, MA, USA November 2014. doi>10.1145/1031495.1031508
- [S-MAC] W. Ye, J. Heidemann, D. Estrin, "An energy-efficient MAC protocol for wireless sensor networks," *Proc. 21st Annual Joint Conference of the IEEE Computer and Communications Societies* (INFOCOM 2002) , pp.1567-1576, 2002. doi: 10.1109/INFCOM.2002.1019408
- [X-MAC] Michael Buettner, Gary V. Yee, Eric Anderson, and Richard Han. 2006. X-MAC: a short preamble MAC protocol for duty-cycled wireless sensor networks. In *Proceedings of the 4th international conference on Embedded networked sensor systems* (SenSys '06). ACM, New York, NY, USA, 307-320. DOI=10.1145/1182807.1182838
- [ContikiMAC] A. Dunkels, "The ContikiMAC Radio Duty Cycling Protocol", SICS Technical Report T2011:13. ISSN 1100-3154. December 2011

Power consumption

- Power consumption is one of the biggest challenges because:
  - nodes typically have a limited source of energy (finite battery);
  - battery difficult to replace (and to waste).



(a) Active Mode

Radio mode	Power consumption (mW)
Transmit (T_x)	14.88
Receive (R_x)	12.50
Idle	12.36
Sleep	0.016

3 orders of magnitude reduction

WeMee — Transmission, reception, idle listening

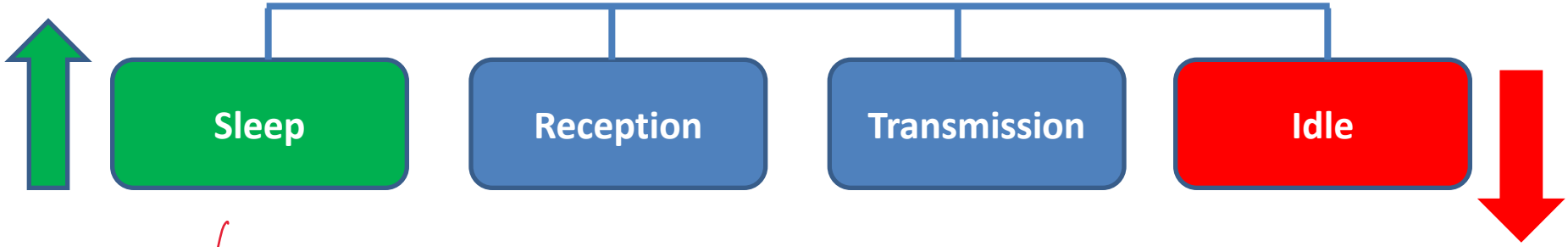
Sources of power consumption

- Useful power consumption:
  - Transmitting or receiving data.
  - Processing queries requests.
  - Forwarding queries and data to the neighbours.
- Wasteful power consumption:
  - Idle listening to the channel "waiting for possible traffic".
  - Retransmitting because of collisions → Collision Avoidance is very important
  - Overhearing: a node receives (and processes) a packet which was not destined to it.
  - Generating and handling control packets (overhead).

Power consumption in WSNs

How to minimize the energy consumption of sensor nodes while meeting the application requirements?

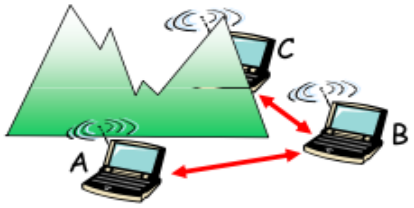
- Use Protocols that aim mainly to increase the sleep periods as much as possible



↳ The typical sleeping portion considered for IoT systems may go up to 99.9%

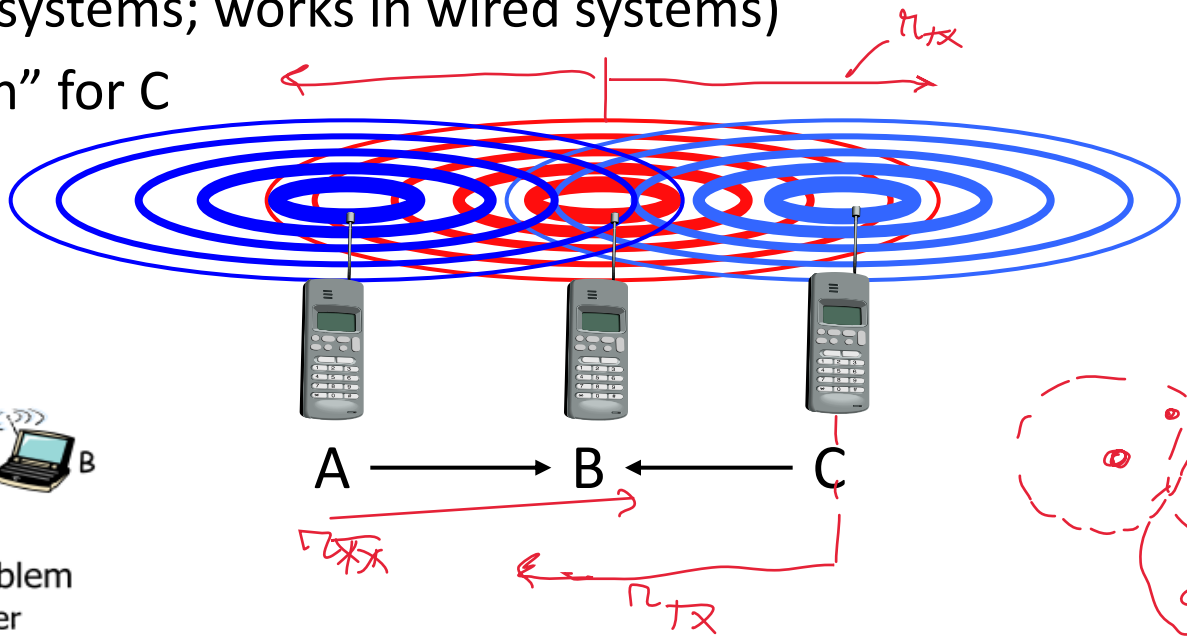
Hidden Terminal Problem

- A sends to B, C cannot receive A
- C wants to send to B, C senses a “free” medium (CS=carrier sensing fails)
- collision at B, A cannot receive the collision (CD=collision deteciont fails in wireless systems; works in wired systems)
- A is “hidden” for C



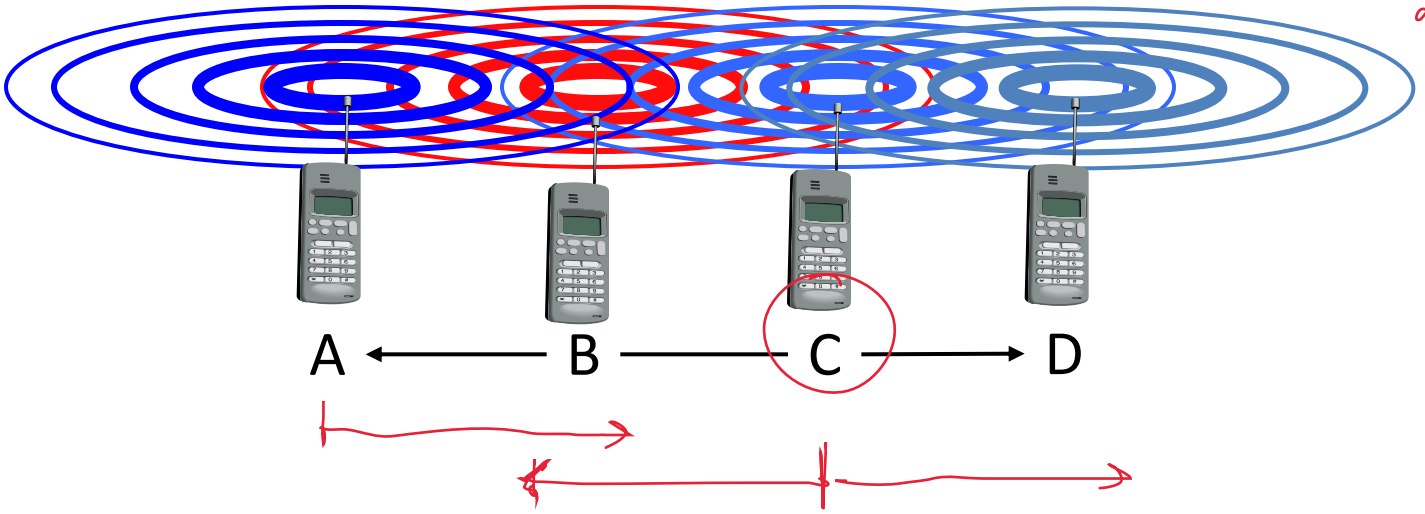
Hidden terminal problem
- B, A hear each other
- B, C hear each other
- A, C can not hear each other

means A, C unaware of their interference at B



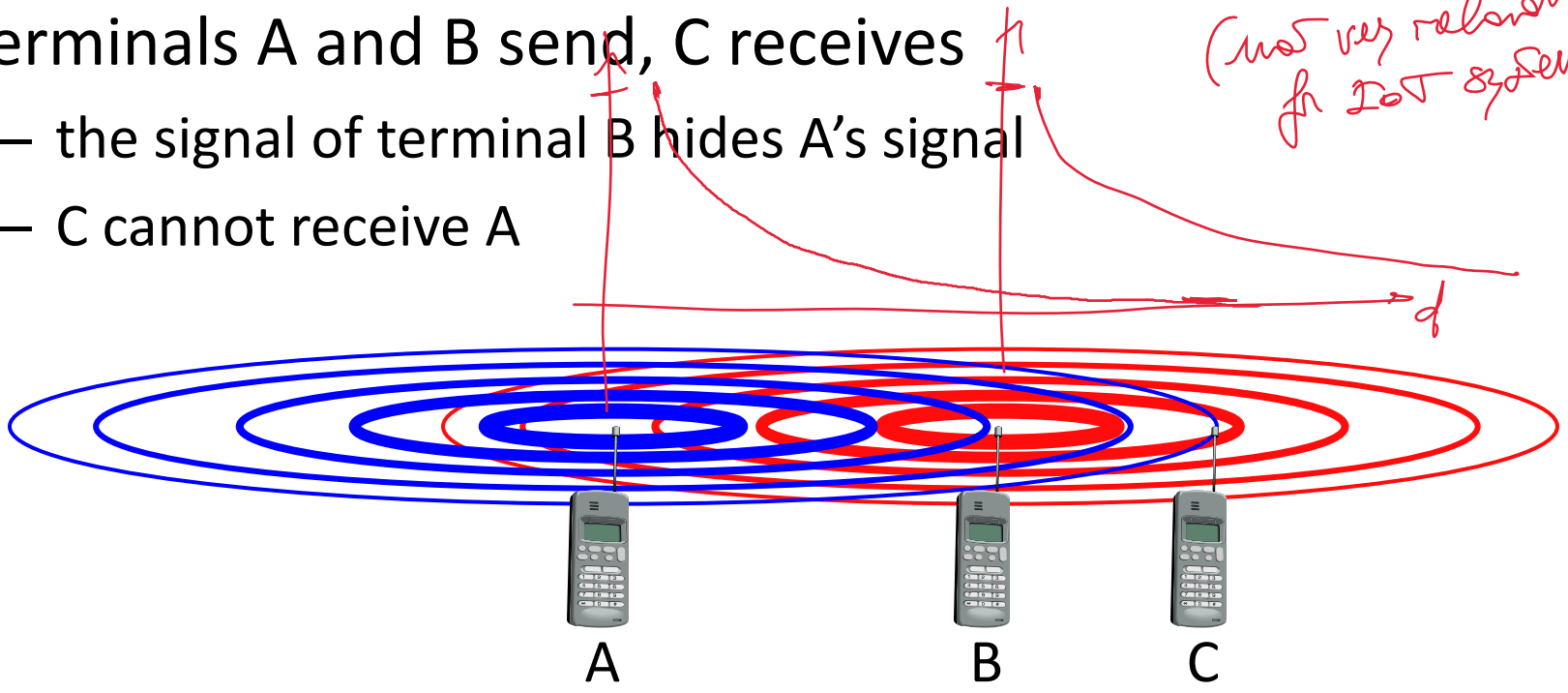
Exposed Terminal Problem

- B sends to A, C wants to send to D
- C has to wait, as it senses a medium in use
- since A is outside the radio range of C waiting is not necessary
- C is "exposed" to B → C is prevented from transmitting even if its transmission would not damage B→A.



Near-Far Problem

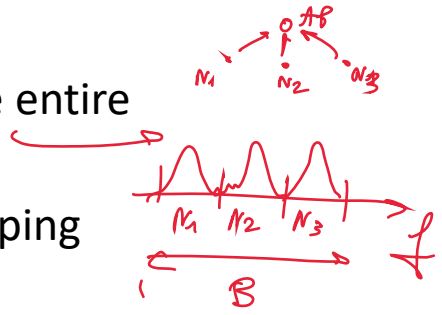
- Terminals A and B send, C receives
  - the signal of terminal B hides A's signal
  - C cannot receive A



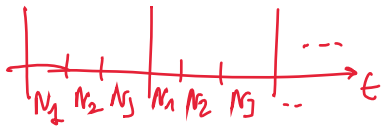
- This is also a severe problem for CDMA networks
- Precise power control required

Centralized Access Methods (Cellular)

- SDMA (Space Division Multiple Access)
  - Segment space into sectors, use directional antennas
  - Use cells to reuse frequencies
- FDMA (Frequency Division Multiple Access)
  - Assign a certain frequency to a transmission channel for the entire transmission duration
  - Permanent (radio broadcast), slow hopping (GSM), fast hopping (FHSS, Frequency Hopping Spread Spectrum → bluetooth)



- TDMA (Time Division Multiple Access)
  - Assign a fixed sending frequency (possibly the entire spectrum) for a certain amount of time
- CDMA (Code Division Multiple Access)
- Combinations!



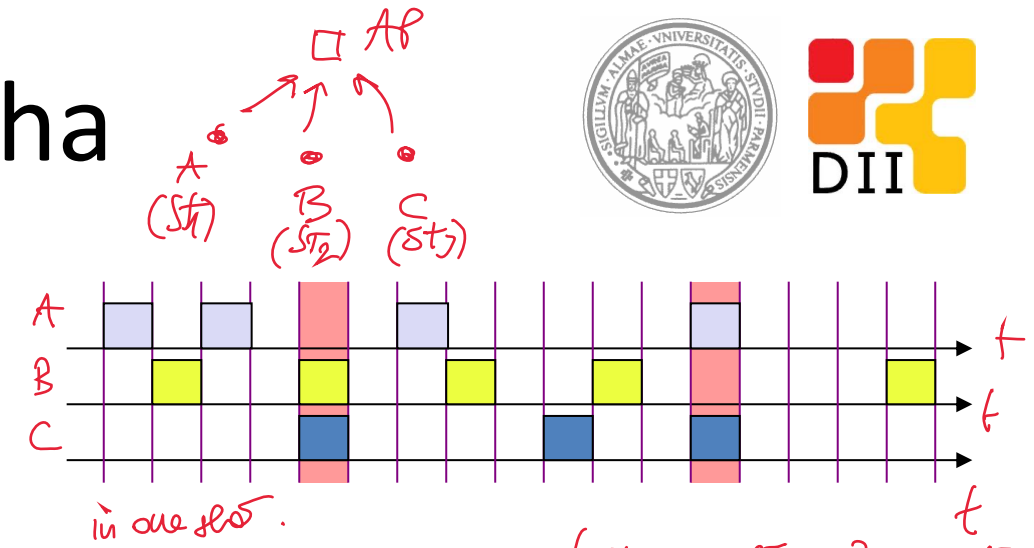
3/4 G

LTE, 5G

Traditional MAC Protocol Classification

- Centralized/Single-Hop Protocols (see previous page)
  - A base station/controller coordinates all traffic (SDMA, FDMA, TDMA, CDMA)
- Contention Protocols (Aloha, Carrier Sense Multiple Access-CSMA)
  - Transmit when you feel like transmitting (Aloha)
  - Retry if collision, try to minimize collisions, additional reservation modes (CSMA)
  - Problem: Receiver must be awake as well
- Scheduling Protocols
  - Use a "pre-computed" schedule to transmit messages
  - Distributed (non-centralized) and adaptive solutions are difficult
- Hybrid protocols
  - E.g. contention with reservation → scheduling (e.g., IEEE 802.15.4 with beacons)
  - Specific ("cross-layer") solutions

Slotted Aloha



- We assume that the stations are perfectly synchronous
- In each time slot each station transmits with probability *p*.

All stations are independent

$$P_1 = \Pr[\text{Station 1 succeeds}] = p(1-p)^{n-1}$$

$$P = \Pr[\text{any Station succeeds}] = nP_1$$

$$\text{maximize } P: \frac{dP}{dp} = n(1-p)^{n-2}(1-pn) \overset{!}{=} 0 \Rightarrow pn = 1 \Rightarrow p^* = \frac{1}{n}$$

$$\text{then, } P = (1-\frac{1}{n})^{n-1} \geq \frac{1}{e}$$

$P = n \cdot p \cdot (1-p)^{n-1}$, $p \in (0,1)$

$P(p^*) = \frac{1}{n}(1-\frac{1}{n})^{n-1}$

$P\{\{St_1 \text{ succeeds}\} \cup \{St_2 \text{ succeeds}\} \cdots \cup \{St_n \text{ succeeds}\}\} = n \cdot P_1$

disjoint events

$P\{St_1 \text{ Tx}\} \cap \{St_2 \text{ does not Tx}\} \cdots \cap \{St_n \text{ does not Tx}\}$

$= P\{St_1 \text{ Tx}\} \cdot P\{St_2 \text{ does not Tx}\} \cdots P\{St_n \text{ does not Tx}\}$ (independence)

$= p \cdot (1-p) \cdots (1-p)$

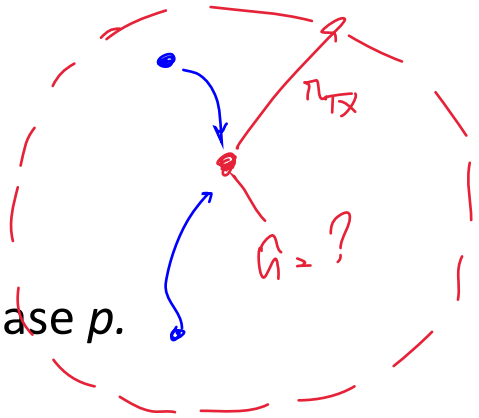
- In Slotted Aloha, a station can transmit successfully with probability at least 1/*e*, i.e., about 36% of the time.
- In pure Aloha (unslotted), a station can transmit successfully with probability at least 1/2e, i.e., about 18% of the time.

$(1-\frac{1}{n})^{n-1} \to e^{-1}$ as $n \to \infty$

$e^{\ln(1-\frac{1}{n})(n-1)} = e^{(n-1)\ln(1-\frac{1}{n})}$, $\ln(1-x) \to -x$ as $x \to 0$

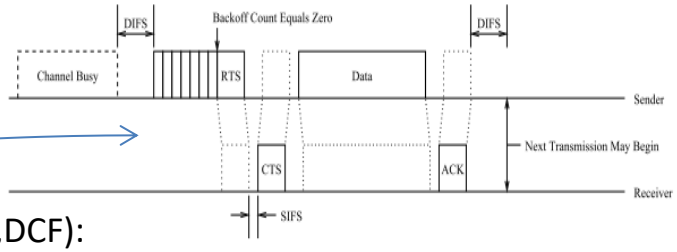
Adaptive Slotted Aloha

- Idea: change the access probability as a function of the number of stations
- How can we estimate the current number of stations in the system?
- Assume that stations can distinguish whether 0, 1, or more than 1 stations transmit in a time slot.
- Model:
  - Number of stations that want to transmit: n.
  - Estimate of n: $\hat{n}$
  - Transmission probability: $p = 1/\hat{n}$
- Idea:
  - If you see that nobody transmits, increase p.
  - If you see that more than one transmits, decrease p.



Carrier Sense Multiple Access (CSMA)

- Listen before transmit: if the channel is free, then transmit
- Various versions:
  - non-persistent: if the device detects activity on the channel, it performs a back off by waiting before attempting to transmit.
  - *p*- persistent: if it detects activity on the channel, it keeps on sensing the channel instead of delaying.
- "Basic" CSMA requires devices to remain in the receive state (not idle) when not transmitting → the transceiver consumes energy too quickly.
- Various variations
  - CSMA/CD: collision detection (Ethernet, IEEE 802.3)
  - CSMA/CA: collision avoidance, LIFO (IEEE 802.15.4, IEEE 802.11-PCF,DCF): combine with Ready-to-send (RTS) and Clear-to-Send (CTS) mechanisms → RTS messages may collide
  - IEEE 802.11 uses Network Allocation Vector (NAV), virtual carrier-sensing mechanism: the MAC frame header contains the packet transmission duration



Backoff

- Backoff protocols rely on acknowledgements.
- Binary exponential backoff:
  - If a packet has collided k times (no ACK for k times), we set $p = 2^{-k}$
  - Or alternatively: wait from random number of slots in $[1,...,2^k]$
- It has been shown that binary exponential backoff is not stable for any arrival rate $\lambda > 0$ if there are infinitely many potential stations
- Interestingly when there is a finite number of stations, binary exponential backoff becomes unstable with $\lambda > 0.568$;
  Polynomial backoff however, remains stable for any $\lambda < 1$.

Network Performance

Moving towards WSNs

- Traditional MAC protocols
  - High throughput
  - Low latency
  - Fairness
  - Mobility
  - Energy? → neglected

- MAC protocols for WSNs:
  - Primary goals
    - Energy conservation
    - Scalability and adaptivity
  - Secondary goals
    - Throughput
    - Latency
    - Fairness

Energy-efficient MAC Protocols

- In WSNs, energy is often more critical than throughput.
  - The radio transceiver should be turned off as much as possible.
- Energy management considerations have a big impact on MAC protocols.
  - Idle listening costs about as much energy as transmitting
- In the following: a few ideas, taken from some known protocols that try to balance throughput and energy consumption:
  - MAC,PAMAS, STEM, B-MAC, X-MAC, WiseMAC, S-MAC, **ContikiMAC**
  - Many of the hundreds of MAC protocols that were proposed have similar ideas…

Unscheduled MAC protocols

- "Evolutions" of CSMA
- Advantages:
  - they can adapt for changes "in the node density, traffic load or the topology" better than scheduled protocols.
  - The nodes do not have to be synchronized together.
- Disadvantages:
  - All nodes need to listen to the channel
- Examples:
  - MACA (1990)
  - PAMAS (1998)
  - STEM (2002)
  - B-MAC (2004), WiseMAC (
  - X-MAC (2006)

MACA (Multiple Access with Collision Avoidance) - 1990

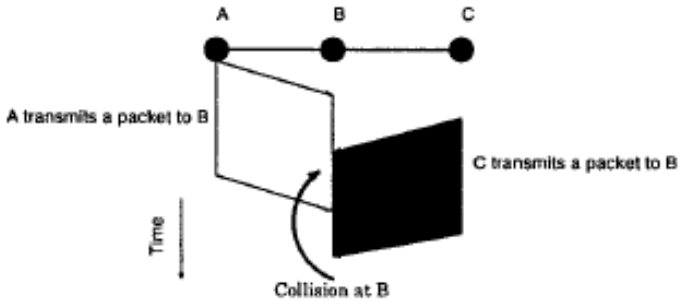
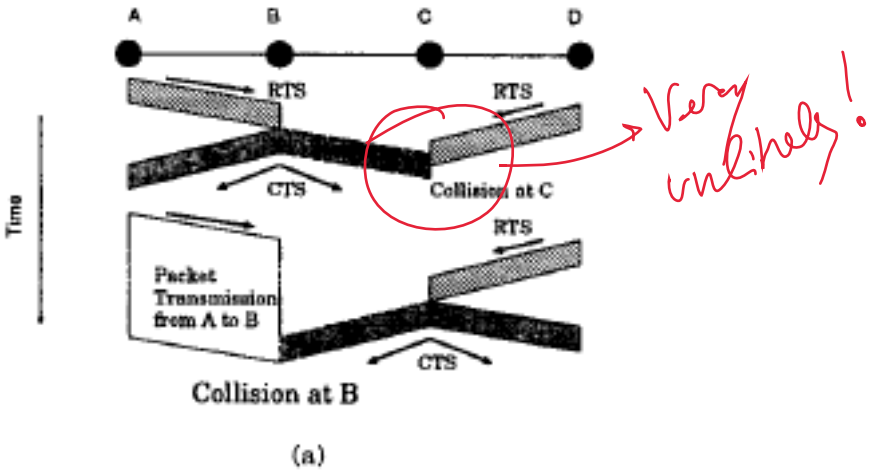


Figure 2: The hidden terminal problem.

- Hidden terminal problem: partially solved with the RTS/CTS mechanism introduced by MACA

Also solves the exposed terminal problem



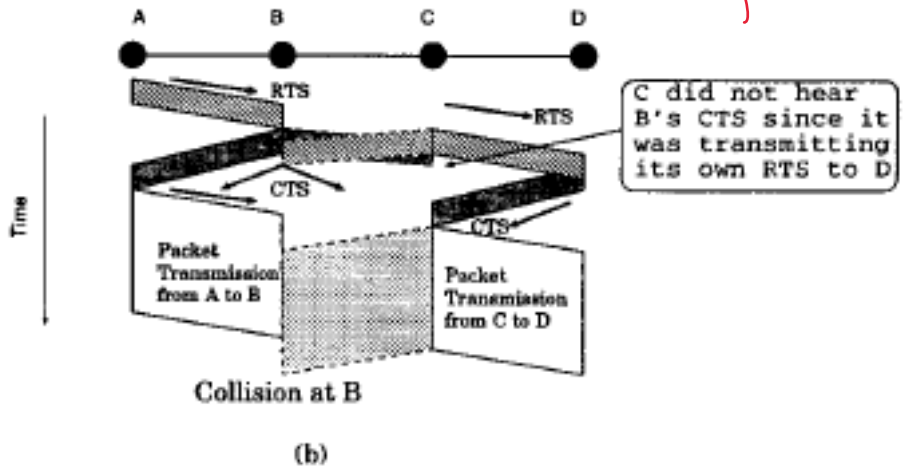


Figure 3: Hidden terminal problem in MACA.

PAMAS (Power Aware Multi-Access protocol with Signaling for Ad-Hoc Networks) - 1998

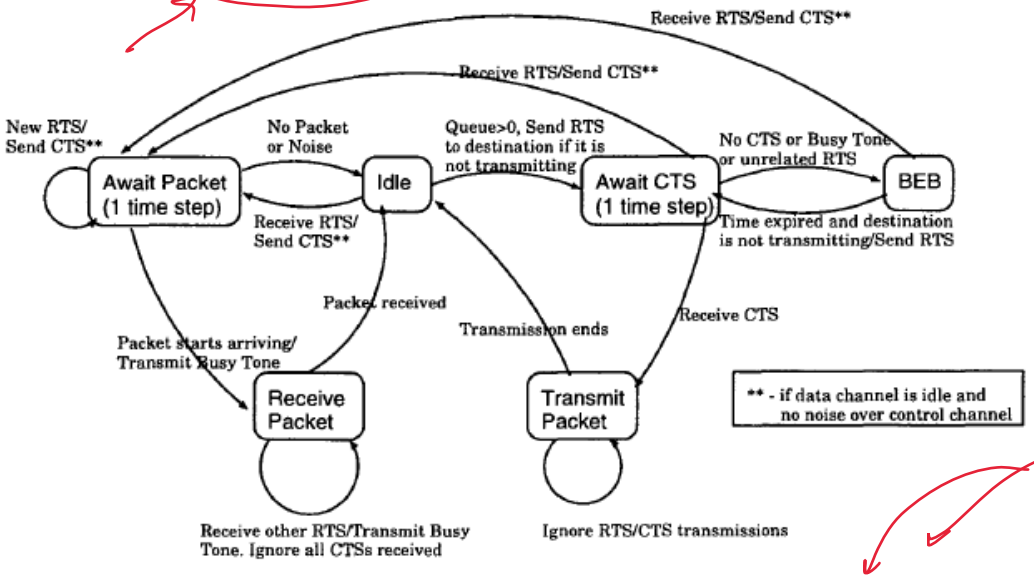
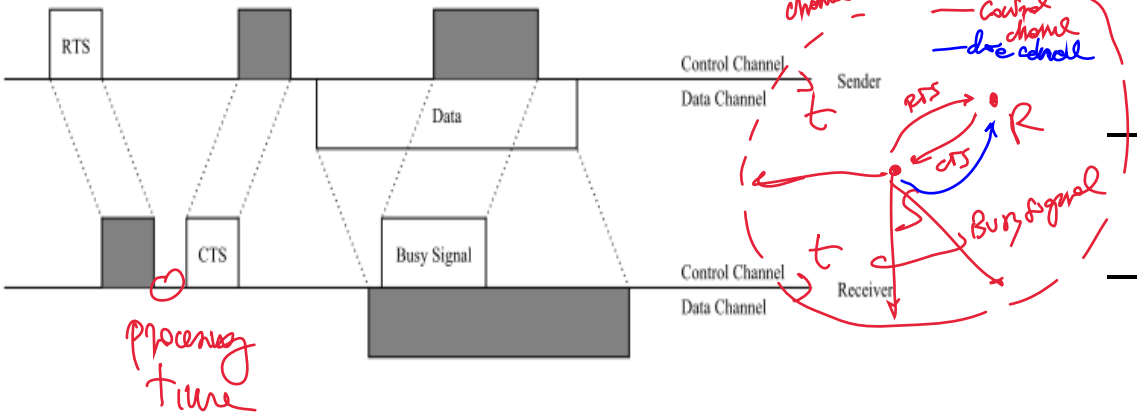


Figure 4: The PAMAS Protocol.



- Basic idea: use a *signaling channel* separated from data channel
- Advantages:
  - Prevent collisions
- Disadvantages:
  - Multiple radio requirement
  - Increase energy consumption
  - Increase device complexity and cost

STEM (Sparse Topology and Energy Management) - 2002

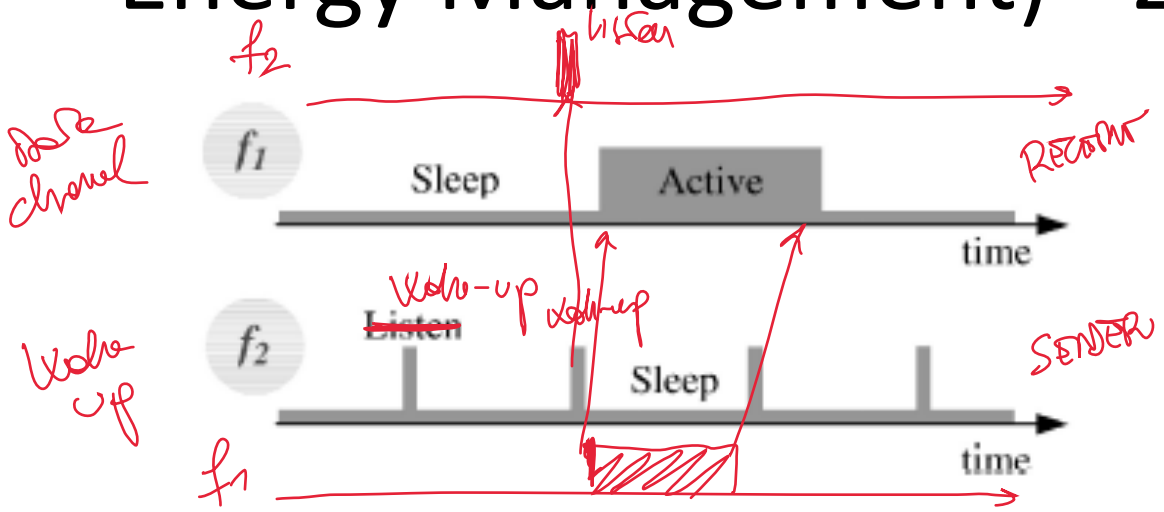


Fig. 3. Separate data and wakeup using two radios.

- Basic idea: uses two different channels: the wakeup channel (f_2) and the data channel (f_1)
- Requires two transceivers in each node

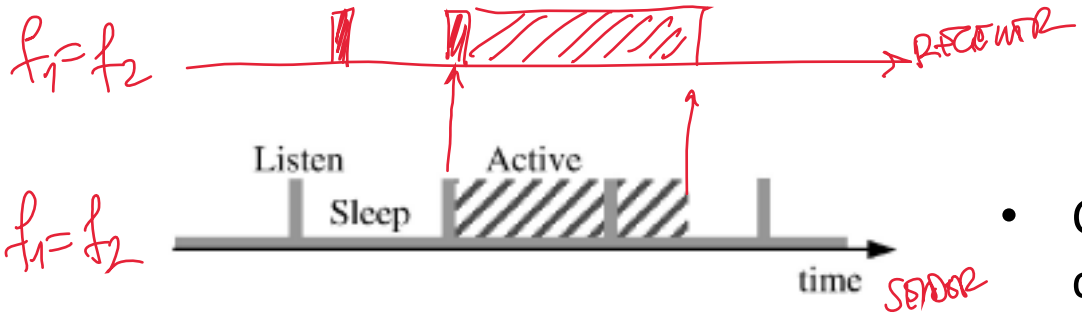


Fig. 4. Separate data and wakeup in time.

- Could "mix" the two channels → synchronization is required

STEM-B and STEM-C - 2002

- STEM-B: a sensor node wakes up a neighbor by transmitting a beacon (no RTS/CTS)
  - Advantages:
    - Lower Latency
  - Disadvantages:
    - More complex
    - High energy consumption
- STEM-T: a sensor node wakes a neighbor by transmitting a busy tone (contains no destination address) of sufficient length so that the destination will have a high probability of sensing
  - Disadvantages:
    - High latency
    - Results in overhearing (stops the transmission from nodes which would not interfere)

"Berkeley MAC" (B-MAC) – 2004

(Versatile low power media access for wireless sensor networks)

- Goals
  - Low power operation
  - Effective collision avoidance
  - Simple implementation (small code)
  - Energy-efficient at both low and high data rates
  - Reconfigurable by upper layers
  - Tolerant to changes on the network
  - Scalable to large number of nodes

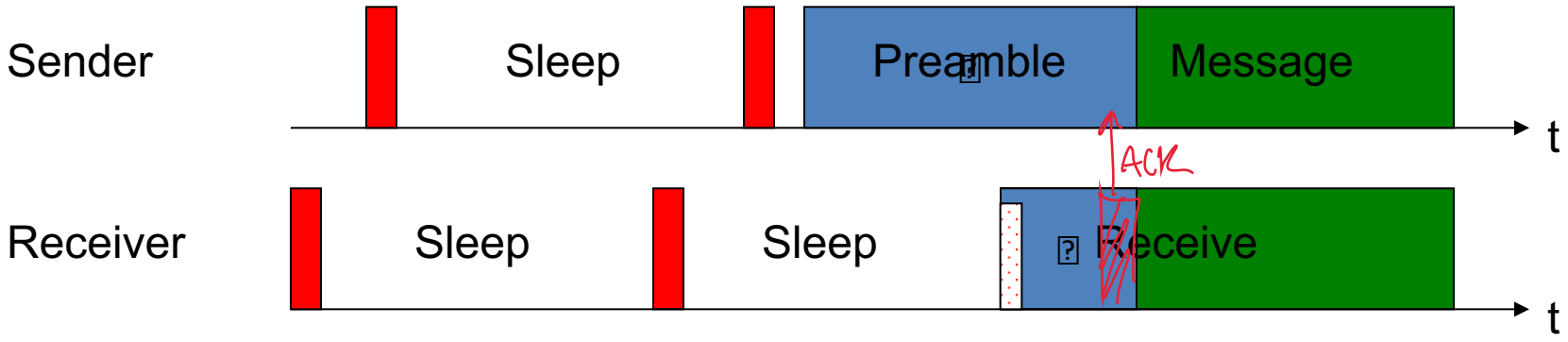
MIRACLE!

- Strategy
  - Nodes wake up for a short period and check for channel activity.
  - Return to sleep if no activity is detected. or if they do not need to transmit.
  - If a sender wants to transmit a message, it sends a long preamble to make sure that the receiver is listening for the packet (STEM-T) → preamble has the size of a sleep interval
- Very robust
  - No synchronization required
  - Instant recovery after channel disruption

B-MAC's Features

- Clear Channel Assessment (CCA)
- Low Power Listening (LPL) using preamble sampling
- Hidden terminal and multi-packet mechanisms not provided: they should be implemented, if needed, by higher layers

- CCA on/off
- Acknowledgements on/off
- Initial and congestion backoff on a per-packet basis
- Configurable check interval and preamble length



B-MAC's Lifetime Model

Operation	Time (s)		I (mA)	
Initialize radio (b)	350E-6	$t_{r_{init}}$	6	$c_{r_{init}}$
Turn on radio (c)	1.5E-3	t_{ron}	1	c_{ron}
Switch to RX/TX (d)	250E-6	$t_{rx/tx}$	15	$c_{rx/tx}$
Time to sample radio (e)	350E-6	t_{sr}	15	c_{sr}
Evaluate radio sample (f)	100E-6	t_{ev}	6	c_{ev}
Receive 1 byte	416E-6	t_{rxb}	15	c_{rxb}
Transmit 1 byte	416E-6	t_{txb}	20	c_{txb}
Sample sensors	1.1	t_{data}	20	c_{data}

Table 2: Time and current consumption (I) for completing primitive operations of a monitoring application using the Mica2 mote and CC1000 transceiver. Identifiers on each operation map back to the activities of acquiring a radio sample in Figure 3.

Notation	Parameter	Default
c_{sleep}	Sleep Current (mA)	0.030
C_{batt}	Capacity of battery (mAh)	2500
V	Voltage	3
$L_{preamble}$	Preamble Length (bytes)	271
L_{packet}	Packet Length (bytes)	36
t_i	Radio Sampling Interval (s)	100E-3
n	Neighborhood Size (nodes)	10
r	Sample Rate (packets/s)	1/300
t_l	Expected Lifetime (s)	-

Table 3: Parameters for a monitoring application running B-MAC. The first three parameters are specific to Mica2 motes; the next three are default values for B-MAC parameters on the Mica2; the remaining parameters are application semantics affecting B-MAC's performance. Each parameter affects the node's total energy consumption, E.

$$t_{tx} = r \times (L_{preamble} + L_{packet}) t_{txb}$$
$$E_{tx} = t_{tx} c_{txb} V$$

$$t_d = t_{data} \times r$$
$$E_d = t_d c_{data} V$$

$$t_{sleep} = 1 - t_{rx} - t_{tx} - t_d - t_{listen}$$
$$E_{sleep} = t_{sleep} c_{sleep} V$$

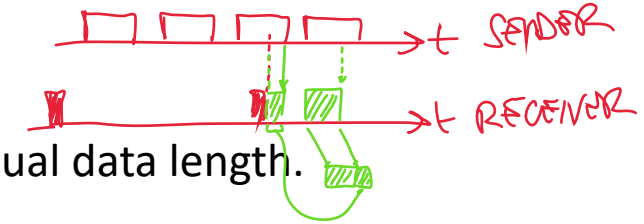
$$E = E_{rx} + E_{tx} + E_{listen} + E_d + E_{sleep}$$

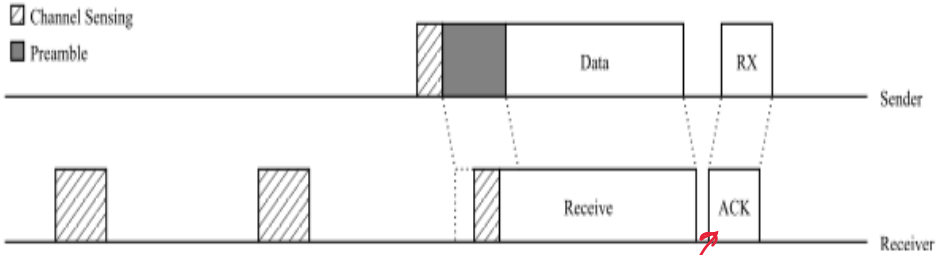
$$t_{rx} \leq nr(L_{preamble} + L_{packet}) t_{rxb}$$
$$E_{rx} = t_{rx} c_{rxb} V$$

$$E_{sample} = 17.3\mu\text{J}$$
$$t_{listen} = (t_{r_{init}} + t_{ron} + t_{rx/tx} + t_{sr}) \times \frac{1}{t_i}$$
$$E_{listen} \leq E_{sample} \times \frac{1}{t_i}$$

B-MAC's Issues

- Problem: All nodes in the vicinity of a sender wake-up and wait for the packet → overhearing
  - Solution 1: Send wake-up packets instead of preamble, wake-up packets tell when data is starting so that receiver can go back to sleep as soon as it received one wake-up packet.
  - Solution 2: Just send data several times such that receiver can tune in at any time and get tail of data first, then head.
- Communication costs are mostly paid by the sender.
  - The preamble length can be much longer than the actual data length.
- Idea: Learn wake-up schedules from neighboring nodes.
  - Start sending preamble just before intended receiver wakes up.
  - **WiseMAC** → encode wake-up pattern in the ACK message



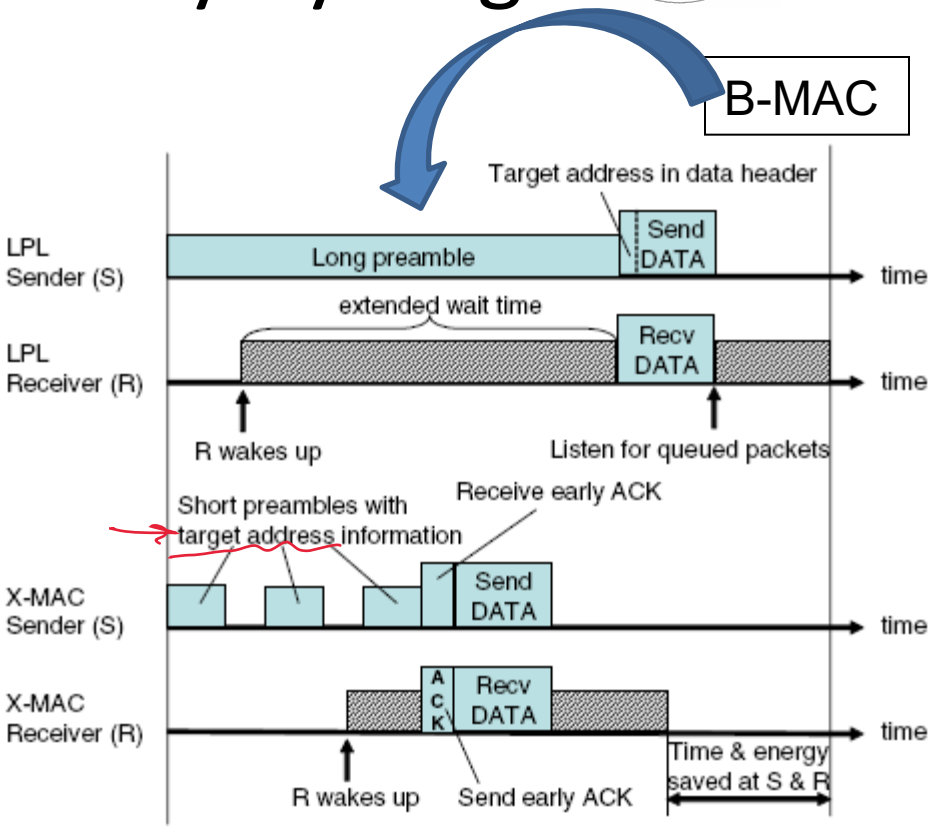


X-MAC: Principles – 2006

- Short preamble
  - Reduce latency and reduce energy consumption
- Target (destination) in preamble
  - Minimize overhearing problem.
- Strobed preamble → the preamble transmission can be interrupted.
  - Reduces latency for the case where destination is awake before preamble completes.
- Reduced per-hop latency and energy
- Dynamic duty-cycling (depending on network conditions)

X-MAC: Asynchronous Duty Cycling

- Overhearing energy loss is proportional to number of receivers in range.
- Short preamble packets.
- Non-receivers return to sleep more quickly.
- **Bad in high density sensor networks** (collisions between strobing packets)



Scheduled MAC protocols

- Idea: try to reduce the energy consumption by *coordinating* sensor nodes with a common schedule
- Advantages:
  - Saving the power from being wasted by turning off the radio outside the allocated time slot.
  - Limits collisions, idle listing, and overhearing
- Disadvantages:
  - When a sensor node enters the network, it must wait till it learns the schedule → some delay exist
  - Cost of increased number of exchanged messages
  - No flexibility to changes in sensor nodes' density or movements.
  - All sensors have to be synchronized.
- Example:
  - S-MAC (2002)
  - ContikiMAC (from 2008 on)

Sensor MAC (S-MAC) - 2002

- Tries to reduce waste of energy from all four sources of energy inefficiency
  1. Collisions – by using RTS and CTS
  2. Overhearing – by switching the radio off when transmission is not meant for that node
  3. Control Overhead – by message passing
  4. Idle listening – by periodic listen and sleep
- At the cost of:
  - some reduction in per-hop fairness and latency
  - but does not reduce end-to-end fairness and latency

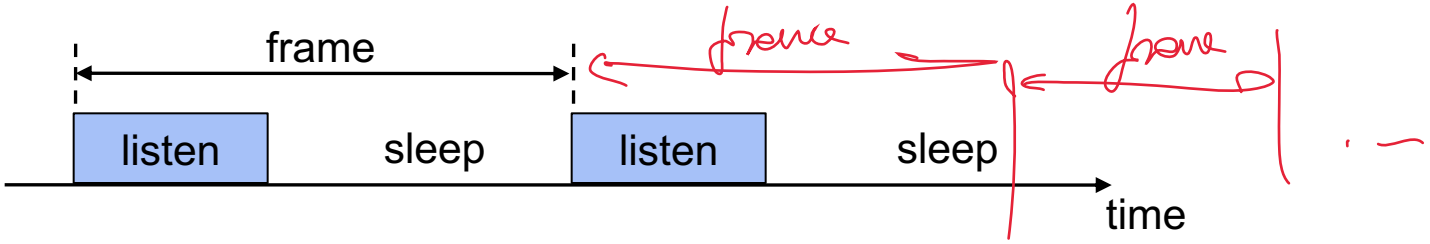
S-MAC: Assumptions

- Network assumptions:
  - Composed of many small nodes deployed in ad hoc fashion (random)
  - Most communications will be between nodes as peers, rather than to/from a single base station
  - Nodes must self-configure → sleep/wake-up schedule
- Application assumptions:
  - Dedicated to a single application or a few collaborative applications
  - Involves in-network processing to reduce traffic and increase life time
  - Applications have long idle periods and can tolerate some latency

S-MAC: Approach

- [Periodic listening and sleeping] Coarse-grained TDMA-like sleep/awake cycles:
  - the sensor node periodically goes to the fixed listen/sleep cycle.
  - A time frame in S-MAC is divided into two parts: one for a listening session and the other for a sleeping session.

increased latency



- [Message passing] All nodes choose and announce awake schedules.
  - synchronize to awake schedules of neighboring nodes.
- [Collision and overhearing avoidance] Uses RTS/CTS to resolve contention during listen intervals.
  - And allows interfering nodes to go to sleep during data exchange.

S-MAC: Possible Scenarios

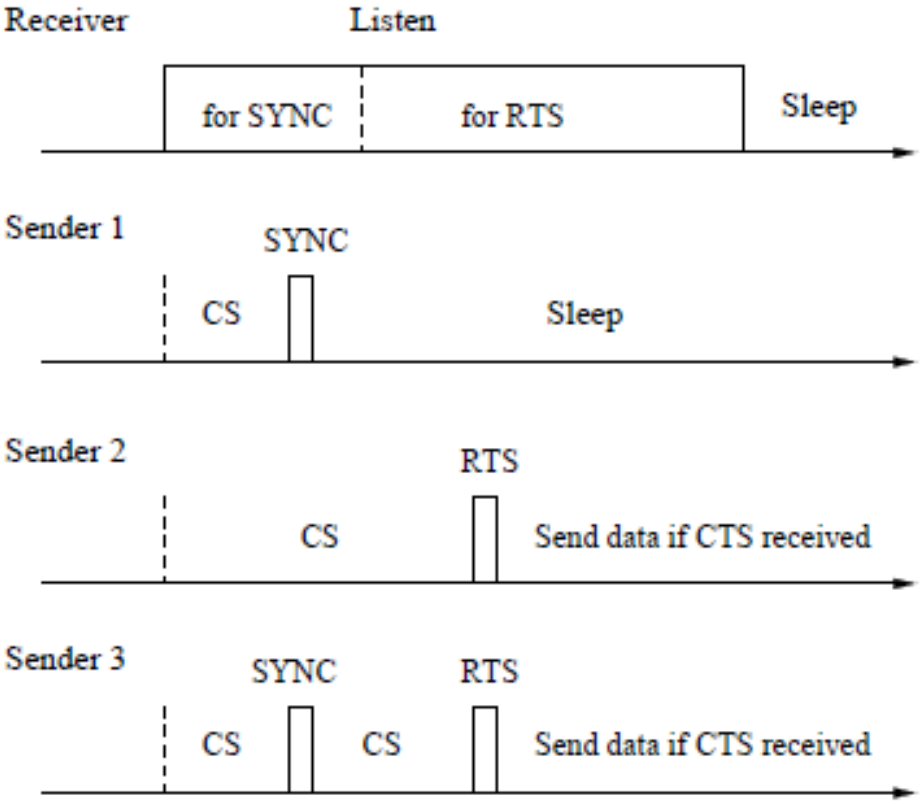
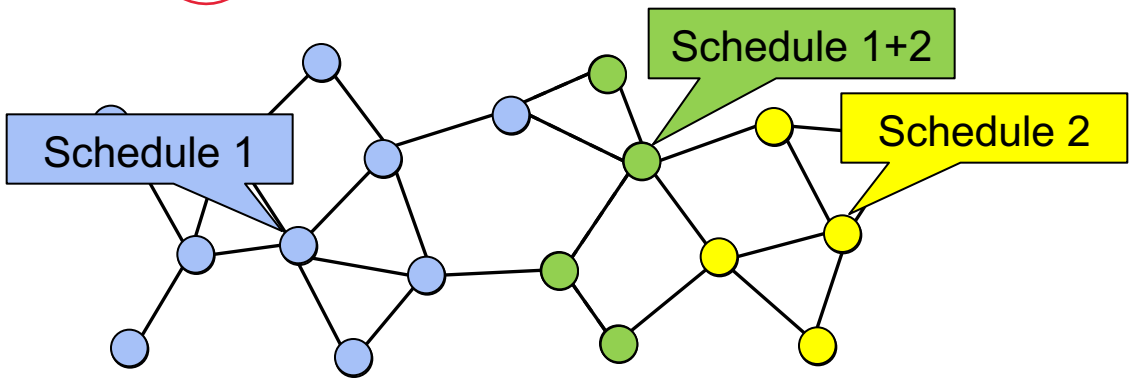


Fig. 3. Timing relationship between a receiver and different senders. CS stands for carrier sense.

Figure 3 also shows the timing relationship of three possible situations that a sender transmits to a receiver. CS stands for carrier sense. In the figure, sender 1 only sends a SYNC packet. Sender 2 only wants to send data. Sender 3 sends a SYNC packet and a RTS packet.

S-MAC: A Problem

- Nodes may have to follow multiple schedules to avoid network partition.

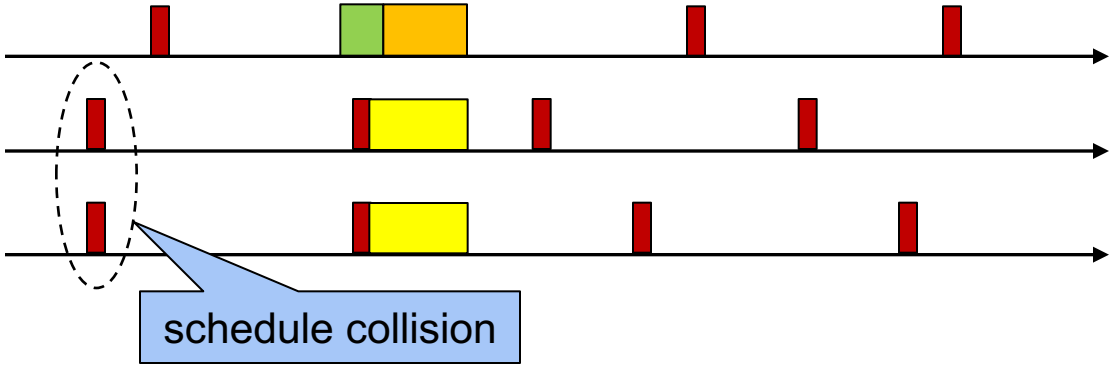


- A fixed sleep/awake ratio is not always optimal.
  - Variable load in the network.
- Idea: Adapt listen interval depending on the current network load → T-MAC

Hybrid MAC Protocols

- Protocols may use information from upper layers to further improve their performance.
  - Information about neighborhood
  - Routing policies

- Minimize costly overhearing of neighboring nodes
  - Inform them to change their channel sniff patterns



- Use randomization to resolve schedule collisions
- Example: I. Rhee, A. Warrier, M. Aia, and J. Min, "Z-MAC: a Hybrid MAC for Wireless Sensor Networks", Sensys 2005.

ContikiMAC – 2011 (Contiki 2.5)

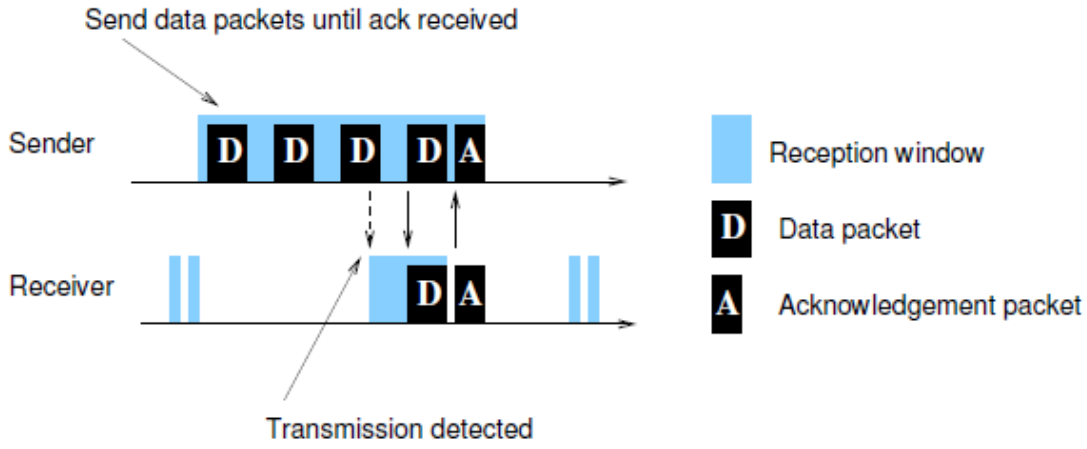


Figure 1: ContikiMAC: nodes sleep most of the time and periodically wake up to check for radio activity. If a packet transmission is detected, the receiver stays awake to receive the next packet and sends a link layer acknowledgment. To send a packet, the sender repeatedly sends the same packet until a link layer acknowledgment is received.

ContikiMAC: Broadcast

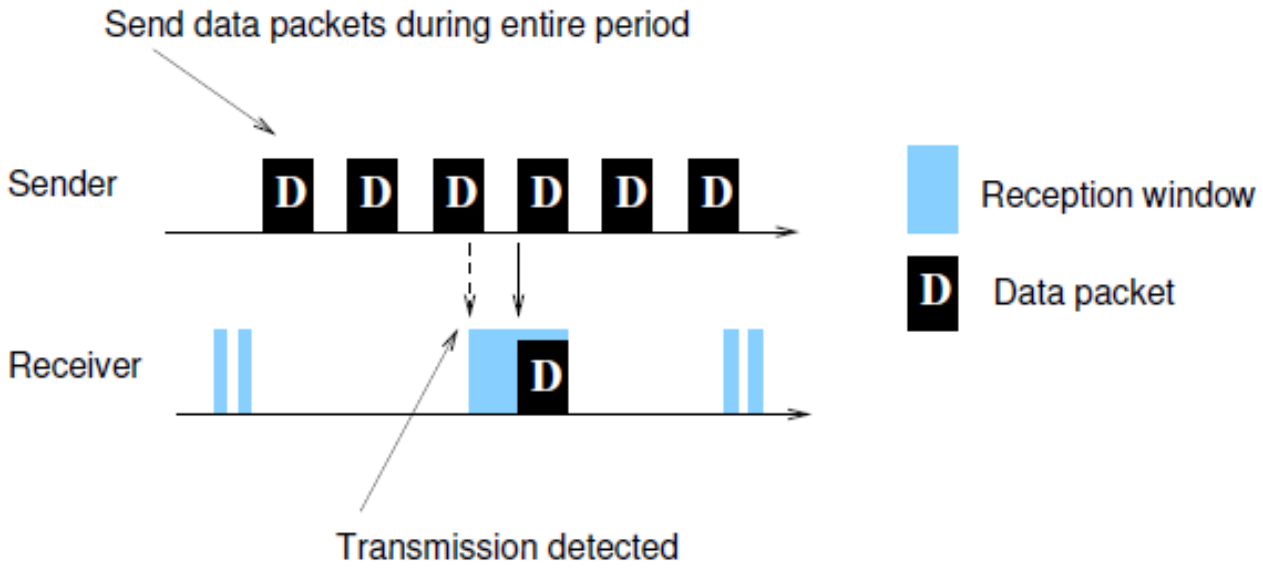


Figure 2: Broadcast transmissions are sent with repeated data packets for the full wake-up interval.

ContikiMAC: CCA timing

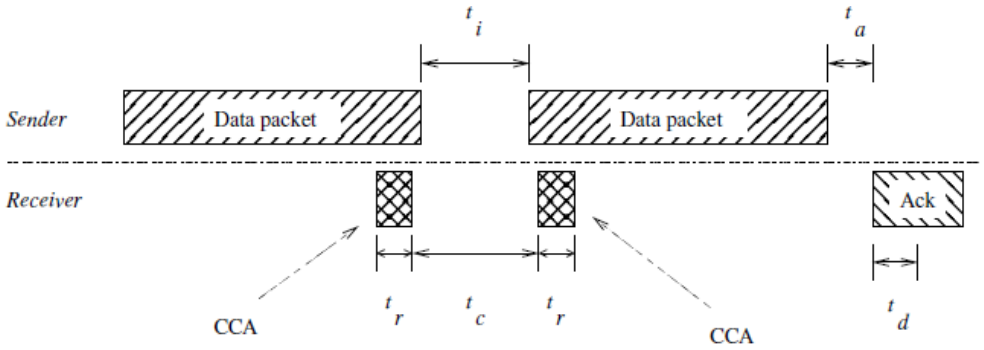


Figure 3: The ContikiMAC transmission and CCA timing.

The ContikiMAC timing is shown in Figure 3. The timing requirements from the figure are:

t_i: the interval between each packet transmission.

t_r: the time required for a stable RSSI, needed for a stable CCA indication.

t_c: the interval between each CCA.

t_a: the time between receiving a packet and sending the acknowledgment packet.

t_d: the time required for successfully detecting an acknowledgment from the receiver.

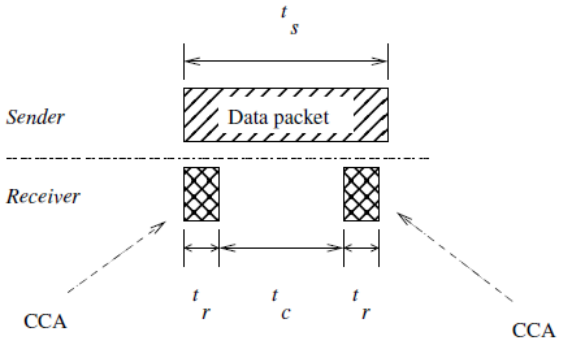


Figure 4: A packet transmission must be long enough so that it does not fall between to subsequent CCAs.

Timing constraints

$$t_a + t_d < t_i < t_c < t_c + 2t_r < t_s.$$

ContikiMAC: Transmission Phase Lock

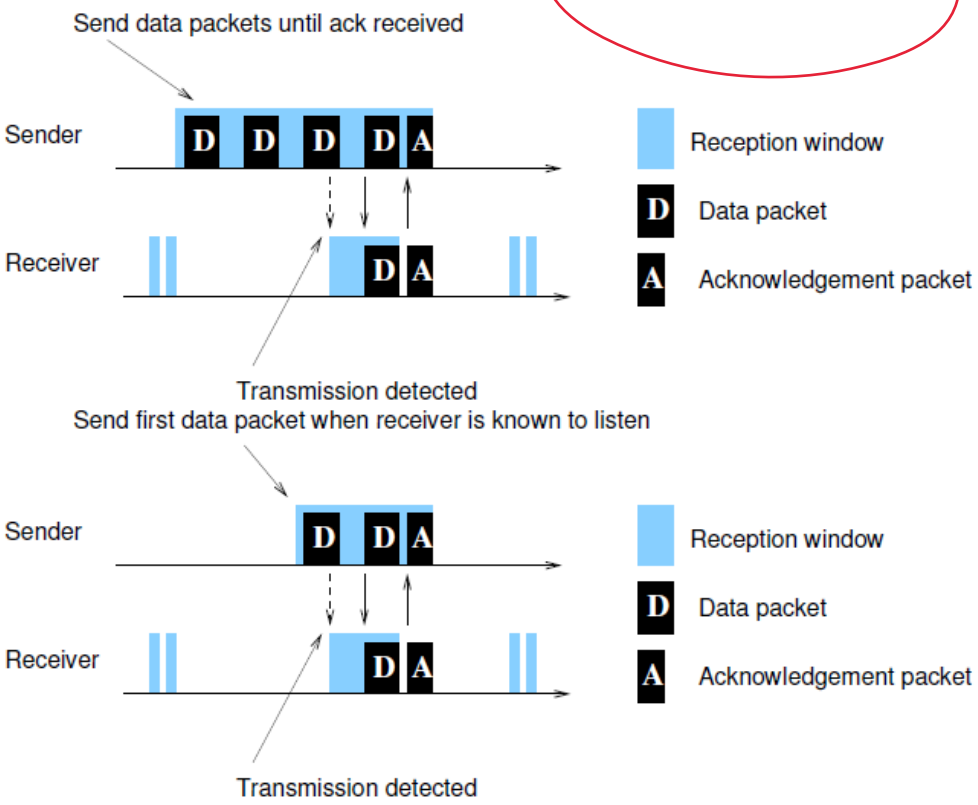
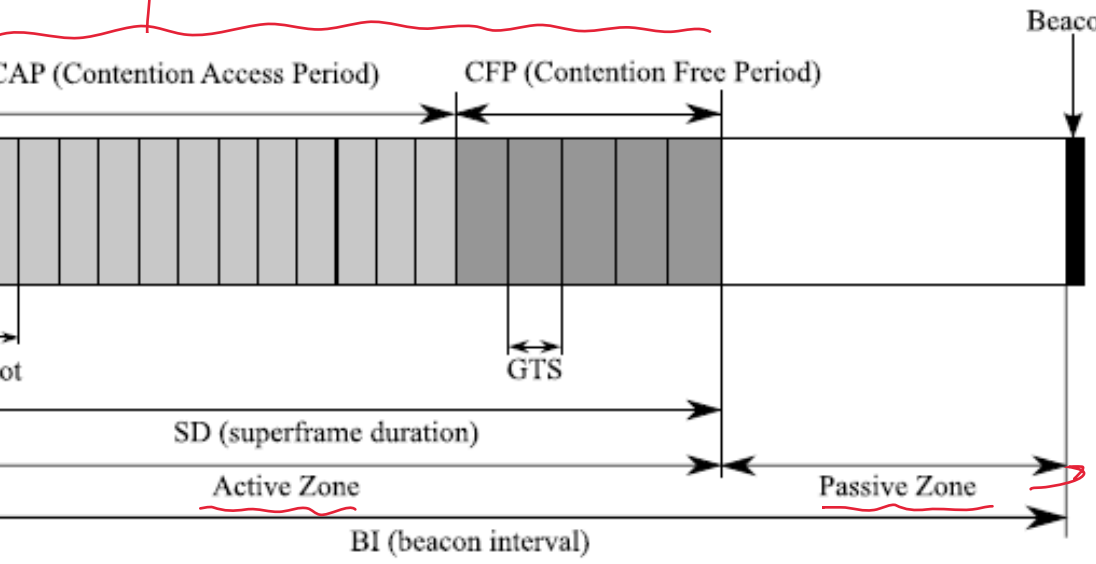


Figure 6: Transmission phase-lock: after a successful transmission, the sender has learned the wake-up phase of the receiver and subsequently needs to send fewer transmissions.

Becon-Enabled Mode

- The use of beacon packets allows to synchronize tne nodes.
- The time is divided into superframes.
- The nodes have, therefore, higher complexity, but duty cycle and channel access are better managed (GTSs).



Beaconless Mode

- No synchronization between nodes.
- Nodes' structures are simpler, but, obviously the performance of beacon-enabled networks is better.
- The MAC protocol differs in terms of time management, synchronization, and packet retransmissions.

CSMA/CA MAC Protocol: Unslotted Version

- The source node waits for a number of slots randomly generated in the interval $[0, 2^{BE_i} - 1]$.
- Once elapsed this time, the node verifies the channel status through a Clear Channel Assessment (CCA), i.e., the energy on the RF interface is measured over $8T_{sym}$.
- If the energy is above a threshold, the channel is busy; otherwise, it is assumed idle.
- If channel is idle, the node can transmit a packet in its queue. Otherwise, NB (number of backoff cycles for this packet) and m (maximum number of backoff cycles, denoted as backoff stage) are compared.
- If $NB = m$, the CSMA/CA is declared as failed. Otherwise, BE is incremented by 1. If $BE_i < BE_{max}$, the procedure is repeated.

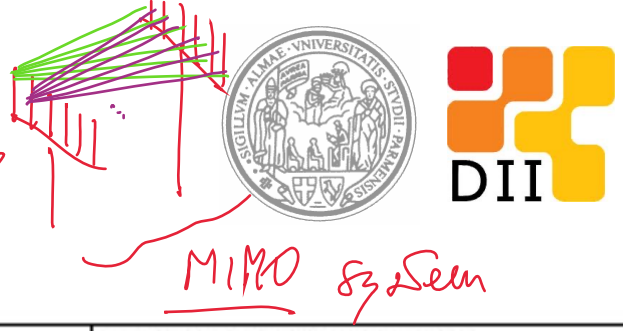
CSMA/CA MAC Protocol: Slotted Version

- The main difference is that all operations must be performed in the correspondence of a backoff boundary.
- CCAs of different nodes can collide. Therefore, two CCAs should be performed to prevent from collisions. However, this leads to waste in channel utilization.
- Obviously, in all cases collisions are still possible!!!

IEEE 802.11: Overview

- IEEE 802.11: wireless communication standard originally designed (late 90's) as a high-speed, short-range communication mechanism for laptops and general purpose PCs.
- WiFi is a brand name of the WiFi Alliance: identify equipment and software that is compatible with other WiFi and 802.11 systems
- Number of 802.11 devices worldwide: billions
- The widespread adoption of 802.11 makes deployment of smart objects easy
- Availability of 802.11 chipsets, routers, and network access cards reduces the cost of hardware for 802.11-enabled smart objects.
- Power consumption: typically one order of magnitude higher than that of 802.15.4 devices → new generation of low-power 802.11 transceivers (significant improvement of power consumption during sleeping periods and low power mode: turn off most of the functions, but resume quickly)

IEEE 802.11: standards



IEEE 802.11	Release Date	Frequency (GHz)	Max Data Rate (Mbps)	Range (m)		Status and Comments
				Indoor	Outdoor	
-1997	1997	2.4	2	20	100	Obsolete
a	1999	5/3.7	54	35/-	120/5k	Legacy systems
b	1999	2.4	11	35	140	Legacy systems
g	2003	2.4	54	38	140	Legacy systems
n	2009	2.4/5	600	70	250	Current systems
ac	2013	2.4/5	450/7,000	35		Next generation – just starting to be deployed.
ad	2012	60	7,000	10		Known as WiGig. Short-reach high data rate data transfers.
af	Est. 2016	0.470-0.710	568		6,000	Being called White-Fi because it uses unused TV spectrum.
ah		0.9	40			In development.
aj		45/60	7,000	10		Modification of 802.11ad for 45GHz band for use in China
ax	Est. 2019	2.4/5	450/10,000	35		Revision to 802.11ac to increase efficiency.

IEEE 802.11: foci of the standards

Task Group	Purpose	Status
PHY	Develop Three Physical Layer Options at 1–2 Mbps	Standard
MAC	Develop One Common MAC Layer	Standard
802.11a	High-Speed (54 Mbps) Physical Layer Extension in the 5.8 GHz Band	Standard
802.11b	High-Speed (11 Mbps) Physical Layer in the 2.4 GHz Band	Standard
802.11e	MAC Enhancements for QoS	Standard
802.11g	Even Higher Speed (54 Mbps) Physical Layer Extension in the 2.4 GHz Band	Standard
802.11i	MAC Enhancements for Enhanced Security	Standard
802.11n	Very High-Speed (100 Mbps) Physical Layer	Draft
802.11r	Mobility Handoffs	Draft
802.11s	Mesh Networks	Draft

IEEE 802.11: Low-Power

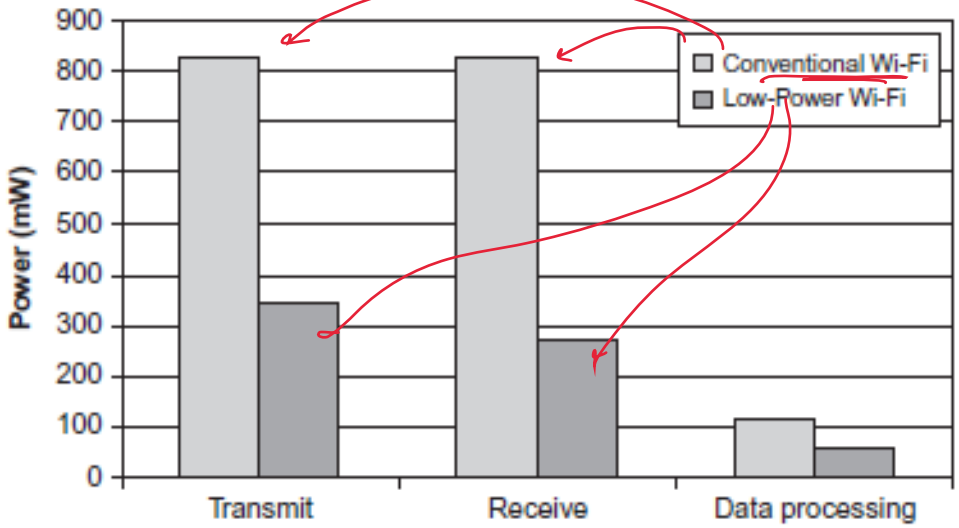


FIGURE 12.15

Comparison of power consumption in transmit, receive, and processing mode for conventional 802.11 and low-power 802.11.

Table 12.1 Power Consumption of Standby and Sleep Modes of a Conventional 802.11 Transceiver and of a Low-power 802.11 Transceiver

	Conventional 802.11 (mW)	Low-power 802.11 (mW)
Standby mode	N/A	0.018
Sleep mode	13	0.2

Note: Conventional 802.11 transceivers do not have a standby mode.

IEEE 802.11: network topology

- Two operational modes: infrastructure (access points) and ad-hoc (direct communications)

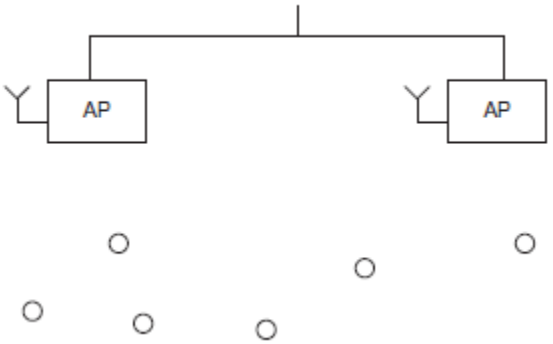


FIGURE 12.11
A network topology of 802.11 in access point mode. Each 802.11 transceiver is associated with an access point. A network may be served by more than one access point. The access points typically have wired network access.

- A node must associate itself with the AP → a group of nodes with an AP is called Basic Service Set (BSS) → with more than one AP, Extended Service Set (ESS) → Service Set IDentifier (SSID)
  - Node probes for a known or unknown SSID
  - AP replies with a probe response packet, with configuration parameters
  - If the network is ok, the node sends an authentication request frame
  - AP replies with association reply frame

IEEE 802.11: PHY layer

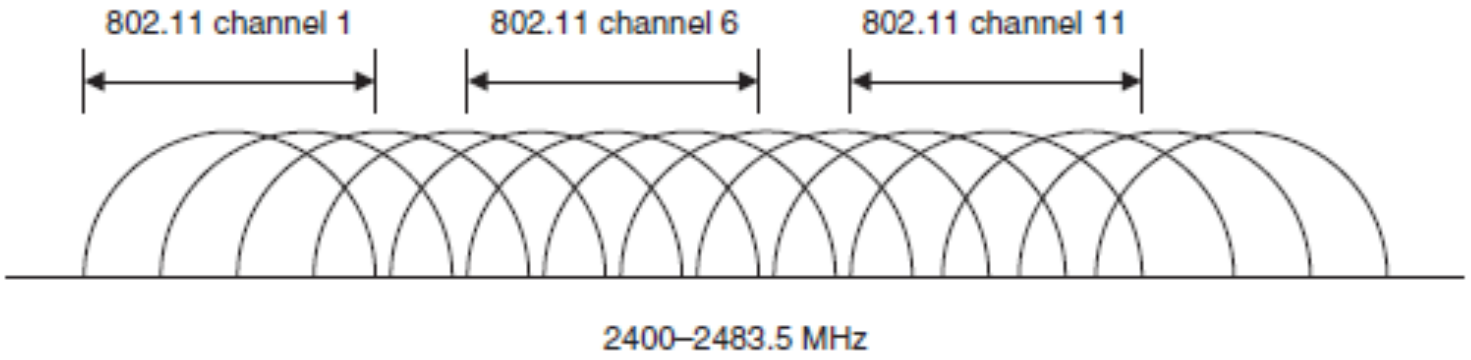


FIGURE 12.12

IEEE 802.11 operates on 14 different channels in the 2.4 GHz band. Channels 1, 6, and 11 do not overlap. Channels 12, 13, and 14 are not available in every country due to radio frequency licensing restrictions.

- Each 802.11 packet is sent with a fixed bit rate, but the bit rate may differ between packets (higher rate with a good communication channel)
- Broadcast packets are sent with a lower bit rate than packets addressed to a specific host to be compatible with older versions

IEEE 802.11: MAC layer

- MAC layer of 802.11: based on Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)

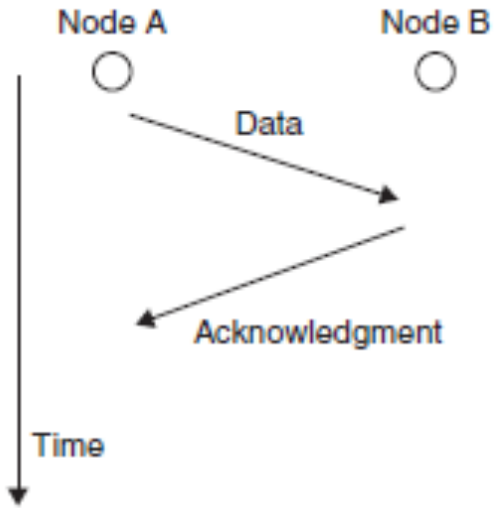


FIGURE 12.13

The IEEE 802.11 MAC layer uses acknowledgment packets that inform the sender that a packet was successfully received by the receiver.

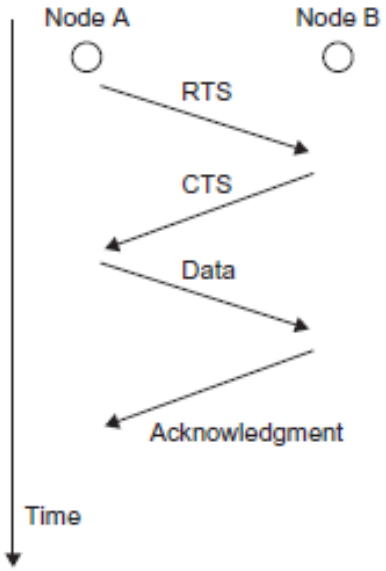


FIGURE 12.14

The RTS/CTS mechanism of the 802.11 MAC layer. To send a packet, a node first sends an RTS packet. If the access point grants the request, it sends a CTS packet and the node can send its packet.

- Mechanism of ready-to-send (RTS) and clear-to-send (CTS) solves the hidden and exposed terminal problems (see later)

PLC: a Quick Overview (1/2)

- Power line communications: eletrical lines are everywhere
- PLC: provides smart objects with data and energy (no need for batteries)

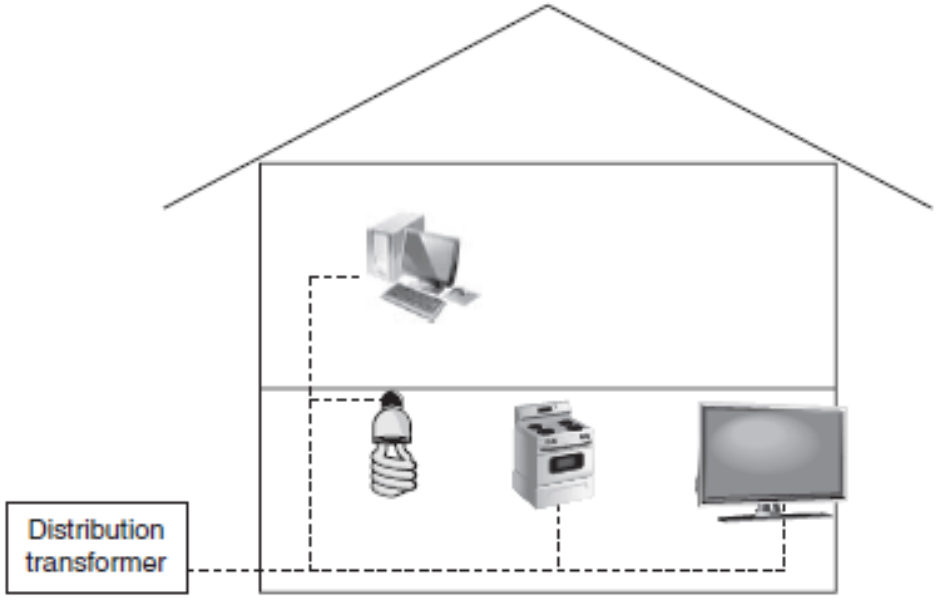


FIGURE 12.16

PLC network within a home.

PLC: a Quick Overview (2/2)

- Data transported across the 50 Hz or 60 Hz electrical distribution network: electrical equipment may add a significant noise
- PLC for short-range communication: only within one electrical domain such as a home or an office building (modulated data signal cannot traverse the electrical voltage transformers outside the electrical domain)
- PHY: copper wires (carry high-power AC signal → noisy; add interferences from plugged-in devices) → very robust communication mechanisms → data rates from kb/s to Mb/s
- MAC: CSMA/CA (unlike transceivers for Ethernet cables, a node transmitting over an electrical line cannot receive)
- Power savings still important: (i) reduce overheating and (ii) avoid compromising overall power savings

UNIVERSITÀ DI PARMA

IoT Lab

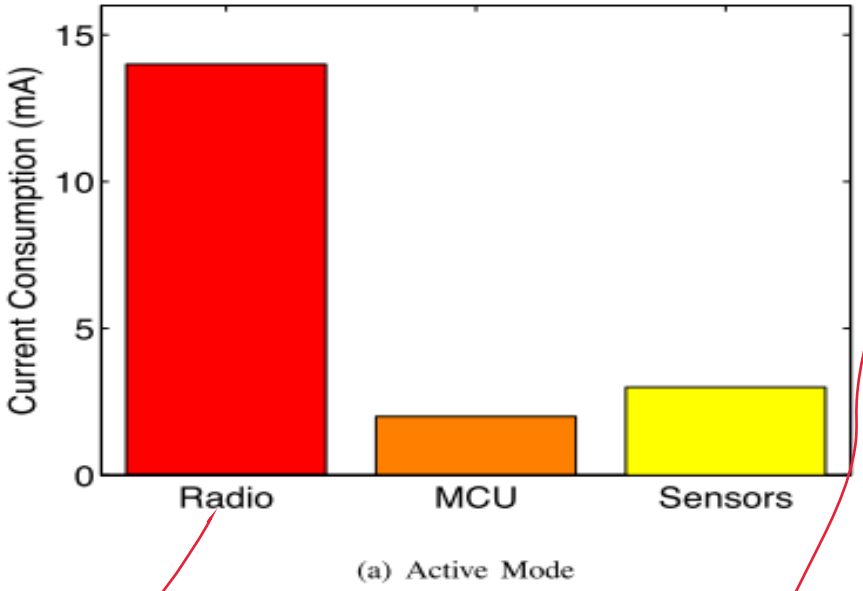
Energy-efficient MAC Protocols

Main sources

- A. Bachir, M. Dohler, T. Watteyne, K. K. Leung, "MAC Essentials for Wireless Sensor Networks," *IEEE Communications Surveys & Tutorials,* vol.12, no.2, pp.222-248, Second Quarter 2010. doi: 10.1109/SURV.2010.020510.00058
- [MACA] P. Karn,"MACA - a New Channel Access Method for Packet Radio", in ARRL/CRRL Amateur Radio 9tt,~ Computer Networking Conference, pp. 134-140, 1990.
- [PAMAS] Suresh Singh and C. S. Raghavendra. 1998. PAMAS—power aware multi-access protocol with signalling for ad hoc networks. *SIGCOMM Comput. Commun. Rev.* 28, 3 (July 1998), 5-26. DOI=10.1145/293927.293928 http://doi.acm.org/10.1145/293927.293928
- [STEM] C. Schurgers, V. Tsiatsis, S. Ganeriwal, M. Srivastava, "Optimizing sensor networks in the energy-latency-density design space," *IEEE Transactions on Mobile Computing*, vol.1, no.1, pp.70,80, Jan-Mar 2002. doi: 10.1109/TMC.2002.1011060
- [B-MAC] J. Polastre, J. Hill, and D. Culler, "Versatile low power media access for wireless sensor networks." Proc. 2nd international conference on Embedded networked sensor systems (SenSys '04), pp. 95-107, Baltimore, MA, USA November 2014. doi>10.1145/1031495.1031508
- [S-MAC] W. Ye, J. Heidemann, D. Estrin, "An energy-efficient MAC protocol for wireless sensor networks," *Proc. 21st Annual Joint Conference of the IEEE Computer and Communications Societies* (INFOCOM 2002) , pp.1567-1576, 2002. doi: 10.1109/INFCOM.2002.1019408
- [X-MAC] Michael Buettner, Gary V. Yee, Eric Anderson, and Richard Han. 2006. X-MAC: a short preamble MAC protocol for duty-cycled wireless sensor networks. In *Proceedings of the 4th international conference on Embedded networked sensor systems* (SenSys '06). ACM, New York, NY, USA, 307-320. DOI=10.1145/1182807.1182838
- [ContikiMAC] A. Dunkels, "The ContikiMAC Radio Duty Cycling Protocol", SICS Technical Report T2011:13. ISSN 1100-3154. December 2011

Power consumption

- Power consumption is one of the biggest challenges because:
  - nodes typically have a limited source of energy (finite battery);
  - battery difficult to replace (and to waste).



(a) Active Mode

Radio mode	Power consumption (mW)
Transmit (T_x)	14.88
Receive (R_x)	12.50
Idle	12.36
Sleep	0.016

3 orders of magnitude reduction

WeMee — Transmission, reception, idle listening

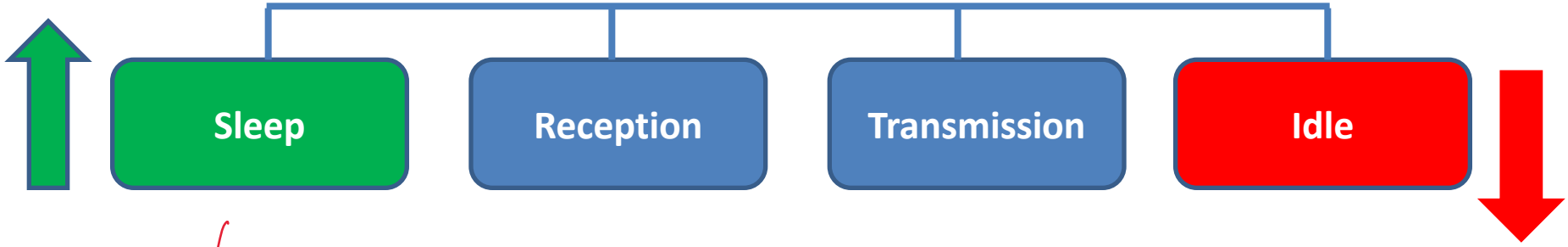
Sources of power consumption

- Useful power consumption:
  - Transmitting or receiving data.
  - Processing queries requests.
  - Forwarding queries and data to the neighbours.
- Wasteful power consumption:
  - Idle listening to the channel "waiting for possible traffic".
  - Retransmitting because of collisions → Collision Avoidance is very important
  - Overhearing: a node receives (and processes) a packet which was not destined to it.
  - Generating and handling control packets (overhead).

Power consumption in WSNs

How to minimize the energy consumption of sensor nodes while meeting the application requirements?

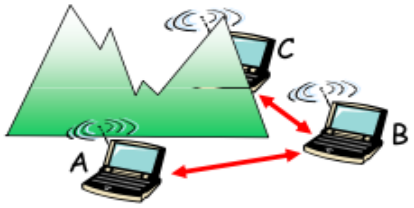
- Use Protocols that aim mainly to increase the sleep periods as much as possible



↳ The typical sleeping portion considered for IoT systems may go up to 99.9%

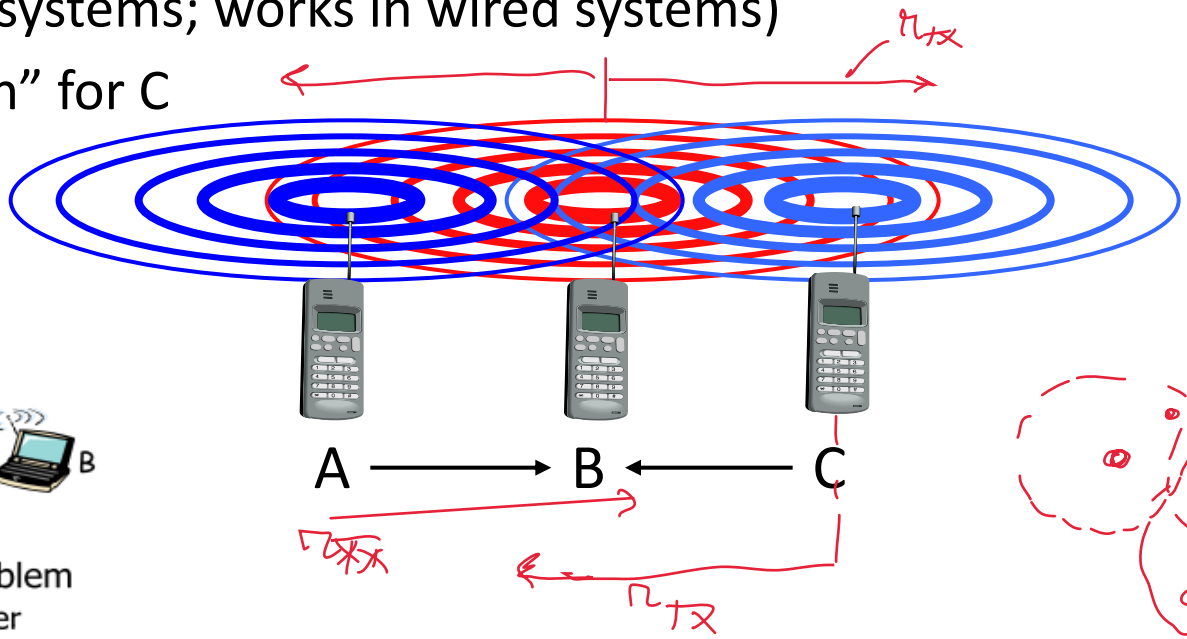
Hidden Terminal Problem

- A sends to B, C cannot receive A
- C wants to send to B, C senses a “free” medium (CS=carrier sensing fails)
- collision at B, A cannot receive the collision (CD=collision deteciont fails in wireless systems; works in wired systems)
- A is “hidden” for C



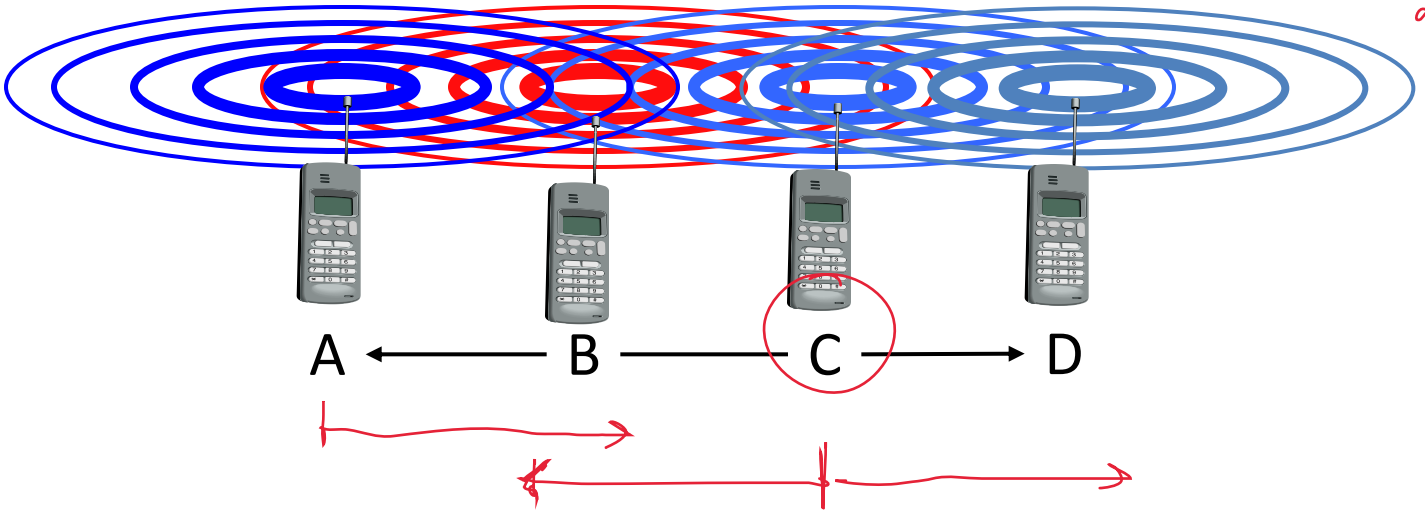
Hidden terminal problem
- B, A hear each other
- B, C hear each other
- A, C can not hear each other

means A, C unaware of their interference at B



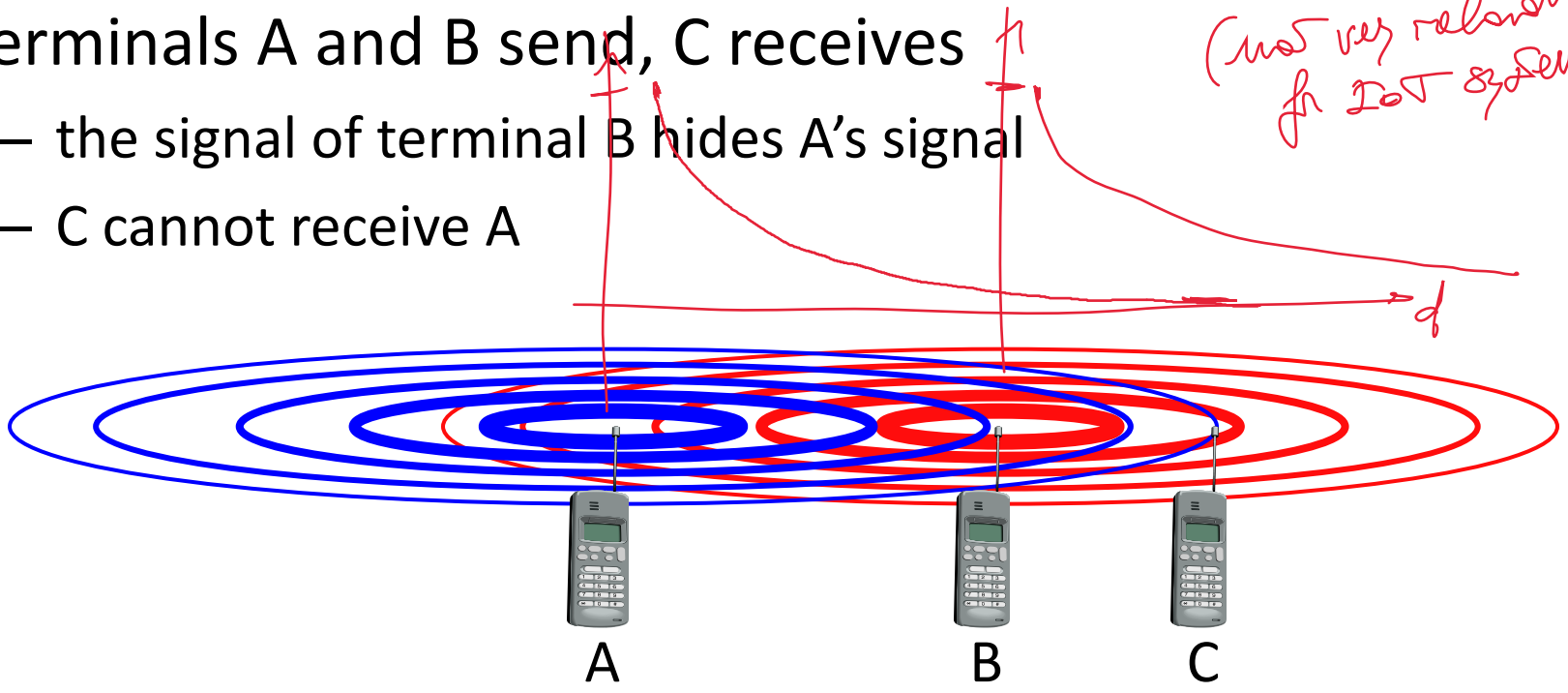
Exposed Terminal Problem

- B sends to A, C wants to send to D
- C has to wait, as it senses a medium in use
- since A is outside the radio range of C waiting is not necessary
- C is "exposed" to B → C is prevented from transmitting even if its transmission would not damage B→A.



Near-Far Problem

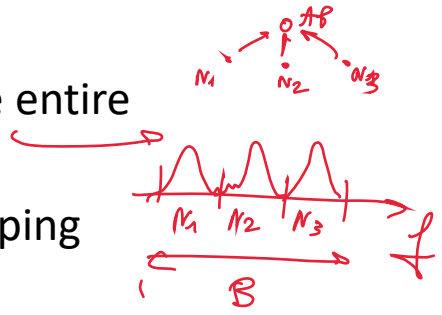
- Terminals A and B send, C receives
  - the signal of terminal B hides A's signal
  - C cannot receive A



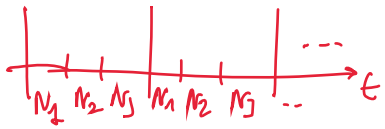
- This is also a severe problem for CDMA networks
- Precise power control required

Centralized Access Methods (Cellular)

- SDMA (Space Division Multiple Access)
  - Segment space into sectors, use directional antennas
  - Use cells to reuse frequencies
- FDMA (Frequency Division Multiple Access)
  - Assign a certain frequency to a transmission channel for the entire transmission duration
  - Permanent (radio broadcast), slow hopping (GSM), fast hopping (FHSS, Frequency Hopping Spread Spectrum → bluetooth)



- TDMA (Time Division Multiple Access)
  - Assign a fixed sending frequency (possibly the entire spectrum) for a certain amount of time
- CDMA (Code Division Multiple Access)
- Combinations!



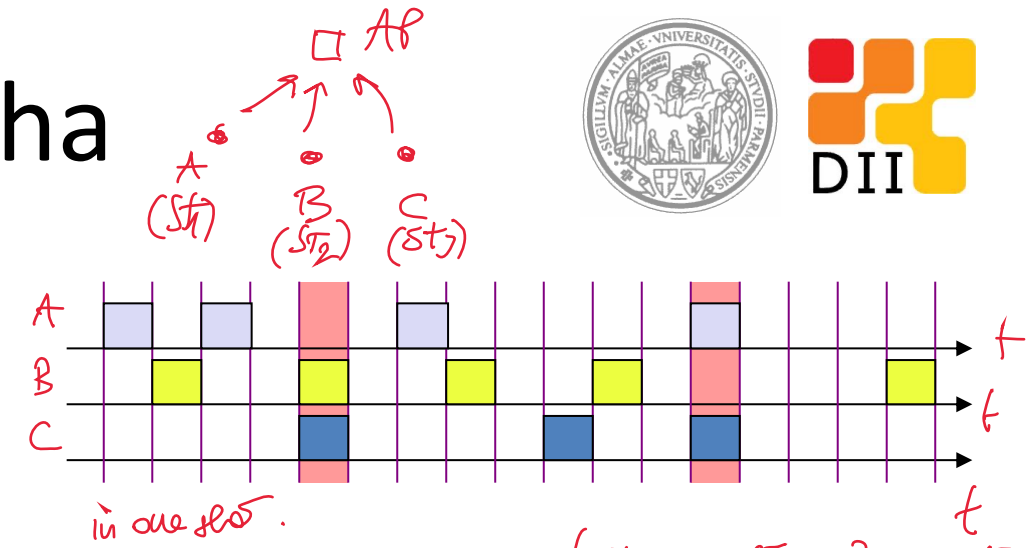
3/4 G

LTE, 5G

Traditional MAC Protocol Classification

- Centralized/Single-Hop Protocols (see previous page)
  - A base station/controller coordinates all traffic (SDMA, FDMA, TDMA, CDMA)
- Contention Protocols (Aloha, Carrier Sense Multiple Access-CSMA)
  - Transmit when you feel like transmitting (Aloha)
  - Retry if collision, try to minimize collisions, additional reservation modes (CSMA)
  - Problem: Receiver must be awake as well
- Scheduling Protocols
  - Use a "pre-computed" schedule to transmit messages
  - Distributed (non-centralized) and adaptive solutions are difficult
- Hybrid protocols
  - E.g. contention with reservation → scheduling (e.g., IEEE 802.15.4 with beacons)
  - Specific ("cross-layer") solutions

Slotted Aloha



- We assume that the stations are perfectly synchronous
- In each time slot each station transmits with probability *p*.

All stations are independent

$$P_1 = \Pr[\text{Station 1 succeeds}] = p(1-p)^{n-1}$$

$$P = \Pr[\text{any Station succeeds}] = nP_1$$

$$\text{maximize } P: \frac{dP}{dp} = n(1-p)^{n-2}(1-pn) \overset{!}{=} 0 \Rightarrow pn = 1 \Rightarrow p^* = \frac{1}{n}$$

$$\text{then, } P = (1-\frac{1}{n})^{n-1} \geq \frac{1}{e}$$

$P = n \cdot p \cdot (1-p)^{n-1}$, $p \in (0,1)$

$P(p^*) = \frac{1}{n}(1-\frac{1}{n})^{n-1}$

$P\{\{St_1 \text{ succeeds}\} \cup \{St_2 \text{ succeeds}\} \cdots \cup \{St_n \text{ succeeds}\}\} = n \cdot P_1$

disjoint events

$P\{St_1 \text{ Tx}\} \cap \{St_2 \text{ does not Tx}\} \cdots \cap \{St_n \text{ does not Tx}\}$

$= P\{St_1 \text{ Tx}\} \cdot P\{St_2 \text{ does not Tx}\} \cdots P\{St_n \text{ does not Tx}\}$ (independence)

$= p \cdot (1-p) \cdots (1-p)$

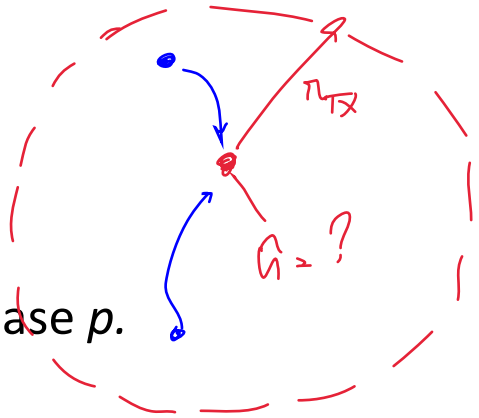
- In Slotted Aloha, a station can transmit successfully with probability at least 1/*e*, i.e., about 36% of the time.
- In pure Aloha (unslotted), a station can transmit successfully with probability at least 1/2e, i.e., about 18% of the time.

$(1-\frac{1}{n})^{n-1} \to e^{-1}$ as $n \to \infty$

$e^{\ln(1-\frac{1}{n})(n-1)} = e^{(n-1)\ln(1-\frac{1}{n})}$, $\ln(1-x) \to -x$ as $x \to 0$

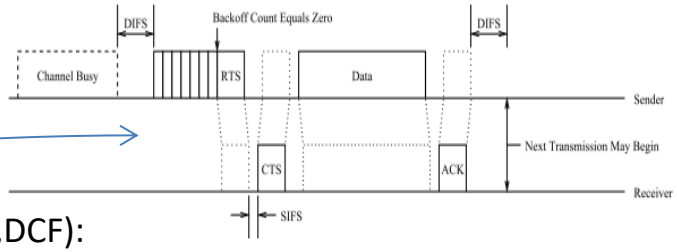
Adaptive Slotted Aloha

- Idea: change the access probability as a function of the number of stations
- How can we estimate the current number of stations in the system?
- Assume that stations can distinguish whether 0, 1, or more than 1 stations transmit in a time slot.
- Model:
  - Number of stations that want to transmit: n.
  - Estimate of n: $\hat{n}$
  - Transmission probability: $p = 1/\hat{n}$
- Idea:
  - If you see that nobody transmits, increase p.
  - If you see that more than one transmits, decrease p.



Carrier Sense Multiple Access (CSMA)

- Listen before transmit: if the channel is free, then transmit
- Various versions:
  - non-persistent: if the device detects activity on the channel, it performs a back off by waiting before attempting to transmit.
  - *p*- persistent: if it detects activity on the channel, it keeps on sensing the channel instead of delaying.
- "Basic" CSMA requires devices to remain in the receive state (not idle) when not transmitting → the transceiver consumes energy too quickly.
- Various variations
  - CSMA/CD: collision detection (Ethernet, IEEE 802.3)
  - CSMA/CA: collision avoidance, LIFO (IEEE 802.15.4, IEEE 802.11-PCF,DCF): combine with Ready-to-send (RTS) and Clear-to-Send (CTS) mechanisms → RTS messages may collide
  - IEEE 802.11 uses Network Allocation Vector (NAV), virtual carrier-sensing mechanism: the MAC frame header contains the packet transmission duration



Backoff

- Backoff protocols rely on acknowledgements.
- Binary exponential backoff:
  - If a packet has collided k times (no ACK for k times), we set $p = 2^{-k}$
  - Or alternatively: wait from random number of slots in $[1,...,2^k]$
- It has been shown that binary exponential backoff is not stable for any arrival rate $\lambda > 0$ if there are infinitely many potential stations
- Interestingly when there is a finite number of stations, binary exponential backoff becomes unstable with $\lambda > 0.568$;
  Polynomial backoff however, remains stable for any $\lambda < 1$.

Network Performance

Moving towards WSNs

- Traditional MAC protocols
  - High throughput
  - Low latency
  - Fairness
  - Mobility
  - Energy? → neglected

- MAC protocols for WSNs:
  - Primary goals
    - Energy conservation
    - Scalability and adaptivity
  - Secondary goals
    - Throughput
    - Latency
    - Fairness

Energy-efficient MAC Protocols

- In WSNs, energy is often more critical than throughput.
  - The radio transceiver should be turned off as much as possible.
- Energy management considerations have a big impact on MAC protocols.
  - Idle listening costs about as much energy as transmitting
- In the following: a few ideas, taken from some known protocols that try to balance throughput and energy consumption:
  - MAC,PAMAS, STEM, B-MAC, X-MAC, WiseMAC, S-MAC, **ContikiMAC**
  - Many of the hundreds of MAC protocols that were proposed have similar ideas…

Unscheduled MAC protocols

- "Evolutions" of CSMA
- Advantages:
  - they can adapt for changes "in the node density, traffic load or the topology" better than scheduled protocols.
  - The nodes do not have to be synchronized together.
- Disadvantages:
  - All nodes need to listen to the channel
- Examples:
  - MACA (1990)
  - PAMAS (1998)
  - STEM (2002)
  - B-MAC (2004), WiseMAC (
  - X-MAC (2006)

MACA (Multiple Access with Collision Avoidance) - 1990

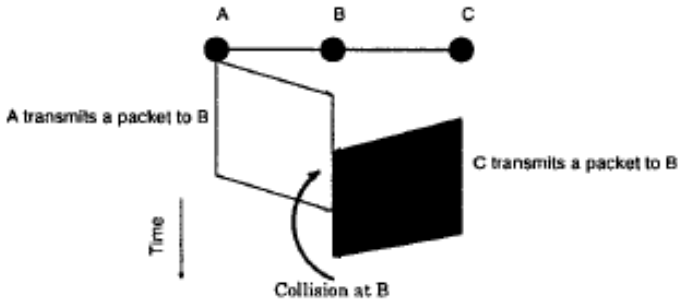
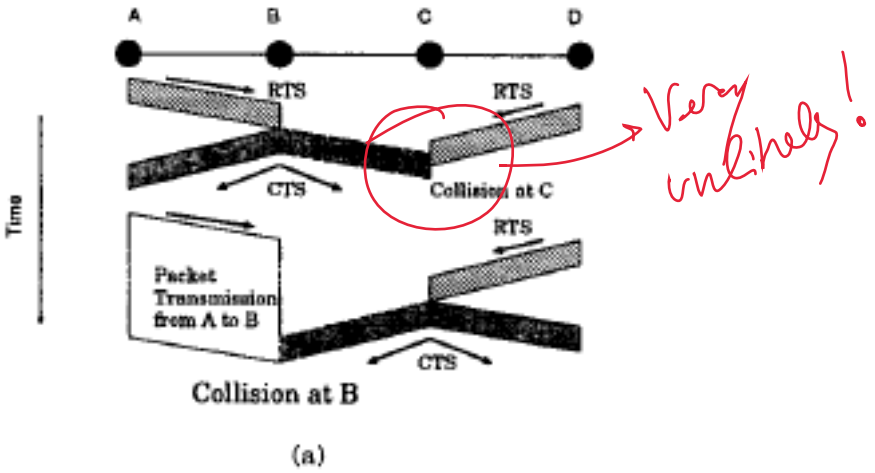


Figure 2: The hidden terminal problem.

- Hidden terminal problem: partially solved with the RTS/CTS mechanism introduced by MACA

Also solves the exposed terminal problem



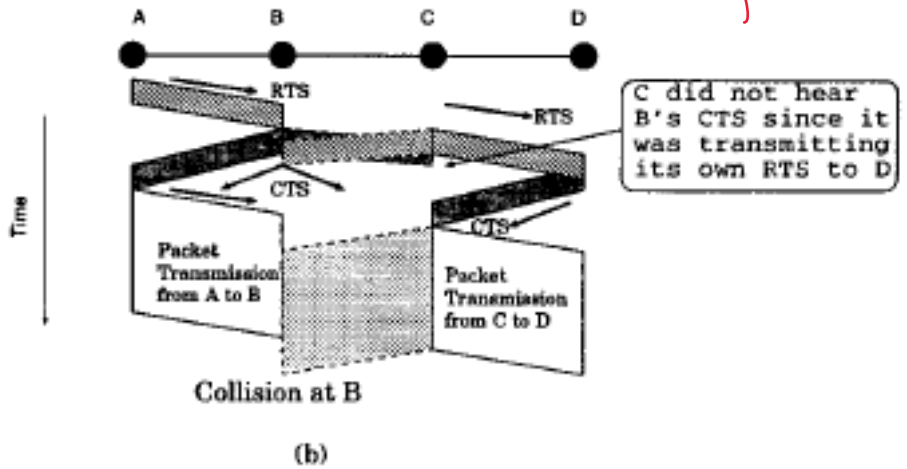


Figure 3: Hidden terminal problem in MACA.

PAMAS (Power Aware Multi-Access protocol with Signaling for Ad-Hoc Networks) - 1998

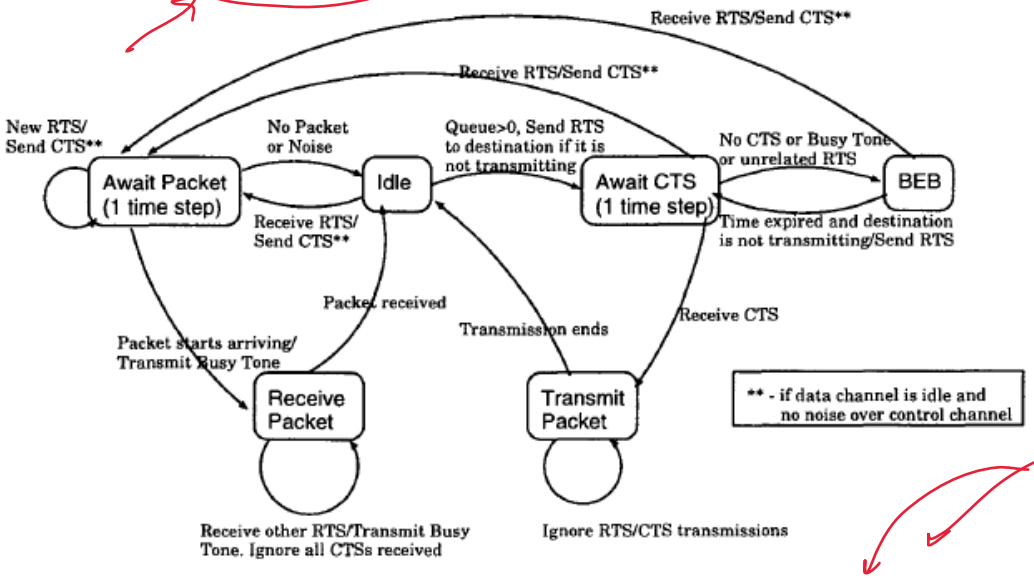
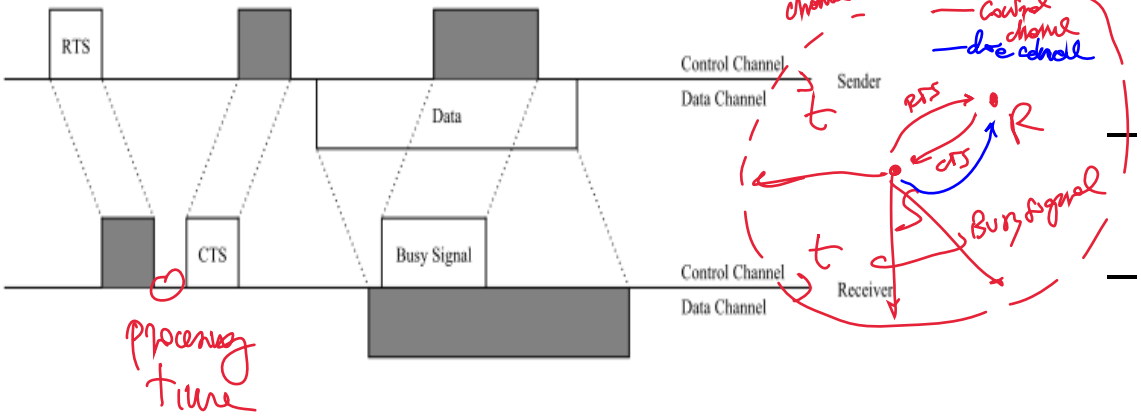


Figure 4: The PAMAS Protocol.



- Basic idea: use a *signaling channel* separated from data channel
- Advantages:
  - Prevent collisions
- Disadvantages:
  - Multiple radio requirement
  - Increase energy consumption
  - Increase device complexity and cost

STEM (Sparse Topology and Energy Management) - 2002

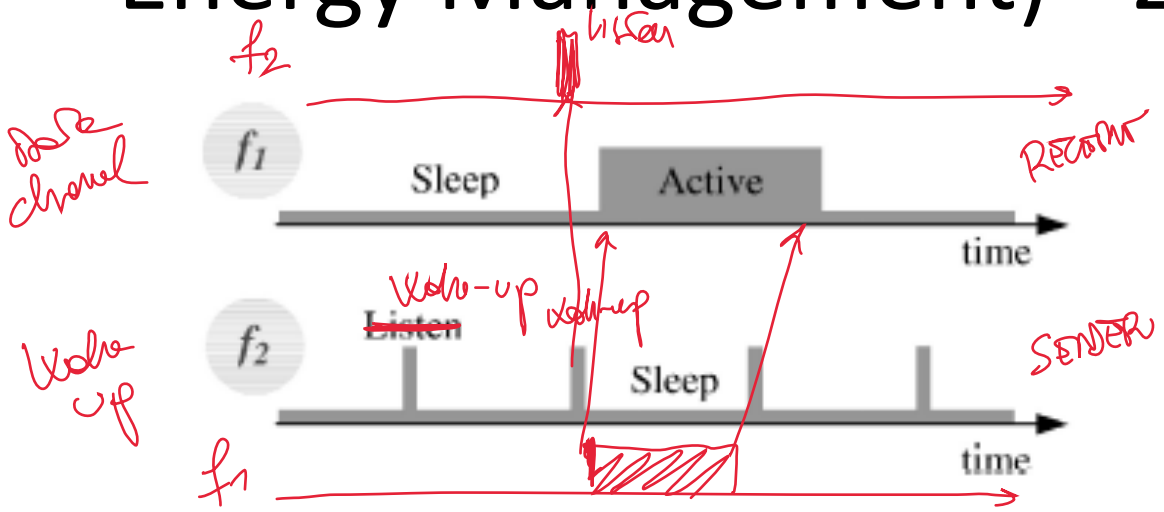


Fig. 3. Separate data and wakeup using two radios.

- Basic idea: uses two different channels: the wakeup channel (f_2) and the data channel (f_1)
- Requires two transceivers in each node

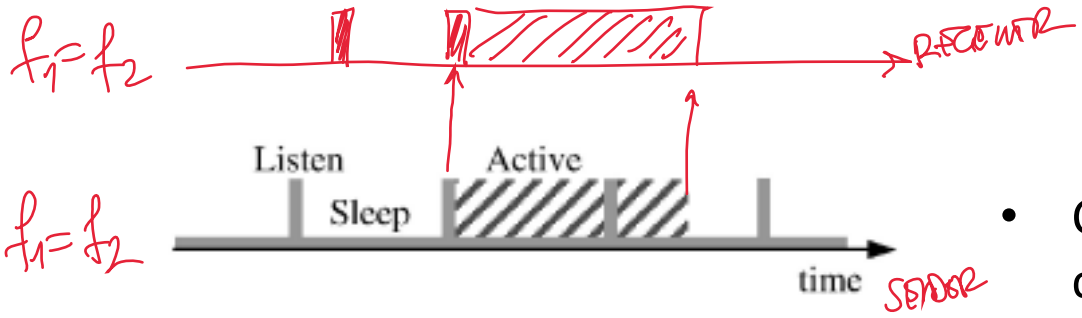


Fig. 4. Separate data and wakeup in time.

- Could "mix" the two channels → synchronization is required

STEM-B and STEM-C - 2002

- STEM-B: a sensor node wakes up a neighbor by transmitting a beacon (no RTS/CTS)
  - Advantages:
    - Lower Latency
  - Disadvantages:
    - More complex
    - High energy consumption
- STEM-T: a sensor node wakes a neighbor by transmitting a busy tone (contains no destination address) of sufficient length so that the destination will have a high probability of sensing
  - Disadvantages:
    - High latency
    - Results in overhearing (stops the transmission from nodes which would not interfere)

"Berkeley MAC" (B-MAC) – 2004

(Versatile low power media access for wireless sensor networks)

- Goals
  - Low power operation
  - Effective collision avoidance
  - Simple implementation (small code)
  - Energy-efficient at both low and high data rates
  - Reconfigurable by upper layers
  - Tolerant to changes on the network
  - Scalable to large number of nodes

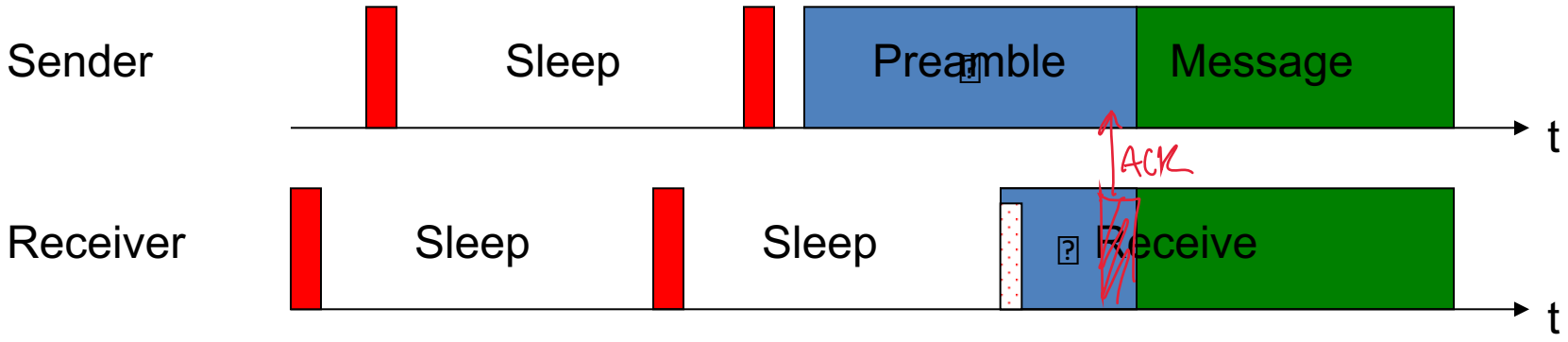
MIRACLE!

- Strategy
  - Nodes wake up for a short period and check for channel activity.
  - Return to sleep if no activity is detected. or if they do not need to transmit.
  - If a sender wants to transmit a message, it sends a long preamble to make sure that the receiver is listening for the packet (STEM-T) → preamble has the size of a sleep interval
- Very robust
  - No synchronization required
  - Instant recovery after channel disruption

B-MAC's Features

- Clear Channel Assessment (CCA)
- Low Power Listening (LPL) using preamble sampling
- Hidden terminal and multi-packet mechanisms not provided: they should be implemented, if needed, by higher layers

- CCA on/off
- Acknowledgements on/off
- Initial and congestion backoff on a per-packet basis
- Configurable check interval and preamble length



B-MAC's Lifetime Model

Operation	Time (s)		I (mA)	
Initialize radio (b)	350E-6	$t_{r_{init}}$	6	$c_{r_{init}}$
Turn on radio (c)	1.5E-3	t_{ron}	1	c_{ron}
Switch to RX/TX (d)	250E-6	$t_{rx/tx}$	15	$c_{rx/tx}$
Time to sample radio (e)	350E-6	t_{sr}	15	c_{sr}
Evaluate radio sample (f)	100E-6	t_{ev}	6	c_{ev}
Receive 1 byte	416E-6	t_{rxb}	15	c_{rxb}
Transmit 1 byte	416E-6	t_{txb}	20	c_{txb}
Sample sensors	1.1	t_{data}	20	c_{data}

Table 2: Time and current consumption (I) for completing primitive operations of a monitoring application using the Mica2 mote and CC1000 transceiver. Identifiers on each operation map back to the activities of acquiring a radio sample in Figure 3.

Notation	Parameter	Default
c_{sleep}	Sleep Current (mA)	0.030
C_{batt}	Capacity of battery (mAh)	2500
V	Voltage	3
$L_{preamble}$	Preamble Length (bytes)	271
L_{packet}	Packet Length (bytes)	36
t_i	Radio Sampling Interval (s)	100E-3
n	Neighborhood Size (nodes)	10
r	Sample Rate (packets/s)	1/300
t_l	Expected Lifetime (s)	-

Table 3: Parameters for a monitoring application running B-MAC. The first three parameters are specific to Mica2 motes; the next three are default values for B-MAC parameters on the Mica2; the remaining parameters are application semantics affecting B-MAC's performance. Each parameter affects the node's total energy consumption, E.

$$t_{tx} = r \times (L_{preamble} + L_{packet}) t_{txb}$$
$$E_{tx} = t_{tx} c_{txb} V$$

$$t_d = t_{data} \times r$$
$$E_d = t_d c_{data} V$$

$$t_{sleep} = 1 - t_{rx} - t_{tx} - t_d - t_{listen}$$
$$E_{sleep} = t_{sleep} c_{sleep} V$$

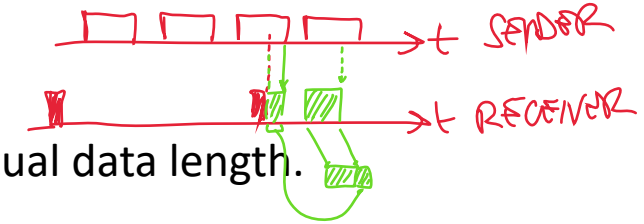
$$E = E_{rx} + E_{tx} + E_{listen} + E_d + E_{sleep}$$

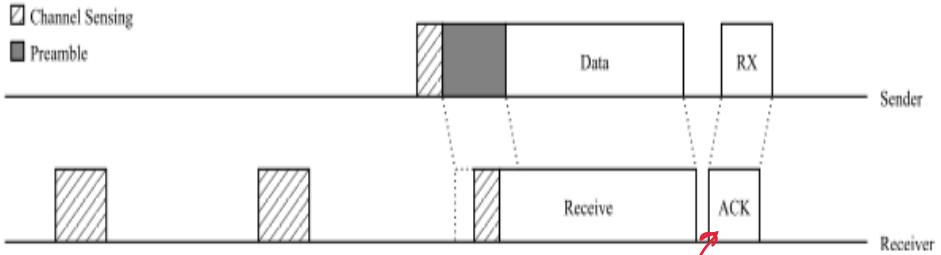
$$t_{rx} \leq nr(L_{preamble} + L_{packet}) t_{rxb}$$
$$E_{rx} = t_{rx} c_{rxb} V$$

$$E_{sample} = 17.3\mu\text{J}$$
$$t_{listen} = (t_{r_{init}} + t_{ron} + t_{rx/tx} + t_{sr}) \times \frac{1}{t_i}$$
$$E_{listen} \leq E_{sample} \times \frac{1}{t_i}$$

B-MAC's Issues

- Problem: All nodes in the vicinity of a sender wake-up and wait for the packet → overhearing
  - Solution 1: Send wake-up packets instead of preamble, wake-up packets tell when data is starting so that receiver can go back to sleep as soon as it received one wake-up packet.
  - Solution 2: Just send data several times such that receiver can tune in at any time and get tail of data first, then head.
- Communication costs are mostly paid by the sender.
  - The preamble length can be much longer than the actual data length.
- Idea: Learn wake-up schedules from neighboring nodes.
  - Start sending preamble just before intended receiver wakes up.
  - **WiseMAC** → encode wake-up pattern in the ACK message



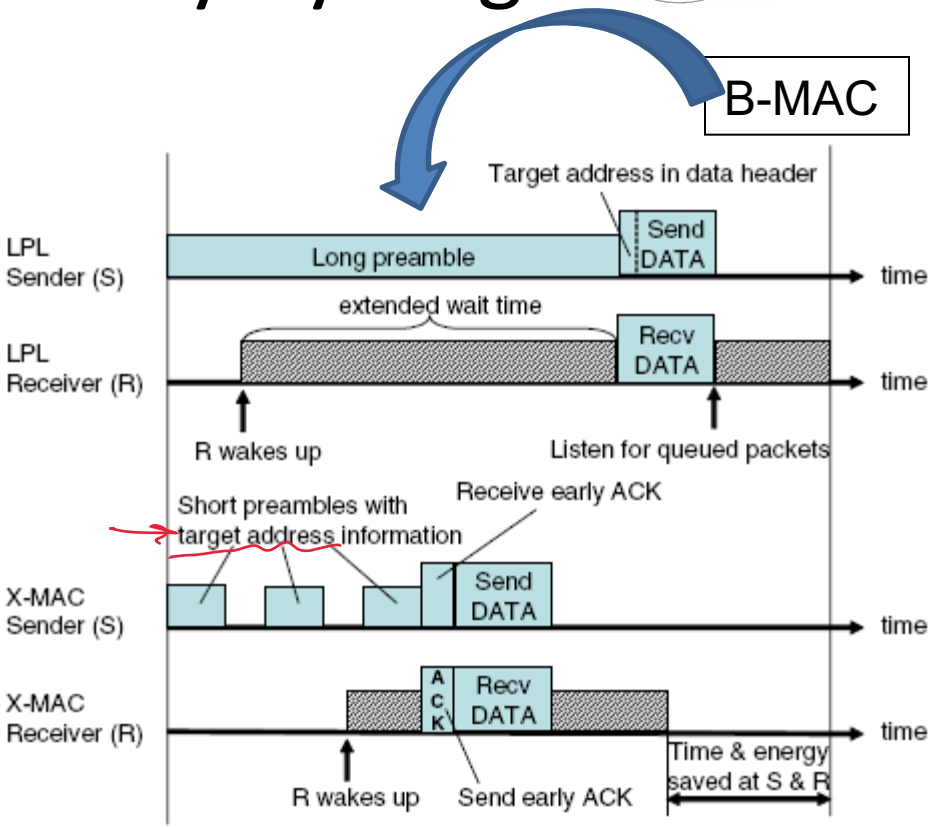


X-MAC: Principles – 2006

- Short preamble
  - Reduce latency and reduce energy consumption
- Target (destination) in preamble
  - Minimize overhearing problem.
- Strobed preamble → the preamble transmission can be interrupted.
  - Reduces latency for the case where destination is awake before preamble completes.
- Reduced per-hop latency and energy
- Dynamic duty-cycling (depending on network conditions)

X-MAC: Asynchronous Duty Cycling

- Overhearing energy loss is proportional to number of receivers in range.
- Short preamble packets.
- Non-receivers return to sleep more quickly.
- **Bad in high density sensor networks** (collisions between strobing packets)



Scheduled MAC protocols

- Idea: try to reduce the energy consumption by *coordinating* sensor nodes with a common schedule
- Advantages:
  - Saving the power from being wasted by turning off the radio outside the allocated time slot.
  - Limits collisions, idle listing, and overhearing
- Disadvantages:
  - When a sensor node enters the network, it must wait till it learns the schedule → some delay exist
  - Cost of increased number of exchanged messages
  - No flexibility to changes in sensor nodes' density or movements.
  - All sensors have to be synchronized.
- Example:
  - S-MAC (2002)
  - ContikiMAC (from 2008 on)

Sensor MAC (S-MAC) - 2002

- Tries to reduce waste of energy from all four sources of energy inefficiency
  1. Collisions – by using RTS and CTS
  2. Overhearing – by switching the radio off when transmission is not meant for that node
  3. Control Overhead – by message passing
  4. Idle listening – by periodic listen and sleep
- At the cost of:
  - some reduction in per-hop fairness and latency
  - but does not reduce end-to-end fairness and latency

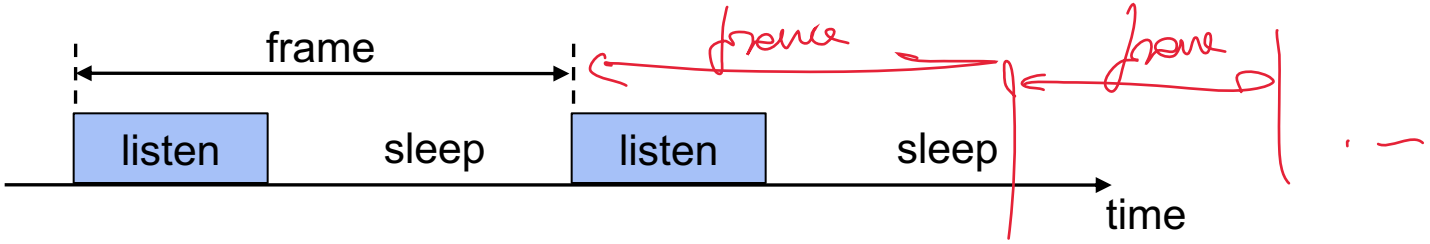
S-MAC: Assumptions

- Network assumptions:
  - Composed of many small nodes deployed in ad hoc fashion (random)
  - Most communications will be between nodes as peers, rather than to/from a single base station
  - Nodes must self-configure → sleep/wake-up schedule
- Application assumptions:
  - Dedicated to a single application or a few collaborative applications
  - Involves in-network processing to reduce traffic and increase life time
  - Applications have long idle periods and can tolerate some latency

S-MAC: Approach

- [Periodic listening and sleeping] Coarse-grained TDMA-like sleep/awake cycles:
  - the sensor node periodically goes to the fixed listen/sleep cycle.
  - A time frame in S-MAC is divided into two parts: one for a listening session and the other for a sleeping session.

increased latency



- [Message passing] All nodes choose and announce awake schedules.
  - synchronize to awake schedules of neighboring nodes.
- [Collision and overhearing avoidance] Uses RTS/CTS to resolve contention during listen intervals.
  - And allows interfering nodes to go to sleep during data exchange.

S-MAC: Possible Scenarios

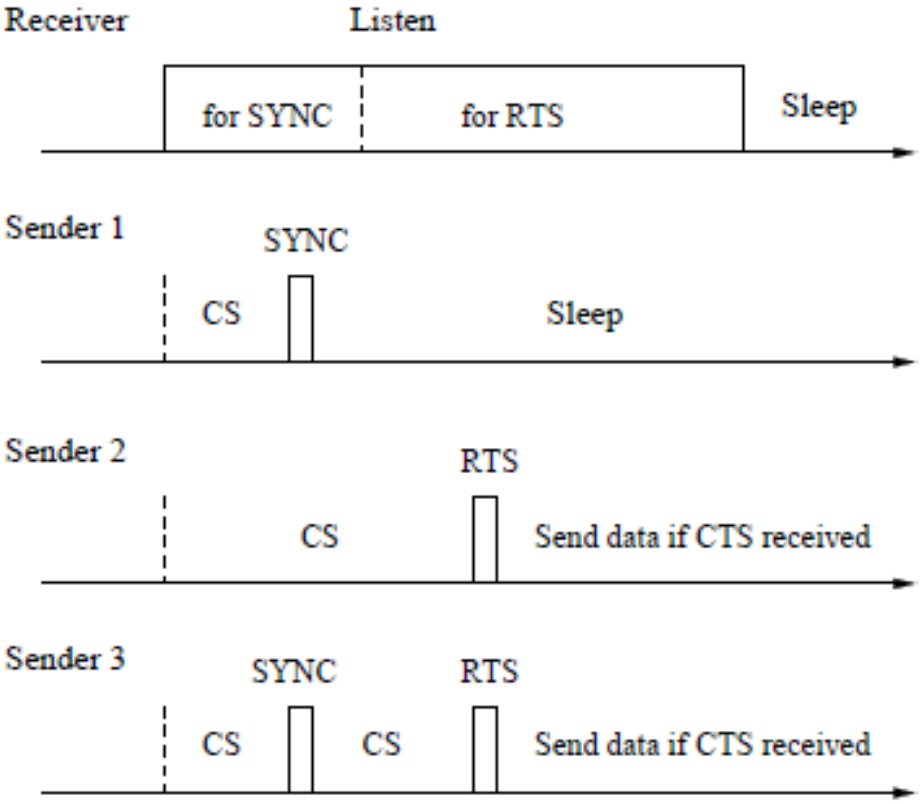
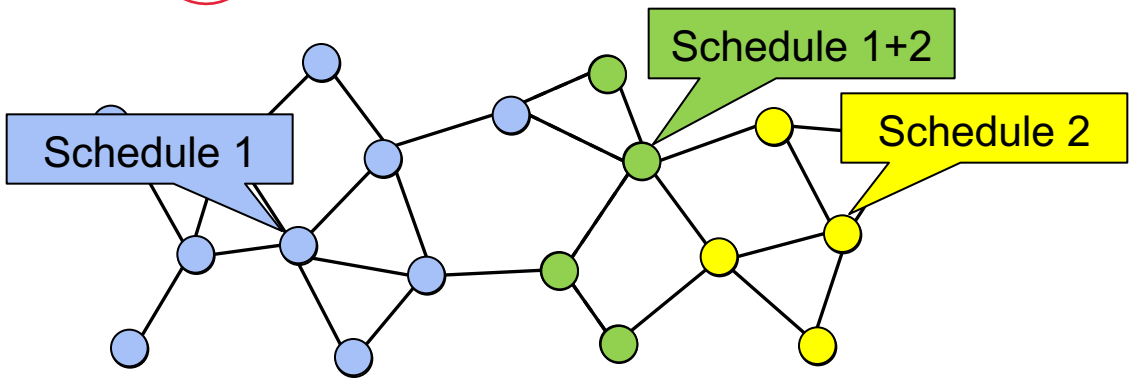


Fig. 3. Timing relationship between a receiver and different senders. CS stands for carrier sense.

Figure 3 also shows the timing relationship of three possible situations that a sender transmits to a receiver. CS stands for carrier sense. In the figure, sender 1 only sends a SYNC packet. Sender 2 only wants to send data. Sender 3 sends a SYNC packet and a RTS packet.

S-MAC: A Problem

- Nodes may have to follow multiple schedules to avoid network partition.

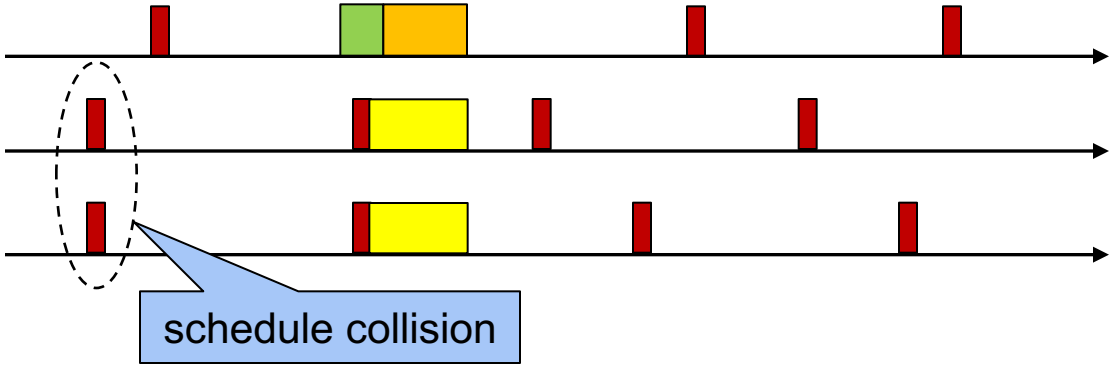


- A fixed sleep/awake ratio is not always optimal.
  - Variable load in the network.
- Idea: Adapt listen interval depending on the current network load → T-MAC

Hybrid MAC Protocols

- Protocols may use information from upper layers to further improve their performance.
  - Information about neighborhood
  - Routing policies

- Minimize costly overhearing of neighboring nodes
  - Inform them to change their channel sniff patterns



- Use randomization to resolve schedule collisions
- Example: I. Rhee, A. Warrier, M. Aia, and J. Min, "Z-MAC: a Hybrid MAC for Wireless Sensor Networks", Sensys 2005.

ContikiMAC – 2011 (Contiki 2.5)

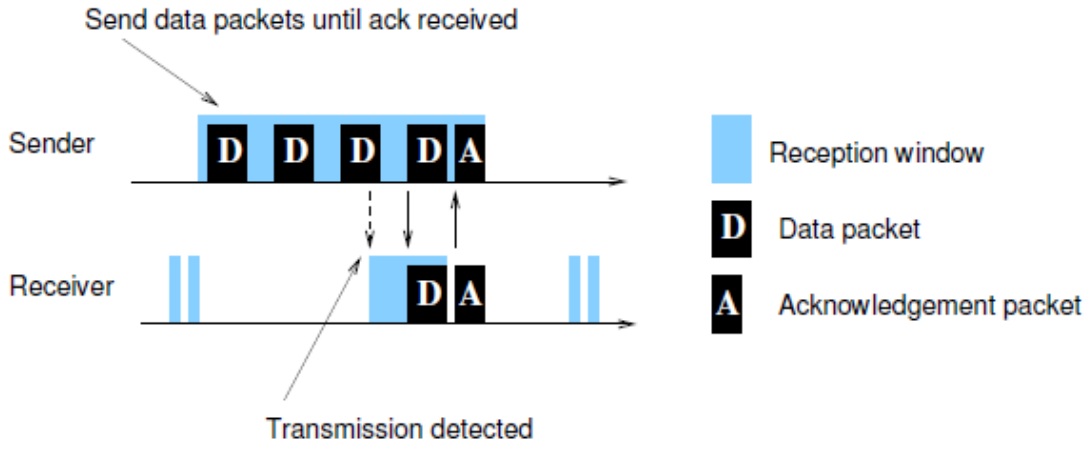


Figure 1: ContikiMAC: nodes sleep most of the time and periodically wake up to check for radio activity. If a packet transmission is detected, the receiver stays awake to receive the next packet and sends a link layer acknowledgment. To send a packet, the sender repeatedly sends the same packet until a link layer acknowledgment is received.

ContikiMAC: Broadcast

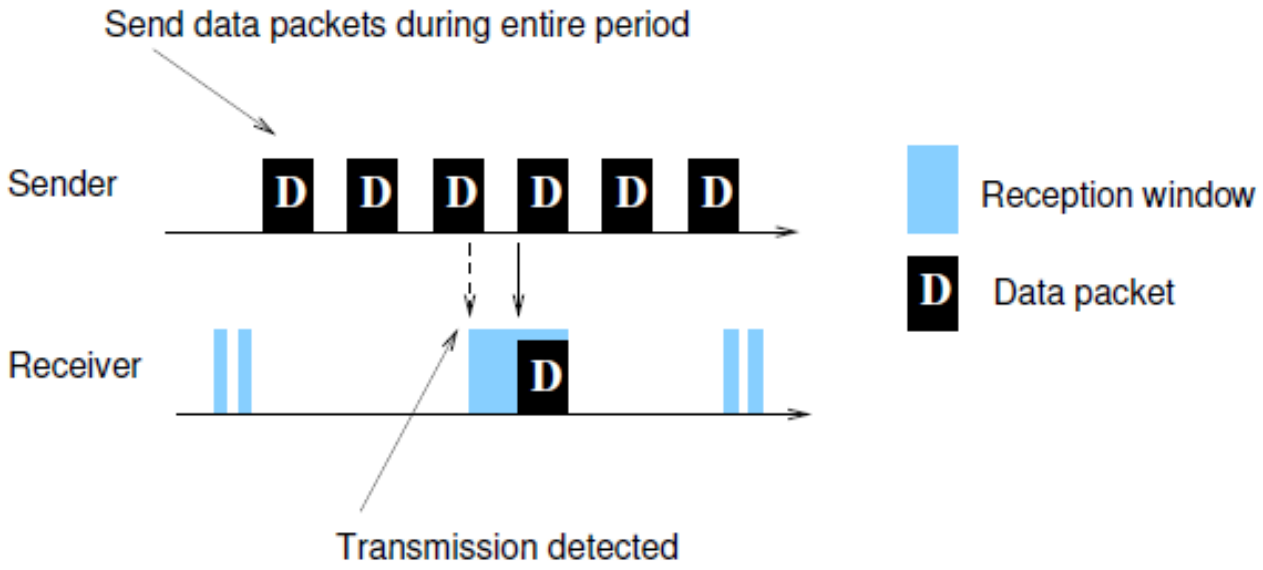


Figure 2: Broadcast transmissions are sent with repeated data packets for the full wake-up interval.

ContikiMAC: CCA timing

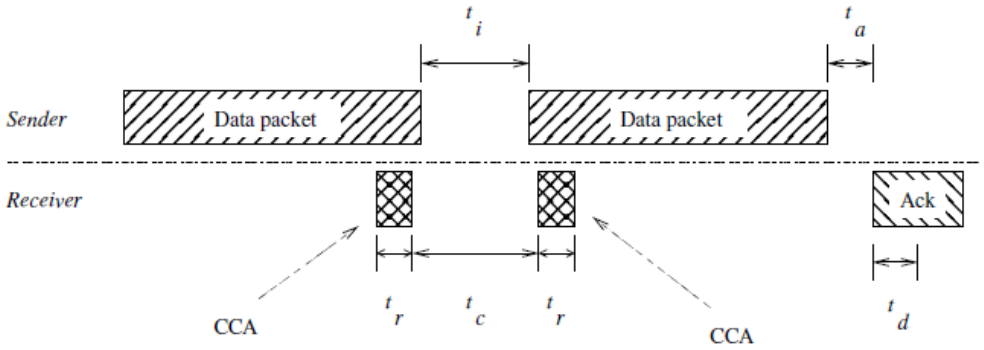


Figure 3: The ContikiMAC transmission and CCA timing.

The ContikiMAC timing is shown in Figure 3. The timing requirements from the figure are:

t_i: the interval between each packet transmission.

t_r: the time required for a stable RSSI, needed for a stable CCA indication.

t_c: the interval between each CCA.

t_a: the time between receiving a packet and sending the acknowledgment packet.

t_d: the time required for successfully detecting an acknowledgment from the receiver.

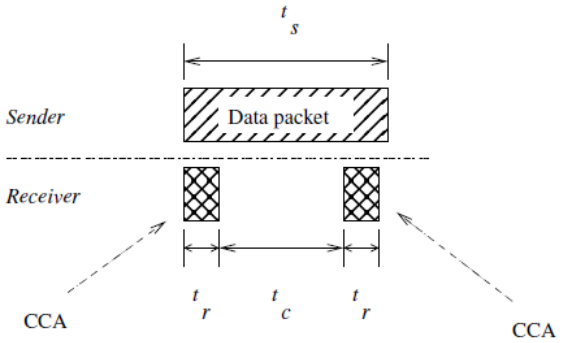


Figure 4: A packet transmission must be long enough so that it does not fall between to subsequent CCAs.

Timing constraints

$$t_a + t_d < t_i < t_c < t_c + 2t_r < t_s.$$

ContikiMAC: Transmission Phase Lock

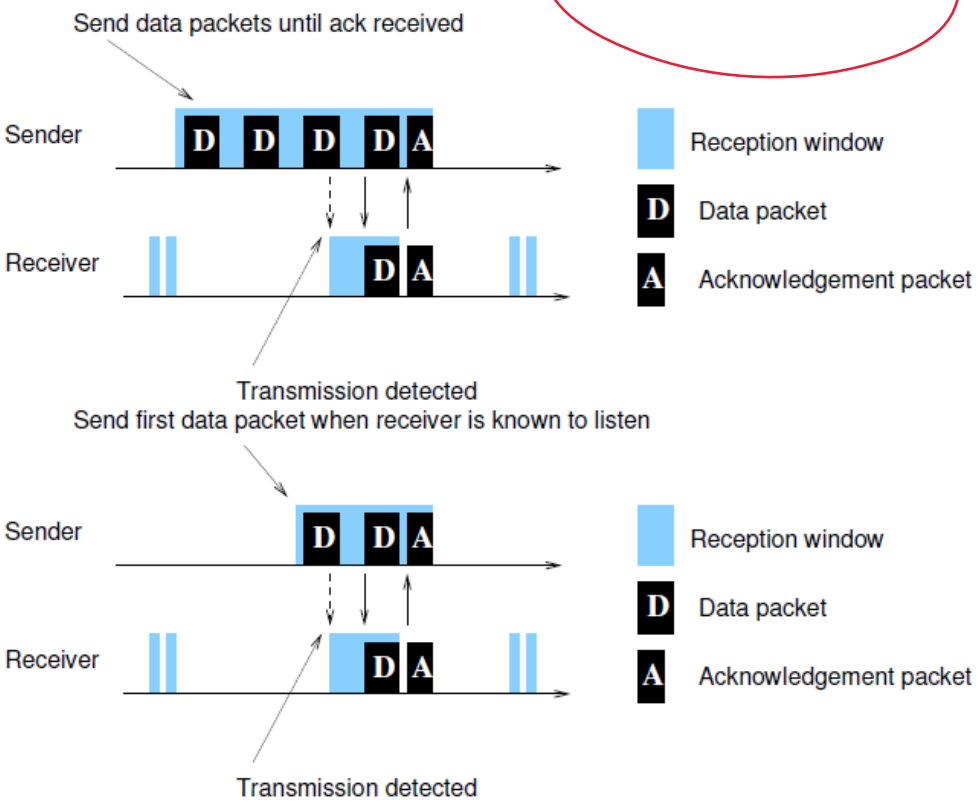


Figure 6: Transmission phase-lock: after a successful transmission, the sender has learned the wake-up phase of the receiver and subsequently needs to send fewer transmissions.

UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

IoT Security

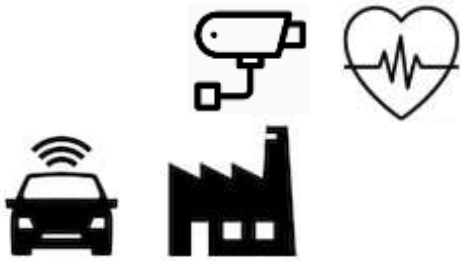
Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Internet of Things, 2025/2026

Security in IoT

- Very important (probably more than for standard Internet) due to the type and value of:
  - **information that is exchanged/stored**
  - **services that are implemented**
    - in terms of damage and/or possible downtime



- Notable IoT scenarios:
  - **Internet of Medical Things IoMT**
    - connected devices in healthcare
      - e.g. monitoring devices, robotic surgery
  - **Smart Cities**
  - **Industrial IoT (IIoT)**
    - Industry IT and OT systems are becoming more integrated an interconnected to external systems

Main issues in IoT security

- In the past most attacks concerned traditional computer systems and computer networks
  - **most of the security mechanisms and protocols has been developed for standard Internet-connected systems**
- IoT brought new security challenges
  - **presence of different highly heterogenous devices (sensors, actuators, etc.)**
  - **devices may suffer from their own vulnerabilities, especially in case of very constrained devices**
    - limited computational power, memory capabilities, communication resources, lack of a user interface
  - **use of different communication protocols, sometimes proprietary protocols designed to be used in protected networks**
  - **applications may be not well tested**
  - **difficulty to apply patches and updates impacts software life cycle**
  - **presence of different attack surfaces and vectors**
    - also for isolated networks

Attack domains

Different attack domains:

- Physical and Hardware
  - **related to the hardware used within the different systems, and may consist in the replacement, destruction, modification or exploitation of hardware components (chips, circuit boards, device ports, or other)**
    - no or limited tamper resistance policies and methodologies
- Communications
  - **exploitation of possible vulnerabilities of communication technologies (both wired and wireless) and protocols at different layers (data-link, network/routing, transport, or application)**
- Software
  - **attacks that try to exploit possible application weaknesses in design or in the implementation**

Different attack domains

- Hardware
  - **attacks related to the physical access of a system, exploiting possible weaknesses in its physical security, and related to the hardware used by different systems**
    - some devices may operate in an unattended fashion
    - may consist in the replacement, destruction, modification or exploitation of hardware components (chips, circuit boards, device ports, or other), e.g.
      - extraction of security parameters from physically unprotected devices
        - » if a group key is used and compromised this way, the whole network may be compromised as well
      - firmware replacement attack
        - » e.g. when a thing is in operation or maintenance phase, its firmware or software may be updated/replaced to allow for new functionality or new features
      - cloning of smart things by untrusted manufacturers and/or malicious substitution of things during the installation
        - » e.g. with lower-quality things, untrusted manufacturer may change functionality of the cloned thing, resulting in degraded functionality

Different attack domains (cont.)

- Communications
  - **exploitation of possible vulnerabilities of communication technologies (both wired and wireless) and protocols at different layers (data-link, network/routing, transport, or application)**
  - **these attacks may lead to data interception (steeling information), modification, communication hijacking and/or deny of service (DoS), e.g.**
    - eavesdropping attacks
      - if the communication channel (often a wireless medium) is not adequately protected
        - » keying materials or security parameters/protocols not correctly set-up
      - an attacker can also gather and track thing's information, deducing behavioral patterns
    - modification and Man-in-the-middle attacks
      - particularly during key establishment and/or device authentication procedure
        - » device authentication/authorization may be not automated and need support of a human decision process, since things usually do not have a priori knowledge about each other

Different attack domains (cont.)

- Communications (cont.)
  - routing attacks
    - routing information in IoT can be spoofed, altered, or replayed, in order to create routing loops, attract/repel network traffic, extend/ shorten source routes, etc.
    - relevant routing attacks may include:
      - » Sinkhole attack (or blackhole attack): an attacker declares himself to have a high-quality route/path to the base station, thus allowing him to do anything to all packets passing through it
      - » Selective forwarding: an attacker may selectively forward pkts or simply drop a packet
      - » Sybil attack: an attacker presents multiple identities to other things

Different attack domains (cont.)

- Communications (cont.)
  - Denial-of-Service (DoS) attacks
    - typically, things have tight memory and limited computation, they are thus vulnerable to resource exhaustion attack
      - » attackers can continuously send requests to be processed by specific things so as to deplete their resources
      - » this is especially dangerous in the IoTs since an attacker might be located in the backend and target resource-constrained devices in an LLN (Low power and Lossy network)
    - DoS attack can be launched by physically jamming the communication channel
      - » thus breaking down the T2T communication channel
    - network availability can also be disrupted by flooding the network with a large number of packets

Different attack domains (cont.)

- Software
  - **attacks that try to exploit possible application weaknesses in design or in the implementation**
    - due to very low maturity of the defined protocols and architecture
      - new software implementations poorly tested
        - » firmware are often released with known vulnerabilities (including backdoors, root users as prime access points) and lack of data encryption usage
      - weak configurations
        - » Various IoT devices have unnecessarily open ports while running vulnerable services, permitting an attacker to connect and exploit a plethora of vulnerabilities
        - » IoT operating systems and embedded firmware/software should be patched appropriately to continuously minimize attack vectors
      - malware

OWASP IoT Top 10 Vulnerabilities

1) Weak Guessable, or Hardcoded Passwords
   - **easy to bruteforce**
   - **publicly available**
     - e.g. https://defpass.com
   - **unchangeable credentials**

2) Insecure Network Services
   - **unneeded or insecure network services running on the device**
     - may be connected to the Internet
     - may lead to unauthorized access to the device

3) Insecure Ecosystem Interfaces
   - **insecure interfaces in the ecosystem outside the device: Web, backend API, cloud, or mobile interfaces**
   - **Common issues:**
     - lack of or weak authentication, authorization, encryption, lack of input and output filtering

Source: Open Web Application Security Project (OWASP), 2018

OWASP IoT Top 10 Vulnerabilities (cont.)

4) Lack of Secure Update Mechanism

- **lack of ability to securely update the device**
  - lack of firmware validation on device and/or secure delivery (un-encrypted in transit)
  - lack of anti-rollback mechanisms or notification of security changes due to updates

5) Use of Insecure or Outdated Components

- **use of deprecated or insecure software components/libraries that could allow the device to be compromised**
  - including third-party software libraries or third-party hardware components from a compromised supply chain

6) Insufficient Privacy Protection

- **user's personal information stored on the device or in the ecosystem that is used insecurely, improperly, or without permission**

OWASP IoT Top 10 Vulnerabilities (cont.)

7) Insecure Data Transfer and Storage

- **lack of encryption or access control of sensitive data anywhere within the ecosystem, including at rest, in transit, or during processing**
  - e.g. most of the times, no TLS or other security communication protocols

8) Lack of Device Management

- **lack of security support on devices deployed in production, including asset management, update management, secure decommissioning, systems monitoring, and response capabilities**

9) Insecure Default Settings

- **devices or systems shipped with insecure default settings**

10) Lack of Physical Hardening

- **allowing malicious users to gain sensitive information that can help in a future remote attack or take local control of the device**

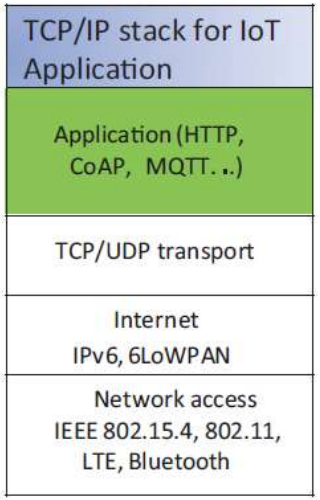
Possible countermeasures

- Countermeasures against physical threats:
  - **security policies**
  - **when possible, protect smart objects and/or apply physical hardening**
  - **safe supplying and installation measures**
    - avoiding untrusted manufacturers and installers
- Countermeasures against communication threats:
  - **security policies on which services and protocols should be used**
  - **secure communication protocols and cryptographic algorithms**
    - to enforce peer authentication/authorization, data protection (authentication/integrity, confidentiality)
    - using proper cryptographic tools
      - ciphers, hash functions, solid key management infrastructure
    - more complicated in IoT scenarios than in standard Internet
- Countermeasures against software threats:
  - **use of well-tested and updated software**
  - **security policies in all software life-cycle**
    - including software update mechanisms

IoT protocols

- A number of protocols have been developed for the IoT
  - **6LowPAN, is located between the network layer and the link layer of the OSI model**
  - **while others are at application level, such as**
    - Message Queuing Telemetry Transport (MQTT)
    - Constrained Application Protocol (CoAP)
  - **other protocols and systens spread on different layers like, e.g.**
    - LoRaWAN

- Security mechanisms and protocols may apply on different layers



Selecting security protocols

- Data authentication, integrity, and confidentiality might be provided at PH and/or at DL layer (e.g., in IEEE 802.15.4 systems), however,
  - **only hop-by-hop security**
    - end-to-end security can be guaranteed only with a high level of trust on intermediate nodes
    - difficult to guarantee due to the dynamic nature of L2 technologies and routing protocols
- For such a reason, security mechanisms at network, transport, or application level should be considered instead of (or in addition to) PH and DL level security
- Standard application protocol like MQTT, CoAP also define their security mechanisms
  - **ofthen using application level authentication and transport layer security**
    - TLS and DTLS

Selecting security protocols (cont.)

- Some important aspects:
  - **use standard secure protocol, with standard cryptographic algorithms**
    - like IPSec, TLS/DTLS
  - **possibly avoid security service duplication**
    - it has impact on the computation and transmission performance
  - **interoperability**
    - many of the Internet security protocols are independent by the used security algorithms
      - algorithms and parameters sometime are negotiated between the communication parties
    - for full interoperability, and to reduce the number of implemented cipher suites, it is important to define a set of mandatory algorithms (targeted for constrained environments) that all objects must implement for minimal support
      - however it is not easy to agree on a set of common protocols and algorithms that can be supported by all devices

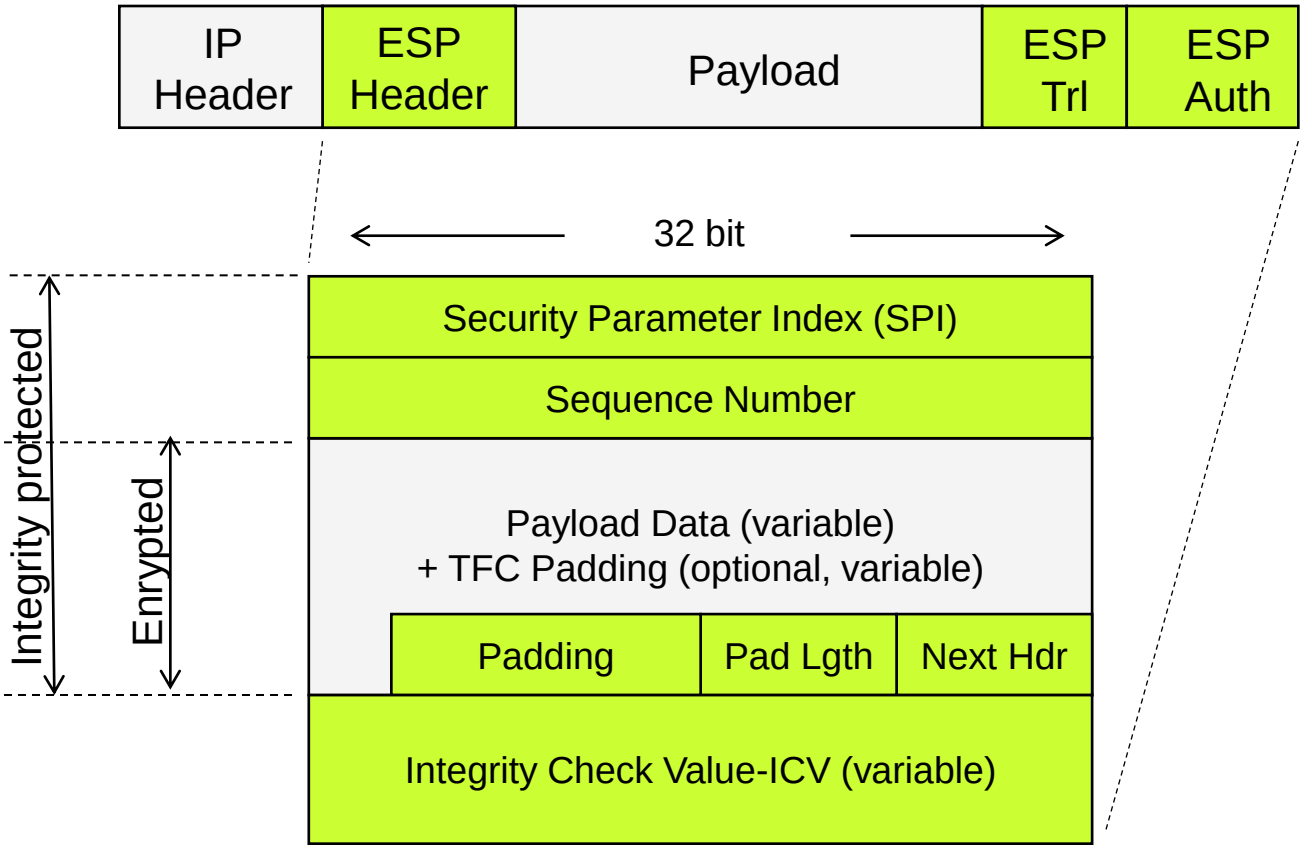
IPSec

- At network layer, an IoT node can secure data exchange in a standard way by using the Internet Protocol Security (IPSec)
- IPSec (RFC 4301) is the Security Architecture for the IP protocol
  - **defined for both IPv4 and IPv6**
    - it was originally developed for IPv6 but found widespread deployment, first, as an extension in IPv4, into which it was back-engineered
    - it was an integral part of the base IPv6 protocol suite, but has been made optional
- IPSec can provide:
  - **confidentiality**
  - **integrity**
  - **data-origin authentication**
  - **protection against replay attacks**

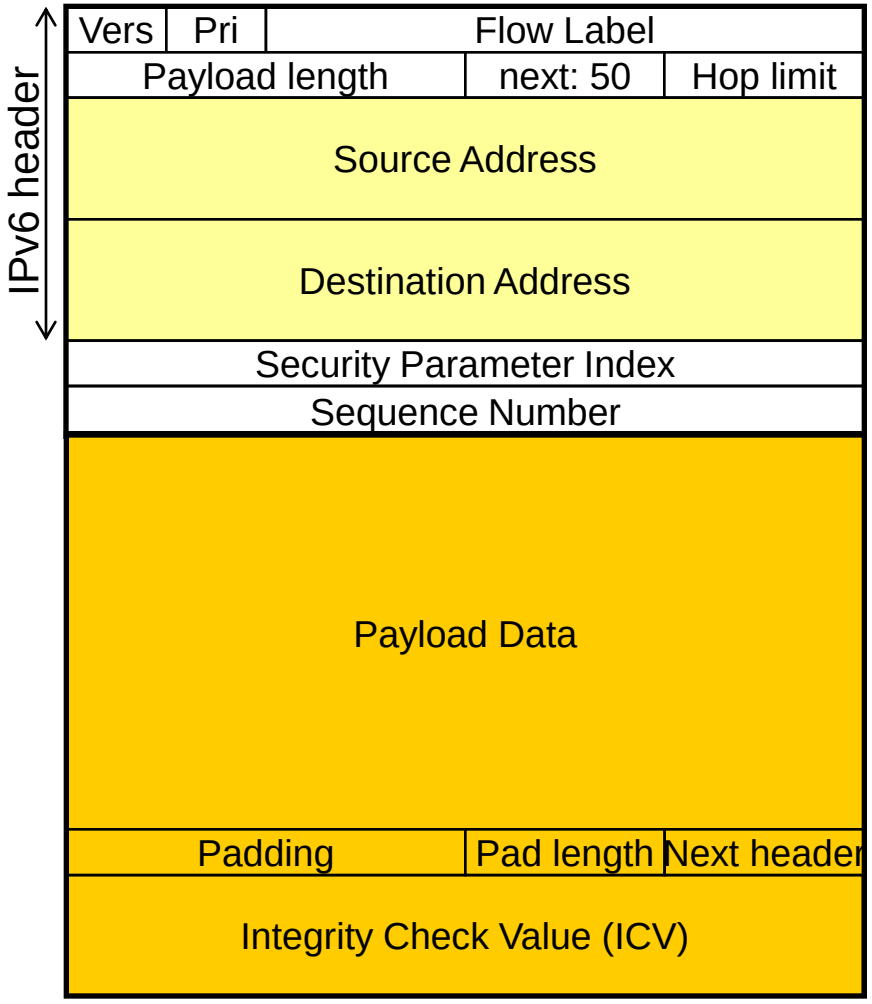
IPSec protocols

- Such security services are implemented by two IPSec security protocols:
  - **Authentication Header (AH)**
    - provides data-origin authentication, integrity, and optionally anti-replay
  - **Encapsulated Security Payload (ESP)**
    - provides confidentiality, data-origin authentication, integrity, and anti-replay capabilities
    - usually preferred since it is all-in-one solution
      - respect to a combination of AH+ESP
- In order to minimize the code, only ESP is implemented and used

IPSec ESP packet format



IPSec ESP packet format (cont.)



- Security Parameter Index (SPI)
- Sequence number
- Payload Data
  - **may including the cipher Initialization Vector**
  - **IPsec implementation can add also Traffic Flow Confidentiality (TFC) padding**
- Padding and Pad length
  - **If the encryption algorithm requires the plaintext to be a multiple of some number of bytes, e.g., the block size of a block cipher**
  - **or to ensure that the resulting ciphertext terminates on a 4-byte boundary**
- Next header
  - **payload type**
- Integrity Check Value (ICV)

IPSec (cont.)

- IPSec ESP defines only the way IPSec control information are encapsulated
  - **the effective security algorithms are specified separately algorithms can be selected amongst a set of available cipher suites**
  - **this makes IPSec usable also in the presence of very resource-constrained devices, if a proper algorithm that guarantees both usability and sufficient security level is selected**
- The keying material and the selected cryptographic algorithms used by IPSec for securing a communication (Security Association) can be:
  - **pre-configured (specifying a priori a pre-shared key, hash function and encryption algorithm) or**
  - **dynamically negotiated by the IKE protocol**
- Unfortunately the IKE protocol uses asymmetric cryptography, which may be computationally too heavy for very small devices
  - **proper IKE extensions may be considered using lighter algorithms**

Transport-level security

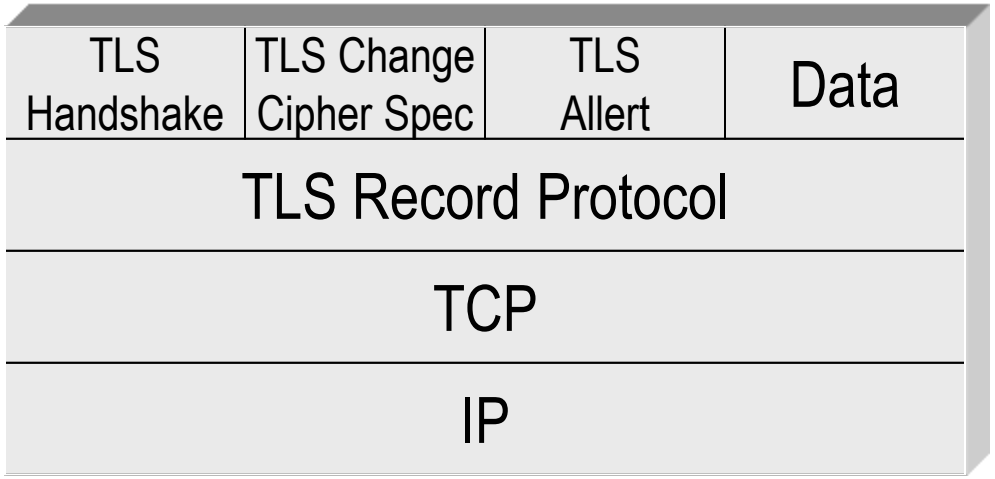
- Similar to the standard Internet, data exchange between applications can be secured at transport layer through
  - **Transport Layer Security (TLS)**
    - widest used secure protocol, running on top of the TCP providing to the application layer the same connection and stream-oriented interface of TCP
  - **Datagram Transport Layer Security (DTLS)**
    - introduced in order to provide a security service similar to TLS on top of UDP
    - can be used to secure datagram traffic for client/server applications

Transport Layer Security (TLS)

- Transport Layer Security (TLS) protocol (version 1.3, 2018)
  - **widest used secure protocol, running on top of the TCP**
  - **same connection and stream-oriented interface of TCP**
- Provides privacy and data integrity between two communicating client/server applications
  - **peer's identity authentication**
    - using asymmetric cryptography (e.g., RSA, DSA, etc.)
  - **negotiation of encryption algorithm and key exchange**
    - protected against eavesdropping, modifying and replay attacks
  - **confidentiality**
    - symmetric cryptography is used
  - **data authentication and integrity**
    - message transport includes a message integrity check using a keyed MAC (e.g. based on SHA-1 hash)
- Uses X.509 certificates

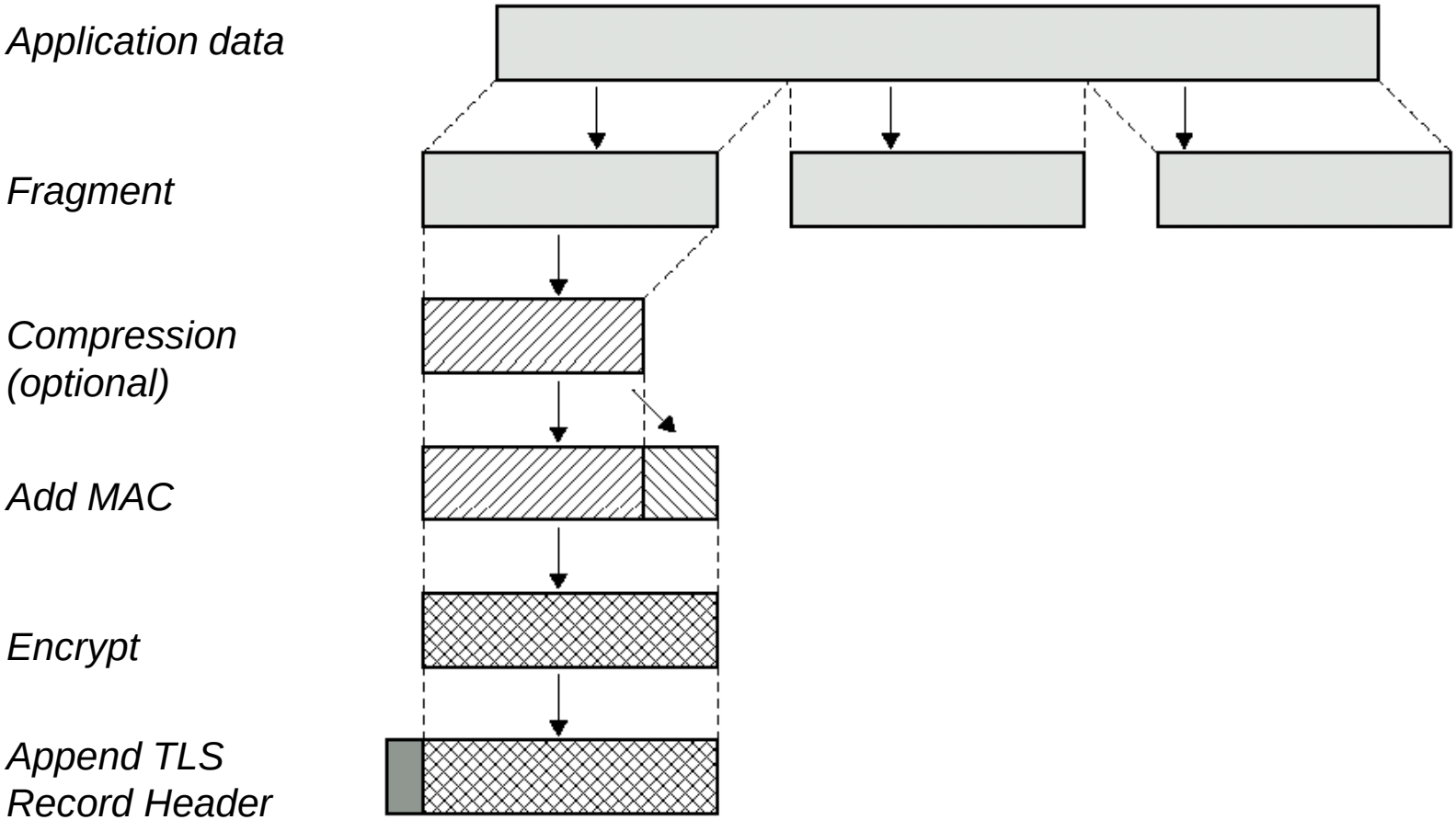
TLS protocol stack

- The protocol is composed of two layers:
  - **TLS Handshake Protocol**
  - **TLS Record Protocol**



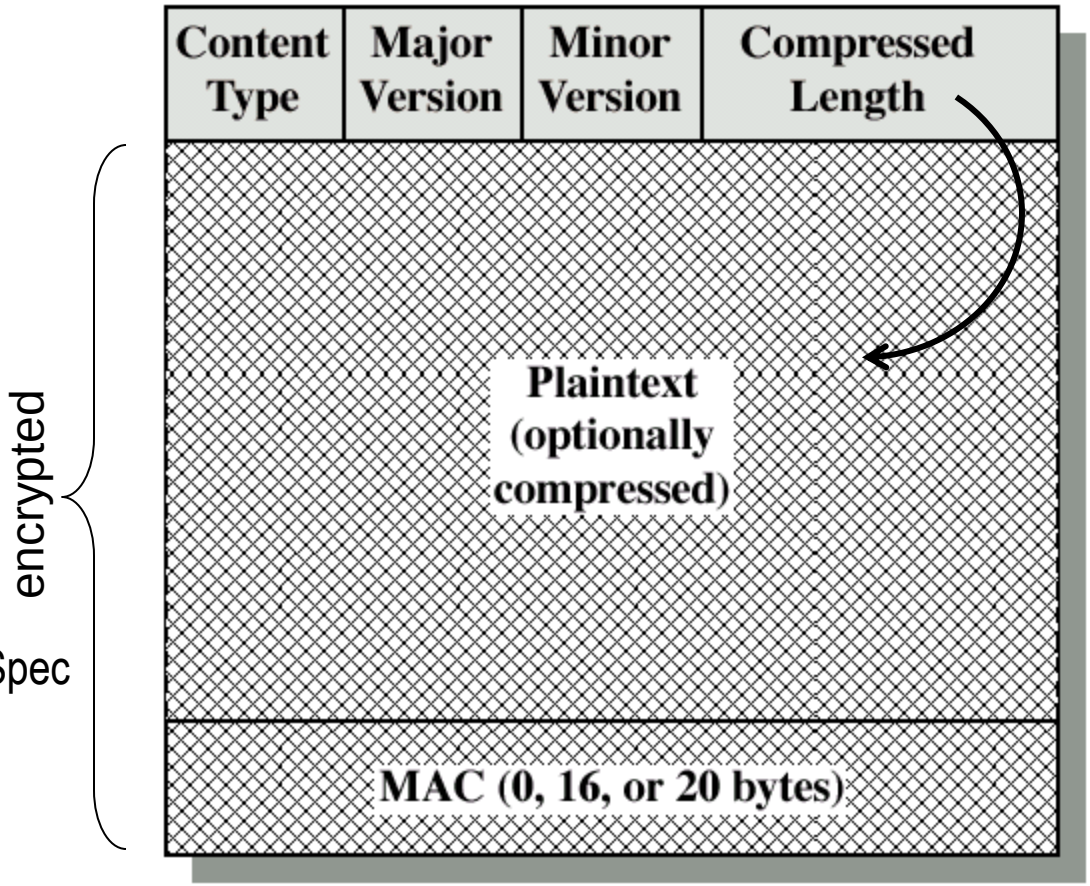
TLS Record Protocol (cont.)

- TLS-RP operation:



TLS Record Protocol (cont.)

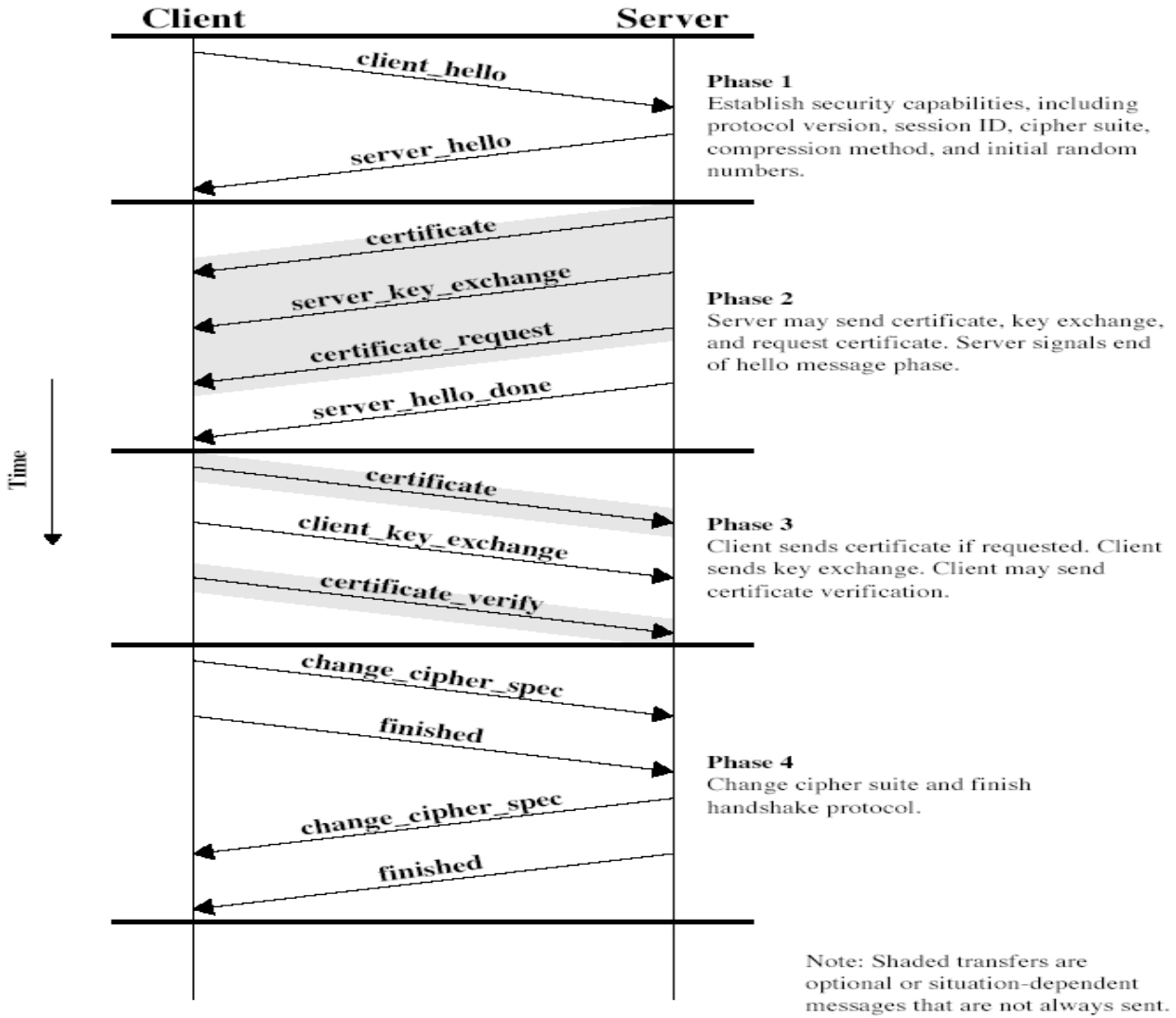
- TLS-RP format
  - **Major Version=3**
  - **Minor Version:**
    - 0 = SSL3.0
    - 1 = TLS1.0
    - 2 = TLS1.1
    - 3 = TLS1.2
    - 4 = TLS1.3
  - **Content-type:**
    - 20 = Change Cihper Spec
    - 21 = Alert
    - 22 = Handshake
    - 23 = Data



TLS Handshake Protocol

- Peer-entity authentication and key exchange is provided by the TLS Handshake phase, performed at the beginning of the communication
- TLS Handshake Protocol allows the server and client to authenticate each other and to negotiate an encryption algorithm and cryptographic keys before the application protocol transmits or receives its first byte of data
  - **the peer's identity can be authenticated using asymmetric, or public key, cryptography (e.g., RSA, DSA, etc.)**
  - **secure negotiation of a shared secret**
    - unavailable to eavesdroppers, or to attackers who can place themselves in the middle of the connection

TLS Handshake Protocol (cont.)



TLS and UDP

- TLS cannot be directly used with UDP, since packets may be lost or reordered (unreliability)
  - **TLS does not allow independent decryption of individual records**
    - the integrity check depends on the sequence number, if record N is not received, then the integrity check on record N+1 will be based on the wrong sequence number and thus will fail
    - the symmetric cipher may also depend on the correct sequence of records
  - **TLS handshake layer assumes that handshake messages are delivered reliably**
    - breaks if those messages are lost

Datagram Transport Layer Security (DTLS)

- Datagram Transport Layer Security (DTLS) Version 1.3
  - **TLS over datagram transport (UDP)**
  - **introduced to provide a security service similar to TLS on top of UDP**
  - **based on TLS**
    - it makes only the minimal changes to TLS required to fix the problems
- Although it is still poorly supported in standard Internet nodes, it is currently the reference security protocol for some IoT protocols and applications
  - **it uses UDP as transport**
  - **It does not suffer from the problems originated by the use of TCP in network-constrained scenarios due to the extremely variable transmission delay and lossy links**

DTLS vs. IPSec

- IPSec and DTLS provide same security features with their own mechanisms. at different stack layers
  - **also the IPSec IKE key agreement reflects almost the same TLS/DTLS handshake function**
- The main advantage of securing communications at transport layer with DTLS consists in allowing applications to directly and easily select which, if any, security service has to be set up
  - **another practical advantage is that the adoption of DTLS can allow for the reuse of the large experience and implementations that came with TLS**
- DTLS has recently received significant attention for constrained networked applications
  - **it has been standardized as the security protocol for CoAP associated to “coaps” URIs**

Hop-by-hop and E2E security

- Providing security at IP layer (through IPSec) or transport layer (through TLS or DTLS) has some advantages
  - **the same standard mechanism and the same implementation can be shared by all applications, resulting in code reuse and reduced code size**
  - **programmers do not have to deal with the implementation of any security mechanism**

Hop-by-hop and E2E security (cont.)

- From the security point of view, one problem of using DTLS or IPSec is that end-to-end (E2E) communication is not guaranteed when intermediate nodes such as proxies or application level gateways are introduced
  - **they provide secure communications hop-by-hop, at IP and transport layers**
  - **in presence of application intermediate nodes, they can assure security only within each hop**
  - **the presence of intermediate nodes may arise for**
    - application reasons
      - caching, data aggregation/processing, application level routing
        - » e.g. IoT gateway, MQTT broker, LoRaWAN App Server (possibly operated by a third party entity)
    - security reasons
      - as security gateway acting as access controller and granting access to the internal network only to trusted nodes
        - » may protect the constrained nodes against flooding or replay attacks carried on by full-powered host attached to the Internet

Hop-by-hop and E2E security (cont.)

- When communication is relayed at application level by intermediate nodes, end-to-end security is guaranteed through transport-level or IP-level mechanisms only if:
  - **communication is secured on each hop**
  - **all intermediate systems are considered trusted**
    - requires additional trust relationships

Application-level security

- A different approach for providing complete end-to-end security is to enforce security directly at application level
  - **simplifies the requirements for underlying layers, and probably reduces the cost, in term of packet size and data processing**
    - since only application data have to be secured and per-data and not per-packet overhead is introduced
  - **multicast communications, and in-network data aggregation is easier to be implemented at application level**
- The main disadvantages are:
  - **the complications introduced for application development**
  - **the overall code size due to poor reuse of software codes**
- These disadvantages are mainly due to the lack of well defined and adopted secure protocols at application level
  - **examples of standards that can be used for this purpose are S/MIME, SRTP, CBOR+COSE**

CBOR

- Concise Binary Object Representation (CBOR)
  - **RFC 8949 (2020, previous RFC in 2013)**
- Data format for binary encoding structured data, designed for
  - **small size**
  - **fast parsing**
  - **extensibility**
- Underlying data model is an extended version of JSON
  - **but compact and machine-efficient**
  - **supports integers, floats, strings, arrays, maps, binary data**
- Deterministic encoding
  - **important for security**
- Alternative of earlier binary serializations
  - such as ASN.1 (with BER and DER), e.g. encoding [1, [2, 3]]
    - ASN.1 BER: 30 0b 02 01 01 30 06 02 01 02 02 01 03
    - CBOR: 82 01 82 02 03

–

Example: JSON vs CBOR

- JSON:

```
{"temp":22,"humidity":48,"battery":2980}
```

  - size= 40 B

- CBOR:

```
A3                                  # map(3)
   64                               # text(4)
      74 65 6D 70                   # "temp"
   16                               # unsigned(22)
   68                               # text(8)
      68 75 6D 69 64 69 74 79       # "humidity"
   18 30                            # unsigned(48)
   67                               # text(7)
      62 61 74 74 65 72 79          # "battery"
   19 0B A4                         # unsigned(2980)
```

  - size= 29 B

- CBOR with numeric keys: `{1:22, 2:48, 3:2980}`

```
A3
01 16       # 1 → 22
02 18 30    # 2 → 48
03 19 0B A4 # 3 → 2980
```

  - size= 10 B

CBOR encoding

- Every CBOR data item starts with an initial byte:
  - **3 bits: Major Type**
    - defines what kind of data follows
      - 0 Unsigned integer
      - 1 Negative integer
      - 2 Byte string (binary data)
      - 3 Text string (UTF-8)
      - 4 Array
      - 5 Map (key-value pairs)
      - 6 Tag (semantic annotation)
      - 7 Floating point / simple values
  - **5 bits: Additional Information**
    - indicates either:
      - the value itself (if small), or
      - how many extra bytes follow

CBOR encoding (cont.)

- CBOR encodes each piece of data as a byte sequence where the first byte tells you the type and size
- Encoding process:
  1) **take the data structure (e.g., JSON)**
  2) **for each element:**
     - write type + length/value
     - append extra bytes if needed
  3) **concatenate everything into a byte stream**

CBOR in IoT Protocols

- Used in:
  - **CoAP (Constrained Application Protocol)**
    - Object Security for Constrained RESTful Environments (OSCORE)
  - **LwM2M (Lightweight M2M)**
    - from the Open Mobile Alliance
  - **Matter (smart home)**
    - from Connectivity Standards Alliance

- Often combined with:
  - **COSE (CBOR Object Signing and Encryption)**

COSE

- CBOR Object Signing and Encryption
  - **RFC 8949 (2020)**
  - **specified how to create and process secure messages using CBOR**
    - analogous to JOSE (Javascript Object Signing and Encryption) with JWT, JWS, JWE) but binary
- Supports:
  - **encryption**
  - **digital signatures**
  - **message authentication (MAC)**

COSE messages

- Most COSE messages follow this pattern:

  [protected headers,
  unprotected headers,
  payload / ciphertext,
  signature / tag]

  - **where:**
    - protected header: CBOR map, integrity protected
      - Typically contains:
        - » algorithm (e.g., AES-GCM, EdDSA)
        - » key ID
    - unprotected headers: CBOR map, used for non-critical metadata
    - payload: plaintext or encrypted
    - signature / tag: ensures authenticity (authentication and integrity)
        - » signature → digital signature
        - » encryption → AEAD tag (e.g., GCM)

Main COSE message types

- COSE_Encrypt / COSE_Encrypt0
  - **for encryption with a AEAD algorithm (e.g., AES-GCM, AES-CCM)**
  - **structure**

    COSE_Encrypt0 = [
    protected,
    unprotected,
    ciphertext
    ]
- COSE_Sign / COSE_Sign1
  - **for digital signature**
  - **structure**

    COSE_Sign1 = [
    protected,
    unprotected,
    payload,
    signature
    ]
- COSE_Mac / COSE_Mac0
  - **for MAC (shared key)**
  - **similar to encryption, but no confidentiality**

OSCORE

- Object Security for Constrained RESTful Environments (OSCORE)
  - **RFC 8613 (2019)**
- Method for application-layer protection of CoAP
  - **provides end-to-end protection between endpoints**
    - designed for constrained nodes and networks supporting a range of proxy operations
    - it uses CBOR+COSE

OSCORE (cont.)

- OSCORE transforms a CoAP message

  [CoAP headers] [CoAP options] [Payload]

  - **into**

    [Outer message] [OSCORE option] [Encrypted inner message]

  - **where**
    - Outer message (visible, partially authenticated):
      - dummy CoAP code (for keep CoAP semantic, POST/Changed)
      - minimal routing info
        - » message ID, token, some options (e.g., Proxy-Uri)
    - Inner message (encrypted + authenticated):
      - CoAP code (GET, POST, etc.)
      - sensitive options (e.g. Uri-Path)
      - payload

Key Agreement and Distribution

- Key distribution and management is a crucial issue that needs to be addressed when security mechanisms have to be adopted
- Asymmetric (public-key) cryptographic algorithms are often the basis for key agreement protocols
  - e.g. Diffie–Hellman
- In IoT environments particular attention must be paid, due to processing constrains
  - **type of used algorithm**
  - **frequency of key exchanges**
- Alternative approaches may involve
  - **key pre-distribution**
  - **use of a third party Key Distribution Center (KDC)**

Security bootstrapping

- All key agreement protocols require that some credentials have been previously installed and configured on nodes, so that the key agreement procedure can occur securely
- Bootstrapping refers to the processing operations required before the network can operate
  - **it is a very important phase in the lifecycle of smart objects and can affect the way they behave in operational conditions**
  - **in smart objects it can be complicated by the absence of UIs**
  - **in this case, the bootstrapping may exploit:**
    - hardware pre-configured settings
    - physical environment where the node is bootstrapping or operates

Lightweight Cryptography

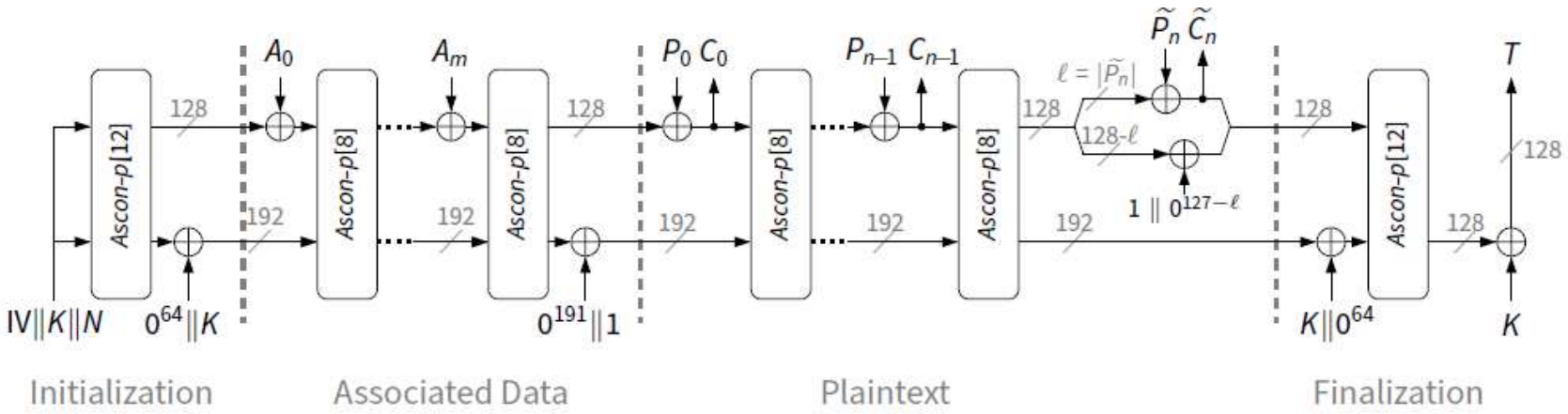
- Smart objects may feature constrained resources, such as low computation capabilities, little memory, and limited battery
- Security mechanism/algorithms must necessarily take into account all these limitations
- Lightweight Cryptography (LWC) aims at the design of ciphers that might meet the requirements set by the use of smart objects
  - **cryptography tailored to (extremely) constrained devices**
  - **algorithms with smaller footprint, low energy consumption, and low computational power needs**
- “Lightweight" should not be mistaken with weak (in terms of cryptographic protection)
  - **aim at providing sufficient security in the environment of restricted resources as can be found in many ubiquitous devices**
  - **trade-offs between security level, cost, and performance**
- Not intended for all-powerful adversaries

NIST lightweight cryptography standard

- NIST published a call for cryptography algorithms for constrained environment, in 2018
  - **received 57 submissions**
  - **10 finalists**
  - **in February 2023 the selection of the Ascon algorithm family**
  - **in November 2024 released the initial public draft**
- Family of symmetric-key cryptographic primitives
  - **Authenticated Encryption with Associated Data (AEAD): Ascon-AEAD128**
  - **Hash: Ascon-Hash256**
  - **Extendable Output Function (XOF): Ascon-XOF128, and Ascon-CXOF128**
- Characterized by lightweight permutation-based primitives
- Viable alternative when the Advanced Encryption Standard (AES) may not perform optimally

Ascon-AEAD128

- Ascon-AEAD128 structure
  - **comprises four phases: initialization, associated data processing, plaintext processing, and finalization**



Energy consumption of asymmetric algorithms

- Energy cost of digital signature algorithms:

Algorithm	Key size ($bits$)	Key generation (mJ)	Sign (mJ)	Verify (mJ)
RSA	1,024	270.13	546.50	15.97
DSA	1,024	293.20	313.60	338.02
ECDSA	163	226.65	134.20	196.23
ECDSA	193	281.65	166.75	243.84
ECDSA	233	323.30	191.37	279.82
ECDSA	283	504.96	298.86	437.00
ECDSA	409	1034.92	611.40	895.98

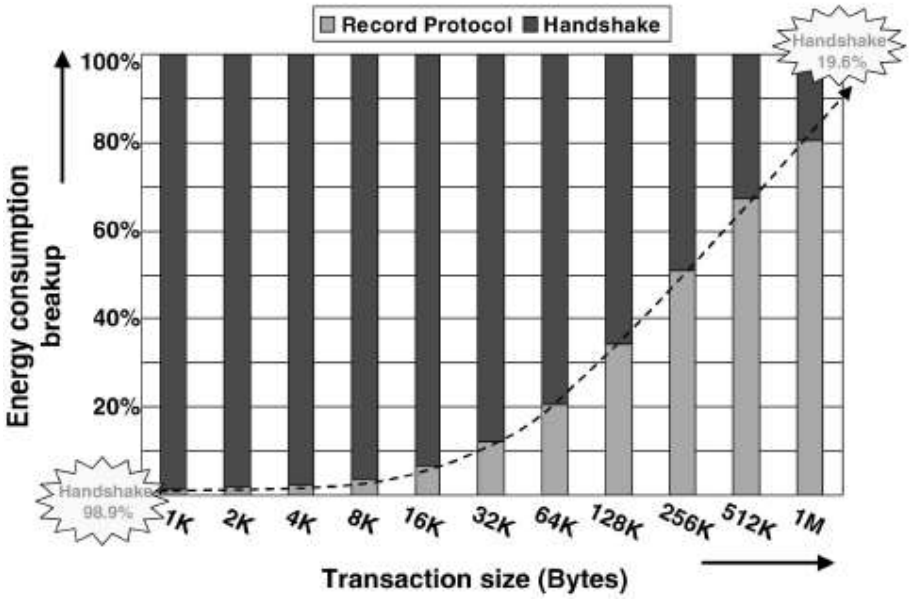
- Energy cost of key exchange algorithms:

Algorithm	Key size ($bits$)	Key generation (mJ)	Key exchange (mJ)
DH	1,024	875.96	1,046.5
ECDH	163	276.70	163.5
DH	512	202.56	159.6

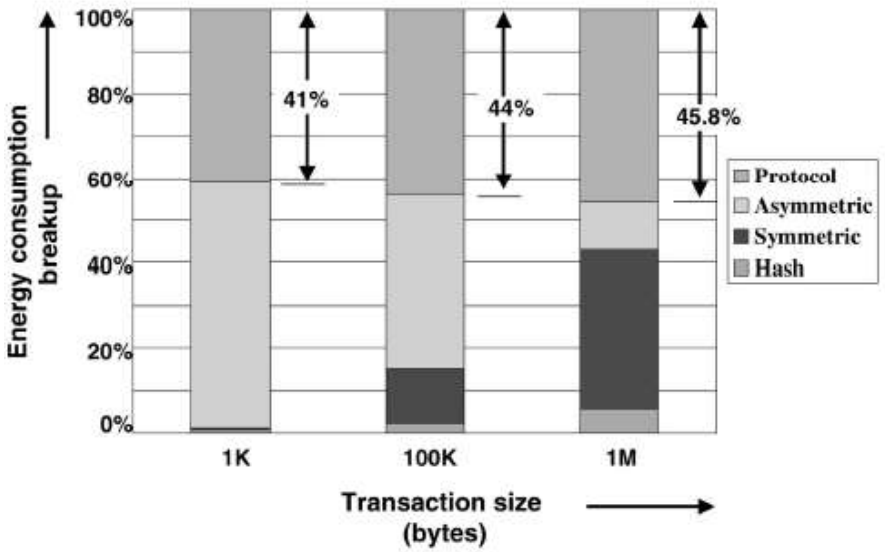
[4] N. R. Potlapally, S. Ravi, A. Raghunathan, and N. K. Jha, "A Study of the Energy Consumption Characteristics of Cryptographic Algorithms and Security Protocols", IEEE Transactions on Mobile Computing, vol. 5, no. 2, February 2006

TLS energy consumption (cont.)

- Handshake vs. RP energy consumption:



- TLS energy consumption into cryptographic and non-cryptographic components:



[4] N. R. Potlapally, S. Ravi, A. Raghunathan, and N. K. Jha, "A Study of the Energy Consumption Characteristics of Cryptographic Algorithms and Security Protocols", IEEE Transactions on Mobile Computing, vol. 5, no. 2, February 2006

Overview on LoRa/LoRaWAN

Antonio Cilfone

Team Leader | Software Engineer, Tesmec Automation S.r.l.

April 22, 2026

Internet of Things, Communication Engineering's Course

Automation is not what you think about...

- Software development in **C** / **C++** for electrical automation systems
- Linux kernel development and cybersecurity work
- Focus on critical infrastructure and energy networks

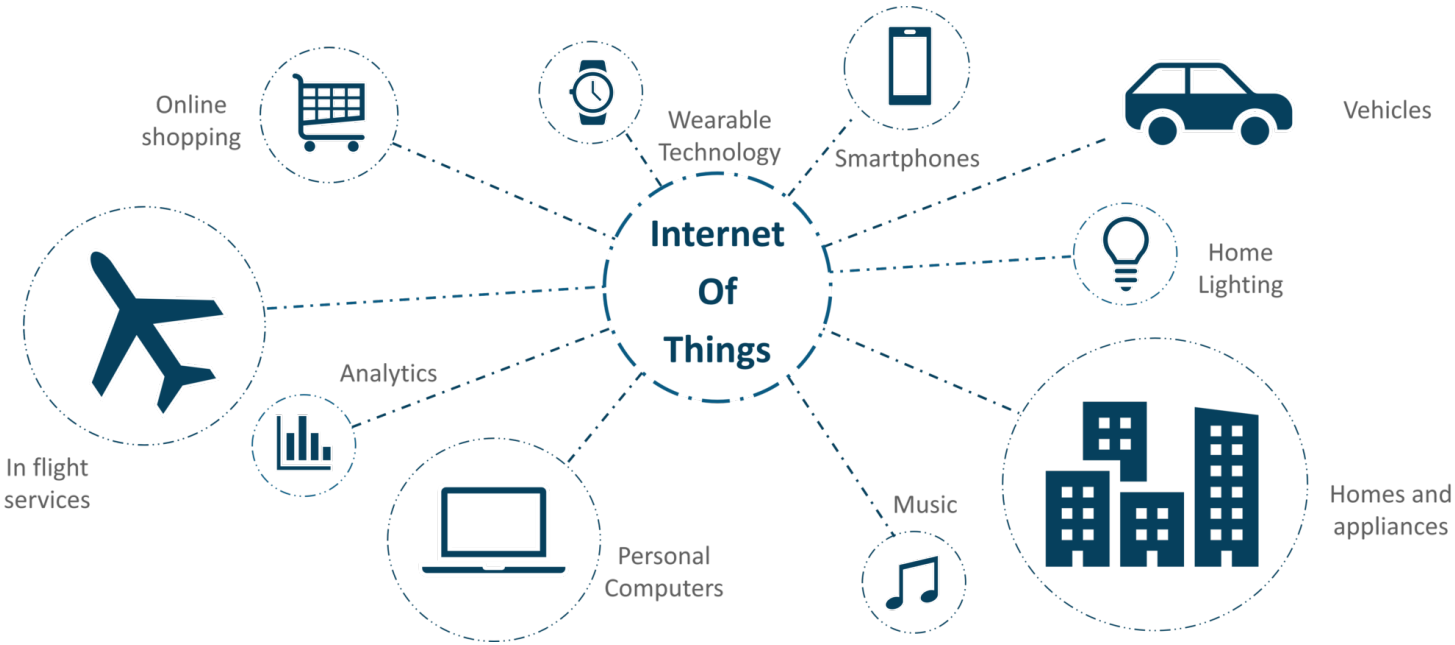
- Systems deployed for:
  - **Terna, Enel** (Italy)
  - **Enedis, RTE** (France)
  - **IPTO (i.e., ADMIE)** (Greece)

- Domain:
  - substation automation
  - protection and control systems
  - real-time industrial communication

Agenda

1. Low Power Wide Area Networks (LPWANs)
2. Use cases
3. LoRa
4. LoRaWAN
5. Current and Future development

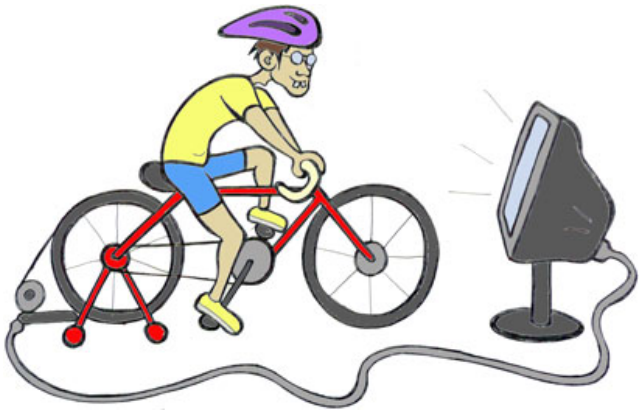
IoT applications



IoT $\rightarrow$ Connecting embedded devices

Challenges at the edge

Battery

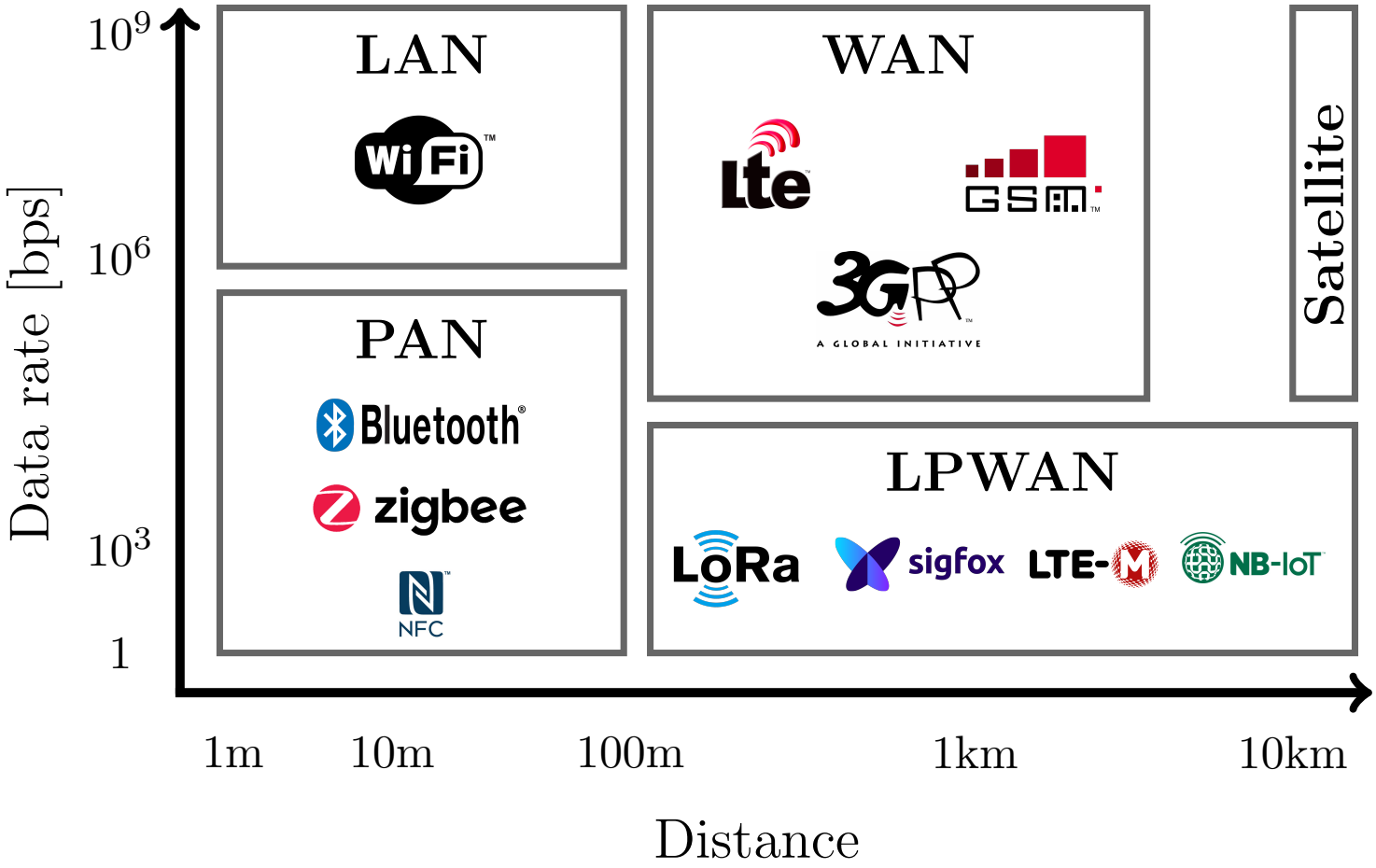


Good range

Security

Cost

Current technologies



Cellular technologies

Cellular networks are widely used for IoT (e.g. home alarm systems)

- Power consumption can be high
- Private deployments require spectrum/licenses

What about 2G?

Ubiquitous, low-cost, energy-aware $\rightarrow$ historically good fit for IoT

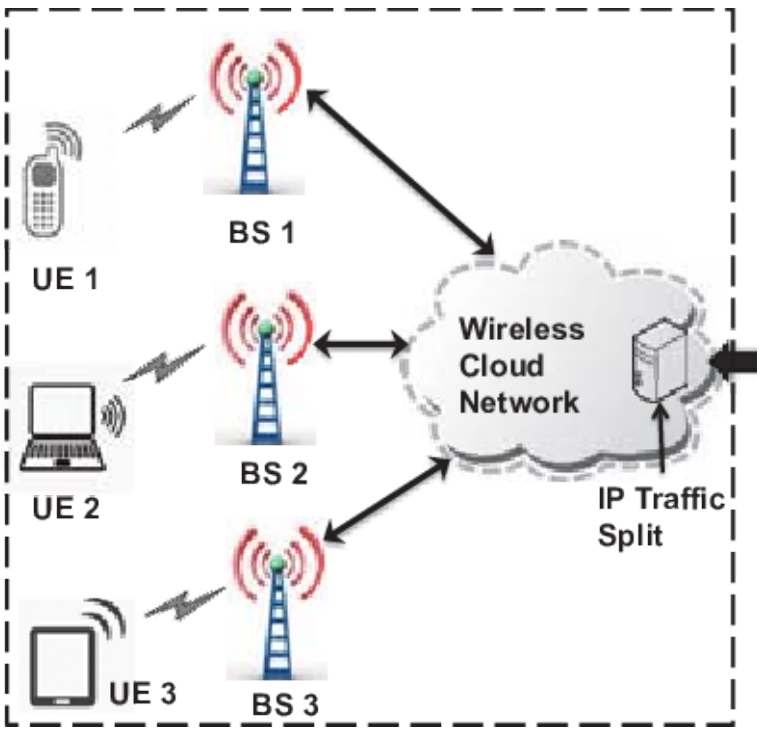
- But legacy networks are disappearing:
  - 2G: already shut down in several countries (e.g. US, Australia)
  - 3G: shut down in Italy

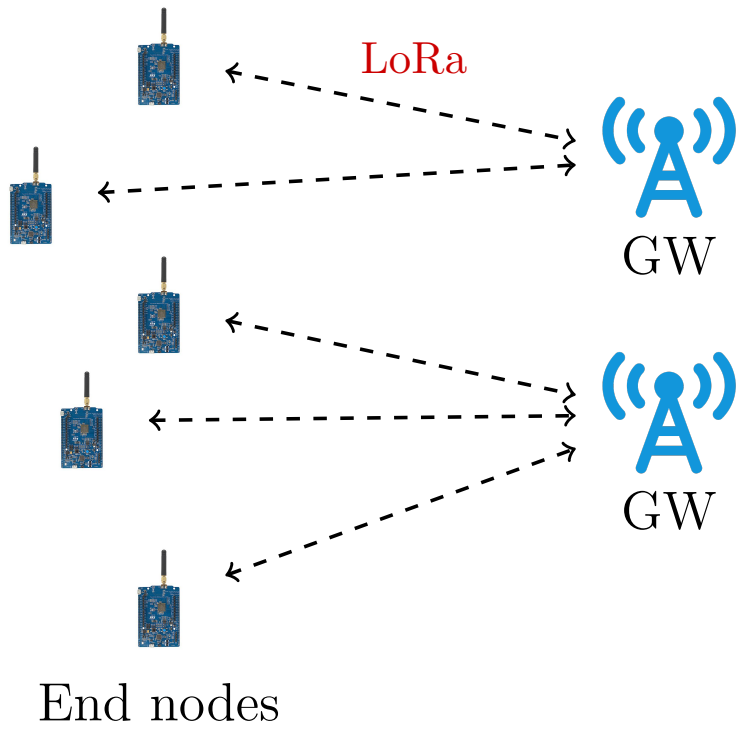
Legacy cellular is not a reliable IoT baseline

LPWAN

LPWAN: like cellular, but optimized for IoT/M2M (node is not connected to a BTS)

- network of GW/BTS
- star topology
- range in kilometers





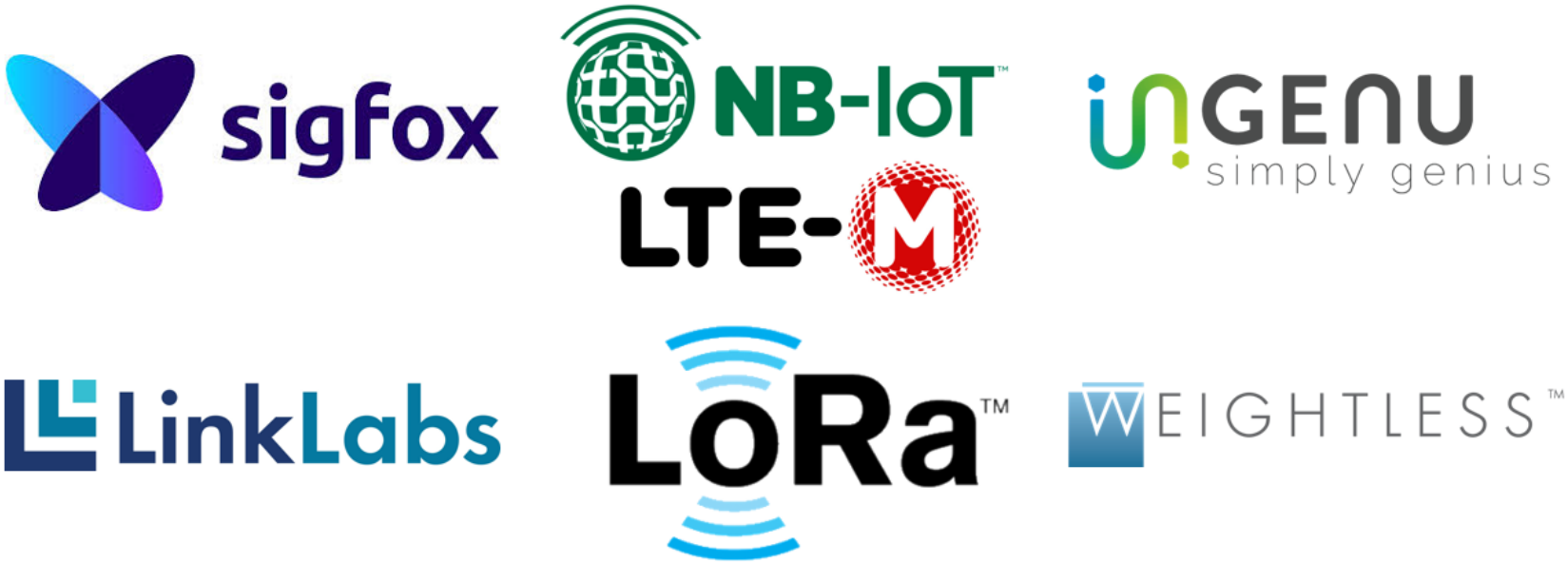
Main features of LPWANs

Key properties

- Low cost
- Low data rate
- Unlicensed spectrum $\rightarrow$ easy deployment

	Range	Battery lifetime
2G	$2 \div 4$ km	days
IEEE 802.15.4	$10 \div 100$ m	months–years
WiFi	$10 \div 50$ m	days
LPWAN	$10 \div 100$ km	years

LPWAN: emerging technologies



Different business models, technical specs (e.g., frequencies)

LoRaWAN recognized as standard for LPWAN by ITU

Definitions

LoRa: PHY layer

LoRaWAN: MAC layer

LoRa Alliance: defines LoRaWAN specifications

Technical comparison

	SigFox	LoRaWAN	NB-IoT
Modulation	BPSK	CSS	QPSK
Frequency	ISM	ISM	LTE bands
Max rate [kbps]	0.1	50	200
Max msg/day	140 (U), 4 (D)	Unlimited	Unlimited
Range [km]	$10 \div 40$	$5 \div 20$	$1 \div 10$
Handover	not applicable	not applicable	no
Localization	Yes (RSSI)	Yes (TDOA)	WIP
Allow private network	No	Yes	No

LPWAN Cost comparison

	SigFox	LoRaWAN	NB-IoT
Spectrum cost [M€/MHz]	free	free	500
Deployment cost [€/BTS]	> 4000	> 100	> 15000
End-device cost [€]	< 2	$3 \div 5$	> 20

LoRa*WAN* vs SigFox: business models

SigFox

- Fully vertically integrated company (end-to-end control)
- Closed ecosystem
- Network and cloud sold as a service
- Hardware ecosystem licensed under strict terms

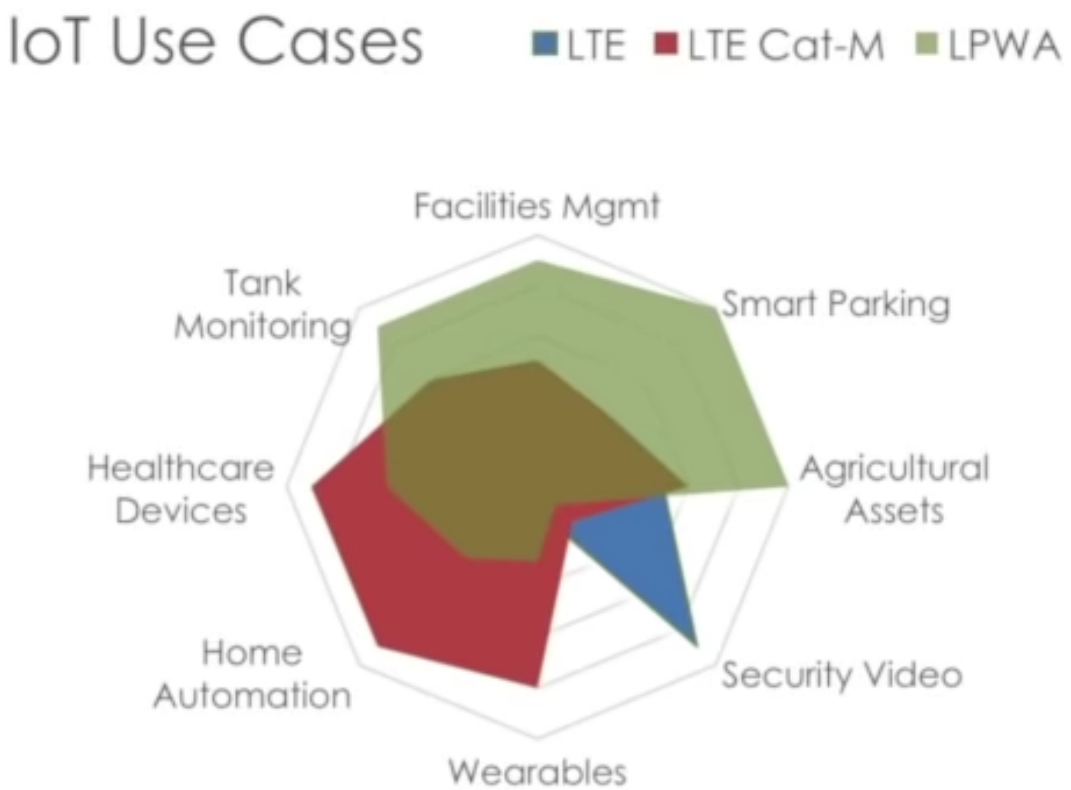
LoRaWAN

- Open specification defined by the LoRa Alliance
- Anyone can deploy networks or build compatible devices
- Semtech provides the radio chips, but ecosystem is multi-vendor
- Strong adoption also in research and private deployments

Key difference: control vs openness of the ecosystem

IoT use cases

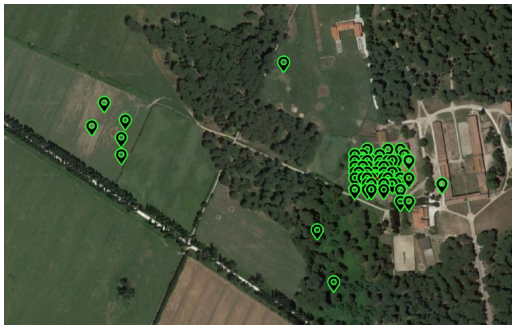
- Smart city
  - Metering
  - Parking
  - Alarms
- Smart agriculture
  - Environmental sensors



Remark

Not only for range of km, think also about indoor with obstacles or underwater/underground (farm monitoring)

IoT use cases: smart farming



LoRaWAN longest range

height = 38.000 m

flights = 8.500 ÷ 12.000 m

Tx power = 25 dBm

WiFi Laptop = 15 dBm

Urban scenario: Padova

Need to send a few bytes occasionally and want years of battery life? $\rightarrow$ LoRa

LoRa vs. LoRaWAN

LoRa: PHY layer

LoRaWAN: MAC, NWK, and APP built on LoRa

LoRaWAN is not the only MAC/NWK implementation: Symphony Link—proprietary

LoRa/LoRaWAN basics

LoRa = PHY layer $\rightarrow$ proprietary

LoRa= Long Range $\rightarrow$ up to tens of km

Based on CSS (Chirp Spread Spectrum)

Low bit rate: $0.25 \div 11$ kbps

ISM bands: EU 868 MHz $\rightarrow$ anyone can deploy

LoRaWAN = MAC layer $\rightarrow$ open

Access method: ALOHA

Duty cycle 1% w/o channel access method

Short payload: max $59 \div 230$ bytes (in EU)

Non IP-compliant

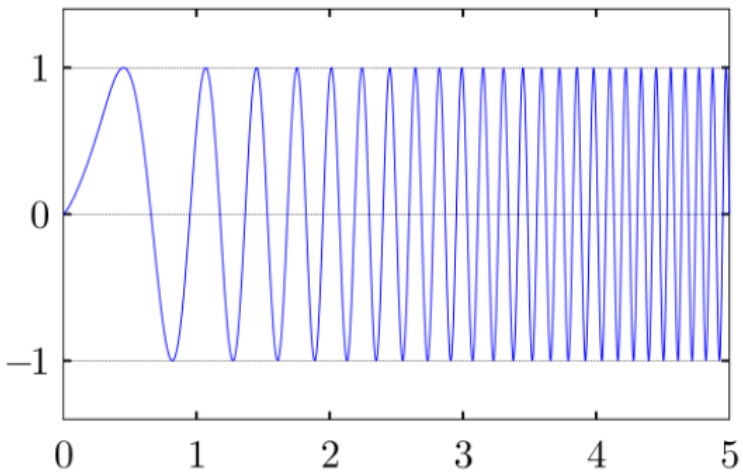
PHY layer – LoRa

LoRa modulation

Based on Chirp Spread Spectrum (CSS)

LoRa modulation

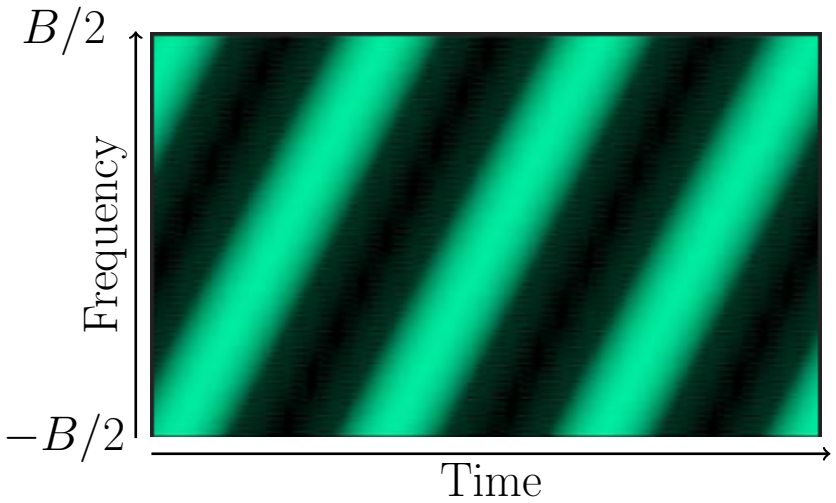
Based on Chirp Spread Spectrum (CSS)



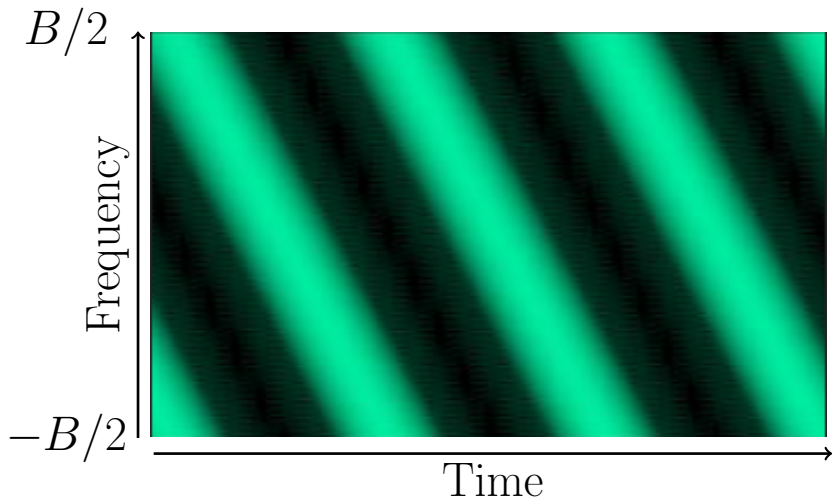
▸ Chirp sound

Up-chirp / down-chirp

Up-chirp: $e^{2\pi jt(f_0+at)}$



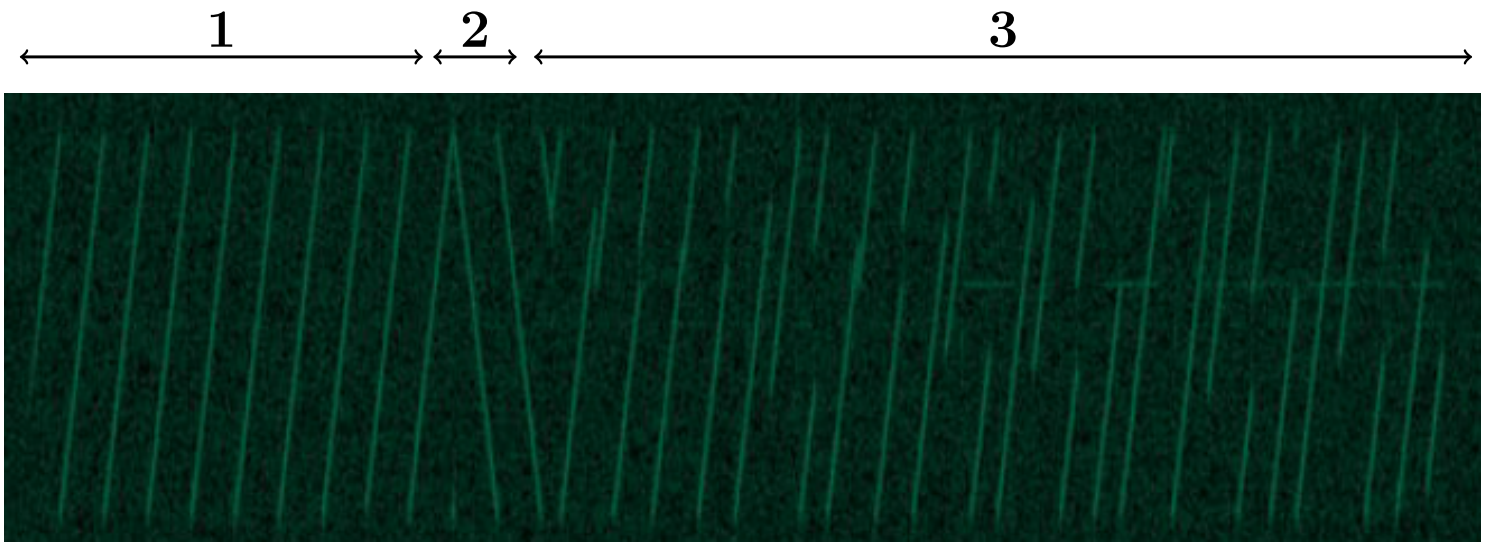
Down-chirp: $e^{-2\pi jt(f_0+at)}$



$$e^{2\pi jt(f_0+at)} \times e^{-2\pi jt(f_0+at)} = 1$$

LoRa Frame

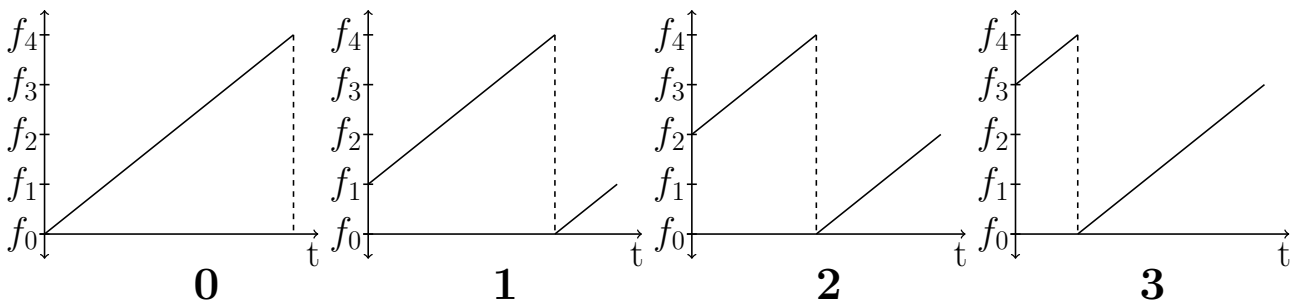
1. Preamble, training sequence
2. Beginning of the PHY data unit
3. Actual data



- The preamble is used for:
  - Time synchronization (Δt estimation)
  - Carrier frequency offset estimation (f_0)
  - Symbol rate alignment

Data encoding before transmission

- Bits are grouped into SF-bit symbols (2^{SF} possible values)
- Each symbol is mapped to a frequency shift (CSS modulation)

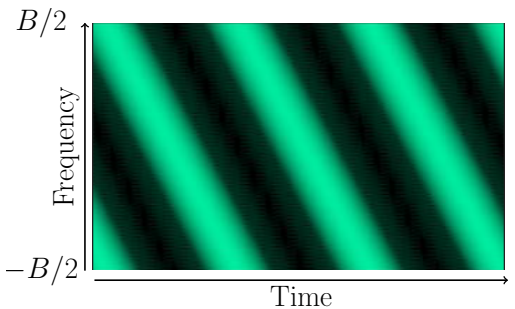


Example SF = 2, $2^{SF} = 4$ symbols

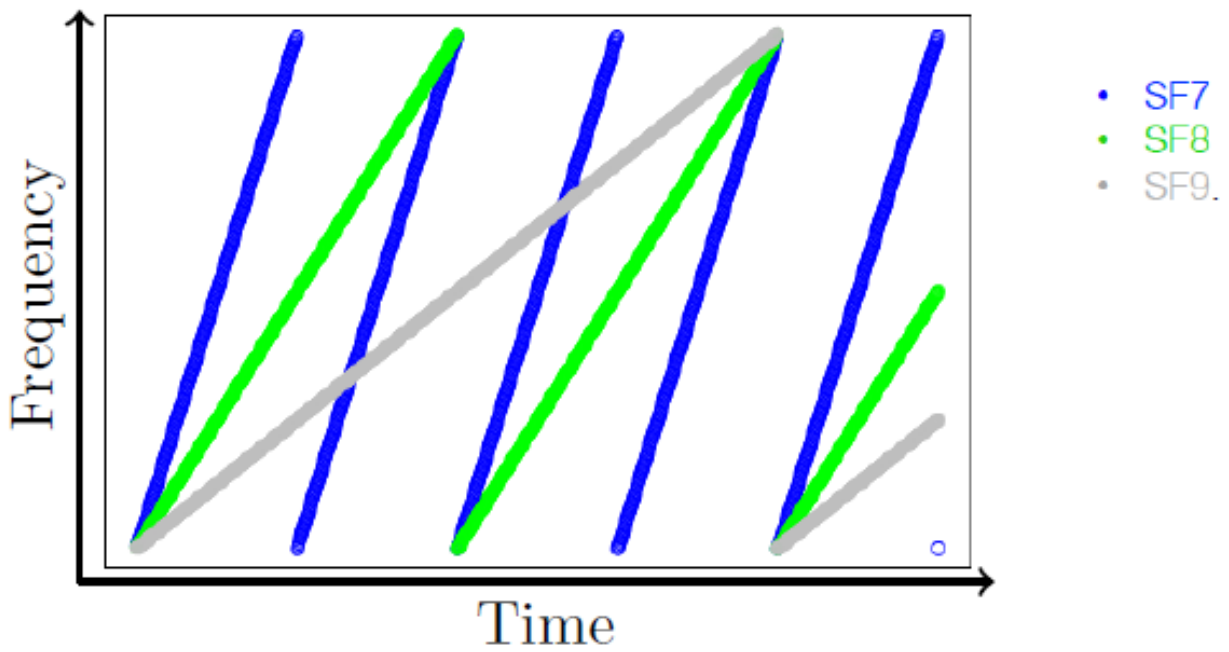
- Signal robustness techniques: Gray coding (reduce bit errors) and whitening (avoid long patterns)
- Error protection: interleaving and FEC (coding rate 4/5 to 4/8)

LoRa Demodulation

- Receiver performs **dechirping**:
  - multiply received signal by a reference down-chirp
  - convert chirps into constant tones (pure frequencies)
- FFT is then used to detect the frequency peak
- Peak position $\rightarrow$ transmitted symbol
- Key idea: **chirp correlation converts time structure into frequency**
- Enables reliable decoding at very low SNR



Spreading Factor (SF)



- SF controls how long a symbol is spread in time
- Higher SF $\rightarrow$ lower data rate, higher robustness
- Each +1 in SF roughly halves data rate and increases airtime

Spreading Factor (SF)

Key relationships

- Number of symbols: 2^{SF}
- Symbol duration: $D = \frac{2^{SF}}{B}$
- Symbol rate: $R_s = \frac{B}{2^{SF}}$
- Bit rate: $R_b = SF \cdot R_s = \frac{SF \cdot B}{2^{SF}}$

Data-rates and sensitivity

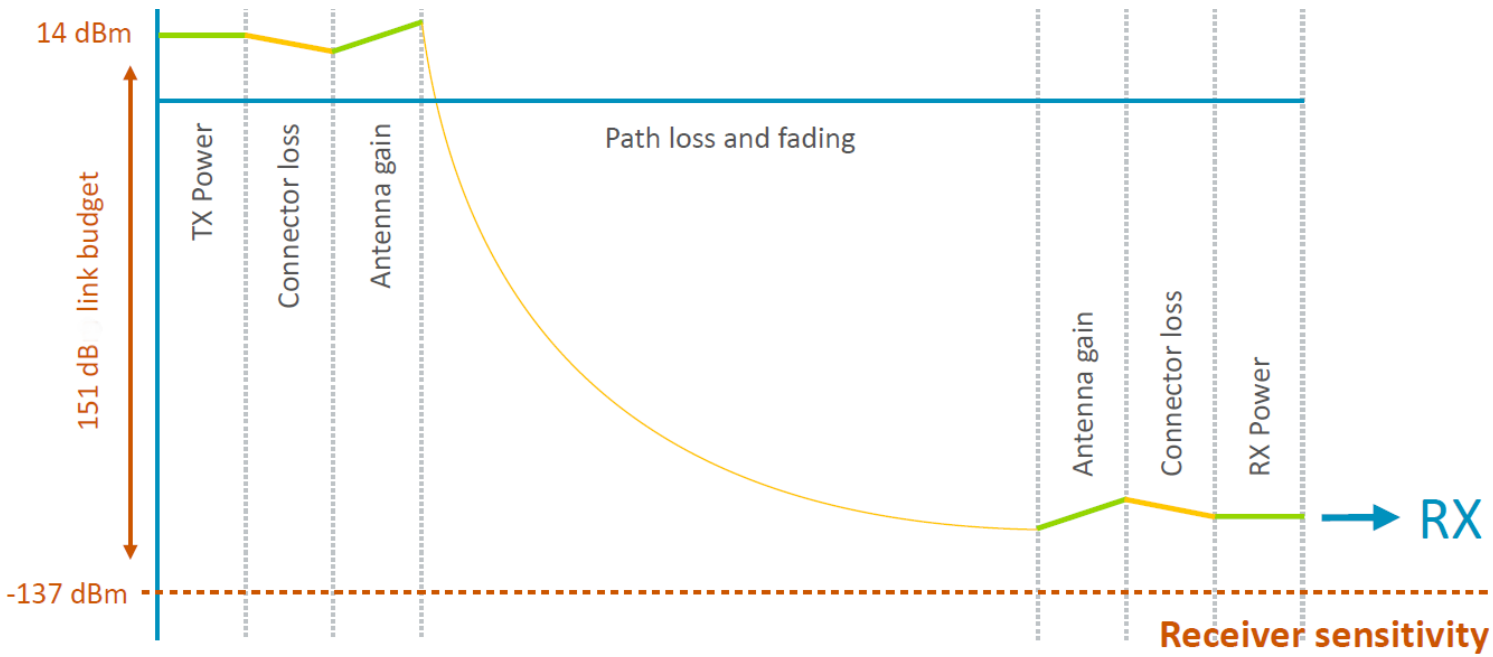
Assuming an antenna with 6 dBi reception gain

Data-rate LoRaWAN	SF	Sensitivity [dBm]
DR5	7	-134.5
DR4	8	-137.0
DR3	9	-139.5
DR2	10	-142.0
DR1	11	- 144.5
DR0	12	-147.0

Sensitivity increases of 2.5 dB with SF

Link budget comparison

	TX Power	RX Sensitivity	Link budget
WiFi	20 dBm	−75 dBm	**95 dB**
Sub-GHz 6LoWPAN	11 dBm	−110 dBm	**121 dB**
LoRa	14 dBm	−137 dBm	**151 dB**



Theoretical max distance in free space

Friis formula

$$P_r(d) = P_t G_t G_r \left(\frac{\lambda}{4\pi d}\right)^{\nu} = P_t G_t G_r \left(\frac{c}{4\pi d f_c}\right)^{\nu}$$

with $\nu = 2 \div 4$ 2 =, path-loss exponent.

Example with $\nu = 2 \rightarrow$ free space

Wi-Fi: 2.4 GHz, with 95 dB link budget ($P_t - P_r$)

550 meters

LoRa: 868 MHz, with 151 dB link budget ($P_t - P_r$)

870 kilometers

LoRaWAN trade-offs

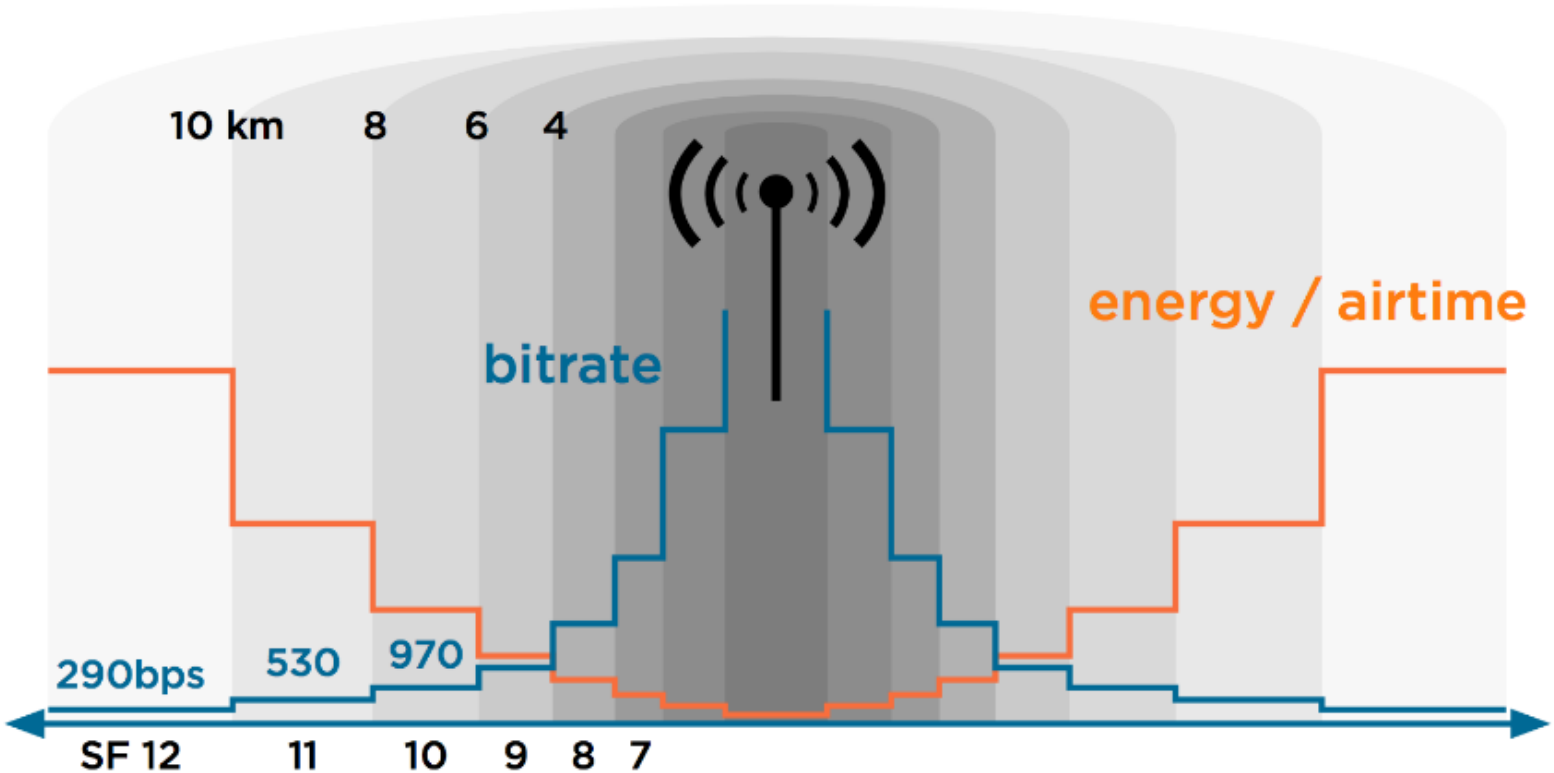
- Higher Spreading Factor $\rightarrow$ longer airtime
- Longer airtime $\rightarrow$ higher collision probability
- Higher SF $\rightarrow$ lower data rate but better sensitivity

Key trade-off

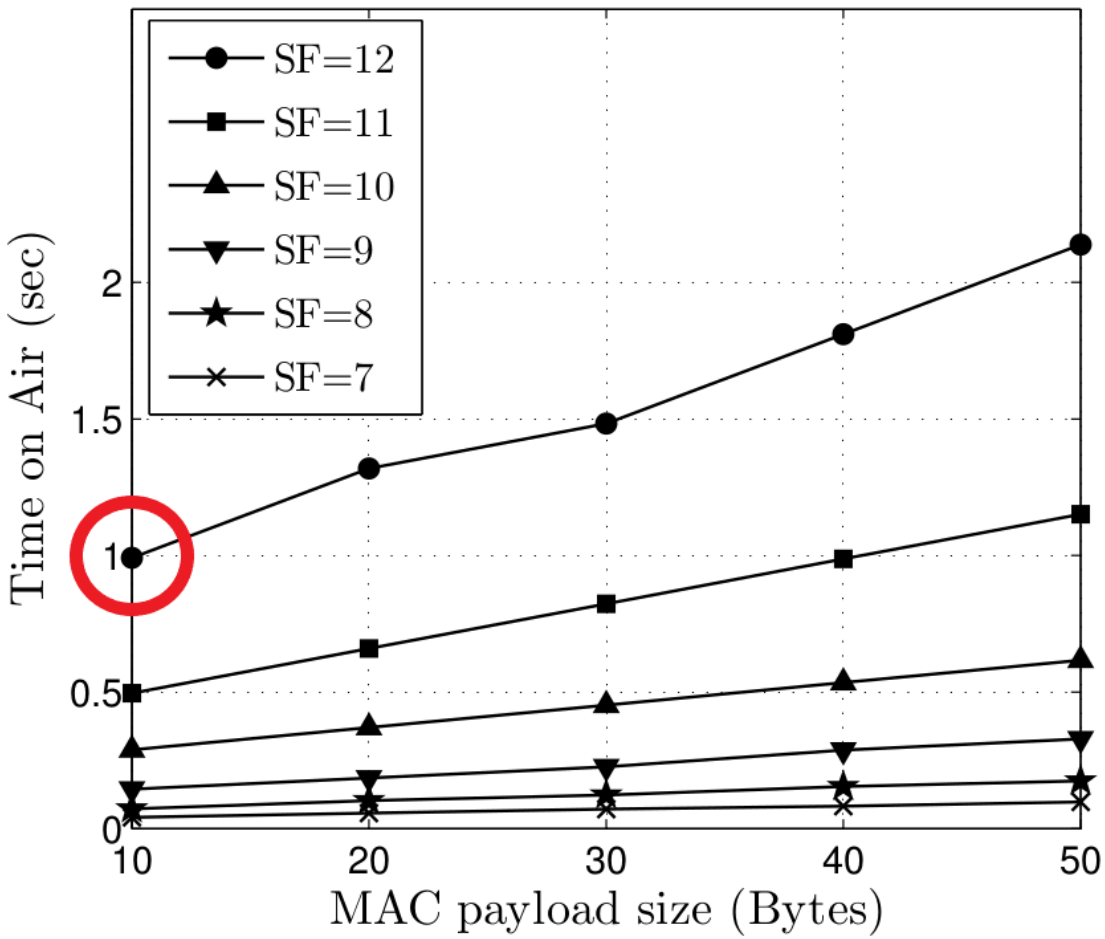
$$\text{Range} \uparrow \longleftrightarrow \text{Time-on-air} \uparrow \longleftrightarrow \text{Battery lifetime} \downarrow$$

- LoRaWAN is not about throughput, but about efficient sparse transmissions

A complete sketch



T_{air} @ code rate 4/5 and BW = 125 kHz



For US915 @ SF = 12, $PL_{\max} = 11$ bytes

In C, 1 char = 1 byte $\rightarrow$ Hello world

Data representation: JSON

Basic example: 54B

{“pressure”: 997.8,“temperature”: 26.6,“humidity”: 55.5}

Short example: 36B

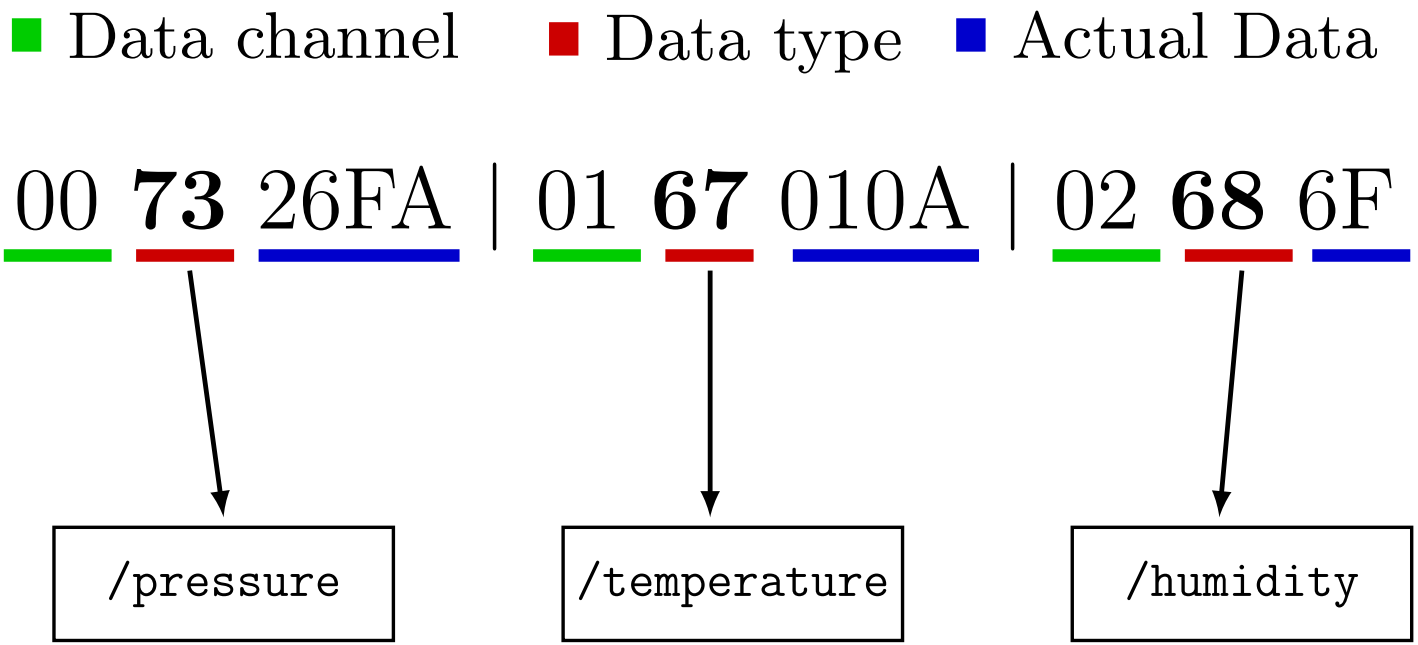
{“prs”: 997.8,“tmp”: 26.6,“hum”: 55.5}

Shortest example: 30B; one letter per sensor: limited

{“p”: 997.8,“t”: 26.6,“h”: 55.5}

Data representation: LPP

Cayenne Low Power Packet (LPP) format



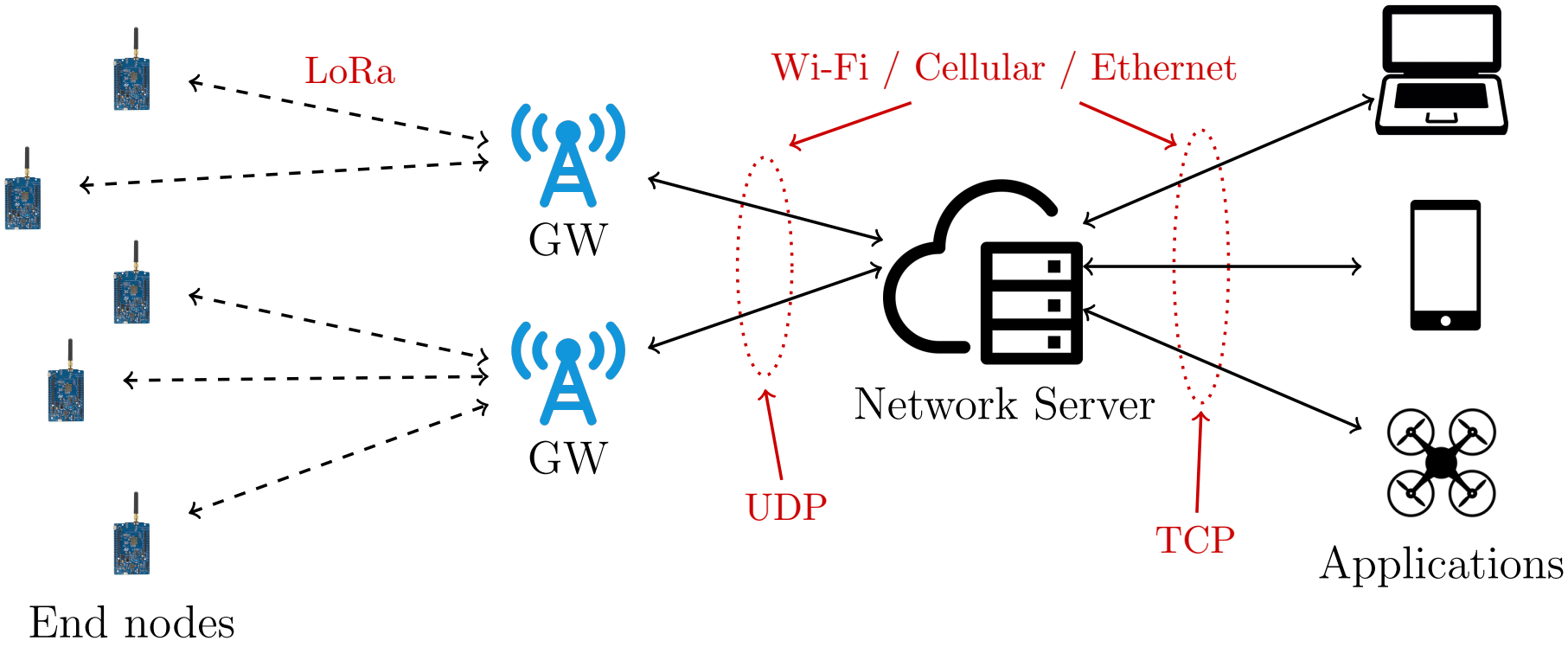
Only 11 bytes!

LPP format

Sensor	HEX value	Length [bytes]	Data Resolution per bit
Analog input	`0x02`	2	0.01 Signed
Temperature	`0x67`	2	0.1 °C Signed MSB
Humidity	`0x68`	1	0.5% Unsigned
Accelerometer	`0x71`	6	0.001 G Signed MSB per axis
Barometer	`0x73`	2	0.1 hPa Unsigned MSB
GPS	`0x88`	9	Lat: 0.0001 ° Signed MSB Lng: 0.0001 ° Signed MSB Alt: 0.01 m Signed MSB

MAC layer – LoRaWAN

Architecture: star of star



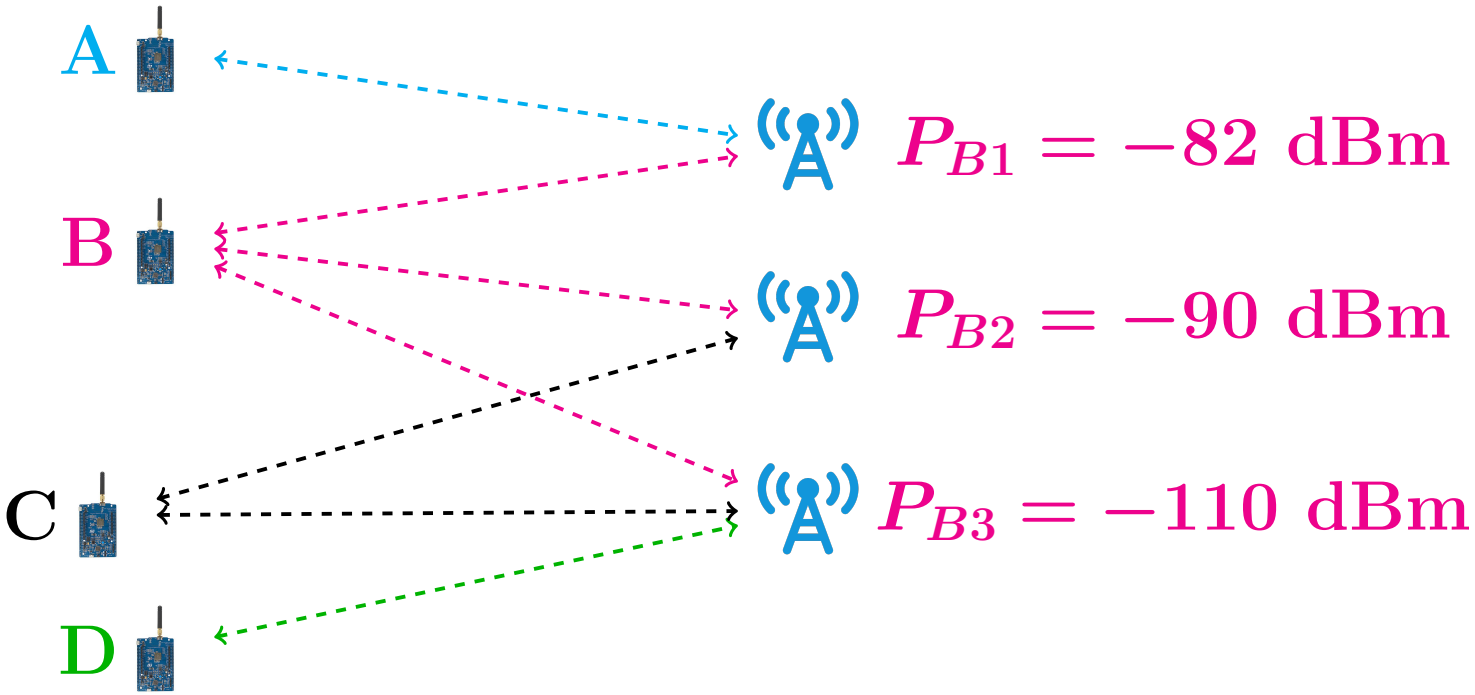
Used by...

- TTN, crowd-sourced public network
- KPN (Netherlands), Orange (France), A2A Smart City (Italy), ...
- Whoever wants to deploy it!

Remark

KPN and Orange are also mobile network operators

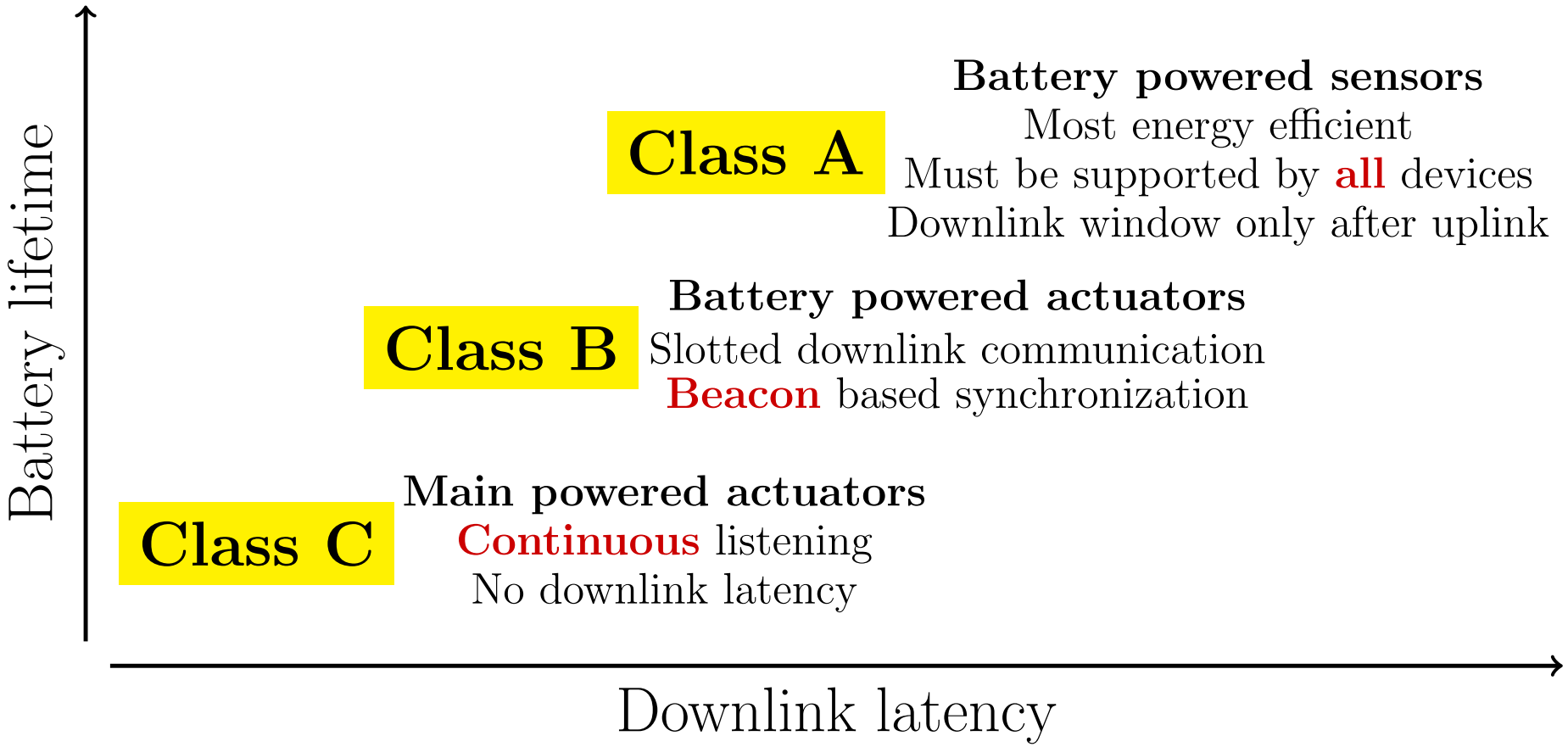
Uplink transmissions



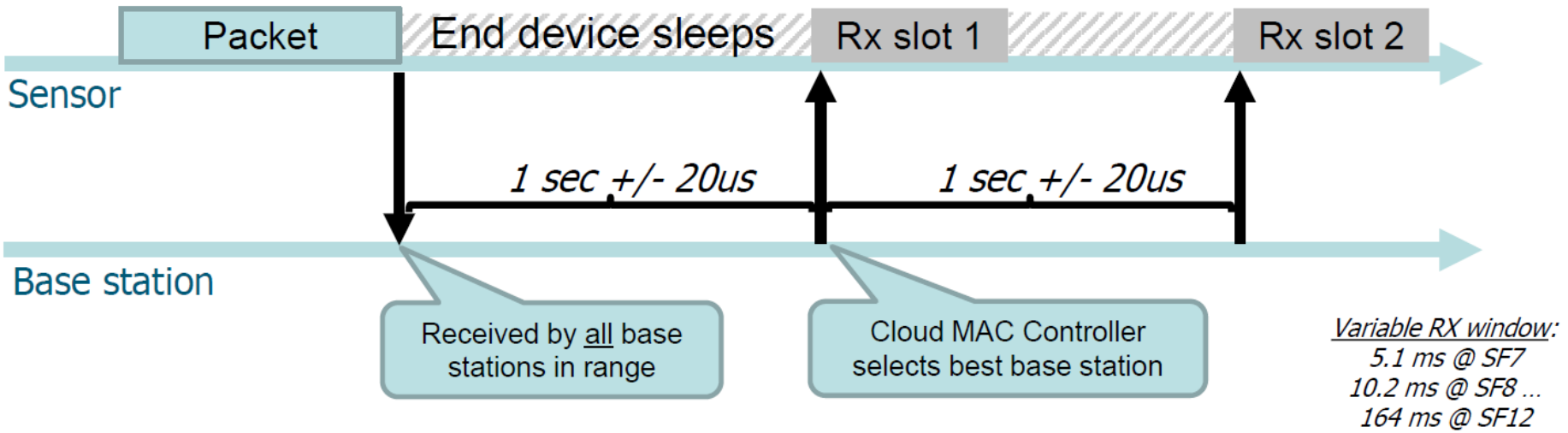
A packet is received by all the GW in the range of the node

Example: packet from B is received by all the GWs
The NS will further process it

Device classes



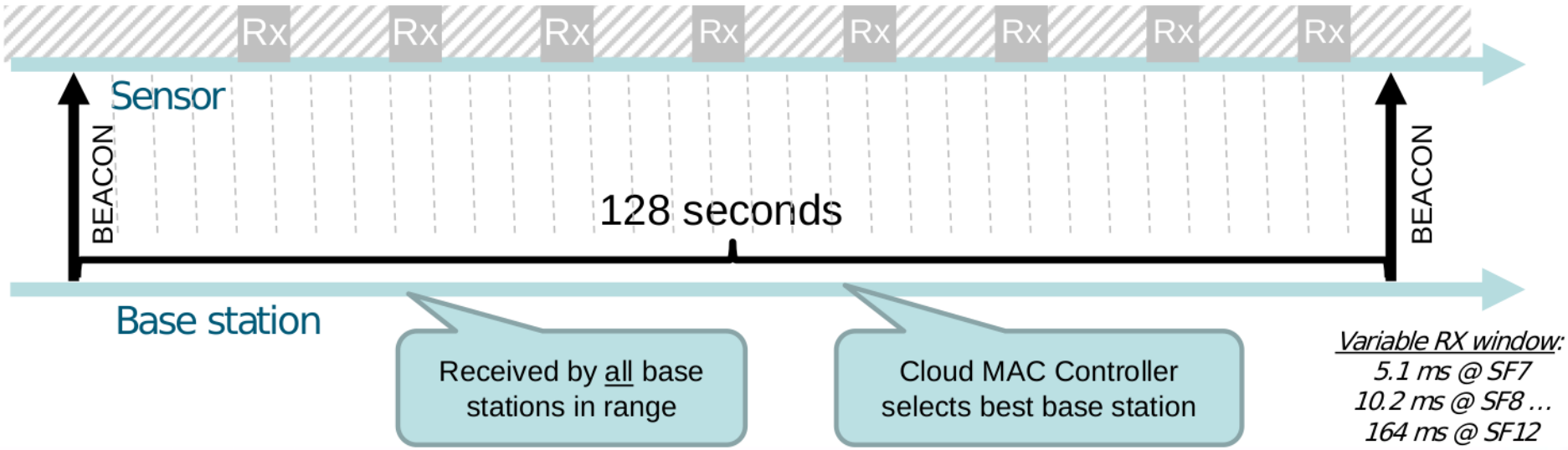
Class A devices



2 RX windows follow the transmission

- at t_1 s (by default $t_1 = 1$s) (same channel as transmission)
- at $t_1 + 1$ s (channel and SF fixed in advance)
  - by default: RX2 at 869.525 MHz (center of h1.6), DR0 (SF12, 125 kHz)

Class B devices



All sensors are **synchronized** with the gateways

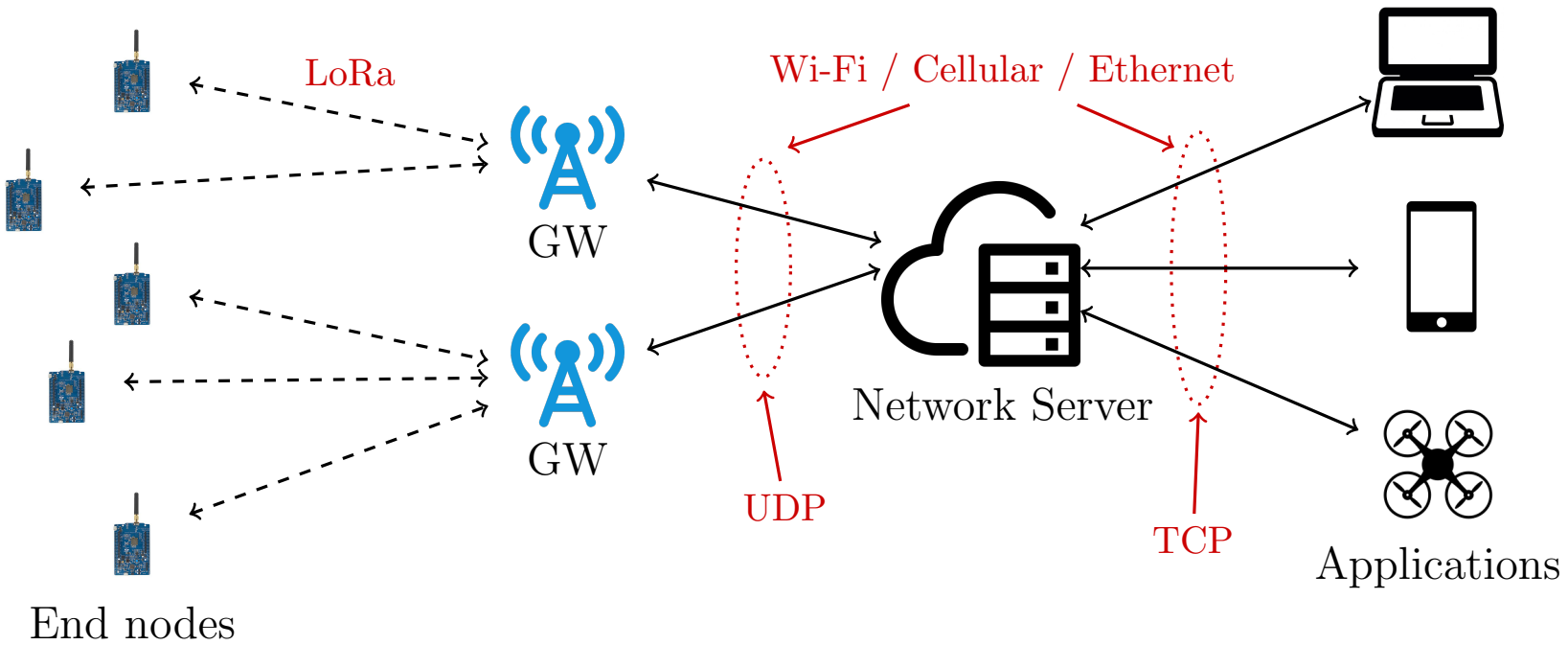
Basic working principles

- ALOHA-based access (low duty-cycle, no scheduling)
- Devices operate in a wake-up $\rightarrow$ transmit $\rightarrow$ sleep cycle
- Designed for ultra-low power operation

- Uplink can be:
  - Unconfirmed (best effort)
  - Confirmed (requires ACK from gateway)

Network Architecture and Security

Network architecture



Devices

SEMTECH
LoRa Picocell Gateway
RX
TX
ANT

pycom
GO INVENT
WiFi
Bluetooth
LoRa
Sigfox
lopy

HDMI

LoRaWAN Network: data path

- **Device:** sends uplink data and receives downlink messages
- **Gateway:**
  - Receives radio packets
  - Forwards them to the Network Server
  - Transparent (no decision logic)
- **Network Server (control core):**
  - Manages sessions and join procedure
  - Removes duplicates
  - Performs ADR (link optimization)

LoRaWAN Network: control and application

- **Network Server:**
  - Selects gateway and timing for downlink
  - Handles MAC-layer control
- **Application Server:**
  - Handles application logic
  - Decrypts payload
  - Generates responses to devices

Network Server and Application Server can be co-located or separated

Adaptive Data Rate (ADR)

- ADR is a **Network Server mechanism** to optimize link efficiency
- It balances data rate, airtime, and energy consumption

- Key parameters:
  - Spreading Factor
  - Transmission Power

Adaptive Data Rate (ADR mechanism)

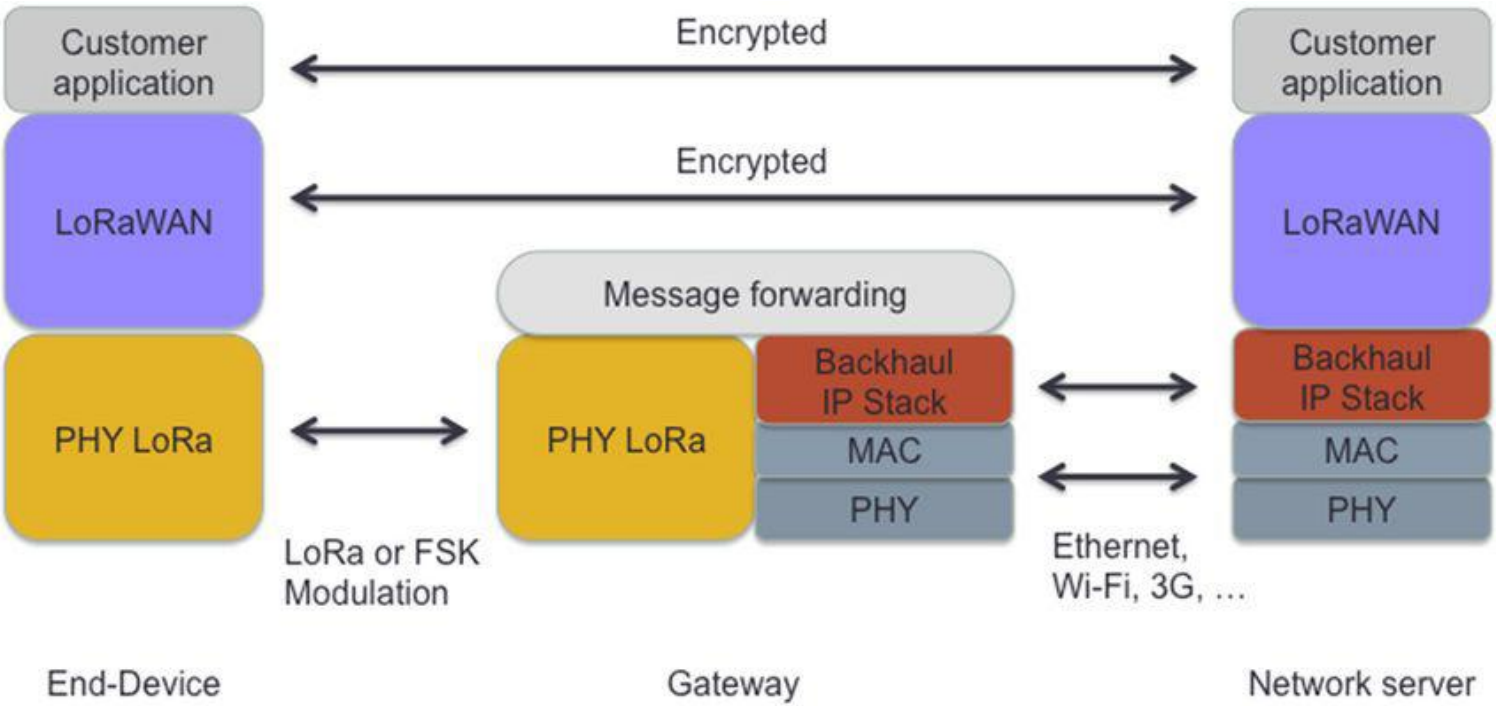
- Network Server estimates link quality (e.g. SNR over time)
- Computes optimal SF and transmission power
- Sends MAC command (LinkADRReq) to the device

- Good channel conditions $\rightarrow$ lower SF, higher data rate
- Poor channel conditions $\rightarrow$ higher SF for reliability

Important

ADR is controlled by the Network Server, but devices can enable/disable it

Protocol stack



Remarks

Available network servers: Brocaar, Gotthard (on github)

LoRaWAN Addressing model

Device identity (hardware level)

DevEUI: 64-bit unique device identifier *(like a MAC address)*

Network identity (routing level)

DevAddr: 32-bit address assigned by the Network Server *(like an IP address)*

Application identity

AppEUI: identifies the application provider (JoinEUI in newer specs)

Application routing

FPort: selects the application inside the device *(like TCP/UDP port)*

Node activation (Join procedure)

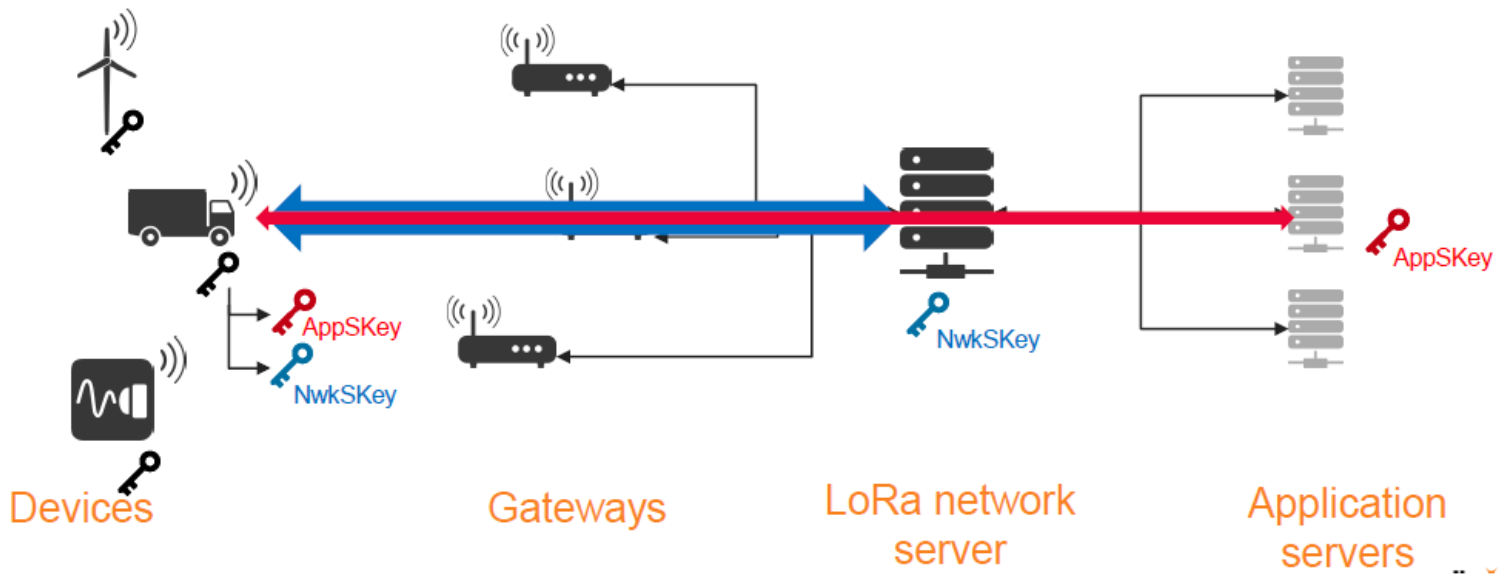
Before communicating on a LoRaWAN network, a device must be activated.

- This process establishes:
  - **DevAddr** (network address)
  - **NwkSKey** (network session key)
  - **AppSKey** (application session key)

- Two activation methods:
  - **OTAA** (Over-The-Air Activation) — dynamic and secure
  - **ABP** (Activation By Personalization) — pre-configured

Keys

- NwkSKey: shared between Device and Network Server (msg integrity)
- AppSKey: shared between Device and Application Server (end-to-end security)



OTAA (Over-The-Air Activation)

- Secure join procedure used when a device connects to the network
- Session keys are generated dynamically at each join

- Join flow:
  - Device sends **Join Request**
  - Network Server authenticates the device
  - Session keys are created
  - Device receives and decrypts **Join Accept**

OTAA: advantages and limitations

Advantages

- Strong security (fresh session keys at each join)
- Scalable key management
- Recommended for real deployments

Limitations

- Join procedure required at each (re)connection
- Slight latency during activation
- Higher implementation complexity (need server for key)

ABP (Activation By Personalization)

- No join procedure at network attachment
- Device is pre-configured offline (no handshake)

- Pre-installed parameters:
  - DevAddr
  - NwkSKey
  - AppSKey

- Keys remain fixed for the entire device lifetime unless manually changed

ABP: advantages and limitations

Advantages

- Very simple deployment
- No join delay (instant communication)
- Useful for testing or constrained environments

Limitations

- No key renewal or rotation
- Poor scalability in large deployments
- Higher security risk if keys are exposed

OTAA vs ABP: key differences

- **1. Join procedure**
  - OTAA: full network handshake (Join Request / Join Accept)
  - ABP: no handshake, direct activation
- **2. Key management**
  - OTAA: session keys generated dynamically at each join
  - ABP: keys are pre-installed and static
- **3. Device state management**
  - OTAA: Network Server maintains session state and join lifecycle
  - ABP: no session lifecycle management (static configuration)
- **4. System complexity**
  - OTAA: requires join protocol + key management infrastructure
  - ABP: minimal logic, manual provisioning

Beyond the App Server

- Rest API
- MQTT
- Web-socket

Platforms

- https://www.chirpstack.io/
- https://www.thethingsnetwork.org/

Current and Future works

Scalability: From Pilot to Production

- $>$125 million devices deployed globally
- $\sim$25% CAGR (compound annual growth)
- Multi-million device deployments in:
  - Utilities (smart metering)
  - Smart cities
  - Industrial IoT
- **Key point:** LoRaWAN has moved beyond pilot phase

LoRaWAN over Satellite (NTN)

- Emerging integration with **Non-Terrestrial Networks (NTN)**
- Currently satellite used as IP backbone
- Direct device-to-satellite communication (GW on the satellite, experimental)

- Enables:
  - Global coverage (remote areas, oceans, deserts)
  - Infrastructure-free deployments

Limitations

- Very constrained link budget
- Limited downlink capacity
- Non-deterministic connectivity (orbital constraints)

Protocol Evolution

Data Rate and Capacity

- Introduction of multi-band operation:
  - sub-GHz $\rightarrow$ long range
  - 2.4 GHz $\rightarrow$ higher throughput
- LR-FHSS (frequency hopping) for improved scalability

Security and Interoperability

- Improved certification processes
- Updated security specifications (e.g., LoRa Alliance TR007)
- Focus on reducing ecosystem fragmentation

Question for you:
What is the most relevant topic right now?

From sensing to decisions

- LoRaWAN acts as a **data acquisition layer**
- Devices remain simple: sense $\rightarrow$ transmit
- Intelligence is centralized in backend systems

- Typical architecture:

  Sensors $\rightarrow$ LoRaWAN $\rightarrow$ Cloud / Analytics / AI $\rightarrow$ Decisions

- Enables:
  - Predictive maintenance
  - Smart monitoring systems
  - Infrastructure optimization

Industrial Integration

- Convergence with industrial standards
- Mapping toward OPC UA (industrial communication standard)
- Integration into OT/SCADA environments
- **Impact:**
  - Increased industrial adoption
  - Better interoperability with existing systems

Network Architecture Evolution

Hybrid Networks

- Combination of:
  - Public networks
  - Private deployments
  - Satellite connectivity

Backhaul Innovation

- Use of Wi-Fi HaLow for long-range backhaul
- Reduced infrastructure costs

Mesh Research

- Multi-hop/mesh under study
- Not part of current standard

Market Growth

- Market size:
  - $\sim$ \$6.7B (2026)
  - $\sim$ \$54B (2033)
- Strong expansion across industries
- **This means:**
  - No longer an experimental technology
  - Established IoT connectivity solution

Critical Summary

Real Advances

- Large-scale deployment
- Satellite integration
- Multi-band support
- Industrial ecosystem integration

Limitations

- Not high-throughput
- No standardized mesh
- Limited latency and QoS

Conclusion: LoRaWAN is evolving, but remains an LPWAN technology

If you want to know more

- **Curiosity is the best protocol**
- https://www.youtube.com/watch?v=NoquBA7IMNc
- Understanding the Limits of LoRaWAN, by Ferran Adelantado
- https://developers.mydevices.com/cayenne/docs/lora/#lora-cayenne-low-power-payload
- https://www.thethingsnetwork.org/docs/lorawan/

UNIVERSITÀ DI PARMA

Overview on LoRa/LoRaWAN

LoRaWAN technology: An overview

Laura Belli, *Ph.D.*

University of Parma

UNIVERSITÀ
DI PARMA

April 23, 2026

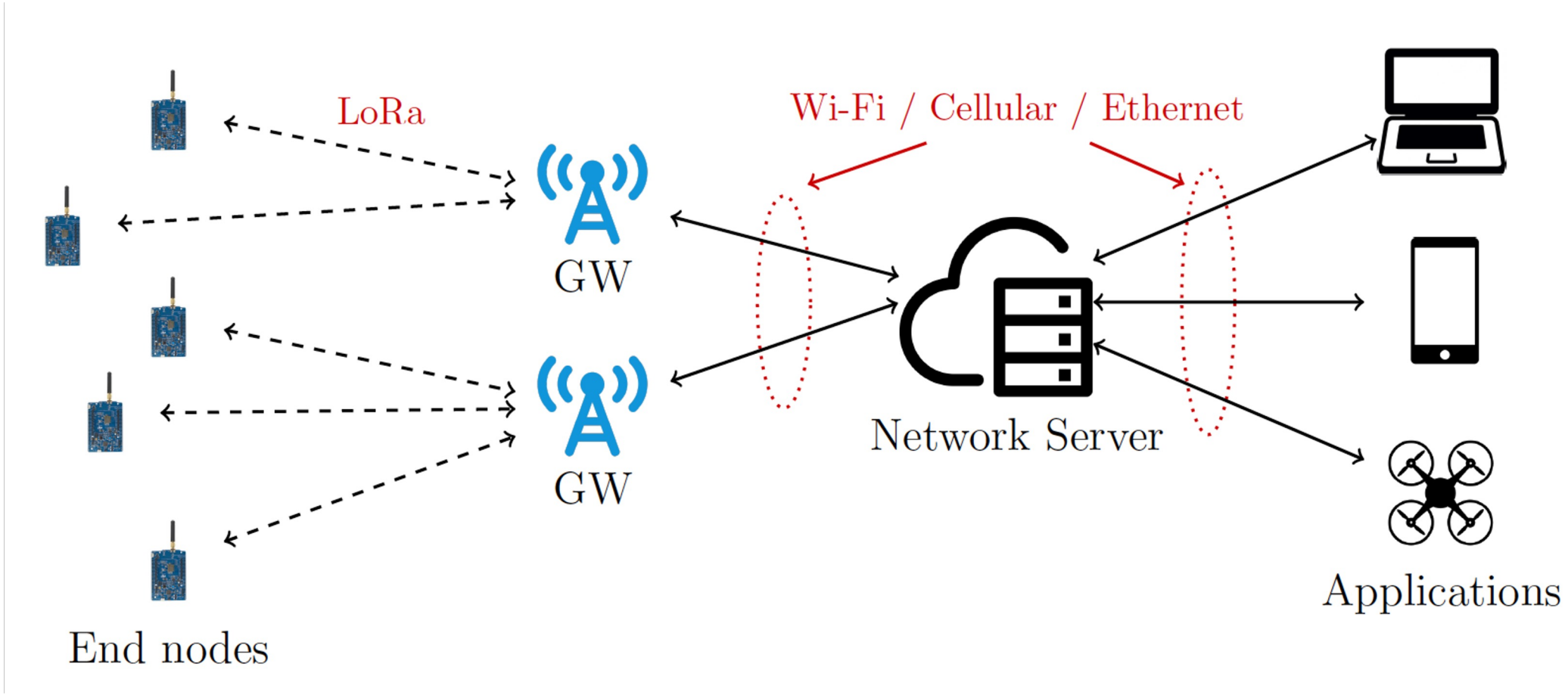
Internet of Things, Communication Engineering Course

Agenda

- LoRaWAN concepts recap
- LoRaWAN Gateways components
- The Things Network (TTN)
- ChirpStack
- LoRaWAN Devices Configuration

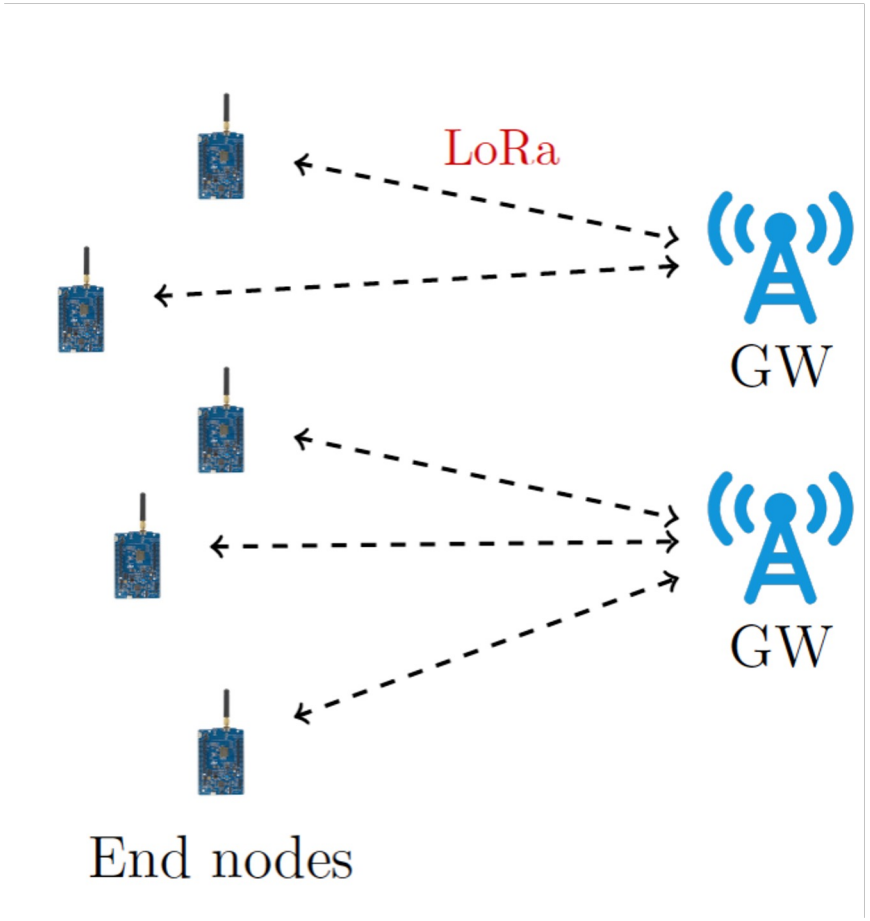
LoRaWAN recap – Network Structure

The structure of a LoRaWAN Network is a “star of stars”



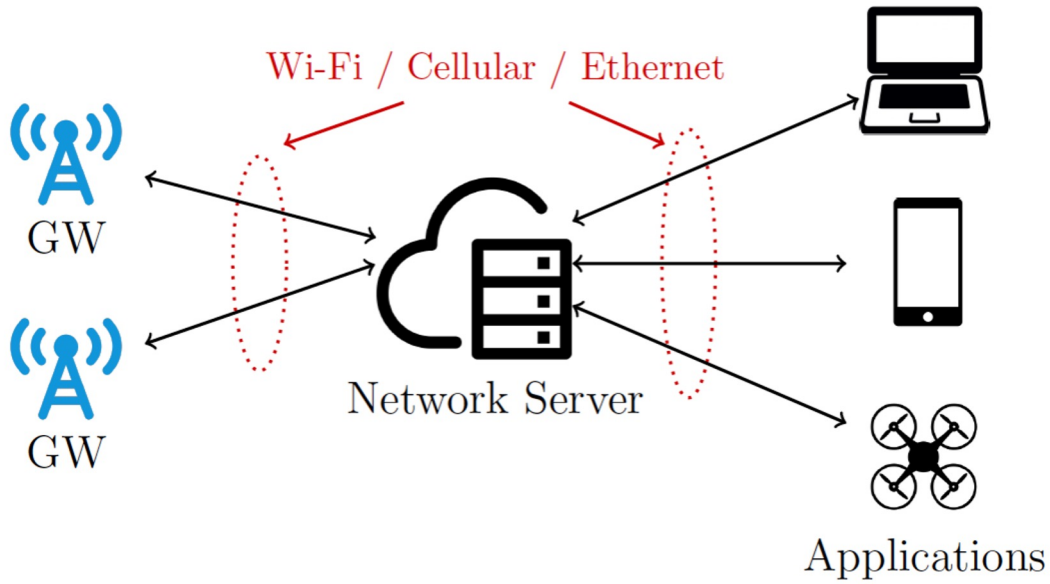
LoRaWAN recap – Network Structure

- End nodes are IoT sensors or actuators
  - They generate data (uplink messages) and can receive data (downlink messages)
  - They have to join the network before sending data
- Gateways:
  - They receive data from end nodes
  - Perform integrity checks (e.g.: Cyclic Redundancy Check CRC)
  - Forward data to the Network Server through IP packets
  - Gateways don't decrypt payloads!



LoRaWAN recap – Network Structure

- Network Server:
  - Handles join requests, downlinks, data rates and duplicated frames
  - Doesn't decrypt payloads!
- Application Server:
  - Receives packets from the Network Server
  - Schedules responses for the end nodes
  - Decrypts payloads
  - Can forward data to external entities



LoRaWAN recap – End Nodes

- LoRaWAN end nodes can follow 3 classes (with an increasing energy consumption level):
  - **Class A**: downlink windows are allowed only after an uplink (generally after 1 or 2 seconds)
  - **Class B**: downlinks are allowed in periodic windows → The gateway should transmit synchronization beacons to correctly manage the communication
  - **Class C**: it's the «continuous listening» mode. The downlink window is always open after the first uplink, except when the device is transmitting
- The more common classes are A and C

LoRaWAN recap – Join methods

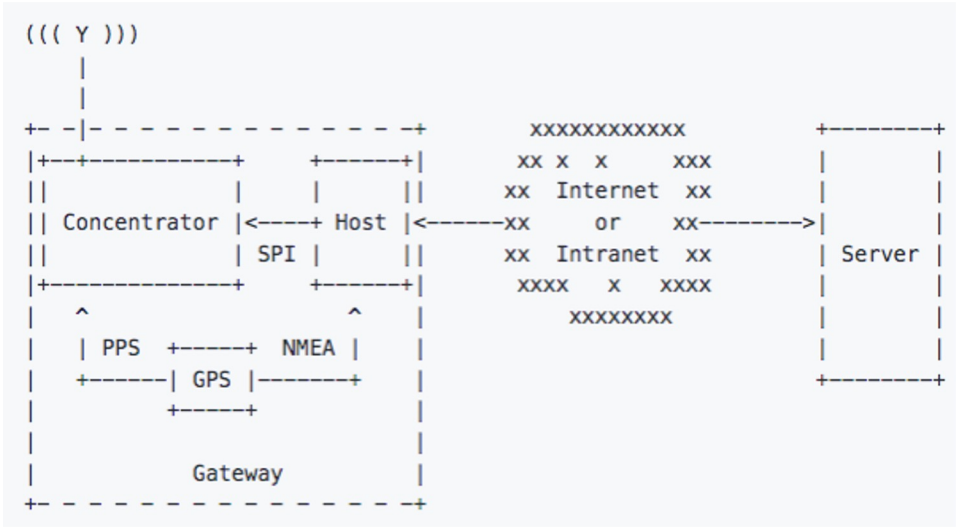
- Before starting sending uplinks, the end nodes have to join the LoRaWAN network
- Two possibilities are available:
  - **ABP**: Activation By Personalization (static)
    - DevAddr (network address)
    - NwkSKey (network session key)
    - AppSKey (application session key)
  - **OTAA**: Over-The-Air Activation (recommended)
    - DevEUI
    - AppEUI/JoinEUI
    - AppKey (application Key)
- In OTAA Session keys are generated dynamically at each join

LoRaWAN Protocol versions

- LoRaWAN version 1.0 (Jan 2015): is the first approved version of LoRaWAN
- LoRaWAN version 1.0.1 (Feb 2016): This minor revision clarified some points that were not clear in the previous version. Moreover, added some frequency plans, and other aspects
- LoRaWAN version 1.0.2 (July 2016): This minor revision added more MAC commands, fixed some errors in the previous version, and moved the section "Physical Layer" to a separate document
- LoRaWAN version 1.1 (Oct 2017): This major revision presented a new architecture
  - A new server called Join Server (JS) is introduced
  - The Network Server entity is split into three servers: Home Network Server (hNS), Serving Network Server (sNS), and forwarding Network Server (fNS)
  - This architecture enables roaming services between network operators
- LoRaWAN version 1.0.3 (Jul 2018): This minor revision related to v1.0.2 reused the same specifications for the class B devices mentioned in LoRaWAN version 1.1, added some MAC commands for class A devices, and deprecated some others
- LoRaWAN version 1.0.4 (Oct 2020): This minor revision related to v1.0.3 (clarifications and security mechanisms)

LoRaWAN Gateways

- Gateways are the bridge between devices and the Network Server
- Devices use low power networks (LoRa) to connect to the Gateway, while the Gateway uses high bandwidth networks like WiFi, Ethernet or Cellular to connect to the Network Server



Each LoRaWAN Gateway:

- Contains an hardware component called Concentrator that is the transmitter/receiver module designed to manage LoRa packets on multiple channels (e.g. Semtech SX1302 or Semtech SX1303)
- Is equipped with a Packet Forwarder, a software component that forwards packets received by the concentrator to the NS through an IP/UDP link, and sends packets that are received from the NS

LoRaWAN Gateways – Packet Forwarders

- The most common Packet Forwarders used in the LoRaWAN market are both developed by Semtech

Semtech's UDP Packet Forwarder:

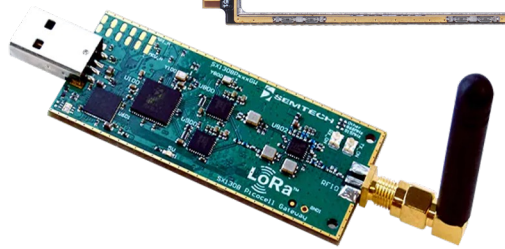
- It is the first of its kind
- Simple and lightweight
- It uses UDP connections to transfer LoRaWAN packets from the gateway to the network server and vice versa
- widely used: many manufacturers preinstall it on their gateways
- No data encryption (between GW and NS)
- UDP can be unreliable because there are no data reception guarantees or confirmation from the UDP protocol

Semtech's LoRa Basic Station:

- Released in early 2019
- It uses WebSockets to transfer packets to the Network Server and vice versa
- Support end to end data encryption (TLS)
- Allows the connection to a Configuration and Update Server (CUPS) to manage and update gateways configurations
- CUPS allow users to access remotely to the gateway
- Better choice for large deployment

LoRaWAN Gateways 2/2

You can usually find two kind of gateways in the market:



- Gateways with a **minimal firmware**, making them low-cost and easy to integrate with other boards, running only the Packet Forwarding software

- Gateways with a complete **operating system**, where the packet forwarding software run as a background program. This gives more control to the gateway administrators to manage and add functionalities to the gateway and to install their own software

LoRaWAN Network and Application Servers

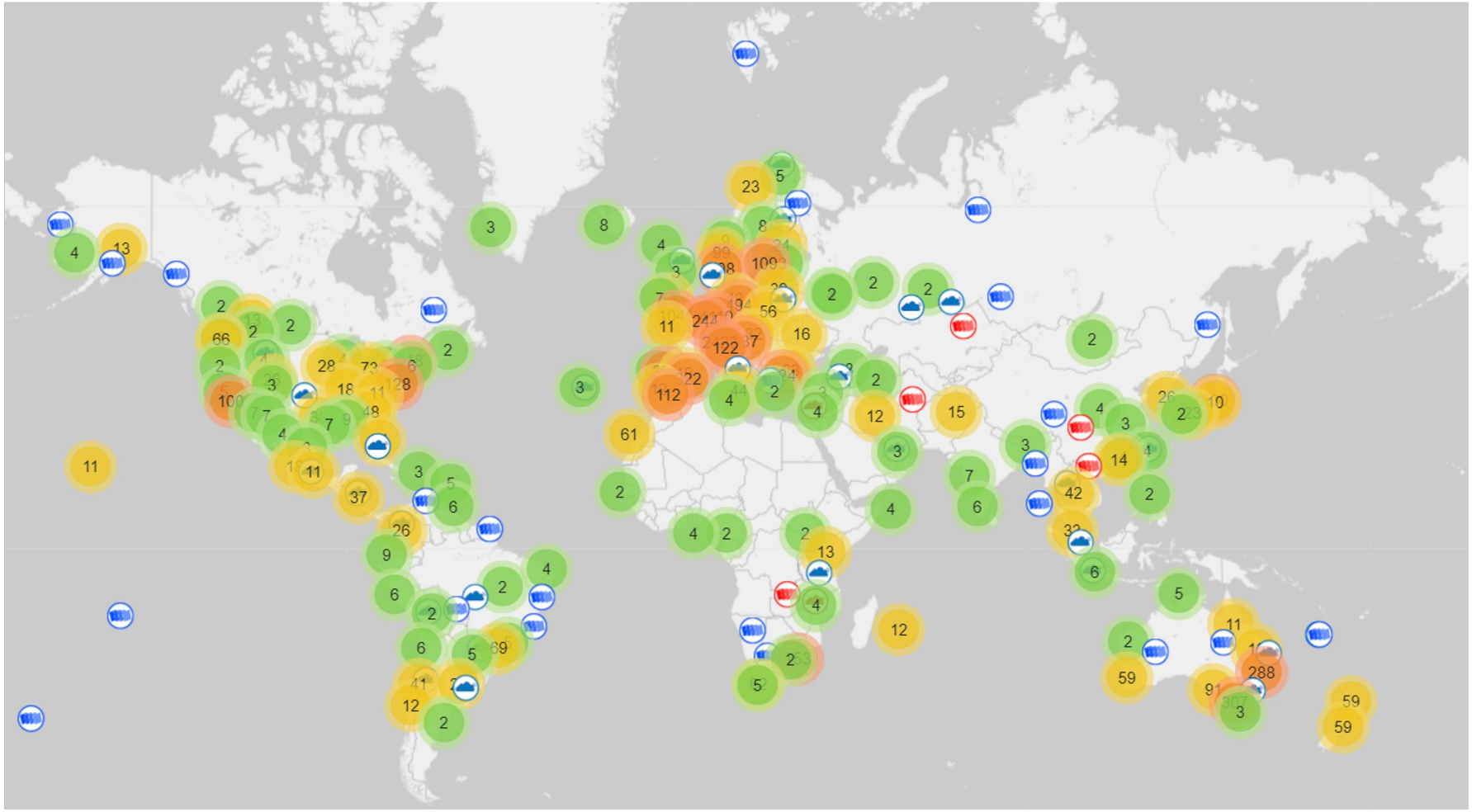
- Network Server and Application Server are often deployed on the same machine
- They can be deployed on the Cloud or managed in a «local» machine (e.g.: your PC or a Raspberry Pi)
- Some open-source options are nowadays available to manage data from a LoRaWAN network:
  - The Things Network (TTN)
  - ChirpStack
- After the Application Server, data can be provided to external entities (applications) with different options: HTTP REST APIs, MQTT, Web Sockets and other…

The Things Network (TTN)

- Is a global collaborative Internet of Things (IoT) ecosystem that creates networks and solutions using the LoRaWAN protocol
- Runs on The Things Stack Community Edition, which is a open and decentralized LoRaWAN Network Server implementation distributed over the globe
- TTN provides developers and students with a set of open tools to get started testing devices, and get familiar with LoRaWAN network components and applications
- It is based on The Things Stack which is an Enterprise LoRaWAN Network Server solution which can be used by companies as a commercial product

The Things Network (TTN)

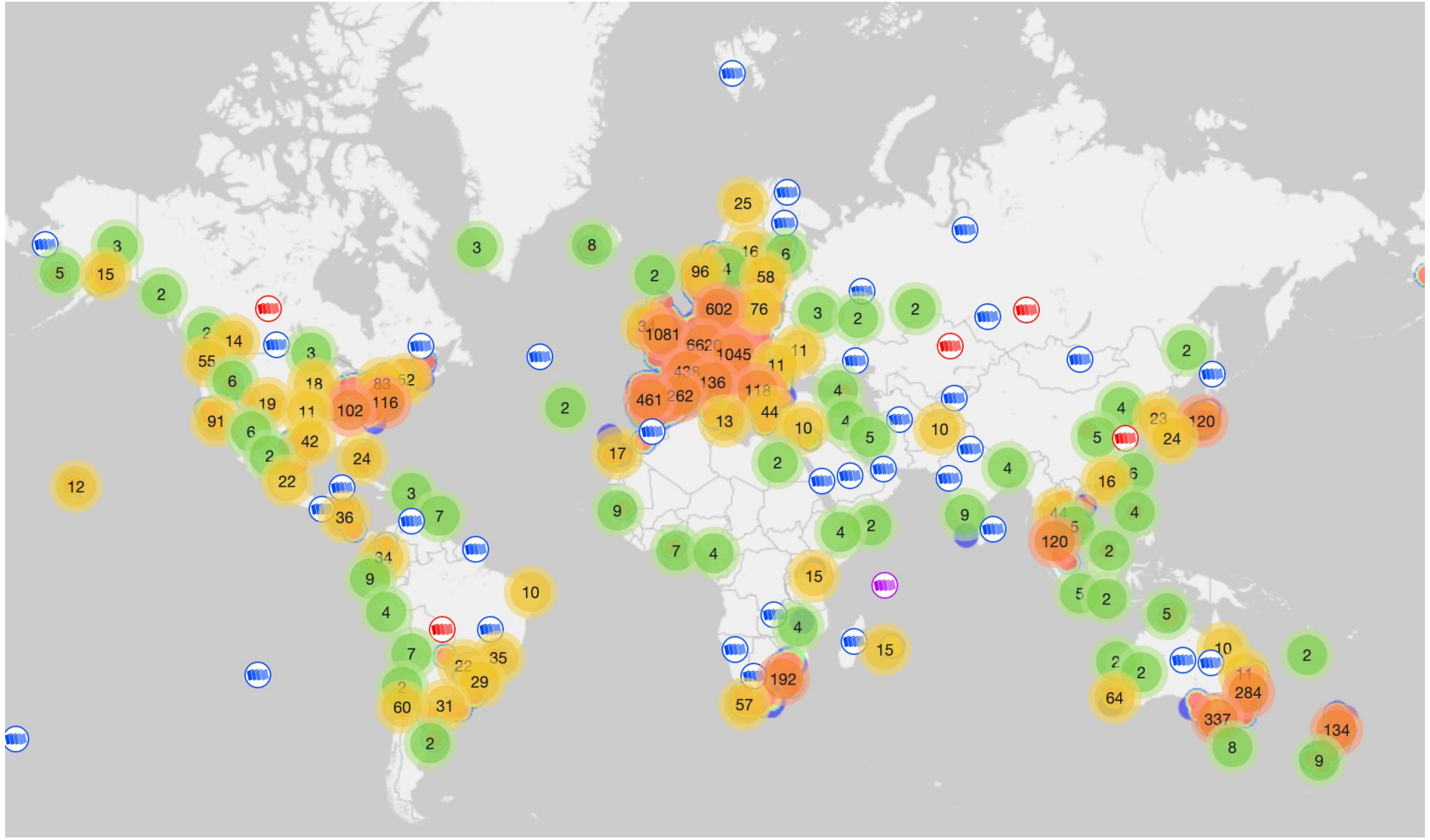
Gateways deployed on TTN (2024)



THE THINGS NETWORK

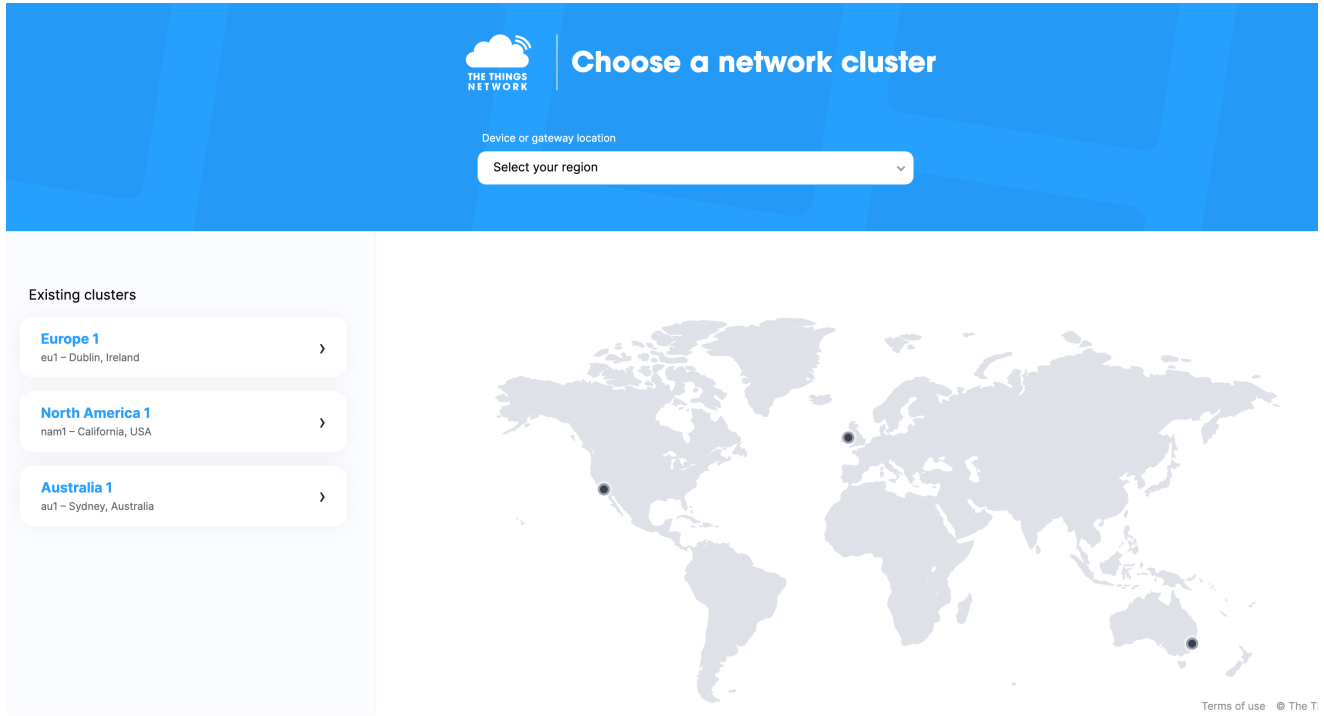
The Things Network (TTN)

Gateways deployed on TTN (2025)



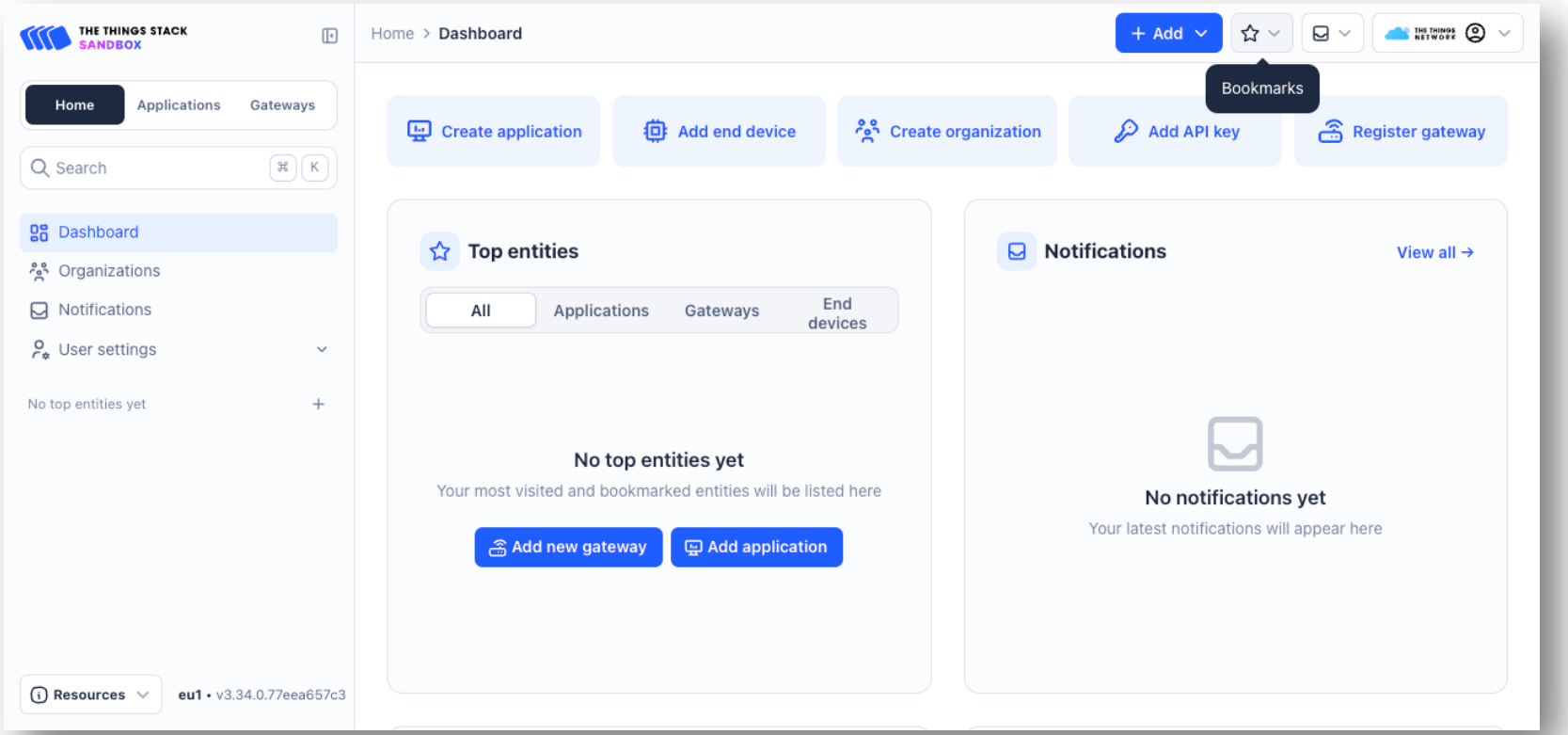
The Things Network (TTN)

- You can create a free account in the Things Stack Community Edition website
- It's a multi-cluster deployment: this means that with your account you can connect your gateways to a cluster, and route all your IoT traffic in that cluster



- This limitation is done to significantly reduce latency: you are assigned to a cluster, so your traffic would not have to cross half the planet
- Currently 3 clusters are available, for different geographic areas

The Things Network: Web console



Each account can access to the TTN Console which is the management application of the Things Stack for LoRaWAN

- It is a web based application accessible from the TTN website
- It is used to register applications, end devices or gateways
- It allows to monitor network traffic, or configure network related options

The Things Network: Gateway entity (1/2)

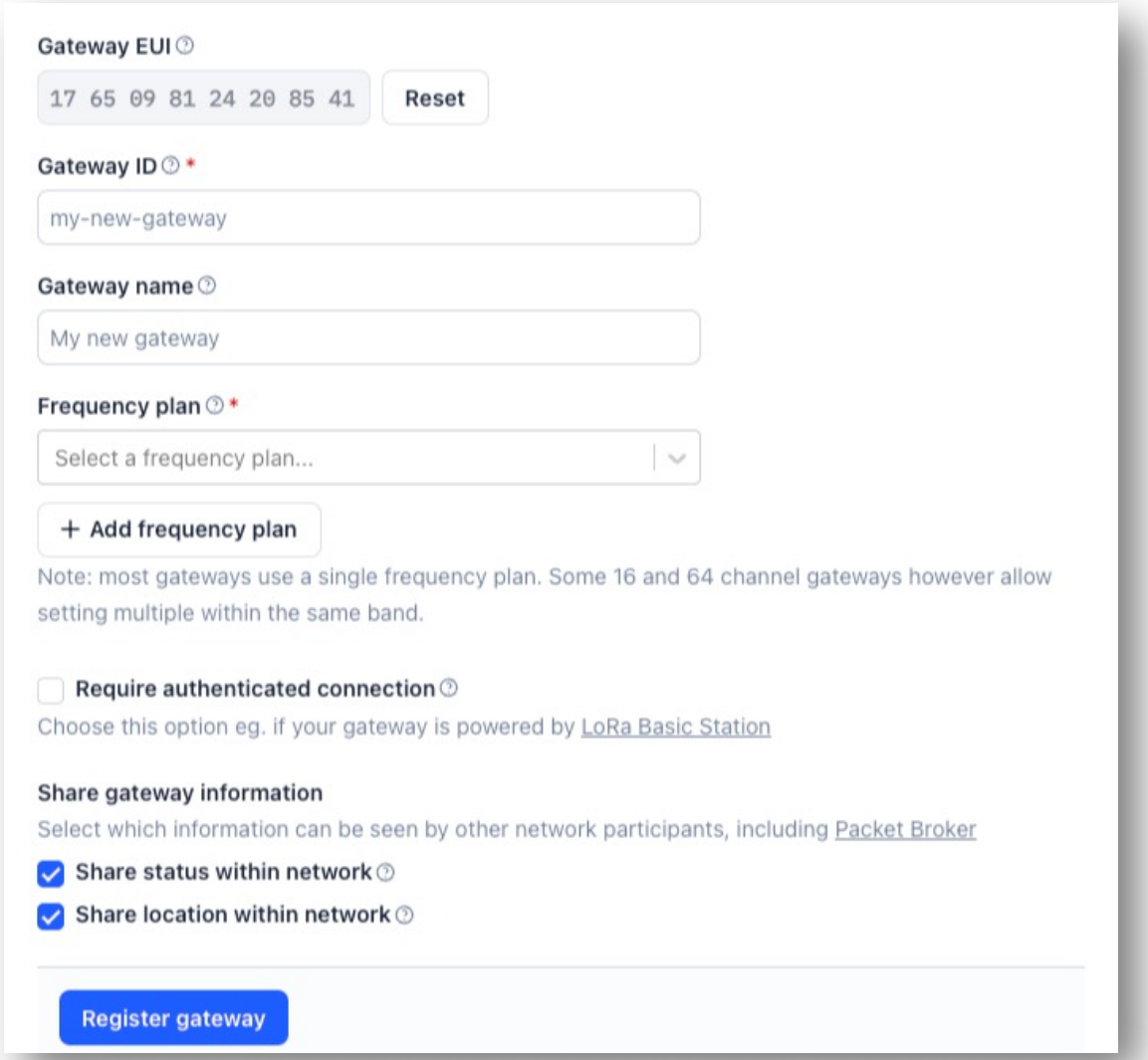
Relevant parameters:

- ID: a unique textual identifier for the gateway

  e.g.: `gw-iotlab-campus-unipr`

- EUI: a unique 64 bits identifier for the gw (provided by manufacturer)

  e.g.: `1765098124208541`

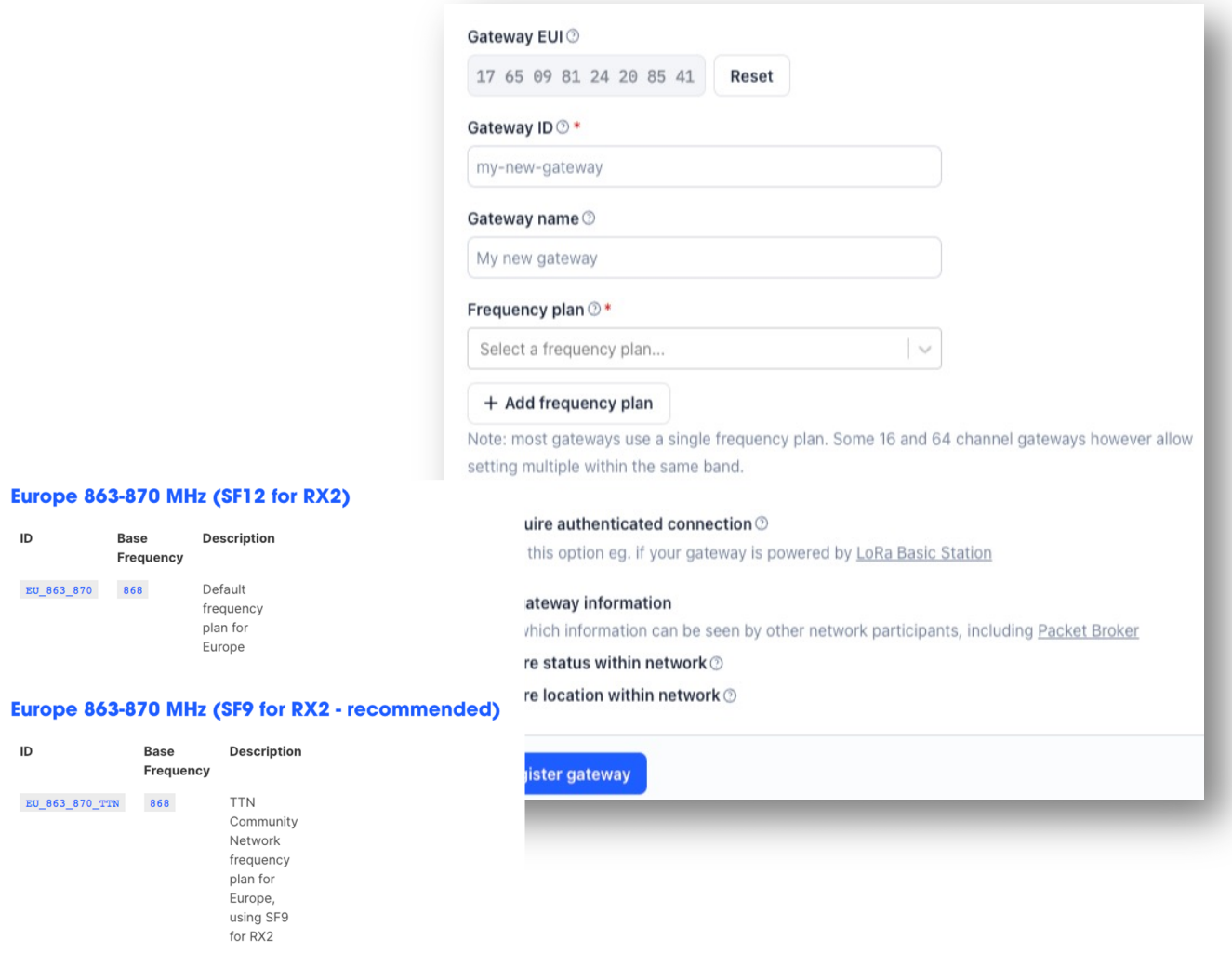


The Things Network: Gateway entity (2/2)

- Frequency plan: defines data rates and channels which comply with the LoRaWAN Regional Parameters for a geographic area

  e.g.: `EU_863_870_TTN`

- Frequency plans are defined by the LoRa Alliance Technical Committee and are published as the LoRaWAN Regional Parameters technical document



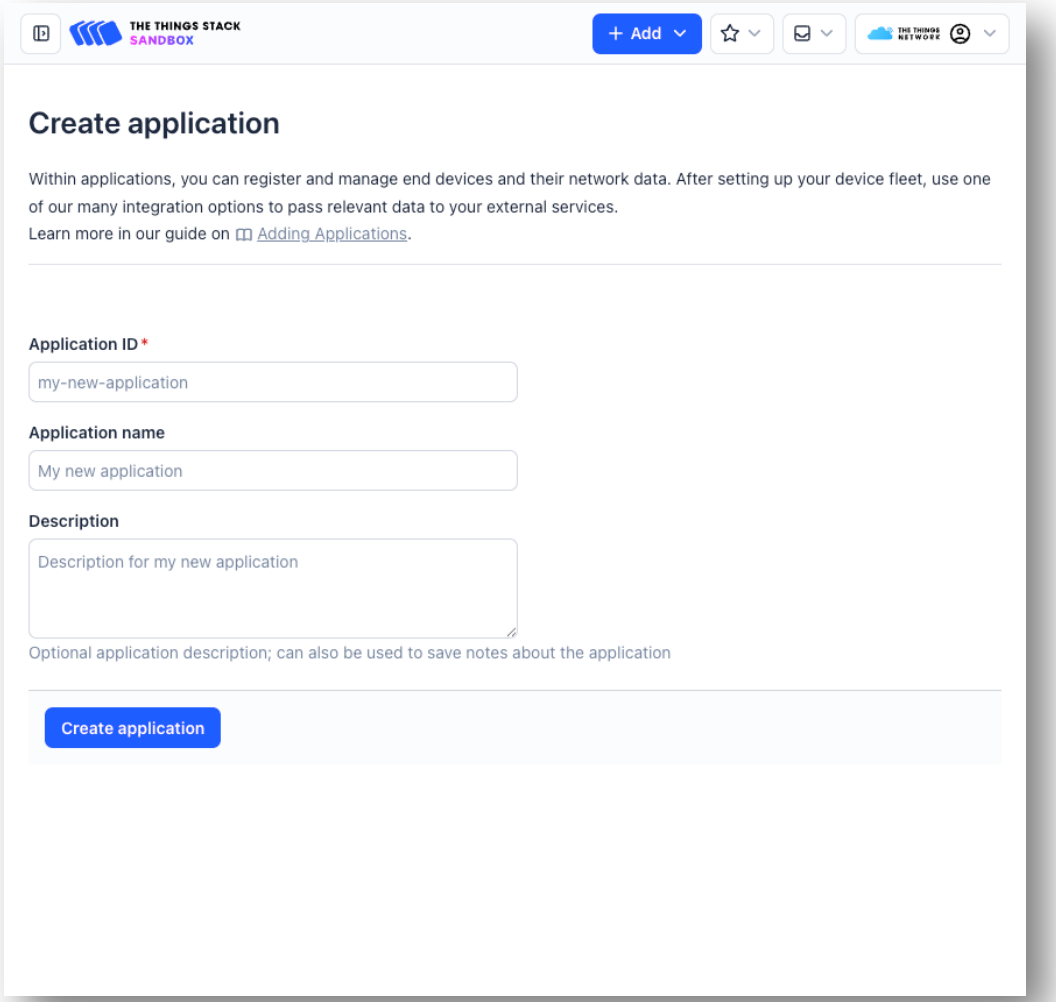
The Things Network: Application entity

- An Application in TTN environment is used to represent a collection of LoRaWAN devices with the same purpose / of the same type

  e.g.: LoRaWAN weather stations collecting data at different locations

- Application also allows to manage integrations for data
- Relevant Parameters:
  - ID: a unique textual identifier for the application

    e.g.: `smart-unipr-campus`



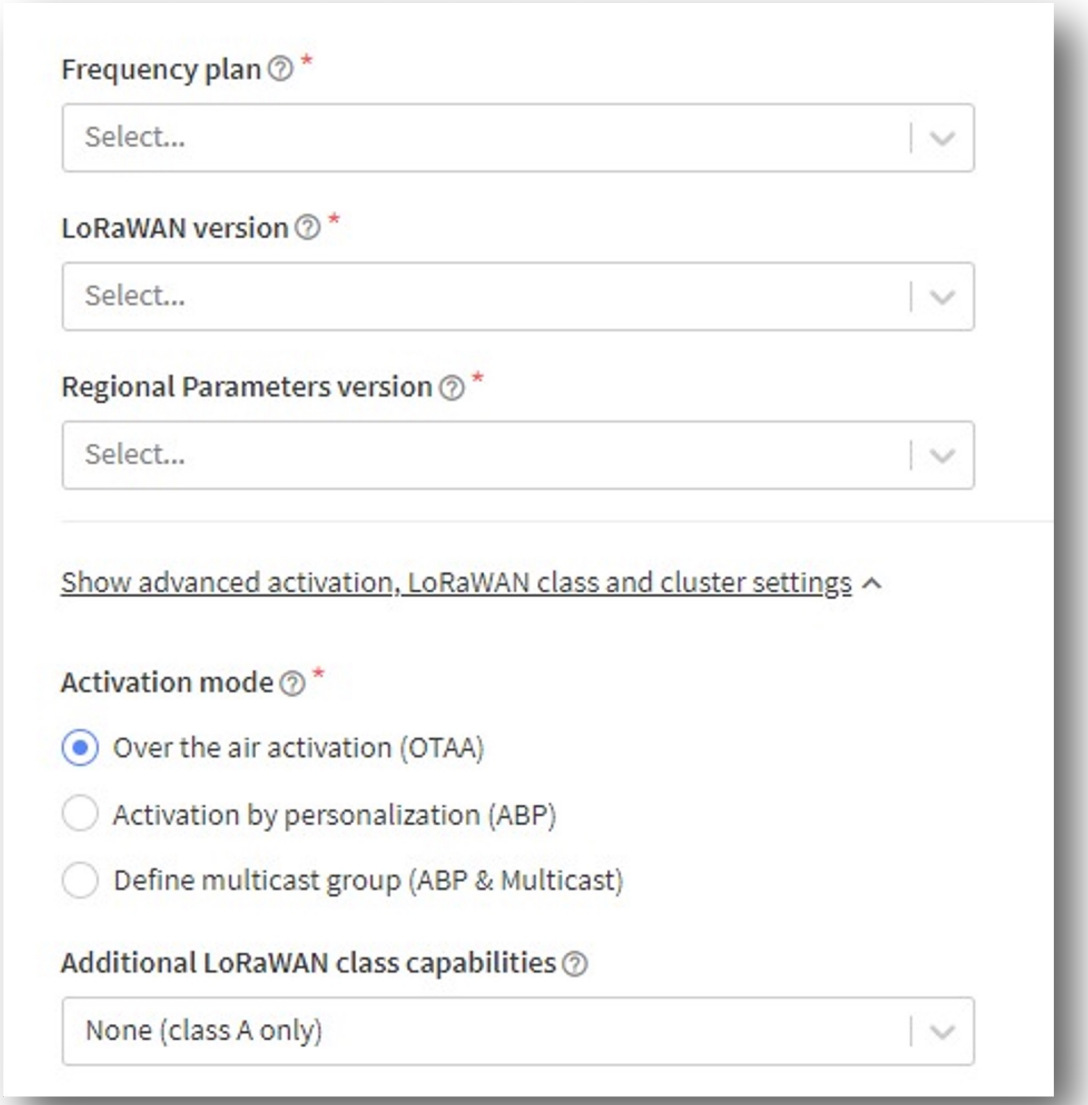
The Things Network: End Device Entity (1/2)

- LoRaWAN end nodes must be registered under an application
- Relevant parameters:
  - Frequency plan: should be the same of the registered gw!

    e.g.: `EU_863_870_TTN`
  - LoRaWAN protocol version: the LoRa Alliance LoRaWAN specification your device conforms to, which defines which MAC features it supports. It is provided by manufacturer

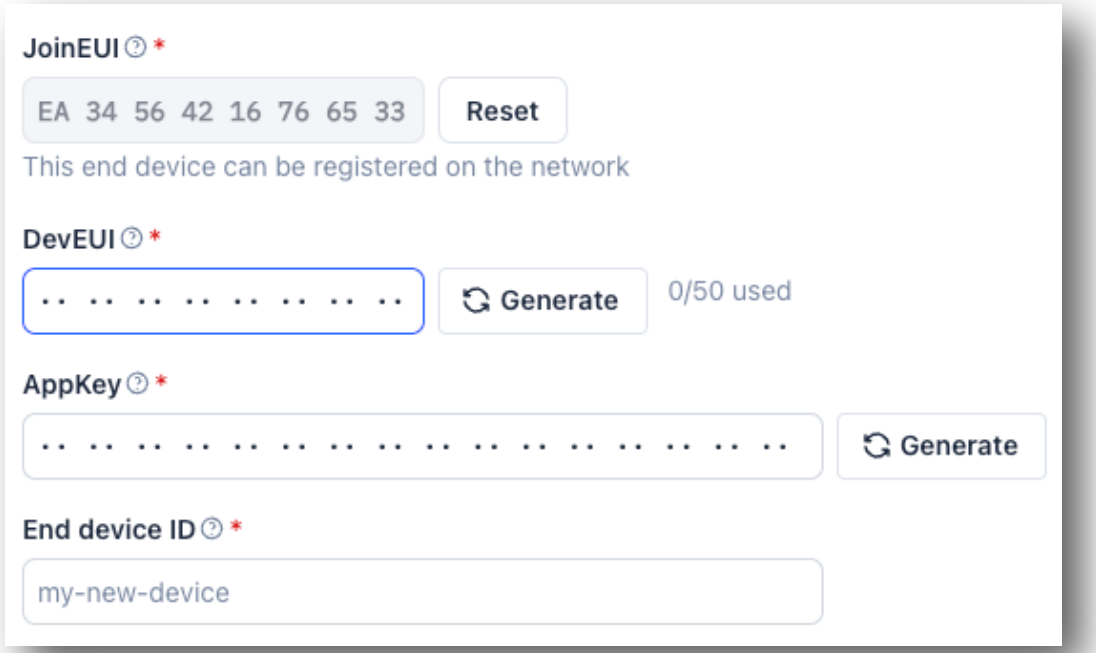
    e.g.: `1.0.2 or 1.0.4`
  - Class capabilities: class A, B, or C

    e.g.: `A`

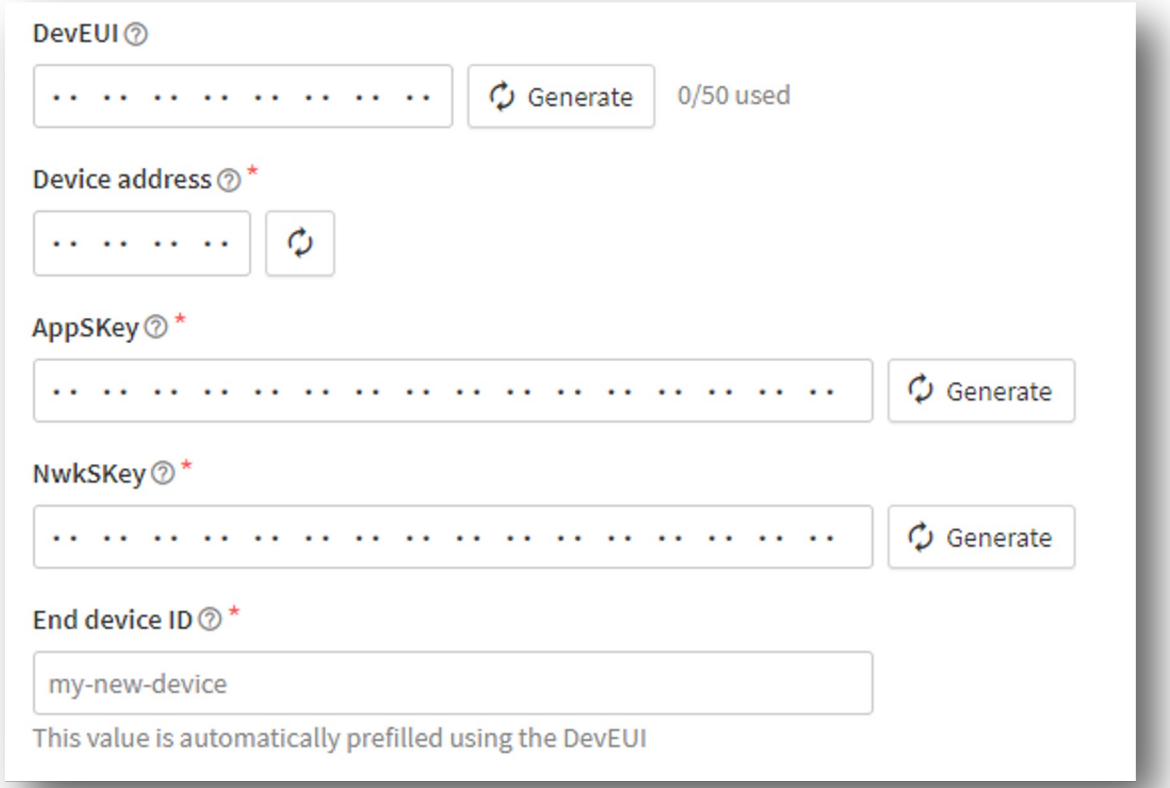


The Things Network: End Device Entity (1/2)

- DevEUI: a 64 bit unique identifier for the device (is provided by the manufacturer and should not be changed)
- End device ID: textual identifier



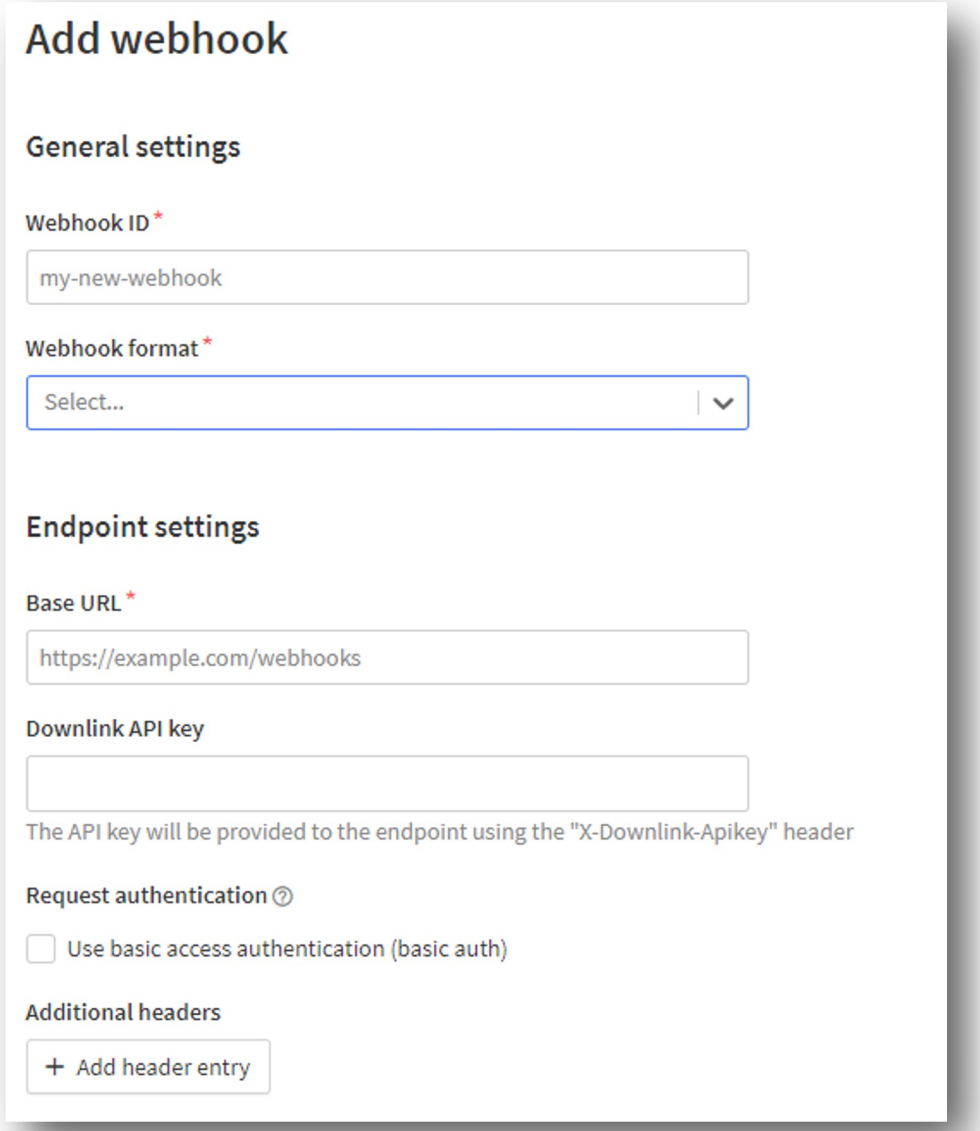
OTAA Activation parameters



ABP Activation parameters

The Things Network: Integrations

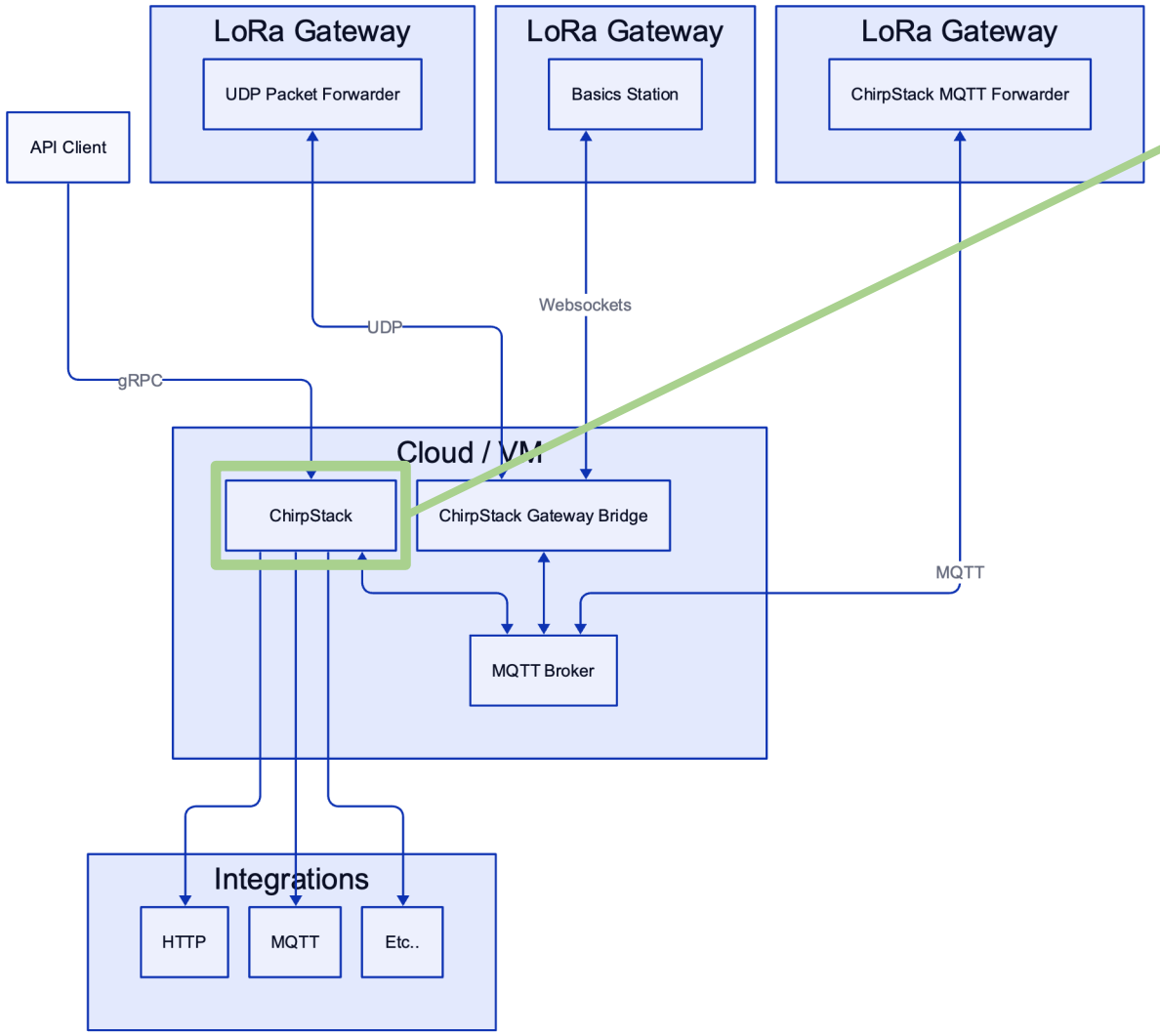
- Integrations connect devices to the external world (e.g.: custom external applications)
- They allow to trigger events whenever a device communicates (join, uplink, downlink)
- The Application server manages integrations using Webhooks (namely HTTP POST calls), MQTT, or other
- Supported data format: JSON or ProtoBuf



ChirpStack

- ChirpStack is an open-source LoRaWAN Network Server stack providing a set of completely open-source software components to build LoRaWAN networks
- Ready-to-use solution that can be installed on private servers or personal PC
- Includes a user-friendly web-interface for LoRaWAN network devices and gateways management and for integration
- All components are licensed under the MIT license and can be used for commercial purposes

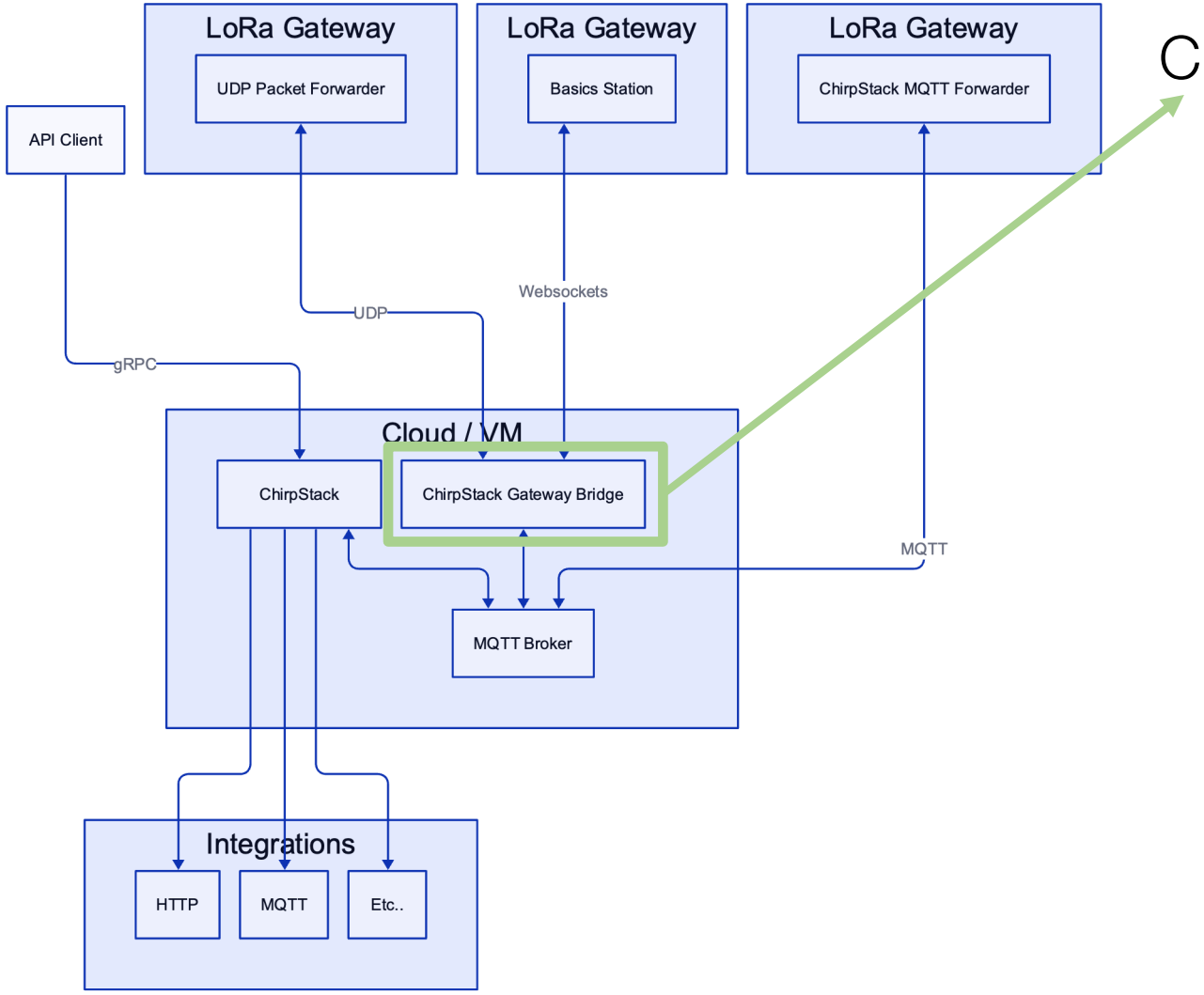
ChirpStack Architecture Main Components



ChirpStack:

- The LoRaWAN Network Server + Application Server implementation
- It provides a web-interface for the gateways, the devices and the integrations management
- Provides API that can be used to integrate or to extend ChirpStack

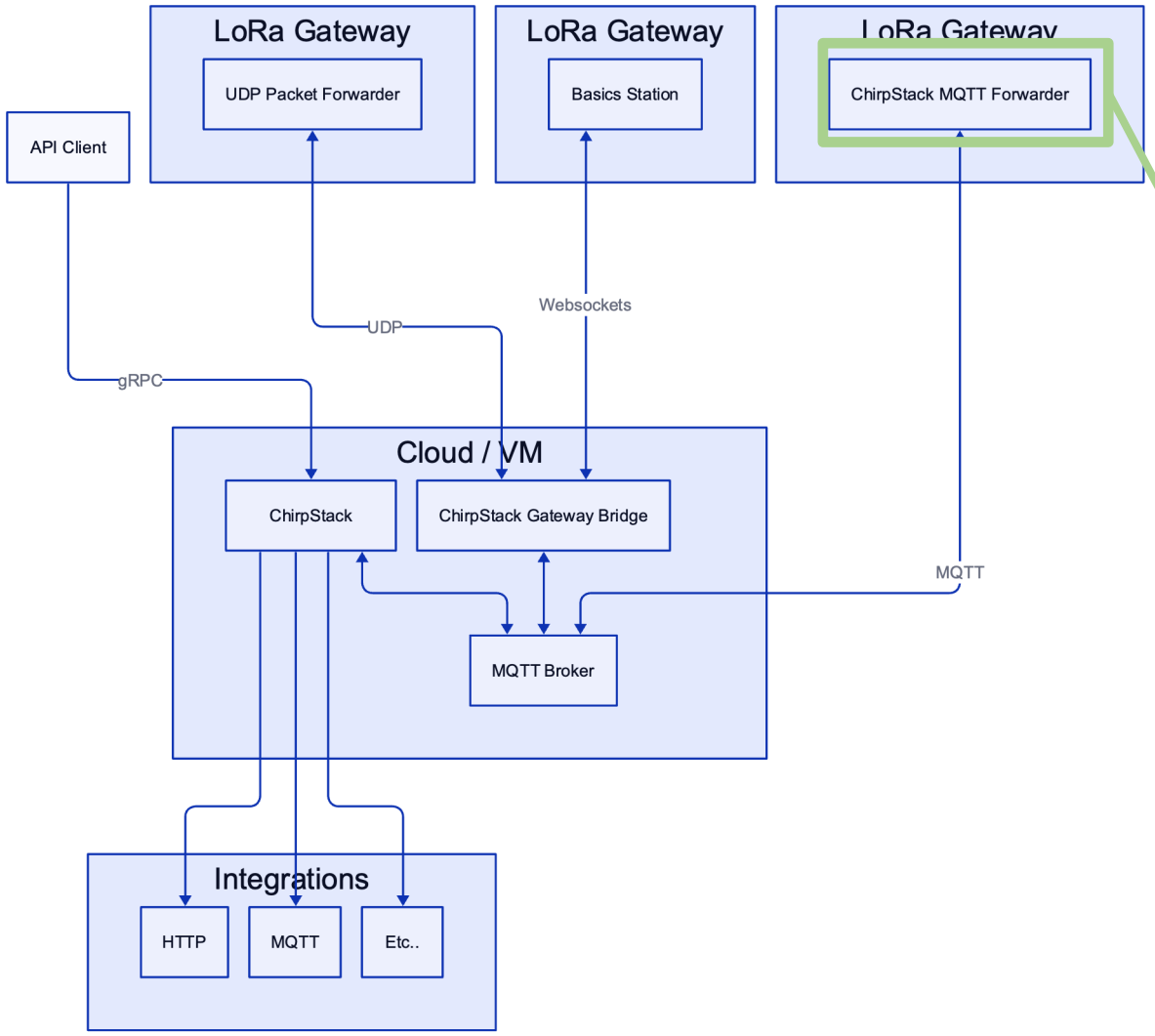
ChirpStack Architecture Main Components



ChirpStack Gateway Bridge:

- An open source bridge that handles the communication with the LoRaWAN gateways and the Network server
- It converts messages from the Packet Forwarder format (used in Semtech UDP Packet Forwarder or Semtech Basic Station) into the ChirpStack data format (MQTT)
- It can be installed on the server or on the gateways

ChirpStack Architecture Main Components



Other relevant components:

- ChirpStack Concentratord: the first ChirpStack implementation of the Packet Forwarder software (simpler)
- ChirpStack MQTT Forwarder: the open source ChirpStack implementation of the Packet Forwarder software (to be installed on gateways) based on the MQTT protocol

ChirpStack Installation (1/2)

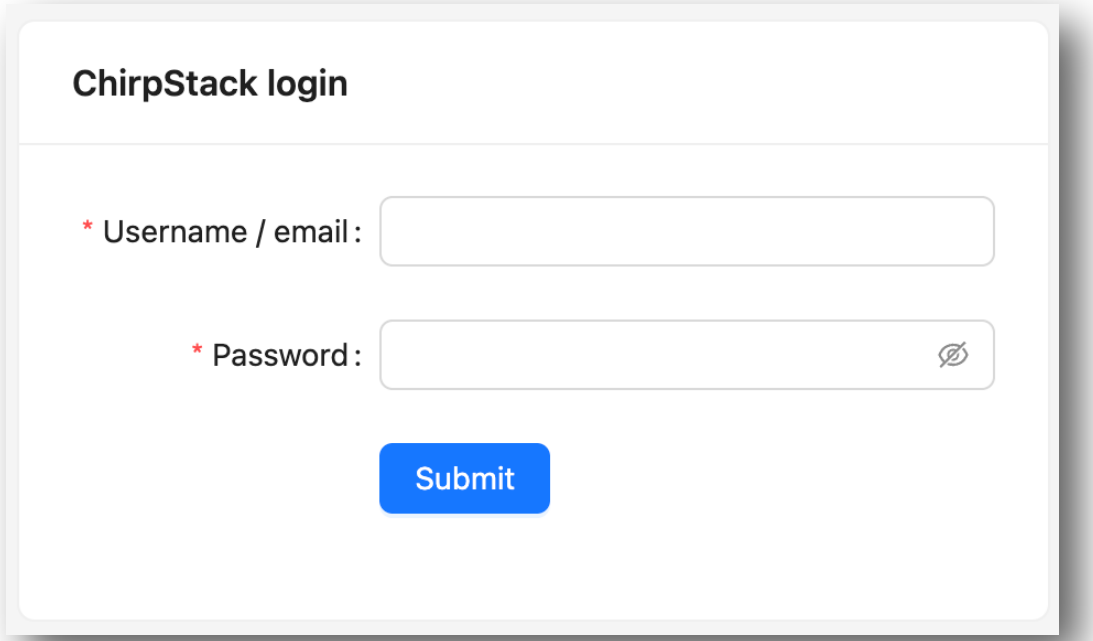
- ChirpStack can be installed on Debian or Ubuntu distributions, but also on Raspberry Pi boards
- Before starting ChirpStack installation you should install some dependencies:
  - **MQTT broker**: a publish/subscribe protocol that allows users to publish information under topics that others can subscribe to. A popular implementation of the MQTT protocol is Mosquitto
  - **Redis**: an in-memory database used to store relatively transient data
  - **PostgreSQL**: the long-term storage database used by the open source packages

ChirpStack Installation (2/2)

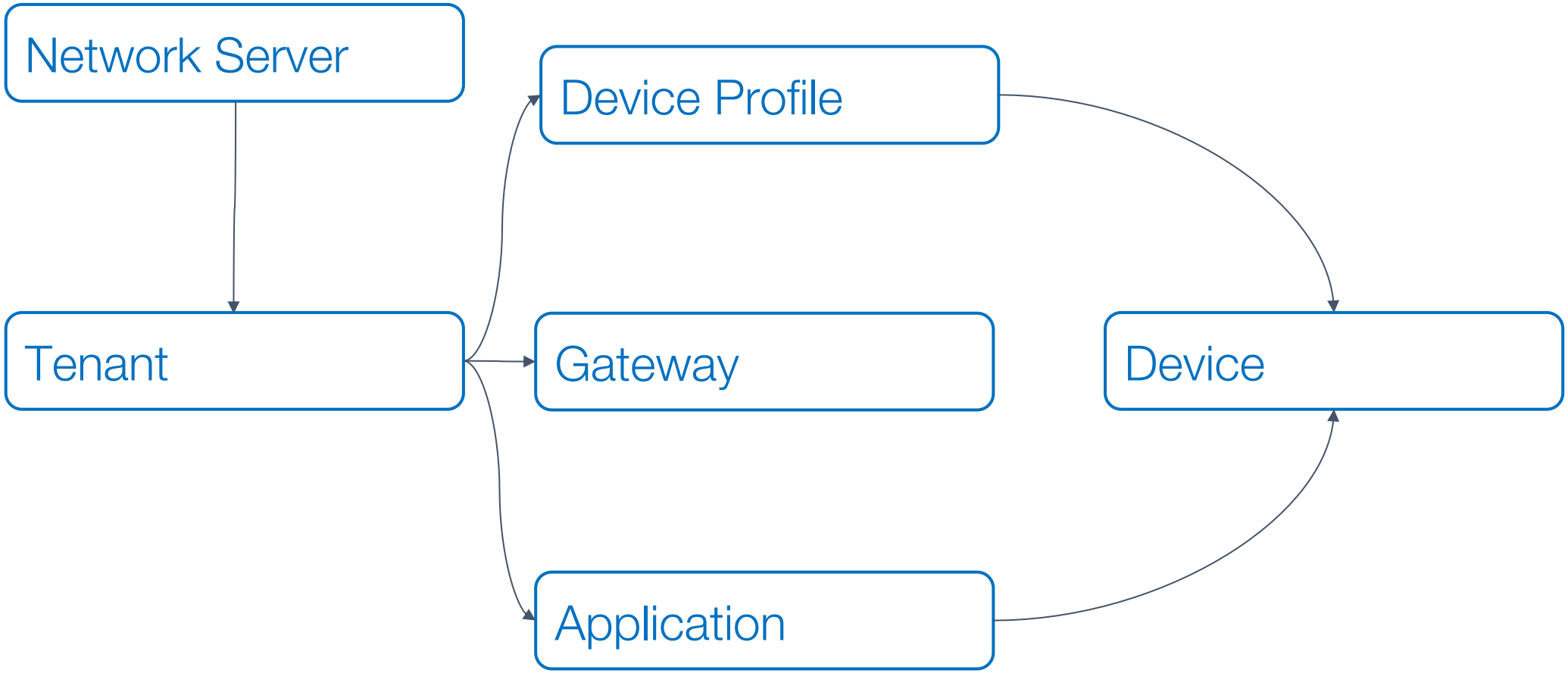
- You can install the ChirpStack Gateway Bridge on the Server in order to manage gateways not mounting it
- The ChirpStack NS supports all regions that are defined by the LoRaWAN Regional Parameters specification.

  → It should be configured for a specific region (e.g.: EU-868) through a JSON configuration file
- After installing all components you can access the web interface as admin at:
- `http://<my-server-ip>:8080`

```
sudo apt install chirpstack-gateway-bridge
…

sudo apt install chirpstack
…

# start chirpstack
sudo systemctl start chirpstack
```



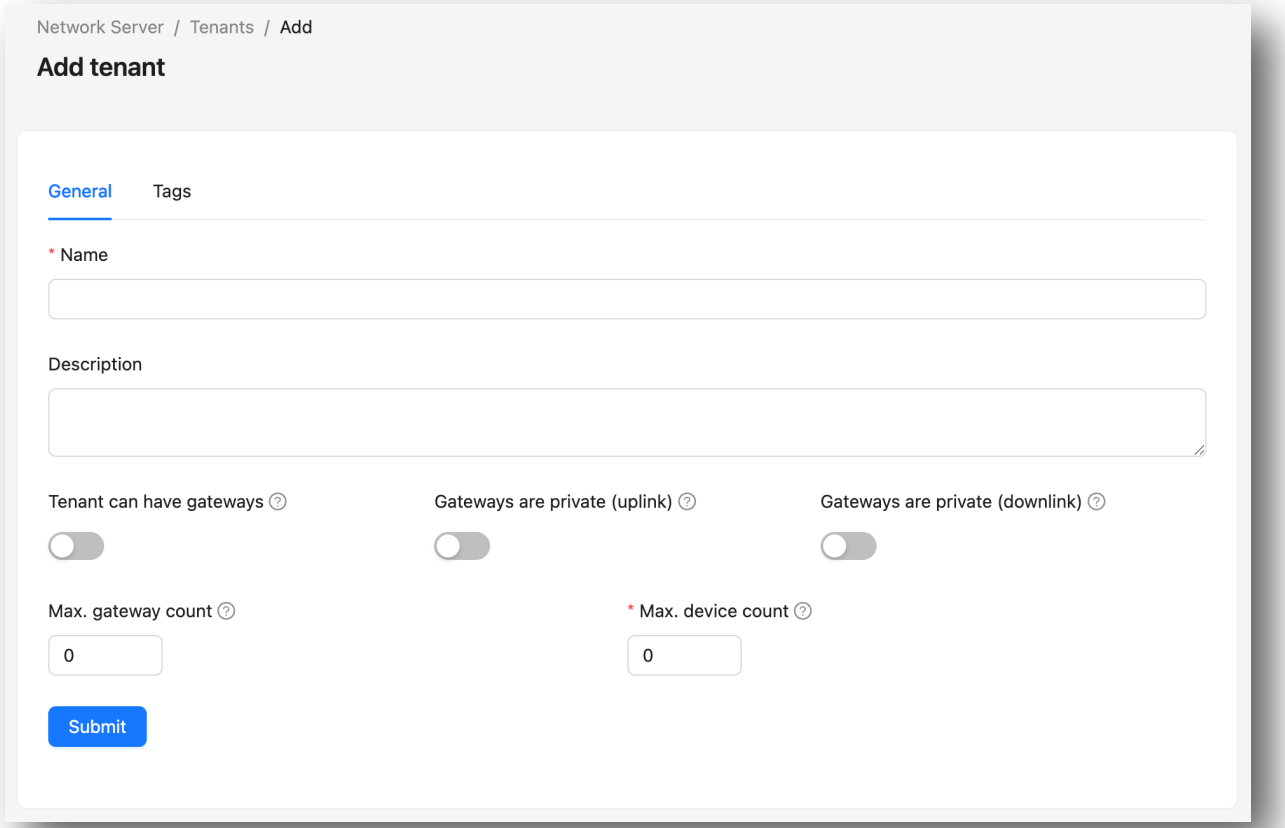
ChirpStack Entities Summary



ChirpStack Entities: Tenants

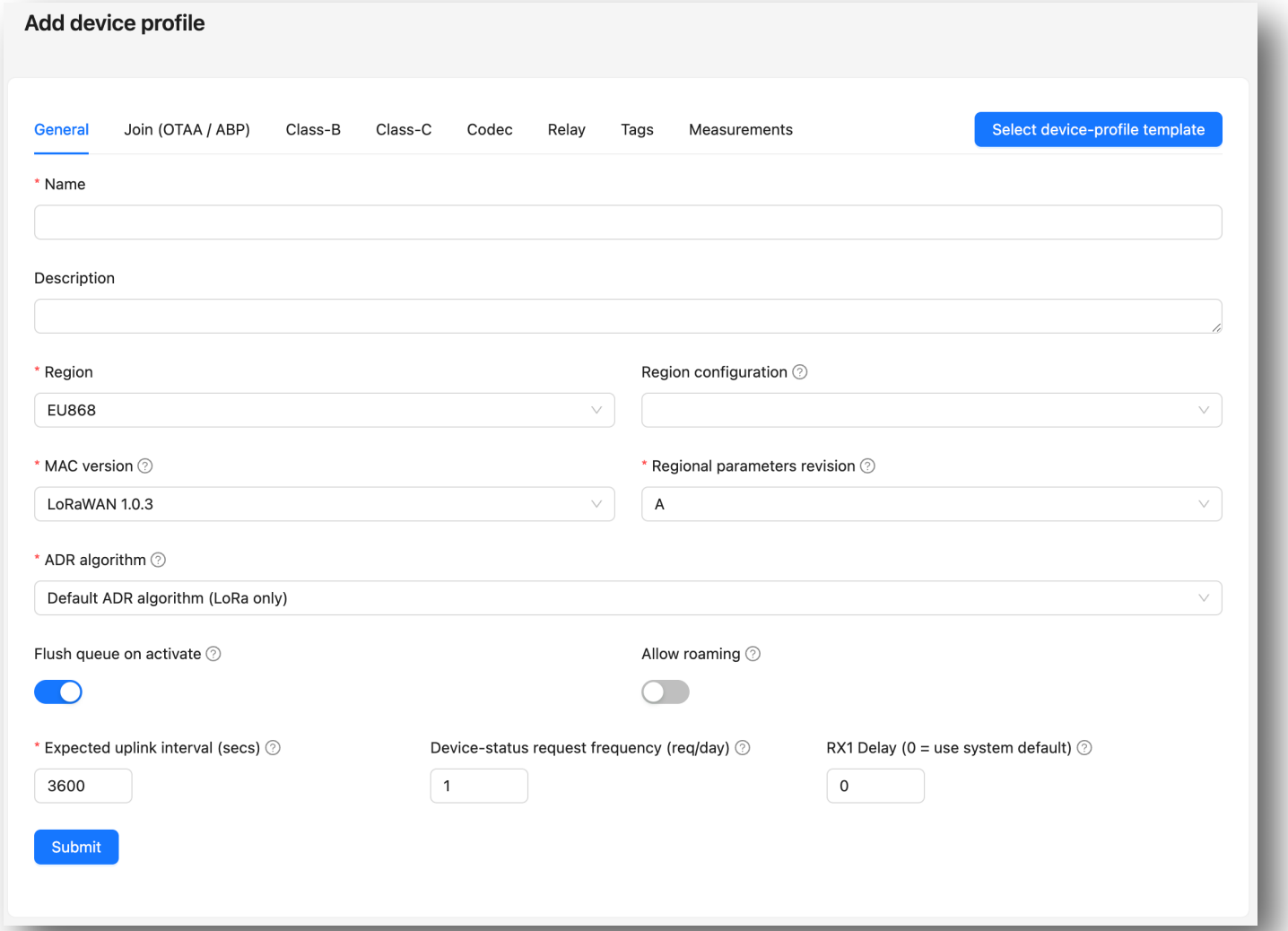
- Tenants can be used to let organizations or teams manage their own applications and optionally their own gateways

  → The admin account has visibility of all tenants
- A tenant can have:
  - Device Profiles
  - Gateways (when allowed)
  - Applications
  - Users



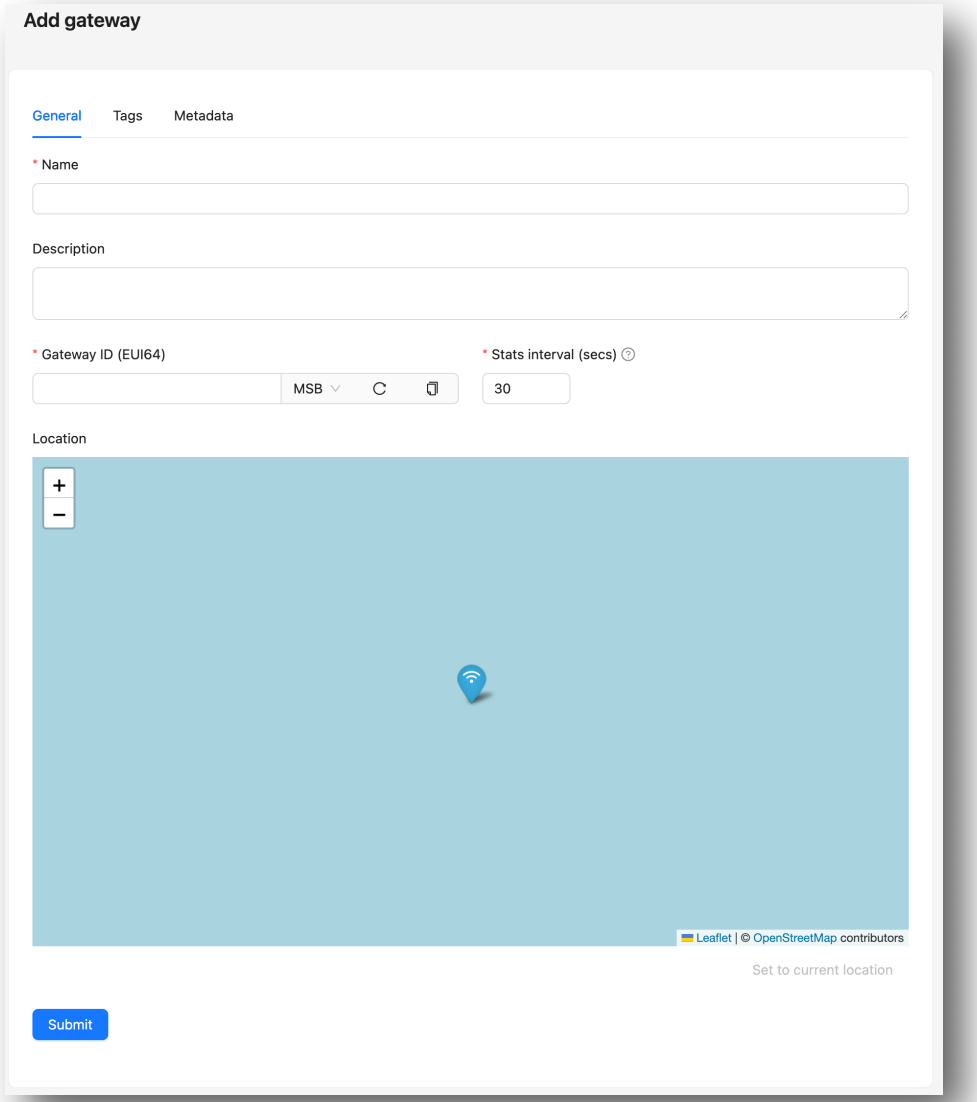
ChirpStack Entities: Device Profiles

- Device Profile: defines the device capabilities and parameters that are needed by the NS for setting the LoRaWAN radio access service
- It allows to group end nodes with the same behavior based on:
  - LoRaWAN protocol version and Regional parameters
  - Activation mode
  - Class
  - Codec
- These information elements shall be provided by the end-device manufacturer
- When creating a Device Profile, you have to specify the Tenant it belongs



ChirpStack Entities: Gateway (1/2)

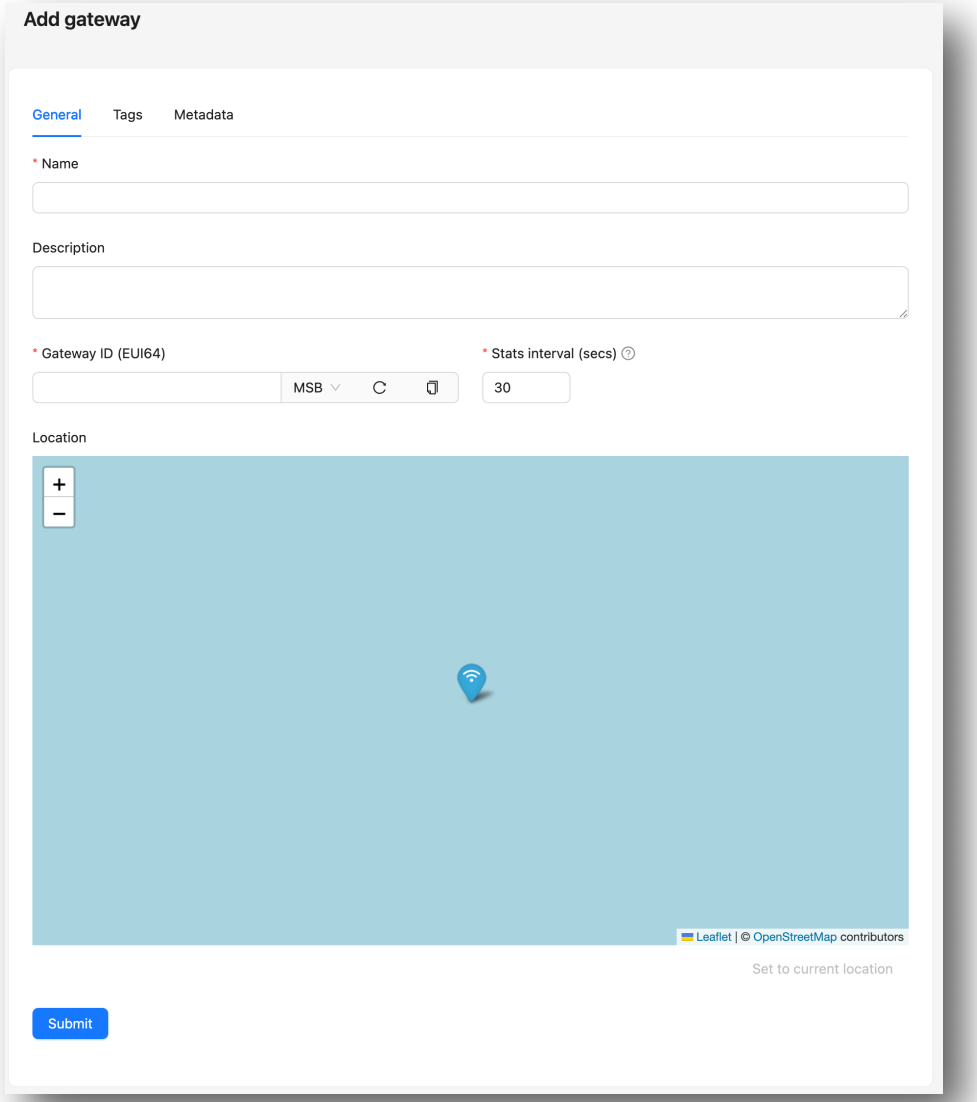
- Gateway: to register the Tenant gateways (if enabled)
- Similar to the TTN Gateway entity
- If a gateway belongs to a tenant does not mean that the usage of the gateway is limited to the tenant. Every device in the whole network will be able to communicate using the gateway
- The tenant will be responsible for managing the gateway details (e.g. name, location) and will be able to see its statistics



ChirpStack Entities: Gateway (1/2)

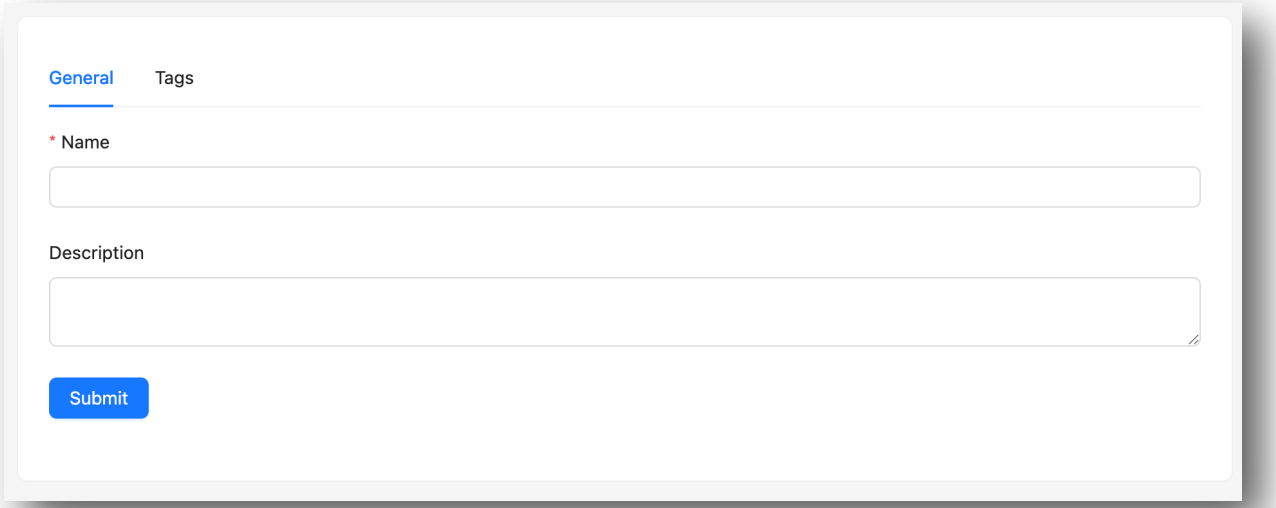
- Relevant parameters:
  - name: a unique textual identifier for the gw
  - ID: a hex encoded unique 64 bit identifier for the gw
    →**e.g.:** c64ae525e519bb9d

  - Geographical location



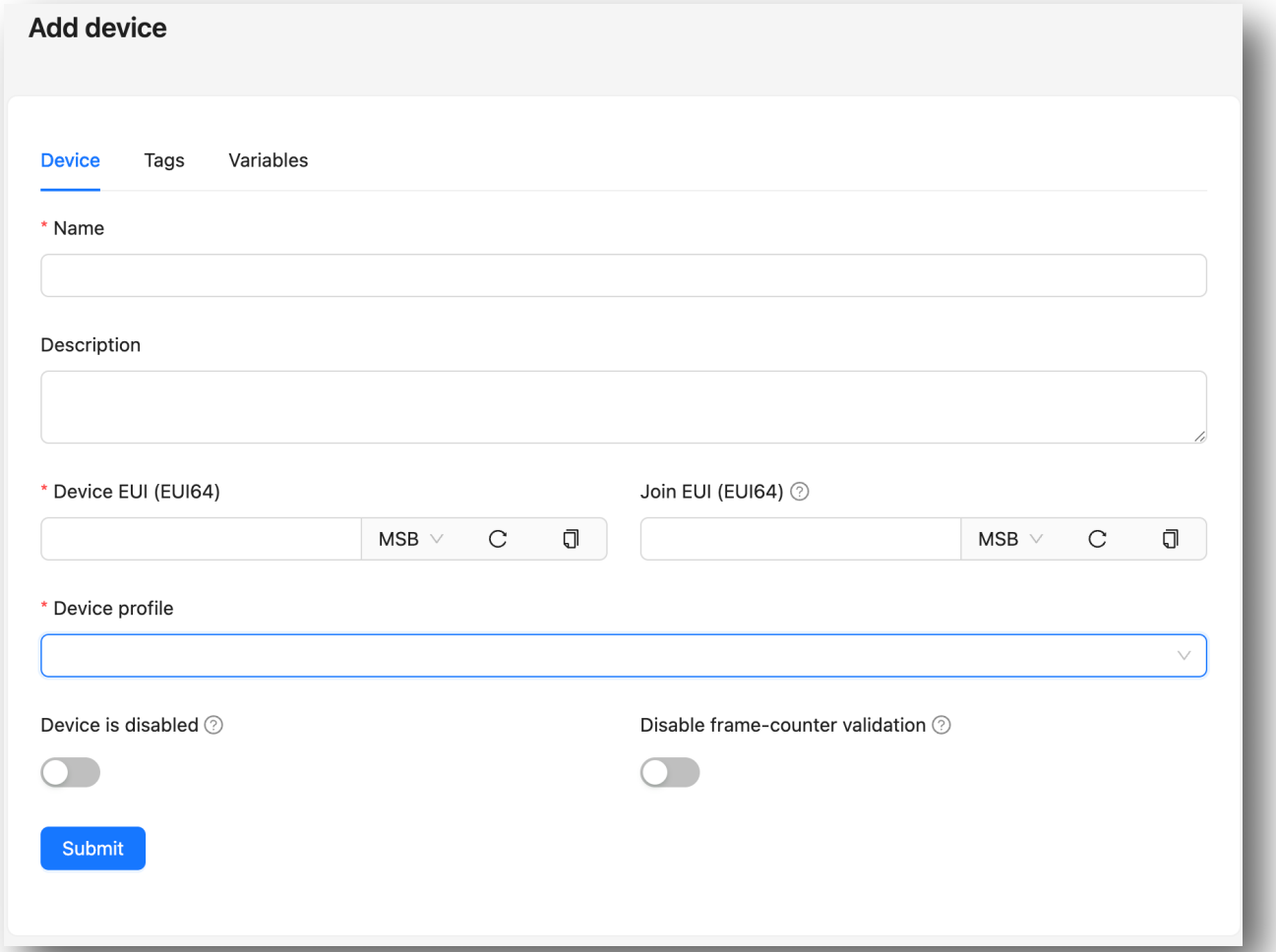
ChirpStack Entities: Application

- Application: is a collection of devices with the same purpose / type
- Similar to the TTN Application entity
- When creating an Application, you need to select the Tenant it belongs
- Multiple devices can be added to an application
- Integrations are intended at Application layer



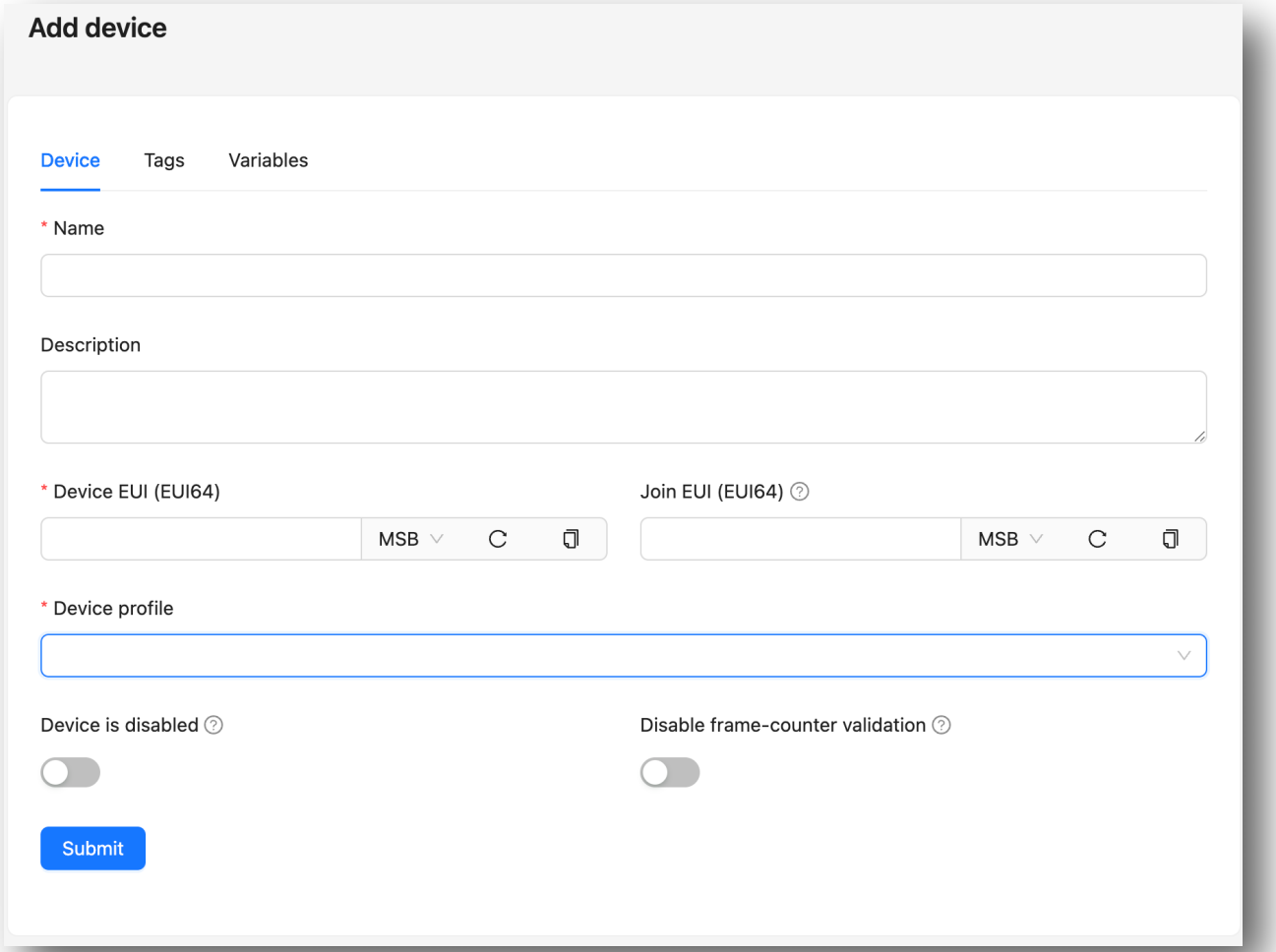
ChirpStack Entities: Device

- Devices: is the end-device connecting to, and communicating over the LoRaWAN network
- Device EUI: a hex encoded 64 bit unique identifier for the device
- When creating a Device, you have to select the application and the Device Profile, where are defined the join activation mode and other parameters



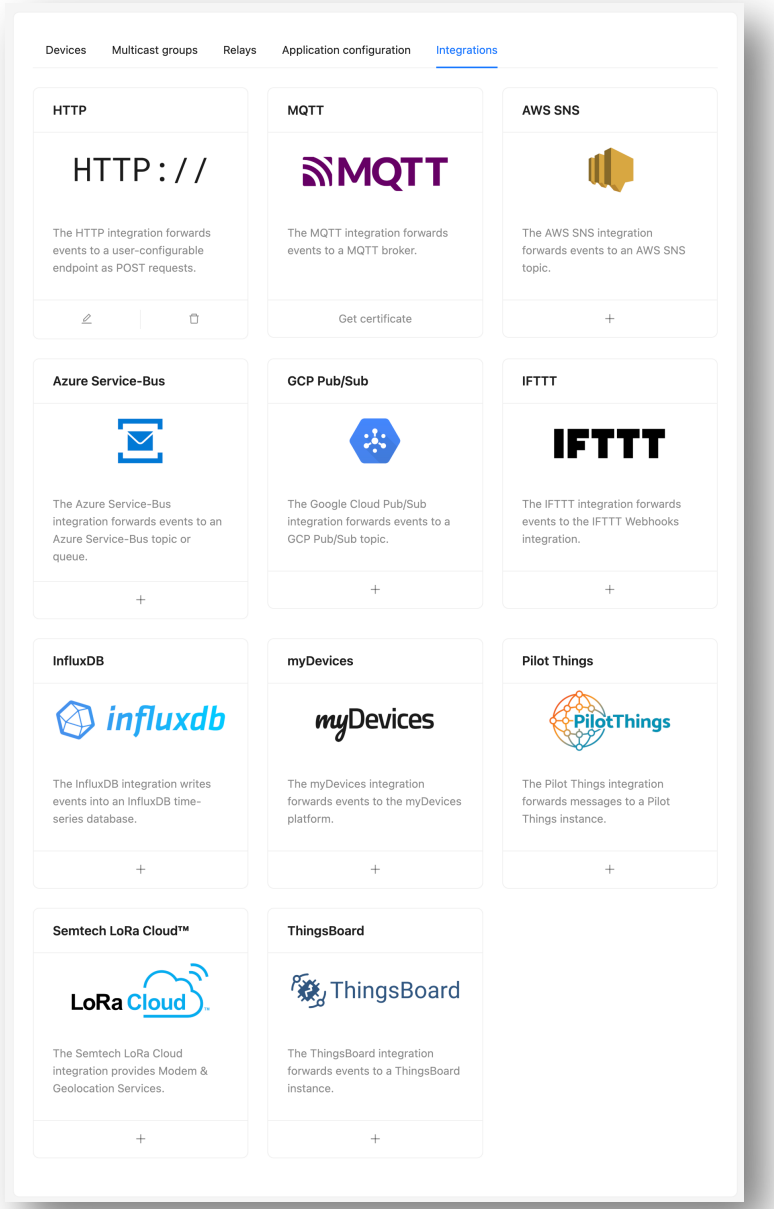
ChirpStack Entities: Device

- Device Activation
- After creating a device, you can manage the activation parameters
- An “activation” tab will appear with different fields depending on the device profile
- OTAA devices:
  - device EUI
  - application-key
  - appEUI (or JoinEUI)
- ABP devices
  - device address
  - network session key
  - application session key



ChirpStack – Integrations

- Like in TTN, Integrations connect devices to the external world custom applications
- They allow to trigger events whenever a device communicates (join, uplink, downlink)
- The Application server manages integrations using HTTP POST calls or MQTT
- ChirpStack also allow the use of HTTP APIs



TTN or ChirpStack?

Use TTN if:

- You simply want to test your devices
- You don’t have a gateway (maybe someone near you has registered his gw)
- You don’t have a host to use for NS installations
- If you want a global coverage

Use ChirpStack if:

- You need full control of network components
- If you want a particular behavior for gateways
- You want host NS and AS on your machine or on your device
- You want to create a private network

Devices: some examples @ IoTLab

- Microchip LoRa Pictail daughter board
- LoRaWAN transceiver module based on the Microchip RN2483 LoRa modem
- Can work as class A and C device
- LoRaWAN v1.0.2 compliant
- Can be connected to a host computer through a standard USB interface
- Can be controlled sending high level textual commands using serial port (e.g. using Putty)

RN2483 commands (some examples)

- Commands syntax is composed by a keyword followed by parameters
- Available keywords are:
  - “sys”: for system level behavior actions (e.g. gathers status or hw information)
  - “radio”: for radio specific configurations (LoRa level)
  - “mac”: for LoRaWAN communication configuration and actions

```
sys get ver //Returns the information on fw version
> RN2483 1.0.5 Oct 31 2018 15:06:52

sys get hweui //Retuns the globally unique EUI for device
> 0004a30b00f56978
```

```
radio set sf sf7
//sets the spreading factor used during transmission
> ok
```

```
mac set class c //Set the class for the device
> ok

mac join otaa //Join the network
> ok
> join accepted
```

Devices: some commercial examples @ IoTLab

Milesight EM500-PP: pipe pressure sensor

Milesight EM500-smtc: soil moisture, temperature and electrical conductivity sensor

Milesight EM500-CO2: CO_2, barometric pressure, humidity, temperature sensors

- Class A end-devices
- Work as standalone battery-powered nodes
- Can be configured through NFC communication and a manufacturer provided software

Devices: some commercial examples @ IoTLab

Milesight UC511:
Solenoid valve controller

Mclimate LorRaWAN water
Valve

Milesight WS523:
Smart Plug,

- Class A end-devices
- Work as standalone battery-powered nodes

Devices: some commercial examples @ IoTLab

Decentlab LoRaWAN temperature leaf sensor

Decentlab LoRaWAN stem dendrometer

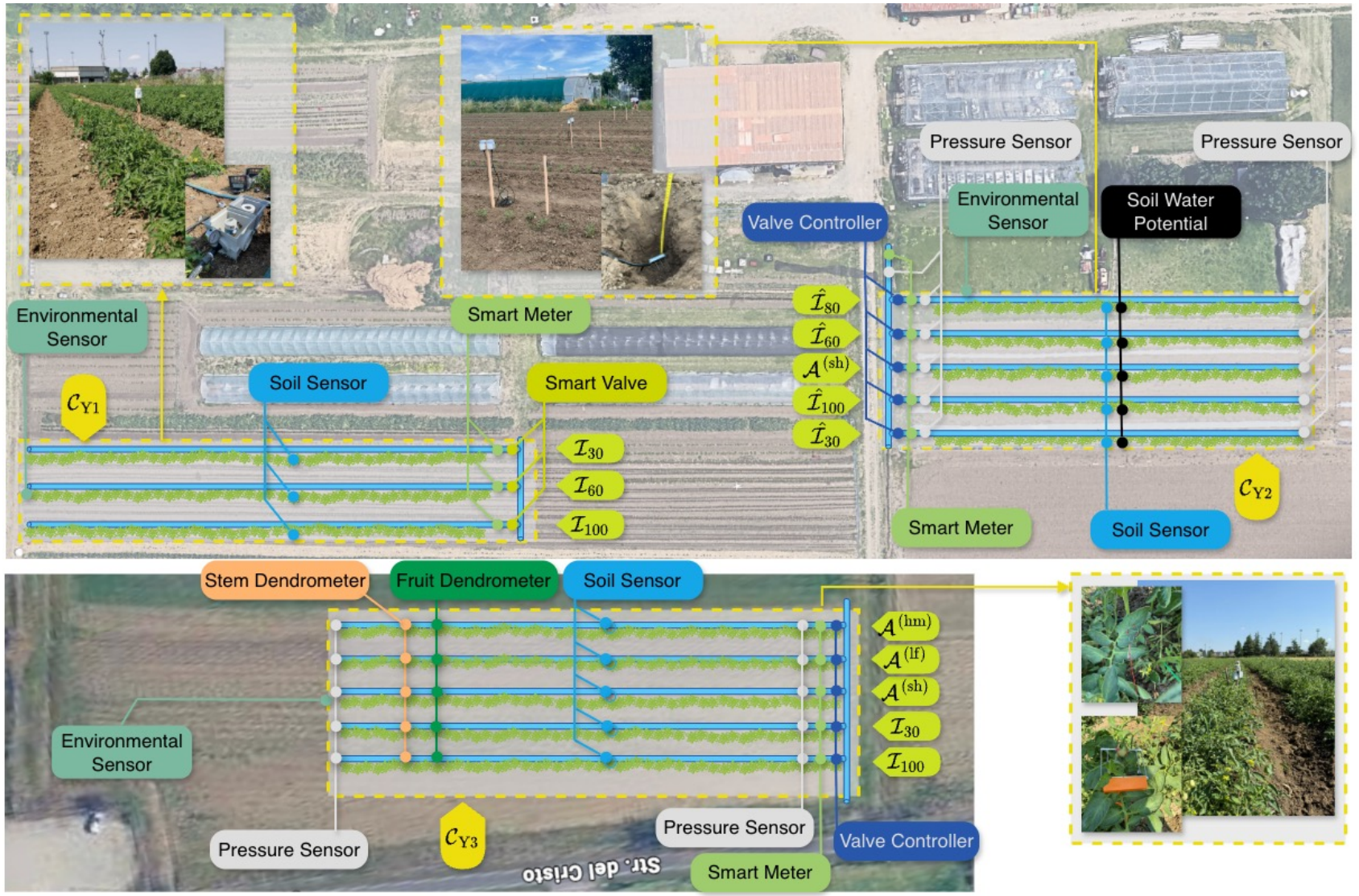
Decentlab LoRaWAN fruit dendrometer

- Class A end-devices
- Work as standalone battery-powered nodes

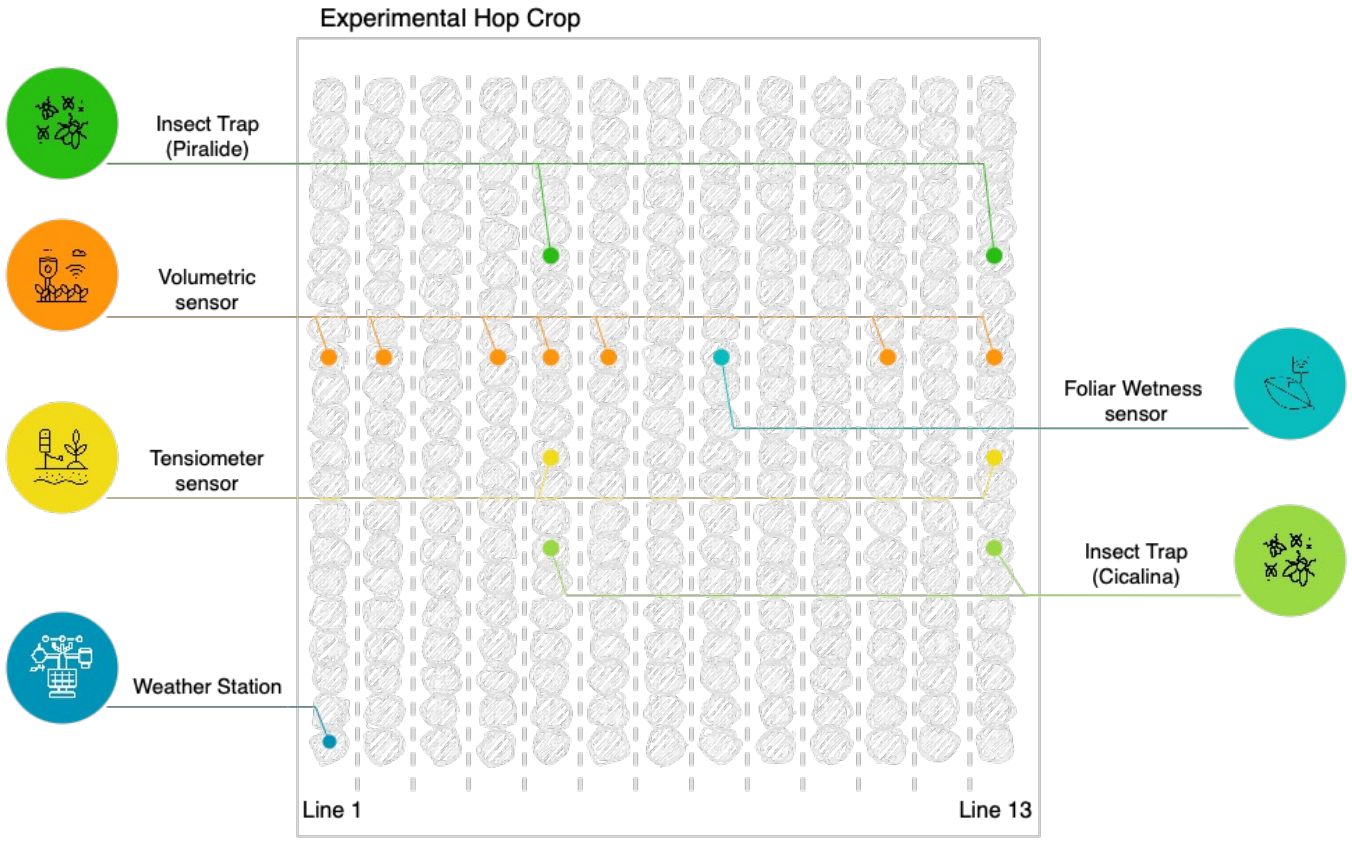
A commercial Gateway

- Milesight UG67 Outdoor gateway
- Include an embedded Network Server, but it’s compatible with TTN and ChirpStack
- Compatible with the EU868 Frequency plan
- Support up to 2000 end devices
- Deployed @iotlab
- Connection to network server trough Ethernet

A smart agriculture project @ IoTLab



A smart agriculture project @ IoTLab



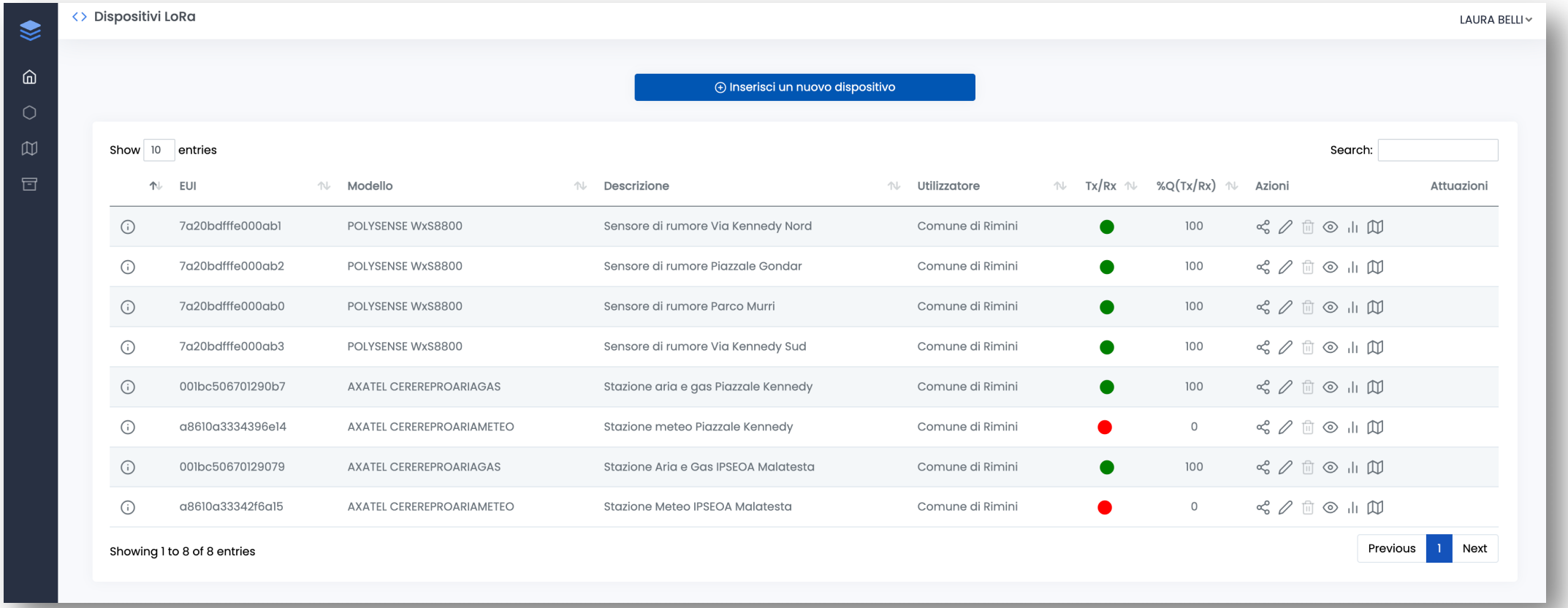
A smart agriculture project @ IoTLab

Smart city scenario: RetePAIoT

- It is a regional project developed by Lepida: a regional company working as a hub to support Emilia Romagna ICT development
- RetePAIoT aims at developing a LoRaWAN IoT network, in which citizens and companies, in addition to the Public Administration (PA) can integrate their own sensors
- It integrates several gateways in the area of the Emilia-Romagna region
- Everyone can access for free through the Italian digital identity SPID or CIE
- If you want to deploy or test a LoRaWAN device, and you don't have a gateway you can use this service
- Possibility to share devices with other people or companies
- Access to the collected data through HTTP REST API

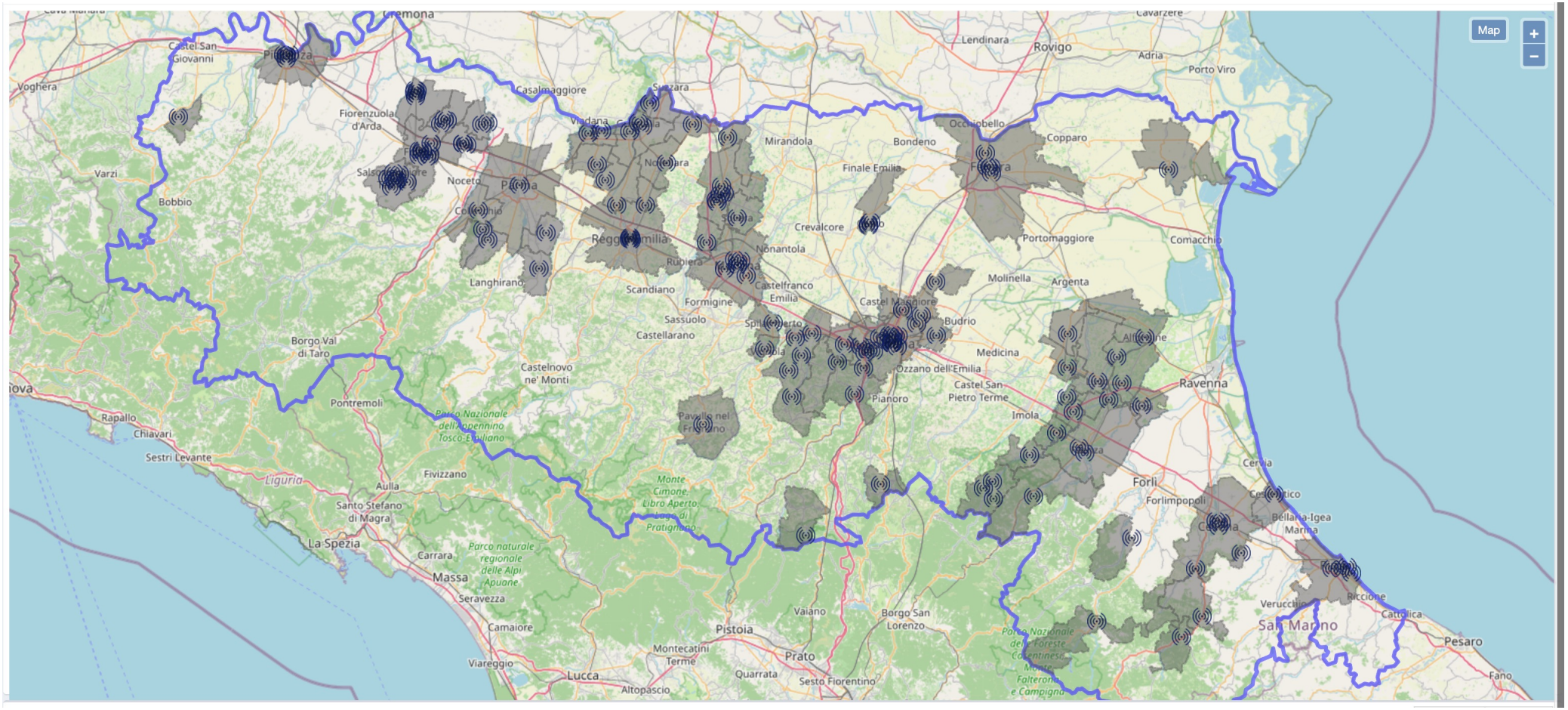
Smart city scenario: RetePaIoT

RetePAIoT dasboard



Smart city scenario: RetePaIoT

RetePAIoT covered areas and gateways



The Things Network: hands on devices

- Open the TTN Console at https://eu1.cloud.thethings.network/
- Create the “UniPR IoT Course 2026” Application
- Register a Soil Sensor (Milesight EM500-SMTC)
  - Frequency Plan: Europe 863-870 MHz (SF12 for RX2)
  - LoRaWAN Version : 1.0.3
  - AppEUI (JoinEUI): 24E124C0002A0001
  - DEVEui: 24E124126D119510
  - AppKey: generate from TTN
  - End device ID: soil-sensor-24e124126d119510
- Save the APP key on the device through the Milesight Toolbox App
- See Uplink and add the payload decoder
- https://github.com/Milesight-IoT/SensorDecoders/tree/main/em-series/em500-smtc

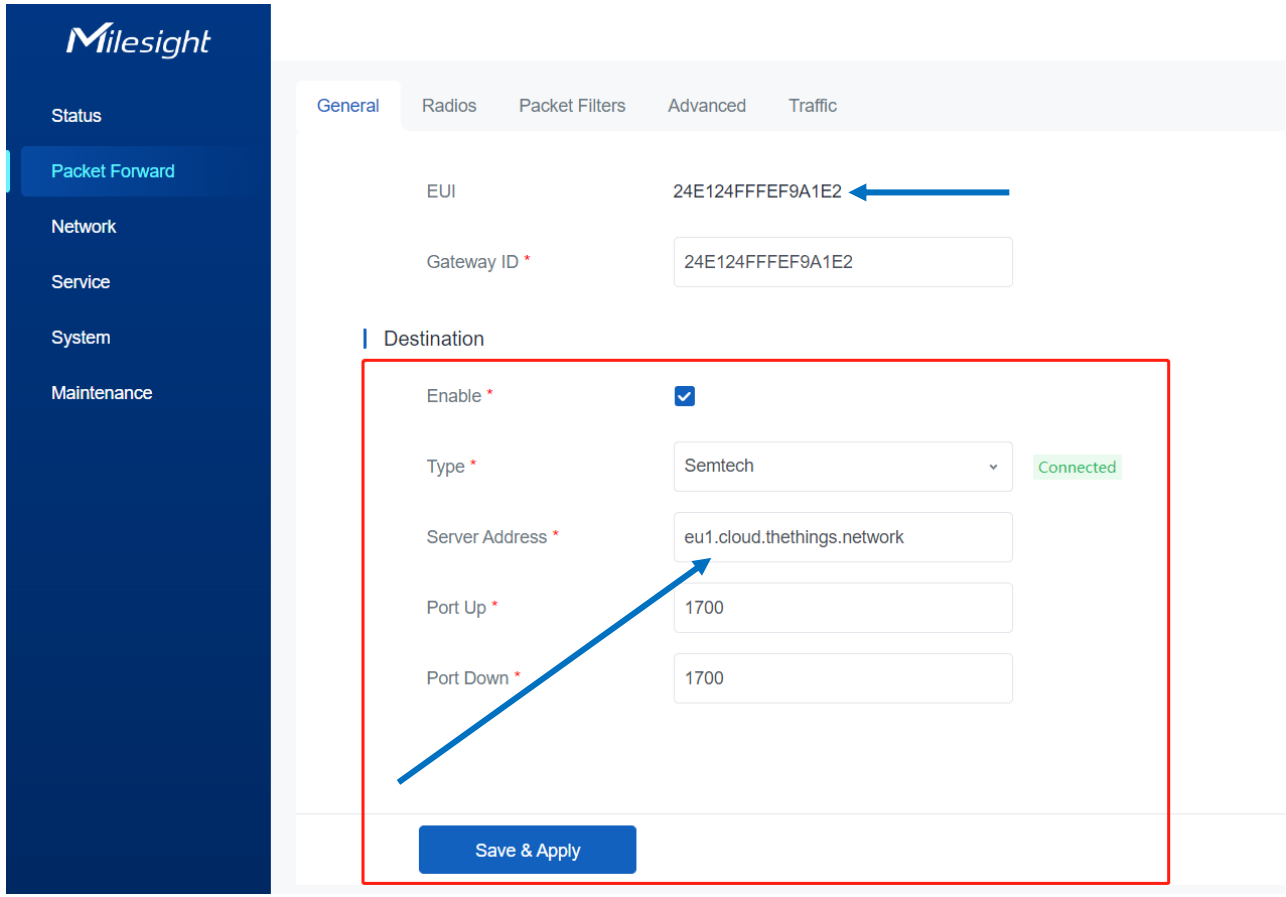
The Things Network: hands on gateways

- Open the TTN Console at
  https://eu1.cloud.thethings.network/
- Create the IoTCourse Application
- Register a new gateway (Milesight UG63)
  - Gateway EUI: 24E14FFFFEF8E338
  - Gateway ID: iot-course-2025-gw
  - Frequency Plan: Europe 863-870 MHz (SF12 for RX2)

- Change the Packet Forwarder configuration in the gateway

Gateway setup

- You need to configure the Packet Forwarder on the MileSight gateway to forward data on our TTN server
- We can do it through the web configuration dashboard provided by the gateway (the GW is connected to the Internet!)
- In this dashboard we can also see the Gateway EUI

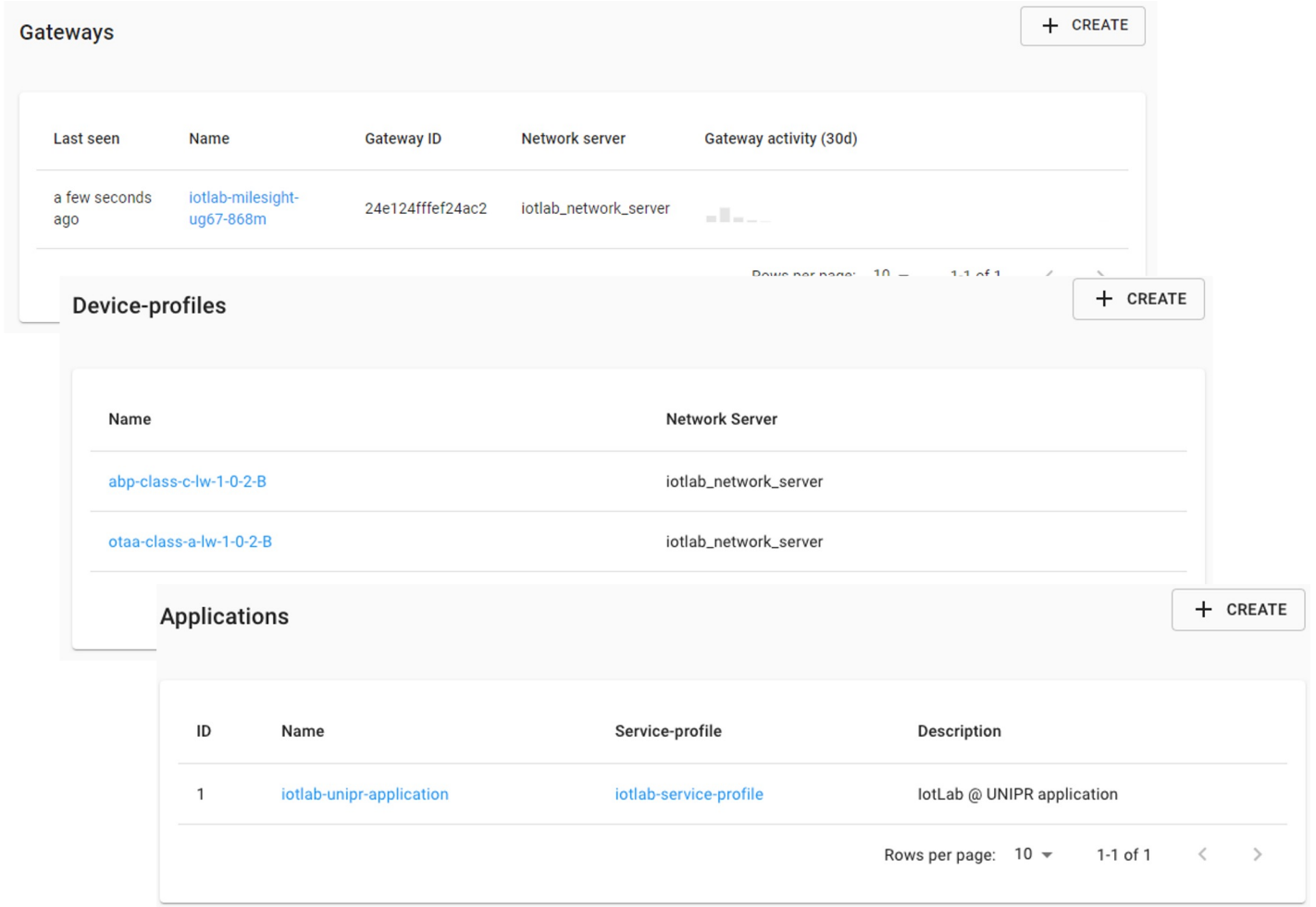


All together in a LoRaWAN Network example

- End node Alpha:
  - Microchip RN2483 LoRa Mote (support 1.0.2 LoRaWAN, regional parameter spec B)
  - Class A
  - Activation: OTAA
  - Sends temperature and light values trough on board application
- End node Beta:
  - Microchip RN2483 LoRa DAUGHTER BOARD (support 1.0.2 LoRaWAN, regional parameter spec B)
  - Class C
  - Activation: ABP
  - Sends CO_2 level
  - Accept downlinks to activate an acoustic signal if the CO_2 level is too high
- Gateway: Milesigth UG67
- Network Server and Application Server:
  - ChirpStack installed on a UNIPR host → we have admin privileges
  - Dashboard available on http://<unipr-host>:8080

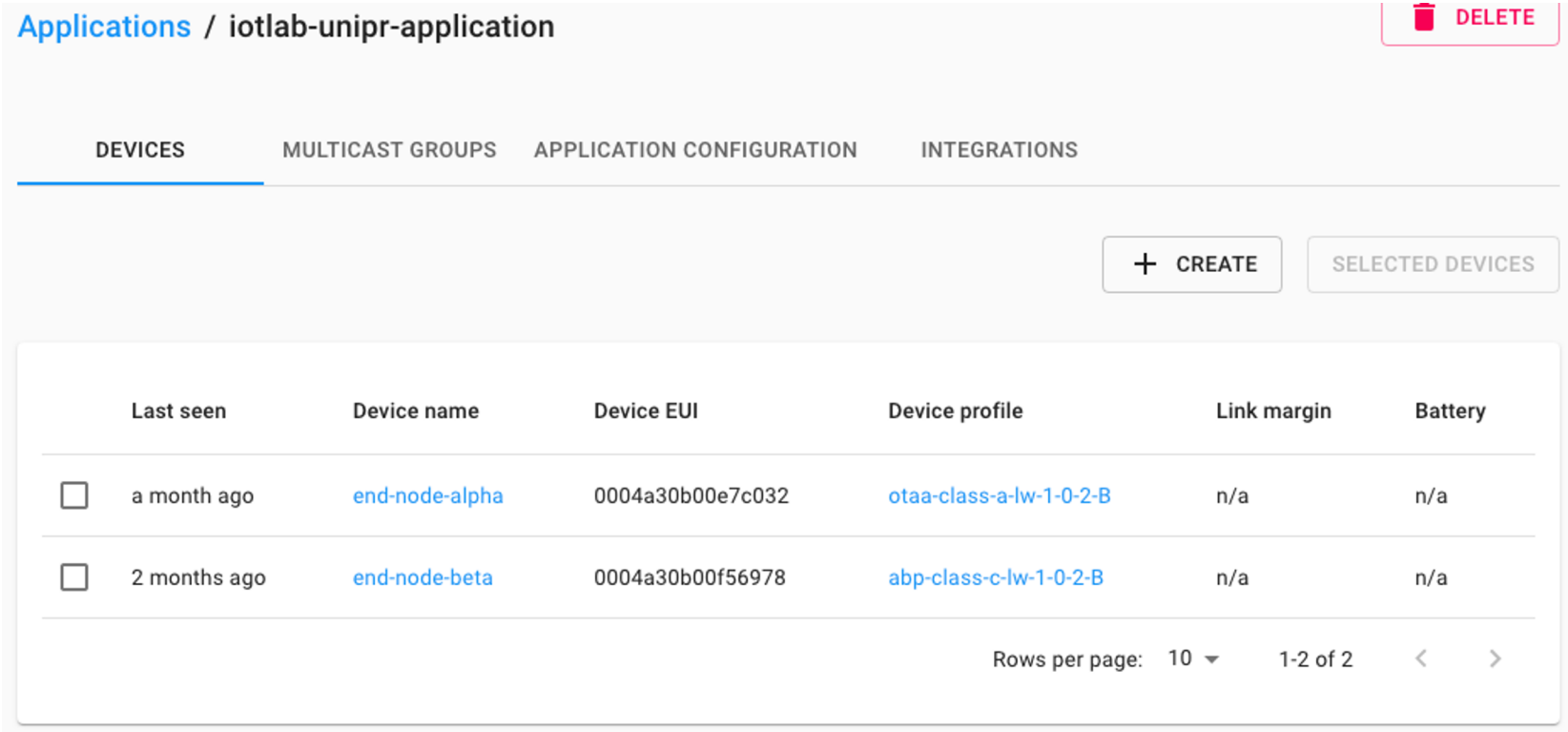
ChirpStack Entities

- Add a Tenant
- Add the Gateway (we know the unique EUI)
- Add 2 different Device Profiles
- Add an Application



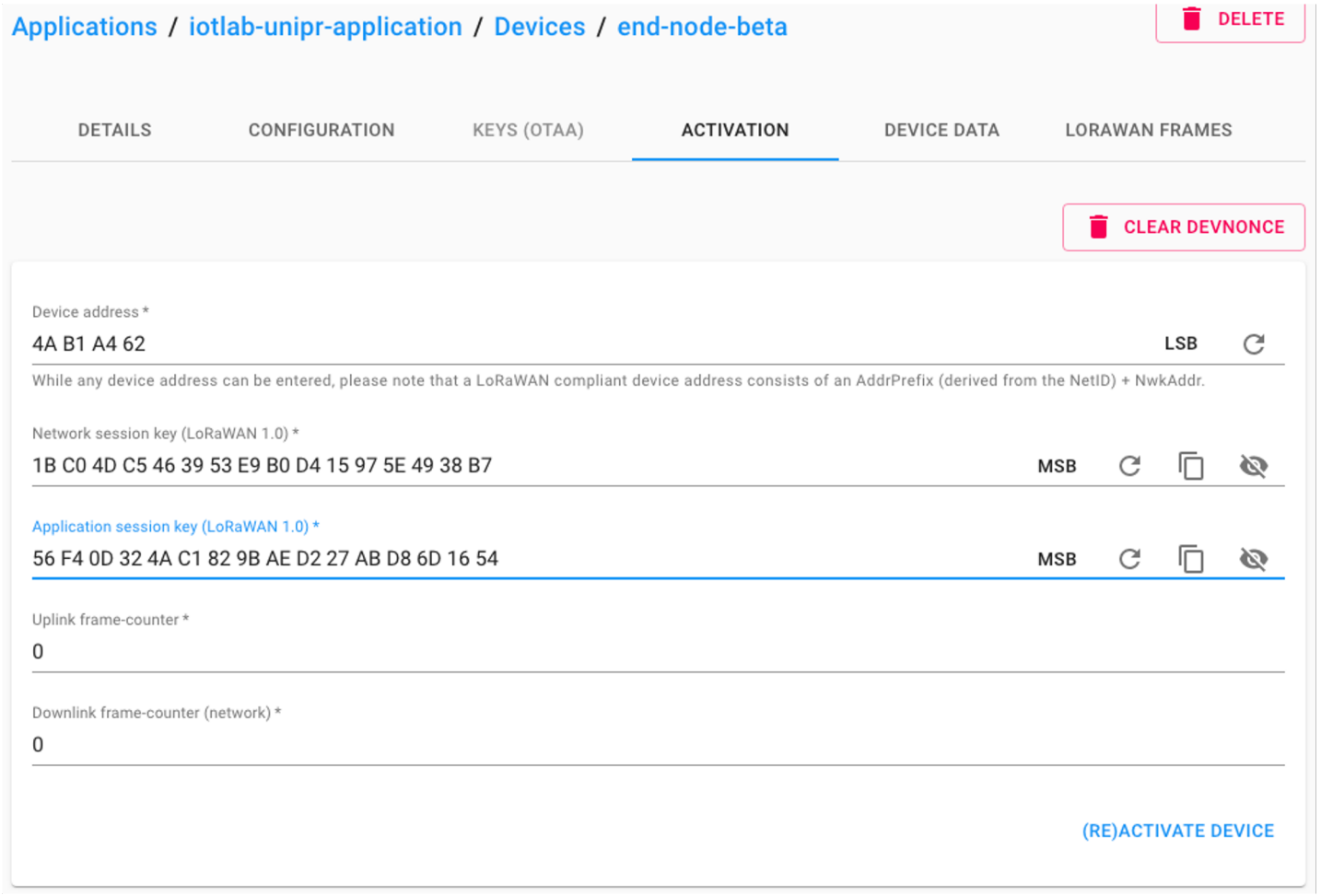
ChirpStack Entities

- Add devices Alpha and Beta
- Connect them to the correct device profile
- Add Activation parameters



ChirpStack Entities

Beta device ABP Activation parameters

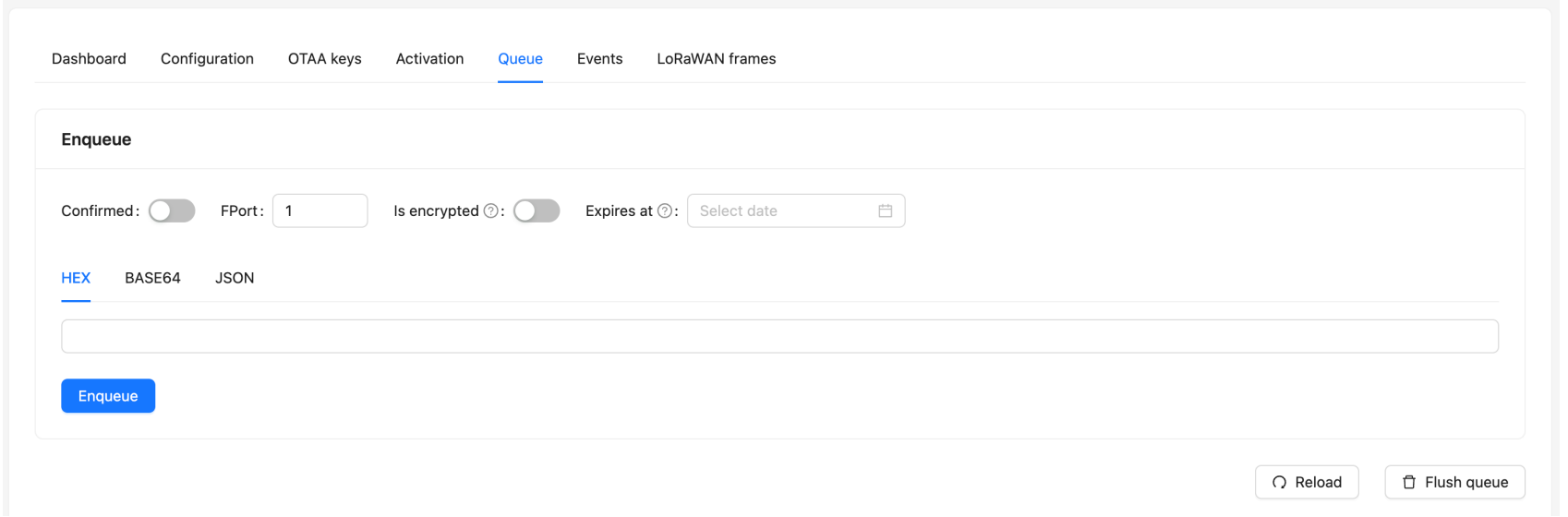


Endnode Beta setup

- Connect the device to a host computer
- Open a serial connection (e.g. with Putty program)
- You can send serial commands
- Suppose the $C0_2$ level is of 735 ppm (373335 hex) and you want to send data on port 22
- If you receive downlinks, in console will appear “mac rx”
- You can do it also with a program (e.g. Python script)

```
sys get hweui //Returns the globally unique EUI for device
> 0004a30b00f56978

mac set class c
> ok

mac set adr on
> ok

mac set devaddr 4AB1A462
> ok

mac set nwkskey 1BC04DC5463953E9B0D415975E4938B7
> ok

mac set appskey 56F40D324AC1829BAED227ABD86D1654
> ok

mac join abp
> join accepted

mac save
> ok
mac tx uncnf 22 373335
> ok
> mac tx ok

> mac rx 60 1
```

Enqueue of downlink messages for Beta



- Downlink messages can be enqueued through the ChirpStack device page
- Parameters:
  - FPort: eg: 60
  - type: confirmed or not
  - content: base64 encoded
- message 1 → MQ==

If you want to know more…

- https://www.thethingsnetwork.org/
- https://www.chirpstack.io/
- https://www.milesight-iot.com/lorawan/gateway/ug67/
- https://www.microchip.com/en-us/product/RN2483
- https://github.com/Lora-net/packet_forwarder
- …

LoRaWAN technology: An overview

IoT Lab

UNIVERSITÀ DI PARMA

April 10, 2025

Internet of Things, Communication Engineering Course

UNIVERSITÀ DI PARMA

IoT Lab

→ LEC. 15
(LEC. 12 → Security - Prof. Veltri
LEC. 13 → 6LoWPAN → Dr. Alfieri
LEC. 14 → 6LoWPAN → Prof. Bellini)

Routing in LLNs & RPL

Main sources

- J. P. Vasseur and A. Dunkels, `Interconnecting Smart Objects with IP,` Morgan Kaufmann, 2010. Chapter 17.

- Brandtr, Clausen, Dawson-Haggerty, Hui, Pister, Thubert, Winter, *RPL (ripple): Routing Protocol for Low Power and Lossy Networks – Walkthrough*, IETF `draft-dt-roll-rpl-01.txt`, IETF 75 – Roll WG – July 2009

From WSNs to RPL (1/2)

- Distance Vector routing (e.g., AODV) is widely used in WSNs
  - trade off between capabilities and resources
- Reduced states in nodes (vs. Link State)
- Simplicity => more control cost over time
- Improvements proposed
  - Loop detection
    - Packets received from a node that is on the way to the destination cause routing control
  - Trickle (adaptive adv period + suppression)
    - Limits the degradations due to node spatial density
  - Explicit link evaluation
    - Prior to selecting a parent as feasible

From WSNs to RPL (2/2)

- Expected Transmission Count (ETX) – MIT
  - Measure of the quality of a path between two nodes in a wireless packet data network
  - A lot more relevant than hop count in wireless
- Wikipedia:

"ETX is the number of expected transmissions of a packet necessary for it to be received without error at its destination. This number varies from 1 to infinity. An ETX of 1 indicates a perfect transmission medium, where an ETX of infinity represents a completely non-functional link. Because ETX is an *expected* transmission count for a future event, as opposed to an actual count of a past event, it represents a probability, and is therefore a real number, and not an integer. For example, if it took 1898 transmissions to transfer 1024 packets without error, the ETX on the link is 1898/1024, or approximately 1.85. Due to varying characteristics of the transmission medium, the number may vary widely."

$$ETX = \frac{\sum_{i=1}^{N} T_i}{N} \xrightarrow{N \to \infty} E[T_i] = E[T]$$

Distance Vector (DV) protocol in IP Networks

- RIP (Routing Information Protocol): #hops as metric
  - Unfit for large, unstable & complex topologies
  - Sync effect, transient loops, count to infinity
  - Partial fixes by split horizon (poison reverse)
- More recent protocols
  - Trade RIP's loops for temporary black holes
  - Path hold-down then route poisoning
  - Quarantine path for which hop counts goes up

The link model

- LLN connectivity might not match the traditional wired link model
  - Links are not a given.
  - Statistical connectivity might cause transient unreachability from/to a rather good neighbor
  - and might enable to receive a packet from far far away from an improbable neighbor
  - Not a RPL-specific issue but a LLN-issue that the IETF ROLL WG has considered

Low-power and Lossy Networks (LLNs)

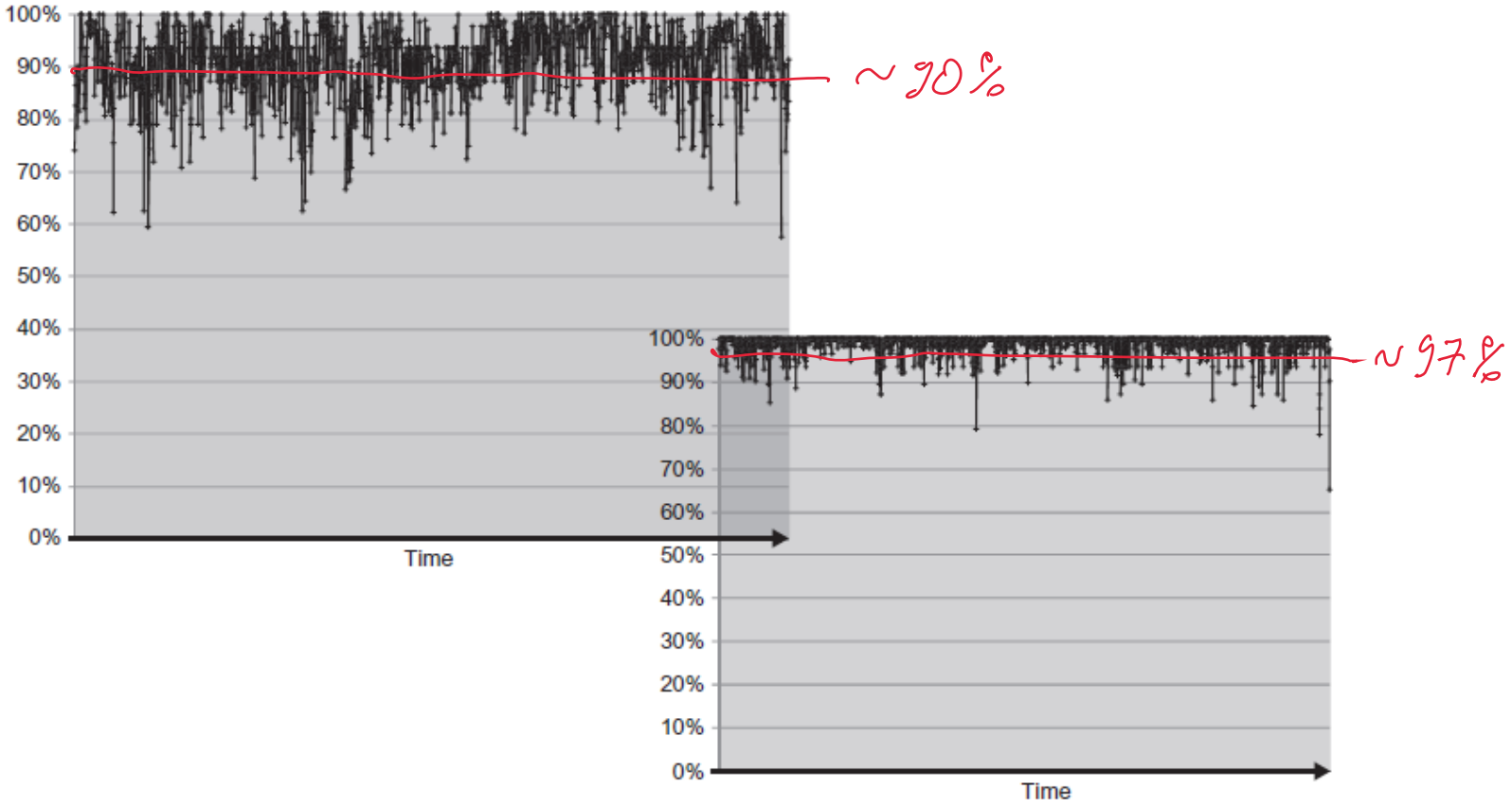


FIGURE 17.1

Packet Delivery Ratio for two IEEE 802.15.4 links.

Characterstics and Applications

- Common Characteristics
  - Low power and Lossy Networks (LLNs) consisting largely of constrained nodes.
  - Lossy and unstable links, typically supporting low data rates, relatively low packet delivery rates.
  - Traffic patterns are not simply point-to-point, but in many cases point-to-multipoint (P2MP) or multipoint-to-point (MP2P)
  - Potentially comprising up to thousands of nodes.
- Four main application scenarios
  - Urban LLNs (including smart grids) [RFC5548]
  - Industrial LLNs [RFC5673]
  - Home Automation LLNs [RFC5826]
  - Building Automation LLNs [RFC5867]

Requirements (1/2)

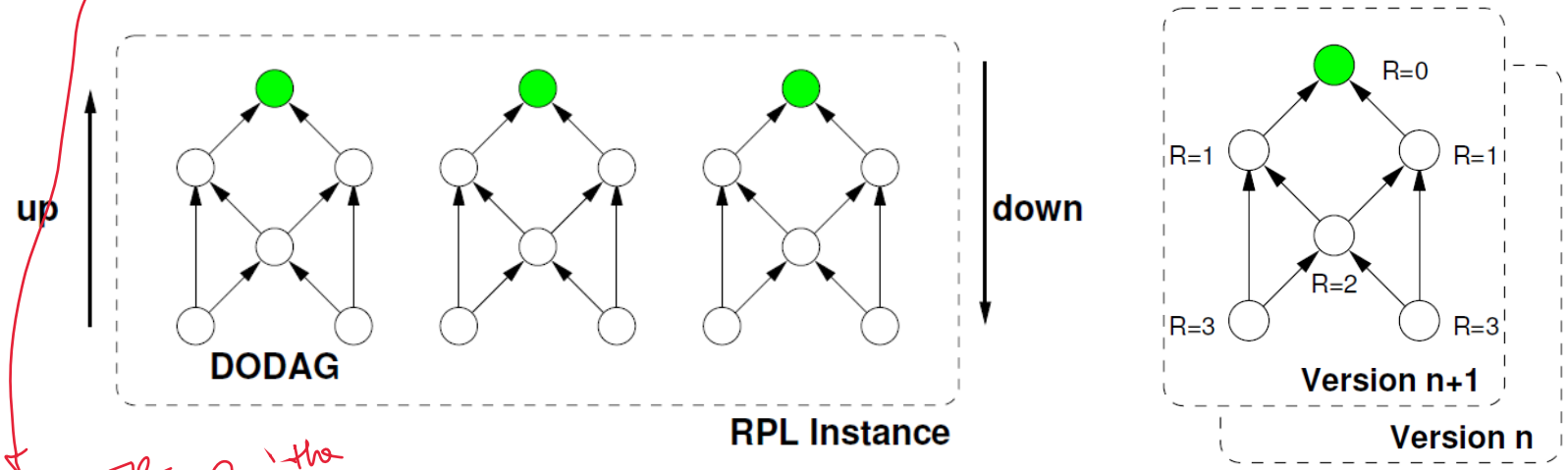
- Unicast/anycast/multicast
- Adaptive routing: new paths are dynamically and automatically recomputed as conditions change in the network (e.g., link/node failure, mobility, etc.)
- Constraint-based routing (energy, CPU, and memory as well as link attributes, such as link latency)
- Traffic characteristics (MP2P-data collection, P2MP-sw updates, P2P, computation of parallel paths)
- Scalability: from 250 to 10000 nodes
- Configuration and management: minimum or zero-configuration; isolation of a misbehaving node to limit/eliminate its impact on other nodes

Requirements (2/2)

- Node attribute: with sleeping nodes in the network (a frequent situation with battery-operated nodes), the routing protocol must discover the capability of a node to act as a proxy
- Performance: ‘‘tricky’’ requirement (e.g., a routing protocol must provide mobility with a convergence time below 0.5 s and it must converge within 0.5 s if no nodes have moved and within 2 s if the destination has moved)
- Security:
  - wide range (low to high);
  - the routing protocol must gracefully handle routing temporal security updates (e.g., dynamic keys) to sleeping devices on their ‘ *awake* ’ cycle to assure that sleeping devices can readily and efficiently access the network

RPL Approach (1/2)

- Design a **modular** routing protocol where the core component of the application would be specified by the RPL specification with optional features activated only where and when needed.
- For example, RPL specifies how to build a destination oriented directed acyclic graph (DODAG), but the characteristics of the DODAG are specified by an objective function.



RPL Approach (2/2)

- For the time being, think of a DODAG as a logical routing topology over a physical network that is built by the routing protocol to meet specific criteria.
- A node may join multiple DODAGs (if the application requires different objectives that must be realized through the use of multiple DODAGs) and mark the traffic according to the DODAG characteristics in support of Quality of Service (QoS) awareness and constrained-based routing.
- DODAG Properties
  - Many-to-one communication (MP2P): upwards
  - One-to-many communication (P2MP): downwards
  - Point-to-point communication (P2P): upwards-downwards

Routing Metrics in IP Nets

- Most of the IP routing protocols used in today's networks such as OSPF use static link metrics.
  - The network administrator is responsible for configuring the link metrics, which may refl ect the link bandwidth, delay, or combine several metrics.
  - Some Service Providers are combining up to three metrics (e.g., delays, bandwidth, cost) in the link metric.
  - Then the routing protocol computes the shortest path taking into account these static link metrics.
- Dynamic link metrics.
  - In ARPANET-2: studies to dynamically compute the link metric based on the averaged queue length to reflect the level of congestion.
  - These strategies were abandoned due to the difficulty in designing stable systems.
  - One of the main challenges with dynamic metrics is to carefully control the rate at which new metrics are advertised. Frequent link metric refreshers provide a high level of accuracy but may also lead to routing oscillation.

Routing in LLNs

- *Metric*: a scalar used to determine the best path according to some objective function (OF).
  - For example, if the link metric is representative of the link propagation delay, the path cost represents the total propagation delay to the destination and the OF may specify ''finding the shortest path based on the propagation delay.''
  - Some metrics may not be additive; for example, the OF may be ''finding the path where the minimum link quality is maximized. ''
- *Constraint*: used to include or eliminate links or nodes that do not meet specific criteria (this is usually referred to as *constraint-based routing*).
  - For example, the OF may not select any path that traverses a node that is battery-operated or a link that does not provide link layer encryption.
- The OF may combine link/node metrics and constraints such as " find the path with the minimum delay that does not traverse any non-encrypted link. "

Objective Function (OF)

- Examples (IP nets):
  - the OF of RIP is to select the path with minimal hop count;
  - OSPF or IS-IS would compute the paths that provide the minimum cost where the path cost is simply the sum of the static link cost along the path.
  - With MPLS TE the OF may be slightly more complex: " find the shortest path according to some metric such as the OSPF/ IS-IS
  - metric or the Traffic Engineering metric that satisfies some constraint such as the available reservable bandwidth or the type of recovery protection provided by the link. "
- LLNs: strong interest in using several OFs because deployments greatly vary with different objectives and a single network may support traffic with very different requirements in terms of path quality.
  - Consider the case of a mixed network with battery- and main-powered nodes, a variety of high and low bandwidth links, and two main applications (telemetry and critical alarms).
  - Extremely useful for each node: supporting both applications to be able to use two paths.
    - One " time-sensitive " path for alarms where the objective is to have a short delay and a highly reliable path with no constraint on the type of nodes along the path to the destination
    - Another " not time-sensitive path " for the telemetry traffic where it is beneficial to not traverse any battery-operated node to preserve energy and where the objective would be to minimize hops to avoid traffic congestion in the network.
  - RPL addresses these requirements by building two DODAGs with each one having its own OF. The OF is used in conjunction with the routing metric to compute the path.

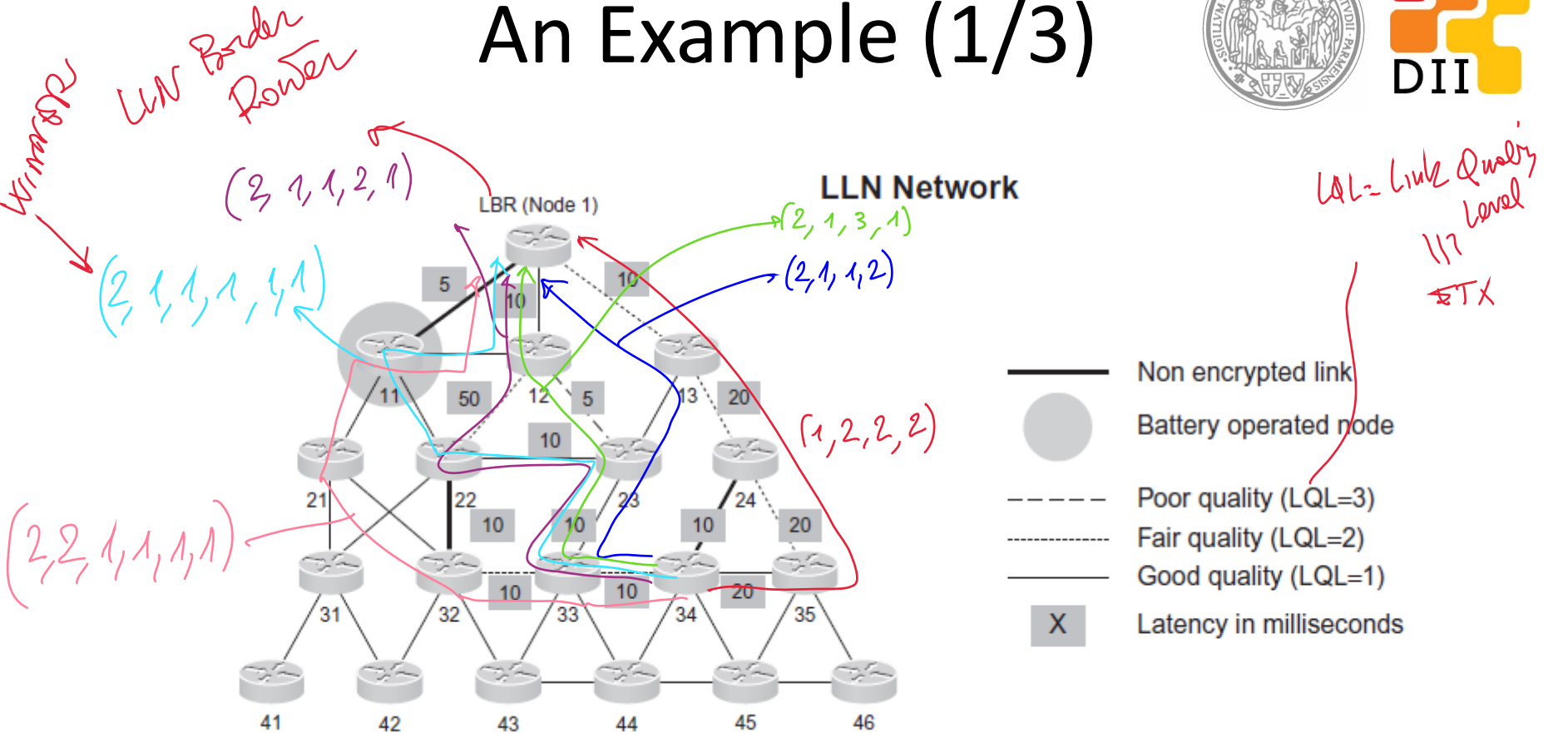
Routing Metrics (1/2)

- *Aggregated* (path cost is defi ned as the sum of the cost of all links along the path) versus *Recorded* (record each individual link metric)
- *Local* (not propagated along the DODAG) versus *Global*
- The Routing Metrics/Constraints Common Header (several flags → routing object: refers to a routing metric or a constraint; is local versus global; if the global metric is aggregated versus recorded; if a constraint is optional or mandatory, and if a metric is additive or reports a maximum/minimum.)
- Routing objects
  - The *Node State and Attributes (NSA) Object*: nodes may act as traffic aggregators (use many bits to describe the state); nodes have limited available resources → 1-bit fl ag set when a node sustainably experiences some level of congestion
  - *Node Energy Object*: several levels of granularity to characterize the node energy: (1) the node power mode; (2) the estimated remaining lifetime; and (3) potentially some detailed set of power-related metrics and attributes.
  - *Hop-count Object*
  - *Throughput Object*
  - *Latency Object*

Routing Metrics (2/2)

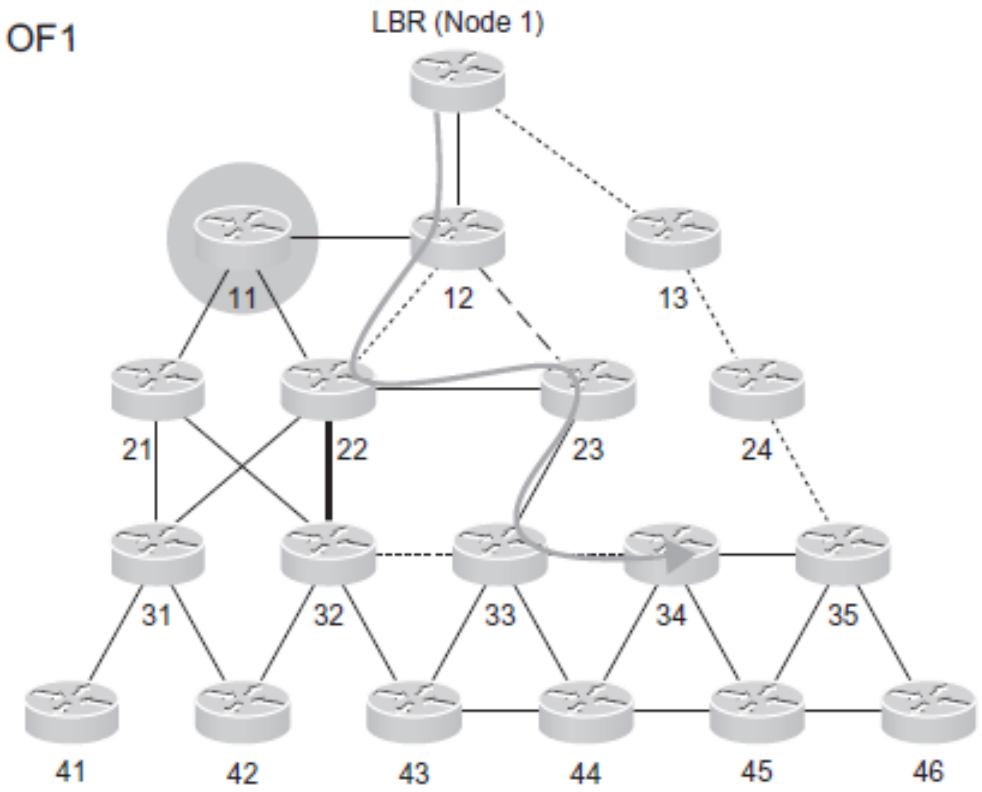
- Link Reliability Object: the most popular reliability metric thus far is the expected transmission (ETX) count metric
  - Characterizes the average number of packet transmissions required to successfully transmit a packet (see before).
  - Tightly coupled to the throughput along a path.
  - Several techniques have been proposed to compute ETX. → IETF does not specify the method for computing ETX values. The ETX is a link-specific quantity and the technique used to compute the ETX value should be independent of the link layer and not specified by the network layer that only carries it for routing protocol decisions.
- Link Colors Attribute: link color is encoded using a bit vector; the meaning of each color is left to the implementer.
  - RPL computes paths over a dynamically built DODAG. The DODAG root uses an OF required for each node along the path reporting path metrics to also report the set of colors of each link along the path.
  - For example, suppose that the color blue is used to indicate the support of the link layer encryption. Upon receiving the path metric, if link colors are recorded, a node may decide to elect as a parent the parent reporting paths with encrypted links (blue links) or with the maximum number of blue links in the absence of a path exclusively made of blue links.

An Example (1/3)



- Routing: from the low-power and lossy network border router (LBR) to node 34
- OF1: " Use the LQL as a global recorded metric and favor paths with the minimum number of low and fair quality links, use the link color as a link constraint to avoid non-encrypted links. "
- OF2: " Find the best path in terms of latency (link latency is used as a global aggregated metric), while avoiding poor quality links and battery-operated nodes. "

An Example (2/3)

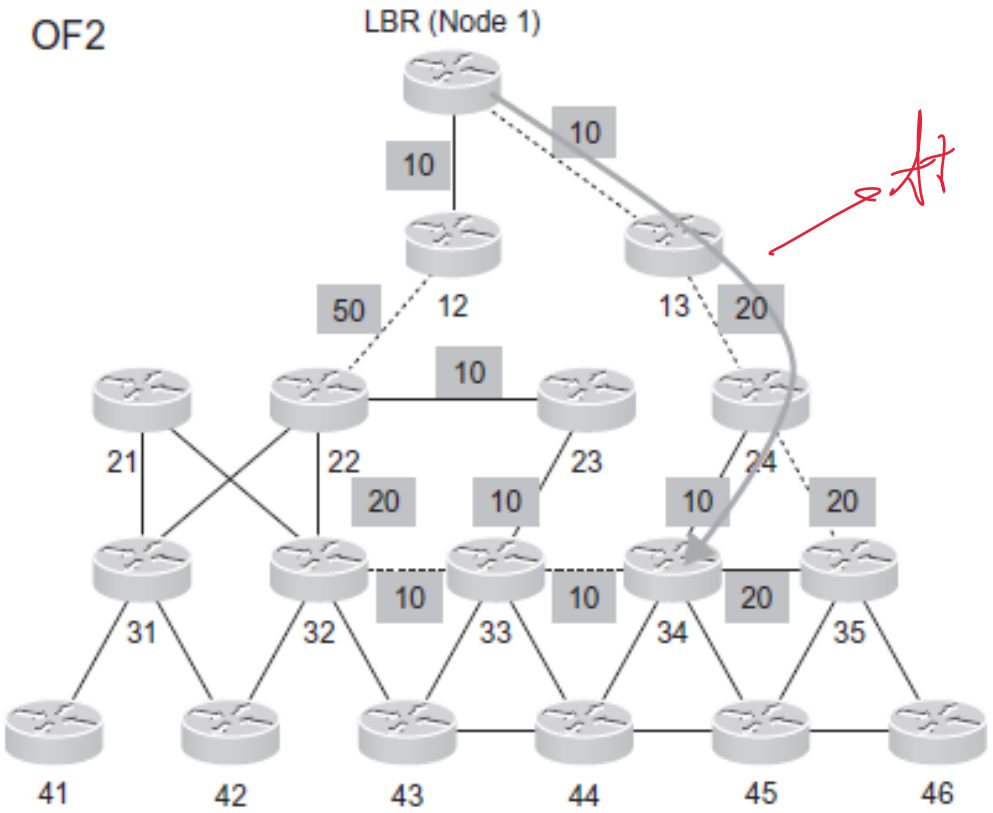


(1,2,2,2) (2,1,1,2,1)

- OF1: " Use the LQL as a global recorded metric and favor paths with the minimum number of low and fair quality links, use the link color as a link constraint to avoid non-encrypted links. "
- Two paths are available with an equivalent aggregated LQL metric: 34-35-24-13-1 and 34-33-23-22-12-1. Since OF specifies using a recorded metric → path 34-33-**23-22**-12-1 is chosen since it only has two links of " fair " quality.

no! Check the

An Example (3/3)



- OF2: " Find the best path in terms of latency (link latency is used as a global aggregated metric), while avoiding poor quality links and battery-operated nodes. "
- Several paths have been pruned because they traverse battery-operated nodes (node 11) and traverse poor quality links (link 12-23). The best path (lowest latency) is 34-24-13-1.

RPL Basics

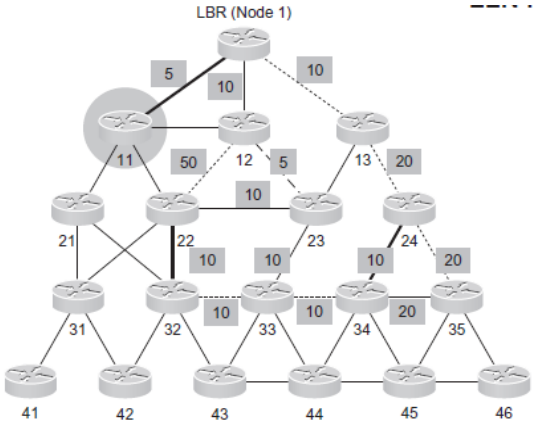
- RPL was designed to be **highly modular**: the core of the routing protocol would address the intersection of the application-specific routing requirements, and additional modules would be added as needed to address specific requirements.
- RPL was designed for LLNs where constrained devices are interconnected by (wireless and wired) lossy links → a node should carefully observe a link and start using it or determine whether to stop using it (thus triggering a global path recomputation in the network).
- Control traffic must be as tightly bounded as possible, ideally the control traffic should decrease as the routing topology stabilizes
- RPL is a distance vector protocol that builds a DODAG where paths are constructed from each node in the network to the DODAG root (typically a sink or an LBR)
- In contrast with tree topologies, DODAGs offer redundant paths, which is a MUST requirement for LLNs. Thus if the topology permits, RPL may provision more than one path between a node and the DODAG root and even other nodes in the network.

RPL Operations (1/5)

- First, one or more nodes are configured as DODAG roots by the network administrator.
- A node discovery mechanism based on newly defined ICMPv6 messages is used by RPL to build the DODAG.
- RPL defines two new ICMPv6 messages:
  - DODAG information object (DIO) messages; → UP & DOWN
  - destination advertisement object (DAO) messages. → ~~DOWN~~ UP
- DIO messages (simply referred to as DIO) are sent by nodes to advertise information about the DODAG, such as: the DODAGID; the OF; the DODAG rank (depth); the DODAGSequenceNumber; along with other DODAG parameters such as a set of path metrics and constraints discussed before.
- When a node discovers multiple DODAG neighbors (that could become parents or sibling), it makes use of various rules to decide whether (and where) to join the DODAG. This allows the construction of the DODAG as nodes join.
- Once a node has joined a DODAG, it has a route toward the DODAG root (which may be a default route) in support of the MP2P traffic from the leaves to the DODAG root (in the up direction).

RPL Operations (2/5)

- RPL uses “ up ” and “ down ” directions terminology.
  - up direction from a leaf toward the DODAG root;
  - down direction refers to the opposite direction.
- The usual terminology of parents/children is used.
- RPL also introduces the “sibling”: two nodes are siblings if they have the same rank in the DODAG (note that they may or may not have a common parent).
- The parent of a node in the DODAG is the immediate successor within the DODAG in the up direction, whereas a DODAG sibling refers to a node at the same rank.
- In the side figure, 13 is a parent of 24, 22, and 23 are siblings, and 43 and 44 are children of 33.
- A DODAG is said to be:
  - *grounded* if it is connected to what RPL calls a “ goal, ” which can be a node connected to an external (non-LLN) private IP network or the public Internet;
  - a non-grounded DODAG is called a *floating* DODAG.



- RPL uses iterations controlled by the DODAG root to maintain the DODAG; the DODAGSequenceNumber is a counter incremented by the DODAG root to specify the iteration number of the DODAG .

RPL Operations (3/5)

- DAO messages: provide routing information in the down direction (for the traffic from the route to the leaf) and for the P2P direction since the DODAG provides defaults routes to the DODAG root from each node in the network.
- DAO messages (simply referred to as DAO) are used to advertise prefi x reachability toward the leaves. DAOs carry prefix information along with a lifetime (to determine the freshness of the destination advertisement-DAs) and depth or path cost information to determine how far the destination is.
- Note that the path in the downward direction is dictated by the DODAG built by RPL in the other direction.
- In some cases, DAOs may also record the set of visited nodes → particularly useful when the intermediate nodes cannot store any routing states.
- If a parent receives destination advertisements that can be aggregated from multiple children, local policy may be used to perform prefix aggregation in an attempt to reduce routing table and the size of DAO messages.
- Note that redundant DAO messages are aggregated along the DODAG. An OF may be specifically designed to maximize prefix aggregation.

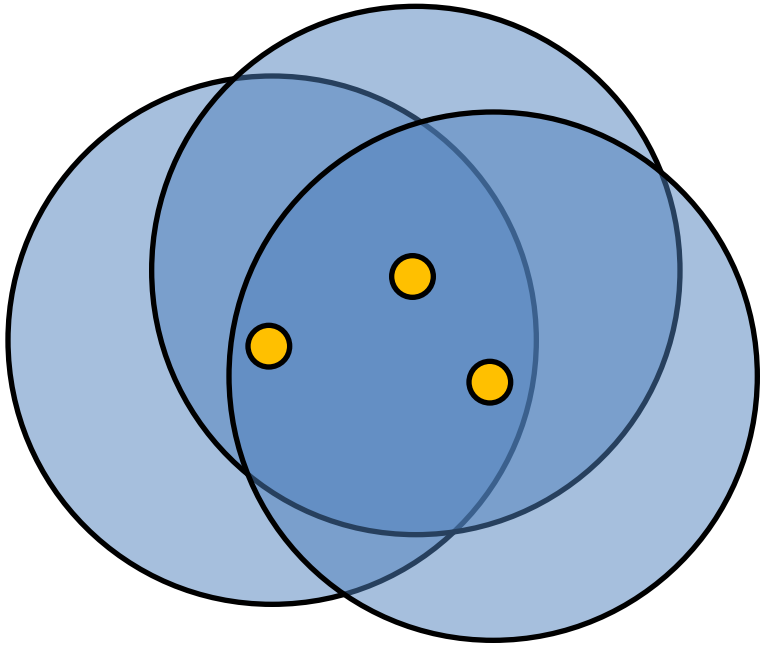
RPL Operations (4/5)

- RPL supports P2P traffic.
  - When node A sends a packet destined to node B, if B is not in direct reach, it forwards the packet to its DODAG parent.
  - From there, if the destination is reachable from one of its children, the packet is forwarded in the down direction.
  - In other words, the packet travels up to a common ancestor at which point it is forwarded in the down direction toward the destination.
- An interesting optimization periodically emits *link-local* multicast IPv6 DAOs -→ if the destination is in direct range (one hop away), a node can send the packet directly to the destination without following the DODAG.
- Sending DIO and DAO messages is governed by the use of trickle timers. The trickle timers use dynamic timers that govern the sending of RPL control messages in an attempt to reduce redundant messages.
  - When the DODAG is unstable (e.g., the DODAG is being rebuilt) RPL control messages are sent more frequently (the DODAG becomes inconsistent).
  - On the other hand, as the DODAG stabilizes messages are sent less often to reduce the control plane overhead, which is very important in LLNs.

skip

Trickle: idea

- Concerns
  - Broadcast is expensive
  - Wireless channel is a shared, spatial resource
- Idea
  - Dynamic adjustment of transmission period
  - Suppress transmissions that may be redundant
- Parameters:
  - T_min: Minimum advertisement period
  - T_max: Maximum advertisement period
  - k: Suppression threshold
- Period adjustment:
  - On receiving *inconsistent* route information, reset to T_min
  - Otherwise, double up to T_max
- Suppression:
  - Increment count when receiving *similar* advertisement
  - At end of period, transmit if c < k, set c = 0
- Proposal:
  - Carry T_min, T_max, and k in DIO

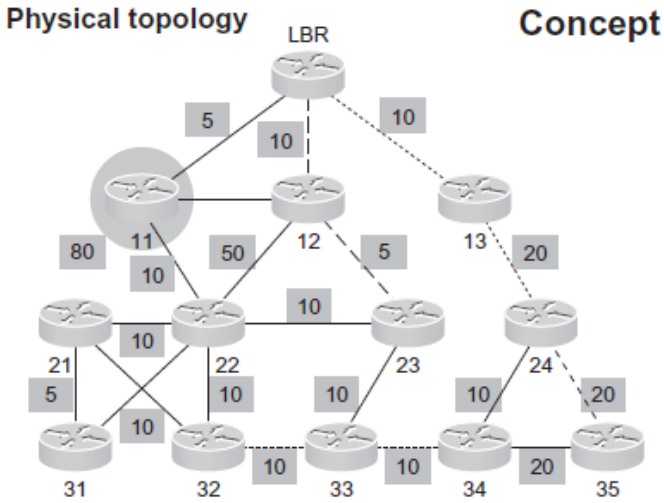


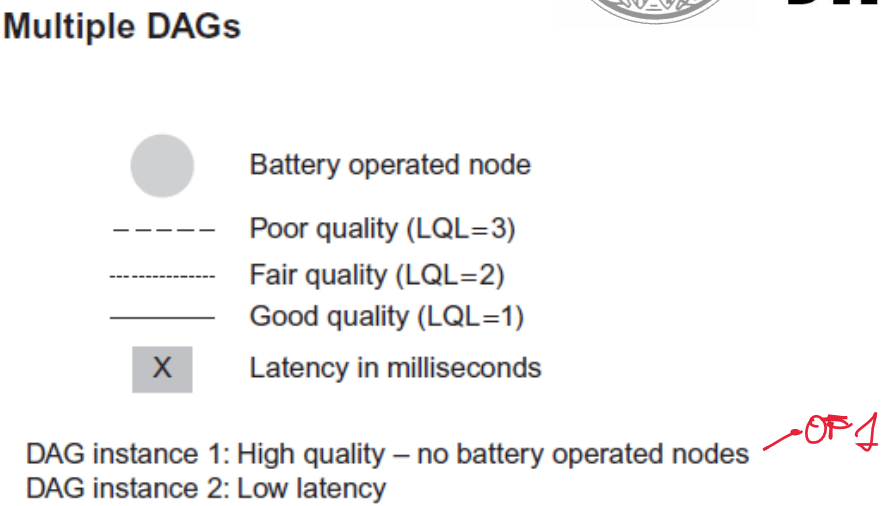
LEC. 15

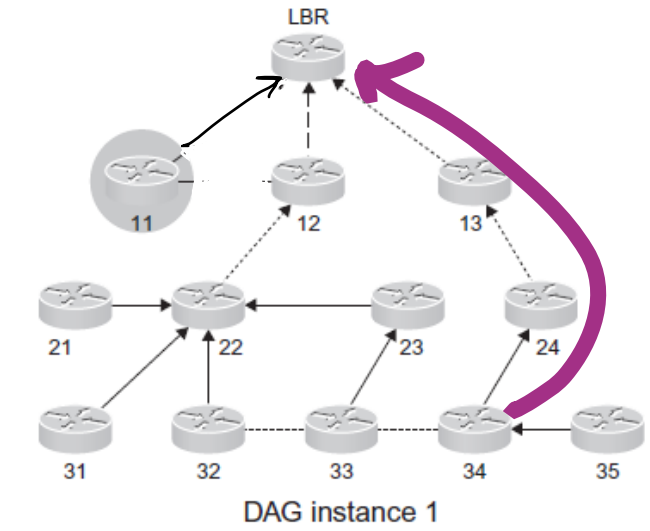
RPL Operations (5/5)

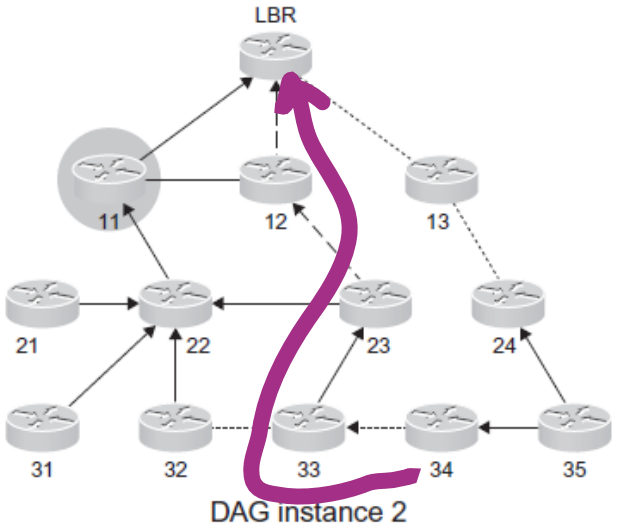
- Once the DODAG is built and routing tables are populated, routing is fully operational.
- As links and nodes fail, paths are repaired using local and global repair mechanisms.
  - Local repairs quickly fi nd a backup path without an attempt to globally reoptimize the DODAG entirely
  - Global repairs rely on a reoptimization process driven by the DODAG root.
- RPL also supports the concept of DODAG instances identifi ed by an Instance ID called the RPLInstanceID. It might be useful to form different topologies according to various sets of constraints and OFs. An RPL node may join multiple DODAG instances; for example:
  - one DODAG optimizes for high reliability
  - another DODAG optimizes for low latency.
- Data packets are then forwarded along the appropriate DODAG according to the application requirements.

Multiple DODAG Instances





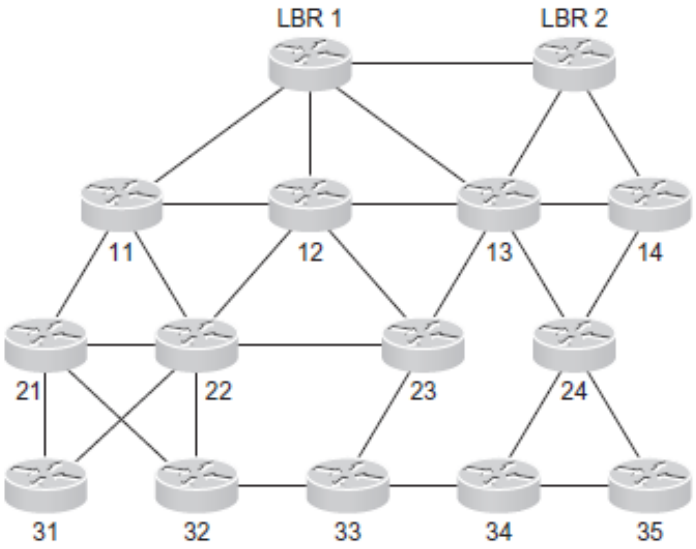




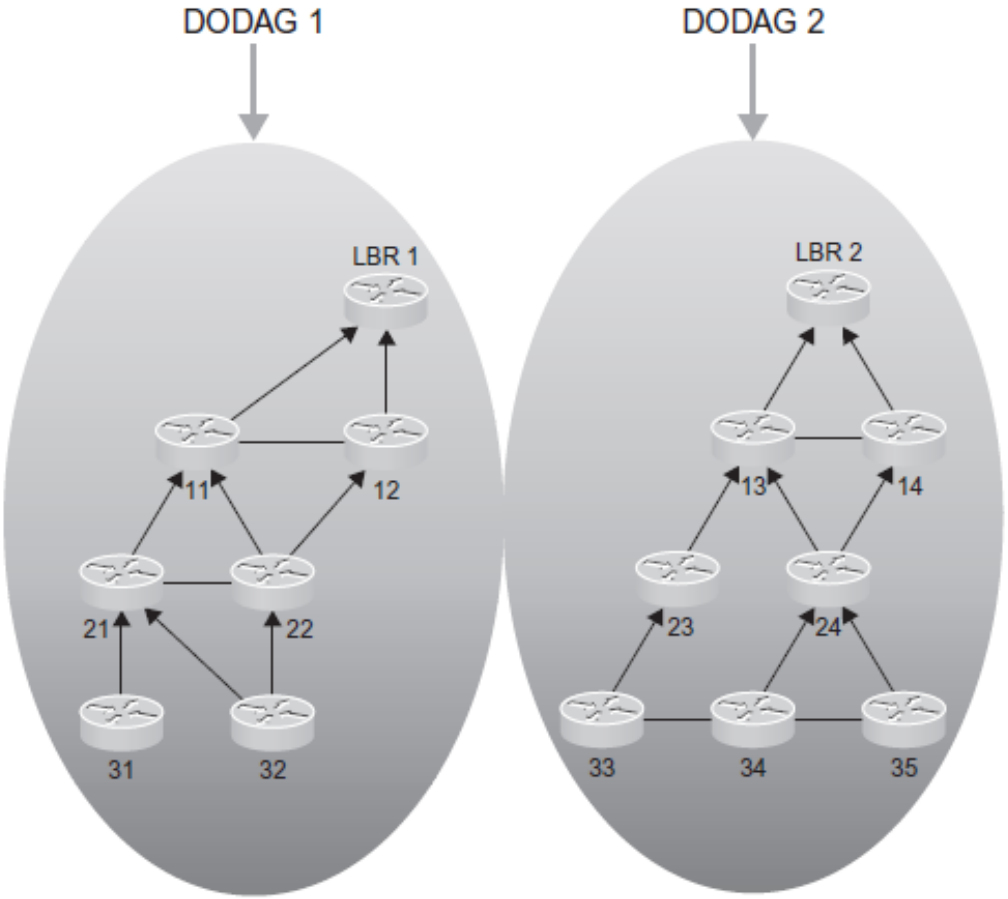
Multiple Disjoint DODAGs

not necessarily

Physical topology



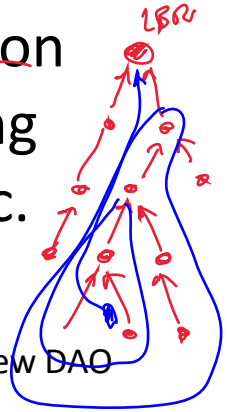
Multiple DODAGs



DIO Messages

- DIO messages are sent by RPL nodes to advertise a DODAG and its characteristics → DIOs are used for DODAG discovery, formation, and maintenance.
- DIOs carry a set of mandatory information augmented with options.
- The following flags and fields are currently defined:
  - Grounded (G): Indicates whether the DODAG is grounded, in other words, the DODAG root is a goal for the OF (e.g., the DODAG root is connected to a non-LLN IP network such as a private network or the public Internet).
  - Destination Advertisement Trigger (T): The T bit is used to trigger a complete refresh of the routes in the down direction (downward routes).
  - Destination Advertisement Stored (S): The S bit is used to indicate that a non-root ancestor is storing routing table entries learned from DAO messages.
  - Destination advertisement supported (A flag): The A flag is set when the DODAG root supports the collection of prefix advertisements and enables the advertisement of prefixes in the DODAG.
  - DODAGPreference (Prf): The Prf is a 3-bit field set by the DODAG root to report its preference. It can be used to engineer the network and make some DODAGs more attractive to join.
- The DODAGSequenceNumber is the sequence number of the DODAG that characterizes the DODAG iteration and is exclusively controlled by the DODAG root.
- The RPLInstanceID is used to identify the DODAG instance and is provisioned at the DODAG root.
- The Destination Advertisement Trigger Sequence Number (DTSN) is an 8-bit integer set by the node sending the DIO → used to maintain the downward routes.
- The DODAGID is a 128-bit integer set by the DODAG root and that uniquely identifies the DODAG.
- The DODAG Rank is the rank of the node sending the DIO message
  - The rank determines the relative position of a node in the DODAG and is used primarily for *loop avoidance*.
  - The rank is computed according to the OF and is potentially subject to local node policy.
  - The rank (although potentially derived from routing metrics) is not a metric.
- When two nodes have the same rank, the nodes are said to be siblings (they are located at a similar level of optimality in the DODAG).

DAO Messages



- DAO messages are used to propagate destination information along the DODAG in the up direction to populate the routing tables of ancestor nodes in support of P2MP and P2P traffic.
- The DAO message includes the following information:
  - DAO sequence: A counter incremented by the node owning the advertised prefix each time a new DAO message is sent.
  - RPLInstanceID: The topology instance ID as learned from the DIO.
  - DAO rank: Corresponds to the rank of the node that owns the prefix.
  - DAO lifetime: It is expressed in seconds and corresponds to the prefix lifetime.
  - Route tag: 8-bit integer that can be used to tag " critical " routes. The priority could be used to indicate whether the route should be stored by the nodes with a lower rank (closer to the DODAG root), which could be useful if nodes have limited memory capacities and must be selective about which destination information to cache. Note that the size of that fi eld has been changed several times and is subject to further changes.
  - Destination Prefix: The Prefix Length field contains the number of valid leading bits in the prefix.
  - Reverse Route Stack (RRS): contains a number of RRCount (another field of the DAO message) IPv6 addresses used in LLNs with nodes that cannot store routing tables.

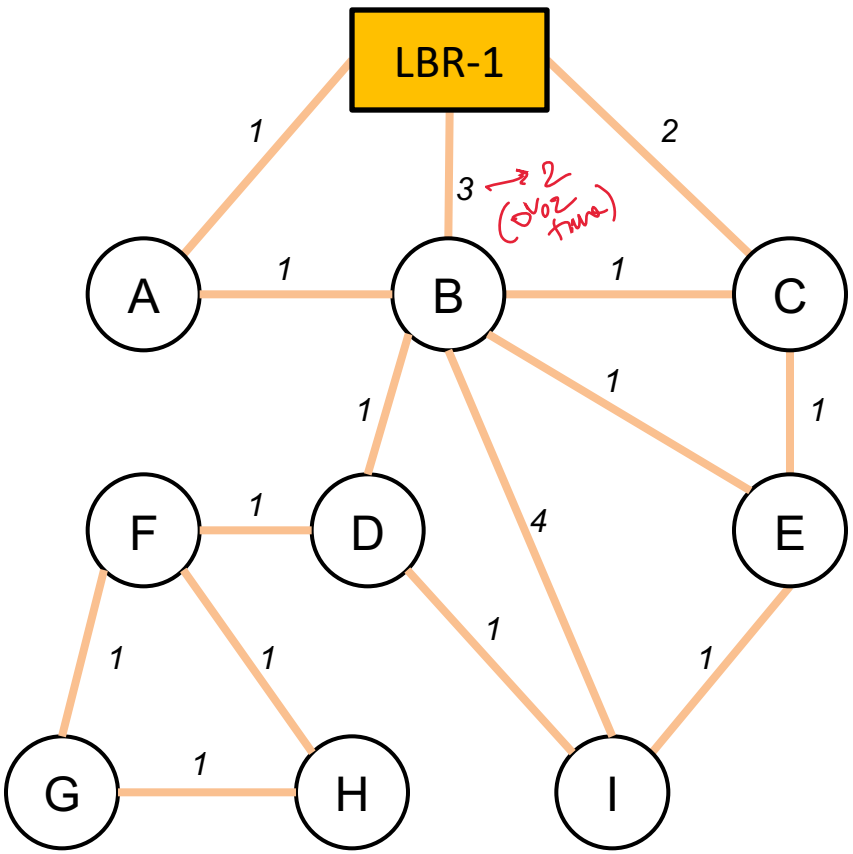
The "perfect" DAG?

- Since there no perfect wires with constant properties there's hardly a perfect DAG
- Conditions change, batteries deplete
- Need to arbiter between conflicting metrics and goals
- Build a structure that is satisfactory for all
- Maintain with probably a slow evolution
- The characteristics of the network, including the metrics, are unpredictable. What we really want is that every node gets an acceptable result most of the time.
- There are many ways in which acceptable can be arranged (for some time).
- We do not look for the illusory "perfect DAG" so we could reach any of a large number of acceptable solutions.

An Open Issue: Carrying IPv6 Addresses

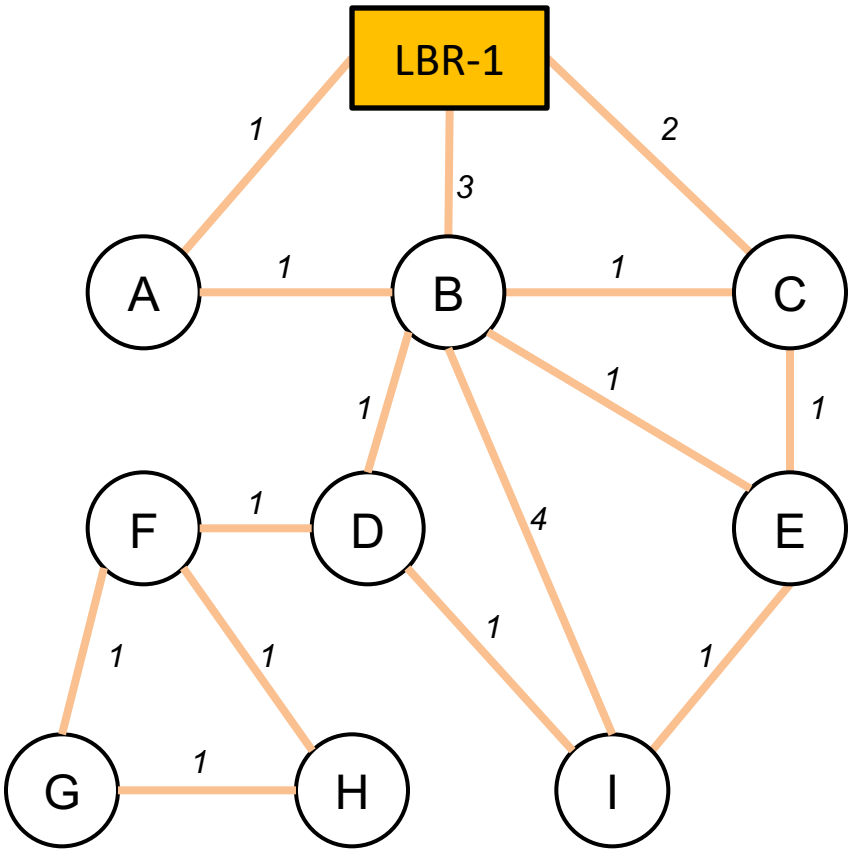
- High communication overhead
  - Carried in DIO, DAO, source routes
- High state overhead
  - Maintain destination routing state
- Options for compressing
  - Use a new namespace (e.g. labels)
  - Compress IPv6 addresses

DAG Construction



- LLN links are depicted
- Links are annotated w/ ETX
- It is expected that ETX variations will be averaged/filtered to be stable enough for route computation
  - Nodes are also to observe the metric and gain confidence before use

DAG Construction



- Example: Objective Code Point (default OF)
- Main characteristics
  - Metric: ETX
  - Objective: Minimize ETX
  - Depth computation:
    Depth ~ ETX
    - Note that a practical computation may be more coarse

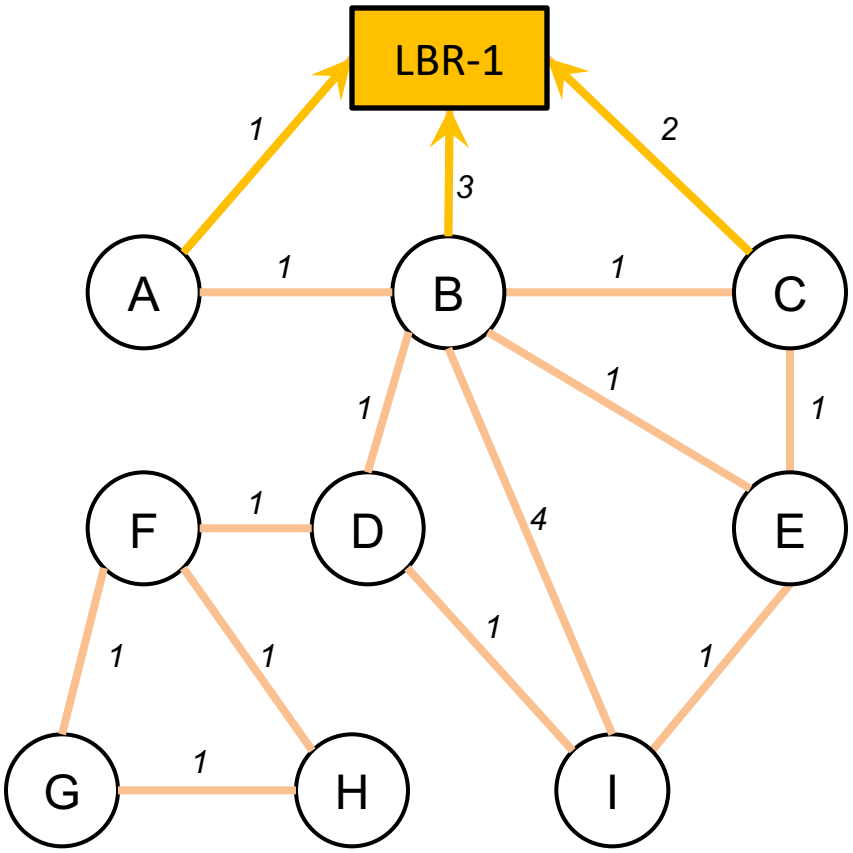
Global aggregated metric

$$ETX_{path} = \sum_{i=1}^{n} ETX_{link_i}$$

Links over the path.

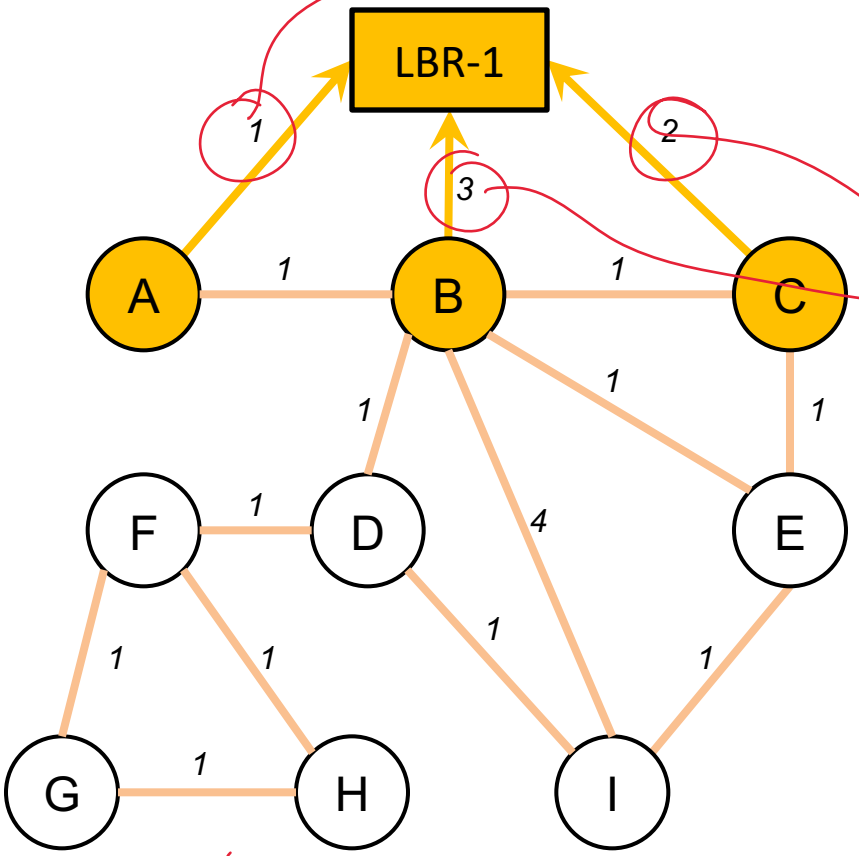
$\{link_i\}_{i=1}^{n}$

DAG Construction



- LBR-1 multicasts RA-DIO
- Nodes A, B, C receive and process RA-DIO
- Nodes A, B, C consider link metrics to LBR-1 and the optimization objective
- The optimization objective can be satisfied by joining the DAG rooted at LBR-1
- Nodes A, B, C add LBR-1 as a DAG parent and join the DAG

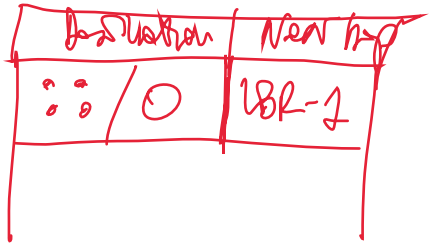
DAG Construction



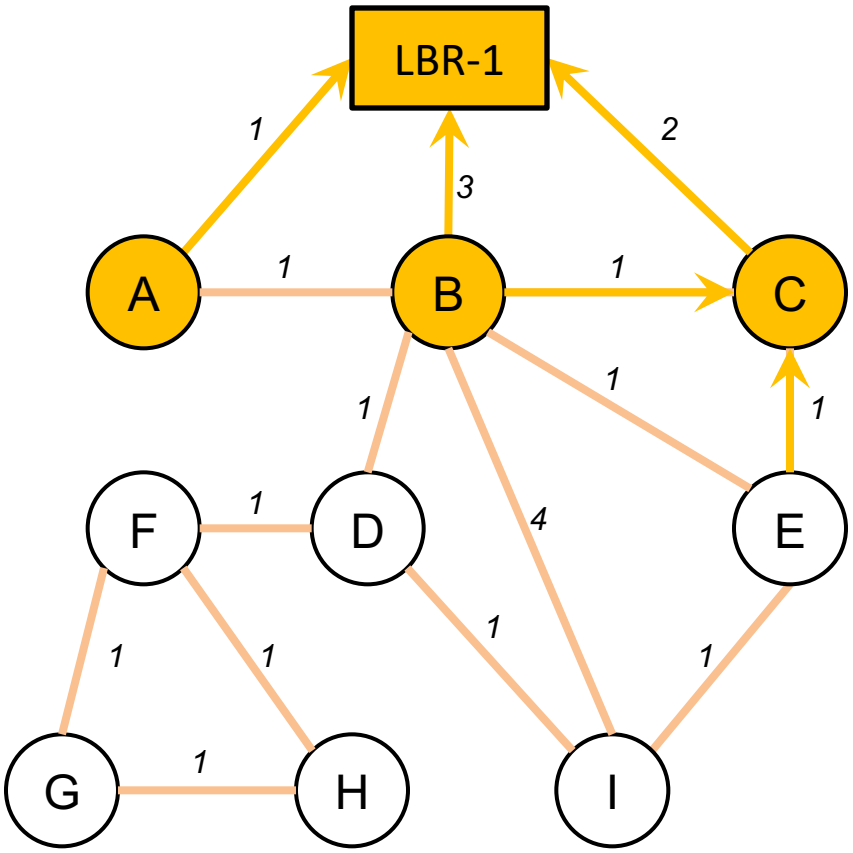
- Node A is at Depth 1 in the DAG, as calculated by the routine indicated by the example OCP (Depth ~ ETX)
- Node B is at Depth 3, Node C is at Depth 2
- Nodes A, B, C have installed default routes (::/0) with LBR-1 as successor

A, B, C set RA timers to (exchanged) trigger a DIO.

A

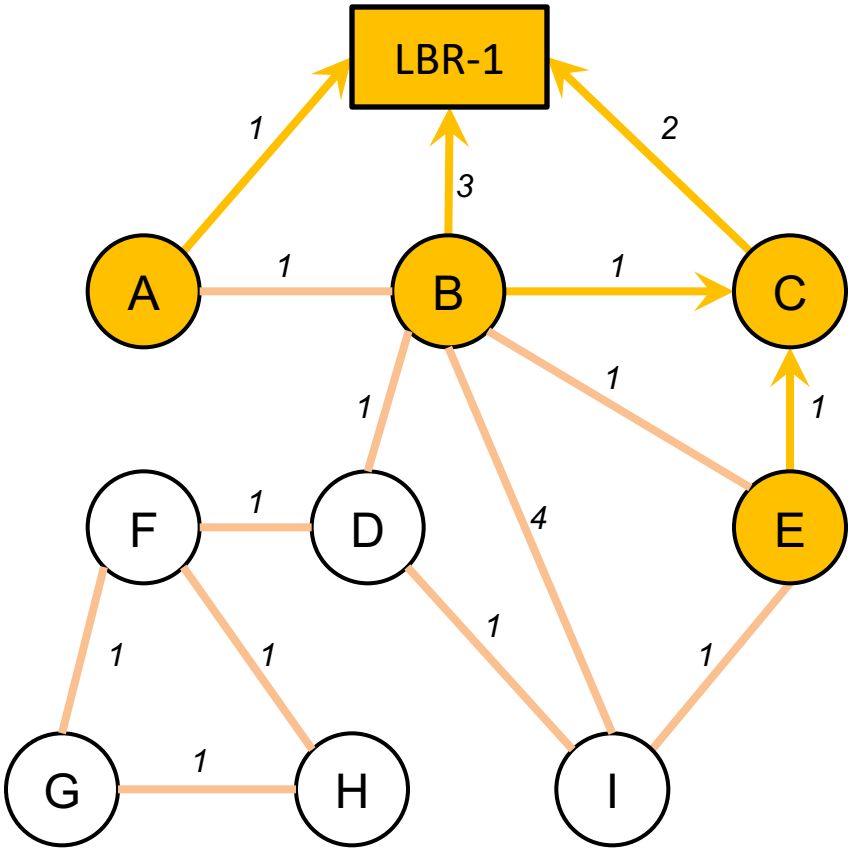


DAG Construction



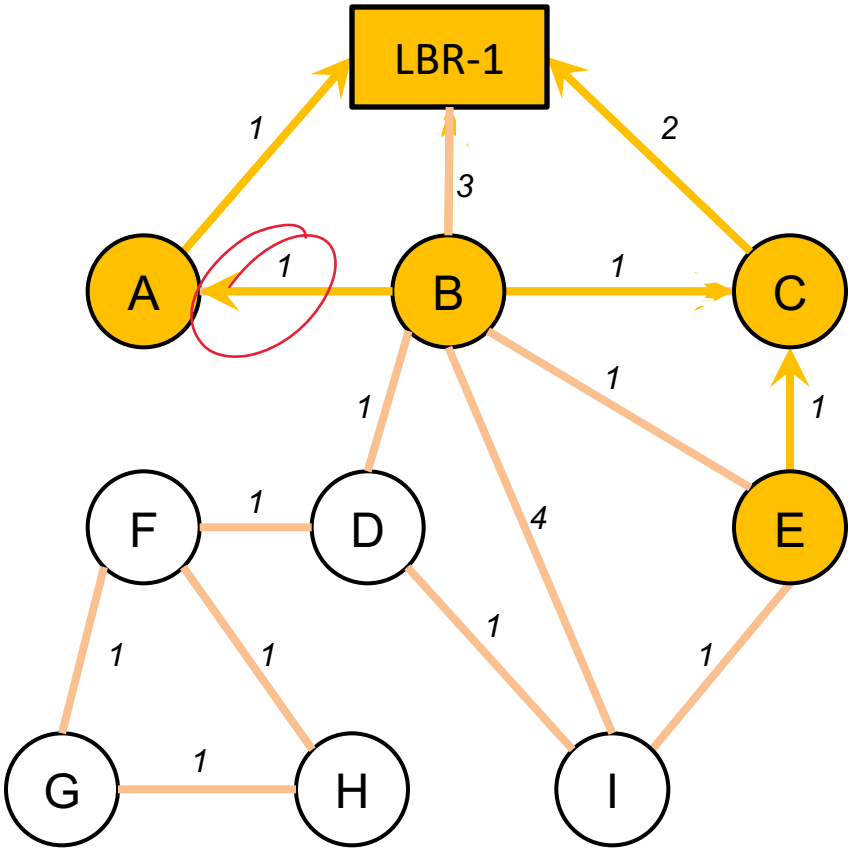
- The RA (Resource Advertisement) timer on Node C expires first (before A and B)
- Node C multicasts RA-DIO
- LBR-1 ignores RA-DIO from deeper node (DIO → there will be the rank=2 of C)
- Node B can add Node C as *alternate* DAG Parent, remaining at Depth 3
- Node E joins the DAG at Depth 3 by adding Node C as DAG Parent

DAG Construction



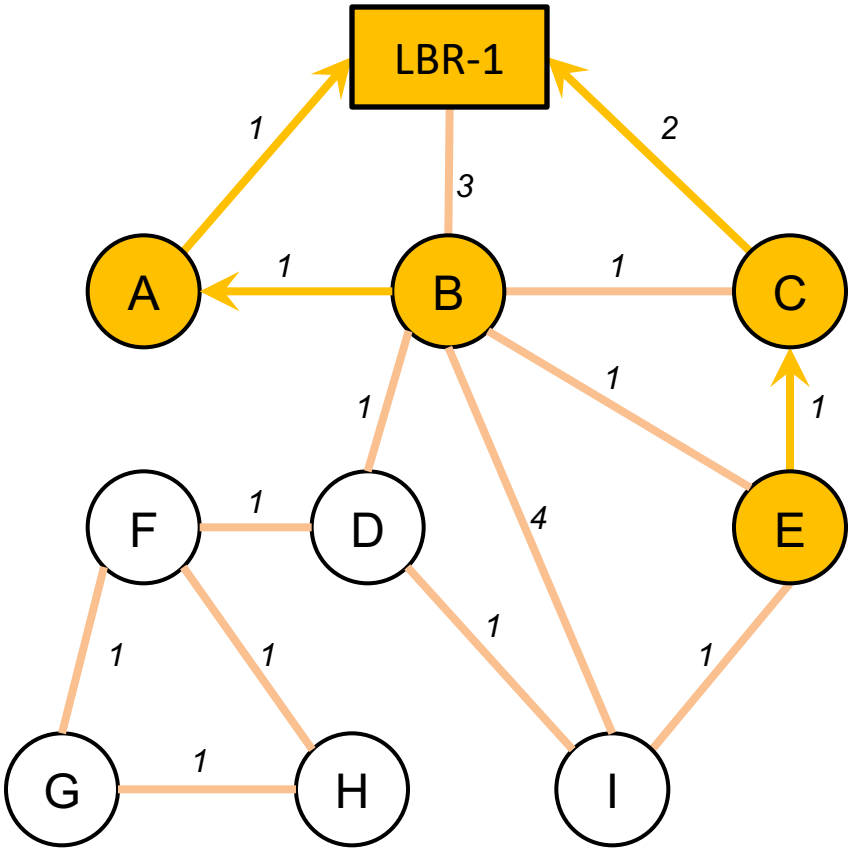
- Node A is at Depth 1, and can reach ::/0 via LBR-1 with ETX 1
- Node B is at Depth 3, with DAG Parents LBR-1, and can reach ::/0 via LBR-1 or C with ETX 3
- Node C is at Depth 2, ::/0 via LBR-1 with ETX 2
- Node E is at Depth 3, ::/0 via C with ETX 3

DAG Construction



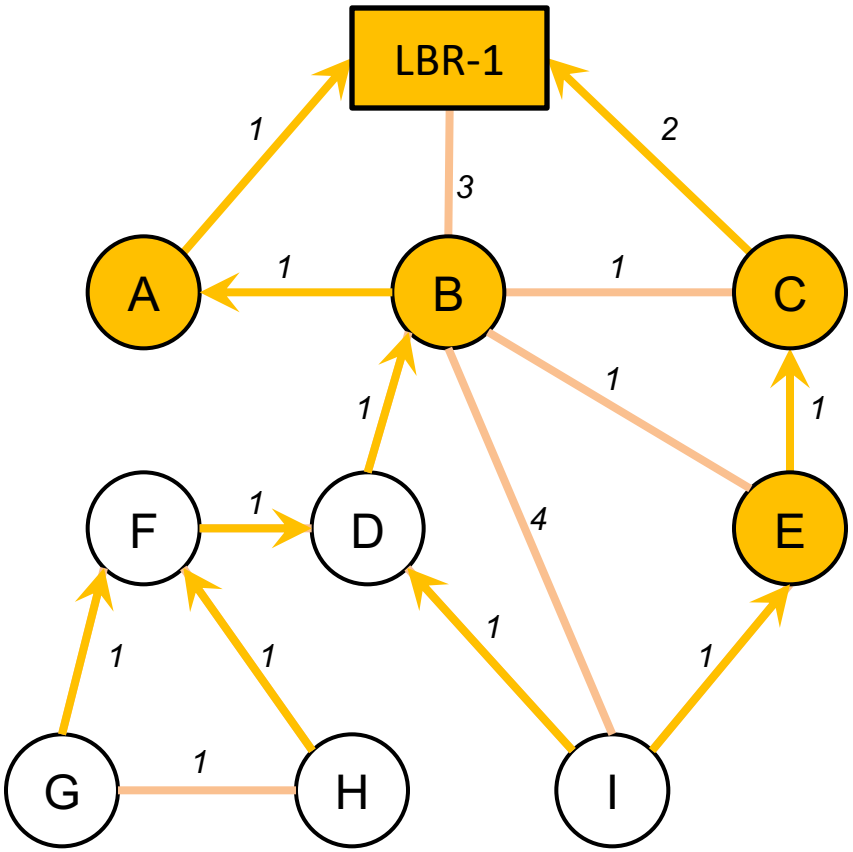
- The RA timer on Node A expires
- Node A multicasts RA-DIO
- LBR-1 ignores RA-DIO from deeper node
- Node B adds Node A
- Node B can improve to a more optimum position in the DAG
- Node B *removes* LBR-1, Node C as DAG Parents

DAG Construction

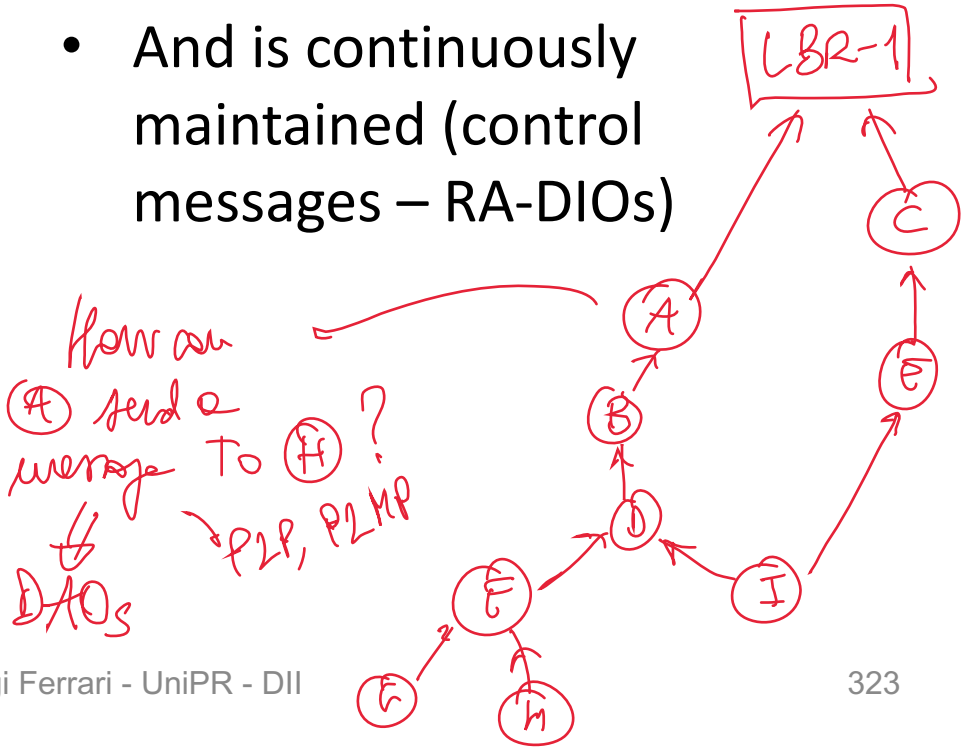


- Node A is at Depth 1, ::/0 via LBR-1 with ETX 1
- Node B is at Depth 2, ::/0 via A with ETX 2
- Node C is at Depth 2, ::/0 via LBR-1 with ETX 2
- Node E is at Depth 3, ::/0 via C with ETX 3

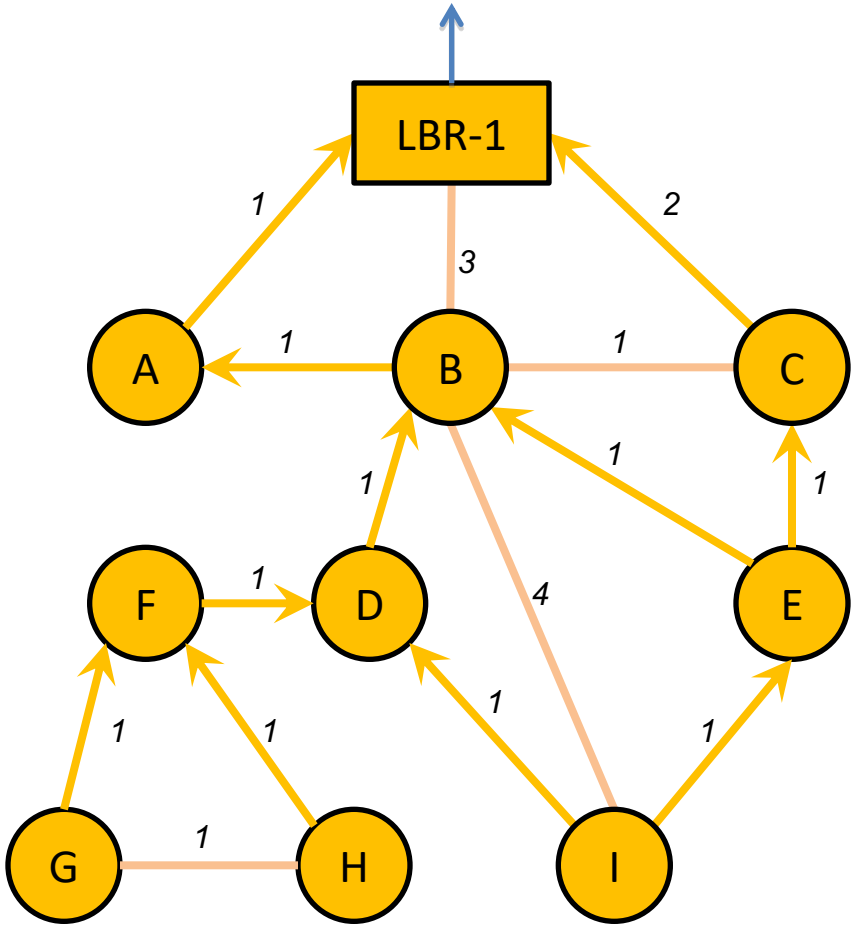
DAG Construction



- DAG Construction continues…
- And is continuously maintained (control messages – RA-DIOs)

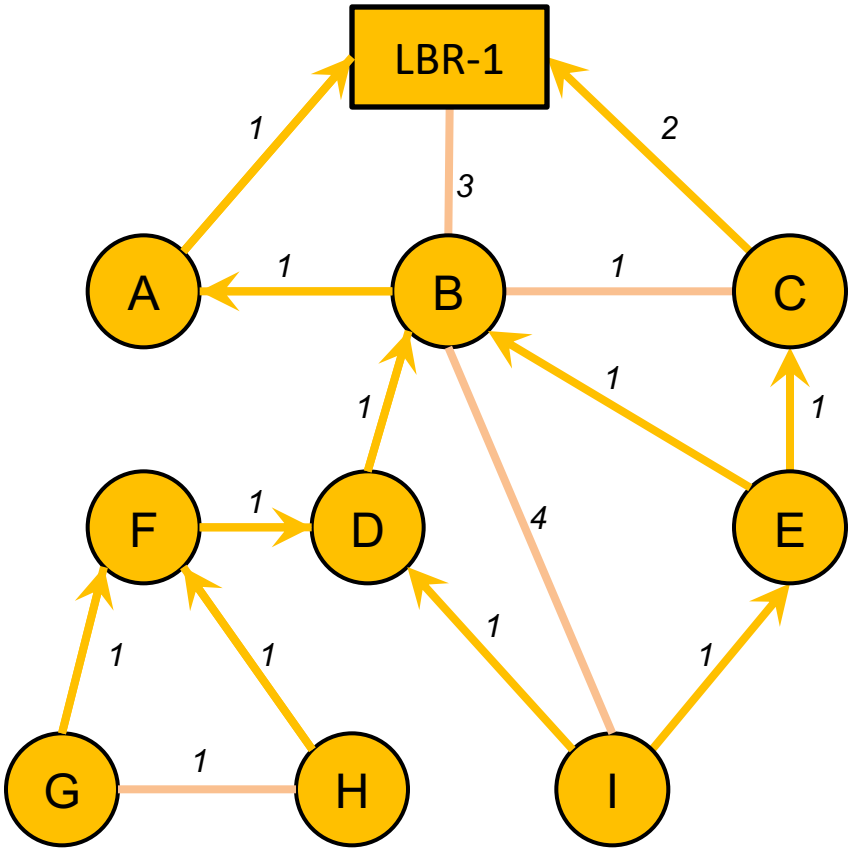


MP2P Traffic



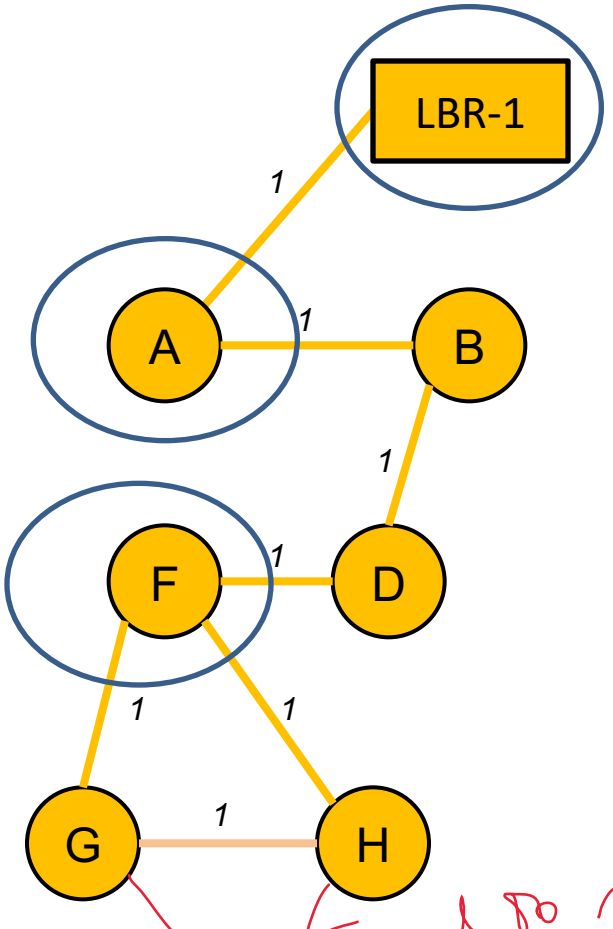
- MP2P traffic flows upwards along DAG, toward DAG Root
- DAG Root may also extend connectivity to other prefixes beyond the DAG root, as specified in the DIO
- Nodes may join multiple DAGs as necessary to satisfy application constraints

Destination Advertisements



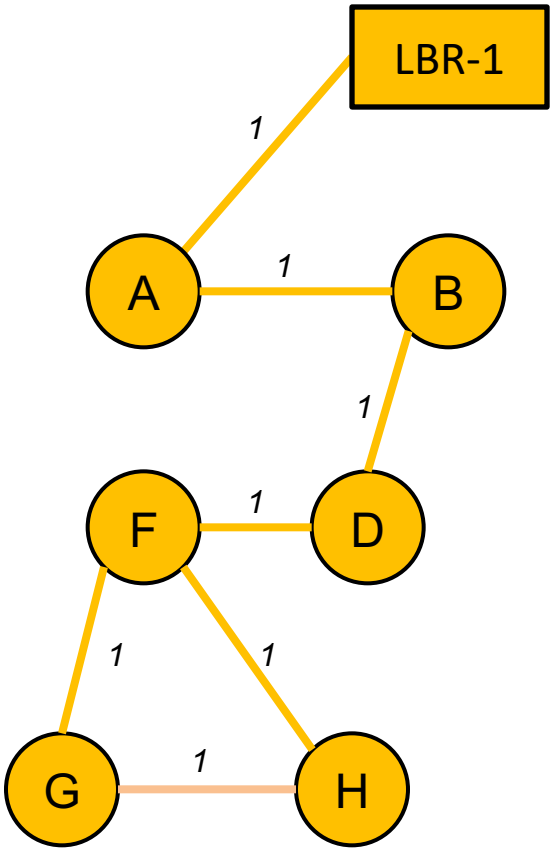
- Destination Advertisements (Das) build up routing state in support of P2MP traffic flows downward, from the sink to other nodes
- DAs use the same DAG
- For simplicity, we will focus on a subset of DAs in the example

Destination Advertisements



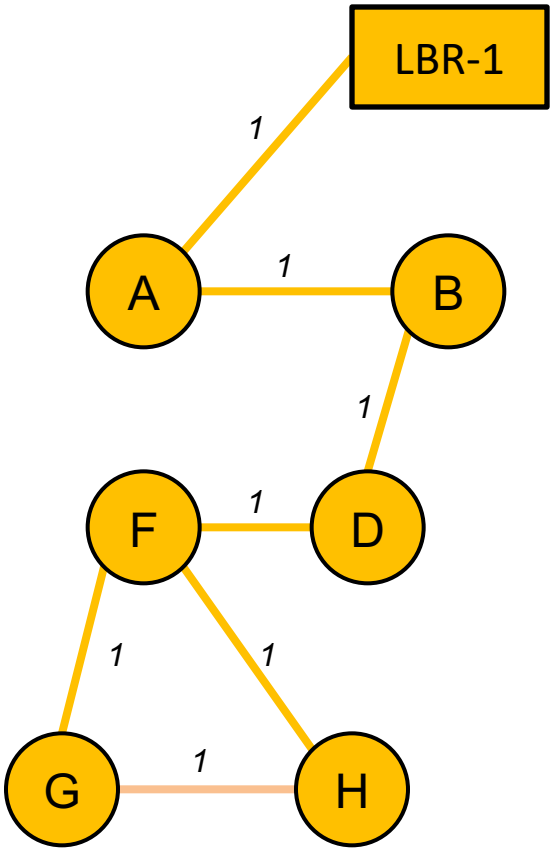
- Some nodes may be able to store routing state for downward flows (LBR-1, A, F)
- Some nodes may not (B, D)
- Some nodes may have a limited ability; DAs may indicate a priority for storage

Destination Advertisements



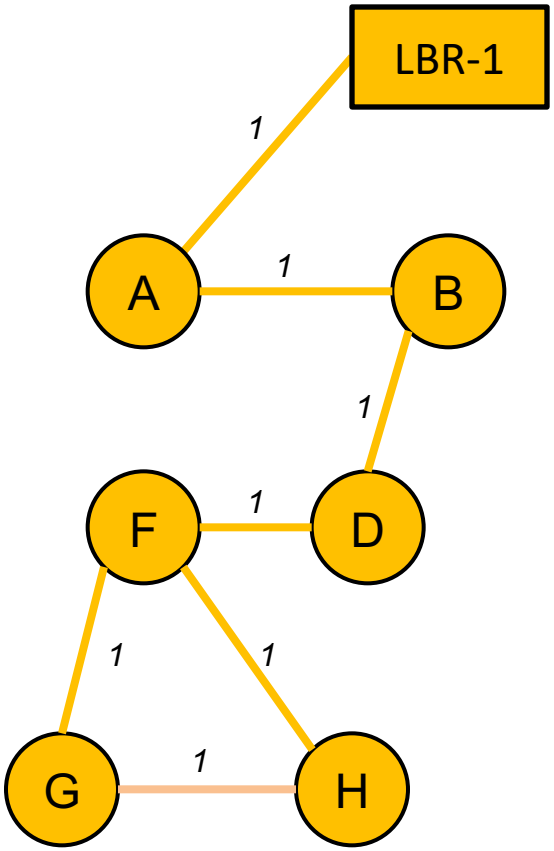
- DAs may be triggered by DAG root or node who detects a change
- DA timers configured such that DAs start at greater depth, and may aggregate as they move up

Destination Advertisements



- LBR-1 triggers Destination Advertisement mechanism in DIO
- G emits NA to F with DAO indicating reachability to destination prefix G::
- F stores G:: via G
- H emits NA to F for destination prefix H::
- F stores H:: via H

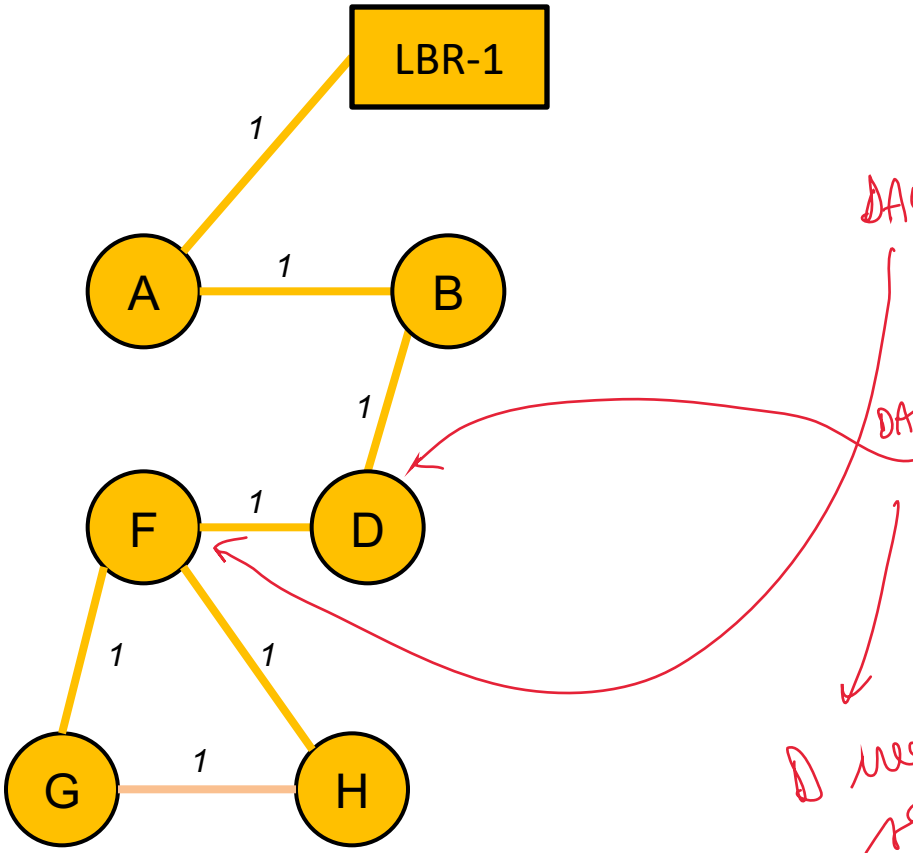
Destination Advertisements



- Suppose in this example F has a prefix F*:: capable of aggregating {F::, G::, H::}
  - The method to provision such a prefix is beyond the scope of RPL
- F emits NA to D with DAO indicating reachability to destination prefix F*::
- D cannot store…

(continued)

Destination Advertisements

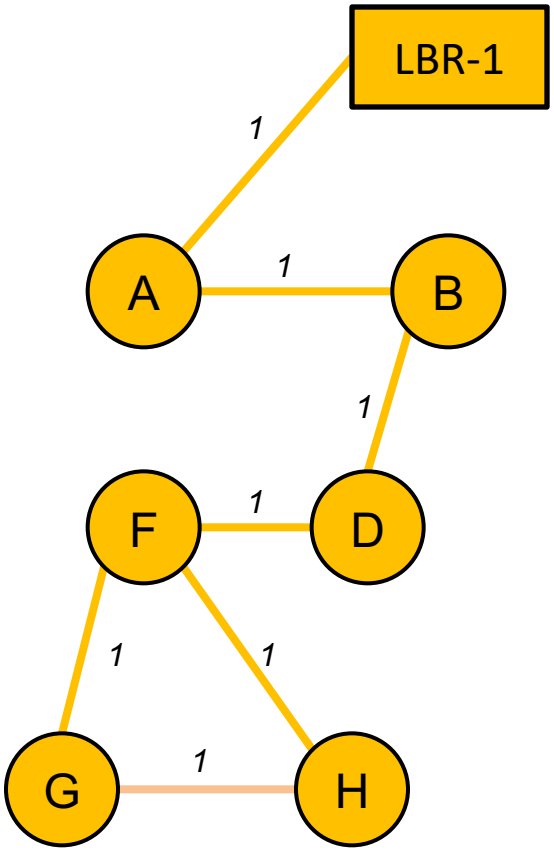


- D adds F to the Reverse Route Stack in the DAO, and passes DAO on to B for F*:: [F]
- D also emits a DAO indicating prefix D:: to B
- B cannot store routing state…

D needs to send 2 DAOs, because the prefix D:: and F*:: cannot be combined.

(continued)

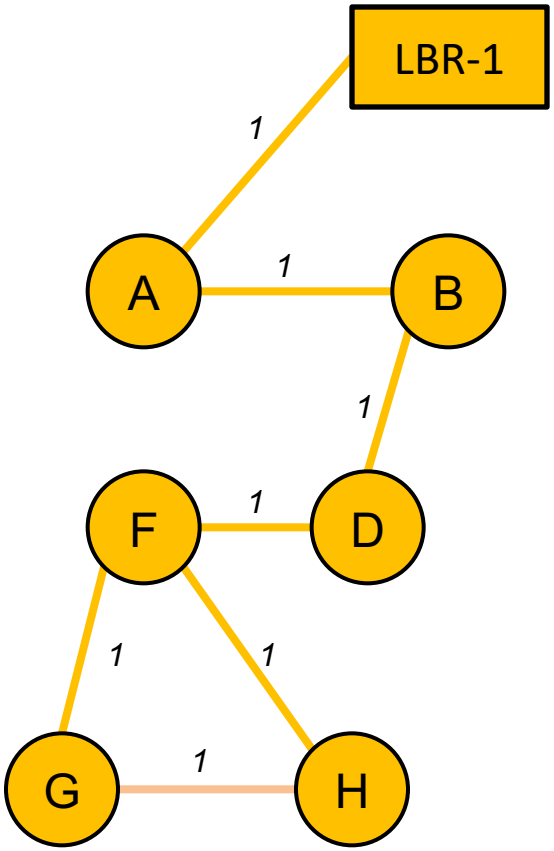
Destination Advertisements



- B adds D to the Reverse Route Stack in the DAO for D::, and passes DAO D:: [D] on to A
- A stores D:: via B, with the piecewise source route [D]
- B also emits a DAO indicating prefix B:: to A
- A stores B:: via B

(continued)

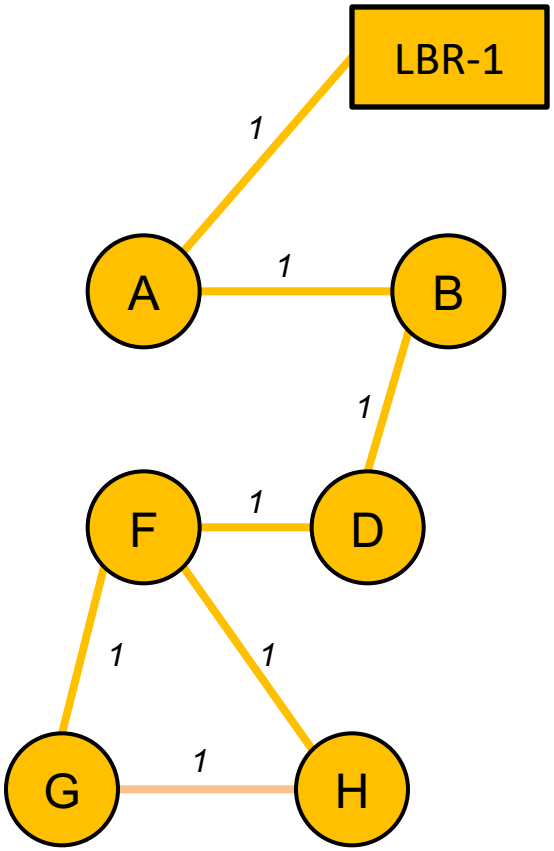
Destination Advertisements



- A emits DAOs to LBR-1 for destination prefixes A::, B::, D::, and F*
- LBR-1 stores A:: via A, B:: via A, D:: via A, and F*:: via A
- A stored B:: via B, D:: via B [D], F* via B [D, F]
- B, D stored nothing
- F stored G:: via G, H:: via H

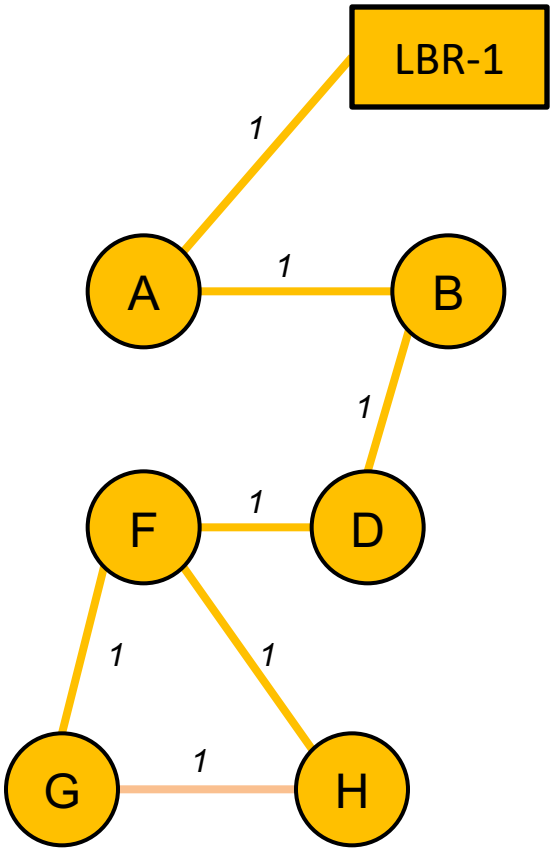
With DAOs, all nodes became aware of the structure of the network

P2MP Traffic



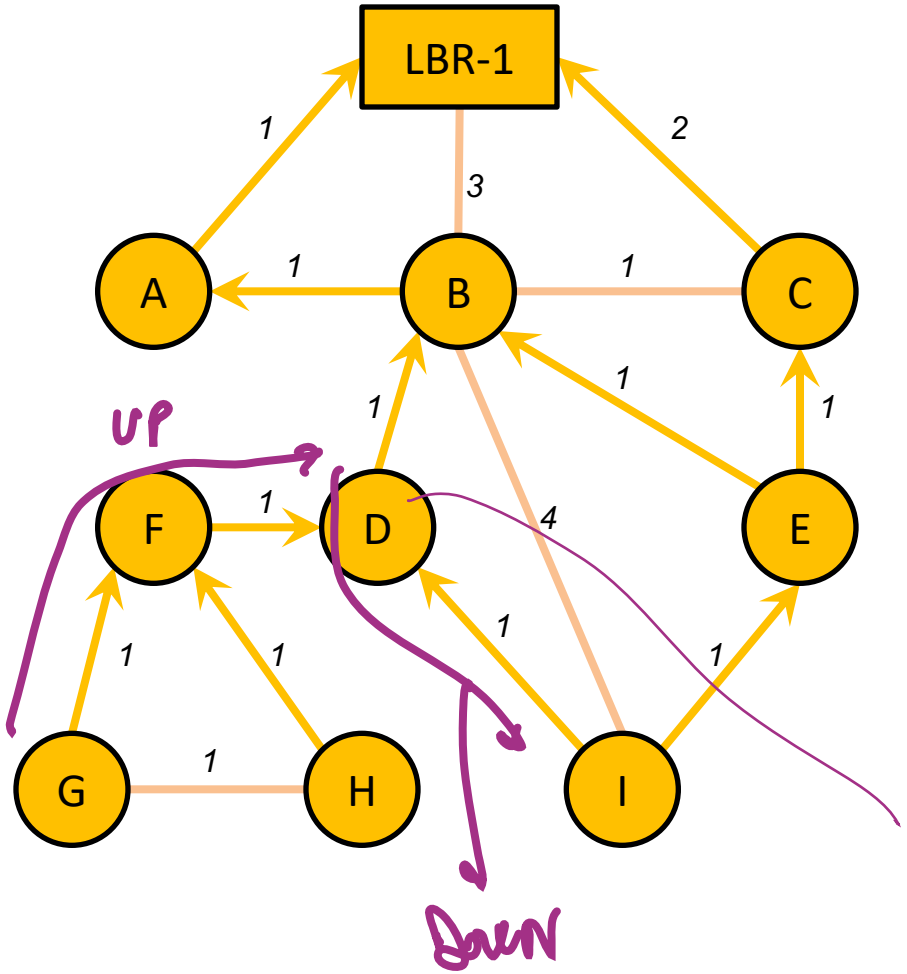
- The routing state setup by Destination Advertisement is used to direct P2MP traffic downward
- LBR-1 directs traffic for G (F*::) to A
- A adds source routing directive, [D, F], and forwards to B
- B uses source routing directive to forward to D...

P2MP Traffic



- D uses source routing directive to forward to F
- F uses routing state to forward to G
- Note the use of source routing to traverse the stateless region of the LLN
  - The details of the source routing mechanism are still to be defined

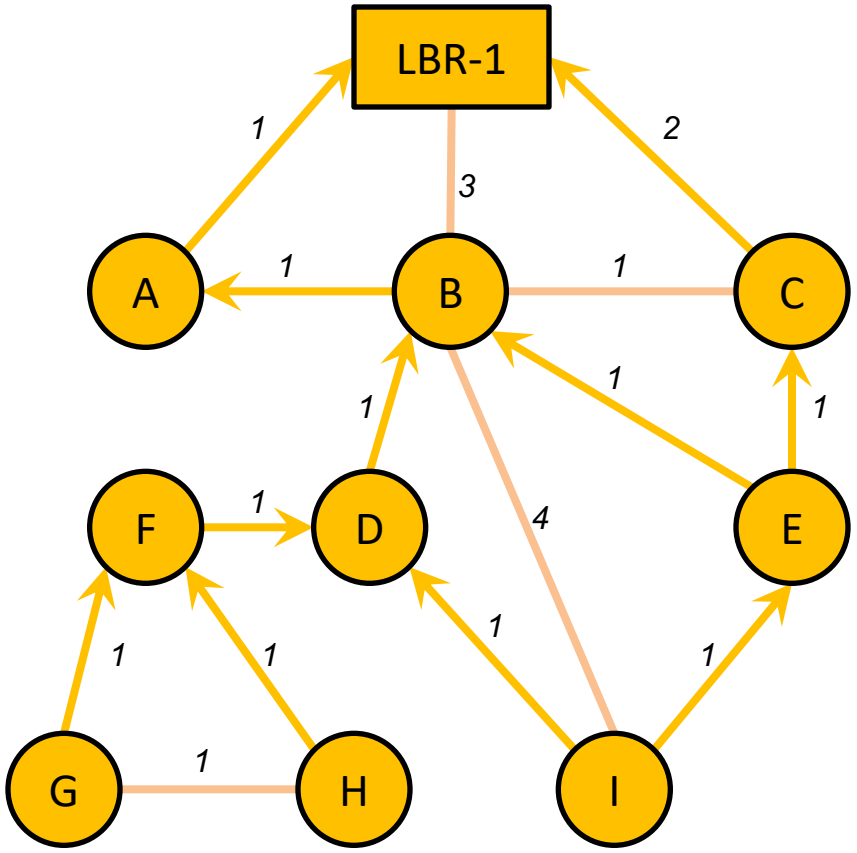
P2P Traffic



- By default, P2P traffic follows the default route until a node is encountered who has a more explicit route
- P2P traffic flows upwards along the DAG until a common node is reached to direct the traffic outwards

You start going down at the first node which knows a downward path to the destination

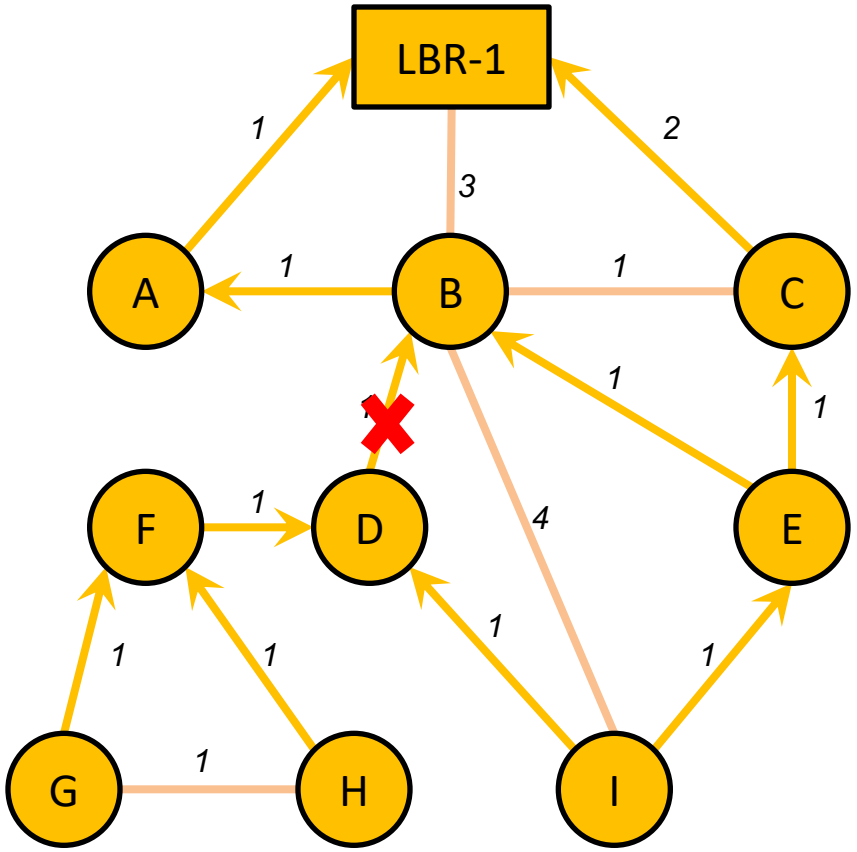
P2P Traffic



- The default scheme allows P2P traffic with minimal state required of nodes – but is not appropriate in all cases
- More optimal P2P routes may be computed and installed if the application requires
  - RPL does not specify nor preclude such a mechanism at this time

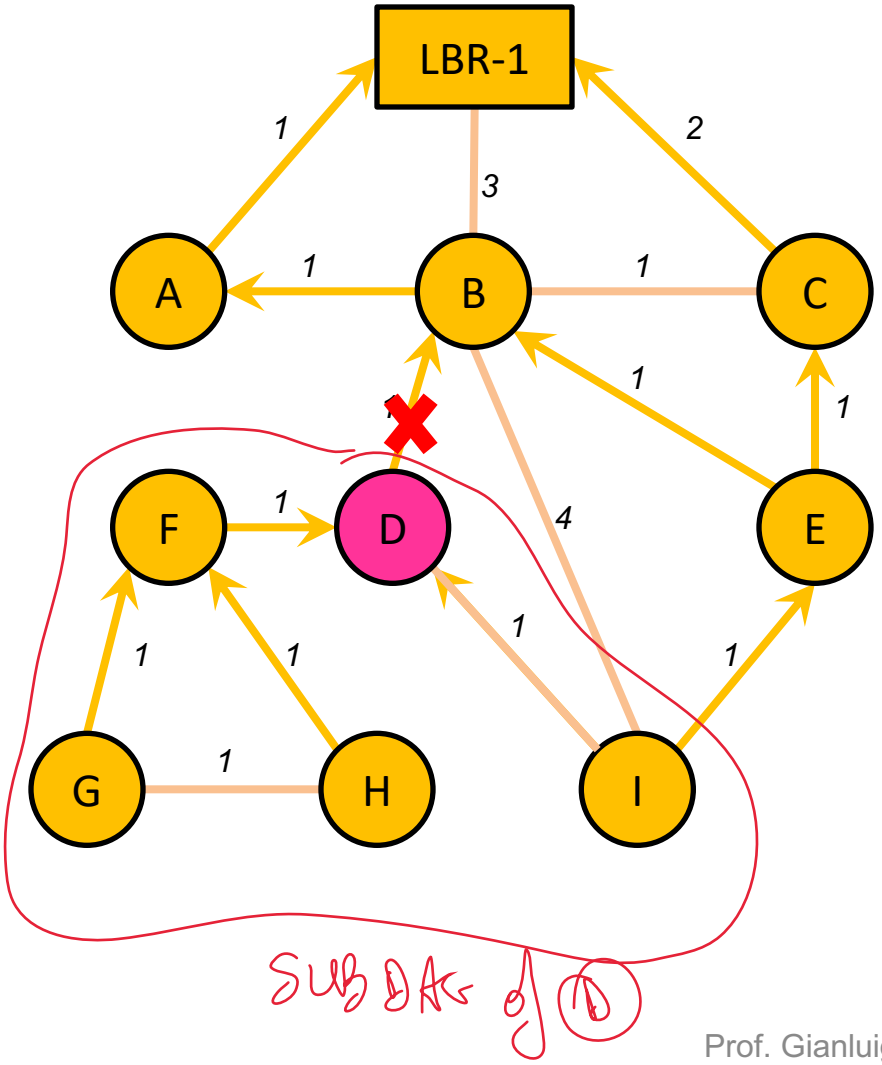
DAG Maintenance

Loop Avoidance



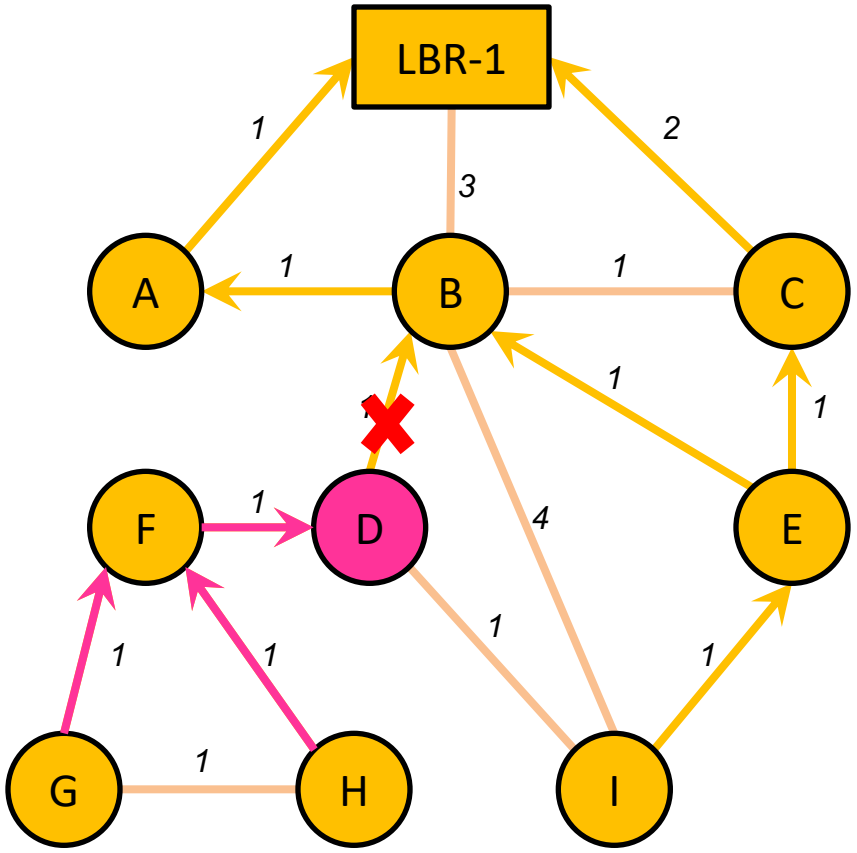
- Consider the case where the link B—D goes bad
- Node D will remove B from its DAG parent set
- Node D no longer has any DAG parents in the grounded DAG, so it will become the root of its own floating DAG

DAG Maintenance
Loop Avoidance



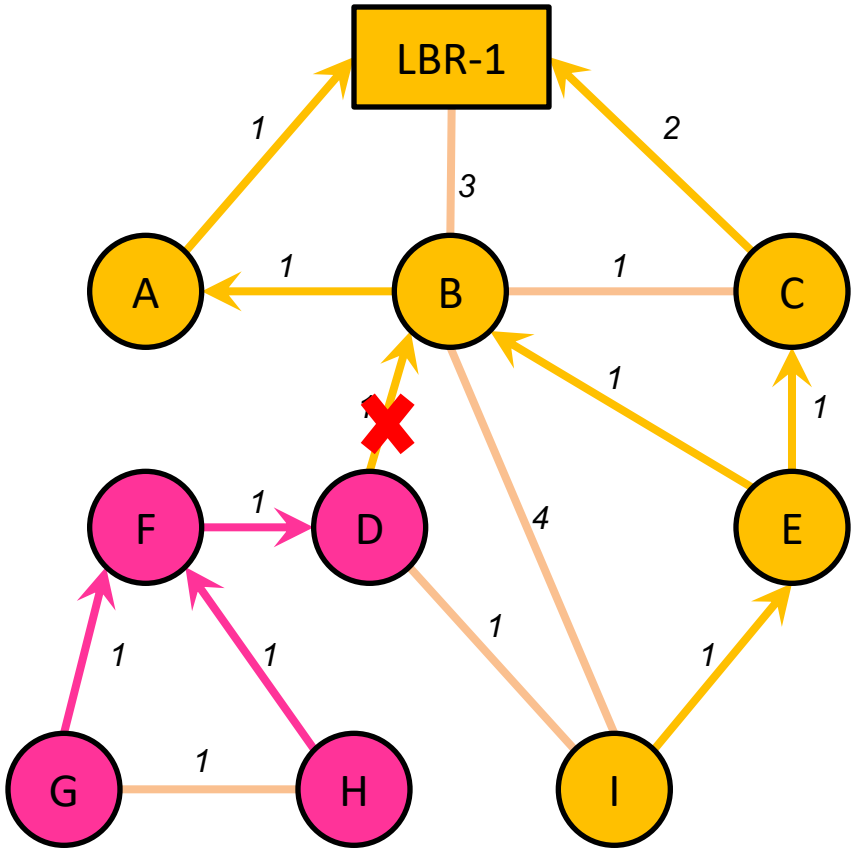
- Node D multicasts an RA-DIO to inform its sub-DAG of the change
- Node I has an alternate DAG Parent, E, and does not have to leave the DAG rooted at LBR-1.
- Node I removes Node D as a DAG Parent

DAG Maintenance
Loop Avoidance



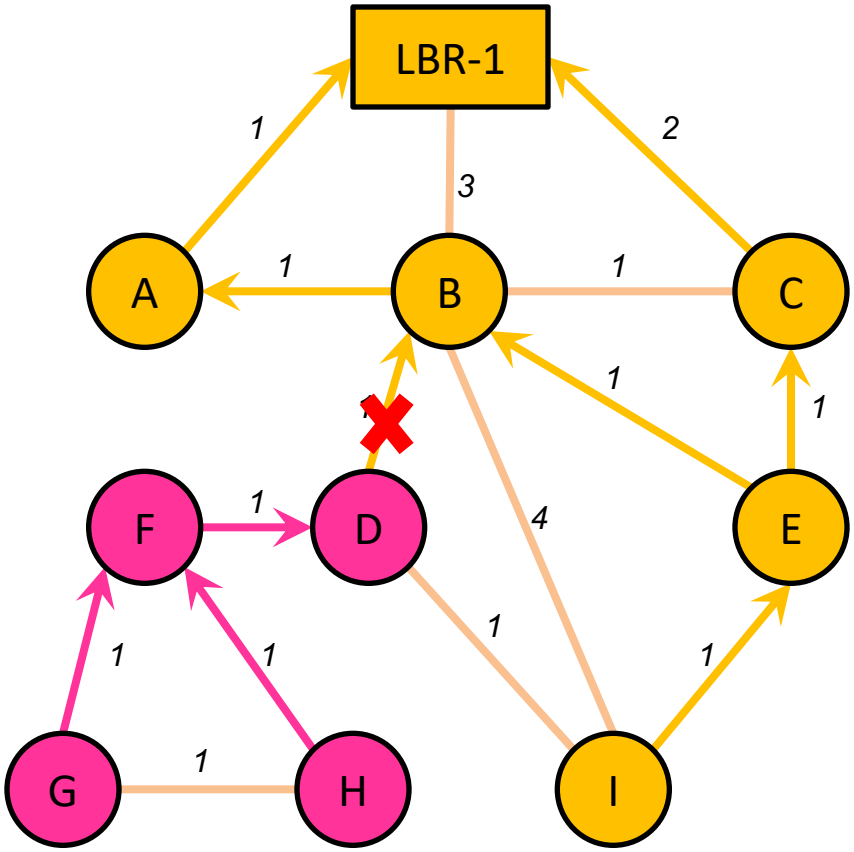
- Node F does not have an option to stay in the DAG rooted at LBR-1 (no alternate DAG Parents), so Node F follows Node D into the floating DAG
- Node F multicasts an RA-DIO
- Nodes G and H follow Node F into the floating DAG

DAG Maintenance
Loop Avoidance



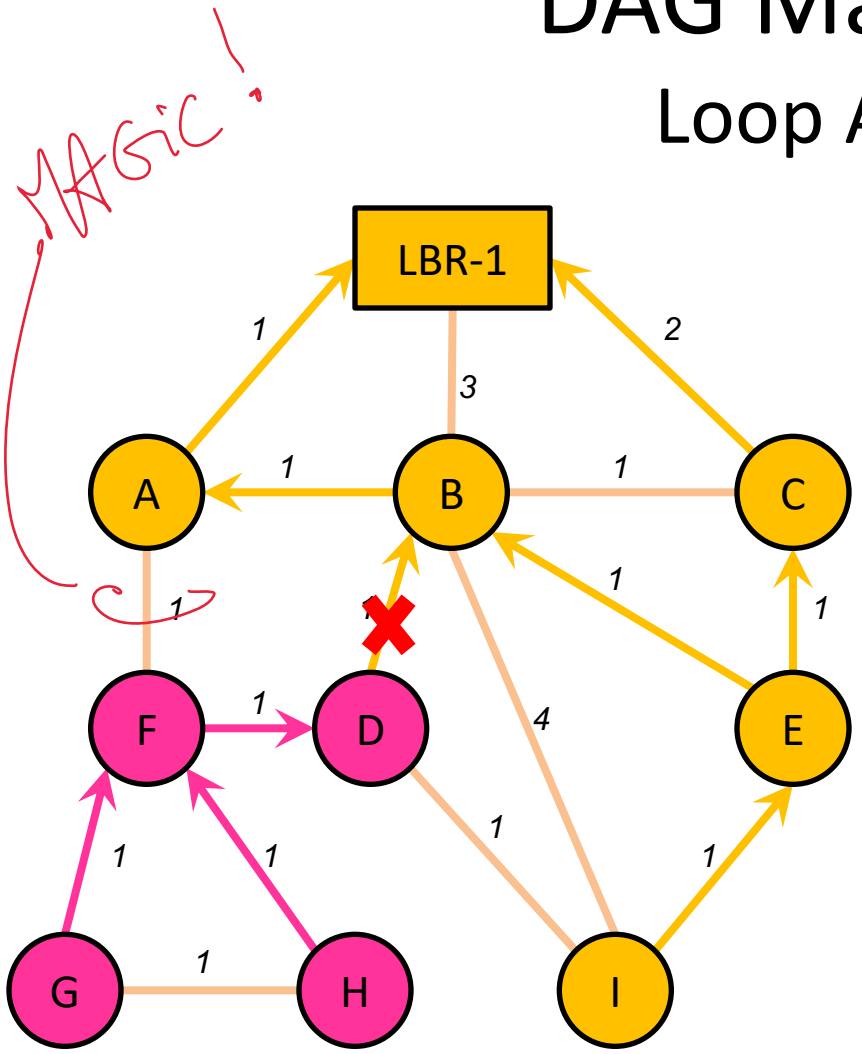
- The sub-DAG of node D has now been frozen
- Nodes contained in the sub-DAG have been identified, and by following node D into the floating DAG, all old routes to LBR-1 have been purged
- The floating DAG seeks to rejoin a grounded DAG…

DAG Maintenance
Loop Avoidance



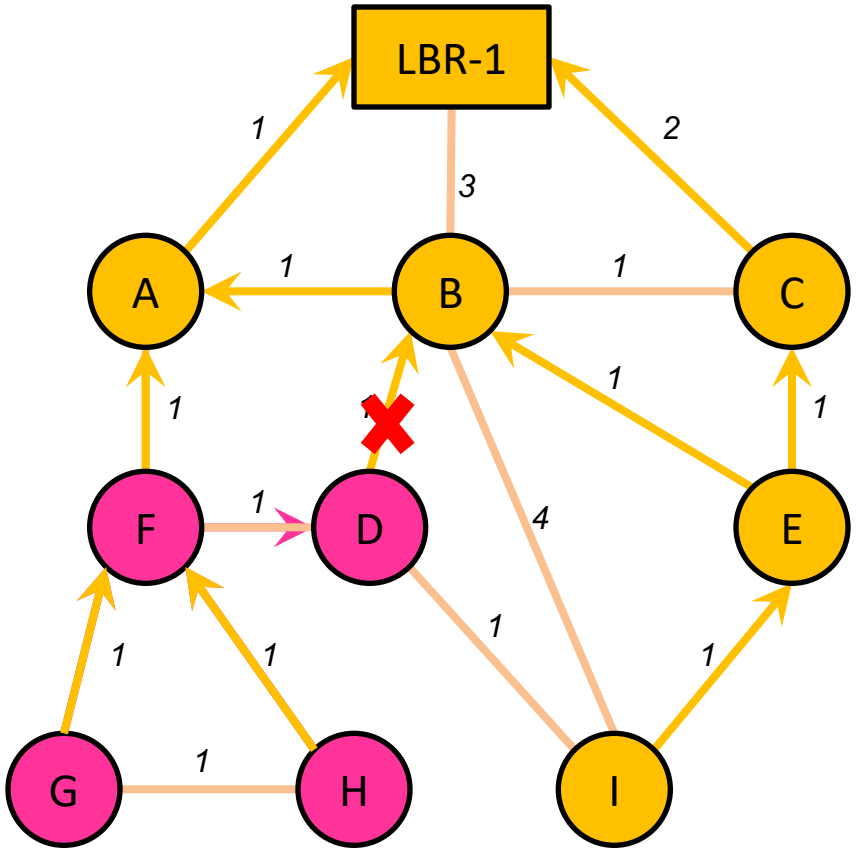
- Node I multicasts an RA-DIO
- Node D sees a chance to rejoin grounded DAG at depth 5 through Node I
- Node D starts a DAG Hop timer of duration α 4 associated with Node I

DAG Maintenance
Loop Avoidance



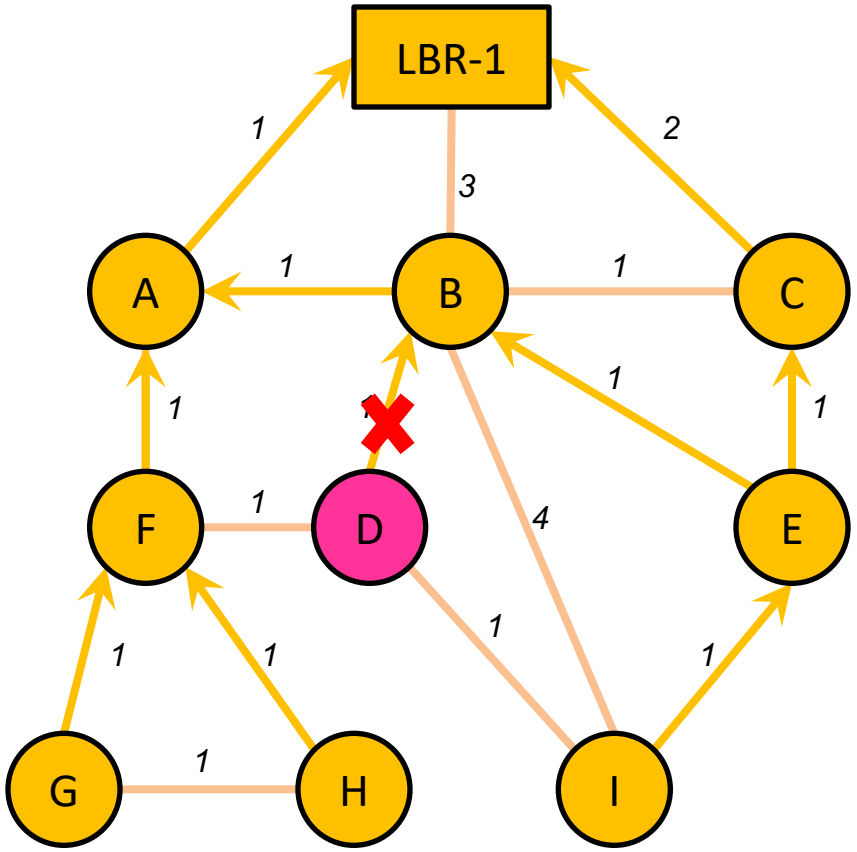
- Suppose a link A—F becomes viable
- Node A multicasts an RA-DIO
- Node F sees a chance to rejoin grounded DAG at depth 2 through Node A
- Node F starts a DAG Hop timer of duration α 1 associated with Node A

DAG Maintenance
Loop Avoidance



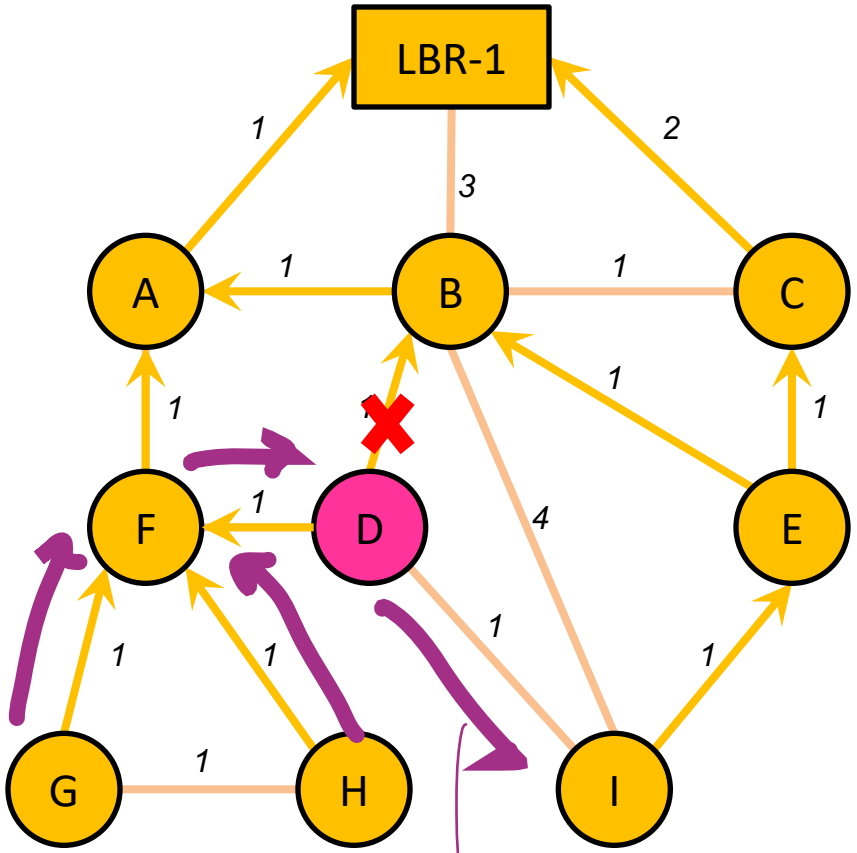
- Node F's DAG Hop Timer expires
- Node F joins to the grounded DAG at depth 2 by adding A as a DAG parent, and removing D
- Node F multicasts an RA-DIO
- Nodes G and H follow Node F to the grounded DAG

DAG Maintenance
Loop Avoidance



- Node D sees a chance to rejoin DAG LBR-1 at depth 3 through Node F
- Node D starts a DAG Hop timer of duration α 2 associated with Node F, in addition the DAG Hop timer already running with duration α 4 associated with Node I

DAG Maintenance
Loop Avoidance



- Node D's DAG Hop timer of duration α 2 tends to expire first
- Node D joins the grounded DAG at depth 3 by adding Node F as a DAG Parent
- The breaking-off and re-joining of the broken sub-DAG is thus coordinated with loop avoidance

Structure if the link A-F had not appeared

Foundations and Emerging Trends in AI for IoT

Armin MAZINANI

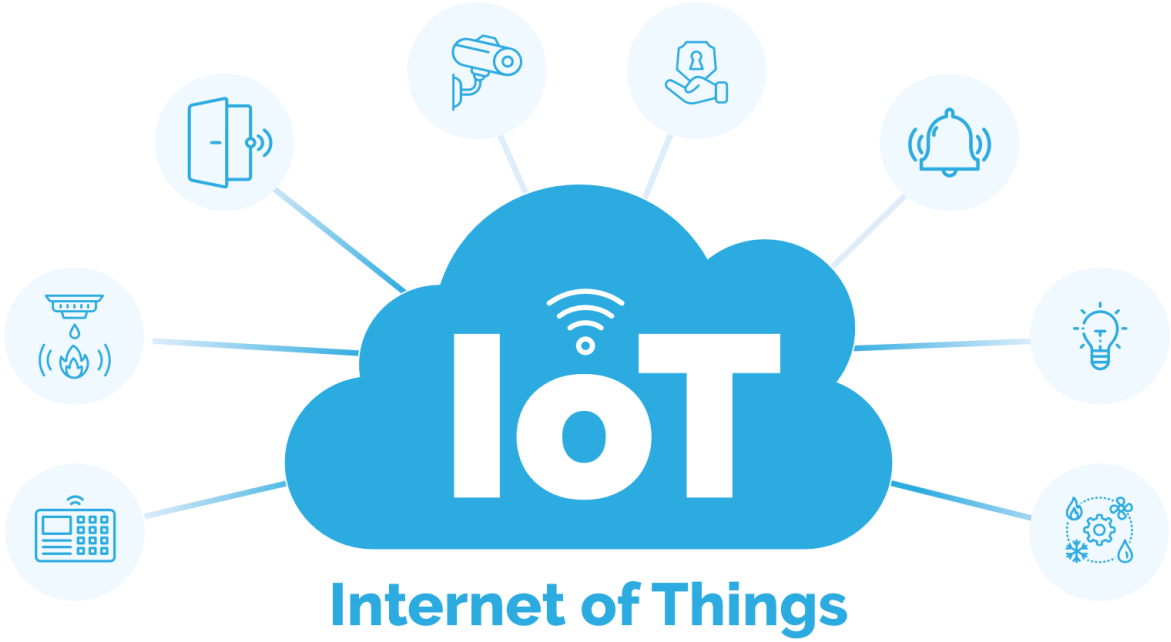
Internet of Things Lab, University of Parma
https://iotlab.unipr.it/

UNIVERSITÀ DI PARMA

IoT Lab

What is IoT?

- Network of connected devices
- Sensors and actuators
- Data collection and communication
- Cloud and edge integration



Motivation: Why AI for IoT?

- **Explosion of IoT devices**
  The rapid growth of connected sensors and smart devices is creating highly distributed and large-scale systems across industries (e.g., healthcare, industry, smart cities, etc.)
- **Massive data generation**
  Devices continuously generate large volumes of heterogeneous data (time-series, signals, images), making traditional processing approaches inefficient
- **Need for intelligent systems**
  To extract meaningful insights from *raw* data, AI techniques are required to enable automatic pattern recognition, prediction, and decision-making
- **Real-time decision-making use cases**
  Many IoT applications (e.g., industrial monitoring, autonomous systems, healthcare) require immediate responses with low latency and high reliability

Overview of AI in IoT

AI for IoT is the integration of **Artificial Intelligence (AI)** technologies (e.g., Machine Learning, ML, and Deep Learning, DL) into **IoT** systems to enable **intelligent decision-making** from the vast amounts of data generated by sensors and connected devices

Key Ideas

- IoT devices generate **continuous streams of data** (e.g., temperature, motion, video, or vibrations)
- AI processes this data to **detect patterns**, **predict outcomes**, or **make real-time decisions** without human intervention

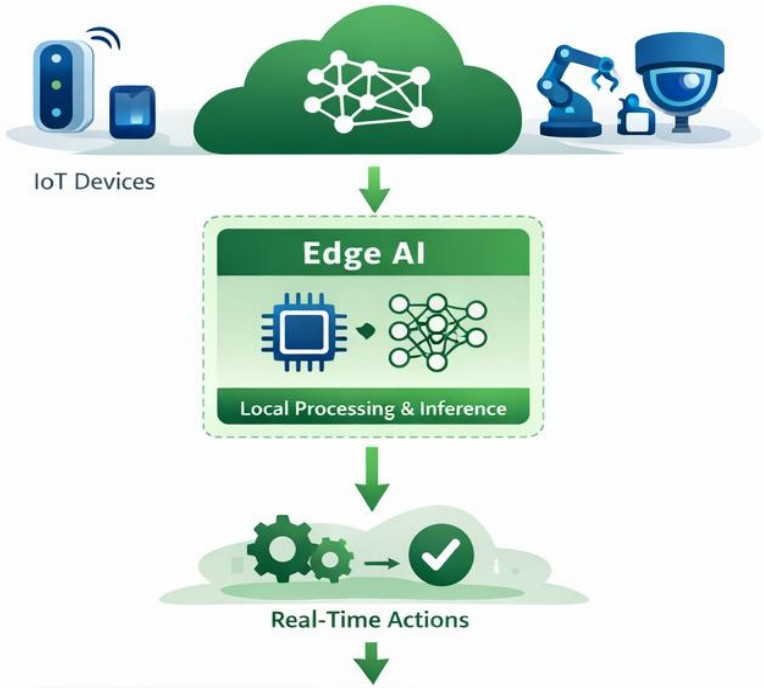
Traditional IoT Architecture

Centralized Processing Model

- High Cloud Dependency
- Higher Latency
- More Bandwidth Usage

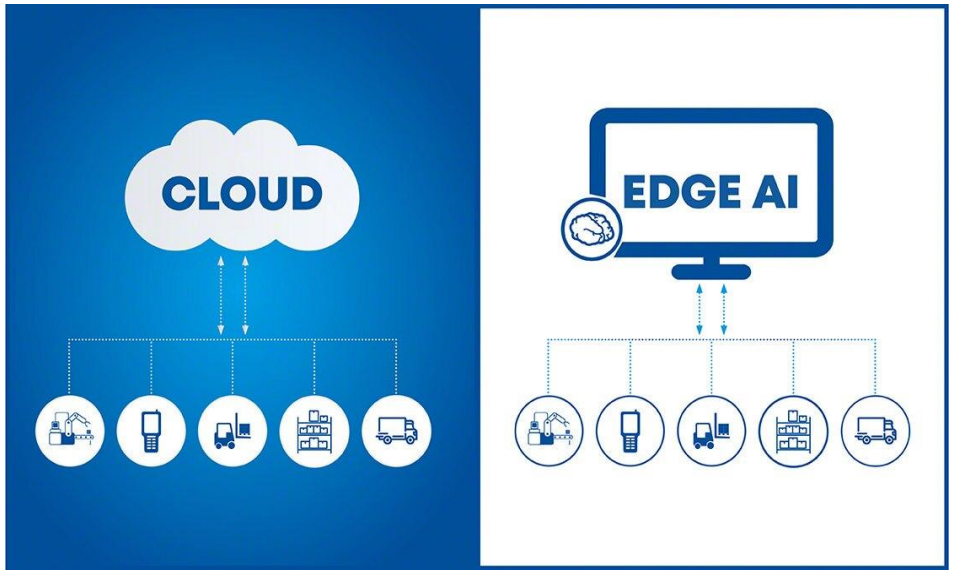
Intelligent Edge AI

Decentralized Processing Model



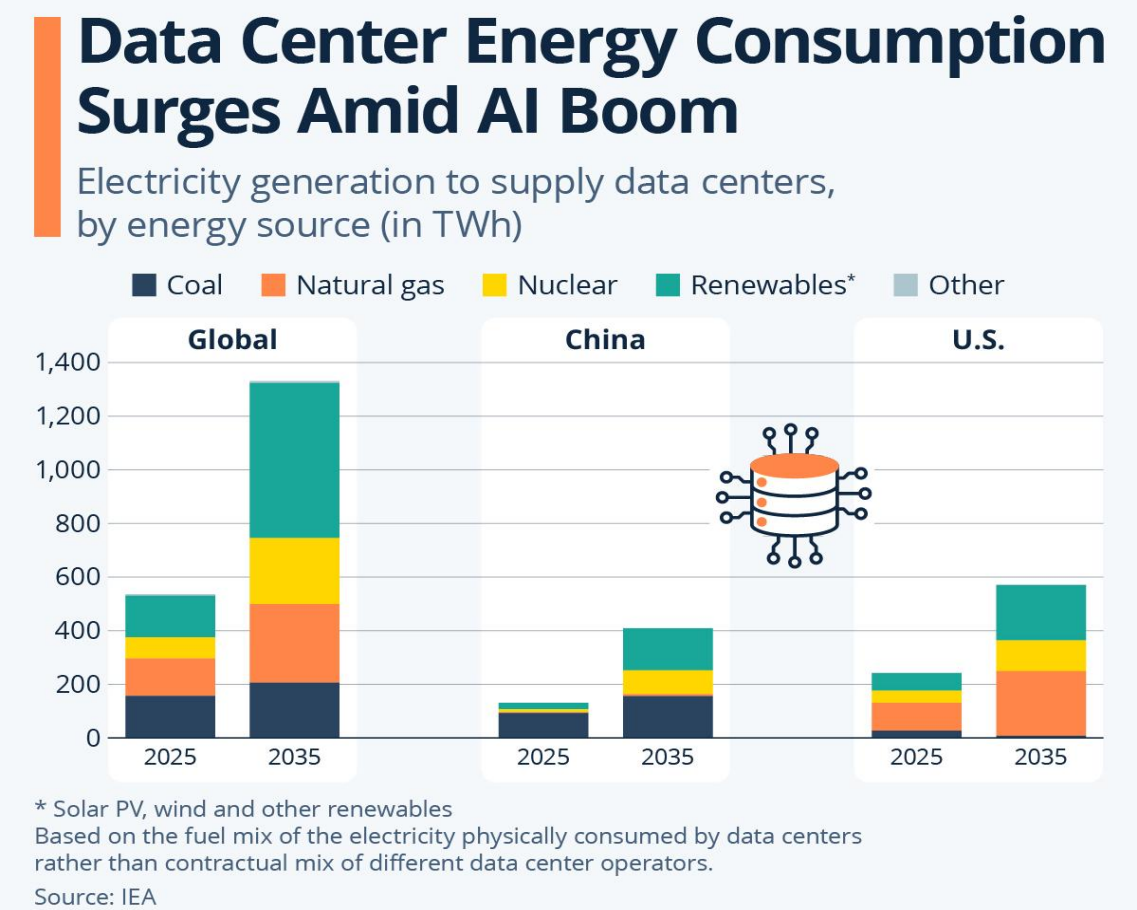
- On-Device Processing
- Faster Decisions
- Reduced Data Transmission

Comparison between Edge AI and Cloud AI



Feature	Edge AI	Cloud AI
Location of computation	On-device (microcontrollers, Raspberry Pi, gateways)	Remote servers / cloud
Latency	Very low, real-time decisions possible	Higher latency due to network delays
Bandwidth usage	Minimal (data processed locally)	High (*raw* data sent to cloud)
Privacy	Better (sensitive data stays on device)	Privacy risk if sending *raw* data to cloud
Power and resource needs	Limited resources → small/lightweight models	High compute power available

Global electricity demand from data centers is expected to more than double in the next five years, largely driven by the widespread uptake of AI



AI, ML & DL

- **AI**

Broad field enabling machines to perform intelligent tasks

Includes reasoning, perception, and decision-making

- **ML**

Subfield of AI that learns patterns from data

Requires feature extraction + training from examples

Examples: Support Vector Machine (SVM), Random Forest, *k*-Nearest Neighbors (*k*-NN)

- **DL**

Subfield of ML based on Neural Networks (NNs)

Automatically learns features from *raw* data

Examples: Convolutional NNs (CNNs), Recurrent NNs (RNNs), Long Short-Term Memory (LSTM), Transformers

Why large models are hard to deploy

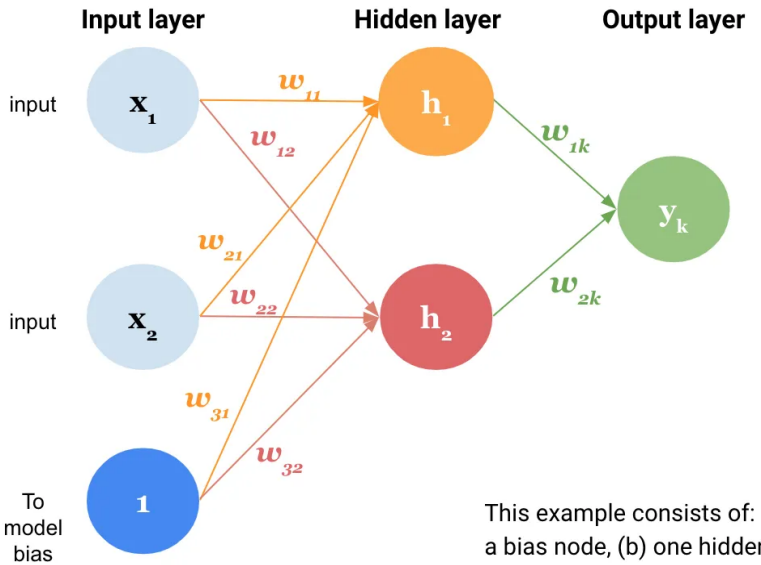
Large memory gap between billion-parameter LLMs and lightweight LLaMA-style models → Model size and numerical precision strongly impact feasibility for deployment on constrained hardware

Model	Parameters	Weight Precision	Model Size (weights)	Inference Profile
LLaMA-2	7B	FP32	≈ 28 GB	Memory-heavy inference
llama-lite	134M	4-bit quantized	≈ 85 MB	Lightweight / edge-friendly

Why DL for IoT?

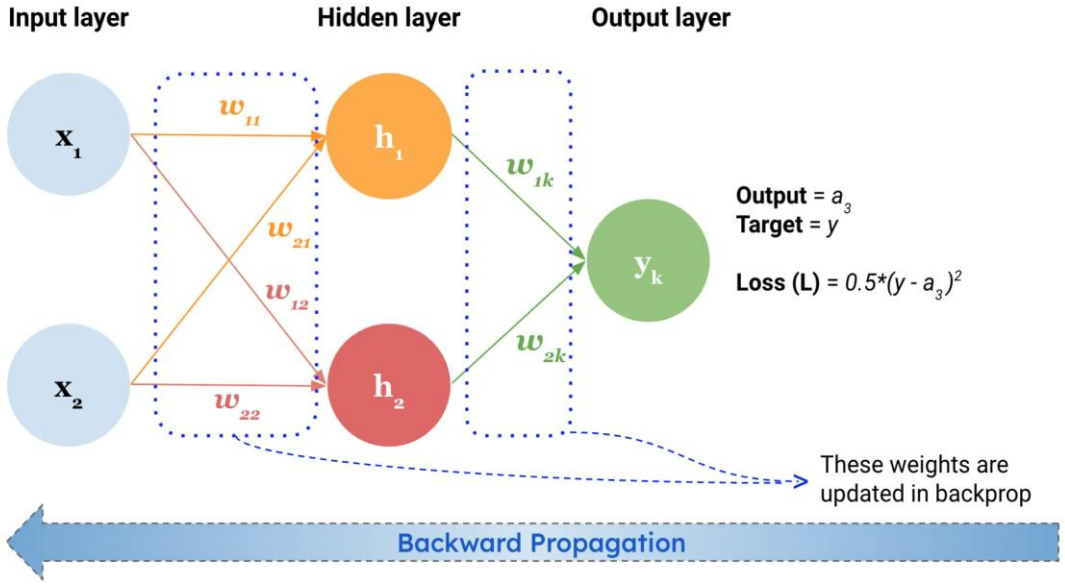
- IoT generates high-dimensional time-series and sensor data
- Traditional ML struggles with complex patterns
- DL enables different operations
  - Automatic feature extraction
  - Better performance on *raw* signals
  - Handling nonlinear patterns
- Suitable for
  - Fault detection
  - Predictive maintenance
  - Activity recognition
  - Cybersecurity

MultiLayer Perceptron (MLP)



x_1 , x_2 : *input data features*
w_{ij} : *weights of the network*
h_1 , h_2 : *nodes in the hidden layer*
y_k : *output variable*

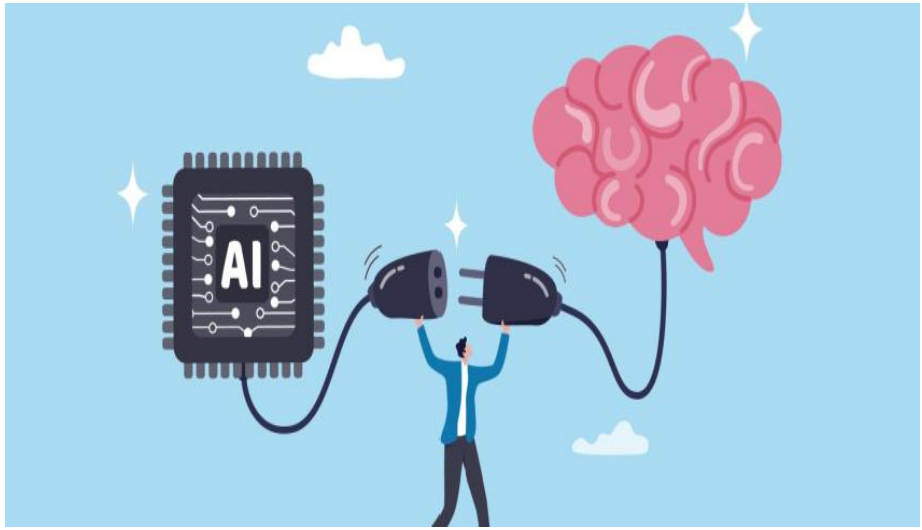
This example consists of: (a) an input layer with two input nodes and a bias node, (b) one hidden layer with two neurons, and (c) an output layer with one neuron



- In stochastic gradient descent, weights (w_{ij}) are updated as below:
  $w_{ij} := w_{ij} + \Delta w_{ij}$
  $\Delta w_{ij} = - \eta * \partial L(w_{ij}) / \partial w_{ij}$,
  where η is the learning rate and $\partial L(w_{ij}) / \partial w_{ij}$ is the gradient of loss w.r.t the model weights
- Intermediate variables calculated during forward prop ($z_1, z_2, z_3, a_1, a_2, a_3$) are used for gradient calculation $\partial L(w_{ij}) / \partial w_{ij}$

AI Deployment Pipeline for IoT

1. Data collection (sensors)
2. Data preprocessing (filtering, normalization)
3. Model training (cloud or local)
4. Model optimization
   - Quantization
   - Pruning
   - Knowledge distillation
5. Deployment
   - Edge device or microcontroller
6. Inference in *real-time*

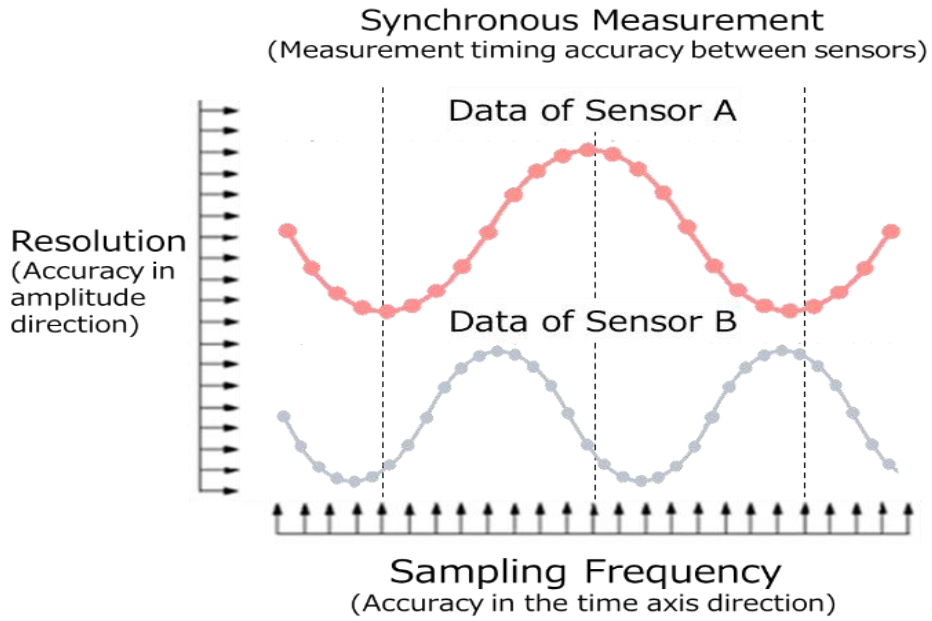


Data Collection (Sensors)

First stage where *raw* data are acquired from physical devices such as sensors, actuators, or embedded systems

Examples include temperature, vibration, current, voltage, audio, or motion signals

The quality and sampling rate of sensors directly affect the performance of the AI model.



Data Preprocessing

Raw sensor data are often noisy, incomplete, or inconsistent

Data are cleaned and transformed to make it suitable for ML

Filtering: Removes noise and unwanted signal components (e.g., low-pass filters, moving average)

Normalization/Scaling: Adjusts values to a common range (e.g., 0–1 or *z*-score) to stabilize training

Segmentation: Splits continuous signals into windows or frames

Handling missing data: Imputation or removal of corrupted samples

Model Optimization

After training, the model is optimized to reduce size, computation, and memory usage—especially important for edge deployment

Quantization: Reduces numerical precision (e.g., float32 → int8) to make models lighter and faster

Pruning: Removes unnecessary weights or neurons to reduce complexity

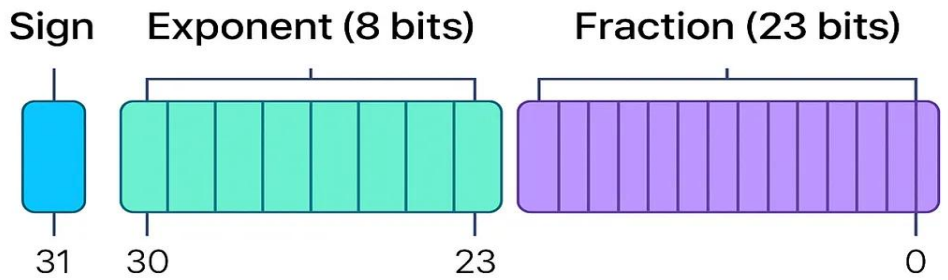
Knowledge Distillation: A smaller “student” model learns from a larger “teacher” model, preserving performance with fewer resources

Quantization

Not all numbers are created equal in computing. There are different **precision types** that define how a number is stored in binary. The choice of precision has a direct impact on a model's memory footprint and computation speed.

- **FP32 (32-bit Floating Point)**
  - FP32 uses 32 bits (4 bytes) per number

FP32 was the standard for training neural networks for years because it offers high precision and range to avoid numerical issues. However, **FP32 consumes a lot of memory and compute. Every weight, activation, and gradient is 4 bytes**.



Quantization

INT8 have no concept of “fractional” bits or exponents.

An 8-bit signed integer can represent values roughly from -128 to +127 (or 0 to 255 if unsigned).

INT8 quantization involves mapping real-valued weights/activations to an 8-bit integer scale.

The primary appeal is efficiency: an INT8 number is 4x smaller than an FP32 value, so models shrink to a quarter of their size.

Quantization

- **Model compression:** smaller weights/activations (e.g., FP32 → INT8) ⇒ lower storage + lower RAM footprint
- **Faster inference :** low-precision / integer arithmetic ⇒ reduced latency (often significant on supported CPUs/accelerators)
- **Lower energy / power:** fewer bits moved + cheaper ops ⇒ reduced energy consumption and better suitability for edge/embedded deployment

Quantization

- INT8 quantization equation

$$q = \text{round}\left(\frac{x}{s}\right) + z$$

$$\hat{x} = s \cdot (q - z)$$

x = real (float32) value

q = quantized integer (e.g., INT8 → −128 to 127)

s = scale (float)

z = zero-point (integer offset)

- How scale and zero-point are computed

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}$$

$$z = \text{round}\left(q_{\min} - \frac{x_{\min}}{s}\right)$$

Quantization

- $x_{\min} = -1.0, x_{\max} = 1.0$
- INT8 $\rightarrow q \in [-128, 127]$

Step 1: scale

$$s = \frac{2}{255} \approx 0.00784$$

Step 2: zero-point

$$z = \text{round}(-128 - (-1/0.00784)) \approx 0$$

So:

$$q = \text{round}(x/0.00784)$$

Post-Training Quantization (PTQ)

PTQ applied **after the model is fully trained**

- Converts trained weights from high precision (e.g., FP32) to lower precision (e.g., INT8)
- No retraining required (or only a small calibration step)
- Uses a small calibration dataset to estimate activation ranges
- Fast and simple to apply

Advantages

- Very easy to implement
- No need to change training pipeline
- Reduces memory and improves inference speed

Quantization-Aware Training (QAT)

QAT is **simulated during training**

Model learns to adapt to low-precision constraints

Advantages

- Higher accuracy compared to PTQ
- Better robustness to INT8/FP16 deployment
- More suitable for critical applications (IoT, medical, industrial AI)

Limitations

- Requires retraining the model
- Higher computational cost during training
- More complex pipeline

Knowledge Distillation (KD)

- **Model compression**: train a small student to mimic a large teacher
- Student learns from teacher **soft predictions** (richer than hard labels)
- Goal: **preserve performance** while **reducing compute** → easier deployment on constrained devices

Teacher–Student Framework

- Teacher: large, accurate, but computationally expensive
- Student: smaller, faster, lightweight
- Training target: match teacher's **probability distribution**, not only the ground-truth class

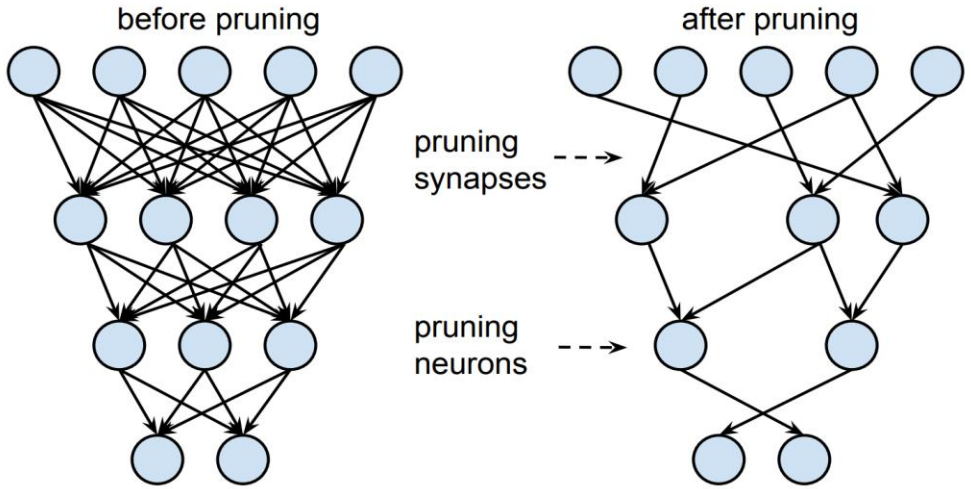
Knowledge Distillation (KD)

- Use **Softmax with temperature (T)** to “soften” teacher outputs
- Higher **T** reveals class similarities (“dark knowledge”) in the teacher distribution
- Student is trained to reproduce these softened outputs (then use **T=1** at inference)

KD = accuracy-aware compression: transfer teacher behavior → smaller model with similar performance at lower cost

NN Pruning

- Pruning is removing parameters from an existing neural network to **reduce size/compute** while trying to keep accuracy.
- **Weight pruning:** set individual weights to zero → sparse model.
  - Removing a weight is essentially setting it to zero; prune weights already close to 0 (low magnitude) to reduce impact.



Tutorials:

KD tutorial

https://colab.research.google.com/drive/1aztIQvrsDdnAv8M75wNqA30D3Xi-JHdt?usp=sharing

Quantization and pruning tutorial

https://colab.research.google.com/drive/1A7QHajTdtkuBEQY3yZzGKj1XCRWMRf2P?usp=sharing

KD

Quantization

Federated Learning (FL)

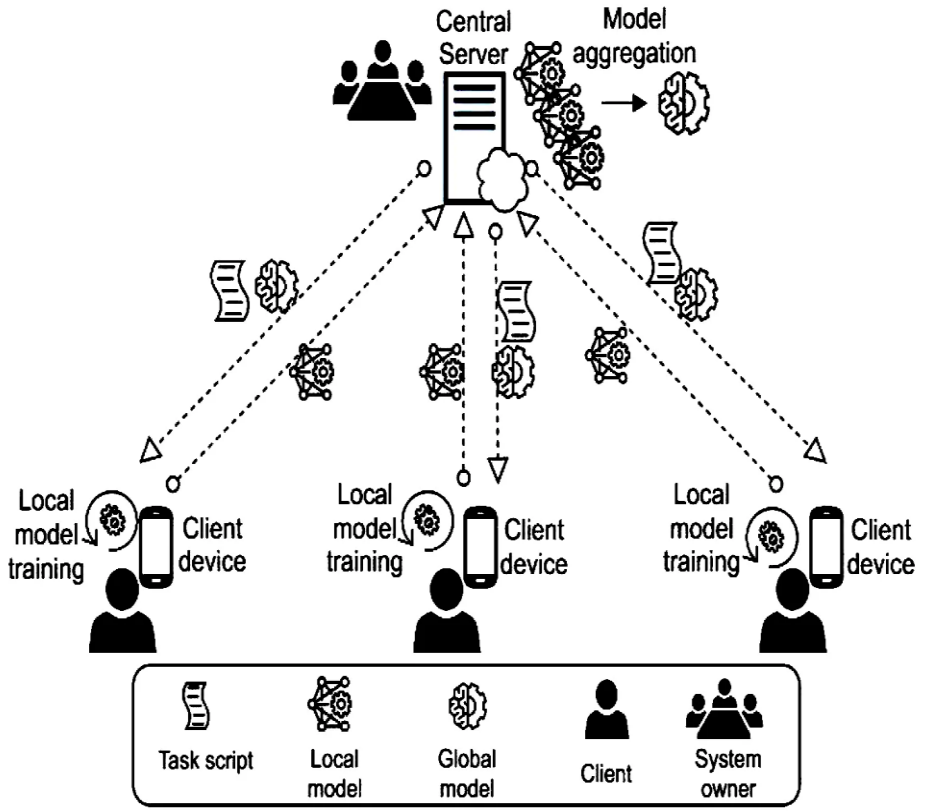
Federated Learning enables multiple clients to collaboratively train a shared model while keeping training data local.

- Local training on edge devices
- Only model updates are shared
- Central server aggregates updates

“Move the model to the data, not the data to the model”

Federated Learning (FL)

- Server initializes global model
- Model is sent to selected clients
- Each client trains locally
- Clients send updates (weights / gradients)
- Server aggregates (e.g., averaging)
- Repeat until convergence



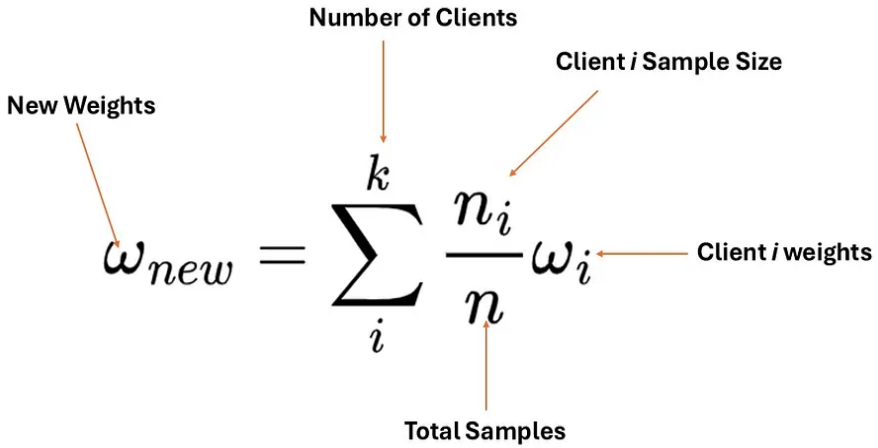
FedAvg

Three clients were involved in a FL round and they each have sent their weights to the server say:

w1=[1.1,0.9] from 100 samples

w2=[1.3,1.0] from 300 samples

w3=[1.2,0.95] from 100 samples



$$\omega_{new} = \frac{100 \cdot \omega_1 + 300 \cdot \omega_2 + 100 \cdot \omega_3}{100 + 300 + 100}$$

$$\omega_{new} = \frac{100 \cdot (1.1, 0.9) + 300 \cdot (1.3, 1.0) + 100 \cdot (1.2, 0.95)}{500}$$

$$\omega_{new} = \frac{(110, 90) + (390, 300) + (120, 95)}{500}$$

$$\omega_{new} = (1.24, 0.97)$$

Tutorials:

FL tutorial:

https://colab.research.google.com/drive/1EPSkcJMhlL3vH_HlZOTF7vlOJbYF2OBe?usp=sharing

FL